
The Complete Guide to Next-Generation Sequencing

From Molecules to Meaning:

Technologies, Applications, and Bioinformatics
for the Modern Genomics Era



A Comprehensive Reference

February 24, 2026

Preface

Why This Book?

The landscape of biological sequencing technologies has expanded at a breathtaking pace. What began with Frederick Sanger’s chain-termination method in 1977—capable of reading a few hundred bases at a time—has evolved into an ecosystem of technologies that can interrogate virtually every layer of biological information: the genome, the transcriptome, the epigenome, the three-dimensional organisation of chromatin, protein–nucleic acid interactions, and even the spatial arrangement of molecules within intact tissues.

For newcomers, this landscape can be overwhelming. Hundreds of protocol names, dozens of platforms, and a dizzying array of acronyms stand between a curious biologist and a productive sequencing experiment. This book aims to be the single reference that bridges that gap.

Who Is This Book For?

This book assumes essentially no prior knowledge of molecular biology or bioinformatics. If you know that DNA is a molecule and that cells contain genes, you have enough background to start reading from [Chapter 1](#). We build every concept from the ground up:

- **Graduate students** entering a genomics or bioinformatics lab for the first time.
- **Wet-lab biologists** who want to understand what happens after they submit samples to a sequencing core.
- **Computational scientists** who want to understand the biology and chemistry behind the data they analyse.
- **Clinicians and translational researchers** exploring how sequencing technologies can be applied in clinical settings.
- **Industry professionals** evaluating sequencing platforms and applications for their organisations.

How to Read This Book

The book is organised into seven parts:

Part I: Foundations

covers the essential molecular biology, the history of sequencing, and the core principles shared by all modern sequencing technologies.

Part II: Platforms and Technologies

provides a detailed survey of current sequencing instruments—short-read and long-read—and the universal concepts of library preparation.

Part III: Genomics

covers DNA-level applications including whole-genome sequencing, exome sequencing, variant discovery, and metagenomics.

Part IV: Transcriptomics

explores RNA sequencing in all its forms: bulk, single-cell, long-read, and spatial.

Part V: Epigenomics

dives into DNA methylation, chromatin accessibility, histone modifications, and three-dimensional genome architecture.

Part VI: Multiomics and Emerging Frontiers

covers the latest technologies that combine multiple measurement modalities and the cutting-edge methods pushing the field forward.

Part VII: Bioinformatics and Data Analysis

walks through computational workflows, experimental design, data standards, and reproducibility—with concrete examples using both Nextflow and Snakemake workflow managers.

Each chapter can largely be read independently, though readers entirely new to the field will benefit from reading Parts I and II first. Throughout the text, coloured boxes highlight key concepts, practical tips, protocol summaries, and important caveats.

A Note on Pace of Change

Sequencing technology evolves rapidly. We have endeavoured to cover the state of the art as of early 2026, but specific instrument models, software versions, and even company names may change. The fundamental principles—the chemistry, the biology, and the analytical strategies—evolve much more slowly and will remain relevant for years to come.

Happy sequencing.

Contents

Preface	iii
I Foundations	1
1 Introduction: The Sequencing Revolution	3
1.1 What Is Sequencing and Why Does It Matter?	3
1.2 A Brief History of Sequencing	4
1.2.1 The Sanger Era (1977–2000)	4
1.2.2 The Human Genome Project (1990–2003)	4
1.2.3 The Next-Generation Sequencing Revolution (2005–2015)	5
1.2.4 The \$1,000 Genome and Beyond (2014–Present)	5
1.3 The Different “Omes”	5
1.3.1 The Genome	6
1.3.2 The Transcriptome	6
1.3.3 The Epigenome	6
1.3.4 The Proteome	7
1.3.5 The Metabolome	7
1.3.6 The Microbiome	7
1.4 How Sequencing Has Transformed Science	7
1.4.1 Medicine and Clinical Diagnostics	7
1.4.2 Agriculture and Food Science	8
1.4.3 Ecology and Conservation Biology	8
1.4.4 Forensic Science	8
1.4.5 Evolutionary Biology and Palaeogenomics	8
1.5 What Modern Sequencing Can Measure	8
1.6 The General Sequencing Workflow	9
1.6.1 Step 1: Biological Sample	9
1.6.2 Step 2: Nucleic Acid Extraction	10
1.6.3 Step 3: Library Preparation	10
1.6.4 Step 4: Sequencing	10
1.6.5 Step 5: Data Analysis	11
1.6.6 Step 6: Biological Insight	11
1.7 The Cost Trajectory and Democratisation of Sequencing	11
1.8 The Current Sequencing Landscape	12
1.9 An Information-Theoretic View of Sequencing	12
1.9.1 Quantitative Comparison of Sequencing Generations	13
1.10 Reading Guide for This Book	14
1.11 Summary	15

2	Molecular Biology Essentials	17
2.1	The Cell: Life's Fundamental Unit	17
2.1.1	Prokaryotic Cells	17
2.1.2	Eukaryotic Cells	17
2.2	DNA Structure: The Molecule of Heredity	18
2.2.1	Nucleotides: The Building Blocks	18
2.2.2	The Double Helix	19
2.2.3	Nucleotide Chemistry in Detail	19
2.2.4	DNA Thermodynamics and the Nearest-Neighbour Model	19
2.2.5	Directionality and Antiparallel Strands	20
2.3	DNA Organisation: From Double Helix to Chromosome	21
2.3.1	Histones and Nucleosomes	21
2.3.2	Chromatin: Euchromatin and Heterochromatin	22
2.3.3	Higher-Order Structures and Chromosomes	22
2.3.4	Ploidy	22
2.4	Genes and the Genome	22
2.4.1	What Is a Gene?	22
2.4.2	Gene Structure: Exons and Introns	23
2.4.3	Regulatory Regions	23
2.4.4	Repetitive Elements	24
2.4.5	Coding vs. Non-Coding DNA	24
2.5	The Central Dogma of Molecular Biology	25
2.5.1	Kinetics of Information Flow	25
2.6	Transcription: From DNA to RNA	26
2.6.1	Transcription in Eukaryotes	26
2.6.2	mRNA Processing	27
2.7	Translation: From RNA to Protein	27
2.7.1	The Genetic Code	27
2.7.2	The Process of Translation	28
2.7.3	Protein Folding and Function	28
2.8	Types of RNA	28
2.9	Epigenetics: Modifications Beyond the Sequence	29
2.9.1	DNA Methylation	30
2.9.2	Histone Modifications	30
2.9.3	Chromatin Remodelling	31
2.10	DNA Replication and Repair	31
2.10.1	DNA Replication	31
2.10.2	DNA Repair	32
2.11	Genetic Variation	32
2.11.1	Single Nucleotide Polymorphisms (SNPs)	32
2.11.2	Insertions and Deletions (Indels)	33
2.11.3	Structural Variants (SVs)	33
2.11.4	Copy Number Variants (CNVs)	33
2.12	Gene Regulation: Controlling When and Where Genes Are Expressed	33
2.12.1	Transcription Factors	34
2.12.2	Enhancers and Super-Enhancers	34
2.12.3	Chromatin Accessibility and Gene Expression	34
2.12.4	Post-Transcriptional Regulation	34
2.13	Summary	35

3	Principles of Modern Sequencing	37
3.1	What All Sequencing Technologies Share	37
3.2	Sequencing by Synthesis (SBS)	38
3.2.1	The Chemistry	38
3.2.2	Cluster Generation	38
3.3	Sequencing by Ligation	39
3.4	Nanopore Sequencing	40
3.4.1	The Principle	40
3.4.2	Key Features of Nanopore Sequencing	40
3.5	Single-Molecule Real-Time (SMRT) Sequencing	41
3.5.1	The Principle	41
3.5.2	SMRTbell Templates and HiFi Reads	42
3.6	Key Sequencing Metrics	43
3.6.1	Read Length	43
3.6.2	Throughput	43
3.6.3	Accuracy and Error Profiles	43
3.7	Paired-End vs. Single-End Reads	44
3.8	Read Depth and Coverage	44
3.8.1	Definitions	44
3.8.2	Why Coverage Matters	45
3.8.3	The Lander–Waterman Model of Coverage	46
3.8.4	Adapter Ligation Efficiency	47
3.9	Multiplexing and Barcoding	47
3.9.1	How Indexing Works	47
3.9.2	Benefits of Multiplexing	48
3.10	Quality Scores: Phred Scores and Q30	48
3.10.1	Phred Quality Scores	48
3.10.2	Q30 as a Benchmark	49
3.10.3	From Phred to Neural Basecallers	49
3.10.4	Quality Score Encoding in FASTQ Files	50
3.11	The Concept of a Sequencing Library	50
3.11.1	What Defines a Library	50
3.11.2	Library Complexity and Saturation	50
3.12	Amplification Bias and PCR Duplicates	51
3.12.1	PCR Duplicates	51
3.12.2	Amplification Bias	51
3.13	GC Bias	51
3.14	Comparison of Major Sequencing Approaches	52
3.15	The FASTQ File Format	53
3.16	Summary	54
II	Sequencing Platforms and Technologies	55
4	Short-Read Sequencing Platforms	57
4.1	Illumina: The Dominant Platform	57
4.1.1	Historical Context and Market Position	57
4.1.2	Chemistry: Bridge Amplification and Cluster Generation	57
4.1.3	Sequencing by Synthesis with Reversible Terminators	58
4.1.4	Two-Channel vs Four-Channel Chemistry	59
4.1.5	Patterned vs Non-Patterned Flow Cells	60
4.1.6	Cluster Density Optimisation	60

4.1.7	Phasing and Pre-Phasing Error Models	60
4.1.8	Two-Channel Signal Deconvolution	61
4.1.9	Index Hopping: Rates and Mitigation	62
4.1.10	Platform Hierarchy	62
4.1.11	Illumina Error Profile	63
4.2	MGI / BGI DNBSEQ	63
4.2.1	DNA Nanoball (DNB) Technology	64
4.2.2	Combinatorial Probe-Anchor Synthesis (cPAS)	64
4.2.3	DNBSEQ Platforms	64
4.3	Element Biosciences AVITI	65
4.3.1	Avidity Sequencing Chemistry	65
4.3.2	AVITI Specifications	65
4.4	Ultima Genomics	65
4.4.1	UG 100 Platform	65
4.4.2	Specifications and Considerations	66
4.5	Singular Genomics G4	66
4.5.1	Chemistry and Specifications	66
4.6	Comprehensive Platform Comparison	67
4.7	How to Choose a Short-Read Platform	67
4.7.1	Throughput Requirements	67
4.7.2	Read Length Requirements	68
4.7.3	Budget and Capital Expenditure	68
4.7.4	Data Ecosystem and Compatibility	68
4.7.5	Practical Considerations	69
4.8	Performance Metrics and Quality Considerations	69
4.9	The Future of Short-Read Sequencing	69
4.10	Summary	70
5	Long-Read Sequencing Platforms	71
5.1	Why Long Reads Matter	71
5.2	Pacific Biosciences (PacBio)	72
5.2.1	Historical Context	72
5.2.2	SMRT Sequencing Principle	72
5.2.3	Continuous Long Reads (CLR) vs HiFi Reads (CCS)	73
5.2.4	PacBio Platforms	74
5.2.5	Kinetic Information and Base Modification Detection	75
5.3	Oxford Nanopore Technologies (ONT)	76
5.3.1	Historical Context	76
5.3.2	Nanopore Sequencing Principle	76
5.3.3	Library Preparation	78
5.3.4	Basecalling: From Signal to Sequence	79
5.3.5	Pore Chemistry Evolution: R9 to R10 and Duplex	79
5.3.6	ONT Platform Hierarchy	80
5.3.7	Ultra-Long Reads	81
5.3.8	Direct RNA Sequencing	81
5.3.9	Adaptive Sampling (Read Until)	82
5.3.10	Base Modification Detection	82
5.4	PacBio HiFi vs Oxford Nanopore: A Detailed Comparison	82
5.4.1	When to Choose PacBio HiFi	82
5.4.2	When to Choose ONT	83
5.5	Hybrid Sequencing Approaches	84
5.6	Emerging Long-Read Developments	84

5.7	Summary	85
6	Library Preparation: From Sample to Sequencer	87
6.1	What Is a Sequencing Library?	87
6.2	Nucleic Acid Extraction	89
6.2.1	DNA Extraction Methods	89
6.2.2	RNA Extraction Methods	90
6.3	Fragmentation	90
6.3.1	Mechanical Fragmentation	91
6.3.2	Enzymatic Fragmentation	91
6.3.3	Chemical Fragmentation	92
6.4	End Repair and A-Tailing	92
6.5	Adapter Ligation	92
6.5.1	Illumina Y-Adapters	93
6.5.2	PacBio SMRTbell Adapters	93
6.5.3	ONT Adapters	93
6.6	Size Selection	93
6.6.1	SPRI Bead-Based Size Selection (AMPure XP)	93
6.6.2	Gel-Based Size Selection	94
6.6.3	The Physics of SPRI Bead Size Selection	94
6.6.4	Adapter Design: Thermodynamic Considerations	95
6.6.5	Low-Input and FFPE Protocols	95
6.7	PCR Amplification	96
6.7.1	Why PCR?	96
6.7.2	How Many Cycles?	96
6.7.3	PCR-Free Protocols	97
6.8	Unique Molecular Identifiers (UMIs)	97
6.8.1	How UMIs Work	97
6.8.2	Applications	97
6.9	Indexing and Multiplexing	97
6.9.1	Single Indexing vs Dual Indexing	98
6.9.2	Index Hopping	98
6.9.3	Index Design Considerations	98
6.10	Library Quality Control	98
6.10.1	Quantification: Qubit Fluorometer	99
6.10.2	Fragment Size Distribution: Bioanalyzer / TapeStation	99
6.10.3	Functional Quantification: qPCR / dPCR	99
6.11	Specific Library Preparation Protocols	99
6.11.1	Illumina DNA Prep (Tagmentation-Based)	99
6.11.2	Illumina Stranded mRNA Prep	99
6.11.3	NEBNext Ultra II Kits	100
6.11.4	KAPA HyperPrep and HyperPlus	100
6.11.5	10x Genomics Chromium (Single-Cell)	100
6.11.6	PacBio SMRTbell Library Preparation	100
6.11.7	ONT Ligation Sequencing Kit (SQK-LSK114)	101
6.11.8	ONT Rapid Sequencing Kit (SQK-RAD114 / SQK-RBK114)	101
6.12	Target Enrichment	101
6.12.1	Hybridisation Capture (Pull-Down)	101
6.12.2	Amplicon-Based Enrichment	102
6.13	Common Failure Modes and Troubleshooting	102
6.14	Decision Framework: Choosing a Library Prep Method	104
6.15	Automation and High-Throughput Library Preparation	105

6.16	Cost Considerations	105
6.17	Summary	106
III	Genomics: Reading the Blueprint	107
7	Whole-Genome and Exome Sequencing	109
7.1	Whole-Genome Sequencing (WGS)	109
7.1.1	What WGS Captures	109
7.1.2	When to Use WGS vs. Targeted Approaches	110
7.1.3	Sequencing Depth Requirements	110
7.1.4	Short-Read WGS Workflow	110
7.1.5	Long-Read WGS	111
7.1.6	De Novo Genome Assembly	112
7.1.7	Telomere-to-Telomere (T2T) Assemblies	115
7.1.8	Clinical WGS	116
7.2	Whole-Exome Sequencing (WES)	116
7.2.1	What the Exome Is	116
7.2.2	Hybridisation Capture Workflow	116
7.2.3	When WES Is Preferred over WGS	117
7.2.4	Limitations of WES	117
7.2.5	Common Capture Kits	117
7.3	Targeted Sequencing and Gene Panels	117
7.3.1	Panel Design Strategies	117
7.3.2	Clinical Gene Panels	118
7.3.3	Ultra-Deep Sequencing for Low-Frequency Variants	118
7.4	Reduced Representation Genotyping: RADseq and GBS	119
7.4.1	The Core Principle	119
7.4.2	RADseq Protocol Variants	119
7.4.3	Genotyping by Sequencing (GBS)	120
7.4.4	Applications	120
7.4.5	Bioinformatics Pipeline	121
7.5	Comparison of Approaches	121
7.6	Visual Comparison: WGS vs. WES vs. Panel	121
7.7	Practical Bioinformatics: WGS Analysis Pipelines	123
7.7.1	Snakemake WGS Pipeline	123
7.7.2	Nextflow WGS Pipeline	126
7.8	Long-Read Alignment and Assembly Workflow	130
7.9	Quality Control Metrics for WGS/WES	131
7.10	Summary	132
8	Variant Discovery and Interpretation	133
8.1	Types of Genetic Variants	133
8.1.1	Single Nucleotide Variants (SNVs) and Single Nucleotide Polymorphisms (SNPs)	133
8.1.2	Insertions and Deletions (Indels)	133
8.1.3	Structural Variants (SVs)	134
8.1.4	Copy Number Variants (CNVs)	134
8.1.5	Repeat Expansions	134
8.1.6	Mobile Element Insertions	135
8.1.7	Complex Rearrangements	135
8.2	TikZ Diagram: Variant Types and Their Scale	135

8.3	Variant Calling Approaches	136
8.3.1	Germline Variant Calling	136
8.3.2	Somatic Variant Calling	136
8.3.3	Structural Variant Calling	137
8.3.4	Copy Number Variant Calling	138
8.3.5	Short Tandem Repeat Analysis	138
8.3.6	Haplotype Phasing	138
8.4	Variant Filtering and Quality Control	139
8.4.1	GATK Variant Quality Score Recalibration (VQSR)	139
8.4.2	Hard Filters	139
8.5	Statistical Foundations of Variant Calling	139
8.5.1	Pair-HMM for Read-Haplotype Alignment	139
8.5.2	Bayesian Genotyping	140
8.5.3	VQSR: The Gaussian Mixture Model in Detail	141
8.5.4	Population Genetics Connections	142
8.6	Variant Annotation	143
8.6.1	Functional Annotation	143
8.6.2	Population Frequency Databases	144
8.6.3	Pathogenicity Prediction	144
8.6.4	Clinical Variant Classification: ACMG/AMP Guidelines	145
8.7	Cancer Genomics	145
8.7.1	Tumor Mutational Burden (TMB)	145
8.7.2	Microsatellite Instability (MSI)	145
8.7.3	Mutational Signatures	146
8.7.4	Tumor Heterogeneity and Clonal Evolution	146
8.7.5	Liquid Biopsy and Cell-Free DNA (cfDNA)	146
8.8	Pharmacogenomics	147
8.9	Decision Flowchart: Variant Calling Pipeline Selection	147
8.10	Practical Workflow: Variant Discovery Pipeline	147
8.10.1	Snakemake Variant Discovery Pipeline	147
8.10.2	Nextflow Variant Discovery Pipeline	151
8.11	Variant Calling Tool Comparison	155
8.12	Summary	157
9	Metagenomics and Microbiome Sequencing	159
9.1	What Is Metagenomics?	159
9.1.1	The Microbiome: What It Is and Why It Matters	159
9.2	16S/18S/ITS Amplicon Sequencing	160
9.2.1	The 16S rRNA Gene	160
9.2.2	Other Marker Genes	161
9.2.3	ASVs vs. OTUs	161
9.2.4	Amplicon Analysis Tools	162
9.2.5	Taxonomic Classification Databases	162
9.3	Shotgun Metagenomics	162
9.3.1	Advantages over Amplicon Sequencing	162
9.3.2	Library Preparation Considerations	163
9.3.3	Taxonomic Profiling	164
9.3.4	Functional Profiling	164
9.3.5	Metagenome-Assembled Genomes (MAGs)	165
9.3.6	Strain-Level Analysis	166
9.4	Metatranscriptomics	166
9.5	Long-Read Metagenomics	167

9.6	Clinical Metagenomics	167
9.6.1	Pathogen Detection	167
9.6.2	Antimicrobial Resistance (AMR) Gene Detection	168
9.7	Diversity Metrics	168
9.7.1	Alpha Diversity (Within-Sample)	168
9.7.2	Beta Diversity (Between-Sample)	169
9.7.3	Ordination Methods	169
9.7.4	Mathematical Foundations of Diversity Indices	170
9.7.5	MAG Quality Standards: MIMAG Tiers in Detail	171
9.7.6	HUMAnN Pathway Abundance Model	172
9.8	TikZ Diagram: Amplicon vs. Shotgun Metagenomics	173
9.9	Comprehensive Tool Comparison	173
9.10	Practical Workflow: Metagenomics Pipelines	173
9.10.1	Snakemake Shotgun Metagenomics Pipeline	173
9.10.2	Nextflow Shotgun Metagenomics Pipeline	179
9.11	Long-Read Metagenomics Workflow Considerations	184
9.12	Summary	185
IV	Transcriptomics: Gene Expression and RNA Biology	187
10	RNA Sequencing: Measuring Gene Expression	189
10.1	Why Measure RNA?	189
10.1.1	Gene Expression as a Window into Biology	189
10.1.2	The Transcriptome	189
10.2	Bulk RNA-seq: An Overview of Approaches	190
10.2.1	mRNA-seq: Poly-A Selection	190
10.2.2	Total RNA-seq: Ribosomal RNA Depletion	191
10.2.3	Choosing Between mRNA-seq and Total RNA-seq	191
10.3	Library Preparation for RNA-seq	192
10.3.1	RNA Extraction and Quality Assessment	192
10.3.2	Strand-Specific Library Preparation	192
10.3.3	Low-Input RNA-seq	193
10.3.4	Degraded RNA and FFPE Samples	193
10.4	Small RNA Sequencing	194
10.4.1	Small RNA Species	194
10.4.2	Small RNA Library Preparation	195
10.4.3	miRNA-seq Analysis Workflow	195
10.5	Long-Read RNA Sequencing	195
10.5.1	PacBio Iso-Seq	196
10.5.2	Oxford Nanopore RNA Sequencing	196
10.5.3	Long-Read Isoform Analysis Tools	196
10.6	Nascent RNA Sequencing	197
10.7	RNA-seq Bioinformatics Workflow	197
10.7.1	Quality Control	197
10.7.2	Trimming	199
10.7.3	Alignment: Splice-Aware Aligners	199
10.7.4	Pseudo-Alignment and Lightweight Quantification	200
10.7.5	Gene and Transcript Quantification	200
10.7.6	Differential Expression Analysis	200
10.7.7	The Negative Binomial Distribution for RNA-seq	201
10.7.8	DESeq2 Dispersion Estimation and Shrinkage	201

10.7.9	Normalisation Methods for RNA-seq	202
10.7.10	DESeq2 vs. edgeR vs. limma-voom: A Detailed Comparison	203
10.7.11	Gene Set Enrichment Analysis	203
10.7.12	Isoform and Splicing Analysis	204
10.7.13	Fusion Gene Detection	205
10.8	Complete Snakemake Workflow for RNA-seq	205
10.9	Complete Nextflow Workflow for RNA-seq	205
10.10	Experimental Design for RNA-seq	205
10.10.1	Biological Replicates	205
10.10.2	Sequencing Depth	205
10.10.3	Batch Effects and Their Mitigation	208
10.11	RNA-seq Approaches: Comprehensive Comparison	208
10.12	Chapter Summary	209
11	Single-Cell RNA Sequencing	211
11.1	Why Single-Cell Resolution?	211
11.1.1	The Limitation of Bulk RNA-seq	211
11.1.2	What Single-Cell Resolution Enables	211
11.2	A Brief History of scRNA-seq	212
11.3	Cell Isolation Methods	212
11.3.1	FACS Sorting	212
11.3.2	Microfluidics	213
11.3.3	Droplet-Based Methods	213
11.3.4	Combinatorial Indexing	213
11.3.5	Plate-Based Methods	213
11.4	10x Genomics Chromium in Detail	214
11.4.1	Gel Bead-in-Emulsions (GEMs)	214
11.4.2	Cell Barcodes and UMIs	215
11.4.3	3' vs. 5' Gene Expression Chemistry	215
11.4.4	Cell Ranger Pipeline	215
11.4.5	Typical Throughput and Sequencing Recommendations	215
11.4.6	10x Chromium Flex (Fixed RNA Profiling)	215
11.5	Key Computational Challenges	216
11.5.1	Sparsity and Dropout	216
11.5.2	Doublet Detection	216
11.5.3	Ambient RNA Contamination	217
11.6	scRNA-seq Analysis Workflow	217
11.6.1	Raw Data Processing	219
11.6.2	Quality Control and Filtering	219
11.6.3	Normalization	219
11.6.4	Feature Selection: Highly Variable Genes	220
11.6.5	Dimensionality Reduction	220
11.6.6	Mathematical Foundations of t-SNE and UMAP	220
11.6.7	Clustering	222
11.6.8	Cell Type Annotation	222
11.6.9	Differential Expression	223
11.6.10	Trajectory Analysis and Pseudotime	223
11.6.11	Cell–Cell Communication	225
11.6.12	Gene Regulatory Network Inference	225
11.7	Analysis Frameworks: Scanpy vs. Seurat	225
11.8	Integration of Multiple Datasets	226
11.8.1	Batch Correction Methods: A Detailed Comparison	226

11.9 Large-Scale Cell Atlases	228
11.10 Practical Considerations	228
11.10.1 How Many Cells to Sequence?	228
11.10.2 Sequencing Depth Per Cell	229
11.10.3 Tissue Dissociation Considerations	229
11.11 Comparison of scRNA-seq Platforms	229
11.12 Snakemake Workflow for scRNA-seq	229
11.13 Nextflow Workflow for scRNA-seq	229
11.14 Chapter Summary	232
12 Spatial Transcriptomics	233
12.1 Why Spatial Context Matters	233
12.2 Categories of Spatial Technologies	233
12.2.1 Sequencing-Based Spatial Methods	234
12.2.2 Imaging-Based Spatial Methods (In Situ Hybridization)	235
12.2.3 In Situ Sequencing	237
12.3 Comparison of Spatial Transcriptomics Platforms	238
12.4 Computational Analysis for Spatial Transcriptomics	238
12.4.1 Spot Deconvolution	239
12.4.2 Spatially Variable Gene Detection	240
12.4.3 Spatial Domain Identification	241
12.4.4 Integration with scRNA-seq	242
12.4.5 Ligand–Receptor Analysis in Spatial Context	242
12.4.6 Data Visualization and Analysis Frameworks	243
12.5 Tissue Handling and Sample Preparation	244
12.5.1 Tissue Sectioning	244
12.5.2 Permeabilization Optimization	244
12.5.3 Imaging	245
12.6 Emerging Directions: Spatial Multi-Omics	245
12.6.1 Spatial Epigenomics	245
12.6.2 Spatial Proteomics	245
12.6.3 Spatial Multi-Omic Integration	245
12.7 Snakemake Workflow for Spatial Transcriptomics	245
12.8 Nextflow Workflow for Spatial Transcriptomics	247
12.9 Practical Experimental Design Considerations	247
12.9.1 Choosing the Right Technology	247
12.9.2 Replication and Sample Size	247
12.9.3 Combining Spatial and Single-Cell Data	247
12.10 Chapter Summary	249
V Epigenomics: Beyond the Sequence	251
13 DNA Methylation Sequencing	253
13.1 What Is DNA Methylation?	253
13.1.1 5-Methylcytosine and CpG Dinucleotides	253
13.1.2 CpG Islands	253
13.1.3 Biological Roles of DNA Methylation	254
13.1.4 Beyond 5mC: Other Cytosine Modifications	254
13.2 Bisulfite Sequencing: The Gold Standard	255
13.2.1 The Bisulfite Conversion Principle	255
13.2.2 Bisulfite Conversion Chemistry in Detail	255

13.2.3	Whole-Genome Bisulfite Sequencing (WGBS)	258
13.2.4	Reduced Representation Bisulfite Sequencing (RRBS)	259
13.3	Enzymatic Methyl-seq (EM-seq)	261
13.3.1	Principle	261
13.4	Distinguishing 5mC from 5hmC	261
13.4.1	Oxidative Bisulfite Sequencing (oxBS-seq)	261
13.4.2	TET-Assisted Bisulfite Sequencing (TAB-seq)	262
13.4.3	ACE-seq	262
13.5	Targeted Methylation Approaches	262
13.5.1	Methyl-Capture Sequencing	262
13.5.2	Methylation Arrays	262
13.6	Long-Read Methylation Detection	262
13.6.1	Nanopore Direct Methylation Detection	262
13.6.2	PacBio Methylation Detection	263
13.7	Comprehensive Comparison of Methylation Methods	263
13.8	Bioinformatics Workflow for Bisulfite Sequencing	263
13.8.1	Snakemake Workflow	265
13.8.2	Nextflow Workflow	267
13.8.3	Differential Methylation Analysis	270
13.8.4	Visualisation	273
13.9	Experimental Design Considerations	273
13.10	Chapter Summary	274
14	Chromatin Accessibility Profiling	275
14.1	Why Chromatin Accessibility Matters	275
14.2	DNase-seq	276
14.2.1	Principle	276
14.2.2	The ENCODE Project and DHS Mapping	276
14.3	ATAC-seq	276
14.3.1	Principle: Tn5 Transposase-Mediated Tagmentation	276
14.3.2	Standard ATAC-seq Protocol	278
14.3.3	ATAC-seq Protocol Variants	278
14.3.4	Tn5 Insertion Bias	278
14.4	Single-Cell ATAC-seq (scATAC-seq)	279
14.4.1	Technology and Platforms	279
14.4.2	The Sparsity Challenge	279
14.4.3	Analysis Tools and Frameworks	280
14.5	Other Accessibility and Nucleosome Methods	280
14.5.1	FAIRE-seq	280
14.5.2	MNase-seq	280
14.5.3	NOMe-seq	281
14.6	Comprehensive Comparison of Accessibility Methods	281
14.7	Bioinformatics Workflow for ATAC-seq	282
14.7.1	Analysis Overview	282
14.7.2	Snakemake Workflow	284
14.7.3	Nextflow Workflow	288
14.7.4	Differential Accessibility Analysis	292
14.7.5	Motif Analysis and Footprinting	292
14.7.6	TF Footprinting Scores: The TOBIAS Framework	292
14.7.7	Nucleosome Positioning from ATAC-seq Fragment Sizes	293
14.8	Experimental Design Considerations	295
14.9	Chapter Summary	295

15 Histone Modifications and Protein–DNA Interactions	297
15.1 The Histone Code	297
15.1.1 Histone Structure and Modifications	297
15.1.2 Key Histone Marks and Their Functions	297
15.1.3 Bivalent Chromatin	297
15.1.4 The Combinatorial Histone Code	298
15.2 ChIP-seq: Chromatin Immunoprecipitation Sequencing	298
15.2.1 Principle	299
15.2.2 Controls in ChIP-seq	299
15.2.3 Antibody Quality	300
15.2.4 Limitations of ChIP-seq	300
15.3 CUT&RUN: Cleavage Under Targets and Release Using Nuclease	300
15.3.1 Principle	300
15.4 CUT&Tag: Cleavage Under Targets and Tagmentation	301
15.4.1 Principle	301
15.4.2 Advantages and Limitations of CUT&Tag	301
15.5 Single-Cell CUT&Tag	302
15.6 PLAC-seq and HiChIP	303
15.7 RNA–Protein Interaction Methods	303
15.7.1 CLIP-seq / eCLIP	303
15.7.2 RIP-seq	303
15.7.3 DAP-seq	304
15.8 Bioinformatics Workflow for ChIP-seq/CUT&Tag	304
15.8.1 Analysis Overview	304
15.8.2 Snakemake Workflow	306
15.8.3 Nextflow Workflow	310
15.8.4 ChromHMM: Chromatin State Segmentation	313
15.8.5 Differential Binding Analysis	315
15.9 Experimental Design Considerations	316
15.10 Chapter Summary	316
16 Three-Dimensional Genome Architecture	317
16.1 Why Three-Dimensional Genome Organisation Matters	317
16.2 Levels of 3D Genome Organisation	318
16.2.1 Chromosome Territories	318
16.2.2 A/B Compartments	318
16.2.3 Topologically Associating Domains (TADs)	319
16.2.4 Chromatin Loops	319
16.3 Hi-C: Genome-Wide Chromosome Conformation Capture	319
16.3.1 Principle	319
16.3.2 In Situ Hi-C vs. Dilution Hi-C	320
16.3.3 Resolution and Sequencing Depth	321
16.3.4 Hi-C Library Preparation Kits	321
16.4 Micro-C	321
16.5 Targeted and Enrichment-Based 3C Methods	322
16.5.1 Capture Hi-C / Promoter Capture Hi-C	322
16.5.2 HiChIP and PLAC-seq	322
16.5.3 ChIA-PET	322
16.6 Long-Read and Alternative 3D Genome Methods	322
16.6.1 Pore-C (Oxford Nanopore)	322
16.6.2 SPRITE	323
16.6.3 GAM (Genome Architecture Mapping)	323

16.6.4	DNA-FISH and Live Imaging	323
16.7	Comprehensive Comparison of 3D Genome Methods	324
16.8	Bioinformatics Workflow for Hi-C	324
16.8.1	Analysis Overview	324
16.8.2	Contact Matrix Normalisation Methods	325
16.8.3	TAD Calling Algorithms	326
16.8.4	Compartment Analysis	327
16.8.5	Polymer Physics Models of Chromatin Folding	328
16.8.6	Snakemake Workflow	329
16.8.7	Nextflow Workflow	332
16.8.8	Visualisation	336
16.8.9	Integration with Other Epigenomic Data	336
16.9	Experimental Design Considerations	337
16.10	Chapter Summary	338

VI Multiomics and Emerging Frontiers 339

17 Single-Cell and Spatial Multiomics 341

17.1	The Promise of Multiomics	341
17.2	Joint RNA + Chromatin Accessibility	342
17.2.1	10x Genomics Multiome (ATAC + GEX)	342
17.2.2	SHARE-seq, SNARE-seq, and Paired-seq	342
17.2.3	Analysis: Linking Peaks to Genes	343
17.2.4	Peak-to-Gene Correlation Framework	343
17.2.5	Tools for Joint RNA + ATAC Analysis	345
17.2.6	Variational Autoencoders for Multiomics: scVI, MultiVI, and the ELBO 345	
17.2.7	Gene Regulatory Network Inference with SCENIC+	347
17.3	Joint RNA + Protein (Surface Markers)	349
17.3.1	CITE-seq: Antibody-Derived Tags + Transcriptome	349
17.3.2	REAP-seq and ASAP-seq	349
17.3.3	TotalVI for Joint RNA + Protein Analysis	349
17.3.4	Antibody Panel Design and Protein Normalization	349
17.4	Joint RNA + DNA	350
17.4.1	G&T-seq: Genome and Transcriptome from the Same Cell	350
17.4.2	scDNA-seq + scRNA-seq Integration	350
17.4.3	Lineage Tracing with CRISPR Barcodes	350
17.5	Triple-Omics and Beyond	351
17.5.1	TEA-seq and DOGMA-seq	351
17.5.2	scNMT-seq: Nucleosome, Methylation, and Transcription	351
17.6	Spatial Multiomics	351
17.6.1	Spatial Chromatin Accessibility	351
17.6.2	Spatial Proteomics	352
17.6.3	10x Visium + Immunofluorescence and DBiT-seq	352
17.7	Computational Integration Strategies	352
17.7.1	Vertical Integration (Same Cells, Multiple Modalities)	352
17.7.2	Weighted Nearest Neighbour (WNN) Mathematics	353
17.7.3	Horizontal Integration (Different Cells, Same Modality)	354
17.7.4	Diagonal Integration (Different Cells, Different Modalities)	354
17.7.5	Benchmarking: scIB and Beyond	354
17.8	Lineage Tracing Technologies	354
17.8.1	CRISPR-Based Lineage Barcoding	355

17.8.2	Natural Lineage Markers	355
17.8.3	Computational Tools for Lineage Reconstruction	355
17.9	Perturbation-seq Approaches	355
17.9.1	Perturb-seq and CROP-seq	355
17.9.2	ECCITE-seq: CRISPR + CITE-seq	356
17.9.3	Analysis of Perturbation Screens	356
17.10	Multiomics Workflow Example	356
17.10.1	Snakemake Workflow	356
17.10.2	Nextflow Workflow	359
17.11	Multiomics Approach Comparison	361
17.12	Visualising the Multiomics Landscape	361
17.13	Summary	361
18	Specialized Sequencing Applications	365
18.1	Ribosome Profiling (Ribo-seq)	365
18.1.1	Principle: Measuring Translation in Real Time	365
18.1.2	P-site Mapping and Translational Efficiency	366
18.1.3	P-site Offset Determination and Metagene Analysis	366
18.1.4	Tools for Ribo-seq Analysis	369
18.1.5	Discovering Novel Translated ORFs	369
18.2	CRISPR Screens	369
18.2.1	Pooled CRISPR Knockout Screens	369
18.2.2	CRISPRi and CRISPRa Screens	370
18.2.3	Guide RNA Library Design	370
18.2.4	Analysis Tools for CRISPR Screens	371
18.2.5	MAGeCK Statistical Framework	371
18.3	Cell-Free DNA and Liquid Biopsy	373
18.3.1	Circulating Tumor DNA (ctDNA)	373
18.3.2	Non-Invasive Prenatal Testing (NIPT)	373
18.3.3	Cancer Detection Applications	373
18.3.4	Fragmentomics	374
18.3.5	cfDNA Fragmentomics: From Size Distributions to Nucleosome Foot- printing	374
18.3.6	Methylation-Based Cancer Detection	376
18.3.7	Technical Challenges and Error Correction	376
18.4	Adaptive Sampling (Oxford Nanopore)	376
18.4.1	Real-Time Selective Sequencing	376
18.4.2	Software and Applications	377
18.5	Optical Genome Mapping (Bionano Saphyr)	377
18.6	Epitranscriptomics: RNA Modifications	378
18.6.1	The RNA Modification Landscape	378
18.6.2	Sequencing-Based Detection of RNA Modifications	379
18.7	Telomere and Centromere Sequencing: T2T Approaches	379
18.8	Chromosome Conformation in Clinical Settings	380
18.9	Practical Workflow: Ribo-seq Processing	380
18.9.1	Snakemake Workflow	380
18.9.2	Nextflow Workflow	382
18.10	Comparison of Specialized Applications	385
18.11	Summary	385

19 Emerging Technologies and Future Directions	387
19.1 Protein Sequencing by Sequencing-Adjacent Technologies	387
19.1.1 Quantum-Si: Single-Molecule Protein Sequencing	387
19.1.2 Nautilus Biotechnology: Proteome Analysis Platform	388
19.1.3 Nanopore-Based Protein Sequencing: Signal Models and Challenges	388
19.2 In Situ Sequencing in Tissues	389
19.2.1 Key In Situ Sequencing Technologies	389
19.3 Single-Cell Proteomics by Mass Spectrometry	390
19.4 Programmable Biology and Synthetic Long Reads	390
19.4.1 Linked-Read Technologies	390
19.4.2 Universal Sequencing Technology	391
19.5 Ultra-Long Nanopore Sequencing Advances	391
19.5.1 Ultra-High Molecular Weight DNA Extraction	391
19.5.2 Applications of Ultra-Long Reads	391
19.6 AI and Machine Learning in Sequencing	392
19.6.1 Neural Network Basecallers	392
19.6.2 AI-Powered Variant Calling	392
19.6.3 Genomic Language Models and Foundation Models	392
19.6.4 Transformer Architecture for Genomic Sequences	393
19.6.5 AlphaFold and the Sequence-to-Structure Connection	394
19.7 Cloud and On-Premise Computing	394
19.7.1 Major Cloud Platforms for Genomics	394
19.7.2 GPU Acceleration	395
19.8 Portable and Point-of-Care Sequencing	396
19.8.1 Field Sequencing Deployments	396
19.8.2 Clinical Rapid Sequencing	396
19.8.3 Veterinary and Agricultural Diagnostics	397
19.9 Democratising Genomics and Healthcare	397
19.9.1 The Falling Cost of Personal Genome Sequencing	397
19.9.2 Options for Obtaining Your Genome Sequence	398
19.9.3 Processing Your Own Genome: A Practical Guide	398
19.9.4 Equity and the Digital Divide in Genomic Medicine	400
19.9.5 The Personal Genome as a Longitudinal Health Tool	401
19.10 Integration of Multi-Omics with Electronic Health Records	401
19.11 Pangenomics: Moving Beyond Single Reference Genomes	402
19.11.1 The Limitations of a Single Reference	402
19.11.2 The Human Pangenome Reference Consortium	402
19.11.3 Graph-Based References	402
19.11.4 Pangenome Graph Formalism and the GFA Format	403
19.11.5 Population-Specific References	404
19.12 Ethical Considerations	404
19.12.1 Genetic Privacy and Insurance Discrimination	404
19.12.2 Direct-to-Consumer Genetic Testing	404
19.12.3 Incidental Findings in Clinical Sequencing	405
19.12.4 Data Sovereignty for Indigenous Populations	405
19.12.5 Equitable Access to Sequencing Technologies	405
19.13 Future Outlook: Where Is the Field Heading?	405
19.14 Practical Workflow: Pangenome Alignment	406
19.14.1 Snakemake Workflow	406
19.14.2 Nextflow Workflow	408
19.15 Visualising the Evolving Technology Landscape	410

19.16	Summary	411
VII	Bioinformatics and Data Analysis	413
20	Bioinformatics Fundamentals	415
20.1	What Is Bioinformatics?	415
20.2	Computing Environment Setup	415
20.2.1	Linux/Unix Basics for Bioinformatics	415
20.2.2	Package Managers	416
20.2.3	Containers: Docker and Singularity/Apptainer	417
20.2.4	HPC Clusters: SLURM, PBS/Torque, SGE	417
20.2.5	Cloud Computing for Genomics	418
20.3	Bioinformatics File Formats	419
20.3.1	FASTA: Reference Sequences	419
20.3.2	FASTQ: Raw Sequencing Reads	419
20.3.3	SAM/BAM/CRAM: Alignment Files	420
20.3.4	BED: Genomic Intervals	421
20.3.5	GFF/GTF: Gene Annotations	422
20.3.6	VCF/BCF: Variant Calls	422
20.3.7	BigWig and BedGraph: Coverage Tracks	422
20.3.8	HDF5 and AnnData (h5ad): Single-Cell Data	423
20.3.9	Hi-C Contact Matrices: Pairs, Cool, and .hic	423
20.3.10	10x Genomics MEX Format	423
20.4	Key Bioinformatics Tools	423
20.4.1	samtools: The Swiss Army Knife of BAM Files	423
20.4.2	bedtools: Interval Arithmetic	424
20.4.3	Picard: Duplicate Marking and Metrics	424
20.4.4	IGV: Visual Inspection	425
20.4.5	deepTools: Coverage and Visualization	425
20.4.6	MultiQC: Aggregating QC Reports	425
20.5	Algorithms and Data Structures for Sequence Analysis	426
20.5.1	Suffix Arrays	426
20.5.2	Burrows-Wheeler Transform and FM-Index	426
20.5.3	Smith-Waterman vs Seed-and-Extend	427
20.5.4	Algorithm Complexity Reference	428
20.6	Programming Languages for Bioinformatics	428
20.6.1	Python for Bioinformatics	429
20.6.2	R for Bioinformatics	429
20.6.3	When to Use Which Language	430
20.7	Workflow Management Systems	430
20.7.1	Why Workflow Managers?	431
20.7.2	Snakemake	431
20.7.3	Nextflow	433
20.7.4	Snakemake vs Nextflow: Detailed Comparison	436
20.7.5	WDL (Workflow Description Language)	436
20.7.6	nf-core: Community-Curated Pipelines	437
20.8	Version Control with Git	437
20.9	Reproducibility Best Practices	438
20.10A	Typical Bioinformatics Computing Stack	439
20.11	Summary	439

21 Complete Analysis Workflows	443
21.1 WGS Variant Calling Pipeline	443
21.1.1 Pipeline Overview	443
21.1.2 Complete Snakemake WGS Workflow	444
21.1.3 Complete Nextflow WGS Workflow	447
21.2 RNA-seq Differential Expression Pipeline	450
21.2.1 Pipeline Overview	450
21.2.2 Complete Snakemake RNA-seq Workflow	450
21.2.3 Complete Nextflow RNA-seq Workflow	452
21.3 ATAC-seq Pipeline	454
21.3.1 Pipeline Overview	454
21.3.2 Complete Snakemake ATAC-seq Workflow	454
21.3.3 Complete Nextflow ATAC-seq Workflow	456
21.4 ChIP-seq / CUT&Tag Pipeline	459
21.4.1 Pipeline Overview	459
21.4.2 Complete Snakemake ChIP-seq Workflow	459
21.4.3 Complete Nextflow ChIP-seq Workflow	461
21.5 scRNA-seq Pipeline	463
21.5.1 Pipeline Overview	463
21.5.2 Complete Snakemake scRNA-seq Workflow	463
21.5.3 Complete Nextflow scRNA-seq Workflow	464
21.6 Single-Cell Quantification Tool Comparison	466
21.6.1 Cell Ranger vs STARsolo vs alevin-fry	466
21.6.2 Single-Cell Tool Recommendation Table	467
21.7 Bisulfite Sequencing Pipeline	467
21.7.1 Pipeline Overview	468
21.7.2 Complete Snakemake Bisulfite-seq Workflow	468
21.7.3 Complete Nextflow Bisulfite-seq Workflow	470
21.8 16S Amplicon Metagenomics Pipeline	471
21.8.1 Pipeline Overview	471
21.8.2 Complete Snakemake 16S Workflow	472
21.8.3 Complete Nextflow 16S Workflow	473
21.9 Master Comparison of nf-core Pipelines	474
21.10 General Best Practices for All Workflows	475
21.11 Resource Requirements by Analysis Type	476
21.12 Error Handling Patterns in Workflows	476
21.12.1 Common Failure Modes	476
21.12.2 Error Handling in Snakemake	477
21.12.3 Error Handling in Nextflow	477
21.13 Summary	478
22 Experimental Design and Best Practices	479
22.1 Why Experimental Design Matters	479
22.2 Biological Replicates vs Technical Replicates	479
22.2.1 Minimum Replicates by Assay Type	480
22.3 Sequencing Depth Guidelines	481
22.4 Read Length Considerations	481
22.5 Power Analysis	481
22.5.1 RNA-seq Power Analysis	481
22.5.2 Formal Power Analysis for Differential Expression	484
22.5.3 The Benjamini-Hochberg Procedure	485
22.5.4 Pilot Study Design	485

22.5.5	Cost-Optimisation Model	487
22.6	Batch Effects	488
22.6.1	Common Sources of Batch Effects	488
22.6.2	How to Minimize Batch Effects	489
22.6.3	Computational Batch Effect Correction	489
22.7	Controls	490
22.7.1	Sequencing Controls	490
22.7.2	Assay-Specific Controls	490
22.8	Multiplexing Strategy	491
22.8.1	How Many Samples per Flow Cell	491
22.8.2	Index Compatibility and Distance	491
22.9	Sample Collection and Storage	492
22.9.1	DNA Storage	492
22.9.2	RNA Storage	492
22.10	Cost Estimation and Budgeting	493
22.11	Decision Tree for Choosing a Sequencing Strategy	493
22.12	Comprehensive Experimental Design Reference Table	493
22.13	Summary	495
23	Data Standards, Sharing, and Reproducibility	497
23.1	Why Data Sharing and Standards Matter	497
23.2	Public Repositories for Raw Sequencing Data	498
23.2.1	The International Nucleotide Sequence Database Collaboration (INSDC)	498
23.2.2	How to Submit Data to SRA/ENA	498
23.2.3	How to Download Data from SRA/ENA	499
23.2.4	Snakemake Workflow for SRA Data Download	499
23.2.5	Nextflow Workflow for SRA Data Download	500
23.3	Processed Data Repositories	500
23.3.1	Gene Expression Omnibus (GEO)	500
23.3.2	ArrayExpress / BioStudies	501
23.3.3	Consortium Data Portals	501
23.4	Reference Genomes and Annotations	502
23.4.1	Where to Download References	502
23.4.2	GENCODE vs RefSeq	502
23.4.3	Genome Builds and LiftOver	502
23.5	Metadata Standards	503
23.5.1	Key Metadata Standards	503
23.5.2	Ontologies for Standardized Terminology	503
23.6	FAIR Principles	504
23.7	Reproducibility in Bioinformatics	504
23.7.1	Pillars of Computational Reproducibility	504
23.7.2	Example Reproducible Project Structure	505
23.7.3	Snakemake Example for Reproducible Analysis	505
23.7.4	Nextflow Example for Reproducible Analysis	506
23.8	Data Management Plans	507
23.8.1	What Funding Agencies Require	507
23.8.2	Storage Considerations	507
23.8.3	Data Security and Patient Consent	508
23.8.4	HIPAA vs GDPR Detailed Comparison	508
23.8.5	Controlled Access Repositories and Federated Analysis	510
23.8.6	Cross-Repository Data Discovery	510
23.8.7	Long-Term Storage Cost Modelling	511

23.9 Community Standards and Consortia	513
23.10 The Data Lifecycle	513
23.11 Practical Data Submission Checklist	515
23.12 Summary	515
VIII Democratisation of Sequencing	517
24 Accessible Sequencing: Democratising Genomics	519
24.1 The Landscape of Accessible Sequencing	519
24.2 Oxford Nanopore for the Individual Researcher	520
24.2.1 MinION and Mk1C	520
24.2.2 Flongle: The Disposable Flow Cell	521
24.2.3 PromethION P2 Solo	521
24.2.4 Computing Requirements for Nanopore Basecalling	522
24.3 Budget Planning and Cost Modelling	523
24.3.1 Itemised Cost Models	523
24.3.2 Reagent Shelf Life	524
24.3.3 Shared Equipment Models and Cost-per-Sample Analysis	524
24.4 Community Labs and Maker Spaces	525
24.4.1 Prominent Community Laboratories	525
24.4.2 Setting Up a Sequencing Corner	525
24.4.3 Minimum Equipment for a Sequencing Laboratory	526
24.5 Field Sequencing	526
24.5.1 Power Requirements	526
24.5.2 Temperature Constraints	527
24.5.3 Data Transfer in the Field	528
24.5.4 Case Studies in Field Sequencing	528
24.5.5 Field Sequencing Setup	528
24.6 Educational and Outreach Applications	530
24.6.1 Designing a Teaching Laboratory Sequencing Module	530
24.6.2 Suggested Curriculum Outline	530
24.6.3 Cost Per Student	531
24.6.4 High School Protocols	531
24.7 What Can You Realistically Do with \$1,000?	532
24.8 Citizen Science Sequencing Projects	533
24.8.1 Environmental DNA Barcoding	533
24.8.2 Food Fraud Detection	534
24.8.3 Antimicrobial Resistance Monitoring in Waterways	534
24.8.4 Concrete Project Examples	535
24.9 Legal, Ethical, and Biosafety Considerations	535
24.9.1 The Nagoya Protocol and Access and Benefit Sharing	536
24.9.2 Dual-Use Research Concerns	536
24.9.3 Ethics Approval for Human Samples	537
24.9.4 Data Sharing Obligations	537
24.9.5 Biosafety Levels and Regulated Organisms	538
24.9.6 Regulatory Frameworks by Region	538
24.10 Building a Sustainable Accessible Sequencing Programme	539
24.10.1 Funding Models	539
24.10.2 Training and Skill Development	539
24.10.3 Data Quality and Reproducibility	540
24.11 Looking Forward: The Future of Accessible Sequencing	540

24.12	Summary	541
25	Practical Protocols: From Sample to Sequencer	543
25.1	Safety First: Laboratory Hazards and Precautions	543
25.1.1	Biosafety Levels	543
25.1.2	Personal Protective Equipment (PPE)	544
25.1.3	Chemical Hazards in Sequencing Workflows	544
25.1.4	Waste Disposal	544
25.2	DNA Extraction Protocols	546
25.2.1	Protocol A: DNA from Human Saliva	546
25.2.2	Protocol B: Environmental DNA (eDNA) from Water Samples	548
25.2.3	Protocol C: DNA from Food and Plant Samples	551
25.2.4	QC Decision Tree	553
25.3	Nanopore Rapid Barcoding Kit (SQK-RBK114.96)	553
25.4	Nanopore Ligation Sequencing Kit (SQK-LSK114)	556
25.4.1	AMPure XP Bead Ratio Optimisation	558
25.5	16S/ITS Amplicon Library Preparation	558
25.5.1	Primer Selection	559
25.5.2	PCR Cycling Conditions	559
25.5.3	Barcoding Strategy and Controls	560
25.6	Loading and Running the Flow Cell	560
25.6.1	Flow Cell QC Check	560
25.6.2	Flow Cell Priming	561
25.6.3	Library Loading	562
25.6.4	MinKNOW Run Setup and Monitoring	563
25.6.5	Nuclease Flush and Reload	563
25.7	Troubleshooting	564
25.8	Sample Collection Best Practices	564
25.8.1	Field Preservation Methods	564
25.8.2	Storage Temperature Requirements	566
25.8.3	Chain of Custody Documentation	567
25.8.4	Metadata Recording	567
25.9	Chapter Summary	568
26	DIY Analysis: From Raw Reads to Results	569
26.1	Setting Up Your Environment	569
26.1.1	Hardware Requirements	569
26.1.2	Installing Software with Conda	570
26.1.3	Test Datasets	571
26.2	Basecalling with Dorado	571
26.2.1	Basecalling Models	571
26.2.2	Running Dorado	572
26.2.3	Understanding BAM Output	573
26.3	Quality Control	573
26.3.1	NanoPlot: Nanopore-Specific QC	573
26.3.2	pycoQC: Sequencing Run QC	574
26.3.3	FastQC and MultiQC	574
26.3.4	QC Decision Table	575
26.4	Walkthrough 1: Bacterial Genome Assembly	575
26.4.1	Step 1: Read Filtering with Chopper	576
26.4.2	Step 2: Assembly with Flye	576
26.4.3	Step 3: Polishing with Medaka	577

26.4.4	Step 4: Assembly Assessment with QUAST	577
26.4.5	Step 5: Completeness with BUSCO	578
26.4.6	Step 6: Annotation with Bakta	578
26.4.7	Expected Results	579
26.4.8	Complete Snakemake Workflow	579
26.4.9	Complete Nextflow Workflow	582
26.5	Walkthrough 2: Species Identification with Kraken2	585
26.5.1	Database Setup	585
26.5.2	Running Classification	586
26.5.3	Abundance Estimation with Bracken	586
26.5.4	Interactive Visualization with Krona	587
26.5.5	Example 1: Environmental DNA from a Pond	587
26.5.6	Example 2: Sushi Species Authentication	588
26.5.7	Interpretation Guidelines	589
26.6	Walkthrough 3: Variant Calling	589
26.6.1	Step 1: Alignment with minimap2	589
26.6.2	Step 2: Variant Calling with Clair3	590
26.6.3	Step 3: Variant Filtering with bcftools	590
26.6.4	Expected Variant Counts	591
26.6.5	Comparison: Clair3 vs Medaka for Variant Calling	591
26.7	Visualization with IGV	592
26.7.1	Loading and Navigating Data	592
26.7.2	Interpreting Alignment Displays	592
26.7.3	Batch Screenshots	593
26.8	Galaxy: No-Code Option	593
26.8.1	Getting Started with Galaxy	593
26.8.2	Building Visual Workflows	594
26.8.3	Limitations vs Command Line	594
26.9	Cloud Computing on a Budget	594
26.9.1	When Local Isn't Enough	594
26.9.2	Cloud Platform Comparison	595
26.9.3	Google Colab for Small Analyses	595
26.9.4	Terra for Production Genomics	596
26.10	Complete End-to-End Snakemake Workflow	596
26.11	Complete End-to-End Nextflow Workflow	600
26.12	Putting It All Together: A Practical Checklist	606

List of Acronyms

609

List of Figures

1.1	Cost per human genome over time.	6
1.2	The general sequencing workflow.	10
1.3	Sequencing platform comparison.	14
2.1	Simplified DNA structure.	21
2.2	Chromatin organisation hierarchy.	23
2.3	The Central Dogma of molecular biology.	25
3.1	Sequencing by synthesis (SBS) principle.	39
3.2	Nanopore sequencing principle.	41
3.3	SMRT sequencing principle.	42
3.4	GC bias in sequencing.	52
4.1	Illumina bridge amplification	58
4.2	Illumina SBS cycle	59
4.3	Short-read platform decision guide	68
5.1	Zero-mode waveguide (ZMW) principle	73
5.2	Nanopore sequencing principle	76
6.1	General library preparation workflow	88
6.2	Tagmentation with Tn5 transposase	92
6.3	Illumina Y-adapter structure	93
6.4	Target enrichment decision flowchart	102
6.5	Library preparation decision flowchart	104
7.1	Visual comparison of genomic coverage achieved by whole-genome sequencing, whole-exome sequencing, and targeted gene panels. WGS covers the entire genome at moderate depth; WES selectively captures exonic regions at higher depth; targeted panels focus on a small gene set at very high depth.	122
8.1	The spectrum of genetic variation, from single nucleotide variants (SNVs) to complex structural rearrangements. Each variant class requires different detection strategies and tools. Sizes are shown on a logarithmic scale.	135
8.2	Decision flowchart for selecting appropriate variant calling tools based on the experimental design, data type, and variant classes of interest.	148
9.1	Comparison of 16S amplicon sequencing and shotgun metagenomics workflows. Amplicon sequencing uses targeted PCR amplification of phylogenetic markers, while shotgun metagenomics sequences all DNA, enabling functional profiling, strain-level resolution, and genome-resolved metagenomics.	174

10.1	Schematic of the dUTP strand-specific library preparation method. The second-strand cDNA is synthesized with dUTP (dashed orange) and subsequently digested by USER enzyme, ensuring that only the first-strand cDNA (blue) is amplified. This preserves strand-of-origin information.	193
10.2	Overview of the bulk RNA-seq bioinformatics workflow. Two main paths exist after trimming: alignment-based quantification (left, using STAR/HISAT2 followed by featureCounts) or pseudo-alignment-based quantification (right, using Salmon/kallisto). Both converge at differential expression analysis. . . .	198
11.1	Schematic of the 10x Genomics Chromium droplet-based scRNA-seq workflow. Single cells and gel beads are co-encapsulated in Gel Bead-in-Emulsions (GEMs), where cell lysis and reverse transcription occur. Each gel bead carries millions of oligonucleotide copies with a shared cell barcode and diverse UMIs. The resulting cDNA is uniquely labeled with a cell barcode and a UMI for each original mRNA molecule.	214
11.2	Standard computational workflow for scRNA-seq analysis. After generating a cell-by-gene count matrix, cells are filtered, normalized, dimensionality-reduced, clustered, and annotated. Downstream analyses include differential expression, trajectory inference, and cell-cell communication.	218
12.1	Overview of spatial transcriptomics technology categories. Sequencing-based methods (left) capture RNA at spatially barcoded positions and sequence it off-tissue. Imaging-based methods (center) visualize individual RNA molecules in situ using fluorescent probes. In situ sequencing methods (right) perform the sequencing reaction directly within the tissue section.	234
13.1	Bisulfite conversion chemistry for methylation detection.	256
14.1	ATAC-seq principle: Tn5 transposase tagmentation of open chromatin.	277
15.1	Comparison of ChIP-seq, CUT&RUN, and CUT&Tag workflows.	302
16.1	Hierarchical levels of 3D genome organisation.	318
16.2	Hi-C proximity ligation principle.	320
17.1	The single-cell multiomics landscape.	362
18.1	Principle of ribosome profiling (Ribo-seq).	367
19.1	The evolving sequencing technology landscape.	410
19.2	Sequencing cost trajectory: 2001–2026.	411
20.1	Bioinformatics computing stack.	440
22.1	Balanced batch design.	489
22.2	Sequencing strategy decision tree.	493
23.1	The NGS data lifecycle.	514
24.1	Field sequencing setup	529
25.1	DNA QC decision tree	554

List of Tables

1.1	Biological layers measurable by sequencing.	9
1.2	Quantitative comparison of sequencing generations.	13
2.1	Nucleobase pKa values and tautomeric equilibria.	19
2.2	Polymerase kinetics comparison.	25
3.1	Typical coverage requirements for common sequencing applications.	45
3.2	Effect of single-end ligation efficiency on overall library yield.	47
3.3	Phred quality scores and corresponding error probabilities.	48
3.4	Basecaller model comparison.	49
3.5	Comparison of major sequencing platforms.	53
4.1	Signal decay due to phasing.	61
4.2	Two-channel base calling scheme.	61
4.3	Index hopping rates by platform and mitigation strategy.	62
4.4	Short-read sequencing platform comparison	67
5.1	CCS accuracy vs. number of passes.	75
5.2	Homopolymer length accuracy.	78
5.3	PacBio HiFi vs ONT comparison	83
5.4	Hybrid sequencing strategies	84
6.1	SPRI bead ratio and size cutoff.	95
6.2	Library preparation troubleshooting guide	103
7.1	Comparison of major exome capture kits.	118
7.2	Comparison of reduced representation genotyping approaches.	121
7.3	Comprehensive comparison of WGS, WES, and targeted panel sequencing. . .	122
7.4	Essential QC metrics for WGS and WES data.	131
8.1	Structural variant callers for short-read and long-read data.	137
8.2	GATK recommended hard filters for SNVs and indels.	139
8.3	Major population frequency databases.	144
8.4	Comparison of variant calling tools for different variant classes and use cases. .	156
9.1	Commonly targeted 16S rRNA variable regions.	161
9.2	Major taxonomic reference databases for amplicon sequencing.	163
9.3	MIMAG quality tiers for metagenome-assembled genomes.	171
9.4	Comparison of key metagenomics analysis tools by application.	175
10.1	Comparison of mRNA-seq (poly-A selection) and total RNA-seq (rRNA depletion) approaches.	191

10.2	Detailed comparison of the three major differential expression tools for bulk RNA-seq.	204
10.3	Recommended sequencing depths for common RNA-seq applications.	208
10.4	Comprehensive comparison of RNA-seq approaches.	209
11.1	Comparison of the Scanpy (Python) and Seurat (R) scRNA-seq analysis frameworks.	225
11.2	Comparison of batch correction and integration methods for scRNA-seq. . . .	227
11.3	Comparison of major scRNA-seq platforms and methods.	232
12.1	Comprehensive comparison of spatial transcriptomics platforms.	238
12.2	Extended comparison including additional platforms.	238
13.1	Comparison of bisulfite-seq alignment tools.	260
13.2	Comparison of DNA methylation profiling methods.	264
14.1	Comparison of chromatin accessibility profiling methods.	281
14.2	ATAC-seq fragment size classes and their biological interpretation.	294
15.1	Key histone modifications and their biological meanings.	298
16.1	Hi-C resolution as a function of sequencing depth (human genome).	321
16.2	Comparison of 3D genome profiling methods.	324
17.1	Comparison of GRN inference methods for single-cell multiomics.	348
17.2	Comparison of single-cell multiomics technologies.	362
18.1	Comparison of CRISPR screen analysis tools.	372
18.2	Comparison of specialized sequencing applications.	385
19.1	Pathways for obtaining personal genome sequencing at different price points (2024 prices, approximate).	401
20.1	Key fields in SAM/BAM alignment records.	420
20.2	Time and space complexity of key sequence analysis algorithms.	428
20.3	Choosing between Python and R for bioinformatics tasks.	430
20.4	Comprehensive comparison of Snakemake and Nextflow workflow management systems.	437
21.1	Comparison of single-cell quantification tools for 10x Chromium data.	468
21.2	Summary of key nf-core pipelines for NGS data analysis.	475
21.3	Approximate computational resource requirements per sample for common NGS analyses. Values assume a human genome and typical parameter settings.	476
22.1	Recommended minimum biological replicates by assay type.	480
22.2	Recommended sequencing depth by application.	482
22.3	Read length recommendations by application.	483
22.4	Comparison of multiple testing correction methods.	486
22.5	Master experimental design reference: depth, replicates, read length, and key considerations for common NGS assays.	494
23.1	Comparison of GENCODE and RefSeq annotations.	502
23.2	Typical data volumes and storage requirements for NGS projects.	508
23.3	Comparison of HIPAA (USA) and GDPR (EU) for genomic data.	509
23.4	AWS S3 storage tiers and approximate costs (as of 2025). Costs vary by region.	512

24.1 Comparison of accessible sequencing platforms	520
24.2 Basecalling compute benchmarks	522
24.3 Itemised cost models for five accessible sequencing projects	523
24.4 Minimum equipment for a nanopore sequencing laboratory	527
24.5 Feasibility matrix: what \$1,000 can accomplish	532
24.6 Citizen science sequencing project examples	535
25.1 Common chemical hazards encountered in NGS sample preparation workflows.	545
25.2 Materials list for saliva DNA extraction.	547
25.3 Troubleshooting guide for saliva DNA extraction.	548
25.4 Typical eDNA yields from 1 L water samples using DNeasy PowerWater kit.	551
25.5 PCR inhibitor mitigation strategies for challenging samples.	553
25.6 SQK-RBK114.96 kit specifications and input requirements.	555
25.7 AMPure XP bead ratio and approximate fragment size cutoff for DNA selection.	558
25.8 Recommended primers for full-length amplicon nanopore sequencing.	559
25.9 PCR cycling conditions for full-length 16S and ITS amplicon generation.	559
25.10 Comprehensive troubleshooting guide for nanopore sequencing workflows.	565
25.12 Recommended storage conditions and maximum holding times for common sample types.	566
25.13 Essential metadata fields for sequencing sample documentation.	567
26.1 Hardware requirements for nanopore data analysis.	570
26.2 Recommended test datasets from the SRA and other public repositories.	571
26.3 Dorado basecalling model comparison (R10.4.1 chemistry, v4.3.0 models).	572
26.4 QC metrics, acceptable ranges, and recommended actions for nanopore data.	575
26.5 Expected results for bacterial genome assembly from nanopore data (SUP basecalling, Flye + Medaka).	579
26.6 Expected variant counts per human genome (30× ONT, SUP basecalling, Clair3).	591
26.7 Comparison of Clair3 and Medaka for variant calling from nanopore data.	592
26.8 Galaxy vs command-line analysis: trade-offs.	594
26.9 Cloud computing options for nanopore analysis (approximate pricing as of 2024).	595

Part I

Foundations

Chapter 1

Introduction: The Sequencing Revolution

Key Concept

Sequencing is the process of determining the precise order of nucleotides—the chemical “letters”—within a molecule of DNA or RNA. This seemingly simple act of reading biological information has become one of the most transformative technologies in the history of science, reshaping medicine, agriculture, ecology, forensics, and our fundamental understanding of life itself.

1.1 What Is Sequencing and Why Does It Matter?

Every living organism on Earth carries an instruction manual written in the language of DNA. This manual, called the **genome**, contains the information needed to build and maintain an organism—from the simplest bacterium to the most complex whale. **Sequencing** is the technology that allows us to read this manual, letter by letter.

To appreciate why this matters, consider an analogy. Imagine you have discovered an ancient library filled with millions of books, written in an alphabet you have only partially deciphered. These books contain the instructions for building every type of cell in the human body, the blueprints for every protein that carries out life’s functions, and the regulatory logic that determines when and where each gene is activated. Sequencing is the technology that lets you read these books. Bioinformatics is the discipline that helps you understand what they say.

The implications are staggering:

- A physician can sequence a tumour’s genome to identify the exact mutations driving a patient’s cancer—and choose a drug that targets those specific mutations.
- An ecologist can sequence environmental DNA (eDNA) from a water sample to catalogue every fish species present in a river, without catching a single fish.
- A forensic scientist can identify a crime suspect from trace amounts of DNA left at a scene.
- An agricultural scientist can sequence crop genomes to breed varieties that resist drought or disease.
- An epidemiologist can track the real-time evolution of a pandemic virus by sequencing thousands of viral genomes per week.

None of this was possible fifty years ago. The story of how we got from reading a single gene to reading entire genomes in a matter of hours is one of the most remarkable technological narratives of the modern era.

1.2 A Brief History of Sequencing

1.2.1 The Sanger Era (1977–2000)

The story of DNA sequencing begins in 1977, when the British biochemist Frederick Sanger developed a method now known as **Sanger sequencing** (also called the **chain-termination method** or **dideoxy sequencing**). The core idea was elegant: Sanger used modified nucleotides—called **dideoxynucleotides** (ddNTPs)—that, when incorporated into a growing DNA strand, terminate the chain at specific positions. By running these terminated fragments on a gel and reading their lengths, researchers could deduce the order of bases in the original DNA molecule.

Sanger sequencing could read approximately 500–1,000 bases at a time, which was revolutionary for its era. Sanger himself used the method to sequence the 5,386-nucleotide genome of bacteriophage ϕ X174, the first complete DNA genome ever sequenced. For this and related work, Sanger received his second Nobel Prize in Chemistry in 1980.

Sanger Sequencing

Sanger sequencing reads DNA by incorporating chain-terminating dideoxynucleotides. Each termination event marks the position of a specific base. By separating fragments by size, the sequence can be reconstructed. This method remained the gold standard for over 25 years and is still used today for targeted validation of short DNA regions.

Throughout the 1980s and 1990s, Sanger sequencing was progressively automated. Fluorescently labelled terminators replaced radioactive labels, capillary electrophoresis replaced slab gels, and robotic systems allowed higher throughput. These improvements set the stage for the most ambitious biological project in history.

1.2.2 The Human Genome Project (1990–2003)

In 1990, an international consortium of scientists launched the **Human Genome Project (HGP)**, an effort to determine the complete sequence of the approximately 3.2 billion base pairs in the human genome. The project was a monumental undertaking: it required over a decade of work by thousands of scientists across 20 institutions in six countries, at a total cost of roughly \$2.7 billion (in 1991 dollars).

A competing private effort, led by Craig Venter and Celera Genomics, used a “shotgun” approach—fragmenting the genome into small pieces, sequencing them all in parallel, and using powerful computers to assemble the overlapping fragments. Both the public consortium and Celera published draft sequences simultaneously in February 2001, and a more complete reference genome was declared finished in April 2003, two years ahead of schedule.

The Human Genome Project achieved several milestones beyond the sequence itself:

- It drove the development of computational tools for sequence assembly and analysis.
- It established norms for open data sharing in genomics (the “Bermuda Principles” required sequence data to be released within 24 hours of generation).
- It catalysed the development of faster, cheaper sequencing technologies.
- It revealed that the human genome contains only about 20,000–25,000 protein-coding genes—far fewer than the 100,000 initially predicted.

1.2.3 The Next-Generation Sequencing Revolution (2005–2015)

The limitations of Sanger sequencing were clear: even with automation, reading an entire human genome required years and billions of dollars. The scientific community needed fundamentally new approaches.

Beginning around 2005, a wave of new technologies—collectively called **next-generation sequencing (NGS)**—dramatically changed the landscape. Instead of reading one DNA fragment at a time (as in Sanger sequencing), these platforms could read millions to billions of fragments *simultaneously*. This massive parallelism was the key innovation.

The major milestones in this era include:

- **2005:** The 454 Life Sciences GS20 became the first commercially available NGS platform, using pyrosequencing (sequencing by synthesis with light detection).
- **2006:** Solexa (later acquired by Illumina) released the Genome Analyzer, which used reversible-terminator sequencing by synthesis. This approach would come to dominate the market.
- **2007:** Applied Biosystems released the SOLiD platform, based on sequencing by ligation.
- **2010:** Ion Torrent introduced semiconductor sequencing, detecting hydrogen ions released during nucleotide incorporation.
- **2011:** Pacific Biosciences released the RS system, the first commercial single-molecule real-time (SMRT) sequencer, capable of reads exceeding 10,000 bases.
- **2014:** Oxford Nanopore Technologies shipped the MinION, a USB-stick-sized sequencer that reads DNA by passing it through a protein nanopore.

1.2.4 The \$1,000 Genome and Beyond (2014–Present)

In January 2014, Illumina announced that its HiSeq X Ten system could sequence a human genome for approximately \$1,000 in reagent costs—a symbolic milestone that had been a stated goal of the genomics community for over a decade. By 2022, the Illumina NovaSeq X series brought this cost below \$200 per genome, and as of 2025–2026, some estimates place the cost of sequencing a human genome at roughly \$100–\$200.

This dramatic cost reduction—faster than Moore’s Law for semiconductor performance (Fig. 1.1)—has fundamentally democratised access to sequencing. What once required a billion-dollar international consortium can now be performed by a single graduate student with a desktop sequencer.

Practical Tip

The cost of *sequencing* a genome is only part of the total cost of a genomics project. Sample collection, DNA/RNA extraction, library preparation, computational infrastructure, data storage, and expert analysis can collectively exceed the sequencing cost itself. When budgeting for a project, consider the complete workflow.

1.3 The Different “Omes”

Sequencing technologies do not just read DNA. Modern methods can interrogate many different layers of biological information. Each layer is described by its own “-ome” and the corresponding “-omics” field:

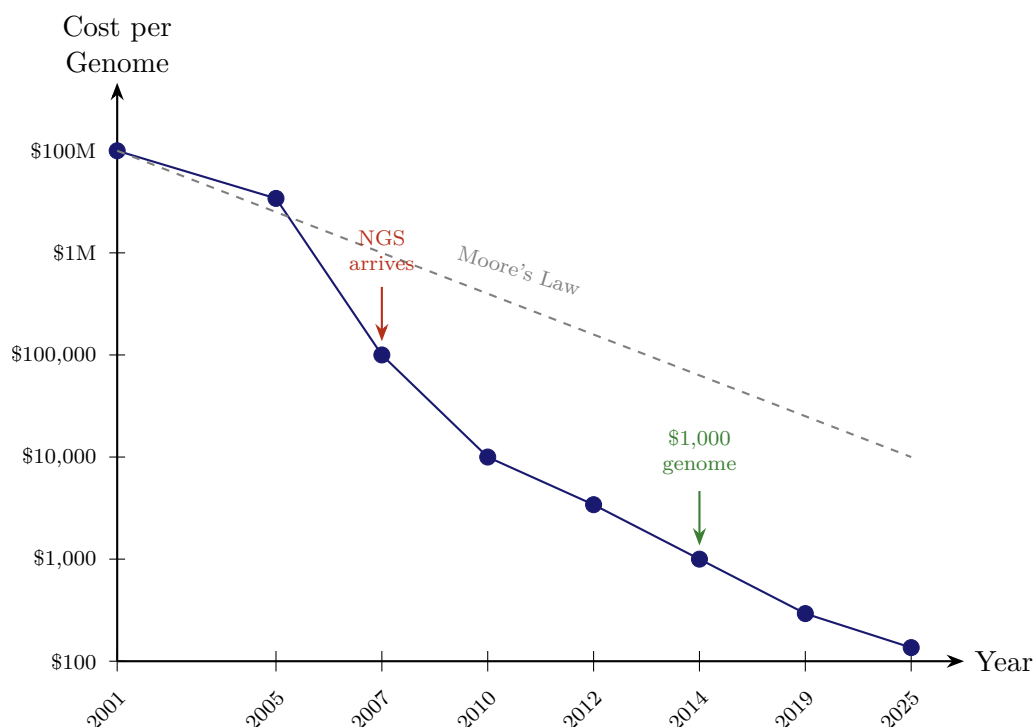


Figure 1.1: The cost of sequencing a human genome has fallen faster than Moore’s Law, particularly after the introduction of NGS technologies around 2005–2007. Costs are approximate and reflect reagent costs.

1.3.1 The Genome

The **genome** is the complete set of DNA in an organism. It includes all the genes (regions that encode proteins or functional RNAs) as well as vast stretches of non-coding DNA that play regulatory, structural, or still-unknown roles. **Genomics** is the study of genomes—their sequence, structure, function, and evolution. The human genome is approximately 3.2 billion base pairs long and is packaged into 23 pairs of chromosomes.

1.3.2 The Transcriptome

The **transcriptome** is the complete set of RNA molecules in a cell or tissue at a given moment. While every cell in your body (with few exceptions) contains the same genome, different cell types express different subsets of genes. A liver cell transcribes a very different set of genes than a neuron. **Transcriptomics** captures this snapshot of gene expression, revealing which genes are “on” and at what level.

1.3.3 The Epigenome

The **epigenome** comprises the chemical modifications to DNA and its associated proteins (histones) that influence gene expression without changing the underlying DNA sequence. These modifications include **DNA methylation** (the addition of methyl groups to cytosine bases) and **histone modifications** (chemical tags on the protein spools around which DNA is wound). **Epigenomics** studies these modifications across the entire genome.

1.3.4 The Proteome

The **proteome** is the complete set of proteins expressed by a cell, tissue, or organism. While the genome provides the blueprint and the transcriptome reflects which blueprints are being read, the proteome represents the actual molecular machines carrying out cellular functions. **Proteomics** traditionally uses mass spectrometry rather than sequencing, though sequencing-based methods (such as Ribo-seq for ribosome profiling) can provide indirect measurements of protein production.

1.3.5 The Metabolome

The **metabolome** is the complete set of small molecules (metabolites) present in a biological sample. These include sugars, lipids, amino acids, nucleotides, and the thousands of other small molecules involved in cellular metabolism. **Metabolomics** typically uses mass spectrometry or nuclear magnetic resonance (NMR), though it is sometimes integrated with sequencing-based multi-omics studies.

1.3.6 The Microbiome

The **microbiome** refers to the community of microorganisms—bacteria, archaea, fungi, viruses—that inhabit a particular environment, such as the human gut, soil, or ocean water. **Metagenomics** uses sequencing to identify and characterise these microbial communities, often without the need to culture individual species in the laboratory. This has revealed that the human body harbours trillions of microbial cells that play critical roles in health and disease.

Multi-Omics

No single “ome” tells the complete story of a biological system. Modern research increasingly integrates data from multiple omics layers—genomics, transcriptomics, epigenomics, proteomics, and metabolomics—to build comprehensive models of cellular function and disease. Some cutting-edge single-cell technologies can even measure multiple omics layers simultaneously from the same individual cell.

1.4 How Sequencing Has Transformed Science

1.4.1 Medicine and Clinical Diagnostics

Sequencing has fundamentally altered the practice of medicine in several ways:

- **Cancer genomics:** Tumour sequencing identifies the specific mutations driving a patient’s cancer, enabling **precision oncology**—the selection of targeted therapies matched to a tumour’s molecular profile. Large-scale projects such as The Cancer Genome Atlas (TCGA) have catalogued mutations across dozens of cancer types.
- **Rare disease diagnosis:** For patients with undiagnosed genetic conditions, **whole-genome sequencing (WGS)** or **whole-exome sequencing (WES)** can identify the causal mutation. Studies have shown that genomic sequencing can provide a diagnosis for 25–40% of previously undiagnosed rare disease patients.
- **Prenatal screening:** Non-invasive prenatal testing (NIPT) sequences cell-free fetal DNA circulating in the mother’s blood to screen for chromosomal abnormalities such as Down syndrome (trisomy 21), with high sensitivity and specificity.

- **Pharmacogenomics:** Sequencing a patient’s genome can reveal genetic variants that affect drug metabolism, helping clinicians choose the right drug and dosage.
- **Infectious disease:** Rapid sequencing of pathogen genomes enables real-time outbreak tracking, identification of antimicrobial resistance genes, and development of targeted diagnostics and vaccines. The COVID-19 pandemic demonstrated this on a global scale, with millions of SARS-CoV-2 genomes sequenced worldwide.

1.4.2 Agriculture and Food Science

Genomic sequencing has accelerated crop and livestock improvement. By sequencing the genomes of important crop species (rice, wheat, maize, soybean), researchers have identified genes associated with yield, drought tolerance, disease resistance, and nutritional quality. Genomic selection—using genome-wide genetic markers to predict an organism’s traits—has dramatically shortened breeding cycles in both plants and animals.

1.4.3 Ecology and Conservation Biology

Environmental DNA (eDNA) sequencing allows ecologists to survey biodiversity by collecting water or soil samples and sequencing the trace DNA shed by organisms. This non-invasive approach can detect rare or elusive species, monitor ecosystem health, and track the spread of invasive species. Population genomics studies use sequencing to assess genetic diversity within endangered species, informing conservation strategies.

1.4.4 Forensic Science

DNA sequencing has become a cornerstone of forensic identification. Beyond traditional short tandem repeat (STR) profiling, NGS-based forensic methods can extract more information from degraded or mixed samples, determine biogeographic ancestry, and even predict physical appearance traits from DNA.

1.4.5 Evolutionary Biology and Palaeogenomics

Sequencing has illuminated the tree of life, resolving evolutionary relationships from viruses to vertebrates. Perhaps most spectacularly, advances in ancient DNA (aDNA) extraction and sequencing have enabled researchers to sequence genomes from organisms that died tens of thousands of years ago—including Neanderthals, Denisovans, woolly mammoths, and cave bears. Svante Pääbo received the 2022 Nobel Prize in Physiology or Medicine for his pioneering work in palaeogenomics.

1.5 What Modern Sequencing Can Measure

The versatility of modern sequencing is remarkable. By varying the sample preparation (called **library preparation**—see [Chapter 3](#)), the same sequencing instrument can be used to measure fundamentally different biological properties:

As shown in [Table 1.1](#), the key insight is that *the sequencing instrument itself is largely agnostic to what is being measured*. The specificity comes from how the sample is prepared before sequencing. This concept—that library preparation defines the experiment—is one of the most important ideas in modern genomics and will be explored in detail throughout this book.

Table 1.1: Major biological layers that can be interrogated using sequencing-based approaches.

What Is Measured	Description	Key Methods
DNA sequence	The order of bases in the genome	WGS, WES
RNA expression	Which genes are active and at what level	RNA-seq, scRNA-seq
DNA methylation	Methyl groups on cytosine residues	WGBS, RRBS, nanopore
Chromatin accessibility	Which genomic regions are “open” and potentially active	ATAC-seq, DNase-seq
Protein–DNA interactions	Where proteins bind to the genome	ChIP-seq, CUT&RUN
Spatial gene expression	Gene expression mapped to tissue location	10x Visium, MERFISH, Slide-seq
3D genome architecture	How the genome folds in three-dimensional space	Hi-C, Micro-C, GAM
Microbial communities	Species composition of complex microbial ecosystems	16S/ITS amplicon, shotgun metagenomics

Important Caveat

Sequencing measures **nucleic acid sequences**. It does not directly measure protein levels, metabolite concentrations, or protein structures. When we say “sequencing measures chromatin accessibility” or “protein–DNA interactions,” we mean that a clever biochemical protocol converts these properties into a DNA sequencing readout. Understanding these conversions is essential for interpreting sequencing data correctly.

1.6 The General Sequencing Workflow

Although each sequencing application has its own specific protocol, all sequencing experiments share a common high-level workflow. Understanding this workflow provides a conceptual framework for the rest of the book.

1.6.1 Step 1: Biological Sample

Every sequencing experiment begins with a biological sample. This could be blood drawn from a patient, a tissue biopsy, a bacterial culture, a water sample from a lake, soil, or even ancient bone. The nature and quality of the starting material profoundly influence every downstream step. A fresh, well-preserved sample will yield high-quality nucleic acids; a degraded or contaminated sample may produce unreliable results.

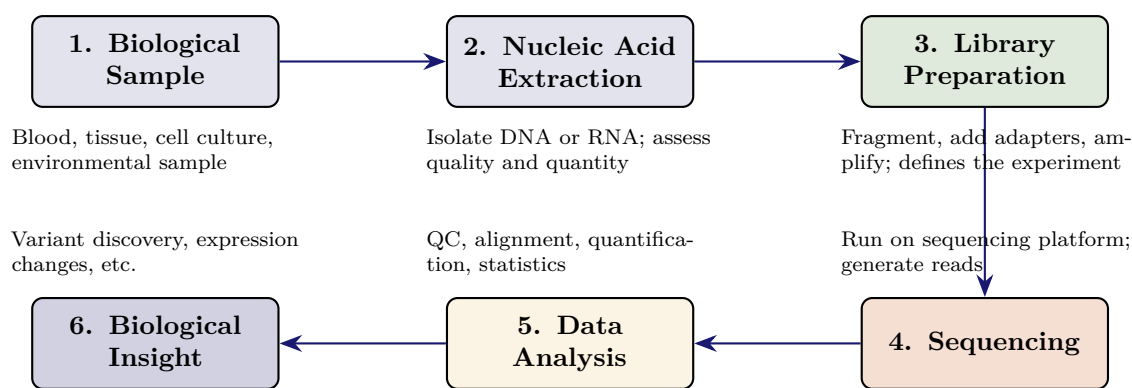


Figure 1.2: The general sequencing workflow, from biological sample to biological insight. Every sequencing experiment follows some version of this pipeline.

1.6.2 Step 2: Nucleic Acid Extraction

The DNA or RNA must be isolated from the sample. Extraction protocols use chemical and physical methods to break open cells (lysis), remove proteins and other contaminants, and purify the nucleic acids. The quality and quantity of extracted material are assessed using instruments such as a spectrophotometer (e.g., NanoDrop), a fluorometer (e.g., Qubit), or electrophoresis (e.g., Bioanalyzer, TapeStation).

1.6.3 Step 3: Library Preparation

This is the step where the nucleic acid is converted into a form compatible with the sequencing instrument. The details vary enormously depending on the application, but the general steps include:

1. **Fragmentation:** Breaking the nucleic acid into pieces of an appropriate size (e.g., 200–500 bp for short-read sequencing).
2. **Adapter ligation:** Attaching short, synthetic DNA sequences (adapters) to the ends of fragments. These adapters allow fragments to bind to the sequencing instrument and serve as priming sites for sequencing.
3. **Amplification:** Using [polymerase chain reaction \(PCR\)](#) to create multiple copies of each library fragment (though some protocols are “PCR-free”).
4. **Selection or enrichment:** Optionally selecting fragments of a specific size or enriching for specific genomic regions (e.g., exome capture).

The resulting collection of adapter-ligated fragments is called a **sequencing library**. This concept is explored in depth in [Chapter 3](#).

1.6.4 Step 4: Sequencing

The library is loaded onto a sequencing instrument, which reads the sequence of each fragment. Different platforms use different chemistries (discussed in [Chapter 3](#) and subsequent chapters), but all produce the same fundamental output: millions to billions of short text strings representing the sequences of the library fragments, along with quality scores indicating the confidence of each base call.

1.6.5 Step 5: Data Analysis

The raw sequencing data (typically in **FASTQ** format) must be processed computationally. A typical analysis pipeline involves:

1. **Quality control:** Assessing data quality and removing low-quality reads or adapter sequences.
2. **Alignment / Mapping:** Aligning reads to a reference genome to determine where each read originated.
3. **Quantification or variant calling:** Counting reads per gene (for expression analysis) or identifying positions where the sample differs from the reference (for variant calling).
4. **Statistical analysis:** Applying statistical methods to identify significant biological signals (e.g., differentially expressed genes, significantly mutated regions).
5. **Visualisation and interpretation:** Presenting results in an interpretable format and placing them in biological context.

1.6.6 Step 6: Biological Insight

The ultimate goal of any sequencing experiment is biological understanding. This might be identifying the mutation causing a patient’s disease, discovering a new gene regulatory mechanism, characterising the microbial community in a specific environment, or any of thousands of other scientific questions that sequencing can address.

Practical Tip

A common mistake for beginners is to focus exclusively on the sequencing step itself, underestimating the importance of sample quality, library preparation, and computational analysis. In practice, the “wetlab” steps (sample prep and library construction) and the “drylab” steps (bioinformatics and statistics) are at least as important as the sequencing itself. Garbage in, garbage out.

1.7 The Cost Trajectory and Democratisation of Sequencing

The cost of sequencing has decreased at a pace that outstrips even the famous Moore’s Law for computer transistors. As illustrated in Fig. 1.1, the cost of sequencing a human genome fell from approximately \$100 million in 2001 to under \$200 by 2025—a decline of roughly six orders of magnitude in 24 years.

This cost reduction has had profound consequences:

- **Democratisation:** Sequencing is no longer restricted to a handful of large genome centres. University core facilities, clinical laboratories, and even field stations now operate sequencing instruments. The Oxford Nanopore MinION, which costs roughly \$1,000 and is the size of a stapler, has been used in Antarctic research stations, the International Space Station, and disease outbreak zones in remote regions of Africa.
- **Scale:** Projects like the UK Biobank (500,000 genomes), the All of Us Research Program (1 million+ participants), and large-scale cancer genomics initiatives would have been unthinkable at earlier cost points.
- **Clinical adoption:** As costs have fallen, sequencing has moved from a research tool to a clinical diagnostic. Many hospitals now offer clinical **WGS** or **WES** for rare disease diagnosis, and tumour sequencing panels are standard of care in oncology.

- **New applications:** Low costs have enabled entirely new categories of experiments, such as single-cell sequencing (profiling thousands to millions of individual cells), spatial transcriptomics (mapping gene expression to tissue locations), and real-time pathogen surveillance.

Important Caveat

While sequencing costs have plummeted, the costs of data storage, computational analysis, and expert interpretation have not decreased at the same rate. For large-scale projects generating petabytes of data, computational costs can exceed sequencing costs. This growing imbalance is sometimes called the “\$1,000 genome, \$1,000,000 analysis” problem.

1.8 The Current Sequencing Landscape

As of the mid-2020s, the sequencing landscape is dominated by a few major technology families:

Short-read sequencing (Illumina)

The most widely used platform, producing reads of 50–300 bp with very high accuracy (>99.9% per base). Dominant for applications requiring high throughput: WGS, RNA-seq, ChIP-seq, ATAC-seq, and most clinical diagnostics.

Long-read sequencing (PacBio SMRT)

Produces reads averaging 15–25 kb (and up to >100 kb), with high consensus accuracy achieved through circular consensus sequencing (HiFi reads). Excellent for genome assembly, structural variant detection, and full-length transcript sequencing.

Long-read sequencing (Oxford Nanopore)

Produces ultra-long reads (routinely >100 kb, with records exceeding 4 Mb). Can directly detect base modifications (e.g., methylation) without chemical conversion. Uniquely portable (MinION) and capable of real-time data streaming.

Emerging platforms

New entrants, including Element Biosciences (AVITI), Ultima Genomics, and others, are introducing competition in the short-read space with novel chemistries and pricing models.

These platforms are covered in detail in Chapter 3 and the chapters that follow.

1.9 An Information-Theoretic View of Sequencing

Sequencing can be framed as an information-recovery problem. A genome of length G bases, drawn from an alphabet of size $|\Sigma| = 4$, carries at most $2G$ bits of information (since $\log_2 4 = 2$). In practice, the information content is lower because of compositional biases. The **Shannon entropy** of a sequence quantifies this:

$$H = - \sum_{i \in \{A, C, G, T\}} p_i \log_2 p_i \quad (1.1)$$

where p_i is the frequency of base i . For a genome with perfectly uniform composition ($p_A = p_C = p_G = p_T = 0.25$), $H = 2$ bits per position. For an AT-rich genome such as *Plasmodium falciparum* (~80% AT), $H \approx 1.72$ bits per position, meaning the sequence is intrinsically more compressible and, in a sense, carries less information per base.

Shannon Entropy and Genome Complexity

Shannon entropy provides a rigorous lower bound on the number of bits needed to encode each position in a genome. This quantity has direct practical consequences: genomes with low entropy (e.g., highly repetitive or AT-biased) are easier to compress (relevant for storage), but harder to assemble (because repeats create ambiguity in fragment overlap). Entropy also constrains the theoretical minimum number of reads required to reconstruct a genome, a result formalised by information-theoretic limits on sequence assembly.

This perspective connects to a practical trade-off: every sequencing experiment must generate enough information (reads $\times$ read length $\times$ accuracy) to overcome the entropy of the target plus the noise introduced by the instrument’s error rate. For a base call with error probability P_e , the mutual information between the true base and the observed call is:

$$I = \log_2 |\Sigma| + P_e \log_2 \frac{P_e}{|\Sigma| - 1} + (1 - P_e) \log_2 (1 - P_e) \quad (1.2)$$

At Q30 accuracy ($P_e = 0.001$), each observed base carries ~ 1.99 bits of information—nearly all of the 2-bit theoretical maximum. At Q10 ($P_e = 0.1$), this drops to ~ 1.53 bits, meaning roughly 30% more sequencing is needed to achieve the same total information content.

1.9.1 Quantitative Comparison of Sequencing Generations

The evolution of sequencing technology can be summarised quantitatively across several dimensions. Table 1.2 provides a comprehensive comparison.

Table 1.2: Quantitative comparison of sequencing technology generations. Costs are approximate and reflect the mid-2020s.

Metric	Sanger	2nd Gen (Illumina)	3rd Gen (PacBio HiFi)	Gen (Q30)	3rd Gen (ONT)	Gen
Read length	500–1,000 bp	50–300 bp	10–25 kb		1 kb–4 Mb	
Per-read accuracy	99.99%	99.9% (Q30)	99.9% (Q30 HiFi)		92–99% (raw)	
Throughput / run	~ 0.1 Mb	0.6–16,000 Gb	15–90 Gb		2–300 Gb	
Reads / run	96	10^6 – 10^{10}	10^5 – 10^7		10^4 – 10^7	
Cost / Gb	\$500,000	\$2–20	\$20–50		\$5–30	
Instrument cost	\$90K	\$20K–1.0M	\$200K–600K		\$1K–48K	
Time to result	2–3 h	12–48 h	12–30 h		1 h–72 h	
Dominant error	Substitution	Substitution	Indel (residual)		Indel (homopolymer)	
Base modifications	No	No (indirect only)	Yes (kinetics)		Yes (direct signal)	

The key insight from this comparison is that *no single platform dominates across all dimensions*. Short-read platforms excel in throughput and cost per base, making them ideal for large-scale surveys (population genomics, RNA-seq). Long-read platforms sacrifice some throughput for the ability to resolve complex genomic features (structural variants, repeat-rich regions, full-length transcripts, base modifications). This complementarity is why many modern studies employ a hybrid approach, combining short and long reads to leverage the strengths of both.

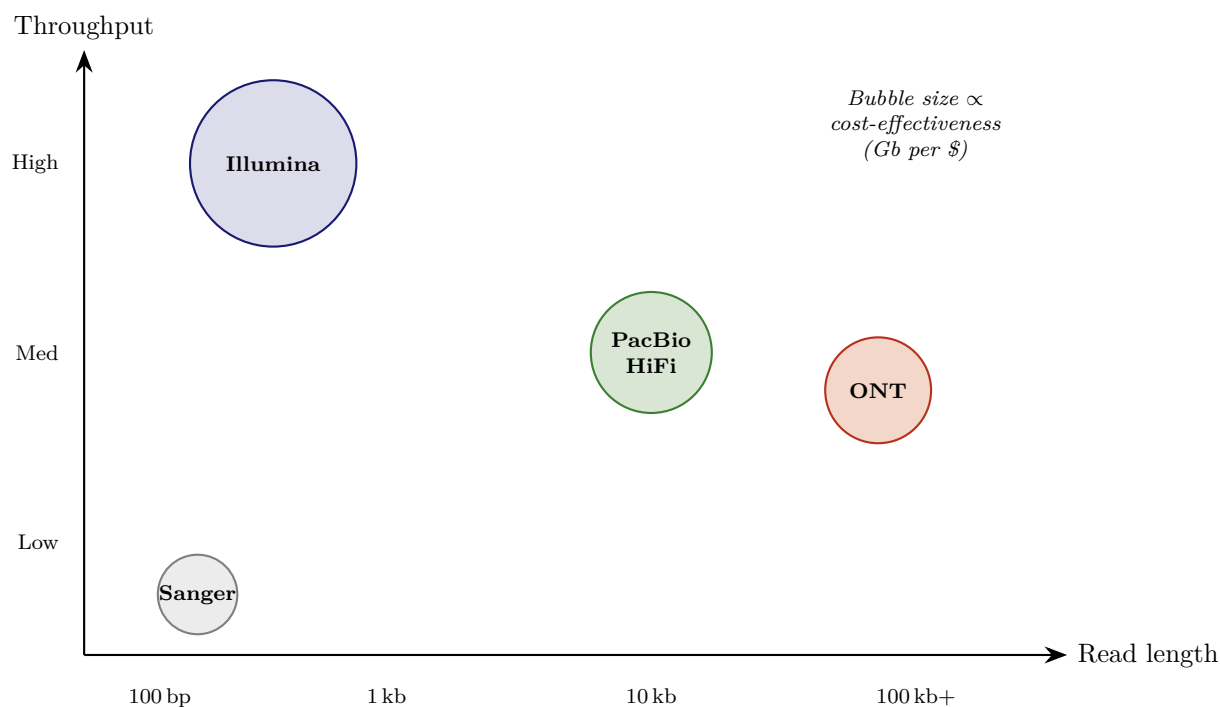


Figure 1.3: Conceptual comparison of major sequencing platforms. The x -axis represents read length, the y -axis represents throughput per run, and bubble size approximates cost-effectiveness. Each technology occupies a distinct niche; the choice depends on the biological question.

1.10 Reading Guide for This Book

This book is organised into seven parts, designed to take you from foundational concepts to advanced applications and analysis:

Part I: Foundations (Chapters 1–3)

You are here. [Chapter 2](#) covers the essential molecular biology you need to understand sequencing technologies—DNA structure, gene expression, epigenetics, and genetic variation. [Chapter 3](#) introduces the core principles shared by all modern sequencing platforms: how reads are generated, what quality scores mean, and key concepts like coverage and multiplexing.

Part II: Sequencing Platforms and Technologies (Chapters 4–6)

A detailed survey of current sequencing instruments. [Chapter 4](#) covers short-read platforms (Illumina and emerging competitors). [Chapter 5](#) covers long-read platforms (PacBio and Oxford Nanopore). [Chapter 6](#) provides a comprehensive guide to library preparation methods.

Part III: Genomics (Chapters 7–9)

DNA-level applications: whole-genome sequencing, exome sequencing, variant discovery and annotation, and metagenomics for microbial community analysis.

Part IV: Transcriptomics (Chapters 10–12)

RNA-level applications: bulk [RNA-seq](#), single-cell [RNA-seq](#) (scRNA-seq), and spatial transcriptomics. These chapters cover both experimental design and computational analysis.

Part V: Epigenomics (Chapters 13–16)

Epigenomic applications: DNA methylation profiling (WGBS and related methods), chromatin accessibility (ATAC-seq, DNase-seq), histone modification mapping (ChIP-seq, CUT&RUN/CUT&Tag), and three-dimensional genome architecture (Hi-C).

Part VI: Multiomics and Emerging Frontiers (Chapters 17–19)

Cutting-edge technologies that measure multiple molecular layers simultaneously, specialised applications (long-read transcriptomics, direct RNA sequencing, spatial multi-omics), and emerging technologies poised to reshape the field.

Part VII: Bioinformatics and Data Analysis (Chapters 20–23)

Computational fundamentals: file formats, alignment algorithms, workflow management (Nextflow, Snakemake), experimental design and statistical considerations, data standards, and reproducibility.

Part VIII: Democratisation of Sequencing (Chapters 24–26)

Hands-on genomics for everyone. Chapter 24 surveys accessible sequencing platforms and cost models for individual researchers and citizen scientists. Chapter 25 provides step-by-step wet-lab protocols—from DNA extraction through library preparation to flow cell loading. Chapter 26 walks you through complete computational analyses, from basecalling to genome assembly and species identification, with full Snakemake and Nextflow workflows.

How to Use This Book

If you are completely new to genomics, we recommend reading Part I (Chapters 1–3) in full before branching into the topics most relevant to your work. If you already have a molecular biology background, you can skim Chapter 2 and start with Chapter 3. Each application chapter (Parts III–VI) is designed to be relatively self-contained, so you can jump directly to the application most relevant to your research.

1.11 Summary

This chapter has provided a bird’s-eye view of the sequencing revolution:

- Sequencing is the process of reading the nucleotide sequence of DNA or RNA, and it has become one of the most transformative technologies in modern science.
- From Sanger sequencing in 1977 through the Human Genome Project and the NGS revolution, the cost and speed of sequencing have improved by many orders of magnitude.
- Modern sequencing can interrogate many biological layers beyond DNA sequence, including gene expression, epigenetic modifications, chromatin structure, protein–DNA interactions, and spatial gene expression patterns.
- The general workflow—sample collection, nucleic acid extraction, library preparation, sequencing, data analysis, and biological interpretation—provides a framework for understanding all sequencing experiments.
- The dramatic decrease in sequencing costs has democratised the technology, enabling applications from clinical diagnostics to ecological surveys to palaeogenomics.

In the next chapter, we will build the molecular biology foundation you need to understand how these technologies work at the chemical and biological level.

Chapter 2

Molecular Biology Essentials

Key Concept

Before we can understand how sequencing technologies work, we need to understand what they are reading. This chapter builds the molecular biology foundation—from the structure of cells and DNA to gene expression, epigenetics, and genetic variation—that underpins every sequencing experiment described in this book. If you already have a strong background in molecular biology, you may wish to skim this chapter and use it as a reference.

2.1 The Cell: Life's Fundamental Unit

All living organisms are composed of cells. A cell is the smallest unit of life capable of independent function: it can take in nutrients, produce energy, grow, and reproduce. There are two fundamentally different types of cells, distinguished by their internal organisation.

2.1.1 Prokaryotic Cells

Prokaryotes—bacteria and archaea—are the simplest and most ancient cells. A prokaryotic cell lacks a membrane-bound nucleus; instead, its DNA floats freely in the cytoplasm in a region called the **nucleoid**. Key features include:

- **No nucleus:** DNA is not enclosed in a membrane-bound compartment.
- **Circular chromosome:** Most prokaryotic genomes consist of a single, circular DNA molecule (though some prokaryotes have linear chromosomes or multiple chromosomes).
- **Small genomes:** Typically 0.5–10 million base pairs (Mb), encoding 500–10,000 genes.
- **Plasmids:** Many prokaryotes carry small, circular, extra-chromosomal DNA molecules called plasmids, which often carry genes for antibiotic resistance or other survival advantages.
- **No membrane-bound organelles:** Prokaryotes lack the internal compartmentalisation seen in eukaryotic cells.
- **Operons:** Prokaryotic genes are often organised into **operons**—clusters of genes transcribed together as a single unit, under the control of a shared promoter.

2.1.2 Eukaryotic Cells

Eukaryotes—animals, plants, fungi, and protists—have larger, more complex cells with a **membrane-bound nucleus** that houses the DNA. Key features include:

- **Nucleus:** The DNA is enclosed within a double membrane called the **nuclear envelope**, which separates the genetic material from the cytoplasm.
- **Linear chromosomes:** Eukaryotic genomes are organised into multiple linear chromosomes. Humans have 23 pairs (46 total).
- **Large genomes:** Ranging from about 10 Mb (some fungi) to over 100 Gb (some plants and amphibians). The human genome is approximately 3.2 Gb.
- **Membrane-bound organelles:** Including **mitochondria** (energy production, with their own small circular genome), **endoplasmic reticulum** (protein and lipid synthesis), **Golgi apparatus** (protein sorting and modification), and in plants, **chloroplasts** (photosynthesis, also with their own genome).
- **Complex gene structure:** Eukaryotic genes are typically split into coding segments (**exons**) interrupted by non-coding segments (**introns**), and each gene usually has its own promoter.

Practical Tip

When sequencing eukaryotic organisms, remember that the cell contains DNA in multiple locations: the nuclear genome (the main focus of most sequencing experiments), the mitochondrial genome, and in plants, the chloroplast genome. Reads from these organellar genomes will appear in your data and must be accounted for during analysis.

2.2 DNA Structure: The Molecule of Heredity

Deoxyribonucleic acid (**DNA**) is the molecule that stores genetic information in all cellular organisms (and many viruses). Understanding its structure is essential for understanding how sequencing works.

2.2.1 Nucleotides: The Building Blocks

DNA is a **polymer**—a long chain made up of repeating subunits called **nucleotides**. Each nucleotide consists of three components:

1. **A sugar:** Deoxyribose, a five-carbon sugar. The carbon atoms are numbered 1' through 5' (pronounced “one prime” through “five prime”).
2. **A phosphate group:** Attached to the 5' carbon of the sugar.
3. **A nitrogenous base:** One of four molecules that project inward from the sugar-phosphate backbone:
 - **Adenine (A)** — a purine (two-ring structure)
 - **Guanine (G)** — a purine (two-ring structure)
 - **Cytosine (C)** — a pyrimidine (single-ring structure)
 - **Thymine (T)** — a pyrimidine (single-ring structure)

Nucleotides are linked together by **phosphodiester bonds** between the 3' carbon of one nucleotide and the 5' carbon of the next, forming a continuous **sugar-phosphate backbone**. The sequence of bases along this backbone encodes genetic information—much like the sequence of letters in a sentence encodes meaning.

2.2.2 The Double Helix

In 1953, James Watson and Francis Crick—building on X-ray crystallography data from Rosalind Franklin and Maurice Wilkins—proposed that DNA has a **double-helical** structure: two strands of nucleotides wound around each other in a right-handed helix.

The two strands are held together by **hydrogen bonds** between complementary bases:

- **A pairs with T** (via two hydrogen bonds)
- **G pairs with C** (via three hydrogen bonds)

This **base-pairing rule** (also called **Chargaff’s rule**) is fundamental to both the biology of DNA and the chemistry of sequencing. Because G-C base pairs have three hydrogen bonds (compared to two for A-T), regions of DNA with high G-C content are more thermally stable and require more energy to separate (“melt”)—a property with important consequences for sequencing, as discussed in [Chapter 3](#).

2.2.3 Nucleotide Chemistry in Detail

The behaviour of nucleotides during sequencing depends critically on their chemical properties. Each nucleotide is a weak polyprotic acid whose ionisation state varies with pH:

Table 2.1: pKa values for the major ionisable groups of the four DNA bases. The dominant tautomeric form at physiological pH (7.4) is the **keto/amino** form, but rare tautomers can cause misincorporation during replication and sequencing.

Base	Ionisable group	pKa	Rare tautomer
Adenine	N-1 (protonation)	3.5	Imino form
Guanine	N-7 (protonation)	3.3	Enol form
	N-1 (deprotonation)	9.2	
Cytosine	N-3 (protonation)	4.5	Imino form
Thymine	N-3 (deprotonation)	9.9	Enol form

Tautomeric forms of nucleobases—rare structural isomers in which a proton shifts between nitrogen and oxygen atoms—are of particular significance. When a base transiently adopts its rare tautomer, it can form non-canonical base pairs (e.g., enol-T pairs with G instead of A, imino-C pairs with A instead of G). This is one of the fundamental mechanisms of spontaneous mutation during DNA replication, with tautomeric mispairing occurring at rates of approximately 10^{-4} to 10^{-5} per base pair.

Tautomers and Sequencing Errors

The same tautomeric equilibria that cause natural replication errors can also affect sequencing accuracy. In sequencing-by-synthesis, a rare tautomer of the template base may cause the polymerase to incorporate the wrong fluorescent nucleotide. This contributes to the small but non-zero substitution error rate observed even in high-fidelity platforms such as Illumina.

2.2.4 DNA Thermodynamics and the Nearest-Neighbour Model

The thermal stability of DNA—its resistance to strand separation (“melting”)—is central to many molecular biology and sequencing techniques, including **PCR**, hybridisation-based capture, and adapter ligation. The **melting temperature** (T_m) is defined as the temperature at which 50% of DNA molecules in solution are in the double-stranded state and 50% are single-stranded.

The most accurate method for predicting T_m is the **nearest-neighbour (NN) model**, which accounts for the fact that the stability of a base pair depends not only on the pair itself but also on its neighbours (stacking interactions):

$$T_m = \frac{\Delta H^\circ}{\Delta S^\circ + R \ln(C_T/4)} - 273.15 \quad (2.1)$$

where ΔH° and ΔS° are the total enthalpy and entropy of duplex formation (summed over all nearest-neighbour pairs), $R = 1.987 \text{ cal}/(\text{mol}\cdot\text{K})$ is the gas constant, and C_T is the total strand concentration. The thermodynamic parameters are calculated as:

$$\Delta H^\circ = \sum_{i=1}^{n-1} \Delta H_i^\circ + \Delta H_{\text{init}}^\circ \quad (2.2)$$

$$\Delta S^\circ = \sum_{i=1}^{n-1} \Delta S_i^\circ + \Delta S_{\text{init}}^\circ + \Delta S_{\text{sym}}^\circ \quad (2.3)$$

where ΔH_i° and ΔS_i° are the enthalpy and entropy contributions of the i th nearest-neighbour pair, $\Delta H_{\text{init}}^\circ$ and $\Delta S_{\text{init}}^\circ$ are initiation parameters (dependent on the terminal base pairs), and $\Delta S_{\text{sym}}^\circ = -1.4 \text{ cal}/(\text{mol}\cdot\text{K})$ is a symmetry correction applied to self-complementary sequences.

Nearest-Neighbour Parameters

The 10 unique nearest-neighbour pairs (e.g., AA/TT, AT/TA, TA/AT, CA/GT, GT/CA, CT/GA, GA/CT, CG/GC, GC/CG, GG/CC) each have experimentally determined ΔH° and ΔS° values. For example, the GC/CG pair ($\Delta H^\circ = -9.8 \text{ kcal/mol}$) is substantially more stable than the AA/TT pair ($\Delta H^\circ = -7.9 \text{ kcal/mol}$). These parameters are used by primer design tools such as **Primer3** and **OligoAnalyzer** to predict hybridisation behaviour.

For practical estimation of short oligonucleotides (primers, adapters) in standard PCR buffer, a salt-adjusted approximation is often used:

$$T_m \approx T_m^{\text{1M NaCl}} + 16.6 \cdot \log_{10}[\text{Na}^+] \quad (2.4)$$

Practical Tip

When designing sequencing adapters or **PCR** primers, ensure that the T_m of all oligos is within $1\text{--}2^\circ\text{C}$ of each other to promote uniform hybridisation and extension. Online tools such as **Primer3**, **IDT OligoAnalyzer**, and **NEB Tm Calculator** implement the full nearest-neighbour model with salt corrections and are freely available.

2.2.5 Directionality and Antiparallel Strands

Each strand of DNA has a **directionality**, defined by the orientation of the sugar-phosphate backbone. One end has a free $5'$ phosphate group (the **$5'$ end**), and the other end has a free $3'$ hydroxyl group (the **$3'$ end**). By convention, DNA sequences are always written in the **$5'$ to $3'$** direction.

Crucially, the two strands of the double helix run in **opposite directions**—they are **antiparallel**. If one strand runs $5' \rightarrow 3'$ from left to right, the complementary strand runs $3' \rightarrow 5'$ from left to right (or equivalently, $5' \rightarrow 3'$ from right to left).

Complementarity and Antiparallel Strands

The two strands of DNA are **complementary** (A pairs with T, G pairs with C) and **antiparallel** (they run in opposite directions). If one strand reads 5'-ATCGGA-3', the complementary strand reads 3'-TAGCCT-5' (or equivalently, 5'-TCCGAT-3' when written in the conventional 5' to 3' direction). This complementarity is the basis for DNA replication, transcription, and many sequencing chemistries.

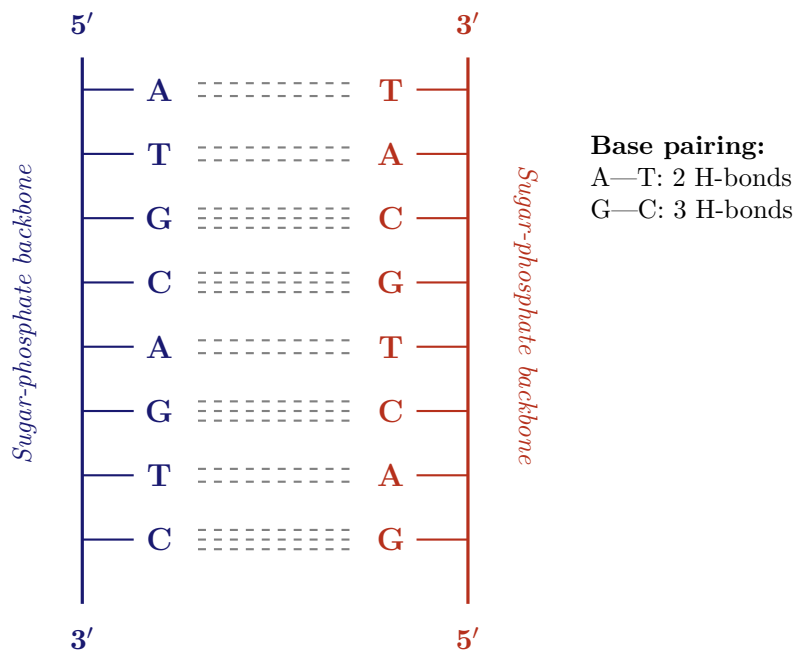


Figure 2.1: Simplified representation of DNA double-stranded structure showing the antiparallel sugar-phosphate backbones (blue and red) connected by complementary base pairs via hydrogen bonds (dashed lines). Note that G-C pairs have three hydrogen bonds, making them stronger than A-T pairs (two hydrogen bonds).

2.3 DNA Organisation: From Double Helix to Chromosome

The human genome contains approximately 3.2 billion base pairs of DNA. If stretched out, the DNA from a single human cell would be about 2 metres long. Yet it must fit inside a nucleus roughly 6 micrometres in diameter—a packing ratio of over 300,000 to 1. This remarkable feat of organisation is achieved through a hierarchy of packaging structures.

2.3.1 Histones and Nucleosomes

The first level of packaging involves proteins called **histones**. DNA wraps around complexes of eight histone proteins (an **octamer** composed of two copies each of histones H2A, H2B, H3, and H4) to form structures called **nucleosomes**. Each nucleosome contains approximately 147 base pairs of DNA wrapped in 1.65 turns around the histone octamer, connected by short stretches of **linker DNA** (typically 20–60 bp).

A fifth histone, **H1** (the linker histone), binds to the DNA as it enters and exits the nucleosome, helping to stabilise higher-order structures.

2.3.2 Chromatin: Euchromatin and Heterochromatin

The chain of nucleosomes is called **chromatin**. Chromatin exists in two broad states:

- **Euchromatin:** Loosely packed, “open” chromatin. Genes in euchromatic regions are generally accessible to the transcription machinery and may be actively expressed. Euchromatin appears light in electron microscopy.
- **Heterochromatin:** Tightly packed, “closed” chromatin. Genes in heterochromatic regions are generally silenced. Heterochromatin appears dark in electron microscopy. It can be further divided into:
 - **Constitutive heterochromatin:** Permanently condensed regions, often found at centromeres and telomeres, typically containing repetitive sequences.
 - **Facultative heterochromatin:** Regions that can switch between open and closed states depending on cell type or developmental stage. The inactive X chromosome in female mammals is a classic example.

2.3.3 Higher-Order Structures and Chromosomes

Nucleosomes are further organised into higher-order structures. During cell division (mitosis or meiosis), chromatin condenses maximally into the dense, compact structures visible under a light microscope as **chromosomes**. Each chromosome consists of a single, continuous DNA molecule complexed with histones and other proteins.

Key chromosome structures include:

- **Centromere:** The constricted region where the two sister chromatids are joined and where the spindle fibres attach during cell division.
- **Telomeres:** Repetitive DNA sequences (TTAGGG in humans) that cap the ends of chromosomes, protecting them from degradation and preventing them from fusing with neighbouring chromosomes.

2.3.4 Ploidy

Ploidy refers to the number of complete sets of chromosomes in a cell. Most human somatic (body) cells are **diploid** ($2n$)—they contain two copies of each chromosome, one inherited from each parent. Human diploid cells have 46 chromosomes (23 pairs). Gametes (sperm and egg cells) are **haploid** (n), containing only 23 chromosomes. Some organisms, particularly plants, can be **polyploid** (e.g., wheat is hexaploid, with six copies of each chromosome), which adds complexity to genome assembly.

2.4 Genes and the Genome

2.4.1 What Is a Gene?

A **gene** is a segment of DNA that contains the instructions for producing a functional product—usually a protein, but in some cases a functional RNA molecule. The human genome contains approximately 20,000–25,000 protein-coding genes, but these account for only about 1.5% of the total genome. The rest—once dismissively called “junk DNA”—includes regulatory sequences, non-coding RNAs, repetitive elements, and regions of unknown function.

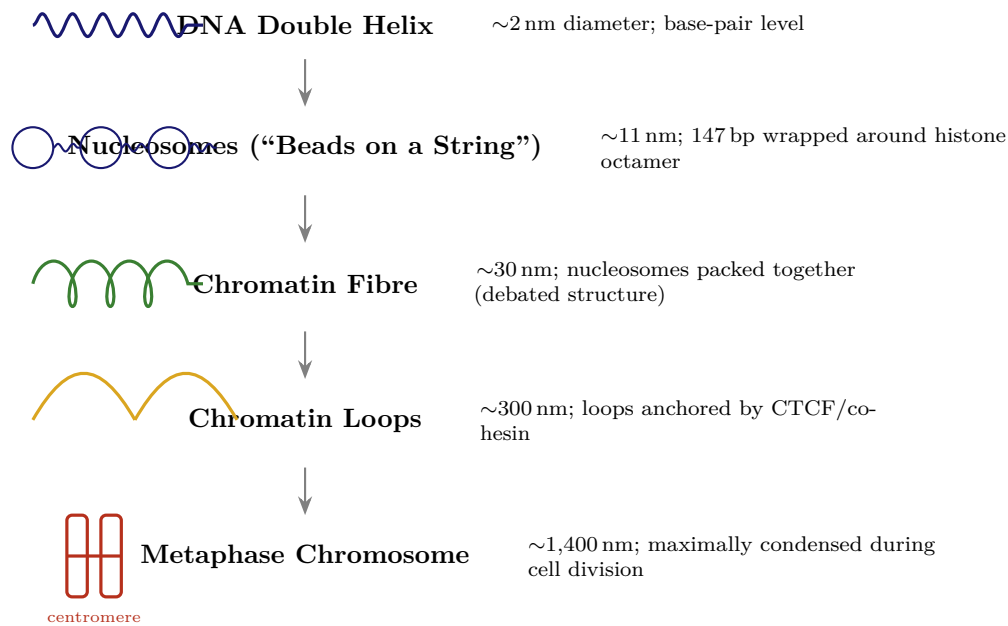


Figure 2.2: The hierarchy of DNA packaging from the naked double helix (~2 nm) through nucleosomes (~11 nm), chromatin fibres (~30 nm), looped domains (~300 nm), to the maximally condensed metaphase chromosome (~1,400 nm). Each level of organisation has implications for gene regulation and is measurable by specific sequencing assays (e.g., [ATAC-seq](#) for nucleosome positioning, [Hi-C](#) for looping interactions).

2.4.2 Gene Structure: Exons and Introns

In eukaryotes, most protein-coding genes have a **split structure**: the coding sequence is divided into segments called **exons**, separated by non-coding sequences called **introns**. When the gene is transcribed, the entire region (exons + introns) is copied into a precursor RNA molecule (pre-mRNA). The introns are then removed through a process called **splicing**, and the exons are joined together to form the mature mRNA (see ??).

This arrangement allows for **alternative splicing**: different combinations of exons can be included in the final mRNA, producing different protein variants (called **isoforms**) from a single gene. Alternative splicing dramatically increases the diversity of proteins that can be produced from a finite number of genes. It is estimated that over 95% of human multi-exon genes undergo alternative splicing.

2.4.3 Regulatory Regions

Genes do not exist in isolation—their expression is controlled by **regulatory DNA elements**:

Promoter

A DNA sequence located immediately upstream (5') of the gene's transcription start site (TSS). The promoter is where **RNA polymerase** and general **transcription factors** bind to initiate transcription. Many promoters contain a TATA box (consensus sequence TATAAA) approximately 25–30 bp upstream of the TSS, though many genes—particularly housekeeping genes—lack a TATA box and instead have CpG-island-associated promoters.

Enhancer

A regulatory DNA element that can increase transcription of a gene, often located thousands or even millions of base pairs away from the gene it regulates. Enhancers work

by binding transcription factors and physically looping to contact the gene's promoter through three-dimensional chromatin folding. Enhancers can function regardless of their orientation or position relative to the gene.

Silencer

A regulatory element that represses transcription, functioning conceptually as the opposite of an enhancer.

Insulator

A boundary element that prevents regulatory signals (enhancers, silencers) from affecting genes on the other side of the insulator. The protein CTCF is a key component of many insulator elements.

2.4.4 Repetitive Elements

Approximately 45–50% of the human genome consists of **repetitive elements**—DNA sequences present in many copies. Major categories include:

- **SINEs (Short Interspersed Nuclear Elements):** The most common are Alu elements (~300 bp), with over one million copies in the human genome.
- **LINEs (Long Interspersed Nuclear Elements):** The most common are LINE-1 (L1) elements (~6 kb), with about 500,000 copies.
- **DNA transposons:** “Cut-and-paste” mobile elements that are mostly inactive in the human genome.
- **Tandem repeats:** Including microsatellites (1–6 bp repeat units, e.g., CACACA...) and minisatellites (longer repeat units).
- **Segmental duplications:** Large blocks (>1 kb) of DNA that are duplicated, often in different locations in the genome.

Important Caveat

Repetitive regions pose a major challenge for sequencing and genome assembly. Short sequencing reads originating from repetitive elements cannot be unambiguously placed in the genome, leading to “multi-mapping” reads and assembly gaps. Long-read sequencing technologies (Chapter 3) have significantly improved our ability to resolve repetitive regions.

2.4.5 Coding vs. Non-Coding DNA

A common misconception is that most DNA “does something” by encoding proteins. In fact, the situation is far more nuanced:

- ~1.5% of the human genome encodes protein sequences (exons of protein-coding genes).
- ~5–10% is estimated to be under evolutionary constraint (functional, including regulatory elements and non-coding RNAs), based on comparative genomics.
- ~45–50% consists of repetitive and transposable elements.
- The function (or lack thereof) of the remaining DNA is an active area of research.

2.5 The Central Dogma of Molecular Biology

In 1958, Francis Crick articulated what he called the **Central Dogma of molecular biology**: the principle that genetic information flows from DNA to RNA to protein. While we now know of exceptions and nuances (reverse transcription, RNA editing, prions), this framework remains the core organising principle of molecular biology.

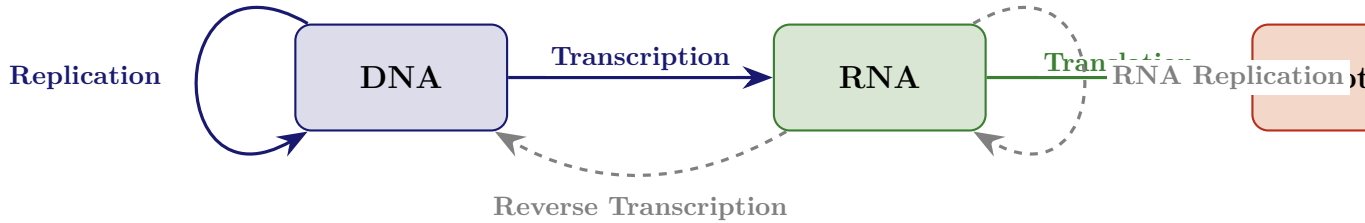


Figure 2.3: The Central Dogma: genetic information flows from DNA to RNA (transcription) to protein (translation). DNA can also be replicated. Dashed arrows indicate processes that occur in special contexts: reverse transcription (used by retroviruses and retrotransposons) and RNA-dependent RNA replication (used by some RNA viruses).

The Central Dogma (Fig. 2.3) describes three main processes:

1. **Replication:** DNA $\rightarrow$ DNA. The faithful copying of the genome before cell division.
2. **Transcription:** DNA $\rightarrow$ RNA. A gene's DNA sequence is copied into a messenger RNA (mRNA) molecule.
3. **Translation:** RNA $\rightarrow$ Protein. The mRNA sequence is decoded by ribosomes to produce a chain of amino acids (a polypeptide) that folds into a functional protein.

2.5.1 Kinetics of Information Flow

The speed and fidelity of the enzymes that execute the Central Dogma are directly relevant to sequencing, because many sequencing chemistries harness these same enzymes (or engineered variants of them). Table 2.2 summarises the key kinetic parameters.

Table 2.2: Kinetic parameters of the major polymerases involved in information flow. Processivity is the average number of nucleotides incorporated before the enzyme dissociates.

Enzyme	Function	Speed (nt/s)	Error rate	Processivity
<i>E. coli</i> DNA Pol III	Replication	1,000	10^{-7}	>500,000
Human DNA Pol δ	Replication (lagging)	30–50	10^{-7}	~500
Human DNA Pol ϵ	Replication (leading)	50–100	10^{-6} – 10^{-7}	>10,000
Phi29 DNA Pol	Isothermal amplification	25–50	10^{-6}	>70,000
<i>E. coli</i> RNA Pol	Transcription	40–80	10^{-4} – 10^{-5}	~10,000
Human RNA Pol II	Transcription	20–50	10^{-4} – 10^{-5}	Variable
Ribosome	Translation (aa/s)	5–20	10^{-3} – 10^{-4}	Complete mRNA

Several insights emerge from these kinetics:

- **Replication is extraordinarily accurate.** After proofreading ($3' \rightarrow 5'$ exonuclease activity), replicative DNA polymerases achieve error rates of $\sim 10^{-7}$ per base pair. With post-replication mismatch repair, the overall error rate drops to $\sim 10^{-9}$ – 10^{-10} per base pair per cell division.
- **Transcription is 100–1,000 $\times$ less accurate** than replication. This is tolerable because (1) each transcript is temporary, (2) many copies of each mRNA are made, and (3) errors in any single transcript are diluted by the population.
- **Sequencing-by-synthesis co-opts polymerase chemistry.** The Phi29 polymerase (used in PacBio SMRT sequencing) combines high processivity ($>70,000$ nt) with moderate speed and good accuracy—ideal for long, continuous reads. Understanding these enzymatic parameters explains why different platforms achieve different read lengths and error rates.

Fidelity Hierarchy

Information fidelity decreases at each step of the Central Dogma: DNA replication ($\sim 10^{-10}$ errors/bp after repair) $>$ transcription ($\sim 10^{-5}$ errors/nt) $>$ translation ($\sim 10^{-4}$ errors/aa). This hierarchy reflects biological priorities: the genome must be copied with near-perfect fidelity across generations, while temporary RNA and protein errors are tolerable.

2.6 Transcription: From DNA to RNA

Transcription is the process of synthesising an RNA copy of a gene's DNA sequence. It is carried out by the enzyme **RNA polymerase**, which reads the template strand of DNA ($3' \rightarrow 5'$) and synthesises a complementary RNA strand in the $5' \rightarrow 3'$ direction.

2.6.1 Transcription in Eukaryotes

Eukaryotic transcription is more complex than prokaryotic transcription and involves three main RNA polymerases:

- **RNA Polymerase I:** Transcribes ribosomal RNA (rRNA) genes (except 5S rRNA).
- **RNA Polymerase II:** Transcribes messenger RNA (mRNA) and most non-coding RNAs, including miRNAs and lncRNAs. This is the polymerase most relevant to gene expression studies.
- **RNA Polymerase III:** Transcribes transfer RNAs (tRNAs), 5S rRNA, and other small RNAs.

The process of mRNA transcription by RNA Polymerase II occurs in three stages:

Initiation

General transcription factors (TFIIA, TFIIB, TFIID, TFIIIE, TFIIF, TFIIF) assemble at the promoter, forming the **preinitiation complex**. TFIID recognises and binds the TATA box (via its TBP subunit). RNA Polymerase II is recruited, and the DNA double helix is unwound ("melted") at the transcription start site.

Elongation

RNA Polymerase II moves along the template strand, synthesising mRNA in the $5' \rightarrow 3'$ direction. The RNA transcript is complementary to the template strand and identical in sequence to the coding (sense) strand, except that uracil (U) replaces thymine (T).

Termination

Transcription continues past the end of the coding sequence until specific termination signals are reached. In eukaryotes, the pre-mRNA is cleaved and polyadenylated (see below), and the polymerase dissociates from the DNA.

2.6.2 mRNA Processing

In eukaryotes, the initial transcript (pre-mRNA) undergoes three critical processing steps before it can serve as a template for protein synthesis:

1. **5' Capping:** A modified guanine nucleotide (7-methylguanosine, m⁷G) is added to the 5' end of the pre-mRNA. The 5' cap protects the mRNA from degradation, facilitates ribosome binding during translation, and aids in nuclear export.
2. **Splicing:** Introns are removed and exons are joined together by a large ribonucleoprotein complex called the **spliceosome**. The spliceosome recognises specific sequences at intron–exon boundaries: the **5' splice site** (donor site, typically GU), the **3' splice site** (acceptor site, typically AG), and the **branch point** (an adenosine within the intron). As mentioned earlier, alternative splicing can produce different mRNA isoforms from the same gene.
3. **3' Polyadenylation:** A tail of 100–250 adenine nucleotides (the **poly-A tail**) is added to the 3' end of the mRNA. The poly-A tail protects the mRNA from degradation, facilitates nuclear export, and aids in translation. In many [RNA-seq](#) protocols, the poly-A tail is used to selectively capture mRNA from the total RNA pool (poly-A selection).

Practical Tip

The poly-A tail is exploited in many RNA sequencing library preparation protocols. Oligo-dT beads (short chains of thymine nucleotides) are used to capture mRNAs via their poly-A tails, enriching for mRNA and depleting abundant ribosomal RNA. However, this approach misses non-polyadenylated RNAs (such as many histone mRNAs and most non-coding RNAs). If your experiment aims to study non-coding RNAs, consider rRNA depletion instead of poly-A selection.

2.7 Translation: From RNA to Protein

Translation is the process by which the nucleotide sequence of an mRNA is decoded to produce a polypeptide (protein). This process takes place on **ribosomes**—complex molecular machines composed of ribosomal RNA (rRNA) and proteins.

2.7.1 The Genetic Code

The genetic code is the set of rules by which the nucleotide sequence of mRNA is translated into the amino acid sequence of a protein. The code is read in **triplets** called **codons**: each codon (a sequence of three nucleotides) specifies one amino acid or a stop signal.

Key properties of the genetic code:

- **64 codons** encode 20 standard amino acids plus 3 stop signals. Because there are more codons than amino acids, the code is **degenerate** (redundant): most amino acids are specified by more than one codon.

- **AUG** (encoding methionine) serves as the **start codon**, signalling the beginning of translation.
- **UAA, UAG, UGA** are **stop codons**, signalling the end of translation.
- The code is nearly **universal**: the same codons specify the same amino acids in virtually all organisms, from bacteria to humans (with minor exceptions in mitochondria and a few organisms).

2.7.2 The Process of Translation

Translation occurs in three phases:

Initiation

The small ribosomal subunit binds to the mRNA and scans for the start codon (AUG). A special initiator tRNA carrying methionine base-pairs with the start codon. The large ribosomal subunit then joins to form the complete ribosome.

Elongation

The ribosome moves along the mRNA in the $5' \rightarrow 3'$ direction, reading one codon at a time. For each codon, a **transfer RNA (tRNA)** molecule carrying the corresponding amino acid binds to the ribosome via complementary base pairing between the codon and the tRNA's **anticodon**. The amino acid is added to the growing polypeptide chain via a **peptide bond**.

Termination

When the ribosome reaches a stop codon, **release factors** bind and trigger the release of the completed polypeptide. The ribosome dissociates from the mRNA.

2.7.3 Protein Folding and Function

The newly synthesised polypeptide chain folds into a specific three-dimensional structure determined by its amino acid sequence. This folding occurs at multiple levels:

- **Primary structure:** The linear sequence of amino acids.
- **Secondary structure:** Local folding patterns, mainly **alpha helices** and **beta sheets**, stabilised by hydrogen bonds.
- **Tertiary structure:** The overall three-dimensional shape of a single polypeptide, stabilised by hydrophobic interactions, hydrogen bonds, ionic bonds, and disulfide bridges.
- **Quaternary structure:** The arrangement of multiple polypeptide subunits into a multi-subunit complex (e.g., haemoglobin is a tetramer of four polypeptide chains).

A protein's function depends critically on its three-dimensional structure. Mutations that alter the amino acid sequence can disrupt folding and cause disease.

2.8 Types of RNA

While mRNA is the most commonly studied RNA in sequencing experiments, cells produce a diverse array of RNA molecules with different functions:

Messenger RNA (mRNA)

Encodes the amino acid sequence of proteins. Represents approximately 3–5% of total cellular RNA by mass but is the most informationally diverse RNA class. This is the primary target of **RNA-seq** experiments.

Ribosomal RNA (rRNA)

The catalytic and structural RNA component of ribosomes. Comprises approximately 80–85% of total cellular RNA. In RNA-seq experiments, rRNA is either depleted or excluded via poly-A selection; otherwise, the vast majority of reads would come from rRNA.

Transfer RNA (tRNA)

Small adapter molecules (~75–90 nucleotides) that carry amino acids to the ribosome during translation. Each tRNA has an anticodon that pairs with the corresponding mRNA codon.

MicroRNA (miRNA)

Small non-coding RNAs (~22 nucleotides) that regulate gene expression post-transcriptionally by binding to complementary sequences in the 3' untranslated regions (3' UTR) of target mRNAs, leading to mRNA degradation or translational repression. A single miRNA can regulate hundreds of target genes.

Long non-coding RNA (lncRNA)

Non-coding RNAs longer than 200 nucleotides with diverse roles in gene regulation, chromatin remodelling, and nuclear organisation. Examples include XIST (involved in X-chromosome inactivation) and HOTAIR (involved in chromatin remodelling). Thousands of lncRNAs have been identified, but the functions of most remain unknown.

Circular RNA (circRNA)

Covalently closed circular RNA molecules formed by back-splicing, in which a downstream splice donor joins to an upstream splice acceptor. circRNAs are resistant to exonucleolytic degradation and have been implicated as miRNA sponges, protein scaffolds, and regulators of transcription.

Piwi-interacting RNA (piRNA)

Small RNAs (~24–32 nucleotides) that interact with Piwi proteins and function primarily in transposon silencing in the germline.

Small nuclear RNA (snRNA)

Components of the spliceosome; essential for pre-mRNA splicing (e.g., U1, U2, U4, U5, U6 snRNAs).

Small nucleolar RNA (snoRNA)

Guide chemical modifications (methylation and pseudouridylation) of rRNA and other RNAs.

RNA Diversity

The transcriptome is far more complex than just mRNA. Non-coding RNAs constitute the vast majority of cellular RNA and play critical roles in gene regulation, splicing, translation, and genome defence. Modern sequencing technologies can profile all of these RNA classes, but different library preparation methods are needed for different RNA types.

2.9 Epigenetics: Modifications Beyond the Sequence

Epigenetics refers to heritable changes in gene expression that occur without changes to the underlying DNA sequence. Epigenetic mechanisms control which genes are active in which cell types, and their dysregulation is implicated in cancer, developmental disorders, and ageing.

2.9.1 DNA Methylation

DNA methylation is the addition of a methyl group ($-\text{CH}_3$) to the carbon-5 position of cytosine, producing **5-methylcytosine (5mC)**. In mammals, this modification occurs predominantly at **CpG dinucleotides**—positions where a cytosine is immediately followed by a guanine on the same strand.

Key points about DNA methylation:

- **CpG islands:** Regions of the genome with a high density of CpG dinucleotides (typically >200 bp, with observed/expected CpG ratio >0.6 and GC content >50%). CpG islands are found at approximately 70% of human gene promoters.
- **Gene silencing:** Methylation of CpG islands in promoter regions is generally associated with **transcriptional silencing**. Methylated promoters recruit proteins that compact chromatin, making genes inaccessible to the transcription machinery.
- **Maintenance and *de novo* methylation:** DNA methyltransferase 1 (DNMT1) maintains methylation patterns during DNA replication by copying methyl marks from the parent strand to the newly synthesised strand. DNMT3A and DNMT3B establish new (“*de novo*”) methylation patterns.
- **Demethylation:** Active demethylation involves oxidation of 5mC to 5-hydroxymethylcytosine (5hmC), 5-formylcytosine (5fC), and 5-carboxylcytosine (5caC) by TET (Ten-eleven translocation) enzymes, followed by base excision repair.

DNA methylation can be measured genome-wide using **WGBS**, which treats DNA with sodium bisulfite to convert unmethylated cytosines to uracil (read as thymine after **PCR**) while leaving methylated cytosines unchanged. This is discussed in detail in later chapters.

2.9.2 Histone Modifications

Histones are not merely passive spools for DNA packaging. The N-terminal “tails” of histone proteins protrude from the nucleosome and can be chemically modified at specific amino acid residues. These modifications—collectively called **histone marks**—influence chromatin structure and gene expression.

Major types of histone modifications include:

- **Acetylation** (e.g., H3K27ac, H3K9ac): Addition of an acetyl group, generally associated with active gene expression. Acetylation neutralises the positive charge of lysine residues, loosening the interaction between histones and negatively charged DNA.
- **Methylation** (e.g., H3K4me3, H3K36me3, H3K27me3, H3K9me3): Addition of one, two, or three methyl groups to lysine or arginine residues. Unlike acetylation, methylation can be associated with either activation or repression, depending on the specific residue and the degree of methylation:
 - H3K4me3 (trimethylation of lysine 4 on histone H3): marks *active* promoters.
 - H3K27me3: marks *repressed* genes (Polycomb repression).
 - H3K9me3: marks *constitutive heterochromatin*.
 - H3K36me3: marks *actively transcribed* gene bodies.
- **Phosphorylation** (e.g., H3S10ph): Addition of a phosphate group. Associated with chromosome condensation during mitosis and with transcriptional activation.
- **Ubiquitylation** (e.g., H2AK119ub, H2BK120ub): Addition of the small protein ubiquitin. Involved in both activation and repression.

The combination of histone modifications at a given genomic locus constitutes what is sometimes called the **histone code**—a complex regulatory layer that is read by effector proteins to control gene expression. These modifications can be mapped genome-wide using ChIP-seq and related techniques (see later chapters).

2.9.3 Chromatin Remodelling

In addition to covalent modifications, chromatin structure is actively remodelled by **ATP-dependent chromatin remodelling complexes** (such as SWI/SNF, ISWI, CHD, and INO80 families). These enzymes use the energy of ATP hydrolysis to slide, eject, or restructure nucleosomes, thereby altering the accessibility of DNA to transcription factors and other regulatory proteins. Chromatin accessibility can be measured genome-wide using ATAC-seq (see later chapters).

The Epigenome as a Regulatory Layer

The epigenome—comprising DNA methylation, histone modifications, and chromatin accessibility—acts as a regulatory layer on top of the DNA sequence. It determines which genes are active in which cell types, without changing the genetic code itself. Every cell in your body (with rare exceptions) has the same genome, but different epigenomes are what make a neuron different from a liver cell. Modern sequencing technologies can map each of these epigenetic marks at single-nucleotide or single-cell resolution.

2.10 DNA Replication and Repair

2.10.1 DNA Replication

Before a cell divides, it must duplicate its entire genome so that each daughter cell receives a complete copy. **DNA replication** is the process by which the double-stranded DNA molecule is copied. Key features include:

- **Semi-conservative:** Each daughter DNA molecule contains one original (parental) strand and one newly synthesised strand. This was demonstrated by the famous Meselson–Stahl experiment in 1958.
- **Bidirectional:** Replication proceeds in both directions from each **origin of replication**. Eukaryotic genomes have thousands of origins to ensure the entire genome is replicated within the S phase of the cell cycle.
- **Key enzymes:**
 - **Helicase:** Unwinds the double helix at the replication fork.
 - **Primase:** Synthesises short RNA primers to provide a 3'-OH group for DNA polymerase.
 - **DNA polymerase:** Synthesises new DNA in the 5' → 3' direction, adding nucleotides complementary to the template strand. DNA polymerase cannot start synthesis *de novo*—it can only extend an existing primer.
 - **Ligase:** Joins Okazaki fragments on the lagging strand.
- **Leading and lagging strands:** Because DNA polymerase can only synthesise in the 5' → 3' direction but the two template strands run antiparallel, replication is continuous on the **leading strand** (synthesised in the direction of fork movement) and discontinuous

on the **lagging strand** (synthesised in short fragments called **Okazaki fragments** that are later joined).

- **Proofreading:** DNA polymerases have a $3' \rightarrow 5'$ exonuclease activity that allows them to correct errors immediately, achieving an error rate of approximately one mistake per 10^9 – 10^{10} base pairs replicated.

2.10.2 DNA Repair

DNA is constantly damaged by environmental factors (UV radiation, chemical mutagens) and by-products of normal metabolism (reactive oxygen species). Cells have evolved multiple DNA repair pathways:

- **Mismatch repair (MMR):** Corrects base-pairing errors that escape proofreading during replication.
- **Base excision repair (BER):** Repairs small, non-helix-distorting lesions (e.g., oxidised or deaminated bases).
- **Nucleotide excision repair (NER):** Repairs bulky, helix-distorting lesions (e.g., thymine dimers caused by UV light).
- **Homologous recombination (HR):** Repairs double-strand breaks using the homologous chromosome or sister chromatid as a template. This is a high-fidelity repair mechanism.
- **Non-homologous end joining (NHEJ):** Repairs double-strand breaks by directly ligating the broken ends together. This is faster but more error-prone than HR, as it can introduce small insertions or deletions.

Defects in DNA repair pathways are associated with increased mutation rates and cancer predisposition. For example, mutations in BRCA1 and BRCA2 (which function in homologous recombination) dramatically increase the risk of breast and ovarian cancer.

2.11 Genetic Variation

No two individuals (except identical twins) have exactly the same genome. The differences between individual genomes—collectively called **genetic variation**—are the raw material for evolution and the basis for individual traits, disease susceptibility, and pharmacological response. Understanding the types of genetic variation is essential for interpreting sequencing data.

2.11.1 Single Nucleotide Polymorphisms (SNPs)

A **single nucleotide polymorphism (SNP)** is a position in the genome where a single nucleotide differs between individuals (or between the two copies of a chromosome within an individual). SNPs are the most common type of genetic variation. The human genome contains approximately 4–5 million SNPs per individual (compared to the reference genome), occurring roughly once every 600–1,000 base pairs.

Most SNPs have no functional consequence (they occur in non-coding, non-regulatory regions). However, some SNPs can:

- Alter the amino acid sequence of a protein (missense variants).
- Create a premature stop codon (nonsense variants).

- Affect gene regulatory elements (eQTLs—expression quantitative trait loci).
- Influence splicing (splice site variants).

2.11.2 Insertions and Deletions (Indels)

Insertion or deletions (indels) are genetic variants in which one or more nucleotides are inserted into or deleted from the genome. Short indels (1–50 bp) are the second most common type of variation. Indels in protein-coding regions can cause **frameshift mutations** if their length is not a multiple of three, disrupting the reading frame of the gene and typically rendering the protein non-functional.

2.11.3 Structural Variants (SVs)

Structural variants (SVs) are large-scale genomic alterations (typically defined as variants >50 bp) that include:

- **Deletions:** Loss of a segment of DNA.
- **Duplications:** Copying of a segment, resulting in extra copies.
- **Inversions:** Reversal of the orientation of a segment.
- **Translocations:** Transfer of a segment from one chromosome to another.
- **Complex rearrangements:** Combinations of the above.

Structural variants affect more total base pairs of the genome than SNPs and indels combined, yet they have historically been harder to detect because short sequencing reads often cannot span them. Long-read sequencing technologies have greatly improved SV detection.

2.11.4 Copy Number Variants (CNVs)

Copy number variations (CNVs) are a subtype of structural variant in which segments of the genome (typically >1 kb) are present in a variable number of copies. CNVs can encompass entire genes, leading to differences in gene dosage between individuals. Some CNVs are common and benign; others are associated with disease (e.g., the association between CNVs in the *SMN1* gene and spinal muscular atrophy).

Important Caveat

Different types of genetic variation require different sequencing strategies for reliable detection. SNPs and short indels are well-detected by short-read WGS at $\geq 30\times$ coverage. Structural variants and CNVs often require long-read sequencing, linked-read sequencing, or specialised algorithms. No single method detects all variant types with equal sensitivity.

2.12 Gene Regulation: Controlling When and Where Genes Are Expressed

Not all genes are active in every cell. Gene expression is tightly regulated at multiple levels, from the accessibility of DNA to the stability of mRNA and protein. Understanding gene regulation is essential because many sequencing assays are specifically designed to measure different aspects of this regulation.

2.12.1 Transcription Factors

Transcription factors (TFs) are proteins that bind to specific DNA sequences (called **motifs** or **binding sites**) and activate or repress transcription. The human genome encodes approximately 1,500–1,600 transcription factors. They can be classified as:

- **Activators:** Bind to enhancers or promoters and recruit co-activators and the transcription machinery, increasing transcription.
- **Repressors:** Bind to silencers or promoters and recruit co-repressors or block activator binding, decreasing transcription.
- **Pioneer factors:** A special class of transcription factors that can bind to closed (nucleosome-occupied) chromatin and initiate the opening process, enabling other factors to bind subsequently.

Transcription factor binding sites can be mapped genome-wide using **ChIP-seq**, **CUT&RUN**, or **CUT&Tag**.

2.12.2 Enhancers and Super-Enhancers

As described in [Section 2.4.3](#), **enhancers** are distal regulatory elements that increase gene transcription. Key features:

- Enhancers can be located tens of kilobases to megabases from the genes they regulate.
- They function by physically contacting promoters through **chromatin looping**.
- Active enhancers are typically marked by H3K27ac, H3K4me1, open chromatin (detectable by **ATAC-seq**), and the presence of bound transcription factors.
- **Super-enhancers** are clusters of enhancers that drive high-level expression of genes important for cell identity. They are densely bound by transcription factors and the Mediator co-activator complex.

2.12.3 Chromatin Accessibility and Gene Expression

The accessibility of DNA to transcription factors and RNA polymerase is a critical determinant of gene expression. Regions of **open chromatin** (depleted of nucleosomes) are more likely to be regulatory elements. Genome-wide maps of chromatin accessibility, obtained through **ATAC-seq** or **DNase-seq**, provide a snapshot of the regulatory landscape of a cell.

2.12.4 Post-Transcriptional Regulation

Gene expression is also regulated after transcription:

- **mRNA stability:** The half-life of mRNAs varies from minutes to hours. Regulatory elements in the 3' UTR, as well as miRNA binding, influence mRNA degradation rates.
- **Translational regulation:** The rate at which ribosomes translate an mRNA can be regulated by upstream open reading frames (uORFs), RNA secondary structures, and RNA-binding proteins.
- **RNA modifications:** Chemical modifications of mRNA, such as N⁶-methyladenosine (m⁶A), affect mRNA stability, splicing, and translation. The study of these modifications is called **epitranscriptomics**.

Multi-Level Gene Regulation

Gene expression is regulated at every step from DNA to functional protein: chromatin accessibility, transcription factor binding, transcription initiation and elongation, mRNA processing and splicing, mRNA export, mRNA stability, translation, and post-translational protein modification. Modern sequencing technologies can interrogate most of these levels, providing an unprecedented view of gene regulation.

2.13 Summary

This chapter has covered the molecular biology essentials needed to understand sequencing technologies:

- **Cells** come in two fundamental types: prokaryotic (no nucleus) and eukaryotic (membrane-bound nucleus and organelles).
- **DNA** is a double-stranded, antiparallel molecule composed of four nucleotides (A, T, G, C) held together by complementary base pairing.
- DNA is **packaged** into nucleosomes, chromatin, and chromosomes through a hierarchy of structures that influence gene expression.
- **Genes** are segments of DNA that encode functional products (proteins or RNAs), and their structure includes exons, introns, promoters, and enhancers.
- The **Central Dogma**—DNA → RNA → Protein—describes the flow of genetic information through transcription and translation.
- Cells produce many **types of RNA** beyond mRNA, including rRNA, tRNA, miRNA, lncRNA, and others, each with distinct functions.
- The **epigenome**—DNA methylation, histone modifications, and chromatin accessibility—provides a regulatory layer that controls gene expression without altering the DNA sequence.
- **Genetic variation**—SNPs, indels, structural variants, and CNVs—underlies individual differences in traits and disease susceptibility.
- **Gene regulation** operates at multiple levels, from chromatin structure to post-transcriptional processing.

With this foundation in place, we are ready to explore how modern sequencing technologies work at the chemical and physical level in [Chapter 3](#).

Chapter 3

Principles of Modern Sequencing

Key Concept

All modern sequencing technologies—despite their remarkably different chemistries—share a common logical framework: fragment nucleic acids, attach synthetic adapters, and read the sequence of individual fragments in a massively parallel fashion. This chapter explains the core principles that underpin every sequencing platform, from the chemistry of base reading to the metrics used to evaluate sequencing quality. Understanding these principles will prepare you for the platform-specific details in subsequent chapters.

3.1 What All Sequencing Technologies Share

Despite their diversity, all modern sequencing technologies share a common workflow with three essential steps:

1. **Fragmenting the nucleic acid:** Genomic DNA or RNA is broken into pieces of a manageable size. DNA can be fragmented mechanically (sonication, acoustic shearing) or enzymatically (tagmentation using Tn5 transposase). RNA is typically fragmented chemically (heat and divalent cations) or enzymatically after conversion to complementary DNA (cDNA).
2. **Attaching adapters:** Short, synthetic DNA sequences—called **adapters**—are ligated to the ends of fragments. Adapters serve multiple critical functions:
 - They allow fragments to bind to the sequencing instrument's flow cell or bead surface.
 - They provide a priming site for sequencing reactions.
 - They contain index (barcode) sequences that identify which sample a fragment came from, enabling multiplexing.
 - On Illumina platforms, they contain sequences needed for bridge amplification (cluster generation).
3. **Reading bases:** The instrument reads the sequence of each fragment, one base at a time (in most technologies) or by monitoring a continuous signal (in nanopore and SMRT sequencing). The output is a collection of **reads**—short (or long) text strings representing the nucleotide sequences of individual fragments, accompanied by quality scores.

The collection of adapter-ligated fragments is called a **sequencing library**. We will return to the concept of a library in detail in [Section 3.11](#).

Practical Tip

Think of a sequencing library like a collection of books in a physical library—each “book” (fragment) has a barcode label (adapter) that identifies it and allows the library system (sequencing instrument) to read it. The quality of the library determines the quality of the data.

3.2 Sequencing by Synthesis (SBS)

Sequencing by synthesis (SBS) is the most widely used sequencing chemistry, employed by Illumina platforms. The fundamental idea is to monitor DNA polymerase as it synthesises a new strand complementary to a template, detecting each nucleotide as it is incorporated.

3.2.1 The Chemistry

Illumina’s **SBS** uses **reversible terminators**—modified nucleotides with two key features:

1. A **fluorescent label** attached to the base (each of the four bases—A, T, G, C—carries a different colour fluorophore).
2. A **removable blocking group** on the 3′-OH of the sugar, which prevents the polymerase from adding more than one nucleotide per cycle.

In each cycle:

1. All four reversible terminators (fluorescently labelled A, T, G, C) are flowed across the flow cell simultaneously.
2. DNA polymerase incorporates the single complementary nucleotide at the 3′ end of each growing strand.
3. The blocking group prevents further incorporation—only one base is added per cycle.
4. The flow cell is imaged: a laser excites the fluorophore, and the emitted colour identifies the incorporated base.
5. The fluorescent label and the 3′ blocking group are chemically cleaved, regenerating a free 3′-OH.
6. The cycle repeats for the next base.

This cycle-by-cycle approach—incorporate, image, cleave, repeat—typically runs for 50–300 cycles, producing reads of 50–300 bp.

3.2.2 Cluster Generation

Before sequencing can begin on Illumina platforms, individual library fragments must be amplified into clusters of identical copies to generate a detectable fluorescent signal. This is accomplished through **bridge amplification** (also called **cluster generation**) on the surface of a **flow cell**:

1. Library fragments are denatured (single-stranded) and flow across the flow cell, which is coated with short oligonucleotides complementary to the adapter sequences.

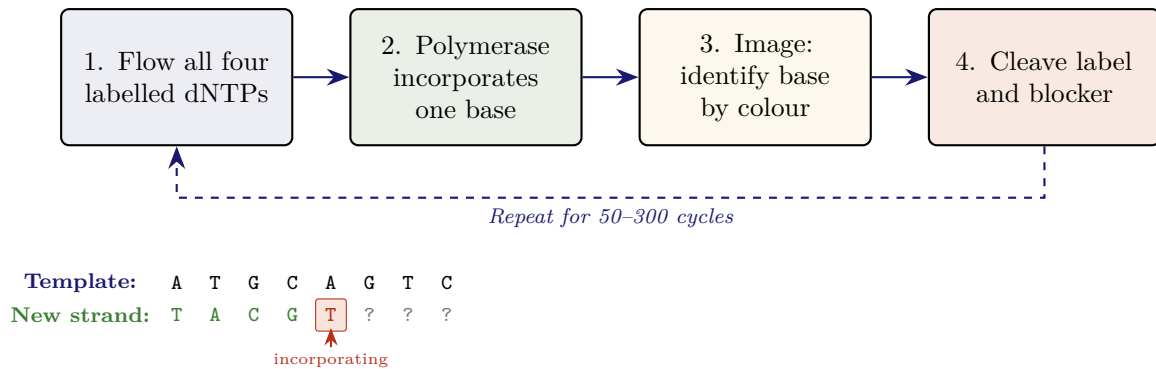


Figure 3.1: The cycle of Illumina sequencing by synthesis (SBS). In each cycle, all four fluorescently labelled reversible terminators are presented to the growing strand. DNA polymerase incorporates one complementary base, which is identified by its fluorescent colour after imaging. The fluorescent label and 3' blocking group are then removed, and the cycle repeats.

2. Fragments hybridise to the surface oligos and are copied by a polymerase.
3. The original template is washed away; the new copy bends over and hybridises to an adjacent surface oligo (forming a “bridge”).
4. The bridge is extended by polymerase, creating a double-stranded bridge.
5. The bridge is denatured, yielding two single-stranded copies tethered to the surface.
6. This process repeats through many cycles, producing a tight cluster of approximately 1,000 identical copies of the original fragment.

Each cluster produces a coherent fluorescent signal during sequencing, effectively amplifying the signal from a single molecule.

Important Caveat

Cluster generation introduces a potential source of error: if two different library fragments land too close together on the flow cell and their clusters overlap, the result is a “mixed” cluster that produces ambiguous base calls. More sophisticated patterned flow cells (used in NovaSeq and newer platforms) mitigate this by arranging cluster positions in a regular grid, but mixed clusters remain a consideration in data quality.

3.3 Sequencing by Ligation

Sequencing by ligation is an alternative approach in which DNA sequence is determined by the successive ligation (joining) of fluorescently labelled oligonucleotide probes to a primer hybridised to the template. Rather than using a polymerase to add single nucleotides, a ligase is used to join short probes whose identity encodes the sequence of the template.

This approach was used by the **SOLiD** (Sequencing by Oligonucleotide Ligation and Detection) platform from Applied Biosystems (later Life Technologies, now Thermo Fisher). In SOLiD sequencing:

1. A universal primer hybridises to the adapter sequence adjacent to the unknown template.
2. A pool of fluorescently labelled 8-mer oligonucleotide probes is added. Each probe interrogates two bases at a time (the first two bases of the probe encode the fluorescent colour in a two-base encoding scheme).

3. DNA ligase joins the probe to the primer if it matches the template.
4. The fluorescent signal is recorded, identifying the dinucleotide.
5. The fluorescent label is cleaved, and the cycle repeats with the next probe.
6. After several rounds, the extended strand is stripped, and the process restarts with a primer offset by one base, eventually covering every position.

The two-base encoding scheme in SOLiD meant that each base was interrogated twice, providing inherent error correction. However, the short read lengths (50–75 bp), relatively slow throughput, and complex data analysis led to SOLiD being largely superseded by Illumina platforms. The sequencing-by-ligation concept is primarily of historical interest, though elements of it persist in some specialised applications.

3.4 Nanopore Sequencing

Nanopore sequencing, commercialised by [Oxford Nanopore Technologies \(ONT\)](#), takes a fundamentally different approach: instead of synthesising a new strand or ligating probes, it reads DNA (or RNA) directly by threading a single molecule through a tiny protein pore and measuring the resulting changes in electrical current.

3.4.1 The Principle

The core of a nanopore sequencer is a biological **nanopore**—a protein channel (currently a modified version of the CsgG protein) embedded in an electrically resistant polymer membrane. A voltage is applied across the membrane, driving a constant ionic current through the pore. When a strand of DNA passes through the pore:

1. A **motor protein** (a helicase) at the top of the pore controls the translocation speed, feeding the DNA strand through the pore one nucleotide at a time.
2. As each nucleotide (or, more precisely, a k-mer of approximately 5–6 nucleotides) passes through the constriction zone of the pore, it partially blocks the ionic current.
3. The degree of current disruption depends on the identity and combination of bases in the constriction zone.
4. A sensitive amplifier measures the current in real time, producing a **signal trace** (a “squiggle”) of current vs. time.
5. A computational algorithm (**basecaller**, typically a neural network) converts the raw signal into a nucleotide sequence.

3.4.2 Key Features of Nanopore Sequencing

- **Ultra-long reads:** There is no theoretical upper limit to read length. Reads exceeding 4 Mb have been demonstrated. In routine practice, reads of 10–100 kb are common with appropriate library preparation.
- **Direct detection of base modifications:** Because the raw signal is sensitive to chemical modifications, nanopore sequencing can detect 5-methylcytosine (5mC), 5-hydroxymethylcytosine (5hmC), N⁶-methyladenosine (m⁶A), and other modifications directly, without bisulfite conversion or antibody enrichment.

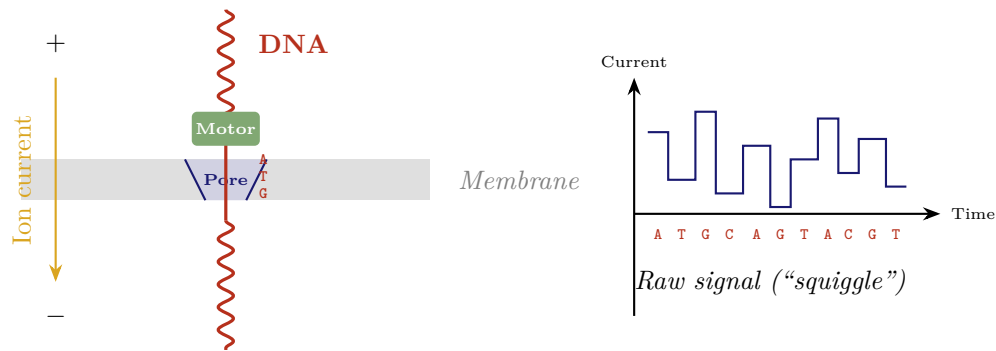


Figure 3.2: The principle of nanopore sequencing. A single strand of DNA is threaded through a protein nanopore embedded in a membrane. A motor protein controls the translocation speed. As bases pass through the pore’s constriction, they modulate the ionic current. The resulting signal trace (“squiggle”) is decoded by a neural network basecaller to determine the DNA sequence.

- **Direct RNA sequencing:** ONT offers a direct RNA sequencing protocol that threads native RNA molecules through the pore, avoiding reverse transcription and PCR amplification entirely.
- **Real-time data:** Data is generated and can be analysed as the sequencing run progresses, enabling applications like adaptive sampling (selectively sequencing or rejecting individual molecules in real time).
- **Portability:** The MinION weighs about 100 grams and connects via USB. It has been used in the field for pathogen surveillance, in space on the ISS, and in Antarctic research stations.
- **Higher error rate:** Raw single-pass accuracy is approximately 92–98% (depending on pore chemistry and basecaller version), with errors primarily being insertions and deletions (indels), especially in homopolymer regions (runs of the same base). Consensus accuracy from multiple reads of the same region can exceed 99.9%.

3.5 Single-Molecule Real-Time (SMRT) Sequencing

Single-molecule real-time sequencing (SMRT sequencing), developed by Pacific Biosciences (PacBio), reads individual DNA molecules in real time by watching a single polymerase as it synthesises a complementary strand.

3.5.1 The Principle

SMRT sequencing uses a **zero-mode waveguide (ZMW)**—a tiny well (approximately 70 nm in diameter and 100 nm deep) fabricated in a metal film on a glass substrate. A single DNA polymerase molecule is immobilised at the bottom of each ZMW, and a single circular DNA template molecule (a **SMRTbell**) is bound to the polymerase.

1. All four fluorescently labelled nucleotides (each with a distinct colour) are present in solution simultaneously.
2. When the polymerase binds and incorporates a nucleotide, the fluorophore is held in the detection volume of the ZMW long enough to emit a characteristic pulse of light.
3. The fluorophore is cleaved from the nucleotide as part of the incorporation process (it is attached to the phosphate group, which is released as pyrophosphate).

4. The sequence of fluorescent pulses over time reveals the sequence of incorporated bases.

The ZMW is so small that even though the solution is filled with fluorescent nucleotides, only nucleotides within the tiny detection volume at the bottom of the well contribute significantly to the signal. This allows detection of single-molecule incorporation events against the background.

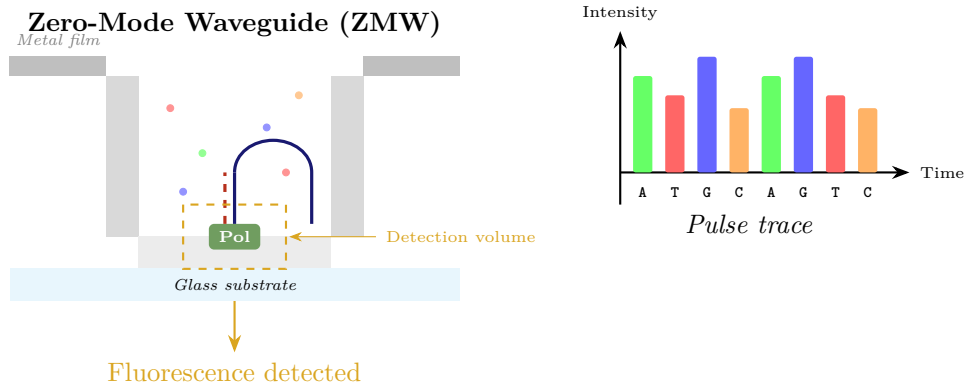


Figure 3.3: The principle of PacBio SMRT sequencing. A single DNA polymerase is immobilised at the bottom of a zero-mode waveguide (ZMW). Fluorescently labelled nucleotides are incorporated one at a time; each incorporation produces a characteristic pulse of light whose colour identifies the base. The ZMW's tiny detection volume ensures that only nucleotides being incorporated contribute to the signal.

3.5.2 SMRTbell Templates and HiFi Reads

A key innovation in PacBio sequencing is the **SMRTbell** library format. The double-stranded DNA insert is flanked by hairpin adapters, creating a single-stranded circular molecule. The polymerase can read around the circle multiple times (called **subreads** or **passes**), generating redundant coverage of the same molecule.

- **Continuous long reads (CLR):** The polymerase reads the SMRTbell once or a few times. CLR reads can be very long (>50 kb) but have a raw single-pass error rate of approximately 10–15% (mostly insertion errors).
- **HiFi reads (CCS):** For shorter inserts (typically 10–25 kb), the polymerase makes many passes around the SMRTbell. The multiple passes are computationally combined via **circular consensus sequencing (CCS)** to produce **HiFi reads** with accuracy exceeding 99.9% (Q30+). HiFi reads combine the advantages of long reads (resolving repeats, structural variants, and full-length transcripts) with the accuracy of short reads.

HiFi Reads: The Best of Both Worlds

PacBio HiFi reads (typically 10–25 kb, >Q20 accuracy) represent a significant advance: they are long enough to span most repetitive elements and structural variants, yet accurate enough for confident variant calling and haplotype phasing. HiFi sequencing has become a cornerstone technology for high-quality genome assembly and comprehensive variant detection.

3.6 Key Sequencing Metrics

To evaluate and compare sequencing technologies and to plan experiments, several key metrics are used.

3.6.1 Read Length

The **read length** is the number of bases sequenced per fragment (per read). It varies dramatically across platforms:

- **Illumina:** Typically 50–300 bp (most commonly 150 bp paired-end).
- **PacBio HiFi:** Typically 10–25 kb.
- **PacBio CLR:** Up to >100 kb.
- **Oxford Nanopore:** Routinely 10–100 kb; records exceed 4 Mb.

Longer reads are advantageous for resolving repetitive regions, detecting structural variants, phasing haplotypes, and assembling genomes. Shorter reads are typically cheaper per base and have lower per-base error rates.

3.6.2 Throughput

Throughput is the total amount of sequence data generated per run, usually expressed in gigabases (Gb). High-throughput instruments (e.g., Illumina NovaSeq X: up to ~16 terabases per year) generate far more data per run than portable devices (e.g., ONT MinION: ~50 Gb per flow cell).

3.6.3 Accuracy and Error Profiles

Accuracy describes how often the sequencer correctly identifies the base. Different technologies exhibit different **error profiles**:

- **Illumina (SBS):** Very high per-base accuracy (>99.9%). Dominant error type is **substitution errors** (one base called instead of another). Errors increase toward the end of reads due to phasing effects and signal decay.
- **PacBio (SMRT):** Raw single-pass accuracy ~85–90%, but HiFi consensus accuracy exceeds 99.9%. Dominant error type is **insertion errors**.
- **Oxford Nanopore:** Raw single-pass accuracy ~92–98% (improving with newer pore chemistries and basecallers). Dominant error types are **insertions and deletions (indels)**, particularly in homopolymer regions (e.g., AAAAAA).

Important Caveat

The error *type* matters as much as the error *rate*. Substitution errors (Illumina) are relatively easy to correct computationally because they rarely occur at the same position in multiple reads. Systematic indel errors (nanopore, PacBio CLR), especially in homopolymers, are more challenging because they can be consistent across reads from the same region.

3.7 Paired-End vs. Single-End Reads

When sequencing a library on Illumina platforms, you can choose to sequence from one end of each fragment (**single-end** reads) or from both ends (**paired-end** reads).

Single-end (SE) sequencing

Each fragment is sequenced from one end only, producing one read per fragment.
Example: 1×75 bp (one read of 75 bases).

Paired-end (PE) sequencing

Each fragment is sequenced from both ends, producing two reads per fragment. The two reads are from opposite ends of the same fragment and are in opposite orientations.
Example: 2×150 bp (two reads of 150 bases each, from opposite ends).

Paired-end sequencing provides several advantages:

- **Improved alignment:** Both reads from a pair should map to locations separated by the expected insert size, providing a constraint that improves mapping accuracy, especially in repetitive regions.
- **Structural variant detection:** Abnormal spacing or orientation of paired reads indicates insertions, deletions, inversions, or translocations.
- **Improved assembly:** Paired reads provide long-range connectivity information that helps bridge gaps in genome assembly.
- **Insert size estimation:** The distance between paired reads provides the fragment size distribution, which is a useful quality metric.

Practical Tip

For most applications (WGS, RNA-seq, ATAC-seq, ChIP-seq), paired-end 150 bp (2×150) sequencing is the standard choice on Illumina platforms. Single-end sequencing may be cost-effective for some applications (e.g., miRNA-seq, where fragments are short) or when the primary goal is counting (e.g., some ChIP-seq experiments).

3.8 Read Depth and Coverage

Two of the most important concepts in experimental design are **read depth** and **coverage**.

3.8.1 Definitions

Sequencing depth (read depth)

The total number of reads generated in a sequencing experiment. Sometimes expressed as the total number of base pairs generated.

Coverage (fold coverage, denoted ×)

The average number of times each base in the target region (e.g., the genome) is read. For example, 30× coverage means that, on average, each position in the genome is covered by 30 independent reads.

Coverage can be calculated as:

$$\text{Coverage} = \frac{N \times L}{G} \quad (3.1)$$

where N is the number of reads, L is the read length (in base pairs), and G is the genome size (in base pairs).

Calculating Coverage

For a human genome ($G \approx 3.2 \times 10^9$ bp) sequenced with 800 million paired-end 150 bp reads ($N = 800 \times 10^6 \times 2 = 1.6 \times 10^9$ reads total, $L = 150$ bp):

$$\text{Coverage} = \frac{1.6 \times 10^9 \times 150}{3.2 \times 10^9} = \frac{2.4 \times 10^{11}}{3.2 \times 10^9} = 75\times$$

This is a rough estimate. Actual coverage will be somewhat lower due to PCR duplicates, low-quality reads, and reads mapping to multiple locations.

3.8.2 Why Coverage Matters

Coverage is critical because sequencing is fundamentally a **sampling process**: each read represents a random sample from the pool of library fragments. Higher coverage means more samples, which translates to:

- **More confident base calls:** At a heterozygous SNP position (where the individual has one copy of each allele), you need enough reads to observe both alleles reliably. At 30 $\times$ coverage, a heterozygous site will have, on average, ~ 15 reads supporting each allele, providing high confidence.
- **Better structural variant detection:** SVs require multiple discordant or split reads to be detected with confidence.
- **More uniform representation:** Higher coverage helps compensate for biases in library preparation that cause some regions to be over-represented and others under-represented.

Typical coverage requirements vary by application:

Table 3.1: Typical coverage requirements for common sequencing applications.

Application	Purpose	Typical Coverage
WGS (germline variants)	SNV and indel detection	30–60 $\times$
WGS (somatic variants)	Tumour mutation detection	60–100 $\times$
WES (exome)	Coding region variant detection	100–200 $\times$
RNA-seq (bulk)	Gene expression quantification	20–50M reads/sample
scRNA-seq	Single-cell expression profiling	20,000–50,000 reads/cell
ChIP-seq	Protein–DNA binding	20–40M reads/sample
ATAC-seq	Chromatin accessibility	50–75M reads/sample
WGBS	DNA methylation	10–30 $\times$
Hi-C	3D genome architecture	200M–1B read pairs

Important Caveat

Coverage is an *average*. Due to biases (GC content, mappability, library complexity), some regions may have far more or far less coverage than the average. For clinical applications, minimum coverage thresholds at specific genomic positions (e.g., $\geq 20\times$ at $\geq 95\%$ of target bases) are often more informative than average coverage alone.

3.8.3 The Lander–Waterman Model of Coverage

The relationship between average coverage and the fraction of the genome covered by at least one read is formalised by the **Lander–Waterman model** (1988). If we assume that reads are distributed uniformly and independently (a Poisson process), then the probability that a given base is *not* covered by any read is:

$$P(\text{uncovered}) = e^{-c} \quad (3.2)$$

where $c = NL/G$ is the average coverage depth. Therefore, the fraction of the genome covered by at least one read is:

$$P(\text{covered}) = 1 - e^{-c} \quad (3.3)$$

More generally, the probability that a position is covered by exactly k reads follows a Poisson distribution:

$$P(X = k) = \frac{c^k e^{-c}}{k!} \quad (3.4)$$

Coverage Thresholds from the Lander–Waterman Model

At 30× average coverage, the Lander–Waterman model predicts that $1 - e^{-30} \approx 99.999999\%$ of the genome is covered by at least one read. However, the fraction covered at $\geq 20\times$ (important for confident variant calling) is:

$$P(X \geq 20 \mid c = 30) = 1 - \sum_{k=0}^{19} \frac{30^k e^{-30}}{k!} \approx 98.5\%$$

This means roughly 1.5% of the genome may fall below 20× even at 30× average coverage—a non-trivial fraction for clinical applications.

The Lander–Waterman model assumes **uniform random sampling**, which is an idealisation. In practice, coverage is not Poisson-distributed due to:

- **GC bias:** Regions with extreme GC content (very high or very low) are under-represented in most library preparations, particularly those involving PCR amplification.
- **Mappability:** Repetitive regions where reads cannot be uniquely placed effectively have zero usable coverage.
- **Library preparation biases:** Fragmentation, adapter ligation, and amplification each introduce systematic deviations from uniformity.

A more realistic model replaces the Poisson with a **negative binomial** distribution, which adds a dispersion parameter r to account for overdispersion:

$$P(X = k) = \binom{k+r-1}{k} \left(\frac{r}{r+c}\right)^r \left(\frac{c}{r+c}\right)^k \quad (3.5)$$

where the variance is $\text{Var}(X) = c + c^2/r$. As $r \rightarrow \infty$, the negative binomial converges to the Poisson.

3.8.4 Adapter Ligation Efficiency

A critical but often overlooked parameter in library preparation is the **adapter ligation efficiency**—the fraction of DNA fragments that successfully receive adapters on both ends and are therefore sequenceable. Fragments with adapters on only one end (or no adapters) are invisible to the sequencing instrument.

Ligation efficiency depends on several factors:

- **End quality:** Blunt-ended or A-tailed fragments ligate more efficiently than fragments with damaged or ragged ends. Enzymatic end repair is a standard step to maximise efficiency.
- **Adapter:insert molar ratio:** Typically 10:1 to 100:1 excess of adapter molecules to fragment ends, ensuring that adapters are not limiting.
- **Ligase and buffer conditions:** T4 DNA ligase at 20 °C for 10–30 minutes is standard. Lower temperatures favour ligation fidelity; longer incubations increase efficiency.
- **Fragment size:** Very short fragments (<100 bp) may ligate to adapters less efficiently due to steric constraints, while very long fragments (>20 kb) may suffer from shearing during handling.

If we denote the probability of successful adapter ligation to one end as p , then the probability that a fragment receives adapters on both ends (and is therefore sequenceable) is p^2 . For typical ligation efficiencies of $p = 0.7$ – 0.9 , this yields:

Table 3.2: Effect of single-end ligation efficiency on overall library yield.

Single-end efficiency (p)	Both-ends efficiency (p^2)	Effective library yield
0.70	0.49	49% of input fragments
0.80	0.64	64% of input fragments
0.90	0.81	81% of input fragments
0.95	0.90	90% of input fragments

Practical Tip

Ligation efficiency is one reason why the recommended minimum input for many library prep kits is 100–500 ng of DNA. Starting with less material means fewer fragments, and if ligation efficiency is suboptimal, the resulting library may have low complexity. For low-input protocols (<10 ng), specialised kits with optimised ligation chemistries or fragmentation-based approaches (which bypass traditional ligation) are preferred.

3.9 Multiplexing and Barcoding

Modern sequencing instruments generate far more data than is needed for a single sample in many applications. **Multiplexing** (also called **pooling**) is the practice of combining multiple samples on a single sequencing run, using short DNA **barcodes** (also called **indexes** or **index sequences**) to identify which reads came from which sample.

3.9.1 How Indexing Works

During library preparation, each sample’s fragments receive a unique barcode sequence incorporated into the adapter. Illumina uses **dual indexing**: two independent index sequences

(i7 and i5) of 8–10 bp each are incorporated into the adapter, one on each end of the fragment. After sequencing, an additional “index read” determines the barcode sequence, and reads are computationally sorted (**demultiplexed**) by barcode into sample-specific files.

3.9.2 Benefits of Multiplexing

- **Cost efficiency:** The sequencing run cost is shared among multiple samples. For example, if a NovaSeq S4 flow cell generates ~ 10 billion read pairs, and your RNA-seq experiment requires 30 million reads per sample, you can multiplex over 300 samples on a single flow cell.
- **Experimental design:** By balancing experimental conditions across lanes and flow cells, multiplexing reduces batch effects.
- **Flexibility:** Different amounts of data can be allocated to different samples by adjusting the ratio of library input.

Index Hopping

On some Illumina platforms with patterned flow cells (e.g., NovaSeq 6000 with the v1.0 reagent kit), a phenomenon called **index hopping** (or **index switching**) can occur: free adapter molecules from one sample can recombine with clusters from another sample during the exclusion amplification step, leading to incorrect sample assignment of a small percentage of reads (typically 0.1–2%). Using **unique dual indexes (UDIs)**—where both the i7 and i5 indexes are unique to each sample—allows computational detection and removal of hopped reads. This problem has been significantly reduced in newer platforms and reagent kits.

3.10 Quality Scores: Phred Scores and Q30

Every base call produced by a sequencing instrument is accompanied by a **quality score** that estimates the probability that the base call is incorrect.

3.10.1 Phred Quality Scores

Quality scores follow the **Phred** scale, defined as:

$$Q = -10 \cdot \log_{10}(P_{\text{error}}) \quad (3.6)$$

where P_{error} is the estimated probability of an incorrect base call. Higher Q values indicate higher confidence:

Table 3.3: Phred quality scores and corresponding error probabilities.

Phred Score (Q)	Error Probability	Accuracy
10	1 in 10 (10%)	90%
20	1 in 100 (1%)	99%
30	1 in 1,000 (0.1%)	99.9%
40	1 in 10,000 (0.01%)	99.99%
50	1 in 100,000 (0.001%)	99.999%

3.10.2 Q30 as a Benchmark

Q30 (a Phred score of 30, corresponding to 99.9% accuracy) is widely used as a benchmark for sequencing quality. The “percent of bases $\geq$ Q30” is a standard metric reported by Illumina instruments. A typical Illumina run produces >85–90% of bases at Q30 or above.

3.10.3 From Phred to Neural Basecallers

The original Phred program (developed for Sanger sequencing in the late 1990s) estimated base quality by examining features of the electropherogram trace—peak spacing, peak resolution, and signal-to-noise ratio—and mapping these features to empirically calibrated error probabilities. Modern basecallers use fundamentally different approaches.

Neural Network Basecalling

Contemporary basecallers—including Illumina’s **DRAGEN**, ONT’s **Dorado**, and PacBio’s **DeepConsensus**—employ deep neural networks (typically recurrent or transformer architectures) that take raw signal data as input and produce base calls with associated quality scores.

For a neural basecaller, at each position the network outputs a vector of **logits** $\mathbf{z} = (z_A, z_C, z_G, z_T)$, which are transformed into probabilities via the **softmax** function:

$$P(b_i = k) = \frac{e^{z_k}}{\sum_{j \in \{A, C, G, T\}} e^{z_j}}, \quad k \in \{A, C, G, T\} \quad (3.7)$$

The called base is the one with the highest probability: $\hat{b}_i = \arg \max_k P(b_i = k)$. The Phred quality score is then derived from the posterior probability:

$$Q_i = -10 \cdot \log_{10}(1 - P(b_i = \hat{b}_i)) = -10 \cdot \log_{10}(P_{\text{error},i}) \quad (3.8)$$

Quality Score Calibration

Neural basecallers can produce **miscalibrated** quality scores: the predicted error probability may not match the actual observed error rate. For instance, a basecaller might assign Q30 to bases that empirically have Q25 accuracy. Well-calibrated quality scores are essential for downstream tools (e.g., variant callers that use quality scores as priors). Tools like **Dorado** include calibration steps, but users should always verify calibration using known reference samples.

Basecalling Models and Accuracy Trade-offs

Modern nanopore basecallers offer multiple model tiers that trade speed for accuracy:

Table 3.4: Comparison of **Dorado** basecalling model tiers for Oxford Nanopore R10.4.1 chemistry. Speed is approximate and depends on hardware.

Model	Modal accuracy	Speed (samples/s)	Architecture
Fast	Q10–Q14 (~ 90 – 96%)	4×10^6	Small CTC/CRF
HAC (high accuracy)	Q16–Q20 (~ 97 – 99%)	1×10^6	Medium CRF
SUP (super accuracy)	Q20–Q25 (~ 99 – 99.7%)	3×10^5	Large transformer

The “fast” model is suitable for real-time applications (e.g., adaptive sampling), while the “SUP” model maximises accuracy at the cost of requiring a high-end GPU and significantly more compute time.

3.10.4 Quality Score Encoding in FASTQ Files

In **FASTQ** files (the standard output format for sequencing data), quality scores are encoded as single ASCII characters. Each character maps to a Phred score via its ASCII value:

$$Q = \text{ASCII value} - 33 \quad (\text{Phred}+33 \text{ encoding, the current standard})$$

For example, the ASCII character “I” has a value of 73, corresponding to $Q = 73 - 33 = 40$. The character “!” (ASCII 33) corresponds to $Q = 0$.

Practical Tip

When you first look at a FASTQ file, the quality string may look like random characters: `IIIIIIFFFFFFFHHHHH`. These are not random—each character encodes the Phred quality of the corresponding base in the sequence line. Tools like **FastQC** decode and visualise these quality scores for you, which is always the first step in analysing sequencing data.

3.11 The Concept of a Sequencing Library

The term **sequencing library** refers to the complete collection of adapter-ligated DNA or cDNA fragments that are ready to be loaded onto a sequencing instrument. The library is the product of the **library preparation** process and represents the molecular “input” to the sequencing experiment.

3.11.1 What Defines a Library

A sequencing library has several defining properties:

- **Insert size:** The length of the original DNA/cDNA fragment (excluding adapters). For short-read sequencing, typical insert sizes are 200–600 bp. For long-read sequencing, inserts can be 10–100 kb or more.
- **Adapter sequences:** Platform-specific synthetic sequences ligated to fragment ends. These contain priming sites, index sequences, and flow cell binding sequences.
- **Complexity:** The number of unique fragments in the library. A high-complexity library contains many different fragments (good); a low-complexity library contains few unique fragments that are over-amplified (bad—leads to redundant data).
- **Concentration:** The number of molecules per unit volume, typically measured in nanomolar (nM). Accurate quantification is essential for optimal loading onto the sequencing instrument.

3.11.2 Library Complexity and Saturation

Library complexity is a critical concept. If a library has low complexity (few unique fragments), sequencing deeply will produce many identical reads (PCR duplicates) without providing new information. This is analogous to reading the same few books in a library over and over—no matter how many times you read them, you do not learn anything new.

Library complexity is influenced by:

- **Input amount:** Starting with more DNA/RNA generally produces higher-complexity libraries.
- **PCR amplification cycles:** Fewer cycles reduce amplification bias and preserve complexity; more cycles are needed for low-input samples but reduce complexity.

- **Fragment size selection:** Stringent size selection can reduce complexity.

3.12 Amplification Bias and PCR Duplicates

Most library preparation protocols include a **PCR** amplification step to increase the amount of library material. While necessary (especially for low-input samples), PCR amplification introduces two problems.

3.12.1 PCR Duplicates

PCR duplicates are multiple reads that originate from the same original library fragment (i.e., they are copies of the same molecule, amplified during PCR). PCR duplicates do not provide independent evidence about the genome; they are redundant copies and can inflate apparent coverage and bias quantitative measurements.

PCR duplicates are identified computationally by their mapping coordinates: reads that align to the exact same genomic start and end positions are flagged as potential duplicates. Tools such as **Picard MarkDuplicates** and **samtools markdup** perform this task. In a typical WGS experiment at 30× coverage, the duplication rate is usually 5–15%.

3.12.2 Amplification Bias

PCR does not amplify all fragments equally. Fragments with extreme GC content (very high or very low), secondary structures, or certain sequence motifs may be amplified less efficiently, leading to under-representation in the final library. This **amplification bias** causes uneven coverage across the genome.

Strategies to mitigate PCR bias include:

- **PCR-free library preparation:** Protocols that skip the amplification step entirely, starting with sufficient input DNA. PCR-free libraries show more uniform coverage and fewer duplicates.
- **Minimising PCR cycles:** Using the fewest cycles necessary.
- **Unique Molecular Identifiers (UMIs):** Short random sequences (**unique molecular identifier (UMI)**) incorporated into the adapter before PCR. UMIs allow computational identification and removal of PCR duplicates even when two independent fragments happen to have the same mapping coordinates. UMIs are particularly important for quantitative applications (RNA-seq, ATAC-seq, single-cell sequencing).

UMIs: Molecular Barcodes for Deduplication

A **UMI** is a short (typically 8–16 bp) random nucleotide sequence attached to each library molecule before PCR amplification. Because each original molecule receives a unique UMI, reads with the same UMI and the same mapping position can be confidently identified as PCR duplicates, while reads with different UMIs at the same position are confirmed as independent molecules. UMIs are essential for accurate quantification in many modern sequencing applications.

3.13 GC Bias

GC bias refers to the uneven representation of genomic regions as a function of their guanine-cytosine content. In an ideal sequencing experiment, all regions would be represented

proportionally regardless of GC content. In practice, regions with very high or very low GC content are often under-represented.

GC bias has multiple sources:

- **PCR amplification:** High-GC regions form stable secondary structures that impede amplification. AT-rich regions may denature too readily.
- **Library preparation:** Some fragmentation methods and adapter ligation steps are sensitive to GC content.
- **Cluster generation:** On Illumina platforms, bridge amplification efficiency can vary with GC content.
- **Sequencing chemistry:** Polymerase incorporation rates can vary with local sequence context.

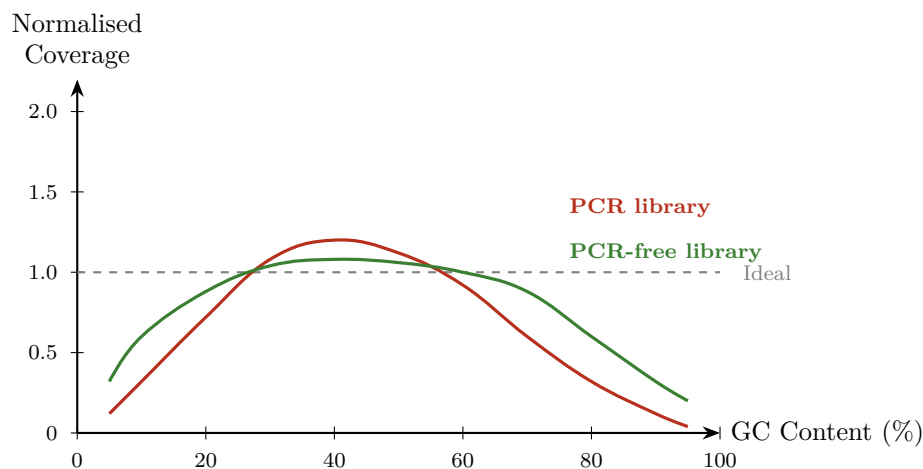


Figure 3.4: Schematic illustration of GC bias. The ideal scenario (dashed grey line) shows uniform coverage across all GC content levels. In practice, libraries prepared with PCR (red) show greater deviation from uniformity than PCR-free libraries (green), with under-representation of both AT-rich and GC-rich regions.

GC bias can be partially corrected computationally during analysis, and PCR-free library preparation significantly reduces (but does not eliminate) the effect (Fig. 3.4).

3.14 Comparison of Major Sequencing Approaches

Choosing a Platform

No single platform is best for all applications. The choice depends on your experimental question:

- **High-throughput quantification** (RNA-seq, ChIP-seq, population-scale WGS): Illumina.
- **Genome assembly, SV detection, haplotype phasing:** PacBio HiFi or ONT.
- **Base modification detection:** ONT (direct) or PacBio (kinetic signatures).
- **Field work and rapid diagnostics:** ONT MinION.
- **Comprehensive analysis:** Many projects now use a combination of short-read and long-read technologies.

Table 3.5: Comparison of the major sequencing technology families as of 2025–2026. Values are approximate and vary by instrument model, chemistry version, and library preparation.

	Illumina (SBS)	PacBio (SMRT)	Oxford Nanopore
Read length	50–300 bp	10–25 kb (HiFi); >100 kb (CLR)	Routinely 10–100 kb; >4 Mb max
Per-base accuracy	>99.9% (Q30+)	>99.9% (HiFi Q30+); ~87% (CLR raw)	~92–98% (raw); >99% (duplex)
Dominant error type	Substitutions	Insertions (CLR); few errors (HiFi)	Indels (especially in homopolymers)
Throughput per run	Up to ~6 Tb (NovaSeq X 25B)	~90 Gb (Revio)	Up to ~50 Gb (PromethION flow cell)
Run time	12–48 hours	12–30 hours	Minutes to 72 hours (user-defined)
Base modifications	Requires chemical conversion (e.g., bisulfite)	Direct detection (kinetics)	Direct detection (current signal)
Instrument cost	\$250K–\$1M+	~\$600K (Revio)	\$1K (MinION) to ~\$300K (PromethION)
Key strengths	Throughput, accuracy, cost per base, ecosystem	HiFi accuracy + length, genome assembly, SV detection	Ultra-long reads, portability, real-time, direct RNA, modifications
Key limitations	Short reads limit repeat resolution and SV detection	Higher cost per base than Illumina	Higher error rate, lower throughput than Illumina

3.15 The FASTQ File Format

The primary output of most sequencing instruments (after base calling) is a set of **FASTQ** files. Each read in a FASTQ file is represented by four lines:

Listing 3.1: Anatomy of a FASTQ record.

```

1 @SEQ_ID read_name and optional description
2 GATTTGGGGTTCAAAGCAGTATCGATCAAATAGTAAATCCATTTGTTCAACTCACAGTTT
3 +
4 !''*(((***+))%%++) (%%%).1***-+*''')**55CCF>>>>>CCCCCCC65

```

- Line 1 (Header):** Begins with @ followed by a sequence identifier and optional description. Contains information such as instrument name, run ID, flow cell ID, tile number, and cluster coordinates.
- Line 2 (Sequence):** The raw nucleotide sequence (A, T, G, C, or N for ambiguous bases).
- Line 3 (Separator):** A + sign (sometimes followed by the same identifier as line 1, but usually just +).
- Line 4 (Quality):** ASCII-encoded Phred quality scores, one character per base, corresponding to the bases in line 2.

For paired-end sequencing, two FASTQ files are produced: one for “read 1” (R1) and one for “read 2” (R2). The files are ordered so that the n th record in R1 and the n th record in R2 correspond to the two ends of the same fragment.

3.16 Summary

This chapter has covered the core principles that underpin all modern sequencing technologies:

- **All sequencing technologies share** a common logic: fragment, attach adapters, and read bases in a massively parallel fashion.
- **Sequencing by synthesis (SBS)**, used by Illumina, reads bases cycle-by-cycle using fluorescently labelled reversible terminators, producing short, highly accurate reads.
- **Sequencing by ligation (SOLiD)** used fluorescent oligonucleotide probes and a ligase; it is now largely of historical interest.
- **Nanopore sequencing** (Oxford Nanopore) reads DNA or RNA directly by threading single molecules through a protein pore and measuring ionic current disruptions, producing ultra-long reads with direct modification detection.
- **SMRT sequencing** (PacBio) watches a single polymerase in a zero-mode waveguide, producing long HiFi reads with high consensus accuracy.
- **Key metrics** for evaluating sequencing include read length, throughput, accuracy, and error profiles, which differ substantially across platforms.
- **Paired-end** sequencing provides two reads from each fragment, improving alignment, assembly, and variant detection.
- **Coverage** (the average number of times each base is read) is a critical parameter for experimental design, calculated as $N \times L/G$.
- **Multiplexing** uses DNA barcodes (indexes) to combine multiple samples on a single sequencing run, reducing costs.
- **Phred quality scores** quantify the confidence of each base call; Q30 (99.9% accuracy) is the standard benchmark.
- A **sequencing library** is the collection of adapter-ligated fragments; its complexity determines the amount of unique information available.
- **PCR duplicates** and **amplification bias** are artefacts of library amplification that can be mitigated by PCR-free protocols and UMIs.
- **GC bias** causes uneven coverage as a function of GC content and is particularly pronounced in PCR-amplified libraries.

With these principles established, the following chapters will dive deeper into specific sequencing platforms (Chapters 4–5), library preparation methods (Chapter 6), and the diverse applications of sequencing across genomics, transcriptomics, and epigenomics.

Part II

Sequencing Platforms and
Technologies

Chapter 4

Short-Read Sequencing Platforms

Short-read sequencing—the generation of millions to billions of relatively short DNA sequence reads, typically 50–600 bases in length—remains the workhorse of modern genomics. Since the commercialisation of the first massively parallel sequencing instruments in the mid-2000s, short-read platforms have driven an extraordinary collapse in the cost of DNA sequencing, from roughly \$100 million per human genome (Sanger-era) to well under \$200 today. This chapter provides a comprehensive survey of the platforms available, beginning with the dominant Illumina ecosystem and extending to the growing number of competitors that have entered the market.

4.1 Illumina: The Dominant Platform

4.1.1 Historical Context and Market Position

Illumina Inc. (San Diego, California) has dominated the short-read sequencing market since approximately 2010, when its HiSeq 2000 platform became the instrument of choice for large genome centres worldwide. The company’s origins trace back to the work of Shankar Balasubramanian and David Klenerman at the University of Cambridge, who developed the sequencing-by-synthesis (SBS) chemistry that underpins all Illumina instruments. Their company, Solexa, was founded in 1998 and acquired by Illumina in 2007 for \$600 million.

By the mid-2010s, Illumina controlled an estimated 80–90% of the global sequencing market by revenue. Although new competitors have emerged, Illumina remains the reference standard against which all other short-read platforms are measured.

Illumina’s Core Innovation

Illumina’s key innovation is the combination of **bridge amplification** on a solid surface (creating clonal clusters of identical DNA molecules) with **reversible-terminator sequencing by synthesis** (incorporating one fluorescently labelled nucleotide per cycle and imaging the entire flow cell). This massively parallel approach enables billions of DNA fragments to be sequenced simultaneously.

4.1.2 Chemistry: Bridge Amplification and Cluster Generation

The Illumina sequencing workflow begins with a prepared DNA library—fragments of double-stranded DNA flanked by platform-specific adapter sequences. The critical steps that occur on the flow cell are:

Step 1: Library denaturation and loading. The double-stranded library is denatured into single strands and loaded onto the flow cell at a carefully controlled

concentration. Too much library leads to overlapping clusters (overclustering); too little wastes capacity.

- Step 2: Hybridisation to lawn oligos.** The flow cell surface is coated with a dense lawn of two types of oligonucleotides (P5 and P7) that are complementary to the adapter sequences on the library fragments. Single-stranded library molecules hybridise to these oligos.
- Step 3: Extension and bridge formation.** A polymerase extends from the surface-bound oligo, creating a double-stranded bridge. The original template is washed away after denaturation, leaving the covalently attached copy.
- Step 4: Bridge amplification.** The newly synthesised strand bends over and hybridises to an adjacent surface oligo of the complementary type, forming a bridge. Polymerase extension creates a double-stranded bridge, which is denatured to yield two surface-bound single strands. This process repeats through approximately 35 cycles of isothermal amplification.
- Step 5: Cluster formation.** Each original library molecule generates a clonal cluster of roughly 1,000 identical copies, tightly localised on the flow cell surface. These clusters are the unit of signal detection during sequencing.
- Step 6: Linearisation.** One strand orientation is selectively cleaved, leaving a lawn of single-stranded molecules all oriented in the same direction, ready for the sequencing primer to hybridise.

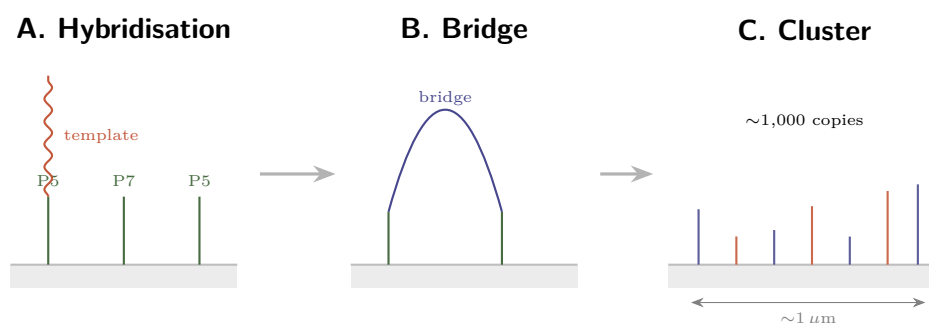


Figure 4.1: Schematic of Illumina bridge amplification. (A) A single-stranded library molecule hybridises to a surface oligo. (B) After extension, the strand bends to form a bridge to an adjacent oligo. (C) Repeated cycles generate a clonal cluster of ~1,000 identical copies.

4.1.3 Sequencing by Synthesis with Reversible Terminators

Once clusters have been generated, the actual sequencing proceeds through iterative cycles of nucleotide incorporation and imaging:

Cycle step 1: Incorporation. All four nucleotides (A, C, G, T) are flowed across the flow cell simultaneously. Each nucleotide carries (i) a fluorescent dye and (ii) a 3'-O-azidomethyl blocking group that prevents incorporation of more than one nucleotide per cycle. The polymerase incorporates exactly one complementary nucleotide at each cluster.

Cycle step 2: Wash. Unincorporated nucleotides are washed away.

Cycle step 3: Imaging. The flow cell is excited with a laser, and a high-resolution camera captures the fluorescent emission from every cluster. Each cluster emits a colour corresponding to the base just incorporated.

Cycle step 4: Cleavage. A chemical cleavage step removes both the fluorescent dye and the 3' blocking group, regenerating a free 3'-OH ready for the next incorporation.

This cycle repeats for every base of the read. A 2×150 bp run therefore requires ~ 300 of these cycles (plus additional cycles for index reads).

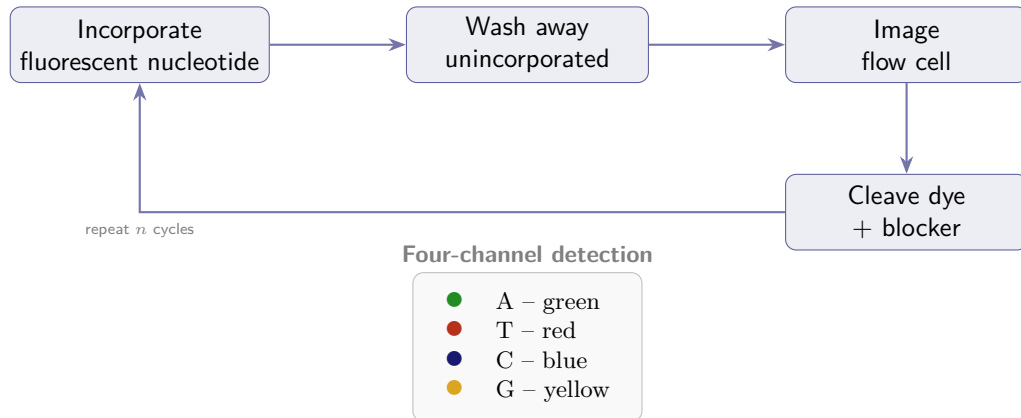


Figure 4.2: The four-step Illumina sequencing-by-synthesis (SBS) cycle. Each cycle adds exactly one base to every cluster, enabling massively parallel read extension.

4.1.4 Two-Channel vs Four-Channel Chemistry

Originally, Illumina instruments used **four-channel chemistry** (4dye/4image), in which each of the four nucleotides carries a spectrally distinct fluorescent dye and each cycle requires four separate images (one per excitation/emission channel). This approach provides unambiguous base identification and is still used on certain instruments such as the NovaSeq 6000 (on some flow cell types) and the older HiSeq series.

To reduce imaging time and cost, Illumina introduced **two-channel chemistry** (2dye/2image), used on the NextSeq, MiniSeq, and NovaSeq X series. In this scheme:

- **A** is labelled with **green dye only** (channel 1 positive, channel 2 negative).
- **C** is labelled with **both green and red dyes** (both channels positive).
- **T** is labelled with **red dye only** (channel 1 negative, channel 2 positive).
- **G** is **unlabelled** (both channels negative; identified by absence of signal).

G-Base Calling in Two-Channel Chemistry

Because G bases are identified by the *absence* of signal, libraries with very high GC content (or low-diversity libraries such as amplicons) can cause problems on two-channel instruments. If most clusters incorporate G at the same cycle, the software cannot distinguish “no signal because G” from “no signal because the cluster is gone.” Low-diversity libraries should be spiked with PhiX (a balanced-composition control library) at 20–50% on two-channel platforms.

4.1.5 Patterned vs Non-Patterned Flow Cells

A second major evolution in Illumina hardware was the introduction of **patterned flow cells**. In traditional non-patterned flow cells (used on MiSeq, HiSeq 2500, and earlier instruments), clusters form at random positions and must be located computationally during the first few cycles. This random seeding limits cluster density because clusters that are too close together merge and become unreadable.

Patterned flow cells, introduced with the HiSeq 3000/4000 and continued in the NovaSeq and NextSeq 1000/2000 series, feature billions of nanowells arranged in a regular grid. Each nanowell captures exactly one library molecule (ideally), producing evenly spaced, uniform clusters. Benefits include:

- **Higher cluster density** (and therefore higher output per flow cell).
- **More uniform cluster size**, improving base-calling accuracy.
- **Simplified imaging**, since cluster locations are known *a priori*.
- **Faster run times**, because fewer imaging frames are needed.

A disadvantage of patterned flow cells is increased sensitivity to index hopping (discussed in Chapter 6), necessitating the use of unique dual indexes (UDIs).

4.1.6 Cluster Density Optimisation

Optimal cluster density is critical for maximising data output while maintaining quality. The relationship between loading concentration, cluster density, and data quality involves a fundamental trade-off:

- **Under-clustering:** Too few clusters per unit area wastes expensive flow cell real estate. The number of reads is proportional to density, so under-loading linearly reduces output.
- **Over-clustering:** Too many library molecules per nanowell (patterned) or too many overlapping clusters (non-patterned) produce mixed signals that fail quality filtering.

For patterned flow cells (NovaSeq, NextSeq 1000/2000), the Poisson statistics of molecular loading govern the fraction of wells with exactly one template. If λ is the average number of molecules per well:

$$P(\text{monoclonal well}) = P(k = 1) = \lambda e^{-\lambda} \quad (4.1)$$

This is maximised at $\lambda = 1$, where $P(k = 1) = e^{-1} \approx 36.8\%$, while $P(k = 0) \approx 36.8\%$ (empty wells) and $P(k \geq 2) \approx 26.4\%$ (polyclonal wells). In practice, manufacturers use exclusion amplification (ExAmp) to increase the effective monoclonal fraction by allowing one template to colonise the well before others, achieving effective occupancy rates of 60–80%.

Cluster Density and Data Quality

Cluster density is specified in units of thousands of clusters per mm^2 (K/ mm^2). Typical target densities are:

- **MiSeq (non-patterned):** 800–1,200 K/ mm^2
- **NovaSeq 6000 S4:** 200–270 K/ mm^2 (per surface, patterned)
- **NovaSeq X Plus 25B:** $\sim 3,500$ K/ mm^2 (patterned)

The percentage of clusters passing filter (%PF) drops sharply when density exceeds the optimal range and is the primary metric for detecting loading problems.

4.1.7 Phasing and Pre-Phasing Error Models

A key source of error in Illumina sequencing is the gradual loss of synchrony among the $\sim 1,000$ copies within each cluster. Over successive SBS cycles, two types of desynchronisation occur:

Phasing (lagging)

A fraction p of molecules in a cluster fail to incorporate a nucleotide in a given cycle and “fall behind” by one base. These lagging molecules will incorporate one cycle late, contributing a shifted signal.

Pre-phasing (leading)

A fraction q of molecules incorporate an extra nucleotide (the 3′ blocking group fails or is removed prematurely) and “run ahead” by one base.

After n cycles, the fraction of molecules still in perfect synchrony with the intended cycle is approximately:

$$f(n) = (1 - p - q)^n \quad (4.2)$$

For typical Illumina phasing ($p \approx 0.1\text{--}0.25\%$) and pre-phasing ($q \approx 0.05\text{--}0.15\%$) rates:

Table 4.1: Fraction of molecules in synchrony as a function of cycle number, assuming $p = 0.20\%$ and $q = 0.10\%$.

Cycle n	In-phase fraction	Quality impact
1	99.7%	Negligible
50	98.5%	Minimal
100	97.0%	Slight degradation
150	95.5%	Noticeable quality drop
200	94.1%	Moderate degradation
300	91.2%	Significant (R2 of 2×150)

This phasing-induced signal decay is the primary reason why quality scores decline toward the end of Illumina reads, and why Read 2 of a paired-end run typically has slightly lower quality than Read 1 (it is sequenced after additional cycles including the index reads). The basecaller software applies a matrix correction at each cycle to computationally deconvolve the phased signal.

4.1.8 Two-Channel Signal Deconvolution

In two-channel chemistry, each cluster produces a two-dimensional signal vector ($I_{\text{green}}, I_{\text{red}}$) at each cycle. The base call is determined by which of the four quadrants the signal falls into:

Table 4.2: Signal intensity patterns for two-channel base calling.

Base	Green (I_1)	Red (I_2)	Classification
A	High	Low	Green-only
C	High	High	Both channels
G	Low	Low	Neither channel
T	Low	High	Red-only

The classification boundary between “high” and “low” is not fixed but is recalibrated at each cycle and each tile using the empirical distribution of intensities. The quality score reflects the distance from the classification boundary: a signal that falls clearly in the A quadrant receives a high Q score, while a signal near the boundary between A and G receives a lower Q score.

Because G bases produce no signal, the basecaller must distinguish G from cluster loss, low signal due to phasing, or optical noise. This is accomplished by tracking each cluster’s signal trajectory across cycles—a cluster that consistently produces signal at most cycles but is dark at the current cycle is called G with high confidence.

4.1.9 Index Hopping: Rates and Mitigation

Index hopping (index switching) occurs when free adapter molecules in the pooled library recombine with other library molecules during the ExAmp on-instrument cluster generation. The rate depends on the platform:

Table 4.3: Index hopping rates by platform and mitigation strategy.

Platform	Typical rate	Mitigation
MiSeq (bridge amp)	<0.01%	Not a concern
HiSeq 2500 (bridge amp)	<0.01%	Not a concern
NovaSeq 6000 (ExAmp)	0.1–2.0%	Use UDIs; filter hopped reads
NovaSeq X (ExAmp v2)	0.01–0.1%	Greatly reduced; UDIs recommended
NextSeq 1000/2000	<0.01%	Bridge amplification; minimal

Unique Dual Indexes (UDIs) solve the index hopping problem by assigning each sample a unique combination of i7 and i5 indexes. With combinatorial dual indexing (CDI), where i7 and i5 indexes are reused across samples in different combinations, a hopped read may be misassigned to the wrong sample. With UDIs, a hopped read will carry an i7 from one sample and an i5 from another—a combination not present in any legitimate sample—and can be computationally discarded.

Important Caveat

For any experiment on a patterned flow cell where sample cross-contamination is unacceptable (e.g., clinical diagnostics, rare variant detection, single-cell experiments), always use UDIs. Combinatorial indexing is only acceptable on platforms that use bridge amplification (MiSeq, NextSeq 1000/2000).

4.1.10 Platform Hierarchy

Illumina offers a portfolio of instruments spanning a wide range of throughput and cost. The major current-generation platforms (as of early 2026) are:

MiSeq and MiSeq™ Dx

The MiSeq is Illumina’s smallest benchtop sequencer, designed for targeted and small-scale experiments. It uses non-patterned flow cells and four-channel chemistry.

- **Read lengths:** up to 2×300 bp (the longest paired-end reads in the Illumina ecosystem).
- **Output:** 0.3–15 Gb per run (depending on the kit version: Nano, Micro, v2, v3).
- **Reads per run:** 1–25 million clusters (single-read equivalents).
- **Run time:** 5–55 hours.
- **Error rate:** < 0.1% per base (substitution-dominated).
- **Typical applications:** 16S/ITS amplicon sequencing, small targeted panels, microbial genomes, validation studies, HLA typing.

The MiSeq Dx is the FDA-cleared clinical version, used in regulated diagnostic laboratories.

NextSeq 1000 and NextSeq 2000

The NextSeq 1000/2000 are mid-throughput benchtop instruments using patterned flow cells and two-channel chemistry. They replaced the NextSeq 500/550 series.

- **Read lengths:** up to 2×150 bp (NextSeq 2000 supports 2×300 bp with the P1 300-cycle kit).
- **Output:** 30–360 Gb per flow cell (P1, P2, P3 cartridges of increasing capacity).

- **Reads per run:** 100 million to 1.2 billion clusters.
- **Run time:** 11–48 hours.
- **DRAGEN on-instrument:** built-in FPGA-accelerated secondary analysis (alignment, variant calling).
- **Typical applications:** gene panels, exomes, small genomes, single-cell gene expression (10x Genomics), RNA sequencing (RNA-seq), Assay for Transposase-Accessible Chromatin using sequencing (ATAC-seq), low-pass whole-genome sequencing (WGS).

NovaSeq X and NovaSeq X Plus

The NovaSeq X series, launched in 2023 as successors to the NovaSeq 6000, are Illumina’s highest-throughput instruments, designed for production-scale sequencing.

- **Read lengths:** up to 2×150 bp.
- **Output:** 1.3 Tb (10B flow cell) to 16 Tb (25B flow cell, NovaSeq X Plus with two flow cells running simultaneously).
- **Reads per run:** up to 25 billion clusters per flow cell (25B).
- **Run time:** approximately 13 hours (10B) to 48 hours (25B 2×150 bp).
- **Cost:** approximately \$2–4 per Gb; roughly \$200 for a $30\times$ human genome.
- **Flow cell types:** 1.5B, 10B, and 25B (referring to billions of nanowells).
- **DRAGEN on-instrument:** full secondary analysis pipeline.
- **Typical applications:** large-scale population WGS, clinical whole-genome/exome sequencing, large RNA-seq projects, single-cell atlases, whole-genome bisulfite sequencing (WGBS).

Choosing an Illumina Flow Cell

The 25B flow cell is only cost-effective when you have enough libraries to fill it. Running a single WGS sample on a 25B flow cell wastes capacity. Many facilities use the 10B flow cell for routine work and the 25B for large population-scale projects or when many samples are multiplexed together.

4.1.11 Illumina Error Profile

Illumina platforms produce predominantly **substitution errors**, with a raw per-base error rate of approximately 0.1–0.5%. The most common systematic errors include:

- **GGC motif errors:** elevated substitution rates in GGC trinucleotide contexts.
- **Quality score drop at read ends:** accumulation of incomplete cleavage (phasing) and leading/lagging strand effects.
- **Index hopping:** on patterned flow cells with exclusion amplification (ExAmp), free adapters can re-hybridise to the wrong nanowell during cluster generation, causing barcode misassignment.

Insertions and deletions (indels) are rare on Illumina, making the platform well-suited for single nucleotide variant (SNV) detection but potentially less reliable for homopolymer-length estimation compared to long-read technologies.

4.2 MGI / BGI DNBSEQ

MGI Tech (a subsidiary of the BGI Group, Shenzhen, China) has developed a distinct sequencing chemistry and is Illumina’s most significant competitor by installed base, particularly in China and parts of Asia and Europe.

4.2.1 DNA Nanoball (DNB) Technology

Rather than bridge amplification, MGI uses **rolling circle amplification (RCA)** to generate signal:

1. Library molecules are circularised by ligation of a splint oligonucleotide.
2. A polymerase with strand-displacement activity performs rolling circle amplification, producing a long concatemer of 200–400 copies of the original molecule, which collapses into a compact **DNA nanoball** (DNB) roughly 200 nm in diameter.
3. DNBs are loaded onto a patterned silicon chip with billions of positively charged spots arranged in a regular grid. Each spot captures a single DNB by electrostatic attraction.

DNA Nanoballs vs Clusters

Unlike Illumina clusters, which are formed *on* the flow cell, DNBs are formed *before* loading. This means DNB formation and chip loading are decoupled, offering independent quality-control checkpoints. DNBs are also inherently clonal (derived from a single circular template), virtually eliminating index hopping.

4.2.2 Combinatorial Probe-Anchor Synthesis (cPAS)

MGI's sequencing chemistry, called **cPAS** (Combinatorial Probe-Anchor Synthesis), differs from Illumina's **sequencing by synthesis (SBS)**:

1. An anchor probe hybridises to a known adapter sequence adjacent to the DNA to be sequenced.
2. A pool of fluorescently labelled sequencing probes (each representing a specific base) competes for hybridisation to the target position.
3. After imaging, the anchor–probe complex is stripped, and a new anchor shifted by one base is applied.

The net result is similar to **SBS**: one base is read per cycle per DNB, and the error profile is dominated by substitutions.

4.2.3 DNBSEQ Platforms

DNBSEQ-G400 (MGISEQ-2000)

A mid-throughput benchtop instrument. Up to 2×200 bp reads; 60–720 Gb per run; two independent flow cell positions. Competes with the Illumina NextSeq.

DNBSEQ-T7

A high-throughput production sequencer. Up to 2×150 bp; up to 6 Tb per run with four flow cells; approximately 24-hour run time. Competes with the NovaSeq 6000/X.

DNBSEQ-T20×2

The latest ultra-high-throughput platform (launched 2024–2025). Two independent sequencing modules, each running two flow cells. Combined output exceeding 10 Tb per run. Aimed at population-scale whole-genome projects.

DNBSEQ-G99

A compact, low-throughput benchtop system designed for clinical panels and targeted sequencing. Competes with the MiSeq.

Intellectual Property Considerations

MGI and Illumina have been involved in extensive patent litigation. Before purchasing MGI instruments, verify the current intellectual property landscape in your jurisdiction. In some markets, licensing agreements are in place; in others, restrictions may apply.

4.3 Element Biosciences AVITI

Element Biosciences (San Diego, California) entered the market in 2022 with the AVITI system, offering what the company terms **avidity sequencing**.

4.3.1 Avidity Sequencing Chemistry

The AVITI chemistry is based on the following key principles:

1. **Rolling circle amplification on the flow cell.** Like MGI, Element uses rolling circle amplification rather than bridge amplification. Library molecules are circularised and then amplified directly on the flow cell surface, producing compact polony (polymerase colony) structures.
2. **Avidity base calling.** Rather than labelling individual nucleotides, Element uses multivalent binding of labelled polymer–nucleotide conjugates. The avidity (multi-point binding) of the polymer to the polony provides high signal-to-noise with natural, unmodified nucleotides used for actual incorporation.
3. **Non-destructive chemistry.** The labelling polymer is removed after imaging without altering the DNA, potentially reducing cycle-to-cycle artefacts.

4.3.2 AVITI Specifications

- **Read lengths:** 2×75 bp to 2×300 bp.
- **Output:** up to 300 Gb per flow cell; the instrument runs two flow cells independently, yielding up to 600 Gb per run.
- **Reads:** up to 1 billion reads per flow cell.
- **Run time:** 16–48 hours depending on read length.
- **Error rate:** < 0.5% raw error; primarily substitutions.
- **Cost:** approximately \$1.50–2.50 per Gb, positioning it competitively with the NextSeq 2000 and lower NovaSeq X tiers.
- **Key differentiator:** Flexible, independent flow cell operation; low instrument cost (approximately \$289,000 list price at launch); open library-prep compatibility.

AVITI's Value Proposition

Element positions the AVITI as a “democratisation” platform—an instrument with high-end data quality at a lower capital cost than a NovaSeq, with the flexibility to run different experiments on its two independent flow cells simultaneously.

4.4 Ultima Genomics

Ultima Genomics (Newark, California) made headlines in 2022 when it announced a target cost of \$1 per Gb—later refined to approximately \$1 per genome at scale.

4.4.1 UG 100 Platform

The Ultima sequencing approach differs substantially from other short-read platforms:

- **Natural nucleotide chemistry.** Instead of reversible terminators, the UG 100 uses *natural* (unmodified) nucleotides. Each sequencing cycle flows a single nucleotide species across the flow cell. The polymerase incorporates as many of that nucleotide as the template requires, and the signal from a reporting chemistry indicates *how many* bases were incorporated.
- **Open, circular flow cell.** The flow cell is a large, spinning silicon wafer. Library amplification is performed off-instrument, and DNA polonies are deposited on the wafer surface.
- **High density, single-colour detection.** The simplified chemistry enables very high feature density and reduced imaging requirements.

4.4.2 Specifications and Considerations

- **Read length:** approximately 300 bp single-end reads.
- **Output:** up to 20 Tb per run.
- **Error profile:** dominated by **homopolymer-length errors** (systematic miscounting of consecutive identical bases), unlike the substitution errors of Illumina and MGI. This is a direct consequence of the single-nucleotide flow chemistry.
- **Applications:** the platform is particularly suited for applications that are tolerant of single-end reads and where cost per base is paramount, such as population-scale WGS and large association studies.

Homopolymer Errors on Ultima

Because the UG 100 uses natural nucleotides in a flow-based scheme, errors in homopolymer runs (e.g., AAAA...) are the dominant error mode. Bioinformatics pipelines designed for Illumina substitution errors may need adjustment. Ultima provides custom variant-calling tools to handle this error profile.

4.5 Singular Genomics G4

Singular Genomics (San Diego, California) developed the G4 platform, a benchtop sequencer designed for mid-throughput applications. The instrument uses four independent flow cells, each of which can be started and stopped independently, providing high flexibility for laboratories running diverse projects.

4.5.1 Chemistry and Specifications

- **Chemistry:** sequencing by synthesis with fluorescent reversible terminators (conceptually similar to Illumina's approach).
- **Read lengths:** up to 2×150 bp.
- **Output:** up to 120 Gb per flow cell; 480 Gb with four flow cells.
- **Reads:** up to 400 million reads per flow cell.
- **Run time:** approximately 8–22 hours per flow cell.
- **Key differentiator:** four independent flow cells on a single instrument, allowing mixed experiment types and staggered starts.

Singular Genomics Status

Singular Genomics filed for bankruptcy in late 2023 and ceased operations. The G4 is included here for completeness, as some instruments remain in the field, and the technology illustrates alternative design choices. However, no ongoing support, consumable supply, or development is expected.

4.6 Comprehensive Platform Comparison

Table 4.4 provides a side-by-side comparison of the major short-read sequencing platforms available as of early 2026.

Table 4.4: Comparison of current short-read sequencing platforms. Values are approximate and may vary with kit versions, software updates, and run configurations.

Platform	Vendor	Max Read Length	Output/Run	Cost/Gb	Run Time
MiSeq v3	Illumina	2 × 300 bp	15 Gb	\$100–200	5–55 h
NextSeq 2000 (P3)	Illumina	2 × 150 bp	360 Gb	\$15–30	11–48 h
NovaSeq X (10B)	Illumina	2 × 150 bp	1.3 Tb	\$2–4	13–24 h
NovaSeq X Plus (25B)	Illumina	2 × 150 bp	16 Tb	\$2–3	24–48 h
DNBSEQ-G99	MGI/BGI	2 × 150 bp	15 Gb	\$60–120	12–24 h
DNBSEQ-G400	MGI/BGI	2 × 200 bp	720 Gb	\$8–15	24–48 h
DNBSEQ-T7	MGI/BGI	2 × 150 bp	6 Tb	\$3–5	24 h
DNBSEQ-T20×2	MGI/BGI	2 × 150 bp	10+ Tb	\$2–3	24 h
AVITI	Element Bio	2 × 300 bp	600 Gb	\$1.50–2.50	16–48 h
UG 100	Ultima	~300 bp SE	20 Tb	\$1–2	20 h
G4*	Singular	2 × 150 bp	480 Gb	\$10–20	8–22 h

*Singular Genomics ceased operations in late 2023. Included for reference.

4.7 How to Choose a Short-Read Platform

Selecting the right short-read platform involves balancing multiple factors. The following decision framework can guide your choice:

4.7.1 Throughput Requirements

- **Low throughput** (< 20 Gb): MiSeq, DNBSEQ-G99. Ideal for amplicons, small panels, microbial genomes.
- **Mid throughput** (20–600 Gb): NextSeq 2000, DNBSEQ-G400, AVITI. Ideal for exomes, targeted panels, [RNA-seq](#), single-cell.
- **High throughput** (> 600 Gb): NovaSeq X, DNBSEQ-T7/T20, Ultima UG 100. Ideal for [WGS](#), population studies, large cohorts.

4.7.2 Read Length Requirements

- **Short reads** (2×50 – 100 bp): sufficient for most RNA-seq, chromatin immunoprecipitation sequencing (ChIP-seq), ATAC-seq.
- **Medium reads** (2×150 bp): the standard for WGS and exome sequencing.
- **Long short-reads** (2×250 – 300 bp): essential for amplicon sequencing (16S, ITS) where forward and reverse reads must overlap to form a contiguous amplicon. MiSeq and AVITI offer the longest paired-end reads.

4.7.3 Budget and Capital Expenditure

- **Instrument cost:** ranges from $\sim \$100$ K (MiSeq) to $\sim \$1$ M+ (NovaSeq X Plus). The AVITI has a competitive list price of $\sim \$289$ K.
- **Cost per Gb:** inversely correlated with throughput. High-throughput instruments are cheapest per Gb but require high sample volumes to be cost-effective.
- **Service contracts and consumables:** ongoing costs can equal or exceed the instrument cost over 3–5 years.

4.7.4 Data Ecosystem and Compatibility

- **Software compatibility:** most bioinformatics tools were developed and validated on Illumina data. Newer platforms may require modified parameters or dedicated tools.
- **Library prep compatibility:** Illumina has the broadest library preparation ecosystem. MGI and Element require platform-specific adapters or adapter conversion steps.
- **Regulatory status:** for clinical applications, consider whether the platform has IVD (in vitro diagnostic) clearance in your jurisdiction.

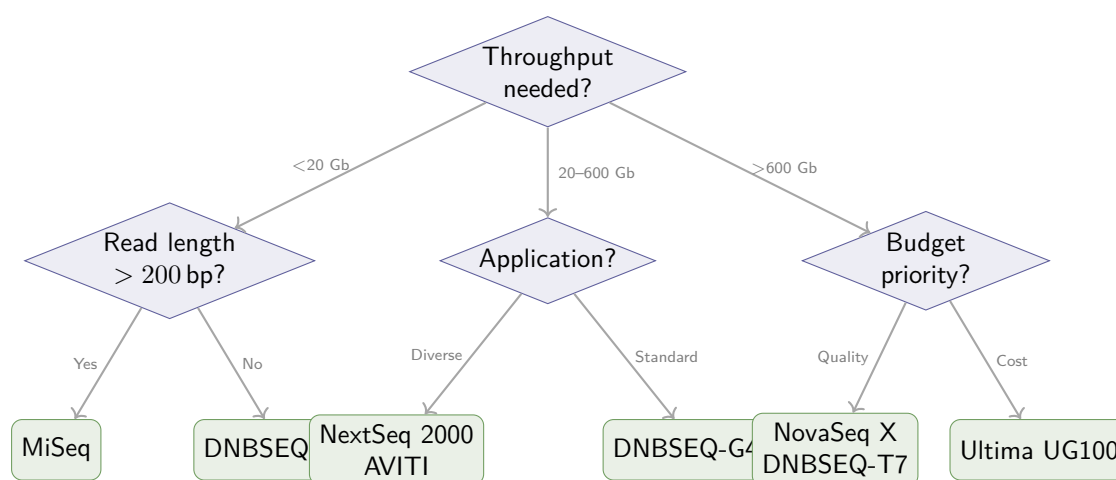


Figure 4.3: Simplified decision flowchart for selecting a short-read sequencing platform. Real decisions involve additional factors including existing infrastructure, bioinformatics support, and vendor relationships.

4.7.5 Practical Considerations

Platform Selection Checklist

Before committing to a platform, evaluate:

1. What is my sample throughput per month (number of samples $\times$ required coverage)?
2. Do I need paired-end reads, and what length?
3. What is my budget for capital equipment vs. consumables?
4. What bioinformatics tools and pipelines does my team use, and are they validated for this platform's data?
5. Do I need clinical/regulatory certification?
6. Is there local service and support from the vendor?
7. Can I share instrument time with collaborators to improve utilisation?

4.8 Performance Metrics and Quality Considerations

When evaluating short-read platform performance, several key metrics are commonly reported:

Q30 (Phred score ≥ 30)

The percentage of bases called with at least 99.9% accuracy. Modern Illumina instruments routinely exceed 85–90% bases $\geq$ Q30 for 2×150 bp reads.

Cluster passing filter (%PF)

The fraction of clusters that pass quality filters during the first 25 cycles. Values below 70–80% indicate library or loading problems.

Error rate by cycle

A cycle-by-cycle plot of mismatch rate against a known reference (often PhiX). Useful for diagnosing phasing problems or chemistry issues.

Duplication rate

The fraction of reads that are PCR or optical duplicates. High duplication reduces the effective unique coverage and is influenced by library complexity and loading concentration.

Insert size distribution

For paired-end libraries, the distribution of distances between the two reads of a pair. A tight, symmetric distribution centred on the expected fragment size indicates good library quality.

PhiX Spike-In

Always include a small fraction (1–5%) of PhiX control library in your sequencing run. PhiX provides a balanced-composition, known-sequence control that enables:

- Calibration of colour balance and focus.
- An independent estimate of the per-base error rate.
- Diversity boosting for low-complexity libraries (increase to 20–50% for amplicons on two-channel systems).

4.9 The Future of Short-Read Sequencing

Several trends are shaping the near-term future of short-read sequencing:

- **Continued cost reduction.** The \$100 genome is now achievable in research settings, and further cost decreases will enable routine clinical WGS at population scale.
- **Integration of on-instrument analysis.** Illumina’s DRAGEN technology and equivalents from other vendors are moving secondary analysis (alignment, variant calling) directly onto the instrument, reducing turnaround time and bioinformatics infrastructure requirements.
- **Increased competition.** The entry of Element, Ultima, and strengthening of MGI provides laboratories with genuine alternatives, driving innovation and price competition.
- **Longer short reads.** Platforms are pushing toward longer reads (2×300 bp and beyond) to improve *de novo* assembly and amplicon analysis while retaining the high accuracy of short-read chemistry.
- **Clinical adoption.** Regulatory clearance for sequencing platforms and companion diagnostic assays will continue to expand, making short-read sequencing increasingly mainstream in clinical care.

4.10 Summary

Short-read sequencing platforms provide the throughput, accuracy, and cost-effectiveness needed for the vast majority of genomics applications. Illumina remains the dominant player, but MGI, Element Biosciences, and Ultima Genomics offer compelling alternatives with distinct technical approaches. When selecting a platform, consider your throughput needs, read length requirements, budget, and bioinformatics compatibility. The field continues to evolve rapidly, with competition driving costs down and capabilities up.

Chapter 5

Long-Read Sequencing Platforms

While short-read sequencing excels at generating massive volumes of highly accurate data, it has an inherent limitation: individual reads of 100–300 bases cannot span many of the repetitive elements, structural variants, and complex genomic regions that are biologically and clinically important. Long-read sequencing technologies—generating reads from thousands to millions of bases—address these limitations and have opened entirely new areas of genomics research. This chapter provides a comprehensive survey of long-read sequencing platforms, focusing on the two dominant technologies: Pacific Biosciences (PacBio) Single Molecule, Real-Time (SMRT) sequencing and Oxford Nanopore Technologies (ONT) nanopore sequencing.

5.1 Why Long Reads Matter

Before diving into platform specifics, it is worth understanding the biological questions that fundamentally require long reads.

The Long-Read Advantage

Long reads provide three capabilities that short reads cannot:

1. **Spanning repetitive regions.** Roughly 50% of the human genome consists of repetitive elements. Short reads cannot be unambiguously placed within repeats longer than the read length. Long reads that span entire repeats resolve their location.
2. **Detecting structural variants.** Insertions, deletions, inversions, and translocations larger than a few hundred bases are poorly detected by short reads. Long reads capture both breakpoints of a structural variant in a single read.
3. **Phasing haplotypes.** Long reads can link distant variants on the same chromosome, enabling haplotype-resolved (phased) assembly without the need for family data.

Specific application areas where long reads are essential or strongly preferred include:

- **De novo genome assembly.** Long reads produce contiguous assemblies (high N50 values) that can span centromeres, telomeres, and segmental duplications. The Telomere-to-Telomere (T2T) Consortium used long reads to complete the first truly gapless human reference genome.
- **Structural variant detection.** Clinical and population genetics increasingly recognise that **structural variants (SVs)** (variants > 50 bp) account for more variable bases per genome than **SNVs**. Long reads detect SVs with far greater sensitivity and precision.

- **Full-length transcript isoform sequencing.** Long reads can capture entire mRNA molecules (including all exons and UTRs) in a single read, eliminating the need for computational transcript assembly from short-read fragments.
- **Direct base modification detection.** Both PacBio and ONT can detect DNA methylation and other base modifications directly from the sequencing signal, without bisulfite conversion or other chemical treatments.
- **Repetitive region characterisation.** Tandem repeats, microsatellites, and mobile element insertions are resolved by reads long enough to span the entire repeat and anchor into flanking unique sequence.
- **Metagenomics.** Long reads can span entire operons or even full microbial chromosomes, improving species-level classification and enabling the assembly of individual genomes from complex communities.

5.2 Pacific Biosciences (PacBio)

5.2.1 Historical Context

Pacific Biosciences (Menlo Park, California) commercialised single-molecule real-time (**single-molecule real-time sequencing (SMRT sequencing)**) sequencing in 2011 with the RS system. The technology has undergone continuous improvement through the RS II, Sequel, Sequel II, Sequel IIe, Revio, and most recently the Vega system. PacBio was acquired by and subsequently merged operations with the former Illumina competitor Omniome, whose sequencing-by-binding (SBB) chemistry now forms the basis of PacBio's short-read instrument, the Onso.

5.2.2 SMRT Sequencing Principle

The fundamental innovation of PacBio is the ability to observe a single DNA polymerase molecule synthesising a complementary strand in **real time**.

Zero-Mode Waveguides (ZMWs)

The heart of the PacBio system is the **SMRT Cell**, a chip containing millions of tiny cylindrical holes called **zero-mode waveguides (ZMWs)**. Each ZMW is approximately 70–100 nm in diameter and 100 nm deep, fabricated in a thin aluminium film deposited on a glass substrate.

The ZMW is so small that its diameter is below the wavelength of the excitation light (~500–600 nm). This means that while laser light can enter the well, it decays exponentially—only the bottom 20–30 nm of the well is illuminated. A single polymerase molecule is immobilised at the bottom of each ZMW, and only the nucleotide being incorporated at the active site dwells long enough to produce a detectable fluorescent pulse.

Evanescent Wave Physics

The operating principle of the ZMW relies on **evanescent wave** behaviour. When light encounters an aperture smaller than its wavelength, it cannot propagate through as a freely travelling wave. Instead, the electromagnetic field decays exponentially with distance from the entrance:

$$I(z) = I_0 e^{-z/d} \tag{5.1}$$

where $I(z)$ is the light intensity at depth z into the waveguide, I_0 is the intensity at the entrance, and d is the **penetration depth**. For a ZMW of diameter a illuminated at wavelength λ :

$$d \approx \frac{a}{2} \cdot \frac{1}{\sqrt{(2.405 \cdot \lambda / (\pi a))^2 - 1}} \quad (5.2)$$

For typical ZMW dimensions ($a \approx 70$ – 100 nm, $\lambda \approx 532$ nm), the penetration depth $d \approx 20$ – 30 nm. This creates an observation volume of approximately 10^{-21} litres (one zeptolitre)—six orders of magnitude smaller than the diffraction limit of conventional optics.

Why the ZMW Works

At micromolar concentrations of fluorescently labelled nucleotides (necessary for polymerase activity), a confocal microscope would observe hundreds of molecules simultaneously, creating overwhelming background fluorescence. The ZMW's zeptolitre observation volume reduces this to an average of ~ 0.01 freely diffusing molecules in the illuminated zone. Only the nucleotide being incorporated by the polymerase dwells long enough (~ 10 – 100 ms) to produce a detectable pulse above the shot noise of rapidly transiting molecules (~ 10 μ s dwell time).

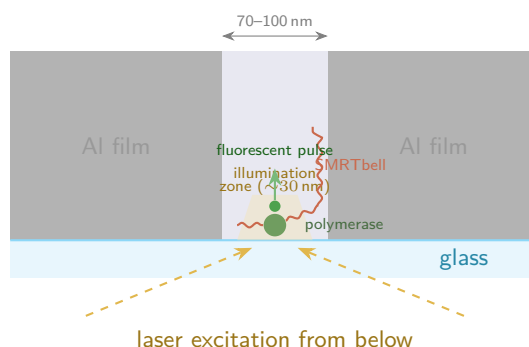


Figure 5.1: Cross-section of a PacBio zero-mode waveguide (ZMW). A single polymerase molecule is immobilised at the bottom of a nanophotonic well. Laser light illuminates only the bottom ~ 30 nm, so only the nucleotide being incorporated produces a detectable fluorescent signal.

The SMRTbell Template

PacBio library molecules are prepared as **SMRTbell** constructs: a double-stranded DNA insert flanked by hairpin adapters on both ends, creating a topologically circular (dumbbell-shaped) molecule. The polymerase initiates synthesis at a sequencing primer annealed in the adapter and proceeds around the circular template continuously.

5.2.3 Continuous Long Reads (CLR) vs HiFi Reads (CCS)

PacBio generates two fundamentally different read types from the same instrument:

Continuous Long Reads (CLR)

In CLR mode, the polymerase reads around the SMRTbell template as many times as it can before it dissociates or the movie time expires. If the insert is long (e.g., 15–20 kb or more),

the polymerase may traverse the insert only once or a few times, yielding a single long but relatively error-prone read. CLR reads can exceed 100 kb, with some reaching beyond 300 kb.

- **Accuracy:** approximately 85–90% per-pass (Q10–Q13).
- **Error type:** predominantly insertions and deletions (indels), particularly in homopolymer contexts. Substitution errors are comparatively rare.
- **Strengths:** maximum read length; essential for spanning very large repeats and producing the most contiguous genome assemblies.

HiFi Reads (Circular Consensus Sequencing, CCS)

If the insert is shorter (typically 10–25 kb), the polymerase traverses the circular template multiple times, generating multiple passes (subreads) of the same molecule. These subreads are computationally aligned and a **consensus** sequence is generated, dramatically reducing the error rate.

- **Passes required:** typically 3–10+ passes for high-quality consensus.
- **Accuracy:** >Q30 (99.9%) median per-read accuracy; the best reads exceed Q40.
- **Read length:** typically 10–25 kb (limited by the requirement for multiple passes within the movie time).
- **Error profile:** residual errors are approximately evenly distributed among substitutions, insertions, and deletions, with no strong systematic biases.

HiFi: The Best of Both Worlds

PacBio HiFi reads combine the advantages of long reads (10–25 kb) with accuracy rivalling or exceeding that of short reads (>Q30). This combination has made HiFi the method of choice for reference-quality genome assembly, clinical structural variant detection, and haplotype-resolved (phased) analysis.

CCS Accuracy Model

The accuracy improvement from circular consensus sequencing can be modelled quantitatively. If the per-pass (single-subread) error rate is ε and a position is covered by N independent passes, the consensus error rate ε_{CCS} at that position follows a binomial model. For a simple majority-vote consensus, an error occurs only when more than half of the passes contain the same error. The probability of the consensus being wrong is approximately:

$$\varepsilon_{\text{CCS}} \approx \sum_{k=\lceil N/2 \rceil}^N \binom{N}{k} \varepsilon^k (1 - \varepsilon)^{N-k} \quad (5.3)$$

For small ε and moderate N , this simplifies to:

$$\varepsilon_{\text{CCS}} \approx \binom{N}{\lceil N/2 \rceil} \varepsilon^{\lceil N/2 \rceil} \quad (5.4)$$

In practice, PacBio’s **ccs** tool uses a more sophisticated approach: it builds a partial-order alignment of all subreads and calls the consensus using a pair-HMM, which accounts for the indel-dominated error profile. The resulting accuracy is:

5.2.4 PacBio Platforms

Revio System

The Revio, launched in late 2022 and widely deployed from 2023 onward, is PacBio’s current flagship production instrument.

Table 5.1: Expected CCS consensus quality as a function of the number of full passes, assuming $\sim 90\%$ single-pass accuracy.

Full passes	Consensus quality	Phred equivalent
1	$\sim 90\%$	Q10
3	$\sim 99\%$	Q20
5	$\sim 99.8\%$	Q27
7	$\sim 99.95\%$	Q33
10	$> 99.99\%$	$> Q40$

- **SMRT Cells:** each cell contains approximately 25 million ZMWs.
- **Throughput:** approximately 90 Gb of HiFi reads per SMRT Cell; with four SMRT Cell positions running in parallel, up to 360 Gb per day.
- **Movie time:** 24 hours per SMRT Cell (standard).
- **Cost:** approximately \$800–1,000 per SMRT Cell; approximately \$1,000–1,500 for a $30\times$ human WGS (HiFi).
- **On-instrument analysis:** HiFi read generation (CCS calling) occurs on-instrument in real time using GPUs.
- **Key improvement over Sequel IIe:** approximately 8-fold increase in throughput per instrument per day, primarily through the 25M ZMW SMRT Cell (vs. 8M on Sequel IIe) and reduced run turnaround.

Vega System

The Vega, announced in 2024 and entering the market in 2025, is PacBio’s next-generation benchtop system.

- **Positioning:** designed for laboratories seeking HiFi data at lower capital cost and smaller scale.
- **Throughput:** lower than Revio per run but sufficient for individual genome projects, targeted long-read panels, and isoform sequencing.
- **SMRT Cell format:** uses a new, smaller SMRT Cell format optimised for lower-throughput applications.
- **Workflow simplification:** streamlined library preparation and reduced hands-on time.

Onso (Short-Read Platform)

PacBio also offers the Onso, a short-read sequencer based on **Sequencing by Binding (SBB)** chemistry (acquired from Omniome). In SBB, the identity of the next nucleotide is determined by a binding event (without incorporation), followed by separate incorporation of a natural nucleotide. This decoupling of recognition from extension is claimed to reduce context-dependent errors. The Onso competes with Illumina’s mid-throughput platforms but is not a long-read instrument and is mentioned here only for completeness.

5.2.5 Kinetic Information and Base Modification Detection

A unique advantage of PacBio SMRT sequencing is that the polymerase’s kinetics—specifically, the **inter-pulse duration** (IPD, the time between fluorescent pulses) and the **pulse width**—are sensitive to chemical modifications on the template DNA. Modified bases (such as 5-methylcytosine, N6-methyladenine, and others) alter the polymerase’s behaviour as it traverses them, producing characteristic kinetic signatures.

- **5mC detection:** PacBio HiFi reads can detect 5-methylcytosine (5mC) at CpG sites with high accuracy, using machine learning models trained on kinetic features. This

enables simultaneous genome sequencing and methylation profiling without bisulfite conversion.

- **m6A detection:** N6-methyladenine produces a strong kinetic signature and is readily detected even from single-pass CLR data.
- **Other modifications:** various other modifications, including damaged bases, can be detected from kinetic signatures.

PacBio Kinetics for Epigenomics

If you need both genome sequence and CpG methylation calls from the same experiment, PacBio HiFi is an excellent choice. A single 30× HiFi whole-genome dataset can provide variant calls, structural variant calls, phased haplotypes, and genome-wide CpG methylation—all without any additional library preparation or chemical treatment.

5.3 Oxford Nanopore Technologies (ONT)

5.3.1 Historical Context

Oxford Nanopore Technologies (Oxford, United Kingdom) was founded in 2005 as a spin-out from the University of Oxford, based on the pioneering work of Hagan Bayley on protein nanopores. The company's first commercial product, the MinION, was released through an early-access programme (MAP) in 2014. ONT has since expanded its product line to include instruments ranging from the pocket-sized Flongle adapter to the production-scale PromethION, and it remains the only company offering portable, real-time, long-read DNA and RNA sequencing.

5.3.2 Nanopore Sequencing Principle

The fundamental principle of nanopore sequencing is elegantly simple: a single strand of DNA or RNA is driven through a tiny protein pore embedded in an electrically resistant membrane. As each nucleotide passes through the narrowest constriction of the pore, it partially blocks the ionic current flowing through the pore. The pattern of current disruption encodes the sequence.

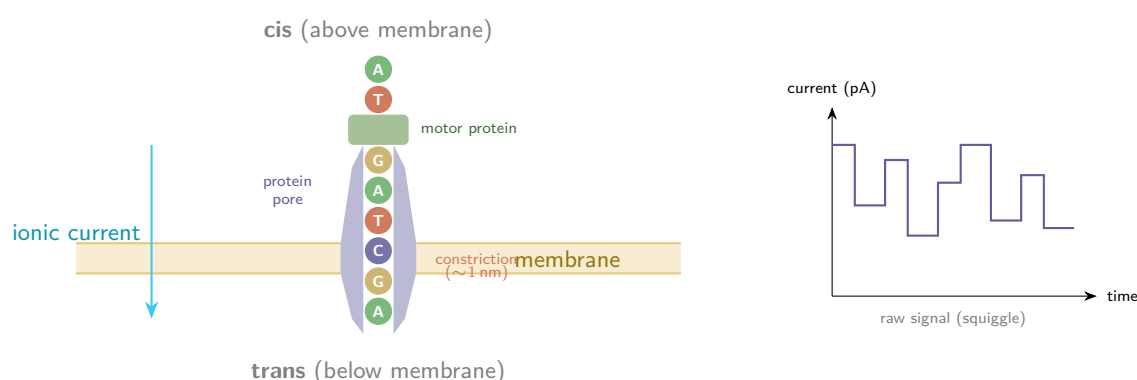


Figure 5.2: The nanopore sequencing principle. A single-stranded DNA molecule is threaded through a protein pore embedded in a lipid membrane. Each nucleotide passing through the constriction produces a characteristic disruption in the ionic current. The resulting “squiggle” signal is decoded by neural network basecallers to produce a DNA sequence.

Key Components

Protein pore.

Currently, ONT uses engineered versions of the *CsgG* protein (R10 series pores) embedded in an electrically resistant polymer membrane. The pore contains a dual-constriction region that samples approximately 5–9 nucleotides simultaneously, providing a richer signal than the single-constriction R9.4.1 pore.

Motor protein.

A helicase enzyme (the “motor protein”) is attached to the DNA and controls the rate at which the strand passes through the pore—approximately 400–450 bases per second. Without the motor protein, the DNA would translocate too fast for accurate current measurement.

Adapter and tether.

Library preparation attaches a sequencing adapter (containing the motor protein) to the DNA. A membrane-associated tether helps localise the adapted DNA near the pore.

ASIC and sensor array.

Each flow cell contains an application-specific integrated circuit (ASIC) with an array of individually addressable electrodes (channels), each capable of hosting one active pore. The ASIC measures the ionic current through each channel at kilohertz sampling rates.

Nanopore Ion Current Model

The measured ionic current through the nanopore depends on the sequence of nucleotides occupying the constriction region at any given moment. For a pore with a sensing region spanning k nucleotides (a “ k -mer context”), each unique k -mer produces a characteristic current level. The measured current at time t can be modelled as:

$$I(t) = \mu_{s(t)} + \eta(t) \quad (5.5)$$

where $\mu_{s(t)}$ is the expected current level for the k -mer $s(t)$ in the constriction at time t , and $\eta(t)$ is Gaussian noise. The signal-to-noise ratio (SNR) for distinguishing between two k -mers s_1 and s_2 is:

$$\text{SNR} = \frac{|\mu_{s_1} - \mu_{s_2}|}{\sigma} \quad (5.6)$$

where σ is the standard deviation of the noise. For the R10.4.1 pore with its dual-constriction design, the effective k -mer context is approximately $k = 9$ bases, giving $4^9 = 262,144$ possible k -mers—each with a distinct (though not always easily separable) current level.

Why Homopolymers Are Hard

Homopolymer runs (e.g., AAAAAA) are the dominant error mode in nanopore sequencing because consecutive identical bases produce *the same current level* as they transit the pore. The only information about homopolymer length comes from the *duration* of the signal plateau, which is stochastic: the motor protein does not advance each base at a perfectly constant rate. This duration-to-length mapping is inherently noisier than the current-level-to-identity mapping used for non-homopolymer contexts.

Stochastic Models of Homopolymer Errors

The time each base spends in the pore constriction (the “dwell time”) follows a stochastic distribution. For a homopolymer of true length L , the total dwell time T is the sum of L independent dwell times:

$$T = \sum_{i=1}^L t_i, \quad t_i \sim \text{Gamma}(\alpha, \beta) \quad (5.7)$$

The basecaller must infer L from T , which is a **deconvolution problem**. The coefficient of variation of the total dwell time decreases as $1/\sqrt{L}$, but the absolute uncertainty in the estimated length grows. In practice:

Table 5.2: Approximate accuracy of homopolymer length calling for nanopore R10.4.1 + SUP basecalling.

True length	Accuracy (% correct)	Common error
1–3	>99%	Rarely miscalled
4–5	95–99%	±1 base
6–8	85–95%	±1 base
9–12	70–85%	±1–2 bases
>12	<70%	±2+ bases

Practical Tip

For applications where homopolymer accuracy is critical (e.g., bacterial gene annotation, frameshift detection), consider using PacBio HiFi reads or supplementing nanopore data with short reads for polishing. Duplex nanopore reads (see below) also significantly improve homopolymer accuracy by providing two independent measurements of each base.

5.3.3 Library Preparation

ONT library preparation is notably simpler than that of most other platforms. Two main approaches are offered:

Ligation Sequencing Kit (LSK).

The standard high-quality preparation. DNA is end-repaired, A-tailed, and then ligated to ONT sequencing adapters containing the motor protein. This kit preserves the longest fragments and gives the highest throughput. Typical hands-on time: 60–90 minutes.

Rapid Sequencing Kit (RAD/RBK).

A transposase-based method that fragments the DNA and simultaneously attaches adapters in a single tagmentation step. Library preparation takes approximately 10 minutes, but the transposase cuts the DNA, limiting the maximum read length to the fragment size produced by the enzyme.

ONT Rapid Library Prep

1. Add transposase to 400 ng of genomic DNA.
2. Incubate at 30°C for 1 min, then 80°C for 1 min.
3. Add rapid adapter.
4. Incubate at room temperature for 5 min.
5. Load onto flow cell.

Total time: <15 minutes from extracted DNA to sequencing.

5.3.4 Basecalling: From Signal to Sequence

Raw nanopore data is not a sequence of bases but a time series of electrical current measurements—the “squiggle.” Converting this signal to a DNA sequence is a computationally intensive process called **basecalling**.

- **Early basecallers** (e.g., Metrichor, Albacore) used hidden Markov models (HMMs) with relatively modest accuracy.
- **Modern basecallers** use deep neural networks. ONT’s current production basecaller is **Dorado**, which succeeded the earlier **Guppy** and **Bonito** basecallers. Dorado uses recurrent neural networks (RNNs) and transformer architectures trained on millions of known-sequence reads.
- **Accuracy modes:** Dorado offers multiple models trading speed for accuracy:
 - **Fast (SUP-fast):** lowest accuracy, highest speed; suitable for real-time applications.
 - **High accuracy (HAC):** good balance of accuracy and speed; the default for most applications.
 - **Super accuracy (SUP):** highest accuracy; requires GPU acceleration and is 5–10× slower than fast mode.
- **Hardware:** production basecalling requires GPU acceleration. A modern NVIDIA GPU (e.g., A100, V100, RTX 4090) can basecall a PromethION flow cell’s data in approximately the same time as the run itself.

5.3.5 Pore Chemistry Evolution: R9 to R10 and Duplex

R9.4.1 Pore

The R9.4.1 pore was ONT’s workhorse for many years. It features a single constriction point that reads approximately 5 nucleotides (a 5-mer) simultaneously. This produces a highly compressed signal that made homopolymer detection challenging. Typical simplex (single-strand) accuracy with SUP basecalling was approximately 97–98% (Q15–Q17).

R10.4.1 Pore

The R10.4.1 pore, which has become the standard pore as of 2024–2025, features a **dual constriction** that reads a longer k-mer context (~9 bases). This provides:

- Significantly improved homopolymer resolution.
- Raw simplex accuracy of approximately 99% (Q20) with SUP basecalling.
- Better discrimination of modified bases.

Duplex Reads

ONT's **duplex sequencing** exploits the fact that when both strands of a DNA duplex are sequenced in succession through the same pore, the complementary information can be combined to produce a consensus. Duplex reads achieve accuracy exceeding Q30 (99.9%), approaching PacBio HiFi quality while retaining the potential for very long reads.

Duplex Reads: ONT's Answer to HiFi

Duplex reads are formed when the complement strand immediately follows the template strand through the same nanopore. The two reads of the same molecule provide independent error profiles that, when combined, yield >Q30 accuracy. The fraction of reads that are duplex depends on library preparation and loading conditions, typically 30–60% of reads.

5.3.6 ONT Platform Hierarchy

Oxford Nanopore offers a range of devices, from portable field instruments to production-scale platforms:

Flongle

- **Format:** a small, single-use flow cell adapter that plugs into a MinION or GridION.
- **Channels:** 126 (vs. 512 on a standard MinION flow cell).
- **Output:** up to 2.8 Gb per flow cell.
- **Cost:** approximately \$90 per flow cell—the lowest entry point for nanopore sequencing.
- **Use cases:** quick experiments, quality checks, small amplicon panels, species identification, teaching.

MinION and MinION Mk1C

- **Format:** the MinION is a USB-powered device roughly the size of a stapler. The Mk1C integrates a touchscreen computer and GPU for on-device basecalling.
- **Channels:** 512.
- **Output:** up to 50 Gb per flow cell (R10.4.1, 72-hour run).
- **Cost:** MinION starter packs begin at approximately \$1,000. Flow cells cost approximately \$500–900.
- **Use cases:** field sequencing (pathogen surveillance, biodiversity), small labs, educational settings, rapid diagnostics, proof-of-concept studies.

Field Sequencing with MinION

The MinION has been used for sequencing in remarkably challenging environments: during the 2014–2016 Ebola outbreak in West Africa, on the International Space Station, in Antarctic field camps, and in tropical forests for real-time biodiversity assessment. Its portability and minimal infrastructure requirements (a laptop and a power source) make it uniquely suited for point-of-need applications.

GridION

- **Format:** a benchtop device that runs up to 5 MinION-format flow cells simultaneously.
- **Channels:** up to 2,560 (5×512).
- **Output:** up to 250 Gb (5 flow cells).
- **Integrated compute:** includes an NVIDIA GPU for real-time basecalling.

- **Use cases:** mid-throughput labs needing flexibility; multiple small projects running in parallel.

PromethION P24 and P48

- **Format:** production-scale instruments running 24 (P24) or 48 (P48) high-capacity flow cells.
- **Channels per flow cell:** approximately 2,675 (roughly 5× a MinION flow cell).
- **Output per flow cell:** up to 290 Gb.
- **Total output:** up to approximately 14 Tb (P48, all flow cells).
- **Run time:** typically 72 hours per flow cell.
- **Cost:** approximately \$600–900 per flow cell; approximately \$1,000–2,000 for a 30× human WGS.
- **Use cases:** population-scale WGS, large cohort studies, production metagenomics, clinical genome sequencing.

5.3.7 Ultra-Long Reads

One of ONT's most remarkable capabilities is the generation of **ultra-long reads** exceeding 1 Mb (one million bases). Achieving ultra-long reads requires:

1. **Ultra-high molecular weight (UHMW) DNA extraction.** Standard extraction methods shear DNA to 20–50 kb. For ultra-long reads, specialised gentle lysis protocols (e.g., agarose plug extraction, Circulomics/PacBio Nanobind kits) are used to preserve megabase-length DNA.
2. **Careful library preparation.** Wide-bore pipette tips, minimal vortexing, and gravity-driven mixing to avoid shearing.
3. **Ligation-based library prep.** Transposase-based (rapid) kits are avoided as they fragment the DNA.
4. **Patience.** Ultra-long molecules take hours to translocate through the pore. A 1 Mb read at 400 bases/second takes approximately 42 minutes.

Ultra-Long Read Caveats

Ultra-long reads (>100 kb) are low-throughput by nature: each long molecule occupies a pore for an extended time, reducing the total number of reads. Ultra-long libraries also have lower yields and are technically demanding to prepare. Use ultra-long protocols when the application demands it (e.g., resolving very large repeats or centromeric regions), not as a default.

5.3.8 Direct RNA Sequencing

ONT is currently the only platform capable of sequencing **native RNA molecules directly**, without reverse transcription to cDNA.

- **Principle.** A poly(A)-tailed RNA molecule is ligated to a sequencing adapter and threaded through the nanopore 3' to 5'. The current signal is decoded by RNA-specific basecalling models.
- **Advantages.** Direct RNA sequencing preserves RNA modifications (m6A, pseudouridine, m5C, and others) as detectable signal perturbations. It also eliminates reverse transcription biases and PCR amplification artefacts.

- **Limitations.** Throughput is lower than cDNA-based approaches (the RNA motor protein translocates more slowly). Accuracy is currently lower than DNA basecalling. Input requirements are high (typically 500 ng–1 µg of poly(A)+ RNA).
- **Applications.** Epitranscriptomics research, full-length transcript isoform detection, viral RNA sequencing.

5.3.9 Adaptive Sampling (Read Until)

One of ONT’s most innovative software features is **adaptive sampling** (formerly called “Read Until”). Because nanopore sequencing produces data in real time, the instrument can make decisions about each read while it is being sequenced:

1. As a DNA molecule begins translocating through the pore, the first few hundred bases are basecalled in real time.
2. This partial sequence is aligned against a reference or target list.
3. If the molecule is **on-target** (matches a region of interest), sequencing continues normally.
4. If the molecule is **off-target**, the voltage across that channel is reversed, ejecting the molecule from the pore. The pore is then free to capture a new molecule.

This enables computational enrichment without physical capture or amplification—effectively a “software pulldown.” Adaptive sampling can enrich for entire chromosomes, specific genes, or panels of genomic regions, achieving 5–10× enrichment over random sequencing.

Adaptive Sampling

Adaptive sampling enables selective sequencing of target regions *without any special library preparation*. A whole-genome library is loaded, and the software selectively sequences regions of interest while rejecting off-target molecules in real time. This is unique to nanopore sequencing and has no equivalent on any other platform.

5.3.10 Base Modification Detection

Like PacBio, ONT can detect base modifications directly from the raw sequencing signal:

- **5-methylcytosine (5mC)** at CpG and non-CpG contexts.
- **N6-methyladenine (6mA).**
- **5-hydroxymethylcytosine (5hmC).**
- **RNA modifications** (m6A, pseudouridine) from direct RNA sequencing.

Modern ONT basecallers (Dorado) include integrated modification calling models that output modification probabilities alongside the base sequence, producing a “modBAM” file containing both sequence and modification information.

5.4 PacBio HiFi vs Oxford Nanopore: A Detailed Comparison

Table 5.3 provides a comprehensive comparison of the two dominant long-read platforms.

5.4.1 When to Choose PacBio HiFi

- **Reference-quality genome assembly.** HiFi reads provide the best single-technology assemblies, with per-read accuracy high enough that polishing with short reads is often unnecessary.
- **Clinical structural variant detection.** The high per-read accuracy minimises false positive SV calls.

Table 5.3: Detailed comparison of PacBio HiFi and Oxford Nanopore Technologies for long-read sequencing. Values represent typical performance as of early 2026.

Feature	PacBio HiFi (Revio)	ONT (PromethION, R10.4.1)
Read length (typical)	10–25 kb	10–100 kb (simplex); ultra-long >1 Mb
Read length (max)	~25 kb (HiFi); >300 kb (CLR)	>4 Mb (record)
Per-read accuracy	>Q30 (99.9%)	Q20 simplex (99%); >Q30 duplex
Consensus accuracy	>Q50 at 30×	>Q50 at 30× (with R10.4.1)
Error type	Balanced (sub, ins, del)	Indels, especially in homopolymers
Throughput/cell	~90 Gb HiFi	~290 Gb (PromethION)
Throughput/instrument	~360 Gb/day (4 cells)	up to 14 Tb (P48)
Run time	24 h per cell	72 h per flow cell
Cost per Gb	\$10–15 (HiFi)	\$3–8
Cost per 30× genome	\$1,000–1,500	\$1,000–2,000
Instrument cost	~\$600K (Revio)	~\$250K (P24); ~\$450K (P48)
Portability	Benchtop only	MinION: fully portable
Real-time analysis	No (batch processing)	Yes (data streams during run)
Adaptive sampling	No	Yes
Direct RNA seq	No	Yes
Methylation (5mC)	Yes (from kinetics)	Yes (from signal)
Library prep time	4–8 hours	10 min (rapid) to 90 min (ligation)

- **Variant calling in difficult regions.** HiFi accuracy in homopolymers and tandem repeats exceeds ONT simplex.
- **Simultaneous methylation and sequence.** A single HiFi dataset provides sequence, SVs, and CpG methylation.
- **Full-length isoform sequencing (Iso-Seq).** PacBio’s Iso-Seq protocol produces full-length transcript sequences with high accuracy.

5.4.2 When to Choose ONT

- **Ultra-long reads are needed.** Only ONT can routinely produce reads >100 kb.
- **Field or point-of-care sequencing.** The MinION is the only truly portable sequencer.
- **Real-time results.** Data is available during the run; adaptive sampling enables dynamic targeting.
- **Direct RNA sequencing.** ONT is the only platform that sequences native RNA.
- **Rapid turnaround.** From sample to sequence in under an hour is achievable with rapid library prep.
- **Cost-sensitive large-scale projects.** ONT’s cost per Gb can be lower, especially on PromethION.
- **Complex epigenomic profiling.** Detection of diverse modification types (5mC, 5hmC, 6mA) from a single dataset.

Complementary Technologies

PacBio and ONT are not mutually exclusive. Many cutting-edge genome projects use both: ONT ultra-long reads for scaffolding and spanning the largest repeats, combined with PacBio HiFi for base-level accuracy and variant calling. The T2T human genome used both technologies extensively.

5.5 Hybrid Sequencing Approaches

Combining short and long reads leverages the strengths of both technologies:

Short reads for accuracy, long reads for contiguity.

A common assembly strategy uses long reads for the initial contig assembly and short reads for error correction (polishing). With PacBio HiFi achieving $>Q30$ natively, the need for short-read polishing has diminished for PacBio-based assemblies, but it remains valuable for ONT-only assemblies.

Short reads for coverage, long reads for phasing.

A cost-effective clinical strategy uses deep short-read WGS for comprehensive variant calling, supplemented by moderate long-read coverage for SV detection and haplotype phasing.

Linked reads as a middle ground.

Technologies such as 10x Genomics Chromium (now discontinued but widely used in existing datasets) and Universal Sequencing Technology's TELL-Seq add long-range information to short reads through molecular barcoding. While not true long reads, linked reads provide phase information and improve SV detection.

Hi-C for scaffolding.

Chromosome conformation capture data (Hi-C, discussed in [Chapter 6](#) and later chapters) provides chromosome-scale scaffolding information that complements both short and long reads.

Table 5.4: Common hybrid sequencing strategies and their applications.

Strategy	Typical Coverage	Primary Application
HiFi only (30×)	30× HiFi	Reference-quality assembly, clinical WGS
ONT + Illumina	30× ONT + 30× Illumina	Cost-effective assembly with polishing
ONT ultra-long + HiFi	30× ONT-UL + 30× HiFi	T2T-grade assembly
HiFi + Hi-C	30× HiFi + Hi-C	Chromosome-scale phased assembly
Illumina + low-pass ONT	30× Illumina + 5–10× ONT	Clinical WGS with SV detection

5.6 Emerging Long-Read Developments

The long-read sequencing landscape continues to evolve:

- **PacBio Vega.** The Vega system aims to make HiFi sequencing more accessible to smaller laboratories with lower capital cost and simplified workflows.
- **ONT pore improvements.** ONT continues to develop new pore proteins with improved resolution and accuracy. Future pores may achieve Q30+ simplex accuracy routinely, further closing the gap with HiFi.
- **ONT P2 Solo.** A lower-cost PromethION variant running two flow cells, targeting labs that need PromethION-class data without the full P24/P48 instrument.
- **Direct protein sequencing.** ONT has demonstrated proof-of-concept for reading amino acid sequences through nanopores, potentially extending the platform beyond nucleic acids.
- **Electronic signal improvements.** Both platforms are investing in improved signal processing and AI-based basecalling, with each generation of basecaller software improving accuracy without hardware changes.
- **Multi-modal data.** The ability of both platforms to simultaneously call bases and modifications is evolving toward richer “multi-omic” readouts from single molecules.

5.7 Summary

Long-read sequencing has matured from an error-prone niche technology to a mainstream tool capable of producing highly accurate, phased genome assemblies and detecting the full spectrum of genetic variation. PacBio HiFi offers the highest per-read accuracy in a long-read format, making it ideal for reference-quality assembly and clinical applications. Oxford Nanopore provides unmatched read length, portability, real-time analysis, and unique capabilities such as direct RNA sequencing and adaptive sampling. The choice between platforms—or the decision to combine them—depends on the specific biological question, required read length and accuracy, budget, and infrastructure constraints. Together, these technologies are completing our picture of genome biology in ways that short reads alone could never achieve.

Chapter 6

Library Preparation: From Sample to Sequencer

Every sequencing experiment, regardless of platform, begins with the transformation of a biological sample into a **sequencing library**: a collection of DNA (or cDNA) fragments that carry the platform-specific adapter sequences required for cluster generation (or equivalent amplification), priming, and sequencing. Library preparation is arguably the most critical step in the entire sequencing workflow—poor library quality cannot be rescued by downstream bioinformatics. This chapter covers the complete library preparation process, from nucleic acid extraction through quality control, with detailed discussion of specific protocols, multiplexing strategies, and common failure modes.

6.1 What Is a Sequencing Library?

Definition of a Sequencing Library

A **sequencing library** is a collection of DNA fragments of defined size range, each carrying platform-specific adapter sequences at both ends. These adapters serve multiple functions:

- **Binding** the fragment to the flow cell surface (or bead, or ZMW).
- **Priming** the sequencing reaction (providing a known sequence for the sequencing primer to hybridise).
- **Indexing/barcoding** the fragment so that multiple samples can be pooled and sequenced together (multiplexing) and then computationally separated after sequencing (demultiplexing).
- **Amplification priming** during optional PCR enrichment of the library.

The general workflow for preparing a sequencing library consists of:

1. Nucleic acid extraction.
2. Fragmentation (for DNA libraries; RNA is typically already fragmented or is fragmented during protocol steps).
3. End repair and A-tailing.
4. Adapter ligation.
5. Size selection (optional, depending on protocol).
6. PCR amplification (optional; PCR-free protocols exist).
7. Library quantification and quality control.

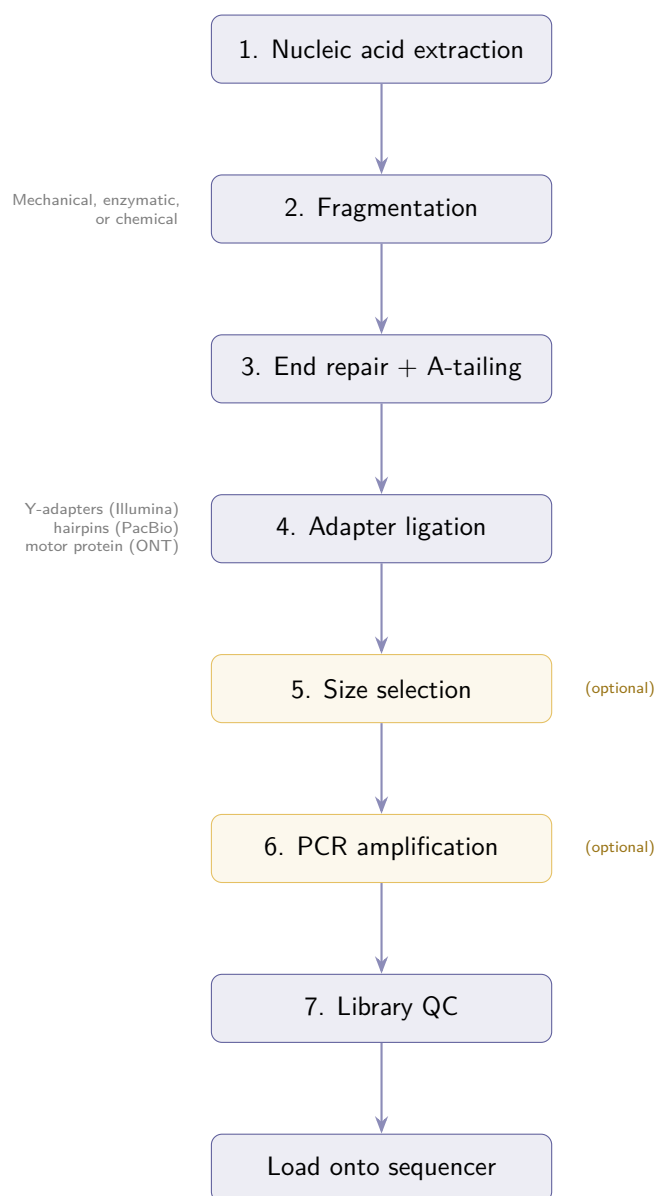


Figure 6.1: Overview of the general sequencing library preparation workflow. Gold-coloured steps are optional depending on the specific protocol. The entire process transforms extracted nucleic acids into adapter-ligated fragments ready for sequencing.

6.2 Nucleic Acid Extraction

The first step is obtaining high-quality nucleic acid from the biological sample. The choice of extraction method depends on the sample type (tissue, blood, cells, FFPE, environmental), the nucleic acid type (DNA vs. RNA), and the downstream application (especially the required fragment length for long-read sequencing).

6.2.1 DNA Extraction Methods

Column-Based Extraction

Column-based kits (e.g., QIAGEN DNeasy, Zymo Quick-DNA) use silica membranes that selectively bind DNA in the presence of chaotropic salts. The workflow consists of lysis, binding, washing, and elution.

- **Advantages:** fast (30–60 minutes), reproducible, minimal hands-on time, good purity ($A_{260}/_{280} \geq 1.8$).
- **Disadvantages:** shears DNA to 20–50 kb (unsuitable for ultra-long read preps). Limited input capacity.
- **Best for:** short-read sequencing, PCR-based applications, standard long-read preps where >50 kb reads are not required.

Bead-Based Extraction

Magnetic bead kits (e.g., Omega Bio-tek Mag-Bind, Beckman Coulter GenFind, PacBio Nanobind) use paramagnetic beads coated with silica or other DNA-binding matrices.

- **Advantages:** automatable (liquid-handling robots), scalable, gentler than columns (can preserve longer fragments with optimised protocols). The PacBio Nanobind (formerly Circulomics) kits use large paramagnetic discs that minimise shearing and can yield DNA >100 kb.
- **Disadvantages:** requires magnetic rack; some kits have lower yields than columns.
- **Best for:** high-throughput extraction, long-read sequencing, automation.

Phenol-Chloroform Extraction

The traditional organic extraction method remains the gold standard for yield and purity from difficult samples.

- **Procedure:** cell lysis with SDS and proteinase K, organic extraction with phenol:chloroform:isoamyl alcohol (25:24:1), aqueous phase recovery, ethanol precipitation or SPRI bead cleanup.
- **Advantages:** highest yields, excellent purity, can handle large and difficult samples (e.g., plant tissue, insects).
- **Disadvantages:** uses toxic chemicals (phenol is corrosive and a suspected carcinogen), time-consuming, difficult to automate, requires a fume hood.
- **Best for:** challenging samples, when maximum yield is essential, ultra-high molecular weight DNA for ultra-long nanopore reads.

DNA Integrity for Long-Read Sequencing

For PacBio HiFi or ONT long-read sequencing, DNA integrity is critical. Degraded or highly sheared DNA will produce short reads regardless of the sequencing platform's capabilities. Always assess DNA integrity using pulsed-field gel electrophoresis (PFGE) or the Agilent Femto Pulse / AATI Fragment Analyzer before committing to a long-read library preparation.

6.2.2 RNA Extraction Methods

RNA is chemically less stable than DNA due to the 2'-OH on the ribose sugar, which makes it susceptible to alkaline hydrolysis. RNA is also readily degraded by ubiquitous RNases. RNA extraction therefore requires strict RNase-free technique.

TRIzol (Guanidinium Thiocyanate-Phenol-Chloroform)

- **Principle:** guanidinium thiocyanate denatures proteins (including RNases); phenol-chloroform extraction separates RNA into the aqueous phase.
- **Advantages:** high yield, works from diverse tissues, extracts total RNA (including small RNAs).
- **Disadvantages:** organic solvents, time-consuming, potential genomic DNA contamination.

Column-Based RNA Extraction

Kits such as QIAGEN RNeasy and Zymo Quick-RNA provide silica column purification with optional on-column DNase digestion.

- **Advantages:** fast, reproducible, DNase treatment integrated, good for automation.
- **Disadvantages:** lower yield than TRIzol from some tissues; may lose small RNAs unless specifically captured.

mRNA Selection vs rRNA Depletion

Total RNA is composed of approximately 80–85% ribosomal RNA (rRNA), which is usually uninformative for gene expression studies. Two strategies address this:

- **Poly(A) selection:** oligo(dT) beads capture polyadenylated mRNA. Effective and widely used, but excludes non-polyadenylated transcripts (many lncRNAs, histone mRNAs, prokaryotic mRNAs).
- **rRNA depletion:** probe-based methods (e.g., Illumina Ribo-Zero, NEB NEBNext rRNA Depletion) hybridise to rRNA and remove it, retaining all other RNA species. Necessary for prokaryotic [RNA-seq](#), total RNA studies, and when non-coding RNAs are of interest.

Choosing Between Poly(A) Selection and rRNA Depletion

- For standard eukaryotic gene expression profiling, **poly(A) selection** provides cleaner data with less intronic/intergenic noise.
- For degraded samples (e.g., FFPE tissue), use **rRNA depletion**—degraded RNA loses its poly(A) tails.
- For prokaryotic or mixed samples, **rRNA depletion** is essential (prokaryotic mRNAs lack poly(A) tails).
- For non-coding RNA studies, use **rRNA depletion** to retain all non-ribosomal transcripts.

6.3 Fragmentation

Genomic DNA is far too long (chromosomes are megabases to hundreds of megabases) to be sequenced directly on short-read platforms. Fragmentation reduces the DNA to a defined size range suitable for the sequencing platform and application.

6.3.1 Mechanical Fragmentation

Sonication (Covaris)

The most widely used mechanical fragmentation method employs focused acoustic energy (Covaris ultrasonicators). DNA in solution is subjected to high-frequency acoustic waves that create cavitation, shearing the DNA at random positions.

- **Fragment size:** tunable from 150 bp to 5 kb by adjusting power, duty cycle, and time.
- **Advantages:** highly reproducible, produces a tight fragment-size distribution, no sequence bias.
- **Disadvantages:** requires specialised equipment (Covaris instrument), limited throughput (one sample at a time for some models), DNA ends are damaged and require end repair.

Nebulisation and Hydroshear

Older mechanical methods include nebulisation (forcing DNA through a small orifice under gas pressure) and hydroshearing (forcing DNA through a small tube). These are largely superseded by Covaris but may be encountered in legacy protocols.

6.3.2 Enzymatic Fragmentation

Tagmentation (Tn5 Transposase)

Tagmentation uses a hyperactive mutant of the Tn5 transposase loaded with sequencing adapters. The transposase simultaneously fragments the DNA and inserts adapters in a single step (“cut-and-paste” transposition). This is the basis of the Illumina DNA Prep (formerly Nextera DNA Flex) kit.

- **Advantages:** extremely fast (minutes), requires very low input (as little as 1 ng), simultaneously fragments and tags, reducing the number of protocol steps.
- **Disadvantages:** Tn5 has known insertion-site preferences (bias toward open chromatin and certain sequence motifs), which can create non-uniform genome coverage. Fragment size is controlled by the enzyme:DNA ratio and is less precisely tunable than Covaris.
- **Applications:** Illumina DNA Prep, ATAC-seq (where the Tn5 insertion bias is actually the signal of interest), ONT rapid library prep.

Quantifying Tn5 Insertion Bias. The hyperactive Tn5 transposase exhibits a measurable sequence preference at its insertion site. Studies have revealed a consensus insertion motif centred on the 9 bp target site duplication:

- **Position +1/+9 (flanking the duplication):** strong preference for weak bases (A/T), with ~70% AT frequency versus the expected 50–60%.
- **Central positions:** mild preference for alternating purine-pyrimidine patterns.
- **Chromatin context:** in native chromatin (as in ATAC-seq), Tn5 preferentially inserts into nucleosome-free regions. On purified DNA, the bias is purely sequence-dependent.

The practical consequence is that tagmentation-based libraries show up to $\pm 20\%$ deviation from expected coverage at extreme GC compositions compared to $\pm 5\text{--}10\%$ for Covaris-based libraries. This bias is modest for most applications but can be significant for quantitative analyses in GC-extreme organisms (e.g., *Plasmodium falciparum* at ~19% GC or *Streptomyces* at ~72% GC).

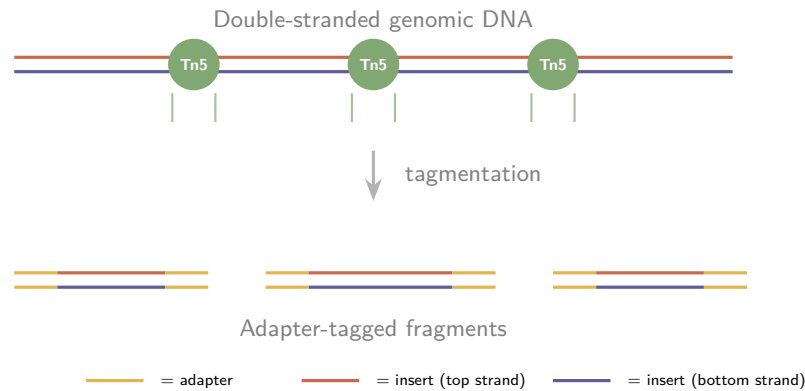


Figure 6.2: Tagmentation: the Tn5 transposase simultaneously fragments double-stranded DNA and inserts sequencing adapters at the cut sites. This combines fragmentation, end repair, and adapter attachment into a single enzymatic step.

Enzymatic Fragmentation (dsDNA Fragmentase)

NEB's dsDNA Fragmentase and similar enzyme cocktails use a combination of a nicking enzyme and a recombinase to produce random double-strand breaks. Fragment size is controlled by incubation time and temperature.

- **Advantages:** no specialised equipment needed, compatible with standard lab equipment.
- **Disadvantages:** less reproducible than Covaris; time-sensitive (over-digestion is easy).

6.3.3 Chemical Fragmentation

RNA fragmentation for RNA-seq is typically achieved by incubation in alkaline buffer or divalent cations (Mg^{2+}) at elevated temperature (e.g., 94°C for 1–8 minutes). The fragmentation time controls the resulting size distribution. This is a standard step in most RNA-seq library prep protocols.

6.4 End Repair and A-Tailing

After fragmentation, DNA ends are heterogeneous: they may have overhangs, blunt ends, damaged bases, or 5'-hydroxyl groups. Before adapters can be ligated, the ends must be repaired:

1. **End repair:** a combination of T4 DNA polymerase (fills in 5' overhangs, removes 3' overhangs) and T4 polynucleotide kinase (phosphorylates 5' ends) converts all fragments to blunt-ended, 5'-phosphorylated molecules.
2. **A-tailing:** Klenow fragment (exo-) adds a single deoxyadenosine (A) overhang to the 3' end of each fragment. This A-overhang is complementary to the T-overhang on most Illumina adapters, providing a non-palindromic sticky end that (i) increases ligation efficiency and (ii) prevents fragment–fragment concatenation.

Most modern kits combine end repair and A-tailing into a single reaction (e.g., NEB Ultra II End Prep Enzyme Mix).

6.5 Adapter Ligation

Adapters are short, synthetic oligonucleotide constructs that are ligated to the prepared DNA fragments. Their structure varies by platform:

6.5.1 Illumina Y-Adapters

Illumina adapters have a distinctive **Y-shape** (forked adapter). The double-stranded region ligates to the DNA fragment, while the single-stranded arms carry the P5 and P7 sequences (for flow cell binding) and primer sites for sequencing and index reads.

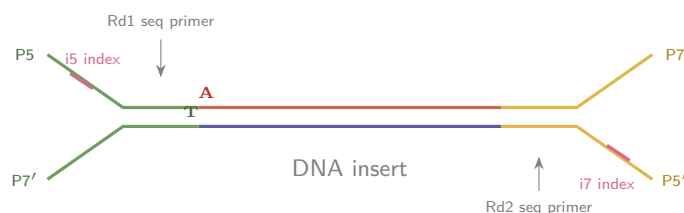


Figure 6.3: Structure of an Illumina Y-adapter (forked adapter) ligated to a DNA insert. The double-stranded stem ligates to the A-tailed fragment via a complementary T-overhang. The single-stranded arms carry the P5 and P7 flow cell binding sequences, index sequences (i5, i7), and sequencing primer annealing sites.

After adapter ligation:

- **Read 1** is sequenced from the Read 1 primer site, reading through the insert toward the opposite adapter.
- **Index 1 (i7)** is read after Read 1, identifying the sample.
- **Index 2 (i5)** is read after resynthesis of the complementary strand.
- **Read 2** is sequenced from the Read 2 primer site in the opposite direction.

6.5.2 PacBio SMRTbell Adapters

PacBio uses **hairpin adapters** that are ligated to both ends of the double-stranded insert, creating a topologically circular molecule (the SMRTbell). This circular structure enables the polymerase to read around the molecule multiple times for HiFi consensus generation.

6.5.3 ONT Adapters

ONT adapters contain the **motor protein** (helicase) pre-loaded onto the adapter. During ligation, the adapter-motor complex is attached to the end of the DNA. When the adapted molecule encounters a nanopore, the motor protein engages and controls the translocation rate.

6.6 Size Selection

After ligation, the library typically contains a range of fragment sizes. Size selection narrows this range to the optimal window for the application and platform.

6.6.1 SPRI Bead-Based Size Selection (AMPure XP)

The most widely used size-selection method employs **Solid Phase Reversible Immobilisation (SPRI) beads** (Beckman Coulter AMPure XP or equivalent). DNA binds to the carboxyl-coated paramagnetic beads in the presence of polyethylene glycol (PEG) and salt; the size selectivity is controlled by the ratio of beads to sample.

- **High bead:sample ratio** (e.g., 1.8×): binds fragments down to ~100 bp.
- **Low bead:sample ratio** (e.g., 0.6×): binds only large fragments (>500 bp), effectively removing small fragments and adapter dimers.

- **Double-sided selection:** a two-step protocol (e.g., 0.55× followed by 0.8×) removes both large and small fragments, yielding a tight size distribution centred on the desired range.

Double-Sided SPRI Size Selection (Target: 300–500 bp)

1. Add 0.55× volume of SPRI beads to the library. Mix and incubate 5 min.
2. Place on magnet. **Transfer supernatant** (contains fragments <600 bp) to a new tube. Discard beads (large fragments bound).
3. Add 0.25× *additional* beads to the supernatant (total ratio now ~0.8×). Mix and incubate 5 min.
4. Place on magnet. **Discard supernatant** (contains fragments <200 bp, including adapter dimers).
5. Wash beads twice with 80% ethanol.
6. Elute in 30 µL EB or water. The eluted library is enriched for 300–500 bp fragments.

6.6.2 Gel-Based Size Selection

- **BluePippin / PippinHT (Sage Science):** automated gel electrophoresis with electroelution of size-selected fractions. Highly precise (narrow size distributions), essential for some applications (e.g., PacBio HiFi libraries targeting a specific insert size).
- **Manual gel extraction:** excision of the desired band from an agarose gel followed by gel purification (e.g., QIAGEN QIAquick). Low throughput, risk of UV damage, but still used in some protocols.

Adapter Dimers

Adapter dimers (two adapters ligated together without an insert) are the most common library preparation artefact. They are preferentially sequenced because they are small and cluster efficiently. Even a small fraction of adapter dimers can consume a large proportion of sequencing reads. Always verify dimer removal by Bioanalyzer/TapeStation before sequencing. If dimers are present, perform an additional SPRI cleanup at 0.8× ratio.

6.6.3 The Physics of SPRI Bead Size Selection

SPRI (Solid Phase Reversible Immobilisation) bead-based selection exploits the principle that DNA solubility in a PEG/NaCl solution depends on fragment length. The mechanism involves molecular crowding: PEG excludes volume, effectively increasing the local concentration of DNA and salt, forcing DNA out of solution and onto the negatively charged carboxyl bead surface (mediated by divalent cation bridging).

The critical PEG concentration for precipitation of a DNA fragment of length L scales approximately as:

$$[\text{PEG}]_{\text{crit}} \propto L^{-0.5} \quad (6.1)$$

This means that longer fragments precipitate (bind to beads) at lower PEG concentrations than shorter fragments. Since the bead:sample ratio directly controls the effective PEG concentration (the beads are suspended in a PEG/NaCl buffer), the ratio controls the size cutoff:

Table 6.1: Approximate relationship between AMPure XP bead:sample volume ratio and the minimum fragment size retained.

Bead:sample ratio	Approximate size cutoff
2.0×	≥50 bp (captures nearly everything)
1.8×	≥100 bp (standard cleanup)
1.0×	≥200 bp
0.8×	≥300 bp
0.65×	≥400 bp
0.5×	≥600 bp
0.4×	≥800 bp

Optimising SPRI Ratios

The exact size cutoff at a given ratio depends on the buffer composition of your sample. Carry-over ethanol, high salt concentrations, or detergents can shift the effective cutoff. When precision matters (e.g., PacBio HiFi library prep), always validate your SPRI ratios empirically using a Bioanalyzer or TapeStation trace before and after selection.

6.6.4 Adapter Design: Thermodynamic Considerations

Sequencing adapters are carefully engineered oligonucleotides whose design is constrained by multiple thermodynamic requirements:

- **Melting temperature (T_m) matching:** The double-stranded portion of Y-adapters must have a T_m high enough to remain annealed at the ligation temperature (typically 20 °C) but low enough to denature cleanly during cluster generation (typically 95–98 °C). A T_m of 60–72 °C is typical for Illumina adapters.
- **Minimal secondary structure:** Self-complementary regions must be avoided to prevent adapter hairpins and self-dimers. The free energy of the most stable intramolecular structure should be $\Delta G > -2$ kcal/mol.
- **Balanced GC content:** Adapter sequences are designed with 40–60% GC content to ensure uniform hybridisation and amplification.
- **Index sequence design:** Barcode sequences must maintain minimum Hamming distance ($d \geq 3$ for single-error correction, $d \geq 5$ for double-error correction) and balanced base composition at each position across the index set. Illumina’s IDT for Illumina indexes are designed with $d \geq 4$ and colour balance at every cycle.

Why Y-Shaped Adapters?

Illumina adapters are “Y-shaped”: the bottom portion is double-stranded (complementary P5 and P7 sequences), while the top portion is single-stranded and non-complementary. This asymmetry ensures that after bridge amplification, the two strands of each cluster have different adapter orientations, enabling independent sequencing of Read 1 (from the P5 end) and Read 2 (from the P7 end).

6.6.5 Low-Input and FFPE Protocols

Standard library preparation protocols assume 100 ng to 1 μ g of high-quality input DNA. Many important sample types—biopsies, circulating cell-free DNA, FFPE (formalin-fixed, paraffin-embedded) tissue, single cells—provide far less or far more degraded material.

Low-input protocols (<10 ng)

Use tagmentation-based methods (Illumina DNA Prep) which work with as little as 1–10 ng, or specialised kits such as the KAPA HyperPlus or Swift Biosciences Accel-NGS. These protocols minimise handling losses through reduced reaction volumes and integrated enzymatic steps.

FFPE samples

Formalin fixation causes extensive DNA damage: cross-links, deamination (C→U, read as C→T substitutions), fragmentation, and oxidative damage (8-oxoG, read as G→T transversions). Specialised FFPE protocols include:

- **Enzymatic repair:** uracil-DNA glycosylase (UDG) removes deaminated cytosines; formamidopyrimidine-DNA glycosylase (Fpg) removes oxidised purines.
- **Adapted fragmentation:** FFPE DNA is already fragmented, so mechanical fragmentation is often skipped or minimised.
- **Higher PCR cycles:** Typically 10–14 cycles (vs. 4–6 for fresh tissue) to compensate for low-complexity input.

Cell-free DNA (cfDNA)

Circulating cfDNA from blood plasma is naturally fragmented (~167 bp fragments, corresponding to mono-nucleosomal DNA). Library prep skips fragmentation and uses end-repair → A-tailing → ligation with UMIs for error suppression.

FFPE Artefacts

C→T and G→A artefacts from formalin damage can be mistaken for real somatic mutations. Always include UDG treatment in FFPE library prep for variant calling applications. Some bioinformatics tools (e.g., `FilterMutectCalls` with the `-orientation-bias-artifact-priors` flag) can also filter these artefacts computationally.

6.7 PCR Amplification

Many library preparation protocols include a PCR amplification step after adapter ligation to increase the total library mass.

6.7.1 Why PCR?

- **Low input:** when starting material is limited (e.g., single cells, FFPE tissue, circulating cell-free DNA), PCR is necessary to generate enough library for sequencing.
- **Enrichment:** PCR selectively amplifies adapter-ligated fragments, enriching them over unligated fragments or adapter dimers.

6.7.2 How Many Cycles?

The number of PCR cycles should be minimised to reduce amplification bias and duplicate rates:

- **Standard WGS** (100 ng–1 µg input): 2–6 cycles, or PCR-free.
- **Low-input protocols** (1–10 ng): 8–12 cycles.
- **Ultra-low input / single cell:** 12–18+ cycles (with significant duplication expected).

6.7.3 PCR-Free Protocols

PCR-free library preparation skips amplification entirely, proceeding directly from adapter ligation to size selection and QC. PCR-free libraries offer:

- **More uniform coverage:** no GC-dependent amplification bias.
- **No PCR duplicates:** every read is an independent observation of the original molecule.
- **Better representation of difficult regions:** AT-rich and GC-rich regions are not under- or over-amplified.
- **Requirement:** sufficient input DNA (typically 100 ng–1 µg).

When to Go PCR-Free

For whole-genome sequencing where uniform coverage and accurate variant calling are paramount (especially clinical WGS), PCR-free library preparation is strongly preferred whenever input material is sufficient. The Illumina DNA PCR-Free Prep and KAPA HyperPrep PCR-Free workflows are widely used.

6.8 Unique Molecular Identifiers (UMIs)

Unique Molecular Identifiers (UMIs) are short random oligonucleotide sequences (typically 8–16 bp) incorporated into library molecules before or during PCR amplification.

6.8.1 How UMIs Work

1. During adapter ligation, each original molecule receives a unique (or nearly unique) random barcode—the UMI.
2. After PCR amplification, all copies (PCR duplicates) of the same original molecule carry the **same UMI**.
3. During analysis, reads sharing the same UMI and mapping position are identified as duplicates of the same original molecule. They can be collapsed into a single consensus read, improving accuracy and enabling accurate quantification of the number of original molecules.

6.8.2 Applications

- **Liquid biopsy / cfDNA:** UMIs are essential for detecting rare variants (e.g., <1% allele frequency) in circulating cell-free DNA, where PCR duplicates dominate.
- **RNA-seq:** UMIs enable absolute transcript counting (how many molecules, not how many reads) and correct for PCR amplification bias.
- **Single-cell sequencing:** 10x Genomics and other single-cell platforms use UMIs to count individual mRNA molecules per cell.
- **Error correction:** consensus of all reads sharing a UMI can suppress sequencing errors, effectively creating a “molecular consensus” analogous to PacBio CCS but in the library prep domain.

6.9 Indexing and Multiplexing

Modern sequencing instruments generate far more data per run than a single sample typically requires. **Multiplexing** (also called barcoding or indexing) allows multiple samples to be pooled into a single sequencing run, with computational demultiplexing after sequencing.

6.9.1 Single Indexing vs Dual Indexing

Single indexing.

Each sample receives a single unique index sequence (typically 6–8 bp i7 index). Sufficient for small experiments with few samples, but limited combinatorial space and no protection against index hopping.

Dual indexing.

Each sample receives two index sequences (i7 and i5), read in separate index reads. Dual indexing provides:

- A much larger combinatorial space (e.g., $96 \text{ i7} \times 96 \text{ i5} = 9,216$ combinations).
- The ability to detect and filter index-hopped reads (see below).

Unique Dual Indexes (UDI).

Each sample receives a unique *pair* of indexes that is not shared with any other sample. UDIs provide unambiguous sample assignment: an index-hopped read will carry an i7 from one sample and an i5 from another, creating a combination not present in any real sample, which can be identified and discarded.

6.9.2 Index Hopping

Index hopping (also called index switching or barcode swapping) occurs when free adapter molecules recombine during cluster generation on patterned flow cells using exclusion amplification (ExAmp). This causes a small fraction (0.1–2%) of reads to be assigned to the wrong sample.

Index Hopping on Patterned Flow Cells

Index hopping is a significant concern on instruments with patterned flow cells (NovaSeq, NextSeq 1000/2000). To mitigate:

1. **Always use Unique Dual Indexes (UDI).** This allows detection and removal of hopped reads.
2. **Clean up libraries thoroughly.** Remove free adapters and adapter dimers with SPRI beads before pooling.
3. **Do not store pooled libraries.** Pool and load promptly; long storage of pooled libraries increases free adapter contamination.

Index hopping does **not** occur on non-patterned flow cells (MiSeq) or on platforms using DNA nanoballs (MGI).

6.9.3 Index Design Considerations

- **Hamming distance:** index sequences should differ by at least 3 bases from every other index in the set, allowing single-error correction during demultiplexing.
- **Colour balance:** on two-channel Illumina instruments, the pool of indexes must provide signal in both colour channels at every cycle of the index read. Illumina provides pre-validated index sets that ensure colour balance.
- **Number of samples:** match the number of indexed samples to the sequencing capacity. Under-multiplexing wastes data; over-multiplexing yields insufficient coverage per sample.

6.10 Library Quality Control

Before loading a library onto a sequencer, three measurements should be performed:

6.10.1 Quantification: Qubit Fluorometer

The Qubit (Thermo Fisher) measures DNA or RNA concentration using fluorescent dyes that bind specifically to double-stranded DNA, single-stranded DNA, or RNA. Unlike spectrophotometric methods (NanoDrop), the Qubit is not confounded by contaminants (free nucleotides, adapters, proteins) and provides an accurate measurement of intact library molecules.

6.10.2 Fragment Size Distribution: Bioanalyzer / TapeStation

Capillary electrophoresis instruments (Agilent Bioanalyzer, Agilent TapeStation, Agilent Fragment Analyzer) separate library fragments by size and provide:

- An electropherogram showing the size distribution.
- Average fragment size.
- Detection of adapter dimers (a sharp peak at ~120–180 bp).
- For RNA: the RNA Integrity Number (RIN), a metric from 1 (degraded) to 10 (intact).

6.10.3 Functional Quantification: qPCR / dPCR

Quantitative PCR (qPCR) using primers that anneal to the adapter sequences measures only **functional** library molecules (those with adapters on both ends that can actually be sequenced). KAPA Library Quantification Kits are the standard. Digital PCR (dPCR) offers absolute quantification without a standard curve.

Library QC Checklist

Before sequencing, verify:

1. **Qubit concentration:** >2 nM (platform-specific threshold).
2. **Fragment size:** peaks at the expected size for your protocol (e.g., 350 bp for WGS with 150 bp inserts + adapters).
3. **No adapter dimers:** no sharp peak at 120–180 bp on Bioanalyzer/TapeStation.
4. **qPCR molarity:** within the loading range for your instrument.
5. **Pool balance:** if multiplexing, all samples should contribute approximately equal molarity to the pool.

6.11 Specific Library Preparation Protocols

6.11.1 Illumina DNA Prep (Tagmentation-Based)

Formerly known as Nextera DNA Flex, this is Illumina's standard whole-genome library preparation kit.

- **Input:** 100–500 ng DNA.
- **Fragmentation:** bead-linked Tn5 transposase (tagmentation).
- **Fragment size:** approximately 350 bp (tuneable by adjusting tagmentation conditions).
- **Indexing:** dual indexing with Nextera CD indexes.
- **PCR:** 5 cycles (standard input) to 12 cycles (low input).
- **Hands-on time:** approximately 2.5 hours.
- **Strengths:** fast, flexible input range, well-validated.
- **Limitations:** Tn5 insertion bias, not PCR-free.

6.11.2 Illumina Stranded mRNA Prep

Illumina's standard mRNA sequencing library preparation.

- **Input:** 25–1,000 ng total RNA.

- **mRNA selection:** poly(A) capture with oligo(dT) beads.
- **Fragmentation:** heat fragmentation in the presence of divalent cations.
- **Reverse transcription:** first-strand synthesis with random hexamers and actinomycin D; second-strand synthesis with dUTP for strand specificity.
- **Strand specificity:** the dUTP-marked second strand is degraded during PCR, preserving only the first strand (which is complementary to the mRNA). This provides strand-specific information, enabling sense/antisense discrimination.
- **Hands-on time:** approximately 5 hours (or 7 hours with rRNA depletion instead of poly(A) selection).

6.11.3 NEBNext Ultra II Kits

New England Biolabs (NEB) offers a widely used family of library prep kits:

- **NEBNext Ultra II DNA Library Prep:** enzymatic fragmentation or pre-fragmented DNA input, end repair, adapter ligation, PCR (or PCR-free). Supports inputs from 500 pg to 1 µg.
- **NEBNext Ultra II FS (Fragmentation/Selection):** integrated enzymatic fragmentation with size selection; “time-based” fragmentation control.
- **NEBNext Ultra II RNA Library Prep:** supports both poly(A)-selected and rRNA-depleted RNA, with strand-specific chemistry.
- **Strengths:** excellent low-input performance, well-validated, available with UMI adapters.

6.11.4 KAPA HyperPrep and HyperPlus

Roche/KAPA Biosystems kits are popular in core facilities:

- **KAPA HyperPrep:** for pre-fragmented DNA (e.g., Covaris-sheared). Single-tube end repair/A-tailing and adapter ligation. Supports PCR-free workflow.
- **KAPA HyperPlus:** adds enzymatic fragmentation (fragmentase) to HyperPrep. Suitable when Covaris is unavailable.
- **Strengths:** excellent for PCR-free WGS, widely used in clinical labs, robust and reproducible.

6.11.5 10x Genomics Chromium (Single-Cell)

The 10x Genomics Chromium platform uses microfluidic partitioning to generate single-cell libraries:

- **Principle:** individual cells (or nuclei) are captured in gel bead-in-emulsion (GEM) droplets, each containing a barcoded gel bead. Cell lysis, reverse transcription, and barcoding occur within each droplet.
- **Output:** each cDNA molecule is tagged with a cell barcode (identifying the cell of origin) and a UMI (identifying the individual molecule).
- **Library structure:** standard Illumina-compatible paired-end library. Read 1 contains the cell barcode + UMI; Read 2 contains the cDNA insert.
- **Applications:** [single-cell RNA sequencing \(scRNA-seq\)](#), single-cell ATAC-seq, single-cell multiome (RNA + ATAC), spatial transcriptomics (Visium).

6.11.6 PacBio SMRTbell Library Preparation

- **Input:** 5–10 µg of high-molecular-weight genomic DNA (for standard WGS HiFi libraries).
- **Fragmentation:** controlled shearing to the target insert size (typically 15–20 kb for HiFi).

- **End repair and adapter ligation:** blunt-end ligation of hairpin adapters to both ends of each fragment, creating the circular SMRTbell.
- **Size selection:** BluePippin or SageELF to remove short fragments (which would consume ZMW reads without contributing useful long reads).
- **Annealing and binding:** sequencing primer annealing, polymerase binding to the SMRTbell.
- **Hands-on time:** 4–8 hours (longer with size selection).
- **Considerations:** the protocol is demanding in terms of DNA quality and quantity. Degraded or impure DNA leads to poor SMRTbell formation and low yields.

6.11.7 ONT Ligation Sequencing Kit (SQK-LSK114)

- **Input:** 1 µg of genomic DNA (recommended; as low as 100 ng with reduced output).
- **Fragmentation:** optional (skip for long-read or ultra-long applications).
- **End repair and A-tailing:** using NEB reagents.
- **Adapter ligation:** ONT adapters with pre-loaded motor protein.
- **SPRI cleanup:** to remove unligated adapters and short fragments.
- **Hands-on time:** approximately 60–90 minutes.
- **Strengths:** preserves long fragments, highest throughput of ONT kits.

6.11.8 ONT Rapid Sequencing Kit (SQK-RAD114 / SQK-RBK114)

- **Input:** 200–400 ng of genomic DNA.
- **Fragmentation:** transposase-mediated (simultaneously fragments and tags).
- **Adapter attachment:** rapid adapters click onto the transposase-tagged ends.
- **Hands-on time:** approximately 10 minutes.
- **Limitations:** transposase fragmentation limits read length to the fragment size produced (~8–15 kb N50).
- **Barcoding variant (RBK114):** the rapid barcoding kit includes 96 barcodes for multiplexing.
- **Best for:** quick experiments, diagnostics, time-critical applications, teaching.

6.12 Target Enrichment

For many applications, sequencing the entire genome is unnecessary or impractical. Target enrichment methods selectively capture specific genomic regions before sequencing, dramatically reducing cost per target base.

6.12.1 Hybridisation Capture (Pull-Down)

Hybridisation capture uses biotinylated oligonucleotide probes (“baits”) that hybridise to regions of interest. Probe-target hybrids are captured with streptavidin-coated magnetic beads, and off-target DNA is washed away.

IDT xGen Lockdown Probes

A widely used panel system supporting custom and predesigned panels (e.g., xGen Exome). Known for high uniformity and flexibility.

Twist Bioscience

Offers highly uniform probes synthesised on silicon chips. Twist panels are cost-competitive and available in custom and catalogue configurations (exome, cancer panels).

Agilent SureSelect

One of the original hybridisation capture systems. Available in multiple formats including SureSelect XT, XT HS (with UMIs), and SureSelect V8 (latest exome design covering 34 Mb).

- **Advantages:** excellent for large target regions (exomes: 30–60 Mb), even coverage, compatible with many library types.
- **Disadvantages:** long protocol (1–2 day hybridisation), higher cost per sample than amplicon methods, requires more input DNA.

6.12.2 Amplicon-Based Enrichment

Amplicon methods use PCR primers to amplify specific targets directly.

AmpliSeq (Thermo Fisher / Illumina)

Highly multiplexed PCR (up to 24,000 amplicon pairs per pool) covering panels from small gene sets to near-exome coverage. Very low input (≤ 10 ng), fast (2–3 hours), and cost-effective for focused panels.

Archer FusionPlex / ArcherDX

Anchored multiplex PCR (AMP), which uses one gene-specific primer and one adapter-specific primer, enabling detection of fusions with unknown partners.

- **Advantages:** fast, low input, low cost for small to medium panels.
- **Disadvantages:** PCR bias, allelic dropout, limited scalability to very large targets, primer design constraints.

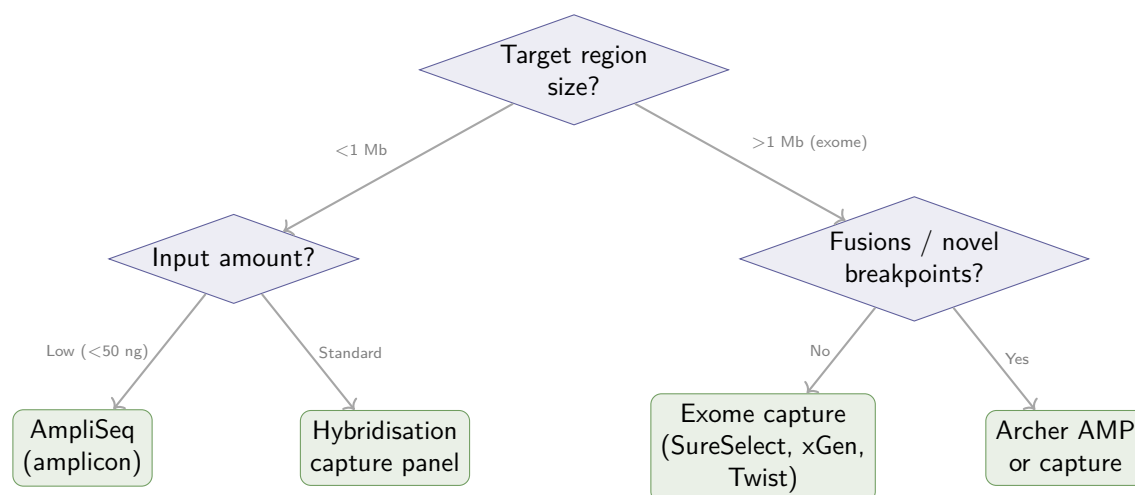


Figure 6.4: Decision flowchart for choosing a target enrichment strategy. For small panels with low input, amplicon-based methods are preferred. For exome-scale targets, hybridisation capture is standard. For fusion detection with unknown partners, anchored multiplex PCR (Archer) or capture-based methods are appropriate.

6.13 Common Failure Modes and Troubleshooting

Library preparation is a multistep biochemical process with many potential points of failure. Table 6.2 summarises the most common problems and their solutions.

Table 6.2: Common library preparation failure modes, their symptoms, likely causes, and solutions.

Symptom	Likely Cause	Diagnosis	Solution
Very low library yield	Degraded input DNA/RNA; failed ligation	Check input on Bioanalyzer; check adapter integrity	Use fresh extraction; verify enzyme activity; increase input
Adapter dimer peak (120–180 bp)	Excess adapters relative to insert; low-input protocol	Bioanalyzer/TapeStation	Additional SPRI cleanup (0.8×); reduce adapter concentration
Library too large (broad, >800 bp)	Insufficient fragmentation	Bioanalyzer	Increase fragmentation time/energy; repeat with optimised parameters
Library too small (<200 bp)	Over-fragmentation; over-digestion	Bioanalyzer	Reduce fragmentation time/energy; handle DNA more gently
High duplication rate after sequencing	Low library complexity; too many PCR cycles; overloading	Picard MarkDuplicates metrics	Increase input; reduce PCR cycles; use PCR-free protocol
Low %PF (clusters passing filter)	Overloading; poor library quality	Run metrics	Reduce loading concentration; verify library QC
Index hopping / sample crosstalk	Free adapters on patterned flow cells; combinatorial indexing	Unexpected barcode combinations	Use UDI; clean libraries; pool immediately before loading
Uneven coverage	GC bias from PCR; Tn5 insertion bias	Coverage plots, GC bias analysis	Use PCR-free protocol; switch from tagmentation to mechanical fragmentation

6.14 Decision Framework: Choosing a Library Prep Method

The choice of library preparation protocol depends on multiple factors. The following framework provides guidance:

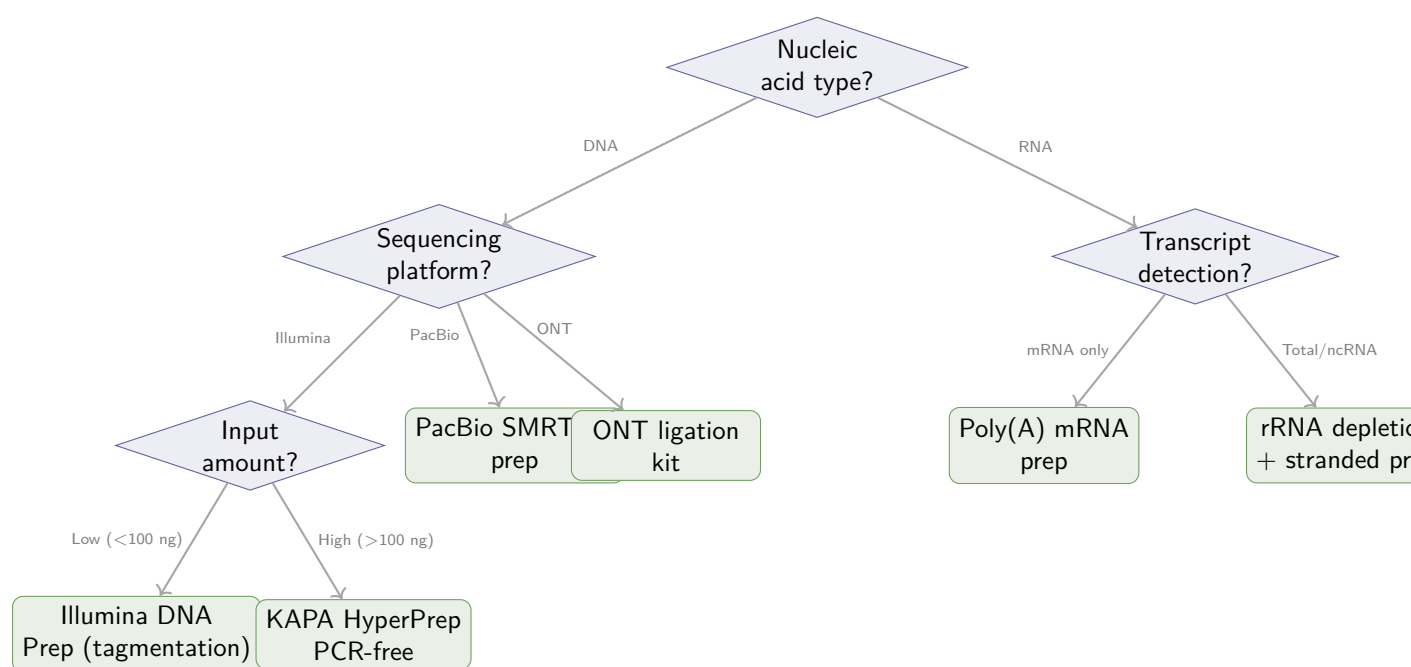


Figure 6.5: Decision flowchart for selecting a library preparation method. The primary decision points are the nucleic acid type (DNA vs. RNA), the sequencing platform, the available input amount, and the target transcript population.

Library Prep Selection Summary

For Illumina DNA WGS:

- High input (>100 ng): KAPA HyperPrep PCR-free or Illumina DNA PCR-Free Prep.
- Low input (<100 ng): Illumina DNA Prep (tagmentation) or NEBNext Ultra II with PCR.
- Ultra-low input / FFPE: NEBNext Ultra II with increased PCR cycles; consider UMIs.

For Illumina RNA-seq:

- Standard mRNA: Illumina Stranded mRNA Prep or NEBNext Ultra II RNA.
- Total RNA / degraded: rRNA depletion (Ribo-Zero / NEBNext rRNA Depletion) + stranded library prep.
- Low input: SMART-Seq (full-length) or NEB Ultra II RNA (directional).

For PacBio HiFi:

- Standard WGS: SMRTbell prep kit 3.0, targeting 15–20 kb inserts.
- Iso-Seq (full-length transcripts): SMRTbell Iso-Seq Express protocol.

For ONT:

- Maximum read length: Ligation sequencing kit (SQK-LSK114) with UHMW DNA.
- Rapid turnaround: Rapid sequencing kit (SQK-RAD114).
- Multiplexed: Rapid barcoding kit (SQK-RBK114) or native barcoding kit (SQK-NBD114).
- Direct RNA: Direct RNA sequencing kit (SQK-RNA004).

6.15 Automation and High-Throughput Library Preparation

For core facilities and large-scale projects, manual library preparation is impractical. Liquid-handling robots (e.g., Beckman Coulter Biomek, Hamilton STAR, Opentrons OT-2) can automate most library preparation steps:

- **Advantages:** reproducibility, reduced hands-on time, throughput (96 or 384 samples per batch), reduced pipetting errors.
- **Considerations:** requires protocol optimisation for robotic execution (tip types, mixing strategies, magnetic plate timing), significant upfront programming effort, and ongoing maintenance.
- **Common automated workflows:** Illumina DNA Prep on Hamilton, KAPA HyperPrep on Biomek, NEBNext Ultra II on Opentrons.

Starting with Automation

If you are new to automated library preparation, start by automating the most time-consuming and error-prone steps (SPRI cleanups, PCR setup, normalisation) while performing fragmentation and adapter ligation manually. Once confident, expand to full protocol automation.

6.16 Cost Considerations

Library preparation costs can be a significant component of the overall per-sample sequencing cost, especially for low-throughput applications:

- **Standard WGS library:** \$30–100 per sample (depending on kit and protocol).
- **RNA-seq library:** \$50–150 per sample (including RNA extraction and QC).

- **Exome capture:** \$100–200 per sample (library prep + capture probes).
- **10x Genomics single-cell:** \$1,500–3,000 per run (capturing 5,000–10,000 cells).
- **ONT rapid sequencing:** \$20–50 per sample (excluding flow cell cost).

Bulk purchasing, automation, and protocol miniaturisation (reducing reagent volumes) can substantially reduce per-sample costs for high-throughput operations.

6.17 Summary

Library preparation is the crucial bridge between biological samples and sequencing data. The choice of extraction method, fragmentation approach, adapter chemistry, size selection, amplification strategy, and quality control steps all directly impact data quality and the success of downstream analysis. Key principles to remember:

1. **Start with high-quality input.** No library preparation protocol can rescue severely degraded nucleic acid.
2. **Minimise PCR.** Use PCR-free protocols when possible; minimise cycle numbers when PCR is necessary.
3. **Use appropriate controls.** Include positive controls, negative controls, and PhiX spike-ins.
4. **QC rigorously.** Always measure library concentration (Qubit), size distribution (Bioanalyzer/TapeStation), and functional molarity (qPCR) before sequencing.
5. **Use UDIs on patterned flow cells.** Prevent index hopping with unique dual indexes.
6. **Match the protocol to the question.** The best library preparation method depends on the nucleic acid type, input amount, sequencing platform, and biological question.
7. **Document everything.** Record kit lot numbers, input amounts, PCR cycles, and QC results. Library preparation metadata is essential for troubleshooting and reproducibility.

Part III

Genomics: Reading the Blueprint

Chapter 7

Whole-Genome and Exome Sequencing

The ability to read the complete DNA sequence of an organism—or strategically selected portions of it—has transformed biology, medicine, and agriculture. In this chapter, we explore the three major strategies for interrogating genomic DNA: **whole-genome sequencing (WGS)**, which captures every nucleotide from telomere to telomere; **whole-exome sequencing (WES)**, which focuses on the protein-coding regions; and **targeted gene panels**, which zoom in on a curated set of genes relevant to a specific clinical question. We examine the technical workflows, compare the trade-offs of each approach, and build complete analysis pipelines in both Snakemake and Nextflow.

7.1 Whole-Genome Sequencing (WGS)

7.1.1 What WGS Captures

Whole-genome sequencing aims to determine the nucleotide sequence of an organism's *entire* genome. For the human genome, this means approximately 3.1 billion base pairs distributed across 22 autosomes and two sex chromosomes, plus the 16.6 kb mitochondrial genome. Critically, WGS captures:

- **Protein-coding exons** (~1.5% of the genome, ~20,000 genes).
- **Non-coding regulatory elements**: promoters, enhancers, silencers, insulators, and super-enhancers that control when, where, and how much a gene is expressed.
- **Intronic sequences**: including splice-regulatory elements and deeply intronic variants that can create cryptic exons.
- **Intergenic regions**: harbouring long non-coding RNA (lncRNA) genes, microRNA genes, and disease-associated variants identified by GWAS.
- **Repetitive elements**: transposable elements (SINEs, LINEs, LTRs), tandem repeats (microsatellites, minisatellites), segmental duplications, and centromeric/telomeric satellite DNA.
- **Structural variation**: large deletions, duplications, inversions, and translocations that are invisible to exome-only approaches.
- **Mitochondrial DNA**: enabling studies of heteroplasmy, maternal lineage, and mitochondrial disease.

Why the Non-Coding Genome Matters

Although only $\sim 1.5\%$ of the human genome encodes proteins, the ENCODE project demonstrated that $\sim 80\%$ shows biochemical activity. Over 90% of disease-associated variants from GWAS fall in non-coding regions. WGS is the only unbiased approach that captures all of these elements.

7.1.2 When to Use WGS vs. Targeted Approaches

The decision of whether to perform WGS, WES, or a targeted panel depends on the scientific question, budget, required depth, and turnaround time. Table 7.3 provides a detailed comparison. In general:

- **Use WGS** when you need a comprehensive, unbiased view of the genome—for example, in rare disease cases where WES was negative, cancer whole-genome analysis, structural variant detection, population genomics, or de novo genome assembly.
- **Use WES** when the primary question concerns protein-coding variants, particularly for Mendelian disease diagnostics where cost efficiency matters.
- **Use targeted panels** when a well-defined set of genes is associated with the phenotype (e.g., oncology panels, pharmacogenomics panels) and ultra-deep coverage is required for detecting low-frequency variants.

7.1.3 Sequencing Depth Requirements

Sequencing depth (also called coverage) is the average number of times each base in the genome is read. The required depth depends on the application:

- **Germline variant calling:** $30\times$ is the community standard for high-confidence SNV and indel detection in diploid genomes. The Genome in a Bottle Consortium validated this threshold extensively.
- **Somatic/cancer sequencing:** $60\text{--}100\times$ for tumor samples (paired with $\sim 30\text{--}40\times$ for the matched normal) to reliably detect subclonal mutations at variant allele frequencies (VAF) as low as 5–10%.
- **Ultra-deep cancer sequencing:** $500\text{--}1000\times$ or more for detecting rare subclonal variants or circulating tumor DNA (ctDNA) in liquid biopsies.
- **De novo assembly:** $30\text{--}50\times$ HiFi long reads, supplemented by $60\text{--}100\times$ ultra-long Oxford Nanopore reads for scaffolding.
- **Population-scale studies:** As low as $4\text{--}10\times$ with imputation (e.g., the TOPMed and All of Us programs) to reduce per-sample cost while maintaining statistical power at the cohort level.

Low-Pass WGS Limitations

Low-pass WGS ($<10\times$) is cost-effective for large population studies when combined with genotype imputation, but it is *not* suitable for clinical variant calling in individual patients. Individual genotype calls at low depth have high error rates, and rare variants unique to a single individual cannot be imputed.

7.1.4 Short-Read WGS Workflow

The canonical short-read WGS workflow proceeds through the following stages:

1. **Library preparation:** Genomic DNA is extracted, fragmented (enzymatic or mechanical), end-repaired, A-tailed, and ligated to platform-specific adapters. PCR amplification

(typically 4–8 cycles) or PCR-free protocols are used. PCR-free libraries reduce amplification bias and are preferred for WGS.

2. **Sequencing:** Libraries are loaded onto the sequencing instrument (e.g., Illumina NovaSeq X, AVITI by Element Biosciences). Paired-end 150 bp reads are standard for WGS.
3. **Quality control:** Raw FASTQ files are assessed with `FastQC` and `MultiQC`. Adapter trimming and quality filtering are performed with `fastp` or `Trimmomatic`.
4. **Alignment:** Trimmed reads are mapped to the reference genome using `BWA-MEM2` (or the original `BWA-MEM`) for short reads. The output is a sorted BAM/CRAM file.
5. **Duplicate marking:** PCR duplicates are flagged with `sambamba markdup`, `Picard MarkDuplicates`, or `biobambam2`.
6. **Base quality score recalibration (BQSR):** GATK's `BaseRecalibrator` adjusts quality scores using known variant sites (dbSNP, Mills indels).
7. **Variant calling:** Germline variants are called with `GATK HaplotypeCaller` or `DeepVariant`. Structural variants are called with `Manta`, `DELLY`, or `Smooove`.
8. **Variant filtering and annotation:** Variants are filtered (VQSR or hard filters) and annotated with `VEP`, `SnPEff`, or `ANNOVAR`.

PCR-Free vs. PCR-Based WGS Libraries

PCR-free libraries are the gold standard for WGS because they:

- Eliminate PCR duplicate reads, increasing effective coverage.
- Reduce GC-composition bias, improving uniformity across AT-rich and GC-rich regions.
- Improve coverage of repetitive regions.
- Require higher DNA input (~100–500 ng) compared to PCR-based protocols (~1–50 ng).

Use PCR-based protocols only when input DNA is limiting (e.g., clinical FFPE samples, forensic samples).

7.1.5 Long-Read WGS

Long-read sequencing technologies from PacBio (HiFi reads, 10–25 kb at >Q30 accuracy) and Oxford Nanopore Technologies (ultra-long reads exceeding 100 kb, some >1 Mb) provide critical advantages for WGS:

- **Structural variant detection:** Long reads span breakpoints of deletions, duplications, inversions, and translocations that are invisible to short reads due to repetitive flanking sequences.
- **Haplotype phasing:** Reads that span multiple heterozygous variants enable direct physical phasing of alleles across kilobases to megabases.
- **Repeat characterisation:** Tandem repeat expansions (e.g., *FMR1* CGG repeats in Fragile X, *HTT* CAG repeats in Huntington's disease, *C9orf72* GGGGCC repeats in ALS/FTD) can be directly measured.
- **Methylation detection:** Nanopore sequencing natively detects 5-methylcytosine (5mC) and other base modifications without bisulfite conversion.
- **De novo assembly:** Long reads are essential for resolving repetitive regions and producing highly contiguous genome assemblies.

Combining Short and Long Reads

A hybrid approach using 30× short-read Illumina data plus 30× PacBio HiFi data provides the best of both worlds: the cost-effective SNV/indel accuracy of short reads and the structural variant resolution and phasing of long reads. This is increasingly being adopted in clinical rare disease programs.

7.1.6 De Novo Genome Assembly

De novo genome assembly is the process of reconstructing a genome sequence without a reference. This is essential for non-model organisms, population-specific reference construction, and resolving complex genomic regions that collapse in reference-based analyses.

Assembly Concepts

Contigs

Contiguous sequences assembled from overlapping reads. A contig has no gaps.

Scaffolds

Ordered and oriented contigs linked by paired-end or long-range information (mate-pair reads, Hi-C, optical mapping), with gaps represented as stretches of N characters.

N50

The length N such that 50% of the total assembly size is contained in contigs (or scaffolds) of length $\geq N$. Higher N50 indicates a more contiguous assembly. Note: N50 alone does not measure *correctness*.

NG50

Like N50, but normalised to the expected genome size rather than the assembly size. Prevents inflated N50 from incomplete assemblies.

BUSCO completeness

Benchmarking Universal Single-Copy Orthologs—measures the fraction of expected single-copy genes present in the assembly. A complete genome should have >95% BUSCO completeness.

QV (consensus quality)

The Phred-scaled accuracy of the consensus sequence. QV50 means one error per 100,000 bases; QV60 means one error per 1,000,000 bases.

Modern Assembly Approaches

Current best-practice de novo assembly combines multiple data types:

1. **PacBio HiFi assembly:** Tools such as `hifiasm` produce highly accurate, haplotype-resolved assemblies from HiFi reads alone.
2. **Ultra-long ONT reads:** Reads >100 kb span centromeric satellites, segmental duplications, and other complex repeats. `Verkko` and `hifiasm` (with ONT integration) use these to connect HiFi contigs.
3. **Hi-C scaffolding:** Chromatin conformation data provides chromosome-scale scaffolding. Tools: `YaHS`, `SALSA2`, `3D-DNA`.
4. **Polishing:** Consensus accuracy is refined using `DeepConsensus` (PacBio), `Medaka` (ONT), or short-read polishing with `Pilon` or `Merqury`-guided assessment.

De Bruijn Graph Formalism

Most modern short-read assemblers—including **SPAdes**, **MEGAHIT**, and the local assembly module within **GATK HaplotypeCaller**—are founded on **de Bruijn graphs**. Understanding this formalism is essential for interpreting assembly behaviour and diagnosing failures.

Definition of a de Bruijn Graph

Given a set of reads and a fixed integer k , the de Bruijn graph $G_k = (V, E)$ is a directed graph defined as follows:

- **Nodes V :** Every unique $(k-1)$ -mer observed in the read set.
- **Edges E :** For each k -mer $s_1 s_2 \dots s_k$ in the reads, a directed edge connects the $(k-1)$ -mer prefix $s_1 \dots s_{k-1}$ to the $(k-1)$ -mer suffix $s_2 \dots s_k$.

The genome reconstruction problem reduces to finding a path through this graph that traverses every *edge* exactly once—an **Eulerian path**.

A connected directed graph possesses an Eulerian path if and only if (i) it is connected (considering the underlying undirected graph) and (ii) at most two nodes have unequal in-degree and out-degree. When these conditions hold, the walk can be computed in $O(|E|)$ time using Hierholzer’s algorithm. In practice, sequencing errors, polymorphisms, and repeats introduce branches, tips, and bubbles that must be resolved by heuristic graph simplification.

Choice of k : the sensitivity–specificity trade-off. The parameter k profoundly affects assembly quality:

- **Small k** increases *sensitivity*: more k -mers survive the read error filter, producing a more connected graph. However, many distinct genomic loci share the same short k -mer, collapsing repeats and creating ambiguous paths.
- **Large k** increases *specificity*: k -mers are more likely to be unique in the genome, resolving repeats and reducing spurious edges. However, k -mers that span sequencing errors are lost, fragmenting the graph.

The number of distinct k -mers in a genome of size G is bounded by:

$$N_k \leq \min(G - k + 1, 4^k), \quad (7.1)$$

where 4^k is the total number of possible k -mers over the DNA alphabet $\{A, C, G, T\}$. For the human genome ($G \approx 3.1 \times 10^9$), $k = 17$ yields $4^{17} \approx 1.7 \times 10^{10} > G$, so most 17-mers are unique; for $k = 11$, $4^{11} \approx 4.2 \times 10^6 \ll G$, meaning extensive k -mer collisions are inevitable. Multi- k assemblers (e.g., **SPAdes**) iteratively build graphs at increasing k values to combine the connectivity of small k with the specificity of large k .

Practical k -mer Selection Rules

For Illumina 150 bp paired-end data, k values in the range 21–77 are typical. As a rule of thumb, k should be (i) odd (to avoid palindromic k -mers), (ii) shorter than the read length, and (iii) large enough that $4^k \gg G$. Tools such as **KMC3** and **Jellyfish2** enable rapid k -mer counting to assess the k -mer spectrum before assembly.

Repeat resolution strategies. Repeats longer than $k - 1$ bases collapse into the same graph path, creating ambiguity. Several strategies address this:

- **Paired-end information:** Mate-pair or paired-end reads spanning a repeat connect the flanking unique sequences, enabling the assembler to thread through the correct path.

- **Read clouds / linked reads:** Barcoded short reads from the same long DNA molecule (10x Genomics Chromium, TELL-Seq, stLFR) provide long-range linkage without long-read sequencing.
- **Long reads:** HiFi and ultra-long ONT reads span most genomic repeats outright, enabling overlap-layout-consensus (OLC) or string-graph assembly that avoids the de Bruijn k -mer limitation entirely.
- **Coverage depth:** Unique regions have single-copy coverage, whereas collapsed repeats exhibit proportionally elevated coverage. Assemblers exploit this signal to estimate repeat copy number.

Assembly Quality Metrics: Formal Definitions

While N50 and NG50 were introduced informally above, their precise mathematical definitions are essential for rigorous benchmarking.

Let an assembly consist of n contigs with lengths $l_1 \geq l_2 \geq \dots \geq l_n$, and let G denote the known or estimated genome size.

N50

The contig length L such that the sum of lengths of all contigs with $l_i \geq L$ constitutes at least half the total assembly size:

$$\text{N50} = L \quad \text{where} \quad \sum_{i: l_i \geq L} l_i \geq \frac{1}{2} \sum_{j=1}^n l_j. \quad (7.2)$$

NG50

Identical to N50, but the denominator is the expected genome size G rather than the assembly size:

$$\text{NG50} = L \quad \text{where} \quad \sum_{i: l_i \geq L} l_i \geq \frac{G}{2}. \quad (7.3)$$

NG50 penalises incomplete assemblies, which may have artificially inflated N50 values.

auN (area under the Nx curve)

A single statistic that captures the entire contiguity distribution rather than a single percentile. It is defined as:

$$\text{auN} = \frac{\sum_{i=1}^n l_i^2}{\sum_{i=1}^n l_i} = \frac{\sum_{i=1}^n l_i^2}{A}, \quad (7.4)$$

where $A = \sum_i l_i$ is the total assembly length. Intuitively, auN is the expected contig length at a randomly chosen assembled base. It avoids the arbitrariness of the 50% threshold and is increasingly preferred for benchmarking.

Contiguity Is Not Correctness

N50, NG50, and auN measure *contiguity* only. A highly contiguous but mis-assembled genome (e.g., with chimeric joins) will have inflated metrics. Always complement contiguity statistics with *correctness* metrics: BUSCO completeness, QV estimated by Merqury, and reference-based mis-assembly counts from QUAST.

Scaffolding Algorithms

Scaffolding bridges gaps between contigs by ordering and orienting them using long-range linkage information. Three major data types are used:

Hi-C

Chromatin conformation capture generates read pairs from DNA fragments that were spatially proximal in the nucleus. Because contact probability decays as a power law with genomic distance ($P_{\text{contact}} \propto d^{-\gamma}$, with $\gamma \approx 1$ for mammalian genomes), Hi-C links contigs across megabase scales and assigns them to chromosomes. Tools: **YaHS**, **SALSA2**, **3D-DNA**, **ALLHiC** (for polyploid genomes).

Linked reads

Technologies such as 10x Genomics Chromium, TELL-Seq, and stLFR attach a shared barcode to short reads derived from the same ~ 50 – 100 kb DNA molecule. Barcode co-occurrence connects contigs separated by up to ~ 100 kb. Tools: **scaff10x**, **Tigmint** (mis-assembly detection), **Arcs**.

Optical maps

Bionano optical mapping generates restriction-enzyme-site patterns along megabase-length single molecules. These patterns are aligned to assembled contigs, enabling gap sizing and scaffolding across repeats invisible to sequence-based methods. Tools: **Bionano Solve**, **OMTools**.

Combining Multiple Scaffolding Data Types

The highest-contiguity assemblies integrate all available evidence. A common strategy: (1) assemble HiFi reads with **hifiasm**, (2) scaffold with linked-read data or ultra-long ONT reads, (3) apply Hi-C scaffolding to achieve chromosome-scale, and (4) validate with optical maps. The VGP (Vertebrate Genomes Project) pipeline exemplifies this multi-layered approach.

7.1.7 Telomere-to-Telomere (T2T) Assemblies

The Telomere-to-Telomere (T2T) Consortium achieved a landmark in 2022 by completing the first gapless human genome assembly, T2T-CHM13. This filled ~ 200 Mb of previously unresolved sequence, including:

- All 22 autosomal centromeres (arrays of α -satellite repeats).
- The five acrocentric short arms (chromosomes 13, 14, 15, 21, 22) containing ribosomal DNA arrays.
- Telomeric repeats $(TTAGGG)_n$ at all chromosome ends.
- Segmental duplications that collapsed in previous assemblies.
- 99 previously unannotated protein-coding genes.

The T2T assembly was achieved using a combination of PacBio HiFi reads ($\sim 30\times$), Oxford Nanopore ultra-long reads ($\sim 120\times$), and Hi-C data, applied to the CHM13 hydatidiform mole cell line (which is nearly homozygous, simplifying assembly).

The Evolution of Human Reference Genomes

GRCh37/hg19 (2009)

The workhorse reference for a decade. Still used in many legacy clinical databases.

GRCh38/hg38 (2013)

Improved representation of centromeres, added alternate loci (ALT contigs). The current standard for most clinical and research pipelines.

T2T-CHM13 (2022)

The first truly complete human genome. No gaps, no unresolved regions. Increasingly adopted for research, but clinical adoption is ongoing.

Human Pangenome Reference (2023–)

A graph-based reference incorporating genetic diversity from hundreds of individuals. Reduces reference bias and improves variant calling in diverse populations. Built using Minigraph-Cactus and PGGB.

7.1.8 Clinical WGS

WGS has entered clinical practice in several domains:

- **Rare disease diagnosis:** WGS achieves diagnostic yields of 25–40% in previously undiagnosed cases, outperforming WES (which yields ~25–30%). The improvement comes from detecting non-coding variants, structural variants, and repeat expansions.
- **Pharmacogenomics:** WGS comprehensively genotypes pharmacogenes (CYP2D6, CYP2C19, CYP3A5, DPYD, TPMT, etc.), including complex structural variants like CYP2D6 gene deletions and duplications.
- **Rapid WGS in NICU/PICU:** Ultra-rapid WGS (<24 hours from blood draw to diagnosis) has been implemented for critically ill neonates, achieving diagnostic rates of ~30–40% and changing clinical management in ~70% of diagnosed cases.
- **Cancer whole-genome sequencing:** Programmes such as the UK's 100,000 Genomes Project and Hartwig Medical Foundation have demonstrated the clinical utility of tumor/normal WGS for treatment selection.

7.2 Whole-Exome Sequencing (WES)

7.2.1 What the Exome Is

The exome comprises the protein-coding regions (exons) of all known genes. In the human genome:

- There are approximately 20,000 protein-coding genes.
- The total target region is approximately 30–60 Mb, depending on the capture kit and whether UTRs (untranslated regions) and miRNA genes are included.
- This represents roughly 1–2% of the total genome.
- Despite its small size, the exome harbours approximately 85% of known disease-causing mutations (as catalogued in ClinVar and HGMD).

7.2.2 Hybridisation Capture Workflow

WES uses hybridisation capture (also called hybrid capture or target enrichment) to selectively enrich exonic sequences:

1. **Library preparation:** Standard WGS library prep—fragmentation, end-repair, adapter ligation, PCR amplification.

2. **Hybridisation:** The library is incubated with biotinylated oligonucleotide probes (baits) complementary to exonic targets. Hybridisation occurs in solution (liquid-phase capture) over 4–16 hours.
3. **Capture:** Streptavidin-coated magnetic beads bind the biotinylated probes and their hybridised library fragments. Non-target fragments are washed away.
4. **Amplification:** Captured fragments undergo PCR amplification (8–12 cycles) to generate sufficient material for sequencing.
5. **Sequencing:** Paired-end 150 bp reads on Illumina platforms, targeting 80–150× mean coverage on target.

WES Uniformity Challenges

Hybridisation capture is inherently non-uniform. Some exons are captured efficiently while others are poorly represented due to:

- Extreme GC content (very AT-rich or GC-rich regions).
- Repetitive sequences flanking target exons.
- Pseudogene interference (probes cross-hybridise to homologous pseudogene sequences).
- Probe design gaps in newly annotated genes.

Typically, 80–90% of target bases achieve $\geq 20\times$ coverage, but some clinically important exons may be consistently undercovered.

7.2.3 When WES Is Preferred over WGS

- **Cost:** WES costs approximately one-third to one-quarter of WGS, making it attractive for large cohort studies and clinical diagnostics in resource-limited settings.
- **Data volume:** A WES dataset is ~5–8 GB per sample (vs. ~80–120 GB for 30× WGS), reducing storage and computational requirements.
- **Interpretation:** Coding variants are more easily interpretable than non-coding variants because the genetic code directly predicts amino acid changes.
- **Mendelian disease diagnostics:** For disorders with a suspected monogenic coding-region cause, WES has been the first-line test in many clinical genetics laboratories.

7.2.4 Limitations of WES

- Misses non-coding regulatory variants, deep intronic variants, and variants in non-captured UTRs.
- Poor detection of structural variants (limited by short insert sizes and non-uniform capture).
- Cannot reliably detect repeat expansions (e.g., trinucleotide repeats).
- Cannot detect epigenetic modifications.
- Capture-dependent biases reduce sensitivity for some exons, particularly in medically important but technically challenging genes (e.g., *GBA*, *SMN1*, *PMS2*).

7.2.5 Common Capture Kits

7.3 Targeted Sequencing and Gene Panels

7.3.1 Panel Design Strategies

Two major approaches exist for targeted sequencing:

Table 7.1: Comparison of major exome capture kits.

Kit	Probe Type	Target Size	Uniformity	Notable Features
Agilent SureSelect V8	RNA baits	~67 Mb	High	Long RNA probes, extensive UTR coverage
IDT xGen Exome V2	DNA baits	~34 Mb	Very High	Lockdown probes, excellent uniformity
Twist Bioscience Exome 2.0	DNA baits	~36 Mb	Very High	Synthetic biology-derived probes, fast hybridization
Roche KAPA HyperExome	DNA baits	~43 Mb	High	Integrated with KAPA library prep

Amplicon-based panels

Use multiplex PCR to amplify specific target regions. Examples include the Ion AmpliSeq technology and Illumina AmpliSeq. Advantages: rapid, low DNA input (~10 ng), simple workflow. Disadvantages: limited target size (<500 genes typically), potential allelic dropout at primer sites, PCR artefacts.

Hybridisation capture-based panels

Similar to WES but with a restricted probe set. Advantages: larger target regions possible, more uniform coverage, better for CNV detection. Disadvantages: longer workflow, higher DNA input required.

7.3.2 Clinical Gene Panels

- **Oncology panels:** Comprehensive cancer gene panels (e.g., MSK-IMPACT with 505 genes, FoundationOne CDx with 324 genes) detect somatic mutations, copy number alterations, microsatellite instability, and tumor mutational burden from FFPE tumor tissue.
- **Hereditary cancer panels:** 50–100+ genes associated with hereditary cancer predisposition (e.g., *BRCA1*, *BRCA2*, *TP53*, *MLH1*, *MSH2*).
- **Cardiology panels:** Genes for cardiomyopathies (*MYH7*, *MYBPC3*), arrhythmias (*SCN5A*, *KCNQ1*), aortopathies (*FBN1*, *TGFBR1*).
- **Neurogenetics panels:** Genes for epilepsy, intellectual disability, neuromuscular disorders, hereditary neuropathies.
- **Pharmacogenomics panels:** Targeted genotyping of key pharmacogenes for pre-emptive drug–gene interaction testing.

7.3.3 Ultra-Deep Sequencing for Low-Frequency Variants

Gene panels enable sequencing depths of 1,000–10,000× or more, which is essential for:

- Detecting somatic mutations at low VAF in heterogeneous tumors or circulating tumor DNA.
- Monitoring minimal residual disease (MRD) after cancer treatment.
- Detecting mosaic variants in genetic disorders.
- Identifying resistance mutations in infectious disease (e.g., HIV drug resistance, tuberculosis).

Unique Molecular Identifiers (UMIs)

When sequencing at ultra-high depth, PCR duplicates become a major source of error. UMIs—random oligonucleotide barcodes ligated to each original DNA molecule before amplification—allow computational identification and removal of PCR duplicates, enabling accurate consensus calling from original molecules. Tools such as `fgbio` and `UMI-tools` handle UMI-aware processing.

7.4 Reduced Representation Genotyping: RADseq and GBS

Whole-genome sequencing of every individual in a large population study is often prohibitively expensive. **Reduced representation sequencing** offers a powerful alternative: by restricting sequencing to a reproducible subset of the genome anchored at restriction-enzyme cut sites, researchers can genotype thousands of single nucleotide polymorphisms (SNPs) across hundreds or thousands of individuals at a fraction of the cost of WGS. Two dominant approaches are **Restriction site Associated DNA sequencing (RADseq)** and **Genotyping by Sequencing (GBS)**.

7.4.1 The Core Principle

Both approaches exploit restriction endonucleases to fragment genomic DNA at predictable, sequence-specific positions. Because the same enzyme cuts the same positions across all individuals of a species, sequencing reads from different samples align to the same genomic loci, enabling comparative SNP genotyping without a prior SNP array or fully sequenced reference genome (though a reference improves analysis considerably).

The fraction of the genome sampled is controlled by enzyme choice:

- **Rare cutters** (6–8 bp recognition sequences, e.g., *SbfI*, *EcoRI*) produce fewer, larger fragments and are suited to large genomes or when fewer, high-coverage loci are desired.
- **Frequent cutters** (4 bp recognition sequences, e.g., *MseI*, *MspI*) produce more fragments, increasing genome coverage but also sequencing costs per sample.
- **Methylation-sensitive enzymes** (e.g., *PstI* in GBS) preferentially cut in low-methylation, gene-rich regions, enriching for functionally relevant loci while reducing repetitive element contamination.

7.4.2 RADseq Protocol Variants

Original RADseq

Digest genomic DNA with a single restriction enzyme, ligate P1 adapters containing a sample-specific barcode, randomly shear with Covaris, size-select fragments (~300–600 bp), ligate P2 adapters, and PCR-amplify. Reads originate from one side of each restriction site. Introduced by Miller et al. (2007) and popularised by Baird et al. (2008).

Double-digest RADseq (ddRAD)

Use two restriction enzymes (one rare cutter + one common cutter) rather than random shearing to define fragment size. This is more reproducible, cheaper, and avoids sonication artefacts. The enzyme pair determines the fraction of the genome sampled. ddRAD has become one of the most widely used approaches.

2b-RAD

Uses Type IIB restriction enzymes (e.g., *BsaXI*) that cut both strands at a defined

distance from their recognition site, producing uniform 36 bp fragments. All loci are represented at equal depth, simplifying analysis.

ezRAD

Replaces P1 adapter ligation with random-primed amplification; requires only commercially available Illumina library prep kits and is accessible to labs without molecular biology expertise.

rapture (RADcap)

Combines RADseq library preparation with hybridisation capture to target a defined subset of RAD loci across thousands of individuals. Enables consistent SNP panels across very large cohorts at reduced cost.

7.4.3 Genotyping by Sequencing (GBS)

Developed by Elshire et al. (2011) at Cornell University, **GBS** simplifies the RADseq protocol by eliminating the random shearing and size-selection steps. Genomic DNA is digested with one or two restriction enzymes (commonly *ApeKI* or the two-enzyme combination *EcoRI*+*MspI*), and adapters are ligated directly to the sticky ends. All fragments are then PCR-amplified and sequenced. GBS is extremely simple to perform, requires only ~100 ng input DNA, and can process 96–384 samples per lane of sequencing.

GBS vs RADseq: Missing Data

GBS's simplicity comes at a cost: because no size selection is performed, fragment sizes are highly variable, leading to uneven locus representation and substantial missing data (loci with no reads in a given sample). Downstream analysis must account for missing genotypes. RADseq variants with size selection (ddRAD, 2b-RAD) achieve more consistent coverage across samples.

Two-Enzyme GBS

Using two enzymes (*EcoRI* + *MspI*) in a double-digest GBS design enriches for fragments flanked by one cut site from each enzyme on opposite ends, reducing the total number of sequenced loci to a more manageable set and decreasing the proportion of repetitive elements.

7.4.4 Applications

RADseq and GBS have transformed population genomics and breeding:

- **Population structure and demographic history** in wild populations (fish, birds, insects, plants), including species with no reference genome, using tools such as **Stacks** and **ipyrad**.
- **Landscape genomics**: correlating allele frequencies with environmental variables (elevation, temperature, precipitation) to identify loci under local adaptation.
- **Phylogenomics**: reconstructing species trees from thousands of loci across divergent taxa, replacing morphological or mitochondrial markers.
- **Quantitative trait locus (QTL) mapping** and genome-wide association studies (GWAS) in crops, livestock, and aquaculture species, where cost per sample is paramount.
- **Conservation genetics**: assessing genetic diversity and inbreeding in threatened or endangered populations.

- **Marker-assisted selection (MAS) in plant breeding:** SNP arrays derived from GBS enable genomic selection models predicting agronomic traits from genotype data alone.

7.4.5 Bioinformatics Pipeline

The canonical RADseq analysis pipeline proceeds as follows:

1. **Demultiplexing:** Assign reads to samples by barcode. `process_radtags` (Stacks) or `cutadapt` with inline barcode handling.
2. **Quality trimming:** Remove adapter sequence and low-quality bases. `Trim Galore`, `fastp`.
3. **Alignment (reference-based):** Align trimmed reads to the reference genome with BWA-MEM or Bowtie2.
4. **Genotyping:**
 - Reference-based: `Stacks` (`ref_map.pl`), `GATK HaplotypeCaller`, `FreeBayes`.
 - *De novo* (no reference): `Stacks` (`denovo_map.pl`) or `ipyrad` cluster loci across samples by sequence similarity.
5. **SNP filtering:** Filter by minimum allele frequency ($MAF > 0.05$), minimum coverage per locus, maximum missing data ($< 50\%$), and Hardy–Weinberg equilibrium.
6. **Downstream analysis:** `PLINK`, `ADMIXTURE`, `TreeMix`, `BayPass`, `OutFLANK` for population structure, admixture, gene flow, and selection scan analyses.

Table 7.2: Comparison of reduced representation genotyping approaches.

Feature	RADseq	ddRAD	2b-RAD	GBS	rapture
Enzymes	1	2	Type IIB	1–2	1–2
Size selection	Yes	Yes	No (uniform)	No	Yes
Fragment uniformity	Moderate	High	Very high	Low	High
Missing data	Low	Low	Low	High	Very low
Protocol complexity	Moderate	Moderate	Low	Low	High
Loci per sample	5K–100K	5K–200K	5K–50K	5K–500K	100–50K
Typical input DNA	1 µg	0.5–1 µg	0.5 µg	100 ng	0.5 µg
Cost per sample	\$\$\$	\$\$	\$\$	\$	\$\$
Best for	Pop. genomics	Pop. genomics	Uniform depth	Agriculture	Large cohorts

Choosing Between RADseq and GBS

For population genomics with a reference genome and moderate sample sizes (< 500), **ddRAD** offers the best balance of missing data control and protocol simplicity. For plant/animal breeding with very large sample sizes and budget constraints, **GBS** minimises per-sample cost. When consistent SNP panels must be maintained across multiple years of breeding, **rapture** (or a traditional SNP array) ensures locus stability. For organisms with no reference genome, use `ipyrad`’s *de novo* clustering with either ddRAD or GBS.

7.5 Comparison of Approaches

7.6 Visual Comparison: WGS vs. WES vs. Panel

Table 7.3: Comprehensive comparison of WGS, WES, and targeted panel sequencing.

Feature	WGS	WES	Targeted Panel
Target region	Entire genome (~3.1 Gb)	Exons (~30–60 Mb)	Selected genes (~0.01–2 Mb)
Genome coverage	100%	~1–2%	<0.1%
Typical depth	30–60×	80–150×	500–10,000×
Data per sample	80–120 GB	5–8 GB	0.1–1 GB
SNV detection	Excellent	Good (exonic only)	Excellent (on-target)
SV detection	Good (esp. with long reads)	Poor	Very limited
CNV detection	Good	Moderate	Moderate (capture-based)
Non-coding variants	Yes	No (mostly)	No
Repeat expansions	Yes (long reads)	No	Limited
Cost per sample*	\$\$\$\$	\$\$	\$
Turnaround time	2–4 weeks	2–4 weeks	1–2 weeks
Best for	Comprehensive analysis, research, rare disease, cancer	Mendelian disease, cost-effective coding-region screen	Clinical diagnostics, known gene sets, ultra-deep

* Approximate relative cost as of 2025–2026. Absolute costs continue to decrease.

Genomic Coverage: WGS vs. WES vs. Targeted Panel

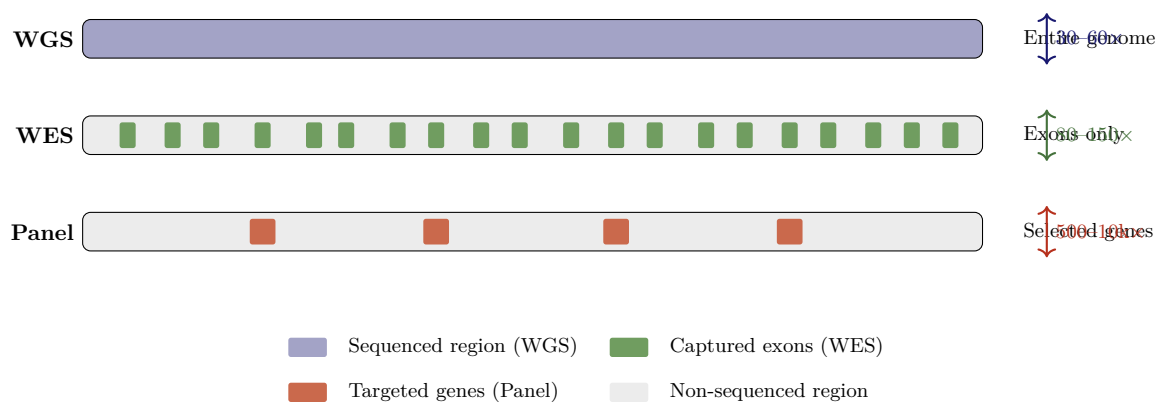


Figure 7.1: Visual comparison of genomic coverage achieved by whole-genome sequencing, whole-exome sequencing, and targeted gene panels. WGS covers the entire genome at moderate depth; WES selectively captures exonic regions at higher depth; targeted panels focus on a small gene set at very high depth.

7.7 Practical Bioinformatics: WGS Analysis Pipelines

In this section, we build a complete WGS germline variant calling pipeline in both Snakemake and Nextflow. The workflow follows the best-practice GATK pipeline: quality control with FastQC, read trimming with `fastp`, alignment with BWA-MEM2, duplicate marking with `sambamba`, base quality score recalibration with GATK, variant calling with both GATK HaplotypeCaller and DeepVariant, and variant filtering.

7.7.1 Snakemake WGS Pipeline

Listing 7.1: Complete Snakemake WGS germline variant calling pipeline.

```

1 # Snakefile for WGS Germline Variant Calling Pipeline
2 # Follows GATK Best Practices
3
4 configfile: "config.yaml"
5
6 SAMPLES = config["samples"]
7 REF = config["reference"]
8 KNOWN_SITES = config["known_sites"] # dbSNP, Mills indels
9
10 rule all:
11     input:
12         expand("results/vcf/{sample}.filtered.vcf.gz", sample=SAMPLES),
13         expand("results/deepvariant/{sample}.dv.vcf.gz", sample=SAMPLES),
14         "results/multiqc/multiqc_report.html"
15
16 rule fastqc:
17     input:
18         r1="data/fastq/{sample}_R1.fastq.gz",
19         r2="data/fastq/{sample}_R2.fastq.gz"
20     output:
21         html1="results/qc/fastqc/{sample}_R1_fastqc.html",
22         html2="results/qc/fastqc/{sample}_R2_fastqc.html",
23         zip1="results/qc/fastqc/{sample}_R1_fastqc.zip",
24         zip2="results/qc/fastqc/{sample}_R2_fastqc.zip"
25     threads: 2
26     shell:
27         """
28         fastqc -t {threads} -o results/qc/fastqc/ \
29             {input.r1} {input.r2}
30         """
31
32 rule fastp_trim:
33     input:
34         r1="data/fastq/{sample}_R1.fastq.gz",
35         r2="data/fastq/{sample}_R2.fastq.gz"
36     output:
37         r1="results/trimmed/{sample}_R1.trimmed.fastq.gz",
38         r2="results/trimmed/{sample}_R2.trimmed.fastq.gz",
39         json="results/qc/fastp/{sample}.fastp.json",
40         html="results/qc/fastp/{sample}.fastp.html"
41     threads: 4
42     shell:
43         """
44         fastp -i {input.r1} -I {input.r2} \
45             -o {output.r1} -O {output.r2} \

```



```

46         --json {output.json} --html {output.html} \
47         --qualified_quality_phred 20 \
48         --length_required 50 \
49         --adapter_sequence auto \
50         --thread {threads}
51     """
52
53 rule bwa_mem2_align:
54     input:
55         r1="results/trimmed/{sample}_R1.trimmed.fastq.gz",
56         r2="results/trimmed/{sample}_R2.trimmed.fastq.gz",
57         ref=REF
58     output:
59         bam="results/aligned/{sample}.sorted.bam"
60     params:
61         rg="@RG\t\tID:{sample}\t\tSM:{sample}\t\tPL:ILLUMINA\t\tLB:lib1"
62     threads: 16
63     shell:
64         """
65         bwa-mem2 mem -t {threads} -R '{params.rg}' {input.ref} \
66             {input.r1} {input.r2} \
67             | samtools sort -@ 4 -o {output.bam} -
68             samtools index {output.bam}
69         """
70
71 rule sambamba_markdup:
72     input:
73         bam="results/aligned/{sample}.sorted.bam"
74     output:
75         bam="results/dedup/{sample}.dedup.bam",
76         bai="results/dedup/{sample}.dedup.bam.bai"
77     threads: 8
78     shell:
79         """
80         sambamba markdup -t {threads} \
81             --tmpdir tmp/ \
82             {input.bam} {output.bam}
83         """
84
85 rule gatk_bqsr:
86     input:
87         bam="results/dedup/{sample}.dedup.bam",
88         ref=REF,
89         known=KNOWN_SITES
90     output:
91         table="results/bqsr/{sample}.recal_data.table",
92         bam="results/bqsr/{sample}.recal.bam"
93     shell:
94         """
95         gatk BaseRecalibrator \
96             -I {input.bam} -R {input.ref} \
97             --known-sites {input.known} \
98             -O {output.table}
99
100         gatk ApplyBQSR \
101             -I {input.bam} -R {input.ref} \
102             --bqsr-recal-file {output.table} \
103             -O {output.bam}

```

```

104     """
105
106 rule gatk_haplotypcaller:
107     input:
108         bam="results/bqsr/{sample}.recal.bam",
109         ref=REF
110     output:
111         gvcf="results/gvcf/{sample}.g.vcf.gz"
112     threads: 4
113     shell:
114         """
115         gatk HaplotypeCaller \
116             -I {input.bam} -R {input.ref} \
117             -O {output.gvcf} \
118             -ERC GVCF \
119             --native-pair-hmm-threads {threads}
120         """
121
122 rule gatk_genotype:
123     input:
124         gvcf="results/gvcf/{sample}.g.vcf.gz",
125         ref=REF
126     output:
127         vcf="results/vcf/{sample}.raw.vcf.gz"
128     shell:
129         """
130         gatk GenotypeGVCFs \
131             -R {input.ref} \
132             -V {input.gvcf} \
133             -O {output.vcf}
134         """
135
136 rule gatk_filter:
137     input:
138         vcf="results/vcf/{sample}.raw.vcf.gz",
139         ref=REF
140     output:
141         vcf="results/vcf/{sample}.filtered.vcf.gz"
142     shell:
143         """
144         gatk VariantFiltration \
145             -R {input.ref} -V {input.vcf} \
146             --filter-expression "QD < 2.0" --filter-name "LowQD" \
147             --filter-expression "FS > 60.0" --filter-name "StrandBias" \
148             --filter-expression "MQ < 40.0" --filter-name "LowMQ" \
149             --filter-expression "MQRankSum < -12.5" --filter-name "MQRankSum" \
150             --filter-expression "ReadPosRankSum < -8.0" --filter-name "ReadPos" \
151             -O {output.vcf}
152         """
153
154 rule deepvariant:
155     input:
156         bam="results/dedup/{sample}.dedup.bam",
157         ref=REF
158     output:
159         vcf="results/deepvariant/{sample}.dv.vcf.gz"
160     threads: 16
161     shell:

```

```

162     """
163     run_deepvariant \
164         --model_type=WGS \
165         --ref={input.ref} \
166         --reads={input.bam} \
167         --output_vcf={output.vcf} \
168         --num_shards={threads}
169     """
170
171 rule multiqc:
172     input:
173         expand("results/qc/fastqc/{sample}_R1_fastqc.zip", sample=SAMPLES),
174         expand("results/qc/fastp/{sample}.fastp.json", sample=SAMPLES)
175     output:
176         "results/multiqc/multiqc_report.html"
177     shell:
178         """
179         multiqc results/qc/ -o results/multiqc/ --force
180         """

```

7.7.2 Nextflow WGS Pipeline

Listing 7.2: Complete Nextflow DSL2 WGS germline variant calling pipeline.

```

1  #!/usr/bin/env nextflow
2  nextflow.enable.dsl=2
3
4  // Nextflow WGS Germline Variant Calling Pipeline
5  // Follows GATK Best Practices
6
7  params.reads      = "data/fastq/*_{R1,R2}.fastq.gz"
8  params.ref        = "reference/hg38.fa"
9  params.known      = "reference/known_sites.vcf.gz"
10 params.outdir     = "results"
11
12 // ---- Processes ----
13
14 process FASTQC {
15     tag "$sample_id"
16     publishDir "${params.outdir}/qc/fastqc", mode: 'copy'
17     cpus 2
18
19     input:
20     tuple val(sample_id), path(reads)
21
22     output:
23     path("*.html,zip"), emit: reports
24
25     script:
26     """
27     fastqc -t ${task.cpus} ${reads}
28     """
29 }
30
31 process FASTP {
32     tag "$sample_id"
33     publishDir "${params.outdir}/trimmed", mode: 'copy'

```

```

34     cpus 4
35
36     input:
37     tuple val(sample_id), path(reads)
38
39     output:
40     tuple val(sample_id),
41         path("${sample_id}_R{1,2}.trimmed.fastq.gz"),
42         emit: trimmed_reads
43     path("${sample_id}.fastp.{json,html}"),
44         emit: reports
45
46     script:
47     """
48     fastp -i ${reads[0]} -I ${reads[1]} \
49         -o ${sample_id}_R1.trimmed.fastq.gz \
50         -O ${sample_id}_R2.trimmed.fastq.gz \
51         --json ${sample_id}.fastp.json \
52         --html ${sample_id}.fastp.html \
53         --qualified_quality_phred 20 \
54         --length_required 50 \
55         --thread ${task.cpus}
56     """
57 }
58
59 process BWA_MEM2 {
60     tag "$sample_id"
61     publishDir "${params.outdir}/aligned", mode: 'copy'
62     cpus 16
63
64     input:
65     tuple val(sample_id), path(reads)
66     path ref
67     path ref_idx // .fa, .bwt.2bit.64, .amb, .ann, .pac
68
69     output:
70     tuple val(sample_id), path("${sample_id}.sorted.bam"),
71         path("${sample_id}.sorted.bam.bai"),
72         emit: aligned_bam
73
74     script:
75     """
76     bwa-mem2 mem -t ${task.cpus} \
77         -R "@RG\tID:${sample_id}\tSM:${sample_id}\tPL:ILLUMINA\tLB:lib1" \
78         ${ref} ${reads} \
79     | samtools sort -@ 4 -o ${sample_id}.sorted.bam -
80     samtools index ${sample_id}.sorted.bam
81     """
82 }
83
84 process SAMBAMBA_MARKDUP {
85     tag "$sample_id"
86     publishDir "${params.outdir}/dedup", mode: 'copy'
87     cpus 8
88
89     input:
90     tuple val(sample_id), path(bam), path(bai)
91

```

```

92     output:
93     tuple val(sample_id), path("${sample_id}.dedup.bam"),
94         path("${sample_id}.dedup.bam.bai"),
95         emit: dedup_bam
96
97     script:
98     """
99     sambamba markdup -t ${task.cpus} ${bam} ${sample_id}.dedup.bam
100    """
101 }
102
103 process GATK_BQSR {
104     tag "$sample_id"
105     publishDir "${params.outdir}/bqsr", mode: 'copy'
106
107     input:
108     tuple val(sample_id), path(bam), path(bai)
109     path ref
110     path ref_fai
111     path ref_dict
112     path known_sites
113     path known_sites_idx
114
115     output:
116     tuple val(sample_id), path("${sample_id}.recal.bam"),
117         path("${sample_id}.recal.bam.bai"),
118         emit: recal_bam
119
120     script:
121     """
122     gatk BaseRecalibrator \
123         -I ${bam} -R ${ref} \
124         --known-sites ${known_sites} \
125         -O ${sample_id}.recal_data.table
126
127     gatk ApplyBQSR \
128         -I ${bam} -R ${ref} \
129         --bqsr-recal-file ${sample_id}.recal_data.table \
130         -O ${sample_id}.recal.bam
131
132     samtools index ${sample_id}.recal.bam
133     """
134 }
135
136 process HAPLOTYPECALLER {
137     tag "$sample_id"
138     publishDir "${params.outdir}/vcf", mode: 'copy'
139     cpus 4
140
141     input:
142     tuple val(sample_id), path(bam), path(bai)
143     path ref
144     path ref_fai
145     path ref_dict
146
147     output:
148     tuple val(sample_id),
149         path("${sample_id}.g.vcf.gz"),

```

```

150         emit: gvcf
151
152     script:
153     """
154     gatk HaplotypeCaller \
155         -I ${bam} -R ${ref} \
156         -O ${sample_id}.g.vcf.gz \
157         -ERC GVCF \
158         --native-pair-hmm-threads ${task.cpus}
159     """
160 }
161
162 process DEEPVARIANT {
163     tag "$sample_id"
164     publishDir "${params.outdir}/deepvariant", mode: 'copy'
165     cpus 16
166
167     input:
168     tuple val(sample_id), path(bam), path(bai)
169     path ref
170     path ref_fai
171
172     output:
173     tuple val(sample_id),
174         path("${sample_id}.dv.vcf.gz"),
175         emit: dv_vcf
176
177     script:
178     """
179     run_deepvariant \
180         --model_type=WGS \
181         --ref=${ref} \
182         --reads=${bam} \
183         --output_vcf=${sample_id}.dv.vcf.gz \
184         --num_shards=${task.cpus}
185     """
186 }
187
188 process MULTIQC {
189     publishDir "${params.outdir}/multiqc", mode: 'copy'
190
191     input:
192     path(reports)
193
194     output:
195     path("multiqc_report.html")
196
197     script:
198     """
199     multiqc . -o . --force
200     """
201 }
202
203 // ---- Workflow ----
204
205 workflow {
206     // Create read-pair channel
207     Channel

```

```

208     .fromFilePairs(params.reads)
209     .set { read_pairs_ch }
210
211     ref          = file(params.ref)
212     ref_fai      = file("${params.ref}.fai")
213     ref_dict     = file(params.ref.replaceAll('.fa$', '.dict'))
214     ref_idx      = Channel.fromPath("${params.ref}.*").collect()
215     known_sites  = file(params.known)
216     known_idx    = file("${params.known}.tbi")
217
218     // Step 1: QC and trimming (in parallel)
219     FASTQC(read_pairs_ch)
220     FASTP(read_pairs_ch)
221
222     // Step 2: Alignment
223     BWA_MEM2(FASTP.out.trimmed_reads, ref, ref_idx)
224
225     // Step 3: Duplicate marking
226     SAMBAMBA_MARKDUP(BWA_MEM2.out.aligned_bam)
227
228     // Step 4: BQSR
229     GATK_BQSR(SAMBAMBA_MARKDUP.out.dedup_bam,
230              ref, ref_fai, ref_dict,
231              known_sites, known_idx)
232
233     // Step 5: Variant calling (two callers in parallel)
234     HAPLOTYPECALLER(GATK_BQSR.out.recal_bam,
235                    ref, ref_fai, ref_dict)
236     DEEPVARIANT(SAMBAMBA_MARKDUP.out.dedup_bam,
237                ref, ref_fai)
238
239     // Step 6: MultiQC report
240     MULTIQC(
241         FASTQC.out.reports
242         .mix(FASTP.out.reports)
243         .collect()
244     )
245 }

```

Choosing Between Snakemake and Nextflow

Both workflow managers are widely used in genomics. Key considerations:

- **Snakemake:** Python-based, file-centric (rules define input/output files), integrates with Conda environments natively, excellent for bioinformaticians comfortable with Python.
- **Nextflow:** Groovy/Java-based, dataflow-driven (channels pass data between processes), strong Docker/Singularity integration, excellent for cloud-native and HPC deployments.
- **nf-core:** A community-curated collection of Nextflow pipelines. The **nf-core/sarek** pipeline implements a production-grade WGS/WES variant calling workflow with extensive QC and reporting.

7.8 Long-Read Alignment and Assembly Workflow

For long-read WGS data, the alignment and variant calling tools differ:

- **Alignment:** minimap2 with preset `-ax map-hifi` for PacBio HiFi or `-ax map-ont` for Oxford Nanopore.
- **Small variant calling:** DeepVariant (PacBio model or ONT model), PEPPER-Margin-DeepVariant for ONT, Clair3.
- **Structural variant calling:** pbsv (PacBio), Sniffles2 (any long reads), cuteSV, SVIM.
- **Phasing:** WhatsHap for read-backed phasing, HiPhase for PacBio, SHAPEIT5 for statistical phasing at population scale.
- **De novo assembly:** hifiiasm for HiFi reads; Verkko for HiFi + ONT ultra-long; Flye for ONT-only assemblies.
- **Methylation:** modkit (PacBio), modbam2bed and modkit (ONT) for extracting base modification information.

7.9 Quality Control Metrics for WGS/WES

After alignment and before variant calling, it is essential to assess data quality. Key metrics include:

Table 7.4: Essential QC metrics for WGS and WES data.

Metric	Description	Typical Threshold
Mean coverage	Average number of reads covering each base	$\geq 30\times$ (WGS)
% bases $\geq 20\times$	Uniformity of coverage	$\geq 90\%$
% mapped reads	Fraction of reads aligning to reference	$\geq 95\%$
% properly paired	Fraction of read pairs with concordant alignment	$\geq 90\%$
% duplicate reads	PCR duplicate rate	$< 15\%$ (PCR-free: $< 5\%$)
Insert size	Fragment length distribution (peak and spread)	300–500 bp
GC bias	Deviation of coverage across GC content bins	Minimal deviation
Contamination	Cross-sample contamination (VerifyBamID)	$< 1\%$
Chimeric reads	Fraction of supplementary/split alignments	$< 5\%$
On-target rate (WES)	Fraction of reads on capture target	$\geq 70\%$

Tools for collecting these metrics include `samtools stats`, `Picard CollectWgsMetrics`, `Picard CollectHsMetrics` (for WES), `mosdepth` (fast coverage calculation), and `VerifyBamID2` (contamination estimation).

7.10 Summary

Chapter Summary

- **WGS** captures the entire genome including coding, non-coding, regulatory, and repetitive regions. It is the most comprehensive but most expensive approach, requiring 30× for germline and 60–100× for somatic applications.
- **WES** captures ~1–2% of the genome (protein-coding exons) at a fraction of the cost of WGS. It is the workhorse for Mendelian disease diagnostics but misses non-coding and structural variants.
- **Targeted panels** offer ultra-deep sequencing of curated gene sets, ideal for clinical diagnostics with well-defined genetic targets.
- **Long-read WGS** (PacBio HiFi, Oxford Nanopore) enables structural variant detection, repeat characterisation, haplotype phasing, and de novo assembly—capabilities that are limited or absent with short reads alone.
- The **T2T-CHM13** assembly and emerging **pangenome references** are transforming how we represent and analyse human genetic variation.
- Both **Snakemake** and **Nextflow** provide robust frameworks for building reproducible WGS/WES analysis pipelines, with community-curated pipelines (e.g., **nf-core/sarek**) available for immediate use.

Chapter 8

Variant Discovery and Interpretation

Every human genome carries approximately 4–5 million genetic variants relative to the reference sequence. The vast majority are benign polymorphisms inherited from our ancestors, but a small fraction can cause disease, influence drug response, or drive cancer. Identifying these variants from sequencing data—and distinguishing the pathogenic needles from the benign haystack—is one of the central challenges of modern genomics. This chapter provides a comprehensive treatment of variant types, calling algorithms, quality control, annotation, and clinical interpretation, with practical pipeline examples in both Snakemake and Nextflow.

8.1 Types of Genetic Variants

Genetic variants span a wide size range, from single nucleotide changes to chromosome-scale rearrangements. Understanding this spectrum is essential for selecting appropriate detection methods.

8.1.1 Single Nucleotide Variants (SNVs) and Single Nucleotide Polymorphisms (SNPs)

An SNV is a change of a single nucleotide at a specific genomic position. When an SNV is present in $\geq 1\%$ of a population, it is conventionally called a **single nucleotide polymorphism (SNP)**. SNVs are the most abundant variant class: a typical human genome contains ~ 4.1 – 4.5 million SNVs compared to the GRCh38 reference.

SNVs in coding regions are classified by their effect on the encoded protein:

Synonymous (silent) The nucleotide change does not alter the amino acid due to codon degeneracy.

Missense The change results in a different amino acid.

Nonsense The change creates a premature stop codon, truncating the protein.

Splice-site The change disrupts a canonical splice donor (GT) or acceptor (AG) site, altering mRNA splicing.

8.1.2 Insertions and Deletions (Indels)

Indels are small insertions or deletions, typically defined as 1–50 bp in length. A typical human genome contains $\sim 500,000$ – $600,000$ indels. Indels in coding regions can be:

Frameshift The insertion or deletion length is not a multiple of 3, shifting the reading frame of all downstream codons. Usually severely disruptive.

In-frame The length is a multiple of 3, adding or removing one or more amino acids without disrupting the reading frame.

Indel Representation Ambiguity

Indels in repetitive regions can be represented at multiple equivalent positions (left-aligned vs. right-aligned). This “indel normalisation” problem can cause the same biological variant to appear as different VCF records. Always use `bcftools norm` or `vt normalize` to left-align and trim indels before comparison or annotation. This is particularly critical when merging call sets from different tools.

8.1.3 Structural Variants (SVs)

Structural variants are genomic rearrangements typically defined as >50 bp. They affect more total bases per genome than SNVs and indels combined. Major SV classes include:

Deletions

Loss of a genomic segment. Detected as reduced coverage and discordant read pairs.

Duplications

Gain of a genomic segment (tandem or dispersed). Tandem duplications appear as increased coverage and characteristic split-read signatures.

Inversions

A genomic segment is reversed in orientation. Detected by discordant read-pair orientation.

Translocations

A genomic segment is moved to a different chromosomal location. Can be balanced (no net gain or loss) or unbalanced. Inter-chromosomal translocations are detected by read pairs mapping to different chromosomes.

Insertions

Novel sequence is inserted at a position. Includes mobile element insertions (Alu, L1, SVA), nuclear mitochondrial insertions (NUMTs), and viral integrations.

A typical human genome contains ~20,000–30,000 structural variants, collectively affecting ~10–20 Mb of sequence.

8.1.4 Copy Number Variants (CNVs)

CNVs are a subset of structural variants involving gains or losses of genomic segments, ranging from kilobases to megabases. They are detected through:

- **Read-depth analysis:** Increased or decreased coverage relative to the diploid expectation.
- **B-allele frequency (BAF):** Deviations from the expected 0/0.5/1 pattern at heterozygous sites indicate gains or losses.
- **Split reads and discordant pairs:** Provide breakpoint-level resolution.

8.1.5 Repeat Expansions

Short tandem repeats (STRs, also called microsatellites) are arrays of 1–6 bp repeat units. Pathogenic expansions of these repeats cause over 50 neurological and neuromuscular diseases:

- *HTT*: CAG expansion >36 repeats causes Huntington’s disease.
- *FMR1*: CGG expansion >200 repeats causes Fragile X syndrome.
- *C9orf72*: GGGGCC expansion (hundreds to thousands of repeats) is the most common genetic cause of ALS and frontotemporal dementia.

- *ATXN1/ATXN2/ATXN3*: CAG expansions cause various spinocerebellar ataxias.
- *DMPK*: CTG expansion causes myotonic dystrophy type 1.
- *RFC1*: AAGGG pentanucleotide expansion causes cerebellar ataxia with neuropathy and vestibular areflexia syndrome (CANVAS).

8.1.6 Mobile Element Insertions

Transposable elements are DNA sequences that can “jump” to new locations. Active human transposons include Alu elements (~300 bp), L1 (LINE-1, up to ~6 kb), and SVA elements. New insertions occur at a rate of approximately one Alu insertion per 20 births and one L1 insertion per 100 births, occasionally causing disease by disrupting genes or regulatory elements.

8.1.7 Complex Rearrangements

Chromothripsis

A catastrophic shattering of a chromosome (or chromosome arm) followed by random reassembly. Can generate dozens to hundreds of rearrangements in a single event. Common in certain cancers (e.g., medulloblastoma, osteosarcoma).

Chromoplexy

Chains of translocations involving multiple chromosomes, often observed in prostate cancer. Unlike chromothripsis, chromoplexy is thought to arise through coordinated rearrangement events.

Kataegis

Clusters of hypermutation (typically C>T and C>G transitions) co-localising with structural variant breakpoints, associated with APOBEC enzyme activity.

8.2 TikZ Diagram: Variant Types and Their Scale

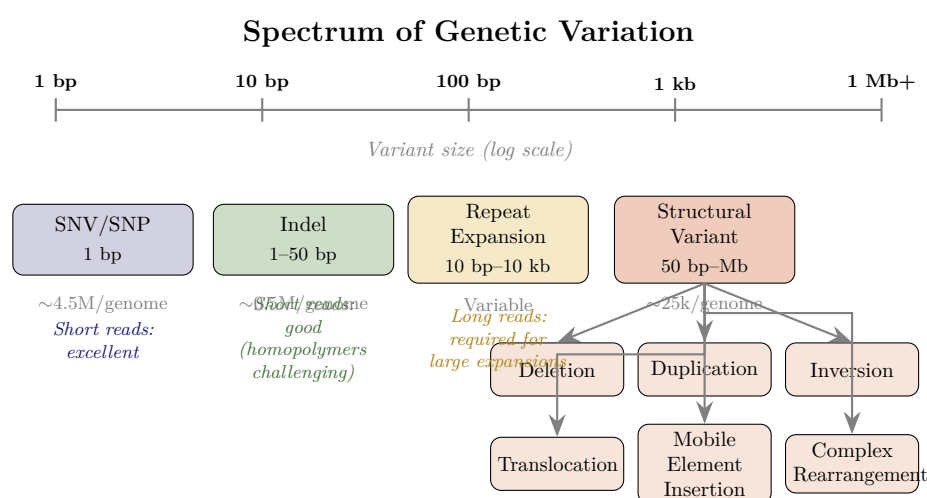


Figure 8.1: The spectrum of genetic variation, from single nucleotide variants (SNVs) to complex structural rearrangements. Each variant class requires different detection strategies and tools. Sizes are shown on a logarithmic scale.

8.3 Variant Calling Approaches

8.3.1 Germline Variant Calling

Germline variants are inherited and present in every cell of the body (or mosaic in a substantial fraction). They are called from a single sample, typically at 30× WGS or higher.

GATK HaplotypeCaller

The GATK HaplotypeCaller is the most widely used germline variant caller. Its approach:

1. **Active region detection:** Identifies regions with evidence of variation.
2. **Local de novo assembly:** Builds a De Bruijn graph of reads in each active region to determine plausible haplotypes.
3. **Pairwise read-haplotype alignment:** Aligns each read to each candidate haplotype using a pair-HMM algorithm.
4. **Genotype likelihood calculation:** Computes likelihoods for each possible diploid genotype.
5. **Output:** GVCF format (genomic VCF) with confidence scores for both variant and reference positions.

For cohort analysis, the GATK Best Practices recommend a three-step process: (1) **HaplotypeCaller** per sample to produce GVCFs, (2) **GenomicsDBImport** to consolidate GVCFs, and (3) **GenotypeGVCFs** for joint genotyping across all samples.

DeepVariant

Google’s DeepVariant reframes variant calling as an image classification problem:

1. Candidate variant sites are identified from pileup data.
2. For each candidate, a multi-channel “pileup image” is constructed encoding read bases, quality scores, mapping quality, strand, and other features.
3. A convolutional neural network (CNN) classifies each site as homozygous reference, heterozygous, or homozygous alternate.
4. Specialised models exist for Illumina WGS, Illumina WES, PacBio HiFi, ONT, and hybrid data.

DeepVariant consistently achieves the highest accuracy in the precisionFDA Truth Challenge and PrecisionFDA v2 benchmarks, particularly for indels.

Joint Calling and GLnexus

When analysing cohorts, joint calling improves sensitivity for rare variants and enables accurate genotyping at multi-allelic sites. **GLnexus** provides scalable joint genotyping from DeepVariant GVCFs:

- Handles hundreds of thousands of samples efficiently.
- Used by the All of Us program and UK Biobank for population-scale joint calling.
- Produces a project-level VCF with unified genotypes across all samples.

8.3.2 Somatic Variant Calling

Somatic variants arise in individual cells during the lifetime of an organism and are central to cancer genomics. They present unique challenges: they may be present at low variant allele frequencies (VAF), the tumor is heterogeneous (mixed clones), and there is no diploid genotype expectation.

Tumor/Normal Paired Analysis

The gold standard for somatic variant calling uses a **matched tumor–normal pair**: the tumor sample and a normal sample (e.g., blood, adjacent normal tissue) from the same patient. Variants present in the tumor but absent from the normal are classified as somatic. This design filters out germline variants and sequencing artefacts that appear in both samples.

Key Somatic Callers**Mutect2 (GATK)**

The most widely used somatic SNV/indel caller. Uses a Bayesian approach with a tumor-normal log-odds model. Supports tumor-only mode (with a panel of normals) and multi-sample calling. Includes `FilterMutectCalls` for artefact filtering using orientation bias models, contamination estimates, and a learned error model.

Strelka2

Developed by Illumina, Strelka2 is fast and accurate, using a mixture model for tumor–normal somatic variant calling. Also supports germline calling.

SomaticSniper

An earlier somatic caller based on a Bayesian genotype comparison between tumor and normal. Less commonly used now but conceptually important.

Consensus approach

Best practice in cancer genomics often involves running multiple callers and taking the intersection (high specificity) or union (high sensitivity) of their call sets, depending on the application.

8.3.3 Structural Variant Calling

SV calling remains more challenging than SNV/indel calling due to the diversity of SV types and the difficulty of resolving breakpoints in repetitive regions.

Table 8.1: Structural variant callers for short-read and long-read data.

Tool	Data Type	SV Types	Approach
Manta	Short reads	DEL, DUP, INS, INV, BND	Discordant pairs + split reads + local assembly
DELLY	Short reads	DEL, DUP, INV, TRA, INS	Paired-end + split-read + read-depth
Smooove	Short reads	DEL, DUP, INV, BND	Wraps LUMPY with better filtering
Sniffles2	Long reads	All SV types	Split-read alignment signatures
pbsv	PacBio	DEL, INS, DUP, INV, BND	CCS/HiFi signatures + local assembly
cuteSV	Long reads	DEL, INS, DUP, INV, BND	Clustering of SV signatures
SVIM	Long reads	All SV types	Multi-signature clustering

8.3.4 Copy Number Variant Calling

CNVnator

Read-depth-based CNV caller that divides the genome into bins and detects deviations from the expected diploid copy number using a mean-shift algorithm.

GATK gCNV (GermlineCNVCaller)

A cohort-based approach that models systematic coverage biases using a panel of normals and detects per-sample CNVs as deviations from the expected copy-number profile.

Spectre

Specifically designed for long-read CNV detection using coverage-based approaches.

ERDS, XHMM, CODEX2

Additional tools for WES-based CNV calling, which use the non-uniform exome capture data to infer copy number changes.

8.3.5 Short Tandem Repeat Analysis

ExpansionHunter

Developed by Illumina, uses paired-end and soft-clipped reads to estimate STR length, even when expansions exceed the read length. Supports a catalogue of >30 known pathogenic repeat loci.

STRique

Nanopore-specific STR quantification using raw signal data, enabling detection of very large expansions.

TRGT

PacBio's tandem repeat genotyping tool for HiFi data, providing allele-resolved repeat length and sequence composition.

STRling

Detects novel (uncatalogued) STR expansions from short-read data by identifying reads with excessive tandem repeat content.

8.3.6 Haplotype Phasing

Phasing determines which alleles at different variant sites reside on the same physical chromosome (haplotype).

WhatsHap

Read-backed phasing using long reads or linked reads. Formulates phasing as a weighted minimum error correction (wMEC) problem.

HiPhase

PacBio's phasing tool optimised for HiFi data, jointly phases SNVs, indels, and structural variants.

SHAPEIT5

Population-based statistical phasing using Hidden Markov Models. Leverages linkage disequilibrium patterns from large reference panels (e.g., TOPMed, 1000 Genomes) to phase common and rare variants.

8.4 Variant Filtering and Quality Control

Raw variant calls contain many false positives due to sequencing errors, mapping artefacts, and systematic biases. Rigorous filtering is essential.

8.4.1 GATK Variant Quality Score Recalibration (VQSR)

VQSR is a machine learning-based approach that uses known, validated variant sites as “truth” training data to build a Gaussian mixture model in the space of variant quality annotations:

1. Training truth sets: HapMap3 sites, Omni 2.5M array sites, 1000 Genomes high-confidence SNPs, Mills gold-standard indels.
2. Annotation features used: QualByDepth (QD), FisherStrand (FS), StrandOddsRatio (SOR), RMSMappingQuality (MQ), MappingQualityRankSumTest (MQRankSum), ReadPosRankSumTest (ReadPosRankSum), InbreedingCoeff.
3. The model assigns each variant a VQSLOD score (log odds of being a true variant).
4. Variants are filtered at a chosen sensitivity tranche (e.g., 99.5% sensitivity retains 99.5% of known truth set variants).

When VQSR Fails

VQSR requires a minimum of ~30 whole-genome samples (for SNPs) or ~1 whole-genome sample (for indels) to build a reliable model. For small sample sizes, targeted panels, or non-human organisms without comprehensive truth sets, fall back to **hard filtering**.

8.4.2 Hard Filters

When VQSR cannot be applied, GATK recommends the following hard filter thresholds for SNVs:

Table 8.2: GATK recommended hard filters for SNVs and indels.

Filter	SNV Threshold	Indel Threshold
QualByDepth (QD)	< 2.0	< 2.0
FisherStrand (FS)	> 60.0	> 200.0
StrandOddsRatio (SOR)	> 3.0	> 10.0
RMSMappingQuality (MQ)	< 40.0	—
MQRankSum	< -12.5	—
ReadPosRankSum	< -8.0	< -20.0

8.5 Statistical Foundations of Variant Calling

The variant calling algorithms described above rely on rigorous probabilistic models. This section formalises the key statistical machinery: the pair-HMM used for read-haplotype alignment, Bayesian genotyping, the VQSR Gaussian mixture model, and connections to population genetics.

8.5.1 Pair-HMM for Read-Haplotype Alignment

At the core of GATK HaplotypeCaller is a **pair hidden Markov model (pair-HMM)** that computes the likelihood of observing a read given a candidate haplotype. The pair-HMM defines three emitting states:

Match / Mismatch (M) Both the read and the haplotype advance by one position. The emission probability depends on whether the bases agree (match) or disagree (mismatch), weighted by the base quality score.

Insertion (I) The read advances by one base, but the haplotype does not. Models insertions in the read relative to the haplotype.

Deletion (D) The haplotype advances by one base, but the read does not. Models deletions in the read relative to the haplotype.

The transition probabilities between states are:

$$\begin{aligned} P(M \rightarrow M) &= 1 - P(M \rightarrow I) - P(M \rightarrow D), \\ P(M \rightarrow I) &= \delta_{\text{open}}, & P(M \rightarrow D) &= \delta_{\text{open}}, \\ P(I \rightarrow I) &= \epsilon_{\text{extend}}, & P(I \rightarrow M) &= 1 - \epsilon_{\text{extend}}, \\ P(D \rightarrow D) &= \epsilon_{\text{extend}}, & P(D \rightarrow M) &= 1 - \epsilon_{\text{extend}}, \end{aligned} \quad (8.1)$$

where δ_{open} is the gap-opening probability and ϵ_{extend} is the gap-extension probability. In GATK, these probabilities are derived from the base insertion and deletion quality scores encoded in the BAM file (tags BI and BD after BQSR).

The likelihood of read r given haplotype h is computed via dynamic programming over the pair-HMM matrix in $O(|r| \times |h|)$ time:

$$P(r \mid h) = \sum_{j=1}^{|h|} [M(|r|, j) + I(|r|, j)], \quad (8.2)$$

where $M(i, j)$, $I(i, j)$, and $D(i, j)$ are the forward-algorithm probabilities of aligning the first i bases of the read to the first j bases of the haplotype, ending in state M , I , or D , respectively. This computation is the most computationally expensive step in HaplotypeCaller and is hardware-accelerated on Intel (AVX-512), GPU (NVIDIA), and FPGA platforms.

Why Pair-HMM Instead of Standard Alignment?

Unlike Smith-Waterman or Needleman-Wunsch alignment, which return a single best alignment (and its score), the pair-HMM *sums over all possible alignments*, yielding the total probability of the read given the haplotype. This marginalisation is critical for accurate genotyping because it avoids committing to a single alignment path when multiple near-optimal alignments exist, particularly in repetitive or low-complexity regions.

8.5.2 Bayesian Genotyping

Given the pair-HMM likelihoods, genotype calling proceeds via Bayes' theorem. For a diploid site with candidate alleles $\{A_0, A_1, \dots, A_m\}$, the possible genotypes are all unordered pairs $G = A_i/A_j$ ($i \leq j$). The posterior probability of genotype G given observed data D (the set of reads overlapping the site) is:

$$P(G \mid D) = \frac{P(D \mid G) \cdot P(G)}{\sum_{G'} P(D \mid G') \cdot P(G')}, \quad (8.3)$$

where $P(G)$ is the genotype prior (often derived from population allele frequencies or a flat prior) and $P(D \mid G)$ is the genotype likelihood.

For a diploid genotype $G = A_i/A_j$, the data likelihood assuming conditionally independent reads $\{r_1, \dots, r_N\}$ is:

$$P(D \mid G = A_i/A_j) = \prod_{n=1}^N \left[\frac{1}{2} P(r_n \mid A_i) + \frac{1}{2} P(r_n \mid A_j) \right], \quad (8.4)$$

where $P(r_n | A_i)$ is the pair-HMM probability of read r_n given allele (haplotype) A_i .

VCF encoding of likelihoods. The VCF format encodes genotype likelihoods in two key fields:

PL (Phred-scaled Likelihood) Defined as $PL(G) = -10 \log_{10} P(D | G)$, normalised so that the most likely genotype has $PL = 0$. Lower PL indicates a more likely genotype.

GQ (Genotype Quality) The Phred-scaled confidence that the called genotype is correct, defined as the difference between the two lowest PL values:

$$GQ = PL_{\text{second-best}} - PL_{\text{best}} = PL_{\text{second-best}}, \quad (8.5)$$

since $PL_{\text{best}} = 0$ by convention. A GQ of 30 means the probability of the called genotype being wrong is $\leq 10^{-3}$.

Minimum GQ Thresholds for Different Applications

Clinical germline analysis typically requires $GQ \geq 20$ (99% confidence). For population genetics studies where genotype accuracy is critical (e.g., identity-by-descent analysis, kinship estimation), a threshold of $GQ \geq 30$ is common. For GWAS, imputation quality metrics (R^2 or INFO score) complement per-sample GQ.

8.5.3 VQSR: The Gaussian Mixture Model in Detail

Variant Quality Score Recalibration (introduced in Section 8.4) trains a **Gaussian mixture model (GMM)** to separate true variants from artefacts in a multi-dimensional annotation space. The procedure is as follows:

1. **Feature extraction:** Each variant site is described by a vector of quality annotations:
 - QualByDepth (QD): variant quality normalised by depth.
 - MappingQuality (MQ): root-mean-square mapping quality of reads.
 - FisherStrand (FS): Phred-scaled p -value of Fisher’s exact test for strand bias.
 - StrandOddsRatio (SOR): a more robust measure of strand bias using a symmetric odds ratio.
 - MQRankSum: rank-sum test comparing mapping qualities of reference vs. alternate reads.
 - ReadPosRankSum: rank-sum test comparing read positions of reference vs. alternate alleles (filters variants concentrated at read ends).
2. **GMM training:** The model fits K Gaussian components to the annotation vectors of known true-positive sites (training resources: HapMap 3, Omni 2.5M, 1000 Genomes high-confidence sites for SNPs; Mills and 1000 Genomes gold-standard indels). The probability density for variant $\mathbf{x}$ under the GMM is:

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k), \quad (8.6)$$

where π_k are mixing weights, $\boldsymbol{\mu}_k$ are component means, and $\boldsymbol{\Sigma}_k$ are covariance matrices. Parameters are estimated via Expectation-Maximisation (EM).

3. **VQSLOD scoring:** Each variant receives a log-odds score comparing its probability under the “true” GMM vs. the “false” (background) distribution:

$$VQSLOD = \log_{10} \frac{p(\mathbf{x} | \text{true})}{p(\mathbf{x} | \text{false})}. \quad (8.7)$$

4. **Sensitivity tranches:** Rather than applying a fixed VQSLOD threshold, GATK defines tranches by the fraction of known truth-set variants retained. A tranche of 99.5% means

the VQSLOD cutoff is set to retain 99.5% of HapMap3 sites. Lower tranches (e.g., 99.0%) are more stringent; higher tranches (e.g., 99.9%) retain more variants but include more false positives.

VQSR Annotation Interactions

VQSR annotations are not independent. For example, variants in low-mappability regions tend to have low MQ, elevated FS, and extreme MQRankSum simultaneously. The multivariate Gaussian model captures these correlations through the full covariance matrix Σ_k , which is why VQSR substantially outperforms univariate hard filters when sufficient training data are available.

8.5.4 Population Genetics Connections

Variant calling and quality control are deeply connected to population genetics theory. Several population-level statistics serve as powerful QC and analytical tools.

Hardy-Weinberg Equilibrium as a QC Filter

Under random mating, the genotype frequencies at a biallelic locus with allele frequencies p (reference) and $q = 1 - p$ (alternate) are expected to follow Hardy-Weinberg Equilibrium (HWE):

$$f(AA) = p^2, \quad f(Aa) = 2pq, \quad f(aa) = q^2, \quad (8.8)$$

where $p^2 + 2pq + q^2 = 1$. Departures from HWE—tested using a χ^2 or exact test—can indicate:

- **Genotyping errors:** Excess homozygotes suggest allelic dropout; excess heterozygotes suggest systematic sequencing artefacts or paralogous mapping.
- **Population stratification:** Mixing individuals from different populations inflates heterozygosity (Wahlund effect).
- **Selection:** Strong selection against homozygotes (e.g., sickle-cell trait in malaria-endemic regions) creates genuine HWE departure.

Applying HWE Filters in Practice

For QC of germline call sets:

1. Compute HWE p -values per site using PLINK2 (`-hardy`) or `bcftools +fill-tags -t HWE`.
2. Remove sites with extreme HWE deviation ($p < 10^{-6}$ is a common threshold for GWAS QC).
3. Apply HWE filters *within* each population stratum, not across the entire cohort, to avoid confounding with population structure.

Site Frequency Spectrum

The **site frequency spectrum (SFS)**, also called the allele frequency spectrum, is the distribution of allele counts across polymorphic sites in a sample. For n diploid individuals ($2n$ chromosomes), the SFS is a vector $\xi = (\xi_1, \xi_2, \dots, \xi_{2n-1})$, where ξ_i is the number of sites at which the derived (or alternate) allele appears exactly i times.

The SFS is a fundamental summary statistic because:

- Under the standard neutral model, the expected unfolded SFS follows $E[\xi_i] = \theta/i$, where $\theta = 4N_e\mu$ is the population-scaled mutation rate. This predicts an excess of rare variants (singletons, doubletons).

- Departures from the neutral expectation reveal demographic history (bottlenecks, expansions) and selection (purifying selection depletes rare variants; balancing selection creates excess intermediate-frequency variants).
- The SFS is also a sensitive diagnostic for data quality: an excess of singletons can indicate sequencing errors rather than rare biological variants.

F_{ST} for Population Differentiation

Fixation index F_{ST} quantifies the degree of genetic differentiation between populations. Using Weir and Cockerham's estimator:

$$F_{ST} = \frac{\sigma_a^2}{\sigma_a^2 + \sigma_b^2 + \sigma_w^2}, \quad (8.9)$$

where σ_a^2 is the variance component due to differences among populations, σ_b^2 is due to differences among individuals within populations, and σ_w^2 is within-individual variance. In the simpler Hudson estimator for two populations:

$$F_{ST} = 1 - \frac{\bar{\pi}_w}{\pi_b}, \quad (8.10)$$

where $\bar{\pi}_w$ is the average within-population nucleotide diversity and π_b is the between-population diversity.

$F_{ST} \approx 0$ No differentiation; allele frequencies are similar across populations.

$F_{ST} \approx 1$ Complete fixation of different alleles in different populations.

Genome-wide average in humans $F_{ST} \approx 0.10$ – 0.15 between continental populations.

Outlier loci Regions with F_{ST} significantly above the genome-wide background are candidates for local adaptation (e.g., *SLC24A5* for skin pigmentation, *LCT* for lactase persistence).

Tools for computing F_{ST} include PLINK2, VCFtools, pixy (which correctly handles missing data), and scikit-allel.

8.6 Variant Annotation

After calling and filtering, variants must be annotated with functional, population, and clinical information to enable interpretation.

8.6.1 Functional Annotation

Functional annotation tools predict the consequence of each variant on genes, transcripts, and proteins:

VEP (Ensembl Variant Effect Predictor)

The most comprehensive annotation tool. Annotates variants with consequences (missense, frameshift, splice donor, etc.), affected transcripts, protein positions, and integrates plugins for pathogenicity scores, population frequencies, regulatory annotations, and more. Outputs in VCF, tab-delimited, or JSON format.

SnpEff

Fast Java-based annotator that classifies variants by impact (HIGH, MODERATE, LOW, MODIFIER). Useful for rapid annotation of large variant sets.

ANNOVAR

Perl-based tool supporting extensive database annotations. Widely used in clinical and research pipelines.

8.6.2 Population Frequency Databases

Population frequency is one of the most powerful filters for identifying pathogenic variants: truly deleterious variants are rare.

Table 8.3: Major population frequency databases.

Database	Description	Samples	Coverage
gnomAD v4	Genome Aggregation Database. The largest publicly available collection of exome and genome variant frequencies.	>807,000	WES + WGS
1000 Genomes Phase 3	Foundational population genetics resource with 26 populations.	2,504	WGS (low-pass)
TOPMed	Trans-Omics for Precision Medicine. Deep WGS from diverse US populations.	>180,000	WGS (30×+)
UK Biobank	Exome sequencing of the UK Biobank cohort.	500,000	WES
ClinVar	Not a frequency database per se, but aggregates clinical significance classifications for variants.	—	—

Population Frequency Filtering

For rare Mendelian disease, filter out variants with a gnomAD population maximum allele frequency >0.01 (1%) for dominant disorders or >0.03 (3%) for recessive disorders. For ultra-rare conditions, use even stricter thresholds (<0.001). Always check population-specific frequencies, not just the global frequency, to avoid filtering out variants that are common in underrepresented populations.

8.6.3 Pathogenicity Prediction

CADD (Combined Annotation Dependent Depletion)

Integrates 63+ annotations into a single deleteriousness score (Phred-scaled). CADD ≥ 20 indicates the top 1% most deleterious variants; ≥ 30 the top 0.1%.

REVEL (Rare Exome Variant Ensemble Learner)

An ensemble method combining 13 individual pathogenicity predictors (SIFT, PolyPhen-2, MutationTaster, etc.). Optimised for rare missense variants. Score range: 0 (benign) to 1 (pathogenic); threshold ≥ 0.5 is commonly used.

AlphaMissense

A deep learning model from DeepMind that predicts pathogenicity of all possible human missense variants using protein structure and evolutionary information from AlphaFold. Achieves state-of-the-art performance.

SpliceAI

Predicts the impact of variants on splicing by modelling the pre-mRNA sequence context using a deep residual neural network. Delta scores ≥ 0.5 indicate high splicing impact.

ClinVar

An NCBI database aggregating variant–disease associations and clinical significance

classifications submitted by clinical laboratories and research groups.

OMIM

Online Mendelian Inheritance in Man. A curated catalogue of human genes and genetic disorders.

8.6.4 Clinical Variant Classification: ACMG/AMP Guidelines

The American College of Medical Genetics and Genomics (ACMG) and the Association for Molecular Pathology (AMP) jointly published a framework for classifying germline sequence variants into five tiers:

1. **Pathogenic**: Strong evidence that the variant causes disease.
2. **Likely pathogenic**: Greater than 90% probability of being disease-causing.
3. **Variant of uncertain significance (VUS)**: Insufficient evidence to classify.
4. **Likely benign**: Greater than 90% probability of being benign.
5. **Benign**: Strong evidence that the variant does not cause disease.

Classification uses a set of criteria (evidence codes) such as:

- **PVS1**: Null variant (nonsense, frameshift, splice site $\pm 1,2$) in a gene where loss-of-function is a known mechanism of disease.
- **PS1**: Same amino acid change as a previously established pathogenic variant.
- **PM2**: Absent from population databases (or at extremely low frequency).
- **PP3**: Computational evidence supports a deleterious effect (multiple in-silico tools agree).
- **BA1**: Allele frequency $>5\%$ in gnomAD (standalone benign).
- **BS1**: Allele frequency greater than expected for the disorder.

Tools such as **InterVar** and **Franklin** automate ACMG classification, but expert clinical review remains essential for final interpretation.

8.7 Cancer Genomics

Cancer genomics applies WGS, WES, and targeted panel sequencing to understand tumor biology and guide therapy.

8.7.1 Tumor Mutational Burden (TMB)

TMB is the total number of somatic non-synonymous mutations per megabase of coding sequence. It serves as a biomarker for response to immune checkpoint inhibitors (ICIs):

- **High TMB** (≥ 10 mutations/Mb): Associated with better response to ICIs (pembrolizumab, nivolumab) across multiple cancer types. FDA-approved as a pan-cancer biomarker.
- TMB is typically measured from WES or large gene panels (≥ 300 genes, ≥ 1 Mb target region).
- Cancers with highest TMB: melanoma (UV-induced), lung cancer (tobacco-induced), microsatellite-unstable cancers (MMR-deficient).

8.7.2 Microsatellite Instability (MSI)

MSI is caused by deficiency in DNA mismatch repair (MMR) genes (*MLH1*, *MSH2*, *MSH6*, *PMS2*). MSI-high (MSI-H) tumors accumulate thousands of somatic mutations, predominantly in microsatellite regions:

- Tools: **MSIsensor2**, **mSINGS**, **MANTIS**.
- MSI-H is an FDA-approved biomarker for pembrolizumab (Keytruda) across all solid tumors—the first tissue-agnostic cancer drug approval.

- Common in colorectal cancer (~15%), endometrial cancer (~30%), and gastric cancer.

8.7.3 Mutational Signatures

Mutational processes leave characteristic “fingerprints” in the genome—patterns of base substitutions, indels, and rearrangements. The COSMIC mutational signatures catalogue (v3.4) includes >100 signatures:

- **SBS1**: Clock-like, associated with 5mC deamination (C>T at CpG). Accumulates with age.
- **SBS4**: Tobacco smoking (C>A mutations).
- **SBS7a/b**: UV light (C>T at CC dinucleotides).
- **SBS6/15/20/26**: Mismatch repair deficiency.
- **SBS3**: Homologous recombination deficiency (BRCA1/2 deficiency)—clinically actionable for PARP inhibitor therapy.
- **SBS13/2**: APOBEC activity (C>T and C>G at TCA/TCT motifs).
- Tools: SigProfiler, MutationalPatterns, deconstructSigs.

8.7.4 Tumor Heterogeneity and Clonal Evolution

Tumors are not monolithic: they consist of genetically distinct subpopulations (clones) that evolve over time:

Clonal mutations

Present in all (or nearly all) cancer cells. These arose early in tumor development or represent the ancestral clone.

Subclonal mutations

Present in a fraction of cancer cells, representing branching evolution.

Variant allele frequency (VAF)

The fraction of reads supporting the alternate allele. In a diploid tumor, a clonal heterozygous mutation has $VAF \approx 0.5 \times \text{tumor purity}$. Subclonal mutations have lower VAF.

Tools

PyClone-VI, SciClone, PhyloWGS, MOBSTER reconstruct clonal architecture from VAF distributions.

8.7.5 Liquid Biopsy and Cell-Free DNA (cfDNA)

Liquid biopsy analyses cell-free DNA (cfDNA) circulating in blood, avoiding the need for invasive tissue biopsies:

- **Circulating tumor DNA (ctDNA)**: Tumor-derived cfDNA fragments, typically 0.01–10% of total cfDNA in cancer patients.
- **Applications**: Non-invasive genotyping, monitoring treatment response, detecting minimal residual disease (MRD), early cancer detection (e.g., Galleri multi-cancer early detection test).
- **Technical requirements**: Ultra-deep sequencing (10,000–100,000× with UMIs), error-suppressed variant calling, tumour-informed or tumour-naïve approaches.
- **Tools**: ctDNAtools, ichorCNA (estimates tumor fraction from cfDNA WGS), MRDetect.
- **Fragmentomics**: Analysis of cfDNA fragment size distributions, nucleosome positioning, and end motifs to infer tissue of origin and detect cancer.

8.8 Pharmacogenomics

Pharmacogenomics uses genetic information to predict individual drug response and guide medication selection and dosing.

Key Pharmacogenes

CYP2D6 Metabolises ~25% of all prescribed drugs. Highly polymorphic with >100 known alleles, gene deletions, and whole-gene duplications. Metaboliser phenotypes: poor, intermediate, normal (extensive), ultra-rapid.

CYP2C19 Affects metabolism of clopidogrel (antiplatelet), proton pump inhibitors, and antidepressants. Loss-of-function alleles (CYP2C19*2, *3) reduce clopidogrel activation.

DPYD Dihydropyrimidine dehydrogenase—metabolises fluoropyrimidine chemotherapy (5-FU, capecitabine). DPD deficiency causes life-threatening toxicity.

TPMT/NUDT15 Affect metabolism of thiopurine drugs (azathioprine, 6-mercaptopurine). Deficiency causes severe myelosuppression.

VKORC1 Affects sensitivity to warfarin (anticoagulant dosing).

HLA-B HLA-B*57:01 predicts abacavir hypersensitivity (HIV therapy); HLA-B*15:02 predicts carbamazepine-induced Stevens-Johnson syndrome.

Resources: PharmGKB database, CPIC (Clinical Pharmacogenetics Implementation Consortium) guidelines, DPWG (Dutch Pharmacogenetics Working Group) recommendations.

8.9 Decision Flowchart: Variant Calling Pipeline Selection

8.10 Practical Workflow: Variant Discovery Pipeline

The following Snakemake and Nextflow pipelines demonstrate a complete germline variant discovery workflow including calling, filtering, and annotation.

8.10.1 Snakemake Variant Discovery Pipeline

Listing 8.1: Snakemake pipeline for germline variant calling, filtering, and annotation.

```

1 # Snakefile for Variant Discovery and Annotation
2 # Assumes aligned, deduplicated BAM files as input
3
4 configfile: "config.yaml"
5
6 SAMPLES = config["samples"]
7 REF = config["reference"]
8 KNOWN_SITES = config["known_sites"]
9 VEP_CACHE = config["vep_cache"]
10 VEP_SPECIES = "homo_sapiens"
11
12 rule all:
13     input:
14         expand("results/annotated/{sample}.annotated.vcf.gz",
15             sample=SAMPLES),
16         expand("results/sv/{sample}.manta.vcf.gz", sample=SAMPLES),
17         expand("results/str/{sample}.eh.vcf.gz", sample=SAMPLES)
18
19 # --- GATK HaplotypeCaller (GVCF mode) ---

```

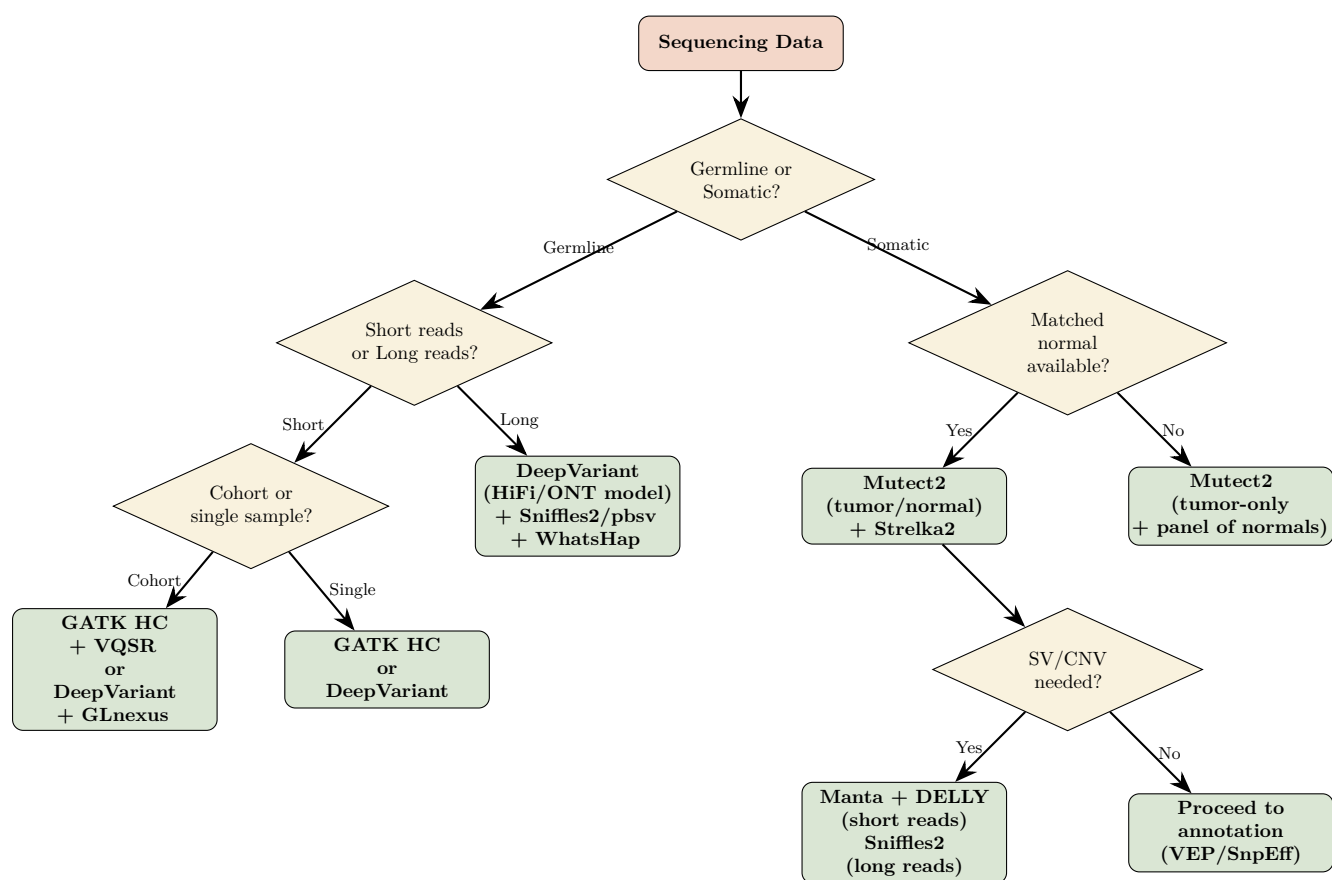


Figure 8.2: Decision flowchart for selecting appropriate variant calling tools based on the experimental design, data type, and variant classes of interest.

```

20 rule haplotypcaller:
21     input:
22         bam="data/bam/{sample}.dedup.bam",
23         ref=REF
24     output:
25         gvcf="results/gvcf/{sample}.g.vcf.gz"
26     threads: 4
27     shell:
28         """
29         gatk HaplotypeCaller \
30             -R {input.ref} -I {input.bam} \
31             -O {output.gvcf} -ERC GVCF \
32             --native-pair-hmm-threads {threads}
33         """
34
35 # --- GenotypeGVCFs ---
36 rule genotype_gvcfs:
37     input:
38         gvcf="results/gvcf/{sample}.g.vcf.gz",
39         ref=REF
40     output:
41         vcf="results/vcf/{sample}.raw.vcf.gz"
42     shell:
43         """
44         gatk GenotypeGVCFs \
45             -R {input.ref} -V {input.gvcf} \
46             -O {output.vcf}
47         """
48
49 # --- Hard Filtering (SNVs) ---
50 rule filter_snps:
51     input:
52         vcf="results/vcf/{sample}.raw.vcf.gz",
53         ref=REF
54     output:
55         vcf="results/filtered/{sample}.snps.filtered.vcf.gz"
56     shell:
57         """
58         gatk SelectVariants -R {input.ref} -V {input.vcf} \
59             --select-type-to-include SNP \
60             -O results/filtered/{wildcards.sample}.snps.raw.vcf.gz
61
62         gatk VariantFiltration \
63             -R {input.ref} \
64             -V results/filtered/{wildcards.sample}.snps.raw.vcf.gz \
65             --filter-expression "QD < 2.0" --filter-name "LowQD" \
66             --filter-expression "FS > 60.0" --filter-name "HighFS" \
67             --filter-expression "MQ < 40.0" --filter-name "LowMQ" \
68             --filter-expression "SOR > 3.0" --filter-name "HighSOR" \
69             --filter-expression "MQRankSum < -12.5" \
70             --filter-name "LowMQRS" \
71             --filter-expression "ReadPosRankSum < -8.0" \
72             --filter-name "LowRPRS" \
73             -O {output.vcf}
74         """
75
76 # --- Hard Filtering (Indels) ---
77 rule filter_indels:

```

```

78     input:
79         vcf="results/vcf/{sample}.raw.vcf.gz",
80         ref=REF
81     output:
82         vcf="results/filtered/{sample}.indels.filtered.vcf.gz"
83     shell:
84         """
85         gatk SelectVariants -R {input.ref} -V {input.vcf} \
86             --select-type-to-include INDEL \
87             -O results/filtered/{wildcards.sample}.indels.raw.vcf.gz
88
89         gatk VariantFiltration \
90             -R {input.ref} \
91             -V results/filtered/{wildcards.sample}.indels.raw.vcf.gz \
92             --filter-expression "QD < 2.0" --filter-name "LowQD" \
93             --filter-expression "FS > 200.0" --filter-name "HighFS" \
94             --filter-expression "ReadPosRankSum < -20.0" \
95             --filter-name "LowRPRS" \
96             --filter-expression "SOR > 10.0" --filter-name "HighSOR" \
97             -O {output.vcf}
98         """
99
100 # --- Merge filtered SNPs and indels ---
101 rule merge_filtered:
102     input:
103         snps="results/filtered/{sample}.snps.filtered.vcf.gz",
104         indels="results/filtered/{sample}.indels.filtered.vcf.gz"
105     output:
106         vcf="results/filtered/{sample}.merged.filtered.vcf.gz"
107     shell:
108         """
109         gatk MergeVcfs \
110             -I {input.snps} -I {input.indels} \
111             -O {output.vcf}
112         """
113
114 # --- VEP Annotation ---
115 rule vep_annotate:
116     input:
117         vcf="results/filtered/{sample}.merged.filtered.vcf.gz"
118     output:
119         vcf="results/annotated/{sample}.annotated.vcf.gz"
120     params:
121         cache=VEP_CACHE,
122         species=VEP_SPECIES
123     threads: 4
124     shell:
125         """
126         vep --input_file {input.vcf} \
127             --output_file {output.vcf} \
128             --format vcf --vcf --compress_output bgzip \
129             --cache --dir_cache {params.cache} \
130             --species {params.species} --assembly GRCh38 \
131             --everything --fork {threads} \
132             --offline --force_overwrite
133         """
134
135 # --- Structural Variant Calling (Manta) ---

```

```

136 rule manta_sv:
137     input:
138         bam="data/bam/{sample}.dedup.bam",
139         ref=REF
140     output:
141         vcf="results/sv/{sample}.manta.vcf.gz"
142     threads: 8
143     shell:
144         """
145         configManta.py \
146             --bam {input.bam} \
147             --referenceFasta {input.ref} \
148             --runDir results/sv/{wildcards.sample}_manta
149
150         results/sv/{wildcards.sample}_manta/runWorkflow.py \
151             -j {threads}
152
153         cp results/sv/{wildcards.sample}_manta/results/variants/\
154         diploidSV.vcf.gz {output.vcf}
155         """
156
157 # --- STR Expansion Analysis ---
158 rule expansion_hunter:
159     input:
160         bam="data/bam/{sample}.dedup.bam",
161         ref=REF,
162         catalog="resources/variant_catalog.json"
163     output:
164         vcf="results/str/{sample}.eh.vcf.gz"
165     shell:
166         """
167         ExpansionHunter \
168             --reads {input.bam} \
169             --reference {input.ref} \
170             --variant-catalog {input.catalog} \
171             --output-prefix results/str/{wildcards.sample}.eh
172
173         bgzip -c results/str/{wildcards.sample}.eh.vcf \
174             > {output.vcf}
175         tabix -p vcf {output.vcf}
176         """

```

8.10.2 Nextflow Variant Discovery Pipeline

Listing 8.2: Nextflow DSL2 pipeline for variant calling, filtering, and annotation.

```

1  #!/usr/bin/env nextflow
2  nextflow.enable.dsl=2
3
4  // Variant Discovery and Annotation Pipeline
5
6  params.bams      = "data/bam/*.dedup.bam"
7  params.ref       = "reference/hg38.fa"
8  params.vep_cache = "resources/vep_cache"
9  params.str_cat   = "resources/variant_catalog.json"
10 params.outdir    = "results"
11

```

```

12 process HAPLOTYPECALLER {
13     tag "${sample_id}"
14     publishDir "${params.outdir}/gvcf", mode: 'copy'
15     cpus 4
16
17     input:
18     tuple val(sample_id), path(bam), path(bai)
19     path ref
20     path ref_fai
21     path ref_dict
22
23     output:
24     tuple val(sample_id), path("${sample_id}.g.vcf.gz"),
25         path("${sample_id}.g.vcf.gz.tbi"),
26         emit: gvcf
27
28     script:
29     """
30     gatk HaplotypeCaller \
31         -R ${ref} -I ${bam} \
32         -O ${sample_id}.g.vcf.gz -ERC GVCF \
33         --native-pair-hmm-threads ${task.cpus}
34     """
35 }
36
37 process GENOTYPE_GVCFS {
38     tag "${sample_id}"
39     publishDir "${params.outdir}/vcf", mode: 'copy'
40
41     input:
42     tuple val(sample_id), path(gvcf), path(gvcf_tbi)
43     path ref
44     path ref_fai
45     path ref_dict
46
47     output:
48     tuple val(sample_id), path("${sample_id}.raw.vcf.gz"),
49         path("${sample_id}.raw.vcf.gz.tbi"),
50         emit: raw_vcf
51
52     script:
53     """
54     gatk GenotypeGVCFs \
55         -R ${ref} -V ${gvcf} \
56         -O ${sample_id}.raw.vcf.gz
57     """
58 }
59
60 process HARD_FILTER {
61     tag "${sample_id}"
62     publishDir "${params.outdir}/filtered", mode: 'copy'
63
64     input:
65     tuple val(sample_id), path(vcf), path(vcf_tbi)
66     path ref
67     path ref_fai
68     path ref_dict
69

```

```

70     output:
71     tuple val(sample_id),
72         path("${sample_id}.filtered.vcf.gz"),
73         path("${sample_id}.filtered.vcf.gz.tbi"),
74         emit: filtered_vcf
75
76     script:
77     """
78     # Filter SNPs
79     gatk SelectVariants -R ${ref} -V ${vcf} \
80         --select-type-to-include SNP \
81         -O ${sample_id}.snps.vcf.gz
82     gatk VariantFiltration -R ${ref} \
83         -V ${sample_id}.snps.vcf.gz \
84         --filter-expression "QD < 2.0" --filter-name "LowQD" \
85         --filter-expression "FS > 60.0" --filter-name "HighFS" \
86         --filter-expression "MQ < 40.0" --filter-name "LowMQ" \
87         --filter-expression "SOR > 3.0" --filter-name "HighSOR" \
88         -O ${sample_id}.snps.filt.vcf.gz
89
90     # Filter Indels
91     gatk SelectVariants -R ${ref} -V ${vcf} \
92         --select-type-to-include INDEL \
93         -O ${sample_id}.indels.vcf.gz
94     gatk VariantFiltration -R ${ref} \
95         -V ${sample_id}.indels.vcf.gz \
96         --filter-expression "QD < 2.0" --filter-name "LowQD" \
97         --filter-expression "FS > 200.0" --filter-name "HighFS" \
98         --filter-expression "SOR > 10.0" --filter-name "HighSOR" \
99         -O ${sample_id}.indels.filt.vcf.gz
100
101     # Merge
102     gatk MergeVcfs \
103         -I ${sample_id}.snps.filt.vcf.gz \
104         -I ${sample_id}.indels.filt.vcf.gz \
105         -O ${sample_id}.filtered.vcf.gz
106     """
107 }
108
109 process VEP_ANNOTATE {
110     tag "$sample_id"
111     publishDir "${params.outdir}/annotated", mode: 'copy'
112     cpus 4
113
114     input:
115     tuple val(sample_id), path(vcf), path(vcf_tbi)
116     path vep_cache
117
118     output:
119     tuple val(sample_id),
120         path("${sample_id}.annotated.vcf.gz"),
121         emit: annotated_vcf
122
123     script:
124     """
125     vep --input_file ${vcf} \
126         --output_file ${sample_id}.annotated.vcf.gz \
127         --format vcf --vcf --compress_output bgzip \

```

```

128     --cache --dir_cache ${vep_cache} \
129     --species homo_sapiens --assembly GRCh38 \
130     --everything --fork ${task.cpus} \
131     --offline --force_overwrite
132     """
133 }
134
135 process MANTA_SV {
136     tag "$sample_id"
137     publishDir "${params.outdir}/sv", mode: 'copy'
138     cpus 8
139
140     input:
141     tuple val(sample_id), path(bam), path(bai)
142     path ref
143     path ref_fai
144
145     output:
146     tuple val(sample_id),
147         path("${sample_id}.manta.sv.vcf.gz"),
148         emit: sv_vcf
149
150     script:
151     """
152     configManta.py \
153         --bam ${bam} \
154         --referenceFasta ${ref} \
155         --runDir manta_run
156     manta_run/runWorkflow.py -j ${task.cpus}
157     cp manta_run/results/variants/diploidSV.vcf.gz \
158         ${sample_id}.manta.sv.vcf.gz
159     """
160 }
161
162 process EXPANSION_HUNTER {
163     tag "$sample_id"
164     publishDir "${params.outdir}/str", mode: 'copy'
165
166     input:
167     tuple val(sample_id), path(bam), path(bai)
168     path ref
169     path str_catalog
170
171     output:
172     tuple val(sample_id),
173         path("${sample_id}.eh.vcf.gz"),
174         emit: str_vcf
175
176     script:
177     """
178     ExpansionHunter \
179         --reads ${bam} --reference ${ref} \
180         --variant-catalog ${str_catalog} \
181         --output-prefix ${sample_id}.eh
182     bgzip -c ${sample_id}.eh.vcf > ${sample_id}.eh.vcf.gz
183     tabix -p vcf ${sample_id}.eh.vcf.gz
184     """
185 }

```



```

186
187 // ---- Workflow ----
188 workflow {
189   // Input channels
190   bam_ch = Channel
191     .fromFilePairs(params.bams, size: 1) { file ->
192       file.name.replaceAll('.dedup.bam', '')
193     }
194     .map { id, files -> tuple(id, files[0],
195       file("${files[0]}.bai")) }
196
197   ref      = file(params.ref)
198   ref_fai  = file("${params.ref}.fai")
199   ref_dict = file(params.ref.replaceAll('.fa$', '.dict'))
200   vep_cache = file(params.vep_cache)
201   str_cat  = file(params.str_cat)
202
203   // Variant calling
204   HAPLOTYPECALLER(bam_ch, ref, ref_fai, ref_dict)
205   GENOTYPE_GVCFS(HAPLOTYPECALLER.out.gvcf,
206     ref, ref_fai, ref_dict)
207
208   // Filtering and annotation
209   HARD_FILTER(GENOTYPE_GVCFS.out.raw_vcf,
210     ref, ref_fai, ref_dict)
211   VEP_ANNOTATE(HARD_FILTER.out.filtered_vcf, vep_cache)
212
213   // SV calling and STR analysis (parallel)
214   MANTA_SV(bam_ch, ref, ref_fai)
215   EXPANSION_HUNTER(bam_ch, ref, str_cat)
216 }

```

8.11 Variant Calling Tool Comparison

Table 8.4: Comparison of variant calling tools for different variant classes and use cases.

Variant Class	Tool	Approach	Data Type	Key Strengths
Germline SNV/indel	GATK HC	Local assembly + pair-HMM	Short reads	Gold standard, cohort support
	DeepVariant	CNN image classification	Short/long reads	Highest accuracy, multi-platform
Somatic SNV/indel	Mutect2	Bayesian somatic model	Short reads	T/N and tumor-only modes
	Strelka2	Mixture model	Short reads	Fast, accurate
Structural variants	Manta	Assembly + PE/SR	Short reads	Fast, comprehensive
	Sniffles2	Split-read signatures	Long reads	Gold standard for LR SVs
	DELLY	PE + SR + RD	Short reads	Multi-algorithm, genotyping
CNVs	GATK gCNV	Cohort denoising	WES/WGS	Handles capture bias
	CNVnator	Read depth binning	WGS	Simple, effective
STR expansions	ExpansionHunter	Spanning/flanking reads	Short reads	Known locus catalogue
	TRGT	HiFi repeat analysis	PacBio HiFi	Allele-resolved, accurate
Phasing	WhatsHap	Read-backed wMEC	Long reads	Physical phasing
	SHAPEIT5	Statistical HMM	Any (population)	Population-scale

8.12 Summary

Chapter Summary

- Genetic variants range from single nucleotide changes (SNVs) to chromosome-scale structural rearrangements, each requiring specialised detection methods.
- **Germline variant calling** (GATK HaplotypeCaller, DeepVariant) identifies inherited variants; **somatic calling** (Mutect2, Strelka2) identifies cancer-specific mutations using tumor–normal pairs.
- **Structural variants and repeat expansions** require long reads or specialised short-read algorithms (Manta, ExpansionHunter) for reliable detection.
- Variant **filtering** (VQSR or hard filters) is essential to remove artefacts, and **annotation** (VEP, gnomAD, CADD, ClinVar) is required for biological and clinical interpretation.
- The **ACMG/AMP 5-tier classification** system provides a standardised framework for clinical variant interpretation.
- Cancer genomics applications include **TMB**, **MSI**, **mutational signatures**, **clonal analysis**, and **liquid biopsy** for non-invasive monitoring.
- **Pharmacogenomics** translates genetic variants in drug-metabolising enzymes and drug targets into actionable prescribing guidance.

Chapter 9

Metagenomics and Microbiome Sequencing

The world teems with microbial life. A single gram of soil contains billions of bacteria from thousands of species; the human gut harbours a community of trillions of microorganisms whose collective genome dwarfs our own. Traditionally, studying these communities required culturing individual organisms in the laboratory—a process that captures only an estimated 1% of environmental microbial diversity. Metagenomics bypasses this “great plate count anomaly” by sequencing DNA directly from environmental or clinical samples, revealing the full genetic repertoire of entire microbial communities. This chapter covers both amplicon-based and shotgun metagenomic approaches, along with the computational tools needed to extract biological insight from complex community data.

9.1 What Is Metagenomics?

Metagenomics is the study of genetic material recovered directly from environmental or host-associated samples. Unlike single-organism genomics, metagenomics captures DNA from all organisms present in a sample—bacteria, archaea, fungi, protists, viruses, and host cells—in a single experiment.

Metagenomics vs. Metatranscriptomics vs. Metaproteomics

Metagenomics Sequences total DNA. Reveals which organisms are present and their *functional potential* (all genes encoded). Cannot distinguish active from dormant organisms.

Metatranscriptomics Sequences total RNA (after rRNA depletion). Reveals which genes are *actively expressed* and by which organisms.

Metaproteomics Identifies proteins via mass spectrometry. Reveals which proteins are actually produced and in what abundance.

Each layer provides complementary information. This chapter focuses on DNA-level metagenomics with a brief treatment of metatranscriptomics.

9.1.1 The Microbiome: What It Is and Why It Matters

The term **microbiome** refers to the community of microorganisms (and their collective genomes) inhabiting a defined environment. Key microbiome niches include:

Human gut microbiome

The most intensively studied microbiome. Contains $\sim 10^{13}$ bacterial cells and $\sim 1,000$

species per individual. Influences digestion, immune system development, pathogen resistance, drug metabolism, and even neurological function via the gut–brain axis. Dysbiosis (microbial imbalance) is implicated in inflammatory bowel disease, obesity, type 2 diabetes, colorectal cancer, and depression.

Human skin microbiome

Varies dramatically by body site (moist vs. dry vs. sebaceous). Dominated by *Cutibacterium*, *Staphylococcus*, and *Corynebacterium*. Relevant to atopic dermatitis, acne, wound healing.

Human oral microbiome

>700 species. Key to dental caries, periodontal disease, and increasingly linked to systemic diseases (cardiovascular disease, Alzheimer’s disease).

Soil microbiome

The most diverse microbial ecosystem on Earth. Critical for nutrient cycling (nitrogen fixation, carbon decomposition), plant health, and agricultural productivity.

Marine microbiome

Oceanic microbes drive global biogeochemical cycles, producing ~50% of Earth’s oxygen via photosynthesis.

Built environment

Indoor microbiomes (hospitals, homes, spacecraft) affect human health and building integrity.

9.2 16S/18S/ITS Amplicon Sequencing

Amplicon sequencing targets a specific phylogenetic marker gene to identify and quantify organisms in a sample. It is the most cost-effective approach for profiling microbial community composition.

9.2.1 The 16S rRNA Gene

The 16S ribosomal RNA gene (~1,540 bp) is the most widely used phylogenetic marker for bacteria and archaea. It is ideal because:

- It is present in all bacteria and archaea (universal).
- It contains both highly **conserved regions** (for universal primer design) and **variable regions** (V1–V9) that differ between taxa, enabling taxonomic classification.
- Extensive reference databases exist (SILVA, Greengenes2) with >10 million curated sequences.

Variable Region Selection

The choice of which variable region(s) to amplify affects taxonomic resolution and which taxa are detected:

Primer Bias

No primer pair amplifies all taxa equally. Every primer set has biases—some taxa are underrepresented or missed entirely. The commonly used 515F/806R primers, for instance, have known biases against certain *Crenarchaeota* and *SAR11* marine bacteria. Always report which primer pair was used, and be cautious about comparing studies that used different primers.

Table 9.1: Commonly targeted 16S rRNA variable regions.

Region	Length	Notes	Common Primers
V1–V2	~300 bp	Good for <i>Staphylococcus</i> differentiation; poor for some Enterobacteriaceae	27F / 338R
V3–V4	~460 bp	Most widely used. Good overall taxonomic resolution. Standard for Illumina MiSeq 2×300 bp	341F / 805R
V4	~250 bp	Used in the Earth Microbiome Project. Compatible with 2×150 bp	515F / 806R
V4–V5	~400 bp	Better discrimination of archaea	515F / 926R
Full-length	~1,500 bp	Highest resolution. Requires long-read sequencing (PacBio, ONT)	27F / 1492R

9.2.2 Other Marker Genes

18S rRNA

The eukaryotic equivalent of 16S rRNA. Used for profiling eukaryotic microbes (protists, micro-algae, nematodes, fungi). Variable regions (V4 and V9 are common) provide taxonomic resolution.

ITS (Internal Transcribed Spacer)

The ITS1 and ITS2 regions flanking the 5.8S rRNA gene are the standard barcodes for fungi. ITS evolves faster than 18S, providing species-level resolution for many fungal taxa. Reference database: UNITE.

9.2.3 ASVs vs. OTUs

A critical methodological decision in amplicon analysis is how to define taxonomic units:

OTU (Operational Taxonomic Unit)

Sequences are clustered at a similarity threshold (traditionally 97% identity). Each cluster represents an “OTU.” This approach is tolerant of sequencing errors but sacrifices resolution and can merge distinct species. Tools: **USEARCH** (UPARSE algorithm), **VSEARCH**, **mothur**.

ASV (Amplicon Sequence Variant)

Error-correction algorithms model the sequencing error process and resolve sequences to single-nucleotide resolution. ASVs are exact biological sequences (after denoising), enabling comparison across studies, strain-level discrimination, and greater ecological precision. This is now the recommended approach. Tools: **DADA2**, **Deblur**.

Why ASVs Have Replaced OTUs

ASVs offer multiple advantages over OTUs:

- **Reproducibility:** ASVs are exact sequences, so results from different studies are directly comparable. OTUs depend on the clustering algorithm and reference database.
- **Resolution:** ASVs can distinguish sequences differing by a single nucleotide, enabling strain-level discrimination.
- **Reusability:** ASV tables can be merged across experiments; OTU tables cannot (different OTU IDs in different studies).
- **No arbitrary threshold:** OTU clustering at 97% is a historical convention with no biological basis for most taxa.

9.2.4 Amplicon Analysis Tools

QIIME 2

The most comprehensive amplicon analysis platform. Provides a plugin-based architecture for denoising (DADA2, Deblur), taxonomy assignment, diversity analysis, statistical testing, and visualisation. Produces interactive visualisations. The community standard for reproducible microbiome research.

DADA2

An R package for high-resolution amplicon denoising. The core algorithm models Illumina error profiles to distinguish true biological sequences from sequencing errors. Can also process PacBio and ONT full-length 16S data.

mothur

A long-established, comprehensive tool for amplicon analysis, particularly popular in the ecological microbiology community. Implements OTU-based workflows.

USEARCH / VSEARCH

Fast sequence analysis tools. VSEARCH is the open-source alternative to USEARCH.

9.2.5 Taxonomic Classification Databases

9.3 Shotgun Metagenomics

Shotgun metagenomics sequences all DNA in a sample (after fragmenting and library preparation), without any targeted amplification. This provides a far richer view of the community than amplicon sequencing.

9.3.1 Advantages over Amplicon Sequencing

- **All domains of life:** Detects bacteria, archaea, fungi, protists, and viruses simultaneously, without primer bias.
- **Functional potential:** Reveals the full gene content of the community—metabolic pathways, virulence factors, antibiotic resistance genes, secondary metabolite biosynthesis clusters.
- **Strain-level resolution:** Sufficient sequence diversity to distinguish closely related strains of the same species.
- **Metagenome-assembled genomes (MAGs):** Shotgun data can be assembled and binned to reconstruct near-complete genomes of uncultured organisms.

Table 9.2: Major taxonomic reference databases for amplicon sequencing.

Database	Target	Description	Taxonomy
SILVA	16S/18S/23S/28S	Comprehensive, curated rRNA database. >10M sequences. Updated regularly.	SILVA taxonomy
Greengenes2	16S	Rebuilt using whole-genome phylogenetics (WoL2 backbone). Provides more accurate taxonomy than the original Greengenes.	GTDB-aligned
UNITE	ITS	The standard for fungal ITS taxonomy. Species hypothesis concept for dynamic species delimitation.	UNITE taxonomy
RDP	16S/28S	Ribosomal Database Project. Provides a naive Bayesian classifier widely used for taxonomic assignment.	RDP taxonomy
GTDB	Whole genomes	Genome Taxonomy Database. Defines taxonomy using genome phylogeny (average nucleotide identity, RED). Not an rRNA database, but provides the taxonomic backbone increasingly used by 16S databases.	GTDB taxonomy

- **No amplification bias:** Coverage reflects actual DNA abundance (modulo extraction bias and differential lysis).

9.3.2 Library Preparation Considerations

- **DNA extraction:** Different extraction protocols lyse different organisms with varying efficiency. Bead-beating protocols improve recovery of Gram-positive bacteria and fungi but may shear DNA. Standardised protocols (e.g., the International Human Microbiome Standards, IHMS) are essential for reproducibility.
- **Host DNA depletion:** For host-associated samples (e.g., human gut biopsies, respiratory samples), host DNA can dominate the library. Depletion strategies include differential lysis (saponin for human cells + DNase treatment), CpG methylation-based depletion (NEBNext Microbiome DNA Enrichment Kit), and propidium monoazide (PMA) treatment to remove extracellular/dead cell DNA.
- **Sequencing depth:** Sufficient depth depends on community complexity. Simple communities (e.g., fermented foods, <50 species) may require only 1–5 million read pairs. Complex communities (e.g., soil, >10,000 species) may require 50–200 million read pairs for adequate functional profiling and MAG recovery.

Estimating Required Sequencing Depth

A useful rule of thumb for human gut metagenomics:

- **5–10M read pairs:** Adequate for taxonomic profiling of dominant species.
- **20–50M read pairs:** Sufficient for functional profiling and detection of less abundant taxa.
- **>100M read pairs:** Required for strain-level analysis, rare species detection, and high-quality MAG recovery.

For soil or other hyper-diverse environments, even 200M read pairs may only scratch the surface. Consider long-read metagenomics for improved assembly of complex communities.

9.3.3 Taxonomic Profiling

Taxonomic profiling assigns each read (or contig) to a taxonomic lineage.

Kraken2 / Bracken

Kraken2 uses exact k-mer matching against a prebuilt database to classify reads at very high speed (>1 million reads/second). Each read is assigned to the lowest common ancestor (LCA) of all species sharing its k-mers. **Bracken** (Bayesian Reestimation of Abundance with KrakEN) then redistributes reads from higher taxonomic levels to species level using a Bayesian model of read-length-specific probabilities. Database: standard Kraken2 database includes RefSeq bacteria, archaea, viruses, and human; custom databases can include fungi, protozoa, and UniVec.

MetaPhlAn 4

Uses a database of clade-specific marker genes (~5.1M markers from ~26,000 species) to estimate relative abundance. Highly specific but may miss novel species absent from the marker database. Reports relative abundance (not absolute counts).

mOTUs3

Uses single-copy phylogenetic marker genes (extracted from metagenomes and reference genomes) for taxonomic profiling. Can detect species lacking reference genomes via metagenomic operational taxonomic units.

Centrifuge

Memory-efficient classifier using compressed indices based on the FM-index and Burrows-Wheeler transform. Handles very large databases including all of NCBI nt.

9.3.4 Functional Profiling

Functional profiling determines what metabolic capabilities are encoded in the metagenome.

HUMAnN 3

The HMP Unified Metabolic Analysis Network tool provides community-level pathway and gene family abundance estimates. It first identifies species present (using MetaPhlAn), then maps reads against species-specific pangenomes, and finally performs translated search against UniRef for unmapped reads. Output: gene family abundances (UniRef90/UniRef50), pathway abundances (MetaCyc), and pathway coverage.

KEGG / KOfam

The Kyoto Encyclopedia of Genes and Genomes provides a hierarchical classification of biological functions (KO numbers, modules, pathways). **KofamScan** assigns KO numbers to predicted proteins using profile HMMs.

eggNOG-mapper

Fast functional annotation using pre-computed orthology assignments from the eggNOG database. Assigns GO terms, KEGG pathways, COG categories, and CAZyme annotations.

antiSMASH

Detects biosynthetic gene clusters (BGCs) for secondary metabolites (antibiotics, siderophores, pigments) in assembled metagenomes.

9.3.5 Metagenome-Assembled Genomes (MAGs)

One of the most powerful applications of shotgun metagenomics is the reconstruction of near-complete genomes from uncultured organisms:

Assembly**MEGAHIT**

Memory-efficient, fast metagenome assembler using succinct de Bruijn graphs with multiple k-mer sizes. Good for large, complex datasets. Lower contiguity than MetaSPAdes for simpler communities.

MetaSPAdes

Extension of the SPAdes assembler for metagenomic data. Uses multiple k-mer sizes with iterative assembly and error correction. Produces more contiguous assemblies for low-to-medium complexity communities but requires more memory.

metaFlye

Long-read metagenome assembler based on the Flye algorithm. Enables highly contiguous MAGs from PacBio HiFi or ONT data.

Binning

Binning groups assembled contigs into genome bins (putative genomes from individual organisms) using two types of signals:

- **Sequence composition:** Tetranucleotide frequency and GC content are species-specific signatures.
- **Differential coverage:** Contigs from the same genome have correlated abundance across multiple samples.

MetaBAT2

Fast, automatic binning using tetranucleotide frequency and coverage. Widely used and generally robust.

CONCOCT

Clusters contigs using Gaussian mixture models on sequence composition and coverage across multiple samples.

MaxBin2

Uses expectation-maximisation (EM) to bin contigs based on tetranucleotide frequency and coverage.

DAS Tool / DAS Tool

A meta-binner that combines results from multiple binning algorithms (MetaBAT2, CONCOCT, MaxBin2) to produce an optimised, non-redundant set of bins using a scoring function based on single-copy gene analysis.

SemiBin2

A deep learning-based binner that uses self-supervised contrastive learning for embedding contigs, followed by clustering. Outperforms traditional binners on many benchmarks.

Quality Assessment**CheckM / CheckM2**

Evaluates MAG quality using lineage-specific sets of single-copy marker genes. Reports **completeness** (fraction of expected markers found), **contamination** (fraction of duplicated markers indicating mixed genomes), and **strain heterogeneity**. CheckM2 uses machine learning for faster evaluation.

MIMAG standards

The Minimum Information about a Metagenome-Assembled Genome standard defines quality tiers:

- **High-quality:** >90% completeness, <5% contamination, 23S/16S/5S rRNA genes and ≥ 18 tRNAs present.
- **Medium-quality:** $\geq 50\%$ completeness, <10% contamination.
- **Low-quality:** <50% completeness, <10% contamination.

GUNC

Detects chimerism (contigs from different species mistakenly binned together) using a clade-separation score.

GTDB-Tk

Assigns taxonomy to MAGs using the Genome Taxonomy Database. Places genomes on a reference phylogeny using 120 bacterial and 53 archaeal marker genes.

9.3.6 Strain-Level Analysis

Shotgun metagenomics can resolve within-species genetic variation:

StrainPhlAn 4

Reconstructs species-specific consensus sequences from metagenomic reads by mapping to species-specific marker genes. Enables phylogenetic analysis of strains across samples and comparison with reference genomes. Useful for tracking transmission events (e.g., *Clostridioides difficile* in hospitals, mother-to-infant transmission).

inStrain

Performs ultra-detailed strain-level analysis: SNV profiling within populations, detection of multiple co-existing strains, gene-level coverage and variation, linkage analysis, and micro-diversity metrics. Operates on mapped reads against reference genomes or MAGs.

9.4 Metatranscriptomics

While metagenomics reveals *what is present*, metatranscriptomics reveals *what is active*. By sequencing total community RNA:

- **Active gene expression** can be quantified: which metabolic pathways are actually being used under specific conditions.
- **Transcriptional responses** to perturbations (diet change, antibiotic treatment, environmental stress) can be measured.
- **Discordance** between genomic potential and actual expression highlights regulatory mechanisms and niche-specific adaptation.

Key technical considerations:

- **rRNA depletion:** Ribosomal RNA constitutes >80% of total RNA. Efficient rRNA removal (Ribo-Zero, poly(A) depletion for eukaryotes) is essential.
- **RNA instability:** RNA degrades rapidly. Samples must be preserved immediately (RNAlater, flash freezing, or direct lysis).
- **Analysis:** Reads are mapped to the corresponding metagenome (if available) or reference genomes. HUMAnN can jointly analyse metatranscriptomic and metagenomic data to compute RNA/DNA ratios as a proxy for gene expression activity per gene copy.

9.5 Long-Read Metagenomics

Long-read sequencing is transforming metagenomics:

Full-length 16S sequencing

PacBio HiFi and ONT can sequence the entire ~1,500 bp 16S gene, providing far greater taxonomic resolution than short amplicons (often species-level vs. genus-level). PacBio's 16S kits produce HiFi reads with >Q30 accuracy.

Strain-resolved metagenomics

Long reads span entire operons and genomic islands, enabling assembly of complete or near-complete genomes directly from complex communities. Tools: **metaFlye**, **hifiasm-meta** (PacBio HiFi metagenome assembler producing MAGs with N50 >1 Mb).

Nanopore adaptive sampling

ONT's real-time selective sequencing ("Read Until") can deplete host DNA in real time by ejecting reads that map to the host reference, enriching for microbial DNA without any laboratory-based depletion step. This is particularly useful for clinical metagenomics where host DNA dominance is a major challenge.

Methylation-resolved metagenomics

Nanopore and PacBio HiFi detect DNA methylation natively. Since methylation patterns are species- and strain-specific, they can be used as an additional binning signal, dramatically improving MAG quality from complex communities.

9.6 Clinical Metagenomics

Clinical metagenomics applies unbiased sequencing to patient samples for pathogen detection and characterisation.

9.6.1 Pathogen Detection

- **Unbiased pathogen identification:** Shotgun metagenomics can detect *any* pathogen—bacterial, viral, fungal, parasitic—in a single test, without prior hypotheses. This is transformative for cases where conventional microbiology fails (culture-negative infections, rare/novel pathogens, polymicrobial infections).
- **Turnaround time:** With nanopore sequencing, pathogen identification can be achieved in hours (vs. days for culture). Real-time analysis with WIMP (What's In My Pot) or EPI2ME enables bedside diagnostics.
- **Cerebrospinal fluid (CSF) metagenomics:** Successfully used for diagnosing encephalitis and meningitis when conventional tests are negative. The UCSF Clinical Microbiology Laboratory has validated a clinical metagenomic next-generation sequencing (mNGS) test for CSF.

- **Respiratory metagenomics:** Used during the COVID-19 pandemic and for pneumonia of unknown aetiology.

9.6.2 Antimicrobial Resistance (AMR) Gene Detection

A critical application of metagenomics is the detection and surveillance of antimicrobial resistance genes:

CARD (Comprehensive Antibiotic Resistance Database)

Curated database of AMR genes, their mechanisms, and associated antibiotics. The Resistance Gene Identifier (RGI) tool detects AMR genes from metagenomic sequences.

ResFinder

Identifies acquired antimicrobial resistance genes from sequence data. Maintained by the Center for Genomic Epidemiology (CGE).

AMRFinderPlus

NCBI's tool for identifying AMR genes, stress response genes, and virulence factors using curated HMMs and BLAST. Covers >5,000 gene families.

deepARG

Deep learning-based AMR gene detection, capable of identifying novel ARGs that are divergent from known sequences.

AMR Gene Detection vs. Phenotypic Resistance

The presence of an AMR gene in a metagenome does not necessarily mean phenotypic resistance. The gene may be:

- On an extrachromosomal element not currently expressed.
- Truncated or carrying inactivating mutations.
- In a different organism than the pathogen of interest.
- Not expressed under the current conditions.

Always integrate genotypic AMR predictions with phenotypic susceptibility testing when making clinical decisions.

9.7 Diversity Metrics

Ecological diversity metrics quantify the structure of microbial communities.

9.7.1 Alpha Diversity (Within-Sample)

Alpha diversity measures the richness and evenness of a single community:

Observed features

The simplest richness measure: the count of distinct ASVs (or OTUs, species, etc.) in a sample. Highly sensitive to sequencing depth—rarefaction or statistical correction is essential.

Chao1 estimator

Estimates total richness including unobserved species using the number of singletons and doubletons: $S_{\text{Chao1}} = S_{\text{obs}} + \frac{f_1^2}{2f_2}$, where f_1 and f_2 are the counts of singletons and doubletons, respectively.

Shannon diversity index (H')

Combines richness and evenness: $H' = -\sum_{i=1}^S p_i \ln p_i$, where p_i is the relative abundance of species i . Higher values indicate more diverse communities. Ranges from 0 (single species) to $\ln(S)$ (all species equally abundant).

Simpson diversity index (D)

Measures the probability that two randomly drawn individuals belong to the same species: $D = \sum_{i=1}^S p_i^2$. Often reported as $1 - D$ (Simpson's diversity) or $1/D$ (inverse Simpson). More sensitive to dominant species than Shannon.

Faith's phylogenetic diversity (PD)

Sums the total branch length of the phylogenetic tree connecting all observed species in a sample. Captures evolutionary diversity.

9.7.2 Beta Diversity (Between-Sample)

Beta diversity measures how communities differ from one another:

Bray-Curtis dissimilarity

Quantifies compositional dissimilarity based on shared species abundances. Ranges from 0 (identical communities) to 1 (no shared species). Does not incorporate phylogenetic information.

Jaccard distance

Based on presence/absence only (ignores abundance). The fraction of species unique to one or the other sample.

UniFrac

Incorporates phylogenetic relationships between taxa:

- **Unweighted UniFrac:** Based on the fraction of unique branch lengths (presence/absence). Sensitive to rare taxa.
- **Weighted UniFrac:** Weights branches by the abundance of taxa on either side. Sensitive to dominant taxa.

Aitchison distance

Euclidean distance on centred log-ratio (CLR) transformed abundances. Appropriate for compositional data (which metagenomic relative abundance inherently is).

9.7.3 Ordination Methods

Ordination reduces high-dimensional distance matrices to 2–3 dimensions for visualisation:

PCoA (Principal Coordinates Analysis)

Eigendecomposition of a distance matrix (e.g., Bray-Curtis, UniFrac). Preserves distances between samples. The most common ordination for microbiome data.

NMDS (Non-metric Multidimensional Scaling)

Preserves rank-order of distances rather than absolute distances. More flexible than PCoA but iterative (requires convergence checking via stress values; stress < 0.2 is acceptable).

PCA (Principal Component Analysis)

Operates on raw abundance data (usually CLR-transformed for compositional data). Preserves Aitchison distances in Euclidean space.

UMAP

Non-linear dimensionality reduction. Preserves local structure better than PCoA/NMDS. Increasingly used for large microbiome datasets.

Statistical Testing for Microbiome Data

When comparing microbial communities between groups (e.g., disease vs. control):

- **PERMANOVA** (`adonis2` in R's **vegan** package): Tests whether group centroids differ significantly in multivariate space. The most widely used test for beta diversity differences.
- **ANCOM-BC2**: Differential abundance testing that accounts for the compositionality of microbiome data. Preferred over naive methods (t-test, Wilcoxon) that ignore compositionality.
- **MaAsLin2**: Multivariate association analysis with linear models. Handles confounders and repeated measures.
- **DESeq2** / **edgeR**: Borrowed from RNA-seq, these tools model count data with negative binomial distributions. Require careful adaptation for compositional microbiome data.

9.7.4 Mathematical Foundations of Diversity Indices

The alpha and beta diversity indices introduced above have precise mathematical formulations. Understanding these foundations helps practitioners choose appropriate metrics and interpret their behaviour under varying conditions.

Shannon and Simpson Indices: Properties and Interpretation

Let a community contain S species with relative abundances $p_1, p_2, \dots, p_S$ (where $\sum_{i=1}^S p_i = 1$). The **Shannon entropy** is:

$$H' = - \sum_{i=1}^S p_i \ln p_i, \quad (9.1)$$

with the convention $0 \ln 0 = 0$. Shannon entropy reaches its minimum of 0 when the community contains a single species ($p_1 = 1$, all other $p_i = 0$) and its maximum of $\ln S$ when all species are equally abundant ($p_i = 1/S$ for all i). The exponential of Shannon entropy, $e^{H'}$, gives the **effective number of species** (Hill number of order 1), which is more interpretable than the raw entropy value.

Simpson's index measures the probability that two randomly drawn individuals belong to the same species:

$$D = \sum_{i=1}^S p_i^2. \quad (9.2)$$

D ranges from $1/S$ (all species equally abundant) to 1 (single dominant species). Because higher values indicate *lower* diversity, derived forms are commonly used:

- **Simpson's diversity**: $1 - D$, ranging from 0 to $1 - 1/S$.
- **Inverse Simpson**: $1/D$, interpretable as the effective number of species (Hill number of order 2). Ranges from 1 to S .

Hill Numbers: A Unified Diversity Framework

All common diversity indices are special cases of the Hill number of order q :

$${}^qD = \left(\sum_{i=1}^S p_i^q \right)^{1/(1-q)}. \quad (9.3)$$

At $q = 0$, ${}^0D = S$ (species richness). At $q = 1$ (the limit as $q \rightarrow 1$), ${}^1D = e^H$ (exponential Shannon). At $q = 2$, ${}^2D = 1/D$ (inverse Simpson). Increasing q gives progressively more weight to abundant species and less to rare ones. Plotting diversity as a function of q yields a *diversity profile* that captures the full richness–evenness spectrum.

Weighted UniFrac: Phylogenetic Beta Diversity

The weighted UniFrac distance incorporates phylogenetic information and abundance data. Given a phylogenetic tree with branches $i = 1, \dots, B$ of lengths b_i , and two samples A and B whose proportional abundances descending from branch i are A_i and B_i respectively, the weighted UniFrac is:

$$U_w = \frac{\sum_{i=1}^B b_i |A_i - B_i|}{\sum_{i=1}^B b_i (A_i + B_i)}, \quad (9.4)$$

where b_i is the length of branch i in the phylogenetic tree, A_i is the fraction of sample A 's total abundance descending from branch i , and B_i is the analogous fraction for sample B . The numerator sums the abundance-weighted branch lengths unique to either sample, while the denominator normalises by the total abundance-weighted tree. The metric ranges from 0 (identical communities) to 1 (no shared lineages).

Choosing Between Unweighted and Weighted UniFrac

Unweighted UniFrac is sensitive to presence/absence differences and thus captures rare taxa effectively—use it when rare community members are biologically important (e.g., pathogen detection). **Weighted UniFrac** is sensitive to abundance differences in dominant taxa—use it when the dominant community structure is of primary interest (e.g., diet-driven changes in the gut microbiome). For a comprehensive analysis, report both.

9.7.5 MAG Quality Standards: MIMAG Tiers in Detail

The **Minimum Information about a Metagenome-Assembled Genome (MIMAG)** standards, published by the Genomic Standards Consortium, provide community-agreed quality tiers for MAGs. These standards are essential for assessing whether a MAG is suitable for downstream analyses such as phylogenomics, metabolic reconstruction, or submission to public databases.

Table 9.3: MIMAG quality tiers for metagenome-assembled genomes.

Quality Tier	Completeness	Contamination	rRNA (23S, 16S, 5S)	tRNAs (≥ 18)	
High quality	>90%	<5%	All present	Yes	Phylog
Medium quality	$\geq 50\%$	<10%	Partial or absent	Not required	Me
Low quality	<50%	<10%	Not assessed	Not required	Gene-

CheckM Workflow for MAG Quality Assessment

CheckM (and its successor **CheckM2**) estimates MAG completeness and contamination using lineage-specific sets of single-copy marker genes:

1. **Gene prediction:** Open reading frames are predicted from the MAG contigs using **Prodigal**.
2. **Marker gene identification:** Predicted proteins are searched against a database of 43 universal single-copy marker genes (for the initial placement) and up to ~200–400 lineage-specific marker genes.
3. **Phylogenetic placement:** The MAG is placed on a reference genome tree to identify the most appropriate lineage-specific marker set.
4. **Completeness estimation:** The fraction of expected lineage-specific marker genes detected in the MAG. If 108 out of 120 expected markers are found, completeness is 90%.
5. **Contamination estimation:** The fraction of marker genes present in more than one copy (indicating that contigs from multiple organisms were incorrectly binned together). Contamination is reported separately from strain heterogeneity (multiple copies from the same species vs. different species).

Contamination vs. Strain Heterogeneity

A MAG with 8% marker gene duplication may have 8% contamination (foreign contigs) or 8% strain heterogeneity (multiple strains of the same species). **CheckM** attempts to distinguish these by examining the amino acid identity of duplicated markers: high identity (>90%) suggests strain heterogeneity, while low identity suggests true contamination. The distinction matters: strain-heterogeneous MAGs may still be biologically meaningful, while contaminated MAGs require re-binning or manual curation.

9.7.6 HUMAnN Pathway Abundance Model

HUMAnN 3 (HMP Unified Metabolic Analysis Network) computes community-level functional profiles through a three-stage pipeline that produces **stratified** (species-attributed) pathway and gene-family abundances. Understanding the underlying model clarifies how to interpret its output.

Tiered Search Strategy

1. **Species identification:** **MetaPhlAn 4** identifies which species are present in the sample and their relative abundances.
2. **Nucleotide-level mapping:** Reads are mapped to the pangenomes of identified species using **Bowtie2**. Each mapping read is attributed to a specific gene family within a specific species—this is the “known known” tier.
3. **Translated search:** Reads that fail to map at the nucleotide level are subjected to translated (protein-level) search against the UniRef90 or UniRef50 database using **DIAMOND**. This captures gene families from organisms not in the pangenome database—the “known unknown” tier.
4. **Unclassified:** Reads that fail both searches are reported as unclassified.

Gene Family Abundance Calculation

For each gene family g in species s , the abundance is computed as reads per kilobase (RPK):

$$\text{RPK}(g, s) = \frac{N(g, s)}{L(g, s)/1000}, \quad (9.5)$$

where $N(g, s)$ is the number of reads mapping to gene family g in species s , and $L(g, s)$ is the effective gene length in bases. RPK values are then normalised to copies per million (CPM) across all gene families, yielding community-normalised abundances.

When multiple genes within a family have varying coverage, HUMAnN uses the **harmonic mean** rather than the arithmetic mean to estimate family-level abundance. The harmonic mean down-weights genes with partial coverage, preventing a gene family from being reported as abundant when only a fragment is detected:

$$\bar{x}_{\text{harmonic}} = \frac{n}{\sum_{i=1}^n \frac{1}{x_i}}, \quad (9.6)$$

where x_i are the RPK values of individual genes within the family and n is the number of genes.

Pathway Abundance and Coverage

Pathway abundance in HUMAnN is computed using the MetaCyc pathway database. For a pathway comprising reactions $\{r_1, r_2, \dots, r_m\}$, the pathway abundance is estimated using a structured approach that accounts for pathway topology (linear chains, branches, optional reactions):

- **Required reactions:** Reactions essential for pathway function must all be present. The pathway abundance is limited by the least abundant required reaction (analogous to Liebig's law of the minimum).
- **Optional reactions:** Reactions that can be bypassed contribute to abundance only when present.
- **Pathway coverage:** The fraction of required reactions that are detected above a minimum abundance threshold. A pathway with high abundance but low coverage may reflect a spurious mapping to a few abundant gene families rather than a genuinely active pathway.

Interpreting HUMAnN Stratified Output

HUMAnN produces three output files per sample:

1. **Gene families** (`_genefamilies.tsv`): Gene family abundance in RPK, stratified by contributing species. Community totals and per-species contributions are reported on separate lines.
2. **Pathway abundance** (`_pathabundance.tsv`): MetaCyc pathway abundance in RPK, stratified by species.
3. **Pathway coverage** (`_pathcoverage.tsv`): Fraction of each pathway's reactions detected (range 0–1).

Use `humann_renorm_table` to convert RPK to copies per million (CPM) or relative abundance before cross-sample comparisons. Use `humann_join_tables` to merge per-sample output into a single community-wide table.

9.8 TikZ Diagram: Amplicon vs. Shotgun Metagenomics

9.9 Comprehensive Tool Comparison

9.10 Practical Workflow: Metagenomics Pipelines

9.10.1 Snakemake Shotgun Metagenomics Pipeline

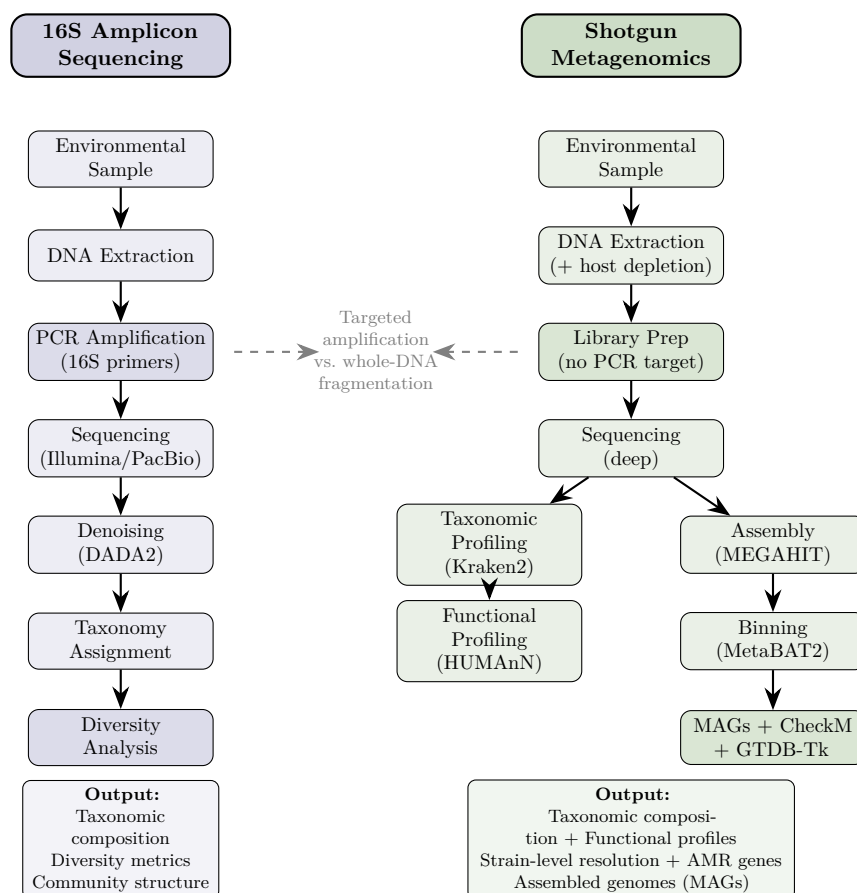


Figure 9.1: Comparison of 16S amplicon sequencing and shotgun metagenomics workflows. Amplicon sequencing uses targeted PCR amplification of phylogenetic markers, while shotgun metagenomics sequences all DNA, enabling functional profiling, strain-level resolution, and genome-resolved metagenomics.

Table 9.4: Comparison of key metagenomics analysis tools by application.

Application	Tool	Approach	Key Features
Amplicon denoising	DADA2	Error model-based denoising	ASV-level resolution, multi-platform
	Deblur	Sequence-error correction	Fast, conservative
Amplicon platform	QIIME 2	Plugin-based framework	Comprehensive, reproducible, interactive viz
	mothur	Monolithic pipeline	OTU-focused, ecological community
Taxonomic profiling	Kraken2/Bracken	k-mer matching + Bayesian reestimation	Fastest, flexible databases
	MetaPhlAn 4	Clade-specific markers	High specificity, lower sensitivity
Functional profiling	mOTUs3	Phylogenetic markers	Detects novel species
	HUMAnN 3	Species-stratified pathway analysis	Community + species-level functions
	eggNOG-mapper	Orthology-based annotation	Fast, comprehensive
Assembly	MEGAHIT	Succinct DBG, multi-k	Fast, memory-efficient
	MetaSPAdes	Multi-k iterative assembly	Higher contiguity
Binning	MetaBAT2	TNF + coverage	Fast, automatic
	SemiBin2	Deep learning embeddings	State-of-the-art accuracy
MAG quality	DAS Tool	Consensus of multiple binners	Optimised non-redundant bins
	CheckM2	ML-based marker gene analysis	Fast, accurate
AMR detection	GUNC	Chimerism detection	Catches contamination
	CARD/RGI	Curated AMR ontology	Gold standard for AMR
	AMRFinderPlus	HMM + BLAST	NCBI-maintained, comprehensive

Listing 9.1: Snakemake pipeline for shotgun metagenomics: QC, taxonomic profiling, functional profiling, assembly, and binning.

```

1 # Snakefile for Shotgun Metagenomics Analysis
2 configfile: "config.yaml"
3
4 SAMPLES = config["samples"]
5 KRAKEN_DB = config["kraken2_db"]
6 METAPHLAN_DB = config["metaphlan_db"]
7 REF_HOST = config["host_reference"] # for host removal
8
9 rule all:
10     input:
11         expand("results/taxonomy/{sample}.kraken2.report",
12               sample=SAMPLES),
13         expand("results/taxonomy/{sample}.metaphlan.txt",
14               sample=SAMPLES),
15         expand("results/functional/{sample}_pathabundance.tsv",
16               sample=SAMPLES),
17         expand("results/mags/{sample}/checkm_results.tsv",
18               sample=SAMPLES),
19         "results/multiqc/multiqc_report.html"
20
21 # --- QC and Host Removal ---
22 rule fastp_qc:
23     input:
24         r1="data/fastq/{sample}_R1.fastq.gz",
25         r2="data/fastq/{sample}_R2.fastq.gz"
26     output:
27         r1="results/trimmed/{sample}_R1.trimmed.fastq.gz",
28         r2="results/trimmed/{sample}_R2.trimmed.fastq.gz",
29         json="results/qc/{sample}.fastp.json",
30         html="results/qc/{sample}.fastp.html"
31     threads: 4
32     shell:
33         """
34         fastp -i {input.r1} -I {input.r2} \
35             -o {output.r1} -O {output.r2} \
36             --json {output.json} --html {output.html} \
37             --qualified_quality_phred 20 \
38             --length_required 50 \
39             --thread {threads}
40         """
41
42 rule remove_host:
43     input:
44         r1="results/trimmed/{sample}_R1.trimmed.fastq.gz",
45         r2="results/trimmed/{sample}_R2.trimmed.fastq.gz",
46         ref=REF_HOST
47     output:
48         r1="results/hostfree/{sample}_R1.fastq.gz",
49         r2="results/hostfree/{sample}_R2.fastq.gz"
50     threads: 8
51     shell:
52         """
53         bowtie2 -x {input.ref} -1 {input.r1} -2 {input.r2} \
54             --very-sensitive -p {threads} \
55             --un-conc-gz results/hostfree/{wildcards.sample}_R%.fastq.gz \
56             -S /dev/null 2> results/qc/{wildcards.sample}.host_stats.txt

```

```

57     """
58
59 # --- Taxonomic Profiling ---
60 rule kraken2:
61     input:
62         r1="results/hostfree/{sample}_R1.fastq.gz",
63         r2="results/hostfree/{sample}_R2.fastq.gz"
64     output:
65         report="results/taxonomy/{sample}.kraken2.report",
66         output="results/taxonomy/{sample}.kraken2.output"
67     params:
68         db=KRAKEN_DB
69     threads: 8
70     shell:
71         """
72         kraken2 --db {params.db} \
73             --paired {input.r1} {input.r2} \
74             --threads {threads} \
75             --report {output.report} \
76             --output {output.output} \
77             --confidence 0.1
78         """
79
80 rule bracken:
81     input:
82         report="results/taxonomy/{sample}.kraken2.report"
83     output:
84         bracken="results/taxonomy/{sample}.bracken.txt"
85     params:
86         db=KRAKEN_DB
87     shell:
88         """
89         bracken -d {params.db} \
90             -i {input.report} \
91             -o {output.bracken} \
92             -r 150 -l S
93         """
94
95 rule metaphlan:
96     input:
97         r1="results/hostfree/{sample}_R1.fastq.gz",
98         r2="results/hostfree/{sample}_R2.fastq.gz"
99     output:
100         profile="results/taxonomy/{sample}.metaphlan.txt"
101     params:
102         db=METAPHLAN_DB
103     threads: 8
104     shell:
105         """
106         metaphlan {input.r1},{input.r2} \
107             --input_type fastq \
108             --bowtie2db {params.db} \
109             --nproc {threads} \
110             -o {output.profile}
111         """
112
113 # --- Functional Profiling ---
114 rule humann:

```

```

115     input:
116         r1="results/hostfree/{sample}_R1.fastq.gz",
117         r2="results/hostfree/{sample}_R2.fastq.gz"
118     output:
119         genefamilies="results/functional/{sample}_genefamilies.tsv",
120         pathabundance="results/functional/{sample}_pathabundance.tsv",
121         pathcoverage="results/functional/{sample}_pathcoverage.tsv"
122     threads: 8
123     shell:
124         """
125         cat {input.r1} {input.r2} >
126         results/functional/{wildcards.sample}_concat.fastq.gz
127         humann --input results/functional/{wildcards.sample}_concat.fastq.gz \
128             --output results/functional/{wildcards.sample}_humann/ \
129             --threads {threads}
130         mv results/functional/{wildcards.sample}_humann/*_genefamilies.tsv \
131             {output.genefamilies}
132         mv results/functional/{wildcards.sample}_humann/*_pathabundance.tsv \
133             {output.pathabundance}
134         mv results/functional/{wildcards.sample}_humann/*_pathcoverage.tsv \
135             {output.pathcoverage}
136         """
137 # --- Assembly and Binning ---
138 rule megahit_assembly:
139     input:
140         r1="results/hostfree/{sample}_R1.fastq.gz",
141         r2="results/hostfree/{sample}_R2.fastq.gz"
142     output:
143         contigs="results/assembly/{sample}/final.contigs.fa"
144     threads: 16
145     shell:
146         """
147         megahit -1 {input.r1} -2 {input.r2} \
148             -o results/assembly/{wildcards.sample} \
149             -t {threads} --min-contig-len 1000
150         """
151
152 rule map_to_assembly:
153     input:
154         r1="results/hostfree/{sample}_R1.fastq.gz",
155         r2="results/hostfree/{sample}_R2.fastq.gz",
156         contigs="results/assembly/{sample}/final.contigs.fa"
157     output:
158         bam="results/assembly/{sample}/mapped.sorted.bam"
159     threads: 8
160     shell:
161         """
162         bowtie2-build {input.contigs} \
163             results/assembly/{wildcards.sample}/contigs_idx
164         bowtie2 -x results/assembly/{wildcards.sample}/contigs_idx \
165             -1 {input.r1} -2 {input.r2} \
166             -p {threads} --very-sensitive \
167             | samtools sort -@ 4 -o {output.bam} -
168         samtools index {output.bam}
169         """
170
171 rule metabat2_binning:

```



```

172 input:
173     contigs="results/assembly/{sample}/final.contigs.fa",
174     bam="results/assembly/{sample}/mapped.sorted.bam"
175 output:
176     bins=directory("results/mags/{sample}/bins")
177 threads: 8
178 shell:
179     """
180     jgi_summarize_bam_contig_depths --outputDepth \
181         results/mags/{wildcards.sample}/depth.txt {input.bam}
182     metabat2 -i {input.contigs} \
183         -a results/mags/{wildcards.sample}/depth.txt \
184         -o {output.bins}/bin \
185         -t {threads} --minContig 1500
186     """
187
188 rule checkm:
189     input:
190         bins="results/mags/{sample}/bins"
191     output:
192         results="results/mags/{sample}/checkm_results.tsv"
193     threads: 8
194     shell:
195         """
196         checkm2 predict \
197             --input {input.bins} \
198             --output-directory results/mags/{wildcards.sample}/checkm \
199             --threads {threads} -x fa
200         cp results/mags/{wildcards.sample}/checkm/quality_report.tsv \
201             {output.results}
202         """
203
204 rule multiqc:
205     input:
206         expand("results/qc/{sample}.fastp.json", sample=SAMPLES)
207     output:
208         "results/multiqc/multiqc_report.html"
209     shell:
210         """
211         multiqc results/qc/ -o results/multiqc/ --force
212         """

```

9.10.2 Nextflow Shotgun Metagenomics Pipeline

Listing 9.2: Nextflow DSL2 pipeline for shotgun metagenomics analysis.

```

1  #!/usr/bin/env nextflow
2  nextflow.enable.dsl=2
3
4  // Shotgun Metagenomics Analysis Pipeline
5
6  params.reads      = "data/fastq/*_{R1,R2}.fastq.gz"
7  params.host_ref   = "reference/human_bowtie2_idx"
8  params.kraken_db  = "databases/kraken2_standard"
9  params.outdir     = "results"
10
11 process FASTP {

```

```

12     tag "$sample_id"
13     publishDir "${params.outdir}/trimmed", mode: 'copy'
14     cpus 4
15
16     input:
17     tuple val(sample_id), path(reads)
18
19     output:
20     tuple val(sample_id),
21         path("${sample_id}_R{1,2}.trimmed.fastq.gz"),
22         emit: trimmed
23     path("${sample_id}.fastp.json"), emit: report
24
25     script:
26     """
27     fastp -i ${reads[0]} -I ${reads[1]} \
28         -o ${sample_id}_R1.trimmed.fastq.gz \
29         -O ${sample_id}_R2.trimmed.fastq.gz \
30         --json ${sample_id}.fastp.json \
31         --qualified_quality_phred 20 \
32         --length_required 50 \
33         --thread ${task.cpus}
34     """
35 }
36
37 process REMOVE_HOST {
38     tag "$sample_id"
39     publishDir "${params.outdir}/hostfree", mode: 'copy'
40     cpus 8
41
42     input:
43     tuple val(sample_id), path(reads)
44     path host_idx
45
46     output:
47     tuple val(sample_id),
48         path("${sample_id}_R{1,2}.hostfree.fastq.gz"),
49         emit: hostfree
50
51     script:
52     """
53     bowtie2 -x ${host_idx}/genome \
54         -1 ${reads[0]} -2 ${reads[1]} \
55         --very-sensitive -p ${task.cpus} \
56         --un-conc-gz ${sample_id}_R%.hostfree.fastq.gz \
57         -S /dev/null
58     """
59 }
60
61 process KRAKEN2 {
62     tag "$sample_id"
63     publishDir "${params.outdir}/taxonomy", mode: 'copy'
64     cpus 8
65
66     input:
67     tuple val(sample_id), path(reads)
68     path kraken_db
69

```

```

70     output:
71     tuple val(sample_id),
72         path("${sample_id}.kraken2.report"),
73         emit: kraken_report
74     path("${sample_id}.kraken2.output"), emit: kraken_out
75
76     script:
77     """
78     kraken2 --db ${kraken_db} \
79         --paired ${reads[0]} ${reads[1]} \
80         --threads ${task.cpus} \
81         --report ${sample_id}.kraken2.report \
82         --output ${sample_id}.kraken2.output \
83         --confidence 0.1
84     """
85 }
86
87 process BRACKEN {
88     tag "$sample_id"
89     publishDir "${params.outdir}/taxonomy", mode: 'copy'
90
91     input:
92     tuple val(sample_id), path(kraken_report)
93     path kraken_db
94
95     output:
96     tuple val(sample_id),
97         path("${sample_id}.bracken.txt"),
98         emit: bracken_out
99
100    script:
101    """
102    bracken -d ${kraken_db} \
103        -i ${kraken_report} \
104        -o ${sample_id}.bracken.txt \
105        -r 150 -l S
106    """
107 }
108
109 process MEGAHIT {
110     tag "$sample_id"
111     publishDir "${params.outdir}/assembly", mode: 'copy'
112     cpus 16
113     memory '32 GB'
114
115     input:
116     tuple val(sample_id), path(reads)
117
118     output:
119     tuple val(sample_id),
120         path("${sample_id}_contigs.fa"),
121         emit: contigs
122
123     script:
124     """
125     megahit -1 ${reads[0]} -2 ${reads[1]} \
126         -o megahit_out -t ${task.cpus} \
127         --min-contig-len 1000

```

```

128     mv megahit_out/final.contigs.fa ${sample_id}_contigs.fa
129     ""
130 }
131
132 process MAP_READS {
133     tag "${sample_id}"
134     publishDir "${params.outdir}/mapped", mode: 'copy'
135     cpus 8
136
137     input:
138     tuple val(sample_id), path(reads), path(contigs)
139
140     output:
141     tuple val(sample_id), path(contigs),
142           path("${sample_id}.sorted.bam"),
143           path("${sample_id}.sorted.bam.bai"),
144           emit: mapped
145
146     script:
147     ""
148     bowtie2-build ${contigs} contigs_idx
149     bowtie2 -x contigs_idx \
150         -1 ${reads[0]} -2 ${reads[1]} \
151         -p ${task.cpus} --very-sensitive \
152         | samtools sort -@ 4 -o ${sample_id}.sorted.bam -
153     samtools index ${sample_id}.sorted.bam
154     ""
155 }
156
157 process METABAT2 {
158     tag "${sample_id}"
159     publishDir "${params.outdir}/mags", mode: 'copy'
160     cpus 8
161
162     input:
163     tuple val(sample_id), path(contigs),
164           path(bam), path(bai)
165
166     output:
167     tuple val(sample_id),
168           path("${sample_id}_bins/"),
169           emit: bins
170
171     script:
172     ""
173     mkdir -p ${sample_id}_bins
174     jgi_summarize_bam_contig_depths \
175         --outputDepth depth.txt ${bam}
176     metabat2 -i ${contigs} -a depth.txt \
177         -o ${sample_id}_bins/bin \
178         -t ${task.cpus} --minContig 1500
179     ""
180 }
181
182 process CHECKM2 {
183     tag "${sample_id}"
184     publishDir "${params.outdir}/mags/quality", mode: 'copy'
185     cpus 8

```

```

186
187   input:
188   tuple val(sample_id), path(bins)
189
190   output:
191   tuple val(sample_id),
192         path("${sample_id}_checkm.tsv"),
193         emit: quality
194
195   script:
196   """
197   checkm2 predict \
198       --input ${bins} \
199       --output-directory checkm_out \
200       --threads ${task.cpus} -x fa
201   cp checkm_out/quality_report.tsv ${sample_id}_checkm.tsv
202   """
203 }
204
205 // ---- Workflow ----
206 workflow {
207     Channel
208         .fromFilePairs(params.reads)
209         .set { reads_ch }
210
211     host_ref = file(params.host_ref)
212     kraken_db = file(params.kraken_db)
213
214     // QC and host removal
215     FASTP(reads_ch)
216     REMOVE_HOST(FASTP.out.trimmed, host_ref)
217
218     // Taxonomic profiling
219     KRAKEN2(REMOVE_HOST.out.hostfree, kraken_db)
220     BRACKEN(KRAKEN2.out.kraken_report, kraken_db)
221
222     // Assembly and binning
223     MEGAHIT(REMOVE_HOST.out.hostfree)
224
225     // Combine hostfree reads with contigs for mapping
226     map_input = REMOVE_HOST.out.hostfree
227         .join(MEGAHIT.out.contigs)
228         .map { id, reads, contigs ->
229             tuple(id, reads, contigs)
230         }
231
232     MAP_READS(map_input)
233     METABAT2(MAP_READS.out.mapped)
234     CHECKM2(METABAT2.out.bins)
235 }

```

Community-Curated Metagenomics Pipelines

Several community-maintained pipelines implement best-practice metagenomics workflows:

- **nf-core/ampliseq**: Nextflow pipeline for amplicon sequencing (16S, 18S, ITS) with DADA2 and QIIME 2 integration.
- **nf-core/mag**: Nextflow pipeline for metagenome assembly, binning, and MAG analysis with comprehensive quality assessment.
- **nf-core/taxprofiler**: Nextflow pipeline for multi-tool taxonomic profiling (Kraken2, MetaPhlAn, mOTUs, Centrifuge, and more) with standardised outputs.
- **Sunbeam**: Snakemake-based extensible metagenomics pipeline with QC, host removal, assembly, and annotation.
- **Atlas**: Snakemake-based metagenome pipeline for assembly, binning, and annotation.

These pipelines handle edge cases, provide extensive QC reports, and are continuously tested—consider using them before building custom pipelines.

9.11 Long-Read Metagenomics Workflow Considerations

For long-read metagenomic experiments, the analysis workflow differs in several key ways:

1. **QC**: Use **NanoPlot** for ONT quality assessment, **chopper** or **Filtlong** for quality and length filtering. For PacBio HiFi, use **pbccs** and quality filtering is minimal (already high accuracy).
2. **Taxonomic classification**: **Kraken2** works well with long reads. **Centrifuge** and **Emu** (designed specifically for full-length 16S long reads) are alternatives.
3. **Assembly**: **metaFlye** for ONT data; **hifiasm-meta** for PacBio HiFi metagenomes. Long reads typically produce far more contiguous assemblies with higher N50.
4. **Binning**: Standard binners (**MetaBAT2**, **SemiBin2**) work, but methylation-aware binning (**bin_reassigner**) can dramatically improve results for ONT and PacBio data by using epigenetic signatures as an additional binning signal.
5. **Polishing**: ONT assemblies may benefit from short-read polishing with **Pilon** or **Polypolish** to improve consensus accuracy.

9.12 Summary

Chapter Summary

- **Metagenomics** bypasses culturing to study all microorganisms in a sample, revolutionising our understanding of microbial communities in health, disease, and the environment.
- **16S amplicon sequencing** is cost-effective for community composition profiling using conserved marker genes. The **ASV** approach (DADA2) has replaced OTU clustering as the standard for sequence resolution.
- **Shotgun metagenomics** provides a comprehensive view: taxonomic profiling across all domains of life (**Kraken2**, **MetaPhlAn**), functional potential (**HUMAnN**), strain-level resolution, and **metagenome-assembled genomes (MAGs)** from assembly and binning.
- **MAG quality** is assessed using **CheckM2** and classified using **GTDB-Tk**, following MIMAG standards for completeness and contamination.
- **Clinical metagenomics** enables unbiased pathogen detection and **AMR gene surveillance**, with databases such as **CARD**, **ResFinder**, and **AMRFinderPlus**.
- **Diversity analysis** uses alpha metrics (Shannon, Simpson, Chao1), beta metrics (Bray-Curtis, UniFrac), and ordination (PCoA, NMDS) to characterise and compare communities.
- **Long-read metagenomics** (full-length 16S, nanopore adaptive sampling, methylation-aware binning) is rapidly improving resolution and enabling genome-resolved metagenomics from complex environments.
- Both **Snakemake** and **Nextflow** provide robust workflow frameworks, with community pipelines (**nf-core/mag**, **nf-core/ampliseq**) available for immediate adoption.

Part IV

Transcriptomics: Gene Expression and RNA Biology

Chapter 10

RNA Sequencing: Measuring Gene Expression

Key Concept

RNA sequencing (RNA-seq) measures the transcriptome—the complete set of RNA transcripts in a cell, tissue, or organism at a given moment. Because gene expression is the primary mechanism through which genotype becomes phenotype, RNA-seq has become one of the most widely used sequencing applications in biology and medicine.

10.1 Why Measure RNA?

Every cell in a multicellular organism carries essentially the same genome, yet a neuron looks and behaves nothing like a hepatocyte. The difference lies predominantly in *which genes are expressed* and at *what levels*. Measuring RNA therefore provides a direct readout of cellular state.

10.1.1 Gene Expression as a Window into Biology

Gene expression governs virtually every biological process:

- **Development.** Transcription factors activate stage-specific gene programs that drive differentiation from a single zygote into hundreds of specialized cell types.
- **Disease.** Cancers are defined by aberrant expression profiles; autoimmune diseases by dysregulated immune genes; neurodegeneration by altered synaptic gene networks.
- **Environmental response.** Cells respond to stress, infection, drugs, and nutrients by rapidly reprogramming their transcriptomes.
- **Circadian rhythm.** Thousands of genes oscillate with 24-hour periodicity in virtually every tissue.

Before RNA-seq, gene expression was measured by microarrays, Northern blots, or quantitative PCR (qPCR). Microarrays could interrogate thousands of genes simultaneously but relied on predefined probes, suffered from cross-hybridization, and had limited dynamic range. RNA-seq overcame all of these limitations: it is unbiased, quantitative across five orders of magnitude, and can detect novel transcripts, splice variants, and allele-specific expression without any prior knowledge of the genome.

10.1.2 The Transcriptome

The transcriptome is far more complex than a simple catalogue of protein-coding messenger RNAs:

Messenger RNA (mRNA)

Protein-coding transcripts, accounting for only ~2–5% of total RNA mass but ~20,000 genes in humans.

Ribosomal RNA (rRNA)

The dominant RNA species by mass (~80–85%), forming the structural and catalytic core of ribosomes (28S, 18S, 5.8S, 5S in eukaryotes).

Transfer RNA (tRNA)

Adapter molecules (~10–15% of RNA) that deliver amino acids during translation.

Long non-coding RNA (lncRNA)

Transcripts >200 nt that do not encode proteins; >60,000 annotated in humans. Functions include chromatin remodeling, transcriptional regulation, and scaffolding (e.g., *XIST*, *MALAT1*, *HOTAIR*).

Small non-coding RNAs

Include microRNAs (miRNAs, ~22 nt), piwi-interacting RNAs (piRNAs, 24–32 nt), small interfering RNAs (siRNAs), small nucleolar RNAs (snoRNAs), and tRNA-derived fragments (tRFs).

Circular RNAs (circRNAs)

Covalently closed loops formed by back-splicing; highly stable, tissue-specific, and implicated in miRNA sponging and translation regulation.

Transcript isoforms

Alternative splicing, alternative polyadenylation, and alternative transcription start sites generate multiple isoforms per gene. Human genes produce an average of ~7 distinct isoforms.

Important Caveat

The vast majority of cellular RNA is ribosomal RNA. Without an enrichment or depletion strategy, an RNA-seq experiment would waste >80% of sequencing reads on rRNA.

10.2 Bulk RNA-seq: An Overview of Approaches

Bulk RNA-seq measures the *average* expression across a population of cells (typically 10^5 – 10^7 cells). Two main strategies exist, distinguished by how they deal with the rRNA problem.

10.2.1 mRNA-seq: Poly-A Selection

The most common approach for studying protein-coding gene expression. Polyadenylated transcripts (mRNAs plus some lncRNAs) are captured using oligo-dT beads that hybridize to the poly-A tail.

Poly-A Selection Protocol Overview

1. Extract total RNA (e.g., TRIzol or column-based methods).
 2. Incubate with oligo-dT magnetic beads; poly-A⁺ transcripts bind.
 3. Wash away rRNA, tRNA, and other non-polyadenylated RNAs.
 4. Elute mRNA and proceed to fragmentation and cDNA synthesis.
- Requires high-quality RNA with intact poly-A tails (RIN $\geq$ 7 recommended).

Advantages:

- Highly enriched for protein-coding transcripts.
- Lower sequencing depth required (fewer “wasted” reads).
- Well-established, widely used—most public datasets use this approach.

Limitations:

- Misses non-polyadenylated transcripts (histone mRNAs, many lncRNAs, most small RNAs, bacterial transcripts).
- Requires high-quality RNA; degraded samples lose poly-A tails, biasing toward 3' ends.
- Cannot capture nascent or nuclear RNA species lacking poly-A tails.

10.2.2 Total RNA-seq: Ribosomal RNA Depletion

Ribosomal depletion removes rRNA from total RNA, leaving all other RNA species intact—mRNAs, lncRNAs, circRNAs, pre-mRNAs, and other non-coding species.

Common rRNA depletion kits include:

- **Illumina Ribo-Zero Plus:** uses DNA probe hybridization and RNase H digestion to deplete rRNA from human, mouse, rat, and bacterial samples.
- **RiboCop (Lexogen):** enzymatic rRNA depletion compatible with degraded samples.
- **NEBNext rRNA Depletion Kit:** probe-based depletion for human, mouse, and rat.
- **QIAseq FastSelect:** targets rRNA during reverse transcription, eliminating a separate depletion step.

Advantages:

- Captures both coding and non-coding RNA.
- Works with degraded/FFPE samples (no reliance on poly-A tail integrity).
- Detects intronic reads, pre-mRNAs, and nascent transcription signals.
- Can reveal circular RNAs (which lack poly-A tails).

Limitations:

- Residual rRNA (~1–10%) consumes some sequencing capacity.
- More complex read composition; requires deeper sequencing for equivalent mRNA coverage.
- May require organism-specific depletion probes for non-model organisms.

10.2.3 Choosing Between mRNA-seq and Total RNA-seq

Table 10.1: Comparison of mRNA-seq (poly-A selection) and total RNA-seq (rRNA depletion) approaches.

Feature	mRNA-seq (poly-A)	Total RNA-seq (rRNA depletion)
Target RNA	Poly-A ⁺ mRNA (+ some lncRNA)	All RNA except rRNA
RNA quality requirement	High (RIN $\geq$ 7)	Moderate (works with RIN $\geq$ 2)
FFPE samples	Poor performance	Suitable
Non-coding RNA	Missed (most)	Captured
Intronic reads	Minimal	Substantial
Circular RNA	Not captured	Captured
Typical depth needed	20–30M reads	40–60M reads
Cost per informative read	Lower	Higher
Best for	Standard differential expression	Degraded samples, ncRNA, comprehensive profiling

Practical Tip

If your primary goal is differential expression of protein-coding genes and your RNA quality is good ($RIN \geq 7$), mRNA-seq is the most cost-effective choice. If you are working with clinical/FFPE samples, interested in non-coding RNAs, or need to capture the broadest possible transcriptome, use total RNA-seq.

10.3 Library Preparation for RNA-seq

10.3.1 RNA Extraction and Quality Assessment

High-quality RNA is the single most critical determinant of a successful RNA-seq experiment. RNA is chemically fragile and ubiquitous RNases in the environment degrade it rapidly.

Extraction methods • **TRIzol / TRI Reagent:** acid guanidinium–phenol–chloroform extraction. Gold standard for yield; suitable for most tissues.

- **Column-based:** RNeasy (Qiagen), PureLink (Invitrogen). Faster, less toxic, but lower yield for difficult tissues.
- **Automated:** KingFisher (Thermo), QIAasympphony. Essential for high-throughput core facilities.

Quality metrics • **RIN (RNA Integrity Number):** Agilent Bioanalyzer / TapeStation score from 1 (fully degraded) to 10 (intact). Based on the 28S/18S rRNA ratio and the electropherogram trace. $RIN \geq 7$ is recommended for poly-A selection.

- **DV200:** Fraction of RNA fragments > 200 nucleotides. Preferred metric for degraded/FFPE RNA because the RIN algorithm performs poorly when rRNA peaks are absent. $DV200 \geq 30\%$ is often the minimum for library construction with specialized FFPE kits.
- **Concentration:** Qubit fluorometer (RNA HS Assay) provides accurate quantitation; NanoDrop UV absorbance gives purity ratios ($260/280 \sim 2.0$ for pure RNA; $260/230 \geq 1.8$).

10.3.2 Strand-Specific Library Preparation

In unstranded RNA-seq, the original strand of the RNA transcript cannot be determined from the sequencing reads. Strand-specific (stranded) protocols preserve this information.

Why Strandedness Matters

- **Overlapping genes:** In complex genomes, genes on opposite strands can overlap. Unstranded data cannot assign reads to the correct gene.
- **Antisense transcription:** Many loci produce antisense transcripts that regulate the sense gene. Without strand information, sense and antisense reads are indistinguishable.
- **Accurate quantification:** Stranded data improves the accuracy of gene-level and transcript-level quantification by 5–15% for overlapping annotations.
- **Novel transcript discovery:** Strand information is essential for correctly assembling and annotating novel transcripts.

The dUTP Method

The most widely used strand-specific protocol:

1. Fragment mRNA and synthesize first-strand cDNA using random primers.
2. Synthesize second-strand cDNA incorporating dUTP instead of dTTP.

3. Ligate sequencing adapters.
4. Digest the dUTP-containing second strand with USER enzyme (uracil-DNA glycosylase + endonuclease VIII).
5. Only the first-strand cDNA (complementary to the original mRNA) is amplified and sequenced.
6. Read 2 corresponds to the sense strand of the original mRNA.

This is the basis of the **Illumina Stranded mRNA Prep** and **Illumina Stranded Total RNA Prep** kits, which have largely become the community standard.

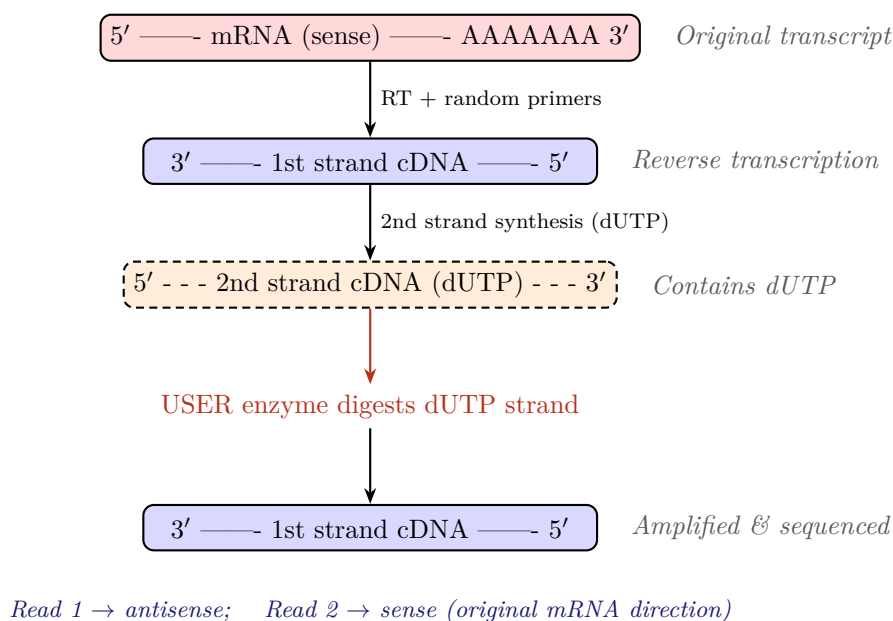


Figure 10.1: Schematic of the dUTP strand-specific library preparation method. The second-strand cDNA is synthesized with dUTP (dashed orange) and subsequently digested by USER enzyme, ensuring that only the first-strand cDNA (blue) is amplified. This preserves strand-of-origin information.

10.3.3 Low-Input RNA-seq

Standard RNA-seq protocols require 100 ng–1 μ g of total RNA. Many biological questions demand profiling from small cell populations, sorted cells, or limited clinical material.

Smart-seq2

Full-length transcript coverage from single cells or picogram-level input. Uses template-switching oligos and limited PCR amplification. Plate-based (typically 96- or 384-well). Excellent sensitivity but low throughput and not strand-specific in the original protocol.

SMART-Seq Stranded (Takara)

Combines Smart-seq template-switching with strand-specific library prep. Input as low as 10 pg (single cells) to 10 ng. Adds strand specificity lacking in original Smart-seq2.

NEBNext Single Cell / Low Input RNA Library Prep

Template-switching approach compatible with 2 pg–200 ng input.

10.3.4 Degraded RNA and FFPE Samples

Formalin-fixed paraffin-embedded (FFPE) tissues are ubiquitous in pathology archives. The fixation and embedding process crosslinks and fragments RNA severely (RIN typically 1–3).

Specialized protocols are needed:

- **Illumina Stranded Total RNA with Ribo-Zero Plus:** works with FFPE RNA down to DV200 of ~30%.
- **KAPA RNA HyperPrep with RiboErase:** optimized for low-quality input; produces libraries from as little as 25 ng FFPE RNA.
- **Takara SMARTer Stranded Total RNA-Seq Kit v3 – Pico Input:** designed for 250 pg–10 ng of degraded RNA.
- **Probe-based approaches (Twist Bioscience RNA Panel, IDT xGen RNA):** use hybridization capture for targeted RNA panels from FFPE samples.

Important Caveat

FFPE RNA-seq data has distinct characteristics: shorter insert sizes, higher duplicate rates, and 3' bias. Specialized QC metrics (DV200, percentage of reads in gene bodies) should be used. The standard RIN score is unreliable for FFPE samples.

10.4 Small RNA Sequencing

Small RNA-seq targets RNA molecules < 200 nt, with a particular focus on microRNAs (miRNAs, ~22 nt), which are key post-transcriptional regulators of gene expression.

10.4.1 Small RNA Species

microRNAs (miRNAs)

19–25 nt single-stranded RNAs that bind the 3' UTR of target mRNAs to induce translational repression or mRNA degradation. ~2,600 mature miRNAs annotated in humans (miRBase v22.1).

piRNAs

24–32 nt RNAs that silence transposable elements in the germline; associate with PIWI-clade Argonaute proteins.

siRNAs

20–24 nt double-stranded RNA-derived species involved in RNA interference.

tRNA-derived fragments (tRFs)

14–35 nt fragments cleaved from mature or precursor tRNAs. Emerging roles in stress response, gene regulation, and cancer.

snoRNAs

60–300 nt; guide chemical modifications of rRNA and snRNA.

10.4.2 Small RNA Library Preparation

Small RNA-seq Library Preparation

1. **Size selection:** Total RNA is size-selected for 15–50 nt fragments using PAGE gel or bead-based selection.
2. **3' adapter ligation:** An adapter is ligated to the 3'-OH of small RNAs (which have a free 3'-OH after Dicer processing).
3. **5' adapter ligation:** An adapter is ligated to the 5' phosphate.
4. **Reverse transcription:** cDNA is synthesized from the adapter-ligated small RNA.
5. **PCR amplification:** Limited-cycle PCR with indexed primers.
6. **Size selection:** Final library is size-selected (~135–170 bp including adapters) to remove adapter dimers.

Adapter Dimers in Small RNA-seq

Adapter dimers (3' and 5' adapters ligated directly to each other without a small RNA insert) are the most common artifact in small RNA-seq. They are ~120–130 bp and can dominate the library if input RNA is limiting. Rigorous gel-based or bead-based size selection is essential. Always check the library size distribution on a Bioanalyzer/TapeStation before sequencing.

10.4.3 miRNA-seq Analysis Workflow

The analysis of miRNA-seq data follows a specialized pipeline:

1. **Adapter trimming:** Critical because inserts (~22 nt) are shorter than reads (typically 50–75 bp). Tools: Cutadapt, Trim Galore, fastp.
2. **Quality filtering:** Remove reads < 15 nt or > 35 nt after trimming.
3. **Mapping to miRNA reference:** Align to mature and precursor miRNA sequences from miRBase using Bowtie (allowing 0–1 mismatches) or dedicated tools.
4. **Quantification:** Count reads mapping to each miRNA.
5. **Normalization and differential expression:** Use DESeq2 or edgeR as for mRNA, but with miRNA-specific considerations (lower library complexity, more discrete count distributions).
6. **Target prediction:** TargetScan, miRDB, DIANA-microT.

Key tools for small RNA analysis include:

- **miRDeep2:** Discovers known and novel miRNAs by modeling the miRNA biogenesis pathway (Drosha/Dicer processing signatures).
- **sRNAbench:** Comprehensive small RNA analysis including miRNA, piRNA, tRFs, and isomiR detection.
- **miRge 3.0:** Fast miRNA alignment and quantification using a Bayesian approach.
- **Oasis 2.0:** Web-based platform for small RNA-seq analysis.

10.5 Long-Read RNA Sequencing

Short-read RNA-seq excels at quantification but struggles with isoform resolution. When multiple transcript isoforms share exons, short reads (~150 bp) cannot span the full transcript, making it impossible to determine which combination of exons belongs to the same molecule. Long-read RNA sequencing solves this by reading full-length transcripts.

10.5.1 PacBio Iso-Seq

The **Isoform Sequencing (Iso-Seq)** protocol from Pacific Biosciences produces full-length transcript sequences without the need for assembly:

1. Reverse-transcribe poly-A⁺ RNA into full-length cDNA using a template-switching oligo.
2. Size-select cDNA (optional; can fractionate to enrich specific size ranges).
3. Generate SMRTbell libraries and sequence on the PacBio Revio or Sequel IIe platform.
4. Use the **IsoSeq3** pipeline to produce high-quality (HQ) full-length transcript isoforms with >99% accuracy using CCS reads.

Typical output: 5–15 million HiFi reads per SMRT Cell on Revio, yielding hundreds of thousands of unique transcript isoforms.

10.5.2 Oxford Nanopore RNA Sequencing

Oxford Nanopore offers two RNA-seq approaches:

Direct RNA sequencing (RNA004)

Sequences native RNA molecules directly through the nanopore without reverse transcription or PCR. This preserves RNA modifications (m⁶A, pseudouridine, m⁵C) and poly-A tail length as part of the raw signal. Requires ~500 ng poly-A⁺ RNA. Currently lower throughput (~1–4M reads per flow cell).

cDNA sequencing (SQK-PCS114)

Reverse-transcribes RNA into cDNA with template-switching, then sequences the cDNA. Higher throughput than direct RNA; compatible with PCR amplification for lower inputs.

Advantages of Long-Read RNA-seq

- **True isoform identification:** Each read represents a single full-length transcript, eliminating assembly artifacts.
- **No computational assembly:** No need for transcript assembly tools that can generate chimeric isoforms.
- **RNA modification detection:** ONT direct RNA-seq detects base modifications in native RNA.
- **Poly-A tail length measurement:** Direct information on polyadenylation dynamics.
- **Fusion transcript detection:** Full-length reads span fusion junctions unambiguously.

10.5.3 Long-Read Isoform Analysis Tools

IsoQuant

Maps long reads to a reference genome and assigns them to annotated or novel isoforms. Handles both PacBio and ONT data with sophisticated error correction.

FLAIR

Full-Length Alternative Isoform analysis of RNA. Corrects, collapses, and quantifies long-read isoforms. Integrates with short-read junction data for hybrid analysis.

Bambu

Reference-guided transcript discovery and quantification for long-read data. Uses a machine-learning model to distinguish genuine novel isoforms from noise.

TALON

Technology-agnostic long-read analysis pipeline; builds a database of observed transcripts across samples and tracks isoform usage changes.

SQANTI3

Structural and quality annotation of novel transcript isoforms; classifies isoforms into categories (full splice match, novel in catalog, novel not in catalog, etc.) and filters artifacts.

10.6 Nascent RNA Sequencing

Standard RNA-seq measures *steady-state* RNA levels, which reflect the balance of transcription and degradation. Nascent RNA-seq methods specifically capture RNA that is *currently being transcribed* by RNA polymerase II, providing a direct readout of transcriptional activity.

GRO-seq (Global Run-On sequencing)

Nuclei are isolated and RNA polymerases are allowed to extend nascent transcripts in the presence of BrUTP (bromouridine), which labels newly synthesized RNA. Labeled RNA is immunoprecipitated and sequenced. Reveals active transcription sites and their directionality, including enhancer RNAs (eRNAs).

PRO-seq (Precision Run-On sequencing)

Similar to GRO-seq but uses biotin-labeled NTPs that cause chain termination after a single nucleotide incorporation. This maps the position of active RNA polymerase at single-nucleotide resolution.

NET-seq (Native Elongating Transcript sequencing)

Immunoprecipitates RNA Pol II and sequences the 3' ends of associated nascent RNA. No run-on step; captures the native polymerase position.

TT-seq (Transient Transcriptome sequencing)

Cells are pulsed with 4-thiouridine (4sU) for 5–10 minutes; newly synthesized RNA incorporates 4sU, which is biotinylated and selected. Captures both stable and unstable nascent transcripts at near-physiological conditions.

Practical Tip

Nascent RNA-seq is essential for studying transcriptional regulation, enhancer activity, and RNA polymerase dynamics. If your biological question is “how much RNA is being produced?” rather than “how much RNA is present?”, consider a nascent RNA method.

10.7 RNA-seq Bioinformatics Workflow

10.7.1 Quality Control

FastQC

Generates per-read-base quality scores, GC content distribution, adapter contamination levels, sequence duplication levels, and overrepresented sequences for each FASTQ file.

MultiQC

Aggregates FastQC (and other tool) reports across all samples into a single interactive HTML report. Essential for spotting batch effects and outlier samples.

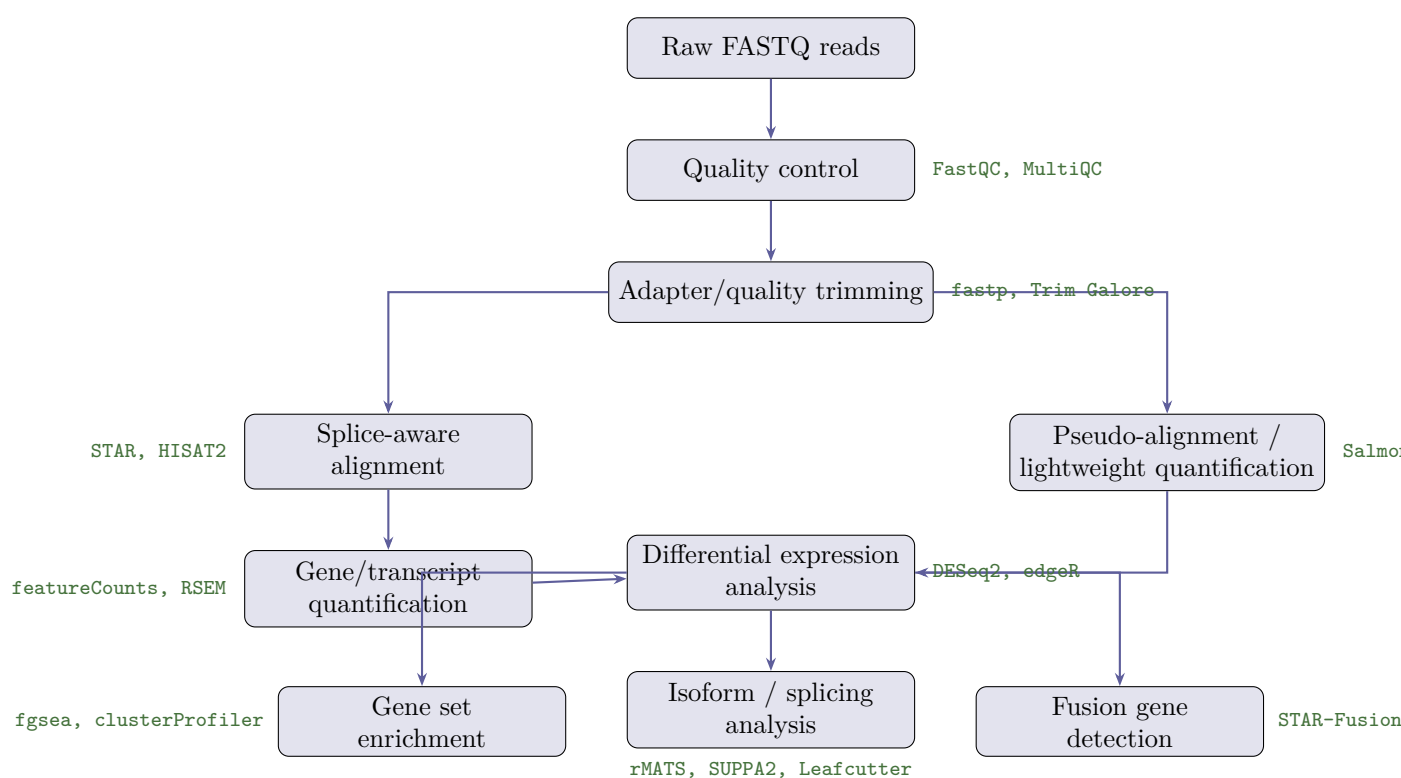


Figure 10.2: Overview of the bulk RNA-seq bioinformatics workflow. Two main paths exist after trimming: alignment-based quantification (left, using STAR/HISAT2 followed by featureCounts) or pseudo-alignment-based quantification (right, using Salmon/kallisto). Both converge at differential expression analysis.

Key RNA-seq QC metrics to monitor:

- Per-base sequence quality (Phred ≥ 30 across the read).
- GC content distribution (should match species expectation; bimodal distribution suggests contamination).
- Adapter content (should be minimal for RNA-seq with appropriate insert sizes).
- Duplication rate (high duplication may indicate low-complexity library or over-amplification).
- After alignment: percentage of uniquely mapped reads, percentage of reads in exons vs. introns vs. intergenic regions, 5'-to-3' gene body coverage (detects RNA degradation bias).

10.7.2 Trimming

fastp

All-in-one tool for adapter trimming, quality trimming, polyG tail removal (common on NovaSeq two-color chemistry), and QC report generation. Fast (multi-threaded C++).

Trim Galore

Wrapper around **Cutadapt** with automatic adapter detection and optional FastQC integration. Widely used in RNA-seq pipelines.

Practical Tip

For modern Illumina data, **fastp** is often sufficient as the sole preprocessing step. It performs adapter trimming, quality trimming, and generates a QC report in a single pass, simplifying the workflow.

10.7.3 Alignment: Splice-Aware Aligners

RNA-seq reads frequently span exon–exon junctions. Standard DNA aligners like BWA cannot handle these split reads because they expect contiguous alignment to the reference.

Important Caveat

Never use BWA or Bowtie2 (DNA aligners) for RNA-seq read alignment. These tools are not splice-aware and will discard or misalign reads that span exon–exon junctions, which can constitute 20–40% of all reads.

STAR (Spliced Transcripts Alignment to a Reference)

The most widely used RNA-seq aligner. Uses a suffix array-based algorithm that is extremely fast but memory-intensive (~30–40 GB RAM for human genome). Features:

- Two-pass mode for improved novel junction detection.
- Built-in gene quantification (`-quantMode GeneCounts`).
- Chimeric read detection for fusion gene calling.
- **STARsolo** mode for single-cell RNA-seq (see [Chapter 11](#)).

HISAT2 (Hierarchical Indexing for Spliced Alignment of Transcripts)

Uses a graph-based FM index that is much more memory-efficient (~8 GB for human). Slightly slower than STAR but suitable for machines with limited RAM. Successor to TopHat2/Bowtie2.

10.7.4 Pseudo-Alignment and Lightweight Quantification

An alternative to alignment-based approaches, pseudo-alignment (also called quasi-mapping or lightweight alignment) estimates transcript abundance without producing a traditional BAM file:

Salmon

Maps reads to the transcriptome (not genome) using selective alignment or quasi-mapping. Corrects for GC bias, sequence-specific bias, and fragment length distribution. Outputs transcript-level quantification (TPM and estimated counts). Run time: minutes rather than hours.

kallisto

Uses pseudo-alignment based on k-mer hashing. Produces transcript-level abundance estimates. Very fast and memory-efficient.

Alignment vs. Pseudo-Alignment

Alignment-based (STAR → featureCounts): Produces BAM files useful for visualization, variant calling, and splicing analysis. Required when you need genome-level information. **Pseudo-alignment** (Salmon/kallisto): Faster, produces transcript-level quantification directly. Excellent for differential expression and isoform quantification. No BAM file for visualization. For most differential expression analyses, both approaches yield highly concordant results.

10.7.5 Gene and Transcript Quantification

featureCounts (Subread package)

Assigns aligned reads (from BAM files) to genomic features (genes, exons) based on a GTF/GFF annotation. Fast and simple. Produces a gene-by-sample count matrix. Handles paired-end reads, multi-mapping reads, and strand-specific assignments.

RSEM

Reference-free estimation of transcript-level expression using an expectation-maximization (EM) algorithm. Often used with STAR alignment (STAR -quantMode TranscriptomeSAM).

StringTie

Assembles transcripts from aligned reads and estimates abundance. Particularly useful for novel transcript discovery and isoform-level quantification.

10.7.6 Differential Expression Analysis

Differential expression (DE) analysis identifies genes whose expression differs significantly between experimental conditions. The three dominant tools all use negative binomial generalized linear models:

DESeq2 (R/Bioconductor)

Uses shrinkage estimation for dispersion and fold changes. Provides independent filtering and Benjamini–Hochberg adjusted p -values. The most cited DE tool. Recommended for studies with ≥ 3 replicates per condition.

edgeR (R/Bioconductor)

Uses empirical Bayes methods for dispersion estimation. Slightly more liberal than DESeq2 in some benchmarks. Offers both exact tests and GLM-based approaches (quasi-likelihood F-tests are recommended for complex designs).

limma-voom (R/Bioconductor)

Transforms count data using the voom method (variance modeling at the observational level) and applies the limma linear modeling framework. Particularly powerful for complex experimental designs with multiple factors and interactions. Often performs well with larger sample sizes ($n > 10$ per group).

Practical Tip

When importing Salmon or kallisto transcript-level counts into R, use the `tximport` or `tximeta` package. These tools correctly aggregate transcript-level estimates to gene-level counts while accounting for changes in transcript length across samples—a bias that simple summing ignores.

10.7.7 The Negative Binomial Distribution for RNA-seq

RNA-seq read counts are *discrete* non-negative integers, making continuous distributions like the normal distribution inappropriate. A natural first model is the Poisson distribution, where the variance equals the mean: $\text{Var}(Y) = \mu$. However, biological replicates exhibit **overdispersion**—the observed variance exceeds the Poisson expectation because biological samples differ from one another beyond what sampling noise alone would predict. A gene with mean expression $\mu = 100$ across replicates may have variance of 500 or more, far larger than the Poisson prediction of 100.

The **negative binomial (NB) distribution** accounts for this overdispersion by introducing a dispersion parameter α (also written ϕ or $1/r$ in different parameterizations):

$$\text{Var}(Y) = \mu + \alpha\mu^2 \quad (10.1)$$

where μ is the mean expression and $\alpha \geq 0$ is the gene-specific dispersion. When $\alpha = 0$, the NB reduces to the Poisson. For most genes in a typical RNA-seq experiment, α ranges from 0.01 to 0.5. The probability mass function of the NB distribution (using the mean-dispersion parameterization) is:

$$P(Y = k) = \binom{k+r-1}{k} \left(\frac{r}{r+\mu}\right)^r \left(\frac{\mu}{r+\mu}\right)^k, \quad r = \frac{1}{\alpha} \quad (10.2)$$

where $r = 1/\alpha$ is the size (or inverse dispersion) parameter.

Why Not Poisson?

The Poisson model underestimates variance in RNA-seq data because it cannot represent biological variability between replicates. Using a Poisson test (e.g., an exact Poisson test in early RNA-seq tools) produces dramatically inflated false-positive rates. All modern DE tools—DESeq2, edgeR, and limma-voom—use models that accommodate overdispersion: the negative binomial GLM (DESeq2, edgeR) or variance-weighted linear models (limma-voom).

10.7.8 DESeq2 Dispersion Estimation and Shrinkage

Accurate estimation of the gene-specific dispersion α_i is the central statistical challenge in RNA-seq differential expression. With only 3–6 replicates per condition, per-gene maximum-likelihood estimates of α_i are highly noisy. DESeq2 addresses this through a three-step **dispersion shrinkage** procedure:

1. **Gene-wise dispersion estimates.** For each gene i , DESeq2 computes a maximum-likelihood estimate $\hat{\alpha}_i^{\text{MLE}}$ by fitting a negative binomial GLM using the Cox–Reid adjusted profile likelihood. These estimates are highly variable when the number of replicates is small.
2. **Fitted dispersion trend.** A smooth function $\alpha_{\text{tr}}(\mu)$ is fitted to the gene-wise estimates as a function of the mean expression μ_i . This trend captures the systematic relationship between dispersion and expression level (lowly expressed genes tend to have higher dispersion). The parametric fit is typically of the form:

$$\alpha_{\text{tr}}(\mu) = \frac{a_1}{\mu} + a_0 \quad (10.3)$$

where a_0 and a_1 are estimated by regression.

3. **Shrinkage toward the trend.** Each gene-wise estimate is shrunk toward the trend using a log-normal prior centered on $\log(\alpha_{\text{tr}}(\mu_i))$. The final maximum a posteriori (MAP) estimate balances the gene-specific data with the global trend:

$$\hat{\alpha}_i^{\text{MAP}} = \arg \max_{\alpha_i} \left[\ell(\alpha_i | Y_i) + \log \pi(\alpha_i | \alpha_{\text{tr}}(\mu_i), \sigma_d^2) \right] \quad (10.4)$$

where ℓ is the log-likelihood and π is the log-normal prior with variance σ_d^2 estimated from the data. Genes with very high dispersion (outliers above the trend) are not shrunk, to avoid underestimating their variance and inflating false positives.

Practical Tip

The **edgeR** package uses a conceptually similar but technically different approach. It estimates a common dispersion across all genes and gene-wise dispersions, then shrinks gene-wise estimates toward the common (or trended) dispersion using a weighted likelihood empirical Bayes procedure. The key difference is that **edgeR** uses a weighted likelihood approach rather than a prior distribution, and its quasi-likelihood F-test (recommended for complex designs) adds a second layer of overdispersion modeling.

10.7.9 Normalisation Methods for RNA-seq

Raw read counts cannot be compared directly across genes or samples because they are confounded by gene length, library size (total sequencing depth), and RNA composition. Several normalisation metrics address different subsets of these biases:

RPKM (Reads Per Kilobase per Million mapped reads)

Normalises for both gene length and sequencing depth. Used for single-end data:

$$\text{RPKM}_i = \frac{c_i \times 10^9}{N \times l_i} \quad (10.5)$$

where c_i is the raw count for gene i , N is the total number of mapped reads in the sample, and l_i is the effective gene length in base pairs.

FPKM (Fragments Per Kilobase per Million mapped fragments)

The paired-end equivalent of RPKM, where N counts fragments (read pairs) rather than individual reads. The formula is identical to RPKM but uses fragment counts, avoiding double-counting of paired-end reads.

TPM (Transcripts Per Million)

A two-step normalisation that first divides by gene length, then normalises to a fixed sum:

$$\text{TPM}_i = \frac{c_i/l_i}{\sum_j (c_j/l_j)} \times 10^6 \quad (10.6)$$

Because TPM values within each sample sum to exactly 10^6 , the relative proportions are directly comparable across samples—a property that RPKM/FPKM lack (their column sums vary with the gene length distribution of expressed genes).

Important Caveat

RPKM and FPKM values are **not directly comparable across samples** because the denominator N includes all mapped reads, and the gene-length normalisation step alters the effective library size differently depending on which genes are expressed in each sample. TPM resolves this by normalising to a fixed sum. For this reason, TPM has largely replaced RPKM/FPKM in modern publications. However, for differential expression analysis, none of these within-sample metrics are appropriate—use the raw counts with DESeq2 or edgeR, which apply their own between-sample normalisation (median-of-ratios or TMM, respectively).

DESeq2 Median-of-Ratios Normalisation

DESeq2 computes a *size factor* s_j for each sample j as the median of the ratios of observed counts to the geometric mean across samples:

$$s_j = \text{median}_i \frac{c_{ij}}{\left(\prod_{j'=1}^m c_{ij'}\right)^{1/m}} \quad (10.7)$$

This approach is robust to highly expressed genes that dominate a few samples and accounts for RNA composition differences—a major advantage over simple library-size normalisation.

10.7.10 DESeq2 vs. edgeR vs. limma-voom: A Detailed Comparison

Practical Tip

In practice, all three tools produce highly overlapping results for well-designed experiments with adequate replication. DESeq2 is the most popular due to its sensible defaults and extensive documentation. edgeR with quasi-likelihood F-tests is preferred for complex designs with multiple factors. limma-voom excels when sample sizes are large (e.g., GTEx-scale studies with hundreds of samples) because its linear model framework is both fast and statistically powerful. For the most robust results, consider running two tools and focusing on the intersection of significant genes.

10.7.11 Gene Set Enrichment Analysis

After identifying differentially expressed genes, the next step is biological interpretation:

GSEA (Gene Set Enrichment Analysis)

The original Broad Institute tool. Uses a ranked list of all genes (not just significant ones) to determine whether predefined gene sets (e.g., GO terms, KEGG pathways, MSigDB collections) are enriched at the top or bottom of the ranking. Avoids the arbitrary cutoff problem of over-representation analysis.

fgsea (R/Bioconductor)

Fast implementation of the GSEA algorithm in R. Orders of magnitude faster than the original Java tool.

Table 10.2: Detailed comparison of the three major differential expression tools for bulk RNA-seq.

Feature	DESeq2	edgeR	limma-voom
Statistical model	Negative binomial GLM	Negative binomial GLM (QL F-test recommended)	Weighted linear model on log-CPM
Normalisation	Median-of-ratios (size factors)	Trimmed mean of M-values (TMM)	TMM (via edgeR) or quantile
Dispersion estimation	Gene-wise MLE $\rightarrow$ trend $\rightarrow$ MAP shrinkage (log-normal prior)	Gene-wise $\rightarrow$ trended $\rightarrow$ empirical Bayes squeeze (weighted likelihood)	Mean-variance trend modeled by voom; precision weights per observation
Shrinkage of fold changes	Optional (apeglm, ashr, normal)	No built-in LFC shrinkage	Empirical Bayes moderation of t -statistics
Multiple testing	Benjamini–Hochberg with independent filtering	Benjamini–Hochberg	Benjamini–Hochberg
Complex designs	Supported (interaction terms, continuous covariates)	Excellent (quasi-likelihood framework is very flexible)	Excellent (full linear model framework; best for large, complex designs)
Speed (20k genes, $n=20$)	~ 1 – 3 minutes	~ 30 seconds– 1 minute	~ 10 – 30 seconds
Best for	Standard two-group comparisons with small n ; strong defaults	Complex experimental designs; large datasets	Very large sample sizes ($n > 20$); complex multi-factor designs

clusterProfiler (R/Bioconductor)

Comprehensive enrichment analysis package supporting Gene Ontology (GO), KEGG, Reactome, Disease Ontology, and custom gene sets. Performs both over-representation analysis (ORA) and GSEA. Provides publication-quality visualization functions (dot plots, enrichment maps, ridge plots).

enrichR

Web-based tool with an R API that queries >200 gene set libraries including ENCODE, ChEA, and other curated databases.

10.7.12 Isoform and Splicing Analysis**rMATS (replicate Multivariate Analysis of Transcript Splicing)**

Detects differential alternative splicing events from RNA-seq data. Classifies events into five types: skipped exon (SE), retained intron (RI), mutually exclusive exons (MXE), alternative 5' splice site (A5SS), and alternative 3' splice site (A3SS). Uses a Bayesian framework to estimate inclusion levels (Ψ / PSI values) and their differences.

SUPPA2

Fast differential splicing analysis using percent spliced-in (PSI) values calculated directly from transcript quantification (Salmon/kallisto output). Does not require BAM files.

Leafcutter

Detects differential intron usage without requiring transcript annotations. Identifies intron excision clusters and tests for differential usage between conditions. Annotation-free approach is particularly valuable for non-model organisms.

10.7.13 Fusion Gene Detection

Gene fusions—where two normally separate genes are joined—are hallmarks of many cancers (e.g., *BCR-ABL1* in CML, *EML4-ALK* in lung cancer).

STAR-Fusion

Uses chimeric read alignments from STAR to detect fusion transcripts. Integrates with the FusionInspector visualization tool and with CTAT (Cancer Transcriptome Analysis Toolkit).

Arriba

Fast and accurate fusion detection from STAR chimeric alignments. Known for low false-positive rate and built-in visualization of fusion events with gene structure diagrams.

10.8 Complete Snakemake Workflow for RNA-seq

10.9 Complete Nextflow Workflow for RNA-seq

Practical Tip

For production use, consider the `nf-core/rnaseq` pipeline, which implements a full-featured, peer-reviewed RNA-seq workflow in Nextflow with extensive QC, multiple aligner/quantifier options, and compatibility with institutional HPC systems and cloud environments. As of v3.14, it supports STAR, HISAT2, Salmon, and RSEM, with automatic strandedness detection.

10.10 Experimental Design for RNA-seq

10.10.1 Biological Replicates

Replication is Non-Negotiable

Biological replicates are essential for statistical inference. Technical replicates (sequencing the same library twice) do not capture biological variability and are rarely needed given the high reproducibility of modern sequencing. The minimum is **3 biological replicates per condition**; **5–6 replicates** substantially increase power to detect smaller fold changes. For heterogeneous clinical samples, even more replicates are advisable.

Power analysis tools such as `RNASeqPower` (R/Bioconductor) and `PROPER` can estimate the number of replicates needed given expected effect sizes, dispersion, and sequencing depth.

10.10.2 Sequencing Depth

Practical Tip

In most cases, adding biological replicates provides more statistical power than increasing sequencing depth beyond 20–30M reads per sample. Budget your sequencing funds toward more replicates rather than deeper sequencing, unless you specifically need isoform-level resolution.

Listing 10.1: Complete Snakemake workflow for bulk RNA-seq analysis with STAR alignment and featureCounts quantification.

```

1 # Snakefile for Bulk RNA-seq Analysis
2 # =====
3 # Pipeline: FastQC -> fastp -> STAR -> featureCounts -> DESeq2
4
5 import pandas as pd
6
7 configfile: "config/config.yaml"
8
9 # Load sample sheet
10 samples = pd.read_csv(config["samples"], sep="\t")
11 SAMPLES = samples["sample"].tolist()
12
13 # Reference files
14 GENOME_DIR = config["star_index"]
15 GTF = config["gtf"]
16 FASTA = config["genome_fasta"]
17
18 rule all:
19     input:
20         "results/multiqc/multiqc_report.html",
21         "results/counts/gene_counts.tsv",
22         "results/deseq2/differential_expression.csv",
23         expand("results/fastqc_raw/{sample}_{read}_fastqc.html",
24             sample=SAMPLES, read=["R1", "R2"]),
25         expand("results/bigwig/{sample}.bw", sample=SAMPLES)
26
27 rule fastqc_raw:
28     input:
29         "data/raw/{sample}_{read}.fastq.gz"
30     output:
31         html="results/fastqc_raw/{sample}_{read}_fastqc.html",
32         zip="results/fastqc_raw/{sample}_{read}_fastqc.zip"
33     threads: 2
34     conda: "envs/qc.yaml"
35     shell:
36         "fastqc -t {threads} -o results/fastqc_raw/ {input}"
37
38 rule fastp:
39     input:
40         r1="data/raw/{sample}_R1.fastq.gz",
41         r2="data/raw/{sample}_R2.fastq.gz"
42     output:
43         r1="results/trimmed/{sample}_R1.fastq.gz",
44         r2="results/trimmed/{sample}_R2.fastq.gz",
45         html="results/fastp/{sample}.html",
46         json="results/fastp/{sample}.json"
47     threads: 4
48     conda: "envs/qc.yaml"
49     shell:
50         """
51         fastp \
52             --in1 {input.r1} --in2 {input.r2} \
53             --out1 {output.r1} --out2 {output.r2} \
54             --html {output.html} --json {output.json} \
55             --detect_adapter_for_pe \
56             --qualified_quality_phred 20 \
57             --length_required 35 \
58             --thread {threads}
59         """
60
61 rule star_index:
62     input:

```

Listing 10.2: Complete Nextflow DSL2 workflow for bulk RNA-seq analysis with STAR alignment and featureCounts quantification.

```

1  #!/usr/bin/env nextflow
2  // RNA-seq Analysis Pipeline (Nextflow DSL2)
3  // =====
4  // Pipeline: FastQC -> fastp -> STAR -> featureCounts -> DESeq2
5
6  nextflow.enable.dsl = 2
7
8  params.reads      = "data/raw/*_{R1,R2}.fastq.gz"
9  params.genome      = "resources/genome.fa"
10 params.gtf         = "resources/genes.gtf"
11 params.star_index  = null
12 params.outdir      = "results"
13 params.strandedness = 2 // 2 = reverse-stranded (dUTP)
14
15 // Print pipeline info
16 log.info ""
17 =====
18 RNA-seq Analysis Pipeline
19 =====
20 reads      : ${params.reads}
21 genome     : ${params.genome}
22 GTF        : ${params.gtf}
23 strandedness : ${params.strandedness}
24 outdir     : ${params.outdir}
25 =====
26 ""
27
28 // --- Processes ---
29
30 process FASTQC {
31     tag "$sample_id"
32     publishDir "${params.outdir}/fastqc", mode: 'copy'
33     cpus 2
34
35     input:
36     tuple val(sample_id), path(reads)
37
38     output:
39     path("*.html"), emit: html
40     path("*.zip"), emit: zip
41
42     script:
43     ""
44     fastqc -t ${task.cpus} ${reads}
45     ""
46 }
47
48 process FASTP {
49     tag "$sample_id"
50     publishDir "${params.outdir}/fastp", mode: 'copy'
51     cpus 4
52
53     input:
54     tuple val(sample_id), path(reads)
55
56     output:
57     tuple val(sample_id), path("*_trimmed_R{1,2}.fastq.gz"),
58         emit: trimmed_reads 207
59     path("*.html"), emit: html
60     path("*.json"), emit: json
61
62     script:

```

Table 10.3: Recommended sequencing depths for common RNA-seq applications.

Application	Rationale	Reads/sample
Differential gene expression	Sufficient to quantify ~15,000 expressed genes	20–30M
Isoform-level analysis	Need to distinguish splice variants	50–100M
Novel transcript discovery	Deeper coverage of low-abundance transcripts	80–150M
Small RNA-seq	Smaller library; many reads are adapter dimers	10–20M
Total RNA-seq (rRNA depleted)	Compensate for residual rRNA reads	40–60M
FFPE RNA-seq	Higher duplicate rate reduces usable reads	50–80M

10.10.3 Batch Effects and Their Mitigation

Batch effects are systematic technical differences between groups of samples processed at different times, by different operators, or on different flow cells. They can be devastating in RNA-seq because they may confound or entirely obscure biological signal.

Prevention (most important):

- Extract RNA from all samples in the same session if possible.
- Prepare libraries in a single batch.
- If batching is unavoidable, **balance experimental groups across batches** (do not process all controls in batch 1 and all treatments in batch 2).
- Record batch information meticulously.

Computational correction (second line of defense):

ComBat (sva package)

Empirical Bayes framework for batch effect adjustment. Operates on normalized expression matrices. Widely used and effective but can remove real biological signal if batch and condition are confounded.

limma::removeBatchEffect

Removes batch effects from a normalized expression matrix for visualization purposes (PCA, heatmaps). For differential expression, include batch as a covariate in the DE model instead.

Inclusion in the DE model

The statistically preferred approach: add batch as a covariate in the DESeq2 or edgeR design formula (e.g., `~ batch + condition`). This separates batch variance from treatment variance without modifying the data.

Important Caveat

If batch is completely confounded with the biological variable of interest (e.g., all treatment samples were processed in January and all controls in March), no computational method can rescue the experiment. The only solution is to re-design and re-run the experiment with proper balancing.

10.11 RNA-seq Approaches: Comprehensive Comparison

Table 10.4: Comprehensive comparison of RNA-seq approaches.

Feature	mRNA-seq	Total RNA-seq	Small RNA-seq	Long-read RNA-seq
Target	Poly-A ⁺ mRNA	All RNA (rRNA depleted)	miRNA, piRNA, tRFs	Full-length transcripts
Read length	2×150 bp	2×150 bp	1×50–75 bp	1–20 kb
Platform	Illumina	Illumina	Illumina	PacBio, ONT
Input RNA	100 ng–1 µg	100 ng–1 µg	100 ng–1 µg	300 ng–5 µg
Quality req.	RIN ≥ 7	RIN ≥ 2 (with rRNA depl.)	RIN ≥ 7	RIN ≥ 7 (Iso-Seq)
Isoform resolution	Limited	Limited	N/A	Excellent
ncRNA coverage	Minimal	Comprehensive	Small ncRNA only	Good
Depth needed	20–30M	40–60M	10–20M	1–10M
Cost/sample	\$\$	\$\$	\$	\$\$\$
Main use case	Standard DE	Degraded samples, ncRNA	miRNA profiling	Isoform discovery

10.12 Chapter Summary

RNA-seq is the workhorse technology of modern transcriptomics, applicable to an extraordinary range of biological questions. In this chapter we covered:

- The rationale for measuring RNA and the complexity of the transcriptome.
- Poly-A selection (mRNA-seq) versus ribosomal depletion (total RNA-seq) and when to use each.
- Library preparation details: strand-specific protocols (dUTP method), low-input protocols (Smart-seq), and degraded RNA handling (FFPE).
- Small RNA sequencing for miRNAs and other small non-coding RNAs.
- Long-read RNA-seq (PacBio Iso-Seq, ONT direct RNA) for full-length isoform characterization.
- Nascent RNA methods (GRO-seq, PRO-seq, NET-seq, TT-seq) for measuring active transcription.
- The complete bioinformatics pipeline: QC, trimming, alignment (STAR, HISAT2), quantification (featureCounts, Salmon), differential expression (DESeq2, edgeR, limma-voom), gene set enrichment, isoform analysis, and fusion detection.
- Practical workflows in both Snakemake and Nextflow.
- Experimental design: replication, sequencing depth, and batch effect management.

In the next chapter, we move from bulk tissue-level measurements to the resolution of individual cells with single-cell RNA sequencing ([Chapter 11](#)).

Chapter 11

Single-Cell RNA Sequencing

Key Concept

Single-cell RNA sequencing (scRNA-seq) measures the transcriptome of individual cells, revealing cellular heterogeneity that is invisible to bulk RNA-seq. It has transformed our understanding of development, immunology, neuroscience, and cancer by enabling the identification of rare cell types, continuous differentiation trajectories, and cell-state-specific gene regulatory programs.

11.1 Why Single-Cell Resolution?

11.1.1 The Limitation of Bulk RNA-seq

Bulk RNA-seq measures the *average* gene expression across a population of 10^5 – 10^7 cells. This averaging has profound consequences:

- **Masking of rare populations:** A cell type comprising 1% of a tissue may be functionally critical (e.g., stem cells, tumor-initiating cells) but its transcriptional signature is diluted beyond detection in bulk data.
- **False intermediate states:** A mixture of two distinct cell states (e.g., activated and resting T cells) appears in bulk as a single “intermediate” state that does not actually exist in any individual cell.
- **Ecological fallacy:** Correlations observed at the bulk level may not hold at the single-cell level. Two genes may appear co-expressed in bulk data simply because they are expressed in different subpopulations that co-vary in proportion.
- **No trajectory information:** Bulk RNA-seq provides a snapshot but cannot reveal the order of events during dynamic processes such as differentiation, reprogramming, or drug response.

11.1.2 What Single-Cell Resolution Enables

1. **Cell type discovery:** Unbiased identification of cell types and subtypes without prior marker knowledge.
2. **Developmental trajectories:** Reconstruction of differentiation paths and branching decisions using pseudotime analysis.
3. **Gene regulatory networks:** Inference of transcription factor activity and regulatory relationships within specific cell types.
4. **Disease heterogeneity:** Characterization of tumor subclones, immune microenvironment composition, and treatment-resistant cell populations.

5. **Cell atlases:** Comprehensive catalogues of all cell types in an organism (Human Cell Atlas, Tabula Sapiens).

11.2 A Brief History of scRNA-seq

2009 — Tang et al.

First scRNA-seq experiment. Manual isolation of single mouse blastomeres, poly-A-tailed cDNA amplification, sequencing on the SOLiD platform. Detected ~5,000 genes per cell.

2011 — Islam et al.

Introduced STRT-seq with template-switching for 5' end capture and early multiplexing using cell barcodes.

2012 — Hashimoto et al. / Ramsköld et al.

Smart-seq protocol: full-length transcript coverage from single cells, greatly improving gene body coverage and isoform detection.

2014 — Smart-seq2 (Picelli et al.)

Improved sensitivity and reduced cost compared to Smart-seq. Became the gold standard for plate-based scRNA-seq.

2015 — Drop-seq & inDrop

Droplet-based microfluidics enabled profiling of thousands of cells in a single experiment. Drop-seq (Macosko et al.) used barcoded beads in nanoliter droplets. inDrop (Klein et al.) used hydrogel beads. Both introduced the concept of **UMI**-based counting.

2016 — 10x Genomics Chromium

Commercialized droplet-based scRNA-seq with the GemCode/Chromium platform. Standardized reagents, automated workflow, and the **Cell Ranger** analysis pipeline made scRNA-seq accessible to any lab with a sequencer.

2017–2019 — Scale-up era

sci-RNA-seq (combinatorial indexing), SPLiT-seq, SHARE-seq, and Slide-seq extended throughput to millions of cells and added spatial and multi-omic dimensions.

2020–present — Atlas and multi-omic era

The Human Cell Atlas project, COVID-19 single-cell studies, and multi-omic platforms (10x Multiome, CITE-seq, ASAP-seq) have made scRNA-seq a routine component of modern biology.

11.3 Cell Isolation Methods

The first step in any scRNA-seq experiment is isolating individual cells and capturing their RNA. The choice of isolation method determines throughput, sensitivity, and cost.

11.3.1 FACS Sorting

Fluorescence-activated cell sorting (FACS) uses fluorescent antibodies to sort cells one-by-one into wells of a 96- or 384-well plate. Each well contains lysis buffer and can be processed with a plate-based scRNA-seq protocol (Smart-seq2, Smart-seq3).

Advantages: Protein-level selection of specific populations (e.g., CD8⁺ T cells); index sorting records surface marker levels for each cell. **Limitations:** Low throughput (hundreds to low thousands of cells); requires viable single-cell suspensions and surface markers.

11.3.2 Microfluidics

The Fluidigm C1 system captures cells in microfluidic chambers using integrated fluidic circuits. Each chip captures 96 or 800 cells.

Advantages: Automated cell capture and lysis; full-length protocols (Smart-seq). **Limitations:** Low throughput; size-dependent capture (requires cells of uniform size); high cost per cell; largely superseded by droplet-based methods.

11.3.3 Droplet-Based Methods

Droplet microfluidics encapsulates individual cells with barcoded beads in nanoliter oil droplets, enabling massively parallel scRNA-seq.

10x Genomics Chromium

The dominant commercial platform (see [Section 11.4](#) for details). Uses Gel Bead-in-Emulsions (GEMs). Throughput: 500–10,000+ cells per reaction.

Drop-seq

Academic protocol using barcoded microparticles and a simple PDMS microfluidic device. Lower cost per cell than 10x but higher doublet rate and lower capture efficiency.

inDrop

Uses deformable hydrogel beads for more uniform barcode delivery. Open-source design.

11.3.4 Combinatorial Indexing

These methods achieve single-cell resolution without physically isolating individual cells. Instead, cells are split across wells, indexed with well-specific barcodes in multiple rounds, and then pooled. The extremely low probability that two cells receive the same combination of barcodes across all rounds provides unique identification.

sci-RNA-seq / sci-RNA-seq3

Achieves profiling of millions of cells in a single experiment. Uses two or three rounds of combinatorial indexing (reverse transcription barcoding → ligation barcoding → PCR barcoding).

SPLiT-seq

Split-pool ligation-based transcriptome sequencing. Uses successive rounds of in-cell barcoding by ligation. No specialized equipment required beyond standard liquid handling.

Practical Tip

Combinatorial indexing methods excel for very large experiments (e.g., organism-scale cell atlases). They are cost-effective per cell but have lower sensitivity (fewer genes per cell) compared to droplet-based or plate-based methods.

11.3.5 Plate-Based Methods

Smart-seq2

Full-length transcript coverage from single cells sorted into wells. Uses oligo-dT priming, template switching, and limited PCR amplification. Highest sensitivity (~5,000–10,000 genes per cell) and gene body coverage of any scRNA-seq method. Enables isoform-level analysis and SNV detection. Throughput: 96–384 cells per run.

Smart-seq3 / Smart-seq3xpress

Adds UMIs to Smart-seq2 while retaining full-length information. Combines the quantitative accuracy of UMI counting with the isoform resolution of full-length coverage.

11.4 10x Genomics Chromium in Detail

The 10x Genomics Chromium platform is the most widely used scRNA-seq system worldwide. Understanding its mechanics is essential for interpreting scRNA-seq data.

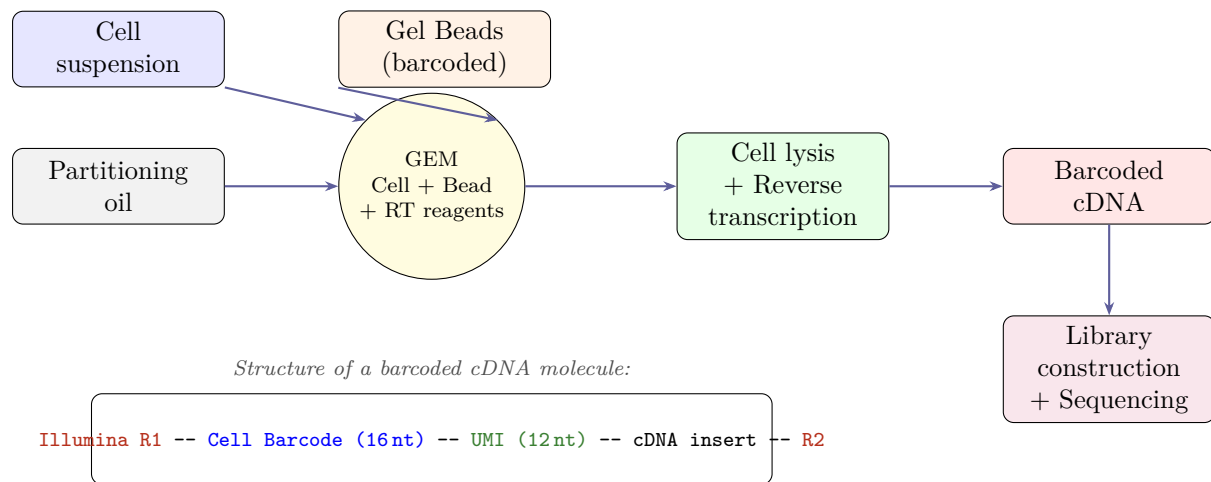


Figure 11.1: Schematic of the 10x Genomics Chromium droplet-based scRNA-seq workflow. Single cells and gel beads are co-encapsulated in Gel Bead-in-Emulsions (GEMs), where cell lysis and reverse transcription occur. Each gel bead carries millions of oligonucleotide copies with a shared cell barcode and diverse UMIs. The resulting cDNA is uniquely labeled with a cell barcode and a UMI for each original mRNA molecule.

11.4.1 Gel Bead-in-Emulsions (GEMs)

The core of the 10x Chromium system is the partitioning of single cells with gel beads into nanoliter aqueous droplets suspended in oil (GEMs):

1. A microfluidic chip combines three streams: cell suspension, gel bead suspension, and partitioning oil.
2. Each GEM ideally contains one cell and one gel bead. The vast majority of GEMs (>90%) contain zero cells (empty) to minimize doublets (GEMs with two or more cells).
3. The gel bead dissolves, releasing oligonucleotides into the droplet.
4. Cell lysis releases mRNA, which is captured by oligo-dT sequences on the bead-derived oligonucleotides.
5. Reverse transcription occurs within the GEM, producing first-strand cDNA tagged with the cell barcode and a unique molecular identifier (UMI).

11.4.2 Cell Barcodes and UMIs

Cell Barcodes and UMIs

Cell barcode (16 nt): Identifies which cell a cDNA molecule came from. All oligonucleotides on a single gel bead share the same cell barcode. Different gel beads have different barcodes. The 10x barcode whitelist contains ~737,000 valid sequences (for 3' v3/v3.1 chemistry).

UMI (12 nt): Unique Molecular Identifier. Each oligonucleotide on a bead has a random 12-mer UMI, enabling distinction between PCR duplicates and distinct original mRNA molecules. After PCR amplification, reads sharing the same cell barcode, UMI, and gene are collapsed to a single count, eliminating amplification bias.

11.4.3 3' vs. 5' Gene Expression Chemistry

3' Gene Expression (GEX)

Captures the 3' end of polyadenylated transcripts (within ~500 bp of the poly-A tail). Most commonly used chemistry. Sufficient for gene-level quantification and cell type identification. Cannot provide full-length transcript or isoform information.

5' Gene Expression (GEX)

Captures the 5' end of transcripts using template-switching. Enables simultaneous profiling of gene expression and immune receptor sequences (TCR/BCR V(D)J repertoires) from the same cells. Preferred for immunology studies.

11.4.4 Cell Ranger Pipeline

Cell Ranger is the standard 10x Genomics analysis pipeline that processes raw Illumina BCL files into a gene-by-cell count matrix:

1. **cellranger mkfastq:** Demultiplexes BCL files into FASTQ files.
2. **cellranger count:** Performs alignment (STAR), barcode counting, UMI deduplication, and generates the filtered feature-barcode matrix.
3. **cellranger aggr:** Aggregates multiple count runs and normalizes across samples.

The key output is the **filtered feature-barcode matrix**: a sparse matrix where rows are genes (~33,000 for human) and columns are cell barcodes that passed the cell-calling algorithm (EmptyDrops-like knee/inflection point detection).

11.4.5 Typical Throughput and Sequencing Recommendations

- **Cells per reaction:** 500–10,000 (recommended loading for ~0.8% multiplet rate per 1,000 cells).
- **Sequencing depth:** 20,000–50,000 reads per cell for 3' GEX. Higher depth yields diminishing returns for gene detection beyond ~50,000 reads/cell.
- **Read configuration:** Read 1: 28 bp (16 bp cell barcode + 12 bp UMI); Index 1: 10 bp (sample index); Read 2: 90 bp (cDNA insert).
- **Expected genes per cell:** 2,000–6,000 median genes/cell, depending on cell type and sequencing depth.

11.4.6 10x Chromium Flex (Fixed RNA Profiling)

The Flex assay uses a probe-based approach to capture gene expression from *fixed* cells:

- Cells are fixed with paraformaldehyde, enabling sample banking and batch processing.

- Probe pairs hybridize to target mRNAs inside fixed cells; probes are ligated and then captured during GEM partitioning.
- Compatible with FFPE tissue sections (after deparaffinization).
- Supports sample multiplexing: up to 128 samples pooled in one Chromium run using probe-barcode-based demultiplexing.
- Lower background noise compared to 3' GEX because probe-based detection avoids ambient RNA contamination.

11.5 Key Computational Challenges

11.5.1 Sparsity and Dropout

scRNA-seq data is extremely sparse: the median gene detection rate per cell is only 15–30% of expressed genes. Many genes that are truly expressed in a cell are recorded as zero counts—a phenomenon called **dropout**. Dropout arises from:

- Low mRNA capture efficiency (~5–15% of transcripts in a cell for droplet-based methods).
- Stochastic sampling: with only ~5,000–10,000 unique mRNA molecules detected per cell, lowly expressed genes are frequently missed.
- Shallow sequencing: not all cDNA molecules in the library are sequenced.

Important Caveat

Dropout means that a zero count in scRNA-seq does not necessarily mean the gene is not expressed. This has profound implications for analysis: methods that treat zeros as true absence can produce misleading results. Many specialized scRNA-seq statistical methods (e.g., MAST, scVI) explicitly model dropout or use zero-inflated distributions.

11.5.2 Doublet Detection

Doublets (or multiplets) occur when two or more cells are captured in the same droplet or well. They appear as single “cells” with hybrid transcriptomes that can create spurious intermediate populations or bridge clusters artifactually.

The expected doublet rate for 10x Chromium is ~0.8% per 1,000 cells loaded (e.g., loading 10,000 cells yields ~8% doublets).

Scrublet

Simulates doublets by combining random pairs of observed cell profiles and scores each real cell by its similarity to simulated doublets. Python-based.

DoubletFinder

Similar simulation-based approach implemented in R/Seurat. Requires parameter optimization (pK, expected doublet rate).

scDblFinder

Fast and accurate doublet detection using a gradient-boosted classifier trained on simulated doublets. R/Bioconductor-based.

Doublet Detection: Principles and Expected Rates

The fundamental principle behind simulation-based doublet detectors such as **Scrublet** is straightforward:

1. **Simulate artificial doublets.** Randomly select pairs of observed cell profiles from the count matrix and sum their expression vectors. This creates synthetic doublets that mimic the transcriptomes of two co-captured cells.
2. **Embed jointly.** Combine the real cells and simulated doublets, then perform dimensionality reduction (PCA followed by KNN graph construction) on the merged dataset.
3. **Score each real cell.** For each observed cell, compute a *doublet score* based on the local density of simulated doublets in its neighborhood. Cells surrounded by many simulated doublets are likely real doublets.
4. **Threshold.** Apply a bimodal threshold (or a user-specified cutoff) to classify cells as singlets or doublets.

The expected doublet rate for droplet-based platforms follows an approximately linear relationship with loading density:

$$r_d \approx 0.008 \times \frac{n_{\text{cells}}}{1000} \quad (11.1)$$

where n_{cells} is the number of recovered cells per reaction. For example, loading 5,000 cells yields an expected doublet rate of $\sim 4\%$, while 10,000 cells gives $\sim 8\%$. This formula assumes Poisson loading statistics and is calibrated for the 10x Chromium platform.

Important Caveat

Simulation-based doublet detectors excel at identifying **heterotypic doublets** (two cells of different types, e.g., a T cell + a monocyte) because these produce distinctive hybrid profiles. **Homotypic doublets** (two cells of the same type) are nearly impossible to detect computationally because their transcriptomes resemble a single cell with higher UMI counts. Homotypic doublets inflate the apparent cell counts but typically do not create artifactual clusters. When reporting cell numbers, account for both detected and undetected (homotypic) doublets.

11.5.3 Ambient RNA Contamination

During droplet generation, free-floating mRNA from lysed or damaged cells (“soup”) is present in the cell suspension. This ambient RNA is captured in every GEM, contaminating the transcriptomes of all cells with a background signature (often enriched for highly expressed genes like hemoglobin in blood or mitochondrial genes from damaged cells).

SoupX

Estimates the contamination fraction using the expression of genes expected to be cell-type-specific (e.g., hemoglobin in non-erythrocyte clusters). Subtracts estimated ambient counts.

CellBender (remove-background)

Deep generative model that distinguishes cell-containing droplets from empty droplets and removes ambient RNA signal. Computationally intensive but highly effective.

DecontX

Bayesian method for ambient RNA decontamination that models each cell’s expression as a mixture of its native profile and the ambient contamination profile.

11.6 scRNA-seq Analysis Workflow

The computational analysis of scRNA-seq data involves a well-established sequence of steps, from raw data processing to biological interpretation.

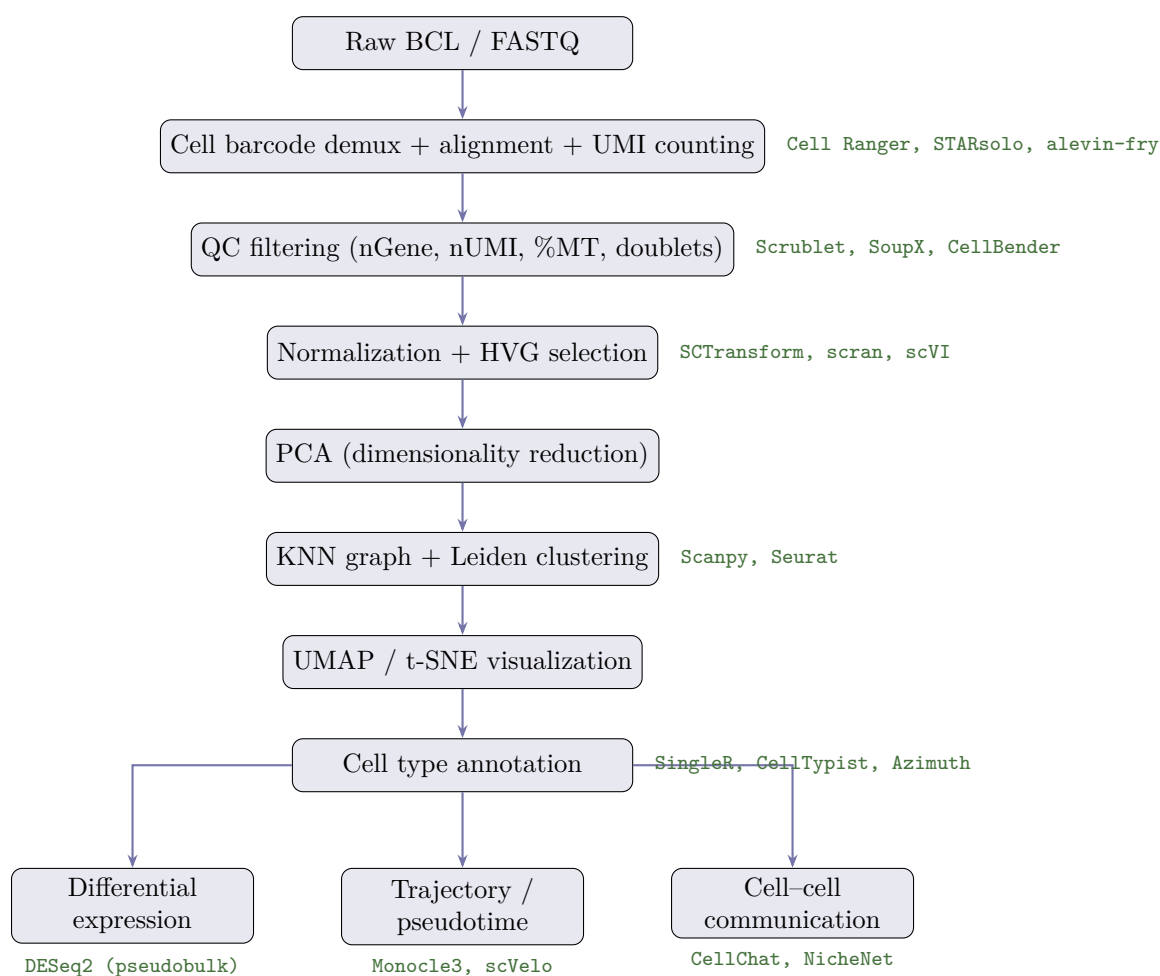


Figure 11.2: Standard computational workflow for scRNA-seq analysis. After generating a cell-by-gene count matrix, cells are filtered, normalized, dimensionality-reduced, clustered, and annotated. Downstream analyses include differential expression, trajectory inference, and cell-cell communication.

11.6.1 Raw Data Processing

Cell Ranger

The default pipeline for 10x Chromium data (see [Section 11.4](#)). Closed-source but free. Produces the filtered count matrix and a web summary report.

STARsolo

Open-source alternative to Cell Ranger, built into the STAR aligner. Produces equivalent results with greater flexibility and often faster run time. Supports 10x, Drop-seq, and other barcode schemes.

alevin-fry (Salmon)

Lightweight mapping-based quantification for scRNA-seq. Uses the Salmon quasi-mapping engine with barcode-aware processing. Produces a count matrix compatible with downstream tools. The `simpleaf` wrapper simplifies execution.

11.6.2 Quality Control and Filtering

After generating the raw count matrix, per-cell QC metrics are computed and cells failing quality thresholds are removed:

- **Number of genes detected (nGene):** Cells with very few genes (< 200 – 500) are likely empty droplets or debris.
- **Number of UMIs (nUMI / nCount):** Total counts per cell. Extremely high values may indicate doublets.
- **Percentage of mitochondrial reads (%MT):** High mitochondrial fraction (> 10 – 20% , depending on tissue) indicates dying or damaged cells whose cytoplasmic mRNA has leaked out while mitochondrial mRNA (enclosed in a double membrane) is retained.
- **Percentage of ribosomal protein genes (%Ribo):** Can help identify low-quality cells or specific cell types.
- **Doublet scores:** From Scrublet, DoubletFinder, or scDblFinder.

Practical Tip

QC thresholds should be adapted to the tissue and experiment. Brain tissue typically has higher mitochondrial content than PBMCs. Use distribution-based thresholds (e.g., > 3 median absolute deviations from the median) rather than fixed cutoffs when possible. The `scater` package in R provides automated outlier detection with `isOutlier()`.

11.6.3 Normalization

Raw UMI counts must be normalized to account for differences in sequencing depth (total UMIs) across cells.

Log-normalization (Seurat default)

Divide each cell's counts by total UMIs, multiply by a scale factor (default 10,000), and log-transform: $\log_2(\text{counts}/\text{total} \times 10,000 + 1)$. Simple and widely used.

scran pooling-based normalization

Pools cells into groups, estimates pool-based size factors, and deconvolves to per-cell size factors. More robust than simple library-size normalization because it handles the sparsity of single-cell data.

SCTransform (Seurat v3+)

Fits a regularized negative binomial regression model to each gene as a function of total

UMIs. Replaces both normalization and variance stabilization in a single step. Handles the mean-variance relationship better than log-normalization for downstream analyses (PCA, clustering).

scVI (scvi-tools)

Deep generative model (variational autoencoder) that models the count generation process, implicitly handling normalization, batch effects, and dropout simultaneously.

11.6.4 Feature Selection: Highly Variable Genes

Not all $\sim 20,000+$ detected genes are informative for distinguishing cell types. Feature selection identifies **highly variable genes (HVGs)**—those with expression variation exceeding what is expected from sampling noise.

- Seurat: fits a variance-stabilizing transformation and selects the top 2,000–5,000 genes by residual variance.
- Scanpy: uses the method of Zheng et al. or the Seurat-style approach to identify HVGs.
- Only HVGs are used for dimensionality reduction and clustering, reducing noise and computation.

11.6.5 Dimensionality Reduction

PCA (Principal Component Analysis)

Linear dimensionality reduction. Typically the first 20–50 principal components are retained, capturing the major axes of variation. PCA is performed on the HVG expression matrix.

t-SNE (t-distributed Stochastic Neighbor Embedding)

Non-linear dimensionality reduction that preserves local structure. Produces 2D embeddings widely used for visualization. Sensitive to hyperparameters (perplexity). Does not preserve global distances.

UMAP (Uniform Manifold Approximation and Projection)

Non-linear method that preserves both local and (approximately) global structure. Faster than t-SNE and currently the most popular visualization method for scRNA-seq. Operates on the PCA-reduced space (not raw expression).

Important Caveat

t-SNE and UMAP are visualization tools, not analytical methods. Do not perform clustering on t-SNE/UMAP coordinates. Cluster on the PCA space or the shared nearest neighbor (SNN) graph. Distances and apparent cluster sizes in t-SNE/UMAP plots can be misleading.

11.6.6 Mathematical Foundations of t-SNE and UMAP

Although t-SNE and UMAP are often treated as black-box visualization tools, understanding their mathematical principles is essential for interpreting their output correctly and tuning hyperparameters.

t-SNE: t-distributed Stochastic Neighbor Embedding

t-SNE operates in two stages. First, in the high-dimensional PCA space, pairwise similarities are converted into conditional probabilities. The similarity of point x_j to point x_i is defined

as:

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)} \quad (11.2)$$

where σ_i is chosen such that the perplexity of the conditional distribution matches a user-specified value (typically 30). The perplexity controls the effective number of neighbors considered for each point.

The joint probability matrix is symmetrized as $p_{ij} = (p_{j|i} + p_{i|j}) / (2n)$. In the low-dimensional embedding (2D), similarities are modeled using a Student's t -distribution with one degree of freedom (Cauchy distribution):

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}} \quad (11.3)$$

where y_i, y_j are the low-dimensional coordinates. The heavy tails of the t -distribution allow nearby points to be modeled faithfully while pushing dissimilar points further apart, alleviating the “crowding problem” that affects Gaussian-based methods like classical SNE.

t-SNE minimizes the Kullback–Leibler divergence between P and Q :

$$KL(P||Q) = \sum_i \sum_{j \neq i} p_{ij} \log \frac{p_{ij}}{q_{ij}} \quad (11.4)$$

via gradient descent. The asymmetry of KL divergence means that t-SNE penalizes placing similar points far apart (“tearing” neighborhoods) more heavily than placing dissimilar points close together. This is why t-SNE excels at preserving local structure but can distort global distances.

UMAP: Uniform Manifold Approximation and Projection

UMAP is grounded in topological data analysis and Riemannian geometry. Its key steps are:

1. **Fuzzy simplicial set construction.** For each point x_i , UMAP finds its k nearest neighbors and converts distances into membership strengths using a smooth kernel:

$$w_{ij} = \exp\left(-\frac{d(x_i, x_j) - \rho_i}{\sigma_i}\right) \quad (11.5)$$

where ρ_i is the distance to the nearest neighbor of x_i (ensuring local connectivity) and σ_i is scaled to match the desired number of neighbors. These weights are symmetrized to form a fuzzy simplicial set (weighted graph) representing the topological structure of the data.

2. **Low-dimensional optimization.** UMAP initializes a low-dimensional embedding (typically using spectral initialization on the fuzzy graph Laplacian) and optimizes it by minimizing a **cross-entropy loss** between the high-dimensional and low-dimensional fuzzy sets:

$$\mathcal{L} = \sum_{(i,j)} \left[w_{ij} \log \frac{w_{ij}}{v_{ij}} + (1 - w_{ij}) \log \frac{1 - w_{ij}}{1 - v_{ij}} \right] \quad (11.6)$$

where $v_{ij} = (1 + a\|y_i - y_j\|^{2b})^{-1}$ is the low-dimensional similarity, with a and b determined by the `min_dist` parameter.

3. **Stochastic gradient descent with negative sampling.** Rather than computing all pairwise interactions, UMAP uses negative sampling to approximate the repulsive term, making it substantially faster than t-SNE for large datasets ($\mathcal{O}(n)$ per epoch vs. $\mathcal{O}(n^2)$ for exact t-SNE).

t-SNE vs. UMAP: Key Differences

- **Global structure:** UMAP’s cross-entropy loss includes a repulsive term for dissimilar points, better preserving global relationships (relative cluster positions) compared to t-SNE’s KL divergence.
- **Speed:** UMAP is typically 5–50× faster than t-SNE for datasets with >50,000 cells, due to negative sampling and graph-based initialization.
- **Reproducibility:** UMAP with fixed random seed produces consistent embeddings; t-SNE results can vary substantially between runs.
- **Key hyperparameters:** For t-SNE, perplexity (30–50 typical); for UMAP, `n_neighbors` (15–30) and `min_dist` (0.1–0.5). Both should be explored rather than left at defaults.

11.6.7 Clustering

Graph-based clustering is the standard approach for scRNA-seq:

1. Construct a *k*-nearest neighbor (KNN) graph in PCA space. Each cell is a node; edges connect cells to their *k* nearest neighbors.
2. Refine edge weights using shared nearest neighbors (SNN graph) or Jaccard similarity.
3. Apply a community detection algorithm:
 - **Louvain algorithm:** Optimizes modularity; fast but can produce poorly connected clusters.
 - **Leiden algorithm:** Improved version of Louvain that guarantees well-connected communities. Generally preferred.
4. The **resolution** parameter controls the granularity of clustering: higher values produce more clusters. There is no single “correct” resolution; it should be guided by biological knowledge.

11.6.8 Cell Type Annotation

Assigning biological identity to clusters is one of the most critical—and most subjective—steps in scRNA-seq analysis.

Manual annotation

Examine marker genes for each cluster (e.g., *CD3D/CD3E* for T cells, *MS4A1/CD79A* for B cells, *LYZ/CD14* for monocytes). Requires domain expertise. The `FindAllMarkers` function in Seurat and `sc.tl.rank_genes_groups` in Scanpy identify cluster-specific marker genes.

SingleR

Automated annotation by correlating each cell’s expression profile to reference transcriptomic datasets (e.g., Human Primary Cell Atlas, Monaco Immune Data, Blueprint Encode). R-based.

CellTypist

Machine-learning-based classifier trained on large reference datasets. Pre-trained models for immune cells, brain cells, and pan-tissue atlases. Python-based.

scType

Fully automated cell type annotation using curated marker gene databases. R-based; no reference dataset required.

Azimuth

Web-based and R-based tool from the Seurat developers. Uses reference mapping to

project query cells onto well-curated reference atlases (PBMC, lung, motor cortex, etc.). Provides both coarse and fine-grained cell type labels with confidence scores.

11.6.9 Differential Expression

Identifying genes that are differentially expressed between clusters or conditions requires methods appropriate for the distributional properties of scRNA-seq data:

Wilcoxon rank-sum test

Non-parametric; the default in both Seurat and Scanpy. Fast and performs reasonably well for cluster markers but does not account for the hierarchical structure of cells within samples.

MAST

Hurdle model that jointly models the rate of expression (logistic regression for zero vs. non-zero) and the expression level (linear model for non-zero values). Accounts for cellular detection rate as a confounding variable.

Pseudobulk approaches

Aggregate counts per cell type per biological replicate, then apply bulk DE tools (DESeq2, edgeR). This approach correctly treats biological replicates as the unit of inference (not individual cells) and has been shown to have the best calibrated false discovery rates in recent benchmarks. **Recommended for comparing conditions (e.g., treatment vs. control) when biological replicates are available.**

The Pseudobulk Principle

In a scRNA-seq experiment with multiple biological samples, individual cells are not independent observations—cells from the same sample share the same biological variation. Treating each cell as an independent replicate inflates the sample size artificially and leads to extreme p -value inflation. The solution is pseudobulk aggregation: sum counts across cells of the same type within each biological replicate, then use bulk DE methods on the resulting replicate-level counts.

11.6.10 Trajectory Analysis and Pseudotime

Many biological processes (differentiation, activation, disease progression) involve continuous changes in gene expression rather than discrete states. Trajectory analysis orders cells along a “pseudotime” axis that recapitulates the dynamic process.

Monocle3

Learns a principal graph (trajectory) through the UMAP embedding using reversed graph embedding. Assigns each cell a pseudotime value representing its position along the trajectory. Supports branching trajectories for divergent differentiation paths.

Slingshot

Fits smooth curves through PCA or diffusion map space. Uses cluster information to define trajectory start and end points. Well-suited for simpler, tree-like trajectories.

scVelo

Infers RNA velocity—the time derivative of gene expression—from the ratio of spliced to unspliced mRNA counts in each cell. RNA velocity vectors indicate the future transcriptional state of each cell, providing directionality to trajectories.

CellRank

Combines RNA velocity with expression similarity to compute transition probabilities between cells. Identifies initial and terminal states and the most likely differentiation fates using Markov chain analysis.

RNA Velocity: Mathematical Framework

RNA velocity exploits the fact that standard scRNA-seq protocols capture both spliced (mature) and unspliced (intron-containing) mRNA molecules. The ratio between unspliced and spliced counts provides information about the *direction* of transcriptional change for each gene in each cell.

The core dynamical model for a single gene describes the time evolution of unspliced (u) and spliced (s) mRNA abundances:

$$\frac{du}{dt} = \alpha(t) - \beta u \quad (11.7)$$

$$\frac{ds}{dt} = \beta u - \gamma s \quad (11.8)$$

where $\alpha(t)$ is the transcription rate, β is the splicing rate (conversion of unspliced to spliced mRNA), and γ is the degradation rate of spliced mRNA. The RNA *velocity* for each gene is defined as ds/dt , representing the expected rate of change of the mature mRNA.

Steady-state model (velocityto)

Assumes that genes are in either an induction or repression phase, each at approximate steady state. At steady state, $ds/dt = 0$ implies $\beta u = \gamma s$, so the expected ratio is $u/s = \gamma/\beta$. Cells deviating above this ratio (excess unspliced) are actively upregulating the gene; cells below are downregulating it. The velocity is estimated as:

$$v_g = \beta u_g - \gamma_g s_g \quad (11.9)$$

where γ_g is estimated from the slope of the upper quantile of the u vs. s phase portrait.

Dynamical model (scVelo)

Relaxes the steady-state assumption by solving the full system of ODEs (Equations 11.7–11.8) using an expectation-maximization algorithm. This jointly infers the kinetic parameters α , β , γ , and a latent time for each cell, providing a more accurate velocity field for genes that are not at steady state (e.g., during rapid transitions).

Multi-omic velocity (MultiVelo)

Extends the velocity framework to incorporate chromatin accessibility (from multi-omic ATAC + RNA data), modeling the cascade from chromatin opening to unspliced to spliced mRNA.

Practical Tip

RNA velocity estimates are gene-specific vectors in expression space. To visualize them on a UMAP or t-SNE embedding, the high-dimensional velocity vectors are projected onto the 2D embedding by computing the correlation between each cell's velocity and the expression difference to its neighbors. Arrows on the embedding indicate the predicted direction of transcriptional change, not physical movement.

11.6.11 Cell–Cell Communication

CellChat

Infers intercellular communication networks from scRNA-seq data using a curated database of ligand–receptor interactions (including multimeric complexes). Models communication probability based on the law of mass action and identifies significant signaling pathways between cell types.

NicheNet

Predicts which ligands from sender cells best explain gene expression changes in receiver cells, integrating prior knowledge of signaling and gene regulatory networks.

LIANA

A framework that benchmarks and combines multiple cell–cell communication tools (CellPhoneDB, CellChat, NATMI, SingleCellSignalR, etc.) to produce consensus interaction scores.

11.6.12 Gene Regulatory Network Inference

SCENIC / pySCENIC

Single-Cell rEgulatory Network Inference and Clustering. A three-step pipeline:

1. Co-expression modules: identify genes co-expressed with each transcription factor (TF) using GRNBoost2 or GENIE3.
2. Regulon pruning: retain only target genes with TF binding motifs in their regulatory regions (cis-regulatory analysis using RcisTarget or pycisTarget).
3. Regulon activity scoring: quantify the activity of each regulon (TF + targets) in each cell using AUCell.

SCENIC regulon activity provides a biologically interpretable dimensionality reduction and can reveal TF-driven cell states invisible in gene expression alone.

11.7 Analysis Frameworks: Scanpy vs. Seurat

Two comprehensive frameworks dominate the scRNA-seq analysis ecosystem:

Table 11.1: Comparison of the Scanpy (Python) and Seurat (R) scRNA-seq analysis frameworks.

Feature	Scanpy (Python)	Seurat (R)
Language	Python	R
Current version	Scanpy 1.10+	Seurat v5
Data structure	AnnData (.h5ad)	SeuratObject / BPCells
Core functions	Preprocessing, clustering, DE, visualization	Preprocessing, clustering, DE, visualization
Normalization	Log-norm, scran (via rpy2)	Log-norm, SCTransform, sketch-based
Integration	scVI, scANVI, BBKNN, Harmony	CCA, RPCA, Harmony, scVI bridge
Scalability	Excellent (sparse arrays, GPU via rapids-singlecell)	Improved in v5 (BPCells on-disk), sketch-based
Visualization	Matplotlib-based (highly customizable)	ggplot2-based (publication-ready)
Ecosystem	scvi-tools, squidpy, CellRank, decoupler	Signac, Azimuth, BPCells
Community	Strong in computational biology / ML	Strong in biomedical research

Practical Tip

Both frameworks are excellent and actively maintained. Choose based on your language proficiency and collaborators' preferences. Many analyses use both: Scanpy for preprocessing and Seurat for visualization (or vice versa). Conversion between AnnData (.h5ad) and SeuratObject formats is straightforward using the `SeuratDisk` or `MuDataSeurat` packages.

11.8 Integration of Multiple Datasets

Combining scRNA-seq datasets from different experiments, batches, or technologies is essential for large-scale analyses but is complicated by batch effects. Integration methods aim to place cells from different batches into a common embedding while preserving biological variation.

Harmony

Fast linear integration that iteratively adjusts PCA embeddings to remove batch effects while preserving biological clusters. Extremely popular due to speed and simplicity. Works as a post-PCA correction.

scVI (Variational Inference)

Deep generative model (variational autoencoder) that learns a batch-corrected latent space. Handles complex batch effects and provides uncertainty estimates. The gold standard for large, complex integration tasks.

BBKNN (Batch Balanced KNN)

Modifies the KNN graph construction to balance neighbors across batches. Simple and effective; does not modify the expression matrix.

Seurat integration (CCA/RPCA)

Identifies “anchors” (mutual nearest neighbors) between datasets using canonical correlation analysis (CCA) or reciprocal PCA (RPCA). Anchors guide the alignment of datasets into a common space. RPCA is faster and recommended for datasets with partially overlapping cell types.

Important Caveat

Over-integration can remove genuine biological differences between datasets. Always verify that integration preserved known biological signals (e.g., marker gene expression for expected cell types). Compare integrated and non-integrated results critically, and use integration metrics (kBET, LISI, silhouette width) to quantify batch mixing and biological conservation.

11.8.1 Batch Correction Methods: A Detailed Comparison

Choosing the right integration method depends on dataset size, batch complexity, and the importance of preserving subtle biological variation. The following table summarizes the key properties of the most widely used methods:

Table 11.2: Comparison of batch correction and integration methods for scRNA-seq.

Method	Approach	Speed	Preservation of biology	Notes
Harmony	Iterative soft clustering on PCA embeddings; adjusts cluster centroids across batches	Very fast (<1 min for 500k cells)	Good; may over-correct rare populations	Post-PCA; does not modify expression matrix; easy to use
MNN (Mutual Nearest Neighbors)	Identifies mutual nearest neighbor pairs across batches; computes correction vectors	Moderate	Good for conserved populations	Implemented in batchelor (R); foundational method
scVI	Variational autoencoder with batch as a covariate in the generative model	Slow (GPU recommended; minutes–hours)	Excellent; learns complex non-linear corrections	Produces a latent space, not corrected counts; best for complex multi-batch designs
BBKNN	Constructs a batch-balanced KNN graph by trimming neighbors per batch	Very fast	Good; minimal distortion of rare types	Corrects the graph, not the embedding or expression; use with Scanpy
Scanorama	Panoramic stitching inspired by image alignment; finds mutual nearest neighbors in SVD space	Fast (scales to millions of cells)	Good; robust to partially overlapping datasets	Can correct expression matrix directly; useful when datasets share only some cell types

Metrics for Evaluating Integration Quality

Integration quality should be assessed along two orthogonal axes:

- **Batch mixing** (technical correction): Do cells of the same type from different batches intermingle? Measured by kBET (k-nearest neighbor batch effect test) and the iLISI (integration Local Inverse Simpson Index). Higher values indicate better batch mixing.
- **Biological conservation**: Are genuine cell types still distinguishable after integration? Measured by cLISI (cell-type LISI), silhouette width, and ARI (Adjusted Rand Index) against known labels. Over-integration reduces these metrics.

The **scIB** (single-cell Integration Benchmarking) framework provides a standardized pipeline for computing these metrics and ranking integration methods for a given dataset.

11.9 Large-Scale Cell Atlases

The field has moved from profiling individual tissues to constructing organism-wide cell atlases:

Human Cell Atlas (HCA)

An international consortium aiming to create a comprehensive reference map of every human cell type. Spans dozens of organs and developmental stages. Data accessible via the HCA Data Portal.

Tabula Sapiens

A multi-organ, single-cell transcriptomic atlas of *Homo sapiens*. Profiled ~500,000 cells from 24 organs of individual donors, enabling cross-tissue cell type comparisons.

Tabula Muris / Tabula Muris Senis

Mouse cell atlases covering 20+ organs using both 10x Chromium (3') and Smart-seq2 (full-length) protocols. Senis extends to aging, covering mice from 1 to 30 months.

CellxGene Census

Aggregated and harmonized single-cell data from multiple studies, providing programmatic access (Python/R APIs) to tens of millions of cells with standardized metadata and cell type annotations.

11.10 Practical Considerations

11.10.1 How Many Cells to Sequence?

- **For common cell types**: 3,000–5,000 cells per sample is often sufficient.
- **For rare populations**: If a target cell type is ~1% of the sample, profiling 10,000–20,000 cells ensures ~100–200 cells of that type.
- **For differential abundance testing**: Multiple biological replicates ($n \geq 3$) are far more important than the number of cells per sample.
- **Trade-off**: More cells at lower depth vs. fewer cells at higher depth. For cell type identification, breadth (more cells) generally wins; for gene-level analyses within a cell type, depth matters more.

11.10.2 Sequencing Depth Per Cell

- **3' 10x Chromium:** 20,000–50,000 reads per cell is standard. Beyond 50,000 reads, the number of newly detected genes saturates rapidly.
- **5' 10x Chromium:** Similar depth for GEX; additional depth for V(D)J libraries (~5,000 reads/cell).
- **Smart-seq2:** 0.5–1 million reads per cell (full-length, no UMIs; deeper sequencing improves isoform detection).
- **Multiplexed experiments:** Calculate total reads as (cells × reads/cell) and ensure the sequencing run provides sufficient output.

11.10.3 Tissue Dissociation Considerations

Preparing a high-quality single-cell suspension is often the most challenging step in a scRNA-seq experiment:

- **Enzymatic digestion:** Collagenase, trypsin, Liberase, or tissue-specific cocktails. Over-digestion kills cells; under-digestion yields clumps and debris.
- **Mechanical dissociation:** Gentle trituration or the Miltenyi gentleMACS for standardized processing.
- **Dead cell removal:** MACS dead cell removal kit or FACS-based viability gating (DAPI, 7-AAD, calcein AM).
- **Dissociation-induced artifacts:** Enzymatic digestion can activate stress-response genes (e.g., *FOS*, *JUN*, *HSPA1A*). Cold-active protease protocols or fixation-based approaches (10x Flex, FANS for nuclei) mitigate this.
- **Single-nucleus RNA-seq (snRNA-seq):** For tissues that cannot be dissociated into viable single cells (e.g., brain, adipose, archived frozen tissue), nuclei are isolated and profiled. Uses the same 10x Chromium platform with nuclei-specific protocols. Detects both cytoplasmic and nuclear RNA (more intronic reads).

Important Caveat

Always check for dissociation-induced gene expression artifacts by examining immediately early gene expression across clusters. If stress genes (e.g., *FOS*, *JUN*, *ATF3*) dominate the first principal components or separate clusters, consider regressing them out or switching to a cold-dissociation or nucleus-based protocol.

11.11 Comparison of scRNA-seq Platforms

11.12 Snakemake Workflow for scRNA-seq

11.13 Nextflow Workflow for scRNA-seq

Practical Tip

For production scRNA-seq analysis, the `nf-core/scrnaseq` pipeline provides a comprehensive Nextflow workflow supporting Cell Ranger, STARsolo, alevin-fry (Salmon), and kallisto-bustools as quantification backends, with automatic whitelist selection and extensive QC reporting.

Listing 11.1: Snakemake workflow for 10x Chromium scRNA-seq analysis using STARsolo and Scanpy.

```

1 # Snakefile for scRNA-seq Analysis (10x Chromium)
2 # =====
3 # Pipeline: STARsolo -> Scanpy (QC, norm, cluster, annotate)
4
5 import pandas as pd
6
7 configfile: "config/config.yaml"
8
9 samples = pd.read_csv(config["samples"], sep="\t")
10 SAMPLES = samples["sample"].tolist()
11
12 STAR_INDEX = config["star_index"]
13 WHITELIST = config["barcode_whitelist"]
14
15 rule all:
16     input:
17         expand("results/starsolo/{sample}/Solo.out/"
18             "Gene/filtered/matrix.mtx",
19             sample=SAMPLES),
20         "results/scanpy/integrated_analysis.h5ad",
21         "results/scanpy/umap_celltypes.pdf",
22         "results/multiqc/multiqc_report.html"
23
24 rule starsolo:
25     input:
26         r1="data/raw/{sample}_R1.fastq.gz",
27         r2="data/raw/{sample}_R2.fastq.gz",
28         index=STAR_INDEX,
29         whitelist=WHITELIST
30     output:
31         bam="results/starsolo/{sample}/"
32             "Aligned.sortedByCoord.out.bam",
33         matrix="results/starsolo/{sample}/Solo.out/"
34             "Gene/filtered/matrix.mtx",
35         barcodes="results/starsolo/{sample}/Solo.out/"
36             "Gene/filtered/barcodes.tsv",
37         features="results/starsolo/{sample}/Solo.out/"
38             "Gene/filtered/features.tsv",
39         log="results/starsolo/{sample}/Log.final.out"
40     threads: 12
41     resources:
42         mem_mb=40000
43     params:
44         prefix="results/starsolo/{sample}/",
45         cb_len=16,
46         umi_len=12
47     conda: "envs/starsolo.yaml"
48     shell:
49         """
50         STAR --runMode alignReads \
51             --genomeDir {input.index} \
52             --readFilesIn {input.r2} {input.r1} \
53             --readFilesCommand zcat \
54             --soloType CB_UMI_Simple \
55             --soloCBwhitelist {input.whitelist} \
56             --soloCBlen {params.cb_len} \
57             --soloUMIlen {params.umi_len} \
58             --soloFeatures Gene \
59             --soloMultiMappers EM \
60             --soloCellFilter EmptyDrops_CR \
61             --outSAMtype BAM SortedByCoordinate \
62             --outSAMattributes NH HI SM AS CR IR UB \

```


Table 11.3: Comparison of major scRNA-seq platforms and methods.

Feature	10x Chromium 3'	Smart-seq2/3	Drop-seq	sci-RNA-seq3
Isolation	Droplet	FACS to plate	Droplet	Combinatorial indexing
Throughput	500–10,000 cells	96–384 cells	2,000–10,000 cells	>100,000 cells
Sensitivity	3,000–6,000 genes/cell	5,000–10,000 genes/cell	1,500–3,000 genes/cell	1,000–2,000 genes/cell
Coverage	3' end	Full-length	3' end	3' end
UMIs	Yes (12 nt)	No (SS2) / Yes (SS3)	Yes	Yes
Cost per cell	\$0.10–0.50	\$5–20	\$0.05–0.20	\$0.01–0.05
Equipment	Chromium Controller	FACS + thermocycler	Custom microfluidics	Standard lab equipment
Isoform info	No	Yes	No	No
Multiplexing	CellPlex, Flex	Limited	Limited	Built-in
Best use case	Standard profiling	Deep per-cell profiling	Budget-friendly scaling	Organism-scale atlases

11.14 Chapter Summary

Single-cell RNA sequencing has revolutionized our ability to dissect cellular heterogeneity and has become an indispensable tool in modern biology. In this chapter we covered:

- Why single-cell resolution is necessary and what bulk RNA-seq misses.
- The historical development of scRNA-seq technologies from 2009 to the present.
- Cell isolation methods: FACS, microfluidics, droplet-based (10x Chromium, Drop-seq, inDrop), combinatorial indexing (sci-RNA-seq, SPLiT-seq), and plate-based (Smart-seq2/3).
- The 10x Genomics Chromium system in detail: GEM formation, cell barcodes, UMIs, 3' vs. 5' chemistry, Cell Ranger, and the Flex assay.
- Computational challenges: sparsity/dropout, doublet detection, and ambient RNA contamination.
- The complete analysis workflow: raw data processing (Cell Ranger, STARsolo, alevin-fry), QC filtering, normalization (scran, SCTransform), feature selection (HVGs), dimensionality reduction (PCA, t-SNE, UMAP), graph-based clustering (Leiden/Louvain), cell type annotation, differential expression (pseudobulk recommended), trajectory analysis (Monocle3, scVelo), cell–cell communication (CellChat, NicheNet), and gene regulatory networks (SCENIC).
- Scanpy vs. Seurat frameworks, and dataset integration methods.
- Large-scale cell atlases: Human Cell Atlas, Tabula Sapiens, Tabula Muris.
- Practical workflows in both Snakemake and Nextflow.

The next chapter extends our view from dissociated single cells to cells in their native tissue context with spatial transcriptomics (Chapter 12).

Chapter 12

Spatial Transcriptomics

Key Concept

Spatial transcriptomics measures gene expression while preserving the physical location of each measurement within a tissue section. By retaining spatial context—lost in both bulk and dissociated single-cell RNA-seq—these technologies reveal how cells are organized, how they communicate with their neighbors, and how tissue architecture relates to function and disease.

12.1 Why Spatial Context Matters

The human body is not a bag of cells: it is a precisely organized structure where cellular function depends critically on spatial position. Conventional scRNA-seq requires tissue dissociation, which destroys the spatial relationships between cells. This loss has several consequences:

- **Tissue architecture is invisible:** The layered organization of the cortex, the crypt–villus axis of the intestine, and the zonal structure of the liver cannot be observed in dissociated data.
- **Cell neighborhoods are lost:** A tumor cell adjacent to a cytotoxic T cell behaves differently from one surrounded by immunosuppressive macrophages. Spatial context determines these interactions.
- **Gradients cannot be detected:** Morphogen gradients during development, oxygen gradients in tumors, and signaling gradients in wound healing require spatial measurement.
- **Rare niches are obscured:** Stem cell niches, germinal centers, and tertiary lymphoid structures are defined by their spatial organization as much as by their cell type composition.
- **Dissociation introduces artifacts:** Enzymatic digestion activates stress-response genes and selectively loses fragile cell types (neurons, adipocytes, syncytiotrophoblasts).

Spatial transcriptomics solves these problems by measuring gene expression *in situ*—within intact tissue sections.

12.2 Categories of Spatial Technologies

Spatial transcriptomics technologies can be broadly divided into three categories based on their detection mechanism: sequencing-based methods that capture RNA at defined spatial positions, imaging-based methods that visualize individual RNA molecules, and in situ sequencing methods that read out RNA sequences directly in tissue.

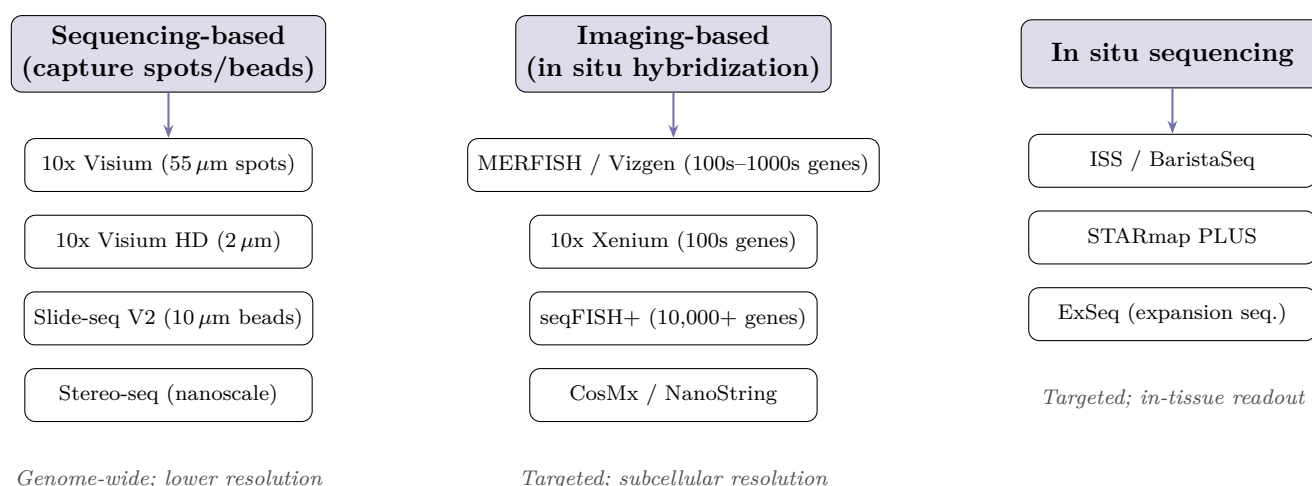


Figure 12.1: Overview of spatial transcriptomics technology categories. Sequencing-based methods (left) capture RNA at spatially barcoded positions and sequence it off-tissue. Imaging-based methods (center) visualize individual RNA molecules in situ using fluorescent probes. In situ sequencing methods (right) perform the sequencing reaction directly within the tissue section.

12.2.1 Sequencing-Based Spatial Methods

These methods capture RNA from intact tissue sections onto spatially barcoded surfaces. The tissue is permeabilized, RNA molecules are captured by underlying barcoded oligonucleotides, and the resulting cDNA is sequenced off-tissue using standard Illumina platforms. The spatial barcode allows each transcript to be mapped back to its location in the tissue.

10x Visium

The most widely adopted spatial transcriptomics platform:

10x Visium Workflow

1. Mount a fresh-frozen or FFPE tissue section (10 μm thick) onto a Visium capture slide.
2. Stain and image the section (H&E for fresh-frozen; immunofluorescence for FFPE).
3. Permeabilize the tissue to release mRNA (fresh-frozen) or after probe hybridization (FFPE).
4. mRNA binds to spatially barcoded oligo-dT probes printed on the slide surface.
5. Reverse transcription incorporates the spatial barcode into cDNA.
6. cDNA is collected, amplified, and sequenced on Illumina.
7. Reads are mapped and demultiplexed by spatial barcode using **Space Ranger**.

Key specifications:

- **Resolution:** 55 μm diameter capture spots arranged in a hexagonal grid with 100 μm center-to-center spacing. Each spot covers 1–10 cells (depending on tissue and cell size).
- **Gene coverage:** Genome-wide (no panel required); typically $\sim 3,000$ – $8,000$ genes detected per spot.
- **Capture area:** 6.5 mm $\times$ 6.5 mm, containing $\sim 5,000$ spots.
- **Sequencing depth:** 50,000–100,000 reads per spot recommended.
- **Sample types:** Fresh-frozen and FFPE (with dedicated Visium for FFPE workflow using probe-based capture).

Important Caveat

Visium spots are not single cells. Each $55\text{ }\mu\text{m}$ spot typically contains multiple cells, creating a mixture of cell types within each spot. Computational deconvolution methods (see [Section 12.4.1](#)) are needed to infer cell type composition per spot.

10x Visium HD

The next-generation Visium platform with dramatically improved resolution:

- **Resolution:** $2\text{ }\mu\text{m} \times 2\text{ }\mu\text{m}$ barcoded squares, providing near-single-cell or subcellular resolution.
- **Binning:** Data can be analyzed at $2\text{ }\mu\text{m}$, $8\text{ }\mu\text{m}$, or $16\text{ }\mu\text{m}$ resolution by binning adjacent squares.
- **Gene coverage:** Genome-wide.
- **Continuous tiling:** No gaps between capture positions, unlike the original Visium.
- **Sample types:** FFPE tissues; fresh-frozen support was added subsequently.
- Analyzed with **Space Ranger** (HD mode).

Slide-seq and Slide-seqV2

- **Principle:** A dense monolayer of DNA-barcoded $10\text{ }\mu\text{m}$ beads (“puck”) is used as the capture surface. Beads are decoded by sequential hybridization (*Slide-seq*) or by sequencing (Slide-seqV2) to determine their spatial position.
- **Resolution:** $10\text{ }\mu\text{m}$ (near single-cell for most tissues).
- **Gene detection:** Lower than Visium (~ 500 – $2,000$ genes per bead), but higher spatial resolution.
- **Open-source:** Protocols and analysis tools are publicly available.

Stereo-seq (BGI)

- **Principle:** DNA nanoball (DNB)-based spatial barcoding on a patterned array. Achieves nanoscale resolution ($\sim 500\text{ nm}$ spot size).
- **Capture area:** Up to full tissue sections (centimeter scale).
- **Resolution:** Subcellular ($\sim 0.5\text{ }\mu\text{m}$), binnable to any desired resolution.
- **Notable application:** Used to create the first spatially resolved transcriptomic atlas of an entire mouse embryo (MOSTA).
- **Gene coverage:** Genome-wide.

Other Sequencing-Based Methods**HDST (High-Definition Spatial Transcriptomics)**

Uses $2\text{ }\mu\text{m}$ barcoded wells. An early demonstration of high-resolution capture-based spatial transcriptomics.

Seq-Scope

Repurposes Illumina flow cells as spatial capture surfaces, achieving ~ 0.5 – $1\text{ }\mu\text{m}$ resolution with genome-wide gene coverage at very low cost per unit area.

12.2.2 Imaging-Based Spatial Methods (In Situ Hybridization)

Imaging-based methods detect individual RNA molecules in intact tissue by hybridizing fluorescent probes directly to transcripts. They achieve subcellular resolution and can image specific panels of 100 – $10,000+$ genes.

MERFISH (Multiplexed Error-Robust FISH)

Developed by the Zhuang lab (Harvard) and commercialized by Vizgen:

How MERFISH Works

1. Design encoding probes for each target gene. Each gene is assigned a unique binary barcode (e.g., a 16-bit code using a modified Hamming distance error-correcting scheme).
2. Hybridize all encoding probes simultaneously to the tissue.
3. Perform multiple rounds of sequential imaging:
 - In each round, readout probes complementary to specific barcode positions bind and fluoresce.
 - Image the tissue, record fluorescent spots.
 - Strip the readout probes (photobleach or chemically remove).
 - Repeat for the next round.
4. Each RNA molecule generates a pattern of ON/OFF signals across rounds, spelling out its binary barcode.
5. Error correction (Hamming distance codes) allows identification and correction of single-bit errors, dramatically reducing false positives.

Key specifications:

- **Gene panel:** 100–1,000+ genes (MERSCOPE platform from Vizgen supports panels of 140–1,000 genes; custom panels possible).
- **Resolution:** Single-molecule, subcellular (~100 nm localization).
- **Throughput:** Millions of cells per tissue section.
- **Imaging time:** 12–48 hours per tissue section depending on panel size and area.

seqFISH+

- An extension of seqFISH that profiles >10,000 genes per cell.
- Uses sequential rounds of hybridization with pseudo-color barcoding (multiple fluorescent channels per round).
- Achieves transcriptome-scale imaging but at lower throughput than MERFISH due to the large number of imaging rounds required.
- Academic protocol; not commercially available as a turnkey system.

10x Xenium

The in situ platform from 10x Genomics:

- **Principle:** Padlock probe-based detection. Pairs of probes hybridize to target RNA; when correctly paired, they are ligated and amplified via rolling circle amplification (RCA), creating a bright fluorescent spot.
- **Gene panel:** Pre-designed panels (human brain, breast cancer, lung, immune, multi-tissue) covering 300–500 genes, plus custom panels up to 480 genes. Higher multiplex panels continue to be released.
- **Resolution:** Subcellular (detects individual transcripts within cells and can resolve nuclear vs. cytoplasmic localization).
- **Throughput:** Large tissue areas (full tissue sections); automated instrument with ~24-hour run time per slide.
- **FFPE compatible:** Works with FFPE tissues, enabling retrospective spatial analysis of archival samples.

- **Integrated analysis:** Results can be directly overlaid with Visium data from adjacent sections for combined genome-wide + targeted spatial analysis.

CosMx Spatial Molecular Imager (NanoString)

- Probe-based in situ hybridization platform.
- Gene panels of 1,000–6,000 genes for RNA and 64–100 proteins simultaneously (multi-omic spatial profiling).
- Subcellular resolution; automated imaging system.
- Compatible with FFPE tissues.
- Multi-omic capability (RNA + protein in the same tissue section) is a key differentiator.

FISH-Based Principles

All imaging-based spatial methods rely on variations of fluorescence in situ hybridization (FISH):

Single-molecule FISH (smFISH)

The foundational method: multiple fluorescent probes tile along a single RNA transcript, producing a diffraction-limited spot visible under a fluorescence microscope. Detects one gene per fluorescence channel (typically 3–5 genes per experiment).

Combinatorial barcoding

MERFISH and seqFISH assign each gene a binary barcode read out across multiple rounds of imaging. This exponentially scales the number of genes: n rounds with error correction can encode thousands of genes.

Sequential hybridization

Probes are applied, imaged, removed, and replaced in successive rounds. Each round reveals a different subset of genes or barcode positions.

12.2.3 In Situ Sequencing

In situ sequencing (ISS) methods perform the sequencing reaction directly within the tissue, reading out short barcodes that identify each transcript.

ISS (In Situ Sequencing)

Padlock probes hybridize to target RNAs; gaps are filled and ligated to form circular DNA. Rolling circle amplification (RCA) creates dense DNA nanoballs that are sequenced in situ by sequencing-by-ligation. Typically reads 4–5 bases (sufficient for barcode identification).

BaristaSeq

An improved ISS method with higher efficiency gap-filling and sequencing-by-synthesis readout. Compatible with tissue clearing for 3D spatial transcriptomics.

STARmap / STARmap PLUS

Uses SNAiL (specific amplification of nucleic acids via intramolecular ligation) probes for highly multiplexed in situ sequencing. STARmap PLUS scales to 1,000+ genes. Compatible with intact tissue (hydrogel-embedded) for volumetric 3D spatial transcriptomics.

ExSeq (Expansion Sequencing)

Combines expansion microscopy (physically enlarging the tissue by embedding in a swellable hydrogel) with in situ RNA sequencing. Achieves nanoscale resolution of RNA localization within subcellular compartments (dendrites, synapses).

12.3 Comparison of Spatial Transcriptomics Platforms

Table 12.1: Comprehensive comparison of spatial transcriptomics platforms.

Feature	Visium	Visium HD	MERFISH	Xenium	Stereo-seq
Category	Seq-based	Seq-based	Imaging	Imaging	Seq-based
Resolution	55 μm	2 μm	Subcellular	Subcellular	0.5 μm
Gene coverage	Genome-wide	Genome-wide	100–1,000	300–500	Genome-wide
Cells per section	~5,000 spots	~millions of bins	Millions	Millions	Millions
FFPE compatible	Yes (v2)	Yes	Limited	Yes	Yes
Instrument	None (slide)	None (slide)	MERSCOPE	Xenium Analyzer	STOmics
Run time	1–2 days (wet lab)	1–2 days (wet lab)	12–48 h imaging	~24 h	1–2 days
Analysis software	Space Ranger	Space Ranger	Vizgen MERSCOPE	Xenium Ranger/Explorer	STOmics tools
Cost per section	\$\$\$	\$\$\$	\$\$\$\$	\$\$\$	\$\$\$
Best use case	Genome-wide discovery	High-res discovery	Targeted, sub-cellular	Clinical FFPE, targeted	Large-area mapping

Table 12.2: Extended comparison including additional platforms.

Feature	Slide-seqV2	seqFISH+	CosMx	STARmap PLUS	Seq-Scope
Category	Seq-based	Imaging	Imaging	In situ seq.	Seq-based
Resolution	10 μm	Subcellular	Subcellular	Subcellular	~0.6 μm
Gene coverage	Genome-wide	10,000+	1,000–6,000	1,000+	Genome-wide
Multi-omic	No	No	RNA + protein	No	No
Commercial	No (academic)	No (academic)	Yes (NanoString)	No (academic)	No (academic)
3D capable	No	Limited	No	Yes	No

Practical Tip

Choosing a spatial technology depends on your biological question:

- **Unbiased discovery (genome-wide):** Visium, Visium HD, or Stereo-seq.
- **Targeted subcellular resolution:** MERFISH (Vizgen), Xenium (10x), or CosMx (NanoString).
- **FFPE clinical samples:** Xenium, Visium FFPE, or CosMx.
- **Combined RNA + protein:** CosMx (NanoString).
- **3D spatial transcriptomics:** STARmap PLUS or ExSeq.

Many studies now combine a genome-wide method (e.g., Visium HD) with a targeted imaging method (e.g., Xenium) on serial sections to get both breadth and resolution.

12.4 Computational Analysis for Spatial Transcriptomics

Spatial transcriptomics data requires specialized computational methods that account for the spatial dimensions in addition to the gene expression matrix.

12.4.1 Spot Deconvolution

For Visium and other methods where each measurement captures multiple cells, deconvolution estimates the proportion of each cell type within each spatial location by leveraging a single-cell RNA-seq reference:

cell2location

Bayesian model that estimates absolute cell type abundances per spot by decomposing the spatial expression profile into a mixture of reference scRNA-seq signatures. Provides uncertainty estimates and handles overdispersion. One of the most widely used and best-performing methods in benchmarks.

RCTD (Robust Cell Type Decomposition)

Part of the `spacexr` R package. Fits a weighted regression model to estimate cell type proportions. Fast and robust; handles platform-specific technical differences between spatial and scRNA-seq data.

Tangram

Deep learning-based method that maps scRNA-seq cells to spatial locations by learning an optimal alignment between the two modalities. Can operate at both spot (Visium) and single-cell (Xenium, MERFISH) resolution.

SPOTlight

Uses non-negative matrix factorization (NMF) to decompose Visium spots into cell type proportions.

stereoscope

Probabilistic method based on negative binomial distributions for deconvolving spatial spots.

Why Deconvolution Needs a scRNA-seq Reference

Deconvolution is fundamentally a source-separation problem: given a mixture signal (the spatial spot), determine the contributing sources (cell types). This requires knowing what each source looks like in isolation, which comes from a scRNA-seq reference of the same tissue. The quality of deconvolution depends critically on the completeness and accuracy of the reference atlas. Missing cell types in the reference will be missed in the spatial data.

Mathematical Framework for Spot Deconvolution

The central idea behind methods like RCTD and `cell2location` is that the observed expression profile at each spatial location is a mixture of cell-type-specific expression signatures. Formally, for spot s with observed gene expression vector Y_s (a vector of length G , one entry per gene), the linear mixture model is:

$$Y_s = \sum_{k=1}^K w_{sk} \cdot \mu_k + \epsilon_s \quad (12.1)$$

where K is the number of cell types in the reference, μ_k is the reference expression profile (mean expression vector) for cell type k , $w_{sk} \geq 0$ is the weight (proportion or abundance) of cell type k in spot s , and ϵ_s represents noise (technical and biological).

RCTD formulation

Models the observed counts at each spot as a weighted sum of cell-type profiles using

a platform-corrected regression framework. The weights w_{sk} are constrained to be non-negative and are estimated via maximum likelihood under a Poisson or negative binomial noise model. RCTD explicitly accounts for platform-specific effects (differences in capture efficiency and gene detection between the spatial and scRNA-seq platforms) by estimating platform-specific normalisation factors.

cell2location formulation

Uses a hierarchical Bayesian model where the expected expression at spot s for gene g is:

$$\mathbb{E}[Y_{sg}] = \sum_{k=1}^K w_{sk} \cdot g_{kg} \cdot y_s \quad (12.2)$$

where g_{kg} is the reference expression of gene g in cell type k (estimated from the scRNA-seq data), y_s is a spot-level scaling factor, and w_{sk} represents the absolute number of cells of type k at spot s . Bayesian inference (using variational inference in **Pyro**) provides full posterior distributions over all parameters, yielding uncertainty estimates for cell type abundances—a key advantage for downstream hypothesis testing.

Practical Tip

When choosing a deconvolution method, consider whether you need *proportions* (what fraction of each spot is cell type k ?) or *absolute abundances* (how many cells of type k ?). RCTD estimates proportions by default; **cell2location** estimates absolute abundances. For comparing cell type composition across spots, proportions suffice. For estimating total cell counts (e.g., immune infiltration density), absolute abundances are more informative.

12.4.2 Spatially Variable Gene Detection

Identifying genes whose expression varies as a function of spatial location, beyond what would be expected by chance:

SpatialDE

Gaussian process regression model that tests whether the spatial variance component of gene expression is significant. Identifies genes with spatial patterns (e.g., gradients, periodic patterns, hotspots) and classifies them by spatial pattern type.

SPARK

Spatial Pattern Recognition via Kernels. Tests for spatially variable expression using a generalized linear spatial model with multiple spatial kernel functions. More powerful than SpatialDE for detecting diverse spatial patterns.

Moran's I

A classical spatial autocorrelation statistic applied to gene expression. Positive Moran's I indicates spatial clustering; negative values indicate spatial dispersion. Simple and fast; implemented in Squidpy and Giotto.

nnSVG

Nearest-neighbor Gaussian process model for identifying spatially variable genes. Scales better than SpatialDE to high-resolution platforms like Visium HD.

Moran's I: Spatial Autocorrelation in Detail

Moran's I is the most widely used statistic for quantifying spatial autocorrelation of gene expression. For a gene with expression values $x_1, x_2, \dots, x_n$ measured at n spatial locations, Moran's I is defined as:

$$I = \frac{n}{\sum_i \sum_j w_{ij}} \cdot \frac{\sum_i \sum_j w_{ij} (x_i - \bar{x})(x_j - \bar{x})}{\sum_i (x_i - \bar{x})^2} \quad (12.3)$$

where $\bar{x} = \frac{1}{n} \sum_i x_i$ is the global mean expression, w_{ij} is the spatial weight between locations i and j (typically 1 if i and j are neighbors and 0 otherwise, or a distance-decay function), and the double sum runs over all pairs of locations.

Interpretation:

- $I > 0$: **Positive spatial autocorrelation** (clustering). Nearby locations tend to have similar expression values. Values close to +1 indicate strong spatial clustering.
- $I \approx 0$: **No spatial autocorrelation** (random spatial pattern). Expression is distributed independently of spatial position.
- $I < 0$: **Negative spatial autocorrelation** (dispersion). Nearby locations tend to have dissimilar expression values, forming a checkerboard-like pattern. This is rare for gene expression data.

The expected value under the null hypothesis of spatial randomness is $\mathbb{E}[I] = -1/(n-1)$, which approaches zero for large n . Statistical significance is assessed either analytically (under normality assumptions) or by a permutation test that randomly shuffles expression values across locations and recomputes I to build a null distribution.

Choosing the Spatial Weight Matrix

The spatial weight matrix $W = \{w_{ij}\}$ encodes the definition of “spatial proximity” and critically affects the results:

- **Binary adjacency:** $w_{ij} = 1$ if spots i and j are direct neighbors (e.g., adjacent hexagons in Visium). Simple and fast.
- **KNN graph:** $w_{ij} = 1$ for the k nearest neighbors of each spot. Robust when spot density varies across the tissue.
- **Distance decay:** $w_{ij} = f(d_{ij})$, e.g., Gaussian kernel $\exp(-d_{ij}^2/2h^2)$ or inverse distance. Captures longer-range spatial correlations (e.g., paracrine signaling gradients).

In `Squidpy`, the default spatial neighbors graph uses Delaunay triangulation or KNN, and Moran's I is computed via `sq.gr.spatial_autocorr(adata, mode="moran")`.

12.4.3 Spatial Domain Identification

Identifying tissue regions (“spatial domains”) with coherent gene expression programs—analogueous to clustering in scRNA-seq but incorporating spatial proximity:

BayesSpace

Bayesian approach that models spatial transcriptomics data on a lattice graph, encouraging neighboring spots to belong to the same cluster. Produces spatially smooth domain assignments. Supports Visium and Slide-seq data.

SpaGCN

Uses a graph convolutional network that integrates gene expression, spatial coordinates, and histology image features to identify spatial domains. Learns optimal representations that combine molecular and morphological information.

STAGATE

Graph attention auto-encoder that identifies spatial domains by learning a latent representation that incorporates spatial neighbor information via attention mechanisms.

BANKSY

Builds a neighborhood expression matrix that combines each spot's own expression with the average expression of its spatial neighbors. Standard clustering on this augmented matrix produces spatially coherent domains without specialized spatial algorithms.

12.4.4 Integration with scRNA-seq

Integrating spatial transcriptomics data with scRNA-seq enables cell type mapping and enhances the resolution of spatial data:

- **Deconvolution** (see [Section 12.4.1](#)): Infer cell type proportions at each spatial location.
- **Imputation**: Predict the expression of unmeasured genes in targeted spatial assays (MERFISH, Xenium) by transferring information from a genome-wide scRNA-seq reference. Tools: **Tangram**, **gimVI** (scvi-tools), **novoSpaRc**.
- **Label transfer**: Project cell type annotations from scRNA-seq reference onto spatial spots using anchor-based integration (Seurat) or probabilistic assignment (cell2location, Tangram).

12.4.5 Ligand–Receptor Analysis in Spatial Context

Spatial data enables direct testing of cell–cell communication hypotheses because the physical proximity of sender and receiver cells is known:

stLearn

Combines spatial distance with ligand–receptor co-expression to identify spatially constrained cell–cell interactions. Only considers interactions between cell types that are physically adjacent.

COMMOT

Uses optimal transport theory to model the spatial flow of signaling molecules from sender to receiver locations. Produces spatial vector fields showing the direction and magnitude of each signaling pathway.

MISTy

Multi-view framework that models gene expression at each spot as a function of its intrinsic state, its immediate neighborhood (juxtacrine signaling), and its broader tissue context (paracrine signaling). Identifies the spatial scale at which different pathways operate.

Squidpy

General-purpose spatial analysis framework (see below) that includes neighborhood enrichment analysis and ligand–receptor co-occurrence testing.

Ligand–Receptor Communication Scoring: Mathematical Framework

Spatial ligand–receptor analysis quantifies the probability or strength of signaling between spatially proximal cell populations. The **CellChat** framework (originally developed for scRNA-seq) and the spatial extension **COMMOT** both model communication probability based on the expression levels of ligand and receptor genes, weighted by spatial proximity.

CellChat communication probability. For a ligand–receptor pair (L, R) between sender cell type A and receiver cell type B , CellChat computes a communication score based on the law of mass action:

$$P_{L,R}^{A \rightarrow B} = \frac{\bar{x}_L^A \cdot \bar{x}_R^B}{K_h + \bar{x}_L^A \cdot \bar{x}_R^B} \quad (12.4)$$

where $\bar{x}_L^A$ is the average expression of ligand L in sender population A , $\bar{x}_R^B$ is the average expression of receptor R in receiver population B , and K_h is a half-saturation constant (Hill function parameterization) that prevents extreme scores when expression is very high. For multi-subunit receptors (e.g., heterodimeric complexes), the receptor expression is taken as the geometric mean of subunit expression levels.

Spatial weighting in COMMOT. The COMMOT framework extends ligand–receptor scoring to spatial transcriptomics by incorporating spatial distance as a modulating factor. For each pair of spatial locations (i, j) with distance d_{ij} , the spatial communication score is:

$$S_{ij}^{L,R} = x_{L,i} \cdot x_{R,j} \cdot K(d_{ij}) \quad (12.5)$$

where $x_{L,i}$ is the expression of ligand L at location i , $x_{R,j}$ is the expression of receptor R at location j , and $K(d_{ij})$ is a spatial kernel function that down-weights interactions at greater distances. Common choices include:

- **Cutoff kernel:** $K(d) = \mathbf{1}[d \leq d_{\max}]$ (binary; interactions only within a maximum range).
- **Gaussian kernel:** $K(d) = \exp(-d^2/2\sigma^2)$ (smooth decay with characteristic scale σ).

COMMOT uses optimal transport theory to aggregate these pairwise scores into **spatial communication vector fields**, showing the net direction and magnitude of signaling flow for each pathway across the tissue. These vector fields can reveal directional signaling patterns such as morphogen gradients or immune cell recruitment.

Important Caveat

Ligand–receptor scoring in spatial transcriptomics assumes that mRNA expression is a reasonable proxy for protein abundance and secretion. This is not always true: post-transcriptional regulation, protein half-life, and secretion dynamics can decouple mRNA and protein levels. Spatial proteomics data (e.g., from CosMx or CODEX) can provide complementary validation for key signaling interactions identified by transcriptomic methods.

12.4.6 Data Visualization and Analysis Frameworks

Squidpy

Python package for spatial omics analysis built on top of Scanpy/AnnData. Provides image processing (segmentation, feature extraction), graph construction (spatial neighbors), spatial statistics (Moran’s I, co-occurrence, centrality scores), and interactive visualization. The primary Python framework for spatial transcriptomics.

Giotto

R and Python package providing a comprehensive suite for spatial data analysis: pre-processing, spatial network construction, spatial gene detection, domain identification, cell–cell interaction, and visualization with interactive viewers.

Seurat (spatial module)

Seurat v5 includes native support for spatial data (Visium, Xenium, MERFISH, Slide-seq). Functions for spatial feature plots, integration with scRNA-seq, and spatial dimensionality reduction.

Voyager

R/Bioconductor package implementing spatial statistics methods (variograms, correlograms, Moran's I, Geary's C) for spatial transcriptomics, built on the SpatialFeatureExperiment data structure.

napari + spatialdata

Python imaging viewer (napari) combined with the spatialdata framework for multimodal spatial data handling (images, coordinates, annotations, expression). Enables interactive exploration of spatial datasets.

12.5 Tissue Handling and Sample Preparation

Spatial transcriptomics requires careful tissue preparation—often more demanding than standard sequencing experiments because tissue morphology must be preserved.

12.5.1 Tissue Sectioning

- **Fresh-frozen (FF):** Tissue is flash-frozen in OCT compound (optimal cutting temperature) or isopentane. Sectioned at 10 μm on a cryostat. Best RNA quality but limited long-term storage.
- **FFPE:** Formalin-fixed, paraffin-embedded. Sectioned at 5–10 μm on a microtome. RNA is crosslinked and fragmented but preserved indefinitely. Compatible with Visium FFPE, Xenium, CosMx.
- **Section quality:** Sections must be uniform in thickness, wrinkle-free, and fully adherent to the capture area. Tissue folds, tears, or bubbles create artifacts in spatial data.

12.5.2 Permeabilization Optimization

For sequencing-based methods (Visium), the tissue must be permeabilized to release mRNA while keeping it localized above its capture spot.

Visium Permeabilization Optimization

A tissue optimization (TO) experiment is performed before the main experiment:

1. Mount serial sections on a Visium TO slide.
2. Permeabilize each section for a different duration (e.g., 3, 6, 12, 18, 24, 30 minutes).
3. Perform reverse transcription with fluorescent nucleotides.
4. Image the fluorescent cDNA footprint under a microscope.
5. Select the permeabilization time that maximizes signal (cDNA fluorescence) while maintaining spatial resolution (sharp edges, no lateral diffusion).

Typical optimal permeabilization time: 6–18 minutes for most tissues, but varies widely (e.g., brain requires less, skeletal muscle requires more).

Important Caveat

Over-permeabilization causes RNA to diffuse laterally, blurring spatial resolution. Under-permeabilization results in poor RNA capture. Each tissue type requires its own optimization. Do not skip the tissue optimization step.

12.5.3 Imaging

- **H&E staining:** Standard histological stain for tissue morphology. Used with fresh-frozen Visium. The H&E image is registered to the spatial gene expression data, allowing correlation of transcriptomic patterns with morphological features.
- **Immunofluorescence (IF):** Antibody-based staining for specific proteins. Used with FFPE Visium and Xenium. Provides protein-level information that complements the transcriptomic data.
- **Cell segmentation:** For imaging-based methods, individual cells must be segmented from the tissue image. Methods include DAPI-based nuclear segmentation (watershed, Cellpose, StarDist) and membrane staining (phalloidin, WGA). Accurate segmentation is critical for assigning transcripts to individual cells.

12.6 Emerging Directions: Spatial Multi-Omics

12.6.1 Spatial Epigenomics

Spatial ATAC-seq

Methods such as spatial-ATAC-seq and Spatial-CUT&Tag map chromatin accessibility and histone modifications in tissue sections. These reveal spatially resolved regulatory landscapes (enhancer usage, TF binding) that complement spatial gene expression.

10x Visium + ATAC

Emerging protocols from 10x Genomics that combine gene expression and chromatin accessibility on the same Visium slide.

12.6.2 Spatial Proteomics

CODEX (CO-Detection by indEXing)

Iterative antibody staining and imaging that profiles 40–60 proteins at single-cell resolution in intact tissues. Uses DNA-conjugated antibodies and fluorescent reporters that are sequentially activated and quenched.

MIBI-TOF (Multiplexed Ion Beam Imaging)

Uses metal-conjugated antibodies and time-of-flight mass spectrometry for highly multiplexed protein imaging (>40 markers).

CosMx multi-omic:

Simultaneous RNA and protein detection on the same tissue section (see above).

12.6.3 Spatial Multi-Omic Integration

The frontier of spatial biology is measuring multiple molecular layers (transcriptome, epigenome, proteome, metabolome) in the same tissue section or in serial sections from the same block:

- Serial section approaches: alternate sections for different assays (e.g., Visium + Xenium + CODEX on serial 10 μ m sections).
- Same-section multi-omics: CosMx (RNA + protein), Visium + IF, and emerging combined ATAC + RNA spatial methods.
- Computational integration: Tools such as MEFISTO (multi-omic factor analysis in space and time) and MOFA+ enable joint analysis of spatial multi-omic datasets.

12.7 Snakemake Workflow for Spatial Transcriptomics

Listing 12.1: Snakemake workflow for 10x Visium spatial transcriptomics analysis.

```

1 # Snakefile for Spatial Transcriptomics (10x Visium)
2 # =====
3 # Pipeline: Space Ranger -> Squidpy / Scanpy analysis
4
5 import pandas as pd
6
7 configfile: "config/config.yaml"
8
9 samples = pd.read_csv(config["samples"], sep="\t")
10 SAMPLES = samples["sample"].tolist()
11
12 REFERENCE = config["spaceranger_ref"]
13
14 rule all:
15     input:
16         expand("results/spaceranger/{sample}/outs/"
17             "filtered_feature_bc_matrix.h5",
18             sample=SAMPLES),
19         "results/squidpy/spatial_analysis.h5ad",
20         "results/squidpy/spatial_domains.pdf",
21         "results/deconvolution/cell2location_results.h5ad"
22
23 rule spaceranger_count:
24     input:
25         fastqs="data/raw/{sample}/",
26         image="data/images/{sample}.tif",
27         reference=REFERENCE
28     output:
29         h5="results/spaceranger/{sample}/outs/"
30             "filtered_feature_bc_matrix.h5",
31         spatial="results/spaceranger/{sample}/outs/spatial/",
32         web="results/spaceranger/{sample}/outs/"
33             "web_summary.html"
34     threads: 12
35     resources:
36         mem_mb=64000
37     params:
38         outdir="results/spaceranger/{sample}",
39         slide=lambda wc: samples.loc[
40             samples["sample"] == wc.sample,
41             "slide_serial"].iloc[0],
42         area=lambda wc: samples.loc[
43             samples["sample"] == wc.sample,
44             "capture_area"].iloc[0]
45     shell:
46         """
47         spaceranger count \
48             --id={wildcards.sample} \
49             --transcriptome={input.reference} \
50             --fastqs={input.fastqs} \
51             --image={input.image} \
52             --slide={params.slide} \
53             --area={params.area} \
54             --localcores={threads} \
55             --localmem=60 \
56             --output-dir={params.outdir}
57         """
58
59 rule squidpy_analysis:
60     input:
61         h5_files=expand(
62             "results/spaceranger/{sample}/outs/"
63             "filtered_feature_bc_matrix.h5"

```

12.8 Nextflow Workflow for Spatial Transcriptomics

Practical Tip

For production spatial transcriptomics analysis, the `nf-core/spatialtranscriptomics` pipeline provides a Nextflow workflow supporting Space Ranger, quality control, normalization, clustering, and basic spatial analysis with standardized outputs.

12.9 Practical Experimental Design Considerations

12.9.1 Choosing the Right Technology

- **Discovery vs. hypothesis-driven:** If you need genome-wide unbiased discovery, use a sequencing-based method (Visium, Visium HD, Stereo-seq). If you have a defined gene panel, imaging-based methods (MERFISH, Xenium) provide higher resolution and single-cell quantification.
- **Sample type:** FFPE samples have the widest compatibility (Visium FFPE, Xenium, CosMx). Fresh-frozen tissue generally yields better RNA quality for sequencing-based methods.
- **Resolution requirements:** If single-cell resolution is essential, use imaging-based methods or Visium HD. Standard Visium (55 μm) requires computational deconvolution for cell type analysis.
- **Throughput:** For large cohorts (tens to hundreds of samples), consider the cost per section and instrument availability. Visium slides are relatively straightforward; imaging instruments have limited throughput (one section per ~ 24 hours).

12.9.2 Replication and Sample Size

- Biological replicates ($n \geq 3$ per condition) are needed for statistical testing of spatial gene expression differences.
- Technical replicates (serial sections from the same block) assess within-sample variability.
- Consider tissue heterogeneity: one section may not represent the entire organ. Multiple sections from different regions may be needed.

12.9.3 Combining Spatial and Single-Cell Data

Recommended Multi-Modal Experimental Design

For comprehensive tissue profiling, a combined approach is often optimal:

1. **scRNA-seq:** Generate a reference atlas of cell types from dissociated tissue (10x Chromium, 5,000–10,000 cells per sample, $n \geq 3$).
2. **Spatial transcriptomics (genome-wide):** Profile serial sections with Visium or Visium HD for unbiased spatial gene expression mapping.
3. **Spatial transcriptomics (targeted):** Profile adjacent sections with Xenium or MERFISH for single-cell resolution of key pathways.
4. **Integration:** Use `cell2location` or `Tangram` to project scRNA-seq cell types onto spatial data; impute unmeasured genes in targeted data.

This multi-modal approach provides genome-wide coverage (Visium), single-cell resolution (Xenium), and a comprehensive cell type reference (scRNA-seq).

Listing 12.2: Nextflow DSL2 workflow for 10x Visium spatial transcriptomics analysis.

```

1  #!/usr/bin/env nextflow
2  // Spatial Transcriptomics Pipeline (Nextflow DSL2)
3  // =====
4  // Pipeline: Space Ranger -> Squidpy -> cell2location
5
6  nextflow.enable.dsl = 2
7
8  params.sample_sheet = "config/samples.tsv"
9  params.reference     = "resources/refdata-gex-GRCh38-2024-A"
10 params.scrna_ref     = "resources/scrna_reference.h5ad"
11 params.outdir        = "results"
12
13 log.info """
14 =====
15 Spatial Transcriptomics Pipeline
16 =====
17 sample_sheet : ${params.sample_sheet}
18 reference    : ${params.reference}
19 scRNA ref    : ${params.scrna_ref}
20 outdir       : ${params.outdir}
21 =====
22 """
23
24 // Parse sample sheet
25 Channel
26     .fromPath(params.sample_sheet)
27     .splitCsv(header: true, sep: '\t')
28     .map { row ->
29         tuple(
30             row.sample,
31             file(row.fastq_dir),
32             file(row.image),
33             row.slide_serial,
34             row.capture_area
35         )
36     }
37     .set { samples_ch }
38
39 process SPACERANGER_COUNT {
40     tag "$sample_id"
41     publishDir "${params.outdir}/spaceranger/${sample_id}",
42         mode: 'copy'
43     cpus 12
44     memory '64 GB'
45
46     input:
47     tuple val(sample_id), path(fastqs), path(image),
48         val(slide), val(area)
49     path(reference)
50
51     output:
52     tuple val(sample_id),
53         path("outs/filtered_feature_bc_matrix.h5"),
54         path("outs/spatial/"),
55         emit: counts
56     path("outs/web_summary.html"), emit: web_summary
57
58     script:
59     """
60     spaceranger count \
61         --id=${sample_id} \
62         --transcriptome=${reference} \
63         --fastqs=${fastqs} \

```

12.10 Chapter Summary

Spatial transcriptomics represents one of the most exciting frontiers in genomics, bridging the gap between molecular profiling and tissue biology. In this chapter we covered:

- Why spatial context matters: tissue architecture, cell neighborhoods, gradients, and the limitations of dissociation-based methods.
- Sequencing-based spatial methods: 10x Visium (55 μm spots), Visium HD (2 μm), Slide-seqV2 (10 μm beads), Stereo-seq (nanoscale), and Seq-Scope.
- Imaging-based spatial methods: MERFISH (Vizgen), seqFISH+, 10x Xenium, and CosMx (NanoString), including the principles of combinatorial barcoding and sequential hybridization.
- In situ sequencing methods: ISS, BaristaSeq, STARmap PLUS, and ExSeq.
- Comprehensive platform comparisons covering resolution, gene coverage, throughput, and cost.
- Computational analysis: spot deconvolution (cell2location, RCTD, Tangram), spatially variable gene detection (SpatialDE, SPARK, Moran's I), spatial domain identification (BayesSpace, SpaGCN, STAGATE), scRNA-seq integration, and ligand–receptor analysis in spatial context (stLearn, COMMOT, MISTy).
- Visualization and analysis frameworks: Squidpy, Giotto, Seurat spatial module, Voyager.
- Tissue handling: sectioning (FF and FFPE), permeabilization optimization, imaging, and cell segmentation.
- Emerging directions: spatial ATAC-seq, spatial proteomics (CODEX, MIBI-TOF), and multi-omic spatial integration.
- Practical workflows in both Snakemake and Nextflow for Visium analysis.

Spatial transcriptomics, combined with the single-cell methods described in [Chapter 11](#) and the bulk approaches in [Chapter 10](#), provides a complete toolkit for transcriptomic analysis at every scale—from population-level averages to individual molecules in their native tissue context.

Part V

Epigenomics: Beyond the Sequence

Chapter 13

DNA Methylation Sequencing

Key Concept

DNA methylation is a fundamental epigenetic modification in which a methyl group ($-\text{CH}_3$) is covalently added to the 5-carbon position of cytosine, forming **5-methylcytosine (5mC)**. This modification occurs predominantly at CpG dinucleotides in mammals and plays critical roles in gene silencing, genomic imprinting, X-chromosome inactivation, and cellular differentiation. Aberrant methylation patterns are hallmarks of cancer and other diseases. Multiple sequencing-based approaches—from bisulfite sequencing to enzymatic methods to direct nanopore detection—allow researchers to map methylation at single-base resolution across entire genomes.

13.1 What Is DNA Methylation?

13.1.1 5-Methylcytosine and CpG Dinucleotides

In mammalian genomes, DNA methylation occurs almost exclusively at **CpG dinucleotides**—sites where a cytosine is immediately followed by a guanine on the same strand (written as 5'-CG-3'; the “p” denotes the phosphodiester bond). The methyl group is added to the 5-carbon of the cytosine ring by enzymes called **DNA methyltransferases (DNMTs)**:

- **DNMT1** (maintenance methyltransferase): copies the methylation pattern from the parental strand to the newly synthesized daughter strand during DNA replication, ensuring epigenetic inheritance.
- **DNMT3A** and **DNMT3B** (de novo methyltransferases): establish new methylation patterns during embryonic development and cell differentiation.
- **DNMT3L**: a catalytically inactive family member that stimulates DNMT3A/3B activity.

Approximately 70–80% of all CpG dinucleotides in the human genome are methylated. The remaining unmethylated CpGs are concentrated in specific genomic regions called **CpG islands (CGIs)**.

13.1.2 CpG Islands

CpG islands are defined as regions of at least 200 bp with a GC content $\geq 50\%$ and an observed-to-expected CpG ratio ≥ 0.6 . The human genome contains approximately 28,000 CpG islands, of which roughly 60–70% overlap with gene promoters. In normal somatic cells,

most CpG islands at gene promoters are unmethylated, allowing gene expression. Methylation of a promoter CpG island typically leads to transcriptional silencing.

CpG Depletion in the Genome

The CpG dinucleotide is under-represented in mammalian genomes (occurring at roughly 20% of the expected frequency), because methylated cytosines spontaneously deaminate to thymine. Over evolutionary time, this mutation pressure has depleted CpGs from most of the genome. CpG islands have been maintained by purifying selection because they serve essential regulatory functions.

In addition to CpG islands, important regulatory methylation occurs at:

- **CpG island shores:** regions 0–2 kb flanking CpG islands, where tissue-specific differential methylation is particularly common.
- **CpG island shelves:** regions 2–4 kb from CpG islands.
- **Enhancers and gene bodies:** intragenic methylation correlates with active transcription, and enhancer methylation is often inversely correlated with enhancer activity.

13.1.3 Biological Roles of DNA Methylation

DNA methylation participates in numerous biological processes:

1. **Gene silencing:** Methylation at promoter CpG islands recruits methyl-binding domain (MBD) proteins, which in turn recruit histone deacetylases and chromatin remodelers to compact chromatin and silence transcription.
2. **Genomic imprinting:** Approximately 100–200 genes in mammals are expressed from only one parental allele (monoallelic expression). The silent allele is marked by DNA methylation at imprinting control regions (ICRs). Classic examples include *H19/IGF2* and *SNRPN* (Prader–Willi/Angelman region).
3. **X-chromosome inactivation:** In female mammals, one X chromosome is silenced to achieve dosage compensation. DNA methylation at CpG islands helps maintain this silencing after the initial inactivation event orchestrated by the *XIST* long non-coding RNA.
4. **Transposable element suppression:** Roughly half the human genome consists of repetitive elements derived from transposons. Heavy methylation of these elements prevents their mobilisation, protecting genomic integrity.
5. **Development and differentiation:** Dramatic methylation reprogramming occurs during two developmental windows—in the early embryo (post-fertilisation) and in primordial germ cells. These reprogramming events erase and re-establish cell-type-specific methylation patterns.
6. **Cancer:** Tumours typically exhibit global hypomethylation (leading to genomic instability and reactivation of transposable elements) combined with focal hypermethylation at tumour-suppressor gene promoters (leading to their silencing). Methylation-based cancer classifiers are now used clinically for tumour typing.

13.1.4 Beyond 5mC: Other Cytosine Modifications

The discovery of additional cytosine modifications has expanded our understanding of the epigenome:

- **5-hydroxymethylcytosine (5hmC):** Generated by TET (ten-eleven translocation) family enzymes that oxidise 5mC. Enriched in neurons and embryonic stem cells. May serve as an intermediate in active DNA demethylation or as a stable epigenetic mark with its own regulatory functions.
- **5-formylcytosine (5fC):** A further oxidation product of 5hmC by TET enzymes. Present at low levels; likely an intermediate in demethylation.
- **5-carboxylcytosine (5caC):** The final TET oxidation product. Recognised and excised by thymine DNA glycosylase (TDG), followed by base excision repair to regenerate unmodified cytosine.
- **6-methyladenine (6mA):** Common in prokaryotes (restriction-modification systems), but recently detected in some eukaryotic genomes. Its functional significance in mammals remains debated, but 6mA has been associated with gene regulation in *Drosophila*, *C. elegans*, and early mammalian development.

13.2 Bisulfite Sequencing: The Gold Standard

13.2.1 The Bisulfite Conversion Principle

Bisulfite sequencing, developed by Frommer et al. in 1992, exploits the differential reactivity of methylated and unmethylated cytosines to sodium bisulfite:

1. Sodium bisulfite **deaminates unmethylated cytosine** to uracil ($C \rightarrow U$).
2. **Methylated cytosine (5mC) is protected** from deamination and remains as C.
3. During **PCR** amplification, uracil is read as thymine (T), while 5mC is read as cytosine (C).
4. By comparing the bisulfite-treated sequence to the reference genome, every C-to-T conversion indicates an unmethylated cytosine, and every retained C indicates a methylated cytosine.

Important Caveat

Standard bisulfite sequencing **cannot distinguish 5mC from 5hmC**. Both modifications are resistant to bisulfite deamination and appear as “C” in the sequencing output. To differentiate these marks, specialised methods such as oxidative bisulfite sequencing (oxBS-seq) or TET-assisted bisulfite sequencing (TAB-seq) are required (see [Section 13.4.1](#)).

13.2.2 Bisulfite Conversion Chemistry in Detail

The bisulfite conversion of cytosine proceeds through a well-characterised three-step chemical mechanism. Understanding this mechanism is essential for appreciating why the reaction discriminates between methylated and unmethylated cytosines, and for diagnosing conversion failures.

The Deamination Mechanism

The conversion of unmethylated cytosine to uracil by sodium bisulfite involves three sequential reactions:

Bisulfite Conversion Chemistry

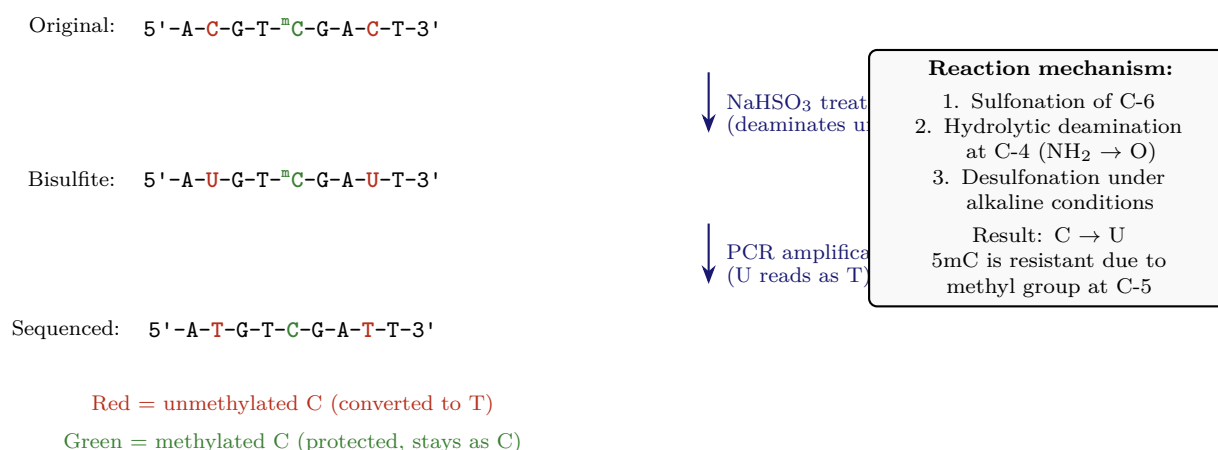
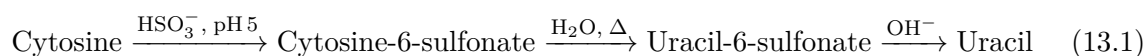


Figure 13.1: Bisulfite conversion chemistry. Sodium bisulfite deaminates unmethylated cytosines to uracil (read as T after PCR), while methylated cytosines (5mC, shown with superscript m) are protected and remain as C. By comparing the sequenced reads to the reference genome, each CpG site can be scored as methylated or unmethylated.

1. **Sulfonation (addition):** Bisulfite ion (HSO₃⁻) attacks the C-6 position of the cytosine ring in a reversible nucleophilic addition, forming **cytosine-6-sulfonate**. This reaction requires acidic conditions (pH 5.0–5.5) and proceeds efficiently only on single-stranded DNA, because bisulfite cannot access cytosines in double-stranded regions.
2. **Hydrolytic deamination:** The amino group (–NH₂) at the C-4 position of cytosine-6-sulfonate undergoes hydrolytic deamination, replacing –NH₂ with a carbonyl group (= O), yielding **uracil-6-sulfonate**. This is the rate-limiting step and is accelerated by elevated temperature (typically 50–70°C) and prolonged incubation (2–16 hours).
3. **Alkali desulfonation:** Under alkaline conditions (pH > 12, achieved during the final desulfonation/clean-up step), the sulfonate group is eliminated from C-6, yielding **uracil**.

The overall reaction can be summarised as:



Why 5-Methylcytosine Is Resistant

The methyl group at the C-5 position of 5mC sterically and electronically hinders the initial sulfonation reaction at C-6. Specifically:

- The electron-donating methyl group increases the electron density at C-5 and C-6, making the ring less electrophilic and thus less susceptible to nucleophilic attack by bisulfite.
- The steric bulk of the methyl group physically impedes access of HSO₃⁻ to C-6.
- As a result, the sulfonation rate for 5mC is approximately **100-fold slower** than for unmodified cytosine under standard bisulfite conditions.

5-Hydroxymethylcytosine (5hmC) is similarly resistant to bisulfite-mediated deamination. The hydroxymethyl group at C-5 provides comparable steric and electronic protection, which is why standard bisulfite sequencing cannot distinguish 5mC from 5hmC.

Conversion Rate and Completeness

The efficiency of bisulfite conversion is characterised by two complementary metrics:

$$\text{Conversion rate} = \frac{\text{Number of unmethylated C read as T}}{\text{Total unmethylated C positions}} \geq 99.0\% \quad (13.2)$$

$$\text{Protection rate} = \frac{\text{Number of methylated C read as C}}{\text{Total methylated C positions}} \geq 99.0\% \quad (13.3)$$

A high-quality bisulfite experiment achieves:

- **Conversion rate** > 99% for unmethylated cytosines (i.e., < 1% false methylation).
- **Protection rate** > 99% for methylated cytosines (i.e., < 1% false non-methylation).
- The non-conversion rate (complement of conversion rate) directly translates to **false-positive methylation calls**, which can be devastating for analyses of lowly methylated regions.

Conversion Rate Quality Control

Conversion rate is routinely assessed using two approaches:

Spike-in controls:

- **Lambda phage DNA** (~0.5–1% of library): the lambda phage genome (*Enterobacteria phage λ*) is unmethylated. After bisulfite treatment, virtually all cytosines in the lambda genome should read as T. The conversion rate is calculated as:

$$\text{Conversion rate}_\lambda = \frac{T_\lambda}{T_\lambda + C_\lambda} \quad (13.4)$$

where T_λ and C_λ are the total thymine and cytosine counts at all cytosine positions in reads mapping to the lambda genome.

- **pUC19 plasmid** (CpG-methylated): used to verify that methylated cytosines are protected. The protection rate should exceed 99%.

Non-CpG context methylation:

- In most somatic mammalian cells, cytosine methylation occurs almost exclusively in the CpG context. Methylation measured in the **CHH and CHG contexts** (where H = A, T, or C) therefore predominantly reflects **incomplete bisulfite conversion** rather than true non-CpG methylation.
- The non-CpG apparent methylation rate serves as an internal conversion control:

$$\text{Estimated non-conversion} \approx \frac{C_{\text{CHH}}}{C_{\text{CHH}} + T_{\text{CHH}}} \quad (13.5)$$

Important Caveat

Incomplete bisulfite conversion inflates methylation estimates across the entire genome, but its impact is most severe at lowly methylated loci. For example, if the non-conversion rate is 2%, a truly unmethylated CpG will appear 2% methylated, while a 50% methylated CpG will be measured as 51%. This makes regions with low methylation levels (enhancers, partially methylated domains in cancer) particularly susceptible to artefacts. Samples with conversion rates below 98% should generally be discarded and re-processed.

Practical Tip

When using cell types known to have genuine non-CpG methylation—such as embryonic stem cells, neurons, or oocytes—the CHH/CHG context cannot be used as an internal conversion control. In these cases, the lambda phage spike-in becomes essential. Always include spike-in controls in experimental designs involving these cell types.

13.2.3 Whole-Genome Bisulfite Sequencing (WGBS)

whole-genome bisulfite sequencing (WGBS) is the gold standard for genome-wide methylation profiling at single-base resolution. It interrogates the methylation status of every cytosine in the genome (approximately 28 million CpG sites in the human genome, plus non-CpG sites).

WGBS Library Preparation**Pre-bisulfite (standard) method:**

1. Fragment genomic DNA to 200–400 bp (sonication or enzymatic).
2. End repair, A-tailing, adapter ligation (methylated adapters to survive bisulfite treatment).
3. Size selection (gel or bead-based).
4. Bisulfite conversion (sodium bisulfite, 2–16 hours at elevated temperature).
5. PCR amplification with adapter primers.
6. Library QC and sequencing.

Post-Bisulfite Adapter Tagging (PBAT):

1. Bisulfite-convert genomic DNA *first*.
2. Synthesise first strand using random primers with adapters.
3. Synthesise second strand with another random-adapter primer.
4. Amplify and sequence.

PBAT avoids DNA loss from bisulfite degradation of adapter-ligated molecules and works with lower input (< 100 ng). It is the basis for low-input and single-cell bisulfite methods.

Coverage Requirements

WGBS requires substantial sequencing depth because bisulfite conversion reduces sequence complexity (all unmethylated Cs become Ts), making alignment more challenging and reducing effective coverage:

- **Minimum useful depth:** 5–10× per strand (10–20× total) for global methylation patterns.
- **Standard depth:** 30× for reliable single-CpG-resolution analysis.
- **DMR detection:** Requires ≥ 10 reads per CpG for statistical testing of differentially methylated regions.
- For a human genome at 30×: approximately 900 million 150 bp PE reads (270 Gb of data).

Practical Tip

Spike-in unmethylated lambda phage DNA (~0.5–1% of the library) or methylated pUC19 plasmid as conversion controls. Lambda DNA should show >99% conversion (all Cs read as T), while pUC19 should remain >99% methylated. Conversion rates < 98% indicate incomplete bisulfite treatment and the data should be discarded.

Alignment Challenges

Bisulfite-converted reads have reduced sequence complexity because all Cs on the original strand that were unmethylated are now Ts. This creates alignment difficulties:

- The effective four-letter alphabet is reduced to a three-letter alphabet on the converted strand.
- Reads can originate from either the Watson (original top) or Crick (original bottom) strand, and bisulfite conversion is strand-specific.
- Specialised aligners handle this by performing *in silico* C-to-T conversion of both reads and reference, aligning in the converted space, and then mapping back to the original reference.

Dedicated Bisulfite Aligners**13.2.4 Reduced Representation Bisulfite Sequencing (RRBS)**

RRBS is a cost-effective alternative to WGBS that enriches for CpG-rich regions of the genome:

RRBS Protocol

1. Digest genomic DNA with **MspI** (recognition site: C⁺ACGG), which cuts at CpG-containing sequences regardless of methylation status.
2. End repair and A-tailing (fragments have predictable CG-rich ends).
3. Adapter ligation.
4. Size selection: typically 150–400 bp fragments, which enriches for CpG islands and CpG-dense regions.
5. Bisulfite conversion.
6. PCR amplification and sequencing.

RRBS covers approximately 10–12% of all CpGs in the human genome (about 3 million CpGs), but captures the majority of CpG islands and promoter-associated CpGs. It requires only 10–30 million reads per sample, making it roughly 10–30× cheaper than WGBS.

Important Caveat

RRBS has important limitations: (1) it is restricted to regions flanking MspI cut sites and therefore misses many enhancers, intergenic regions, and CpG-poor regulatory elements; (2) the fragment ends contain CpG sites that were part of the MspI recognition sequence—these should be excluded from methylation calling due to end-repair artefacts (tools like Trim Galore’s `-rrbs` flag handle this); (3) coverage is highly non-uniform, with some CpGs having thousands of reads and others having none.

Table 13.1: Comparison of bisulfite-seq alignment tools.

Aligner	Algorithm	Speed	Memory	Notes
Bismark	Bowtie2/HISAT2 back- end; three- letter aligner	Moderate	Moderate	Most widely used; excellent documentation
bwa-meth	BWA- MEM back- end; in silico conver- sion	Fast	Low-moderate	Faster than Bismark; good for large datasets
BSBolt	Combined aligner and methy- lation caller	Fast	Moderate	Integrated pipeline; PBAT support
BSMAP	Seed- based; wild- card ap- proach	Fast	High	Legacy tool; still used
gemBS	Full pipeline (align- ment + calling)	Fast	High	Designed for large cohorts

13.3 Enzymatic Methyl-seq (EM-seq)

Enzymatic methyl-seq (EM-seq), commercialised as the NEBNext Enzymatic Methyl-seq (EM-seq) kit by New England Biolabs, is a gentler alternative to bisulfite conversion that avoids the harsh chemical treatment responsible for DNA degradation.

13.3.1 Principle

EM-seq uses a two-step enzymatic process:

1. **TET2 oxidation:** The TET2 enzyme (plus an oxidation supplement) converts 5mC to 5-carboxylcytosine (5caC) and 5hmC to 5caC. These oxidised forms are resistant to the subsequent deamination step.
2. **APOBEC3A deamination:** The APOBEC3A enzyme deaminates *unmodified* cytosines to uracils, mimicking the bisulfite conversion outcome. Importantly, APOBEC3A does not deaminate the oxidised forms (5caC), so originally methylated and hydroxymethylated cytosines are protected.

The net result is the same readout as bisulfite sequencing (unmethylated C $\rightarrow$ T, methylated C $\rightarrow$ C), but with critical advantages:

- **Less DNA damage:** bisulfite treatment degrades 90–99% of input DNA; EM-seq preserves DNA integrity.
- **Lower input requirements:** works reliably with 10–100 ng of DNA (vs. 200 ng–1 μ g for standard WGBS).
- **Better coverage uniformity:** reduced GC bias and more even coverage across the genome.
- **More accurate at low-methylated regions:** less false methylation signal from incomplete conversion.

Practical Tip

EM-seq is rapidly becoming the preferred method for genome-wide methylation profiling, especially when input DNA is limited (FFPE samples, liquid biopsies, sorted cell populations). If you are starting a new methylation project and do not have a specific reason to use bisulfite, consider EM-seq as the default choice.

13.4 Distinguishing 5mC from 5hmC

13.4.1 Oxidative Bisulfite Sequencing (oxBS-seq)

Standard bisulfite sequencing reads both 5mC and 5hmC as “C.” To distinguish them:

oxBS-seq adds a chemical oxidation step (potassium perruthenate, KRuO_4) *before* bisulfite treatment. This oxidises 5hmC to 5-formylcytosine (5fC), which is then deaminated by bisulfite (read as T). Thus:

- In standard BS-seq: 5mC $\rightarrow$ C, 5hmC $\rightarrow$ C, C $\rightarrow$ T.
- In oxBS-seq: 5mC $\rightarrow$ C, 5hmC $\rightarrow$ T, C $\rightarrow$ T.
- 5hmC levels at each site = BS-seq methylation – oxBS-seq methylation.

13.4.2 TET-Assisted Bisulfite Sequencing (TAB-seq)

TAB-seq specifically maps 5hmC:

1. Protect 5hmC by glucosylation using β -glucosyltransferase (β -GT) to form 5-glucosylhydroxymethylcytosine (5ghmC).
2. Oxidise 5mC to 5caC using recombinant TET1 enzyme.
3. Bisulfite conversion: unmodified C and 5caC are deaminated (read as T), while 5ghmC is protected (read as C).
4. Result: only 5hmC positions read as C; everything else reads as T.

13.4.3 ACE-seq

APOBEC-Coupled Epigenetic Sequencing (ACE-seq) uses APOBEC3A enzyme instead of bisulfite for 5hmC detection. APOBEC3A deaminates C, 5mC, and 5fC but *not* 5hmC (or its glucosylated form). Combined with β -GT protection, this provides a bisulfite-free 5hmC mapping method with less DNA damage.

13.5 Targeted Methylation Approaches

13.5.1 Methyl-Capture Sequencing

Target-enrichment (hybridisation capture) can be combined with bisulfite sequencing to interrogate a defined subset of the genome:

- **Agilent SureSelect Methyl-Seq:** captures 3.7 million CpGs (84 Mb target region) covering CpG islands, shores, shelves, and ENCODE regulatory elements.
- **Roche SeqCap Epi CpGiant:** captures 5.5 million CpGs.
- These methods provide deep coverage of biologically relevant CpGs at a fraction of WGBS cost.

13.5.2 Methylation Arrays

Illumina methylation arrays use bisulfite-converted DNA hybridised to bead arrays with methylation-specific probes:

- **Infinium MethylationEPIC v2.0:** interrogates ~935,000 CpG sites. The most widely used array platform, especially for large epidemiological cohorts.
- Provides a beta value (0–1) for each CpG representing the fraction of methylated alleles.
- Advantages: low cost per sample, high throughput, extensive normalization pipelines.
- Limitations: fixed probe set (biased towards CpG islands), limited to pre-selected sites.

13.6 Long-Read Methylation Detection

13.6.1 Nanopore Direct Methylation Detection

Oxford Nanopore sequencing can detect DNA modifications **directly from the raw ionic current signal** without any chemical conversion. As a DNA strand passes through the nanopore, modified bases produce subtly different current signatures compared to unmodified bases:

- **Simultaneous detection** of 5mC, 5hmC, and 6mA from a single sequencing run.
- **No chemical conversion** means no DNA damage, no conversion bias, and no loss of long-range information.
- **Single-molecule resolution:** methylation is called on individual reads, enabling analysis of allele-specific methylation and methylation heterogeneity.
- **Long reads** provide methylation information in the context of phased haplotypes.

Key tools for nanopore methylation calling:

- **Dorado:** ONT's current basecaller with integrated modification calling (replaces Guppy). Outputs modified base probabilities in BAM tags (MM/ML).
- **modkit:** ONT's tool for summarising, filtering, and extracting modification calls from BAM files.
- **Remora:** Deep learning framework for training custom modification-detection models.
- **Megalodon (legacy):** Earlier ONT tool for per-read modification calling; superseded by Dorado.
- **DeepMod / DeepSignal:** Third-party deep-learning tools for nanopore modification detection.

Practical Tip

Nanopore methylation detection accuracy has improved dramatically. With R10.4.1 chemistry and Dorado's super-accuracy models, per-CpG-site accuracy exceeds 95% at 10× coverage, approaching bisulfite sequencing accuracy. The key advantage is the ability to phase methylation across long reads without any chemical treatment.

13.6.2 PacBio Methylation Detection

PacBio SMRT sequencing detects DNA modifications through **kinetic signatures**—changes in the inter-pulse duration (IPD) when the polymerase encounters a modified base:

- Most sensitive for **6mA** detection (strong kinetic signature).
- **5mC detection** from native DNA is possible but requires very high coverage (> 50×) due to the weaker kinetic signature.
- **Improved with HiFi + 5mC model:** PacBio's *jasmine* tool and newer *primrose* model enable CpG methylation calling from HiFi reads, though sensitivity is lower than nanopore or bisulfite methods.
- Best used as an additional data layer when HiFi sequencing is already being performed for other purposes (e.g., genome assembly, variant calling).

13.7 Comprehensive Comparison of Methylation Methods

13.8 Bioinformatics Workflow for Bisulfite Sequencing

The standard WGBS/EM-seq analysis pipeline follows these steps:

1. **Quality control:** FastQC, MultiQC to assess raw read quality.

Table 13.2: Comparison of major DNA methylation profiling approaches.

Method	Resolution	Coverage	Input	Cost	Key Features
WGBS	Single base	Genome-wide	200 ng–1 μ g	\$\$\$\$	Gold standard; complete CpG coverage
RRBS	Single base	CpG-rich (~10%)	10–100 ng	\$\$	Cost-effective; biased to CpG islands
EM-seq	Single base	Genome-wide	10–100 ng	\$\$\$\$	Less DNA damage; low input
oxBS-seq	Single base	Genome-wide	1–2 μ g	\$\$\$\$\$	Distinguishes 5mC from 5hmC
TAB-seq	Single base	Genome-wide	1 μ g	\$\$\$\$\$	Maps 5hmC specifically
Methyl-capture	Single base	Targeted (~3–5M CpGs)	250 ng	\$\$\$	Good compromise of cost and coverage
EPIC array	Single probe	~935K CpGs	250 ng	\$	Cheapest per sample; large cohorts
Nanopore	Single base	Genome-wide	1 μ g	\$\$\$	No conversion; 5mC+5hmC+6mA; long reads
PacBio HiFi	Single base	Genome-wide	5–10 μ g	\$\$\$\$	CpG methylation from kinetics; long reads

2. **Adapter trimming:** Trim Galore (wrapper around Cutadapt) with bisulfite-specific options.
3. **Alignment:** Bismark or bwa-meth against a bisulfite-converted reference genome.
4. **Deduplication:** Remove PCR duplicates (Bismark `deduplicate_bismark` or Picard).
5. **Methylation extraction:** Extract per-CpG methylation levels (Bismark `bismark_methylation_extractor` or MethylDackel).
6. **Quality assessment:** Check conversion rate (lambda control), M-bias plots, coverage distribution.
7. **Differential methylation analysis:** Identify DMRs using DSS, dmrseq, methylKit, or BSsmooth.
8. **Visualisation:** IGV tracks, heatmaps, genome browser.

13.8.1 Snakemake Workflow

Listing 13.1: Snakemake workflow for WGBS analysis with Bismark.

```

1 # Snakefile for WGBS analysis using Bismark
2 configfile: "config.yaml"
3
4 SAMPLES = config["samples"]
5 GENOME_DIR = config["genome_dir"]
6
7 rule all:
8     input:
9         "results/multiqc/multiqc_report.html",
10        expand("results/methylation/{sample}.bismark.cov.gz",
11              sample=SAMPLES),
12        expand("results/methylation/{sample}_CpG_report.txt",
13              sample=SAMPLES),
14        "results/dmr/dmrseq_results.tsv"
15
16 rule trim_galore:
17     input:
18         r1 = "raw_data/{sample}_R1.fastq.gz",
19         r2 = "raw_data/{sample}_R2.fastq.gz"
20     output:
21         r1 = "results/trimmed/{sample}_R1_val_1.fq.gz",
22         r2 = "results/trimmed/{sample}_R2_val_2.fq.gz",
23         report1 = "results/trimmed/{sample}_R1.fastq.gz_trimming_report.txt",
24         report2 = "results/trimmed/{sample}_R2.fastq.gz_trimming_report.txt"
25     threads: 4
26     conda: "envs/trim_galore.yaml"
27     shell:
28         """
29         trim_galore --paired --cores {threads} \
30             --output_dir results/trimmed \
31             {input.r1} {input.r2}
32         """
33
34 rule bismark_genome_preparation:
35     input:
36         genome = GENOME_DIR
37     output:
38         directory(GENOME_DIR + "/Bisulfite_Genome")

```

```

39     conda: "envs/bismark.yaml"
40     shell:
41         """
42         bismark_genome_preparation {input.genome}
43         """
44
45 rule bismark_align:
46     input:
47         r1 = "results/trimmed/{sample}_R1_val_1.fq.gz",
48         r2 = "results/trimmed/{sample}_R2_val_2.fq.gz",
49         genome = GENOME_DIR,
50         idx = GENOME_DIR + "/Bisulfite_Genome"
51     output:
52         bam = "results/aligned/{sample}_pe.bam",
53         report = "results/aligned/{sample}_PE_report.txt"
54     threads: 8
55     resources:
56         mem_mb = 32000
57     conda: "envs/bismark.yaml"
58     shell:
59         """
60         bismark --genome {input.genome} \
61             -1 {input.r1} -2 {input.r2} \
62             --parallel {threads} \
63             --output_dir results/aligned \
64             --temp_dir results/tmp \
65             --non_directional # Add if using PBAT library
66         """
67
68 rule bismark_dedup:
69     input:
70         bam = "results/aligned/{sample}_pe.bam"
71     output:
72         bam = "results/dedup/{sample}_pe.deduplicated.bam",
73         report = "results/dedup/{sample}_pe.deduplication_report.txt"
74     conda: "envs/bismark.yaml"
75     shell:
76         """
77         deduplicate_bismark --paired --bam \
78             --output_dir results/dedup \
79             {input.bam}
80         """
81
82 rule methylation_extraction:
83     input:
84         bam = "results/dedup/{sample}_pe.deduplicated.bam",
85         genome = GENOME_DIR
86     output:
87         cov = "results/methylation/{sample}.bismark.cov.gz",
88         cpg = "results/methylation/{sample}_CpG_report.txt",
89         mbias = "results/methylation/{sample}_pe.deduplicated.M-bias.txt"
90     threads: 4
91     conda: "envs/bismark.yaml"
92     shell:
93         """
94         bismark_methylation_extractor --paired-end \
95             --comprehensive --merge_non_CpG \
96             --multicore {threads} \

```



```

97         --cytosine_report --genome_folder {input.genome} \
98         --bedGraph --gzip \
99         --output results/methylation \
100         {input.bam}
101     """
102
103 rule dmr_analysis:
104     input:
105         cpg_reports = expand(
106             "results/methylation/{sample}_CpG_report.txt",
107             sample=SAMPLES
108         )
109     output:
110         dmrs = "results/dmr/dmrseq_results.tsv",
111         plots = "results/dmr/dmr_plots.pdf"
112     conda: "envs/dmrseq.yaml"
113     script:
114         "scripts/run_dmrseq.R"
115
116 rule multiqc:
117     input:
118         expand("results/trimmed/{sample}_R1.fastq.gz_trimming_report.txt",
119             sample=SAMPLES),
120         expand("results/aligned/{sample}_PE_report.txt",
121             sample=SAMPLES),
122         expand("results/dedup/{sample}_pe.deduplication_report.txt",
123             sample=SAMPLES),
124         expand("results/methylation/{sample}_pe.deduplicated.M-bias.txt",
125             sample=SAMPLES)
126     output:
127         "results/multiqc/multiqc_report.html"
128     conda: "envs/multiqc.yaml"
129     shell:
130         """
131         multiqc results/ -o results/multiqc --force
132         """

```

13.8.2 Nextflow Workflow

Listing 13.2: Nextflow DSL2 workflow for WGBS analysis.

```

1  #!/usr/bin/env nextflow
2  nextflow.enable.dsl=2
3
4  // Parameters
5  params.reads      = "raw_data/*_R{1,2}.fastq.gz"
6  params.genome     = "reference/genome.fa"
7  params.outdir     = "results"
8
9  // Processes
10 process TRIM_GALORE {
11     tag "$sample_id"
12     cpus 4
13     publishDir "${params.outdir}/trimmed", mode: 'copy'
14     conda 'bioconda::trim-galore=0.6.10'
15
16     input:

```

```

17     tuple val(sample_id), path(reads)
18
19     output:
20     tuple val(sample_id), path("*_val_{1,2}.fq.gz"), emit: trimmed
21     path "_trimming_report.txt", emit: reports
22
23     script:
24     """
25     trim_galore --paired --cores ${task.cpus} \
26         ${reads[0]} ${reads[1]}
27     """
28 }
29
30 process BISMARK_INDEX {
31     cpus 8
32     memory '32 GB'
33     conda 'bioconda::bismark=0.24.2'
34
35     input:
36     path genome
37
38     output:
39     path "BisulfiteGenome", emit: index
40
41     script:
42     """
43     mkdir -p genome_dir
44     cp ${genome} genome_dir/
45     bismark_genome_preparation genome_dir/
46     mv genome_dir/BisulfiteGenome BisulfiteGenome
47     """
48 }
49
50 process BISMARK_ALIGN {
51     tag "$sample_id"
52     cpus 8
53     memory '32 GB'
54     publishDir "${params.outdir}/aligned", mode: 'copy'
55     conda 'bioconda::bismark=0.24.2'
56
57     input:
58     tuple val(sample_id), path(reads)
59     path index
60
61     output:
62     tuple val(sample_id), path("*.bam"), emit: bam
63     path "_report.txt", emit: report
64
65     script:
66     """
67     bismark --genome . \
68         -1 ${reads[0]} -2 ${reads[1]} \
69         --parallel ${task.cpus} \
70         --temp_dir tmp/
71     """
72 }
73
74 process DEDUPLICATE {

```

```

75     tag "$sample_id"
76     publishDir "${params.outdir}/dedup", mode: 'copy'
77     conda 'bioconda::bismark=0.24.2'
78
79     input:
80     tuple val(sample_id), path(bam)
81
82     output:
83     tuple val(sample_id), path("*.deduplicated.bam"), emit: bam
84     path "*.deduplication_report.txt",                emit: report
85
86     script:
87     """
88     deduplicate_bismark --paired --bam ${bam}
89     """
90 }
91
92 process METHYLATION_EXTRACTION {
93     tag "$sample_id"
94     cpus 4
95     publishDir "${params.outdir}/methylation", mode: 'copy'
96     conda 'bioconda::bismark=0.24.2'
97
98     input:
99     tuple val(sample_id), path(bam)
100    path genome
101
102    output:
103    tuple val(sample_id), path("*.cov.gz"),          emit: coverage
104    tuple val(sample_id), path("*CpG_report.txt"),emit: cpG_report
105    path "*.M-bias.txt",                            emit: mbias
106
107    script:
108    """
109    bismark_methylation_extractor --paired-end \
110        --comprehensive --merge_non_CpG \
111        --multicore ${task.cpus} \
112        --cytosine_report --genome_folder . \
113        --bedGraph --gzip \
114        ${bam}
115    """
116 }
117
118 process DMR_ANALYSIS {
119     publishDir "${params.outdir}/dmr", mode: 'copy'
120     conda 'bioconda::bioconductor-dmrseq=1.18.0'
121
122     input:
123     path cpG_reports
124
125     output:
126     path "dmrseq_results.tsv", emit: dmrs
127     path "dmr_plots.pdf",      emit: plots
128
129     script:
130     """
131     Rscript ${projectDir}/scripts/run_dmrseq.R \
132         --input_dir . \

```

```

133     --output_dir .
134     """
135 }
136
137 process MULTIQC {
138     publishDir "${params.outdir}/multiqc", mode: 'copy'
139     conda 'bioconda::multiqc=1.21'
140
141     input:
142     path '*'
143
144     output:
145     path "multiqc_report.html"
146
147     script:
148     """
149     multiqc . --force
150     """
151 }
152
153 // Workflow
154 workflow {
155     // Create channel from read pairs
156     reads_ch = Channel
157         .fromFilePairs(params.reads, checkIfExists: true)
158
159     // Run pipeline
160     TRIM_GALORE(reads_ch)
161     BISMARK_INDEX(file(params.genome))
162     BISMARK_ALIGN(TRIM_GALORE.out.trimmed, BISMARK_INDEX.out.index)
163     DEDUPLICATE(BISMARK_ALIGN.out.bam)
164     METHYLATION_EXTRACTION(DEDUPLICATE.out.bam, file(params.genome))
165     DMR_ANALYSIS(METHYLATION_EXTRACTION.out.cpg_report.map{ it[1] }.collect())
166     MULTIQC(
167         TRIM_GALORE.out.reports
168         .mix(BISMARK_ALIGN.out.report)
169         .mix(DEDUPLICATE.out.report)
170         .mix(METHYLATION_EXTRACTION.out.mbias)
171         .collect()
172     )
173 }

```

Practical Tip

The nf-core community maintains `nf-core/methylseq`, a production-ready Nextflow pipeline for bisulfite sequencing that supports both Bismark and bwa-meth backends. For most users, this is the recommended starting point rather than building a pipeline from scratch. It includes comprehensive QC, supports EM-seq libraries, and integrates with institutional compute clusters.

13.8.3 Differential Methylation Analysis

Identifying differentially methylated regions (DMRs) between conditions (e.g., tumour vs. normal, treated vs. control) is a central goal of methylation studies. Major tools include:

- **DSS (Dispersion Shrinkage for Sequencing):** Bayesian hierarchical model based on the beta-binomial distribution. Handles biological variability through dispersion

shrinkage. Works well with as few as two replicates per group. Provides both differentially methylated loci (DMLs) and DMRs.

- **dmrseq**: Specifically designed for detecting DMRs (not individual CpGs). Uses a two-stage approach: (1) identifies candidate DMR regions based on methylation differences, then (2) tests significance using a permutation-based statistic. Excellent for detecting regions with consistent, moderate methylation differences.
- **methyKit**: R/Bioconductor package with a complete workflow: reads coverage files, performs tiling-window or per-CpG analysis, and calls DMRs. Uses logistic regression or Fisher's exact test. Good for exploratory analysis.
- **BSmooth**: Smoothing-based approach that borrows information across nearby CpGs to improve estimation of local methylation levels. Particularly useful when coverage is moderate (5–15×).

Beta-Binomial DMR Model (DSS)

The DSS (Dispersion Shrinkage for Sequencing) package implements a rigorous statistical framework for identifying differentially methylated loci (DMLs) and regions (DMRs) from bisulfite sequencing data. The model accounts for both sampling variation (from finite read coverage) and biological variation (from inter-replicate differences) through a hierarchical Beta-Binomial framework.

Per-CpG read-level model. At a given CpG site j in sample i , let X_{ij} be the number of reads showing methylation (C retained) out of n_{ij} total reads covering that site. Given the true methylation proportion θ_{ij} , the read counts follow a Binomial distribution:

$$X_{ij} \mid \theta_{ij} \sim \text{Binomial}(n_{ij}, \theta_{ij}) \quad (13.6)$$

Biological variability model. Across biological replicates within the same condition, the true methylation proportion θ_{ij} is not fixed but varies. DSS models this variability with a Beta prior:

$$\theta_{ij} \sim \text{Beta}(\alpha_j, \beta_j) \quad (13.7)$$

where the Beta distribution is parameterised by the mean methylation level $\mu_j = \alpha_j / (\alpha_j + \beta_j)$ and the dispersion $\phi_j = 1 / (\alpha_j + \beta_j + 1)$. The dispersion parameter ϕ_j captures biological variability: larger ϕ_j implies greater heterogeneity across replicates.

Marginal distribution. Integrating over θ_{ij} , the marginal distribution of X_{ij} is Beta-Binomial:

$$X_{ij} \sim \text{Beta-Binomial}(n_{ij}, \alpha_j, \beta_j) \quad (13.8)$$

This has variance $\text{Var}(X_{ij}) = n_{ij} \mu_j (1 - \mu_j) [1 + (n_{ij} - 1) \phi_j]$, which is larger than the Binomial variance by the factor $[1 + (n_{ij} - 1) \phi_j]$, properly accounting for biological over-dispersion.

Differential methylation testing. To test whether methylation differs between two groups (e.g., treatment vs. control) at each CpG site, DSS estimates μ_j and ϕ_j for each group using moment estimators with **dispersion shrinkage**—borrowing information across all CpG sites to stabilise the dispersion estimates, especially when the number of replicates is small. It then applies a **Wald test** for the difference in mean methylation:

$$W_j = \frac{\hat{\mu}_{j,1} - \hat{\mu}_{j,2}}{\sqrt{\widehat{\text{Var}}(\hat{\mu}_{j,1}) + \widehat{\text{Var}}(\hat{\mu}_{j,2})}} \quad (13.9)$$

Under the null hypothesis of no differential methylation, W_j approximately follows a standard normal distribution, yielding p-values that are corrected for multiple testing.

DMR construction. DMLs are individual CpG sites with significant p-values. DSS then merges nearby DMLs into contiguous DMRs by requiring that significant CpGs within a DMR are separated by no more than a specified distance (default: 50 bp) and that the region contains a minimum number of CpGs.

BSmooth Local-Likelihood Smoothing

The BSmooth algorithm addresses a fundamental challenge in WGBS analysis: at typical sequencing depths (10–30×), individual CpG sites are covered by relatively few reads, leading to noisy per-site methylation estimates. BSmooth improves these estimates by **borrowing information from neighbouring CpGs** through local-likelihood smoothing.

Motivation. Consider a CpG site covered by only 5 reads, of which 3 show methylation. The raw estimate of $\hat{\theta} = 0.60$ has a wide confidence interval and is unreliable. However, if the 10 neighbouring CpGs within a 1 kb window all show similar methylation levels (0.55–0.70), we can borrow strength from these neighbours to obtain a more precise estimate.

Smoothing procedure. BSmooth fits a **local-likelihood weighted regression** across CpG positions within a sliding window:

1. For each CpG site, define a smoothing window containing a minimum number of CpGs (default: 70 CpGs or 1 kb, whichever spans more sites).
2. Within this window, fit a weighted logistic regression where CpGs closer to the target site receive higher weights (using a tricube kernel function).
3. The fitted value at the target CpG provides the smoothed methylation estimate $\tilde{\theta}_j$.
4. Weights also incorporate coverage: CpGs with higher read depth contribute more to the fit.

Why smoothing works. Methylation levels at adjacent CpGs are highly correlated—nearby CpGs tend to share similar methylation states, especially within CpG islands and shores. Smoothing exploits this spatial autocorrelation to reduce noise without sacrificing genuine biological variation that occurs over larger genomic distances.

DMR detection with BSmooth. After smoothing, BSmooth computes a t-statistic at each CpG comparing the smoothed methylation profiles between two groups. Contiguous regions where the t-statistic exceeds a threshold are identified as candidate DMRs, and their significance is assessed using area-under-the-curve statistics.

Practical Tip

BSmooth is especially valuable for moderate-coverage WGBS experiments (5–15×), where per-CpG estimates are noisy but there are sufficient neighbouring CpGs to support smoothing. At very high coverage (>30×), the benefit of smoothing diminishes because individual CpG estimates are already precise. Conversely, in targeted sequencing (RRBS), smoothing may be unreliable because CpG coverage is fragmented and non-contiguous.

Important Caveat

A common pitfall in DMR analysis is confusing **statistical significance** with **biological significance**. With sufficient sequencing depth, even very small methylation differences (e.g., 5%) can be statistically significant. Most biologically meaningful DMRs show methylation differences ≥ 20 –25%. Always apply both a statistical threshold (e.g., FDR < 0.05) and a minimum methylation difference threshold (e.g., $|\Delta\beta| \geq 0.2$).

13.8.4 Visualisation

Methylation data can be visualised in multiple ways:

- **Genome browser tracks:** Load Bismark coverage files (.bedGraph or .cov.gz) into IGV or the UCSC Genome Browser. Each CpG is displayed as a bar indicating methylation level (0–100%).
- **Methylation heatmaps:** Cluster samples and CpG regions using tools like `methyKit::clusterSamples` or `deepTools computeMatrix + plotHeatmap` for methylation around specific genomic features (e.g., TSSs, enhancers).
- **M-bias plots:** Generated by Bismark; show methylation level as a function of read position. Systematic biases at read ends (common in RRBS) indicate positions that should be excluded from methylation calling.
- **PCA/MDS plots:** Principal component analysis or multi-dimensional scaling of methylation data to assess sample relationships and batch effects.
- **Violin/bean plots:** Distribution of methylation levels across genomic features (promoters, gene bodies, repeats) by sample or condition.

13.9 Experimental Design Considerations

Choosing a Methylation Method

The choice of method depends on the experimental question:

- **Comprehensive, unbiased survey:** WGBS or EM-seq.
- **CpG island-focused, budget-limited:** RRBS.
- **Large clinical cohorts:** EPIC array.
- **Distinguishing 5mC from 5hmC:** oxBS-seq, TAB-seq, or nanopore.
- **Low-input or degraded DNA:** EM-seq or PBAT-WGBS.
- **Haplotype-resolved methylation:** Nanopore or PacBio long-read sequencing.
- **Targeted regions of interest:** Methyl-capture or amplicon bisulfite sequencing.

Key design parameters to consider:

1. **Biological replicates:** At least 3 per condition for DMR analysis; more improve statistical power.
2. **Sequencing depth:** See coverage recommendations above; under-sequenced WGBS wastes money without providing reliable single-CpG resolution.
3. **Batch effects:** Bisulfite conversion efficiency can vary between batches. Process all samples in a single batch or randomise samples across batches. Include conversion controls.
4. **Cell-type composition:** Bulk tissue methylation is a weighted average of constituent cell types. Cell-type deconvolution (e.g., MethylCIBERSORT, EpiDISH) or cell sorting prior to sequencing may be necessary for accurate interpretation.
5. **Paired samples:** When possible, pair tumour and normal samples from the same individual to reduce inter-individual variability.

13.10 Chapter Summary

DNA methylation is the most extensively studied epigenetic modification, with a rich toolkit of sequencing-based methods for its detection. Bisulfite sequencing (WGBS and RRBS) remains the historical gold standard, but enzymatic methods (EM-seq) offer a gentler, more accurate alternative for new projects. Long-read technologies from Oxford Nanopore and PacBio enable direct detection of modifications without chemical conversion, providing haplotype-resolved methylation information. The choice among these methods depends on the biological question, available input material, budget, and the need to distinguish between different cytosine modifications. Rigorous bioinformatics—from specialised alignment to appropriate statistical testing of DMRs—is essential for extracting reliable biological insights from methylation data.

Chapter 14

Chromatin Accessibility Profiling

Key Concept

Chromatin accessibility refers to the degree to which nuclear DNA is physically accessible to regulatory proteins such as transcription factors, RNA polymerases, and chromatin remodelers. In eukaryotic cells, DNA is packaged around histone octamers into nucleosomes, and the local density and positioning of these nucleosomes determines whether a genomic region is “open” (accessible) or “closed” (compacted). Open chromatin regions typically correspond to active regulatory elements—promoters, enhancers, silencers, and insulators. Sequencing-based methods such as ATAC-seq and DNase-seq allow researchers to map these accessible regions genome-wide, providing a snapshot of the regulatory landscape of any cell type.

14.1 Why Chromatin Accessibility Matters

The human genome contains approximately 3.2 billion base pairs, yet only a small fraction of this sequence is accessible in any given cell type. The accessible fraction—typically 2–4% of the genome—encompasses the active **cis-regulatory elements** that control gene expression:

- **Active promoters:** Regions immediately upstream of transcription start sites (TSSs) where the transcription machinery assembles.
- **Enhancers:** Distal regulatory elements that boost transcription of target genes, often located tens to hundreds of kilobases away.
- **Insulators/boundary elements:** Regions that prevent inappropriate enhancer–promoter interactions, often bound by CTCF.
- **Silencers:** Elements that repress transcription of target genes.
- **Transcription factor (TF) binding sites:** Accessible DNA motifs where sequence-specific TFs bind to regulate gene expression.

Chromatin accessibility profiling provides a direct readout of regulatory element activity. Unlike histone modification mapping, which requires specific antibodies for each mark, accessibility assays capture *all* open regulatory elements in a single experiment. Changes in accessibility between conditions (e.g., disease vs. healthy, differentiated vs. stem cells) reveal the regulatory programs that drive cell-state transitions.

14.2 DNase-seq

14.2.1 Principle

DNase I is an endonuclease that preferentially cleaves DNA that is not protected by nucleosomes or tightly bound proteins. Regions of open chromatin are hypersensitive to DNase I digestion, and these **DNase I hypersensitive sites (DHSs)** have been used since the 1980s to identify regulatory elements.

In **DNase-seq**:

1. Intact nuclei are isolated from cells.
2. Nuclei are treated with a controlled amount of DNase I enzyme.
3. The small DNA fragments released from hypersensitive sites are size-selected (typically < 300 bp).
4. Fragments are adapter-ligated and sequenced.
5. Reads pile up at DHSs, producing peaks in the coverage signal.

14.2.2 The ENCODE Project and DHS Mapping

DNase-seq was the method of choice for the **Encyclopedia of DNA Elements (ENCODE)** project, which mapped DHSs in hundreds of human cell types and tissues. The ENCODE DHS maps identified approximately 2.9 million DHSs across the human genome, revealing that ~40% of the genome is accessible in at least one cell type, even though only 2–3% is accessible in any individual cell type. This work demonstrated the vast regulatory diversity of human cells and provided a foundational resource for interpreting non-coding genetic variants.

Practical Tip

While DNase-seq has been largely superseded by ATAC-seq for new experiments (due to ATAC-seq's simpler protocol and lower cell requirements), the massive DNase-seq datasets generated by ENCODE remain invaluable reference resources. The ENCODE portal (<https://www.encodeproject.org>) provides freely accessible DHS maps for hundreds of cell types.

14.3 ATAC-seq

14.3.1 Principle: Tn5 Transposase-Mediated Tagmentation

Assay for Transposase-Accessible Chromatin using sequencing (ATAC-seq), introduced by Buenrostro et al. in 2013, has become the dominant method for chromatin accessibility profiling. The key innovation is the use of a hyperactive **Tn5 transposase** loaded with sequencing adapters. The Tn5 transposase simultaneously fragments DNA and inserts sequencing adapters into accessible chromatin regions—a process called **tagmentation**:

1. The Tn5 transposase can only access DNA that is not wrapped around nucleosomes or occluded by other proteins.
2. When it inserts into accessible DNA, it creates a 9 bp staggered cut and ligates sequencing adapters to both fragment ends.
3. Nucleosome-protected regions are refractory to Tn5 insertion.

4. After tagmentation, DNA is purified and amplified by PCR with primers complementary to the inserted adapters.
5. Sequencing reveals the locations where Tn5 inserted, which correspond to accessible chromatin.

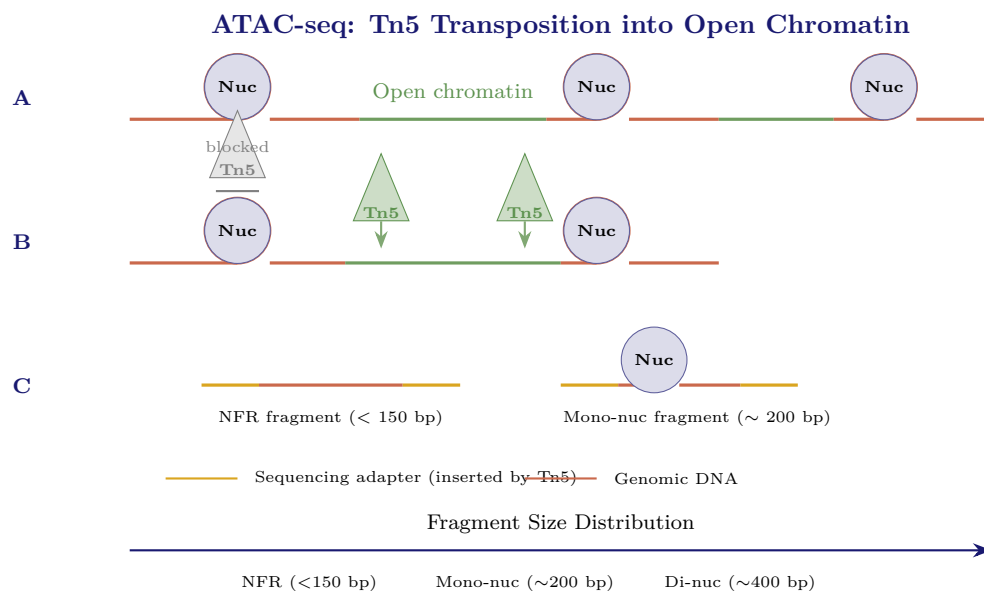


Figure 14.1: ATAC-seq principle. (A) Chromatin consists of nucleosome-wrapped regions (closed) and nucleosome-free regions (open). (B) The Tn5 transposase, pre-loaded with sequencing adapters, preferentially inserts into accessible (open) regions. Nucleosome-bound DNA blocks Tn5 insertion. (C) The resulting fragments include short nucleosome-free region (NFR) fragments and longer mono-nucleosome fragments, producing a characteristic fragment size distribution with peaks at < 150 bp (NFR), ~200 bp (mono-nucleosomal), and ~400 bp (di-nucleosomal).

14.3.2 Standard ATAC-seq Protocol

Standard ATAC-seq Protocol (Omni-ATAC)

1. **Cell preparation:** Harvest 50,000–100,000 cells. Count cells and assess viability (>90% recommended).
2. **Lysis:** Gently lyse cells with a non-ionic detergent (e.g., 0.1% NP-40, 0.1% Tween-20, 0.01% digitonin in Omni-ATAC) to release nuclei while keeping nuclei intact.
3. **Tagmentation:** Incubate nuclei with Tn5 transposase (e.g., Illumina Tagment DNA Enzyme) at 37°C for 30 minutes. The enzyme simultaneously fragments and adapter-tags accessible DNA.
4. **DNA purification:** Clean up tagmented DNA using a column or SPRI beads.
5. **PCR amplification:** Amplify library using indexed primers (5–12 cycles, determined by qPCR to avoid over-amplification).
6. **Size selection (optional):** Remove large fragments (>1000 bp) by bead-based selection. Some protocols skip this.
7. **QC:** Assess fragment size distribution on Bioanalyzer/TapeStation. A successful ATAC-seq library shows the characteristic nucleosomal laddering pattern.
8. **Sequencing:** Paired-end 50 or 75 bp reads. 50–75 million read pairs per sample recommended.

14.3.3 ATAC-seq Protocol Variants

Several optimised ATAC-seq protocols have been developed for specific applications:

- **Omni-ATAC:** The recommended standard protocol. Uses a combination of detergents (NP-40 + Tween-20 + digitonin) for more complete lysis, resulting in cleaner signal and reduced mitochondrial contamination. Works across diverse cell types.
- **Fast-ATAC:** Optimised for blood cells and other suspension cells. Cells are tagmented directly without a separate lysis step (Tn5 + detergent in a single reaction). Reduces hands-on time and improves results for difficult cell types.
- **miniATAC-seq:** Designed for very low cell numbers (500–5,000 cells). Uses modifications to reduce material loss at each step.
- **ATAC-seq on tissues:** Fresh or flash-frozen tissues are dissociated into single nuclei (mechanical dissociation or enzymatic digestion), then standard ATAC-seq is performed. snATAC-seq on nuclei avoids the need for live cells.
- **mtscATAC-seq:** Modified protocol that retains mitochondrial DNA information alongside chromatin accessibility in single cells, enabling clonal tracking based on mitochondrial mutations.

14.3.4 Tn5 Insertion Bias

The Tn5 transposase has a modest sequence preference for insertion. This **Tn5 bias** can affect footprinting analysis (where exact cut-site positions are used to infer TF binding). Bias correction methods include:

- **Naked DNA control:** Tagment purified genomic DNA (no chromatin) with Tn5 to empirically measure insertion bias.
- **Computational correction:** Tools like TOBIAS and seqOutBias model and correct for the Tn5 sequence preference.
- **k-mer-based normalisation:** Normalise cut-site frequencies by the local sequence composition.

Important Caveat

A major source of wasted sequencing in ATAC-seq is **mitochondrial reads**. Because mitochondrial DNA is abundant and lacks histones (and is therefore fully accessible), mitochondrial reads can constitute 30–60% of a standard ATAC-seq library. The Omni-ATAC protocol reduces this to ~5–15%, but filtering mitochondrial reads remains an essential bioinformatics step. Always report the fraction of mitochondrial reads as a QC metric.

14.4 Single-Cell ATAC-seq (scATAC-seq)

14.4.1 Technology and Platforms

Single-cell ATAC-seq (scATAC-seq) profiles chromatin accessibility in individual cells, revealing cell-type-specific regulatory landscapes within complex tissues. The dominant commercial platform is the **10x Genomics Chromium Single Cell ATAC** system:

1. Nuclei are isolated and tagmented in bulk with Tn5.
2. Tagmented nuclei are partitioned into nanoliter-scale gel bead-in-emulsion (GEM) droplets, each containing a single nucleus and a barcoded gel bead.
3. Within each GEM, linear amplification attaches cell-specific barcodes to the tagmented fragments.
4. After breaking the emulsions, all barcoded fragments are pooled, amplified, and sequenced.
5. Demultiplexing by barcode assigns fragments to individual cells.

Other platforms include **sci-ATAC-seq** (combinatorial indexing without droplets, enabling very high cell throughput), **SHARE-seq** (simultaneous ATAC + RNA from the same cell), and **10x Multiome** (combined scATAC + scRNA-seq from the same nucleus).

14.4.2 The Sparsity Challenge

scATAC-seq data is inherently **extremely sparse**. Each cell has only two copies of each autosomal locus (diploid genome), and the detection efficiency is far below 100%. A typical scATAC-seq experiment recovers only 5,000–20,000 unique fragments per cell. The resulting cell-by-peak matrix is >95% zeros, making analysis fundamentally different from scRNA-seq:

- Individual peaks in single cells are essentially **binary** (0 or 1 fragment).
- Clustering and dimensionality reduction must use methods suited to binary/sparse data.
- Feature aggregation (peaks → gene activity scores, or peaks → TF motif scores) is essential to accumulate signal.

14.4.3 Analysis Tools and Frameworks

- **ArchR:** Comprehensive R package for scATAC-seq analysis. Uses Arrow files for memory-efficient processing. Implements iterative LSI (latent semantic indexing) for dimensionality reduction, peak calling with MACS2, gene activity scores, motif enrichment, and trajectory analysis.
- **Signac:** Extension of Seurat for scATAC-seq. Provides integration with scRNA-seq data, chromVAR motif analysis, gene activity scoring, and multimodal analysis.
- **SnapATAC2:** Python-based tool using an anndata framework. Fast and memory-efficient; supports spectral embedding, cell clustering, and peak calling.
- **cisTopic:** Uses Latent Dirichlet Allocation (topic modelling) to identify co-accessible regulatory modules. Each “topic” represents a set of genomic regions that tend to be accessible together, often corresponding to specific cell types or regulatory programs.

Key analysis steps in scATAC-seq:

1. **Cell filtering:** Remove low-quality cells based on unique fragment count, TSS enrichment, and nucleosome signal.
2. **Dimensionality reduction:** LSI (TF-IDF normalisation → SVD) or topic modelling.
3. **Clustering:** Graph-based clustering (Louvain, Leiden) on the LSI embedding.
4. **Peak calling:** Call peaks per cluster (pseudo-bulk) using MACS2/MACS3.
5. **Gene activity scores:** Estimate gene activity by summing accessibility signal in gene body and promoter regions.
6. **Motif enrichment:** Identify enriched TF motifs in cluster-specific peaks using chromVAR or HOMER.
7. **Integration with scRNA-seq:** Transfer cell-type labels from scRNA-seq to scATAC-seq using integration methods (e.g., Seurat WNN, ArchR gene activity integration).

Practical Tip

When designing a scATAC-seq experiment, aim for at least 25,000 unique fragments per cell. Libraries with <5,000 fragments per cell are typically too sparse for meaningful analysis. Use TSS enrichment (>4 is good; >8 is excellent) as the primary QC metric for library quality.

14.5 Other Accessibility and Nucleosome Methods

14.5.1 FAIRE-seq

Formaldehyde-Assisted Isolation of Regulatory Elements (FAIRE-seq) was an early accessibility method. Cells are crosslinked with formaldehyde, then DNA is sheared and subjected to phenol-chloroform extraction. Protein-free (accessible) DNA partitions into the aqueous phase, while protein-bound (nucleosomal) DNA partitions into the organic phase. FAIRE-seq is now rarely used due to high background and replacement by ATAC-seq.

14.5.2 MNase-seq

Micrococcal nuclease (MNase) digests linker DNA between nucleosomes but is impeded by histone-bound DNA. By sequencing the protected fragments:

- **Nucleosome positioning maps** are generated, showing where nucleosomes sit along the genome.
- Well-positioned nucleosomes flanking TSSs and TF binding sites are characteristic features of active genes.
- MNase-seq complements ATAC-seq: ATAC-seq maps open regions, while MNase-seq maps nucleosome positions.

14.5.3 NOMe-seq

Nucleosome Occupancy and Methylome Sequencing (NOMe-seq) provides **simultaneous measurement of chromatin accessibility and DNA methylation on the same DNA molecule**:

1. Treat intact nuclei with GpC methyltransferase (M.CviPI), which methylates cytosines in GpC dinucleotides that are accessible.
2. Nucleosome-protected GpC sites are not methylated by M.CviPI.
3. Bisulfite sequencing then reads both: (a) endogenous CpG methylation (from the cell's own methylation), and (b) exogenous GpC methylation (marking accessible chromatin).
4. On long reads (nanopore or PacBio), this provides single-molecule, phased measurements of both methylation and accessibility.

14.6 Comprehensive Comparison of Accessibility Methods

Table 14.1: Comparison of major chromatin accessibility profiling approaches.

Method	Input Cells	Reads Needed	Resolution	Time	Key Features
DNase-seq	10^6 – 10^7	30–50M	~1 bp (cut site)	2–3 days	ENCODE standard; historical
ATAC-seq	50K–100K	50–75M	~1 bp (cut site)	~3 hours	Simple; dominant method
Omni-ATAC	50K–100K	50–75M	~1 bp	~3 hours	Improved ATAC; less mito contamination
scATAC-seq	5K–10K nuclei	20–50K/cell	Single cell	~4–5 hours	Cell-type-resolved accessibility
FAIRE-seq	10^6 – 10^7	20–30M	~200 bp	1–2 days	Superseded; no enzyme needed
MNase-seq	10^6 – 10^7	100–200M	~10 bp	1–2 days	Nucleosome positions
NOMe-seq	10^5 – 10^6	100–200M (WG)	Single molecule	1–2 days	Accessibility + methylation jointly

14.7 Bioinformatics Workflow for ATAC-seq

14.7.1 Analysis Overview

The standard ATAC-seq bioinformatics pipeline includes the following steps:

1. **Quality control:** FastQC on raw reads. Check for adapter contamination, insert size distribution.
2. **Adapter trimming:** Trim Galore or fastp to remove Nextera adapter sequences.
3. **Alignment:** Bowtie2 in paired-end mode with specific settings (e.g., `-X 2000` to allow large insert sizes, `-very-sensitive`).
4. **Filtering:**
 - Remove mitochondrial reads (`samtools view` excluding chrM).
 - Remove PCR duplicates (Picard MarkDuplicates or samtools markdup).
 - Remove low-quality alignments (`MAPQ < 30`).
 - Remove reads in ENCODE blacklist regions.
 - For paired-end data, keep only properly paired reads.
5. **Tn5 offset adjustment:** Shift reads by $+4/-5$ bp to centre on the Tn5 cut site (accounts for the 9 bp duplication).
6. **Peak calling:** MACS2 or MACS3 with appropriate settings (`-nomodel -shift -100 -extsize 200` for Tn5 cut-site-centred calling, or using the paired-end BAMPE mode).
7. **Quality metrics:**
 - Fragment size distribution (nucleosomal laddering).
 - TSS enrichment score.
 - FRiP (Fraction of Reads in Peaks).
8. **Differential accessibility:** DiffBind or DESeq2 on a consensus peak set.
9. **Motif analysis:** HOMER `findMotifsGenome.pl`, chromVAR, MEME Suite.
10. **Footprinting:** TOBIAS or HINT-ATAC for TF footprint detection.

MACS2 Dynamic Poisson Model for Peak Calling

MACS2 (Model-based Analysis of ChIP-Seq), the most widely used peak caller for both ChIP-seq and ATAC-seq data, employs a dynamic Poisson model that adapts to local background variation across the genome. Understanding this model clarifies how peaks are identified and why certain parameter choices matter.

Global background estimation. MACS2 first estimates a genome-wide average background rate λ_{genome} by dividing the total number of reads by the effective genome size:

$$\lambda_{\text{genome}} = \frac{N_{\text{reads}}}{G_{\text{effective}}} \quad (14.1)$$

where N_{reads} is the total number of mapped reads and $G_{\text{effective}}$ is the mappable genome size (e.g., 2.7×10^9 for human).

Local background estimation. Because read density varies across the genome due to copy number variation, mappability, and genuine biological heterogeneity, a global rate alone is insufficient. MACS2 estimates local background rates using sliding windows of different sizes centred on each candidate peak:

- λ_{1k} : background estimated from a 1 kb window around the peak.
- λ_{5k} : background estimated from a 5 kb window.
- λ_{10k} : background estimated from a 10 kb window.

The local lambda used for testing at each position is the maximum of all estimates, ensuring conservative peak calls:

$$\lambda_{\text{local}} = \max(\lambda_{1k}, \lambda_{5k}, \lambda_{10k}, \lambda_{\text{genome}}) \quad (14.2)$$

This dynamic approach prevents false-positive peak calls in regions with elevated background (e.g., copy number gains, accessible but non-regulatory regions).

Significance testing. Given the local background rate λ_{local} , MACS2 models the read count in a peak region as a Poisson random variable. The p-value for observing k or more reads under the null (background) model is:

$$p = P(X \geq k) = 1 - \sum_{i=0}^{k-1} \frac{\lambda_{\text{local}}^i e^{-\lambda_{\text{local}}}}{i!} \quad (14.3)$$

Multiple testing correction. Because MACS2 tests millions of candidate positions across the genome, multiple testing correction is essential. MACS2 applies the **Benjamini-Hochberg (BH)** procedure to control the false discovery rate (FDR). The output q-value column reports BH-adjusted p-values, and the default threshold of $q < 0.05$ (or $q < 0.01$ for ATAC-seq) corresponds to controlling the FDR at 5% (or 1%).

Practical Tip

For ATAC-seq data, MACS2 is typically run in **BAMPE** mode (paired-end), which uses the actual fragment boundaries rather than extending reads from a single end. This avoids the need for the **-nomodel -shift -extsize** parameters required for single-end data and provides more accurate peak boundaries. When using **BAMPE** mode, set **-keep-dup all** because PCR duplicates should already have been removed in the filtering step.

IDR: Irreproducible Discovery Rate

The **Irreproducible Discovery Rate (IDR)** framework, developed for the ENCODE project, provides a principled statistical method for assessing the reproducibility of peak calls across biological replicates. IDR is particularly important for defining a set of high-confidence peaks.

The copula mixture model. IDR models the ranked peak signals from two replicates as arising from a mixture of two components:

1. **Reproducible signal component:** True peaks that are consistently detected across replicates. The signals from the two replicates are correlated (modelled by a copula with high correspondence).
2. **Irreproducible noise component:** Spurious peaks that appear in one replicate but not the other (or at very different ranks). The signals from the two replicates are essentially uncorrelated.

Formally, for each peak i , let $(x_{i,1}, x_{i,2})$ be the signal intensities (e.g., $-\log_{10}(p)$) in replicates 1 and 2. The IDR model assumes:

$$f(x_{i,1}, x_{i,2}) = \pi_1 \cdot f_{\text{repro}}(x_{i,1}, x_{i,2}) + (1 - \pi_1) \cdot f_{\text{irrepro}}(x_{i,1}, x_{i,2}) \quad (14.4)$$

where π_1 is the proportion of reproducible peaks, f_{repro} is a bivariate distribution with high correlation (the reproducible component), and f_{irrepro} is a bivariate distribution with low or zero correlation (the irreproducible component).

IDR score. For each peak, the IDR framework computes the posterior probability of belonging to the irreproducible component. The IDR score is the expected proportion of irreproducible discoveries at a given rank threshold. Peaks with **IDR** < 0.05 are considered reproducible—analogueous to an FDR threshold of 5% but applied to replicate concordance rather than statistical significance.

Practical use. The ENCODE consortium recommends the following IDR workflow for ATAC-seq and ChIP-seq:

1. Call peaks on each individual replicate using lenient thresholds (e.g., $p < 0.01$ without FDR correction).
2. Run IDR on pairs of replicates to obtain a conservative, reproducible peak set (IDR < 0.05).
3. Call peaks on the pooled data (all replicates merged) for the final peak set used in downstream analysis.
4. Self-consistency analysis: split a single replicate into pseudo-replicates and verify that the IDR curve is well-behaved.

Important Caveat

IDR is designed for **sharp/narrow peaks** (such as those from TF ChIP-seq, H3K4me3, or ATAC-seq). It is not appropriate for broad marks (H3K27me3, H3K36me3) because broad peaks have fundamentally different signal characteristics that violate the IDR model assumptions. For broad marks, use overlap-based reproducibility metrics or the proportion of peaks shared between replicates.

Quality Metrics in Detail

Fragment Size Distribution: A successful ATAC-seq library produces a characteristic “nucleosomal ladder” pattern when fragment sizes are plotted:

- **Peak 1** (< 150 bp): nucleosome-free region (NFR) fragments—the primary signal of interest.
- **Peak 2** (~200 bp): mono-nucleosomal fragments.
- **Peak 3** (~400 bp): di-nucleosomal fragments.
- The ratio of NFR to mono-nucleosomal fragments indicates signal quality. Higher NFR enrichment is better.

TSS Enrichment Score: Calculated as the ratio of ATAC-seq signal at TSSs versus flanking regions (± 2 kb). The ENCODE consortium recommends TSS enrichment ≥ 6 for acceptable quality and ≥ 10 for high quality.

FRiP (Fraction of Reads in Peaks): The proportion of reads that fall within called peaks. ENCODE recommends FRiP ≥ 0.3 (30%). Low FRiP indicates high background or poor tagmentation.

14.7.2 Snakemake Workflow

Listing 14.1: Snakemake workflow for ATAC-seq analysis.

```

1 # Snakefile for ATAC-seq analysis
2 configfile: "config.yaml"
3
4 SAMPLES = config["samples"]
5 GENOME = config["genome"] # Bowtie2 index prefix
6 BLACKLIST = config["blacklist"]
7 EFFECTIVE_GENOME = config["effective_genome_size"]
8
9 rule all:
10     input:
11         "results/multiqc/multiqc_report.html",
12         expand("results/peaks/{sample}_peaks.narrowPeak",
13             sample=SAMPLES),
14         expand("results/bigwig/{sample}.bw", sample=SAMPLES),
15         "results/diffbind/differential_peaks.tsv",
16         "results/motifs/homer_results/knownResults.html",
17         expand("results/qc/{sample}_fragment_sizes.pdf",
18             sample=SAMPLES)
19
20 rule trim_adapters:
21     input:
22         r1 = "raw_data/{sample}_R1.fastq.gz",
23         r2 = "raw_data/{sample}_R2.fastq.gz"
24     output:
25         r1 = "results/trimmed/{sample}_R1_val_1.fq.gz",
26         r2 = "results/trimmed/{sample}_R2_val_2.fq.gz"
27     threads: 4
28     conda: "envs/trim_galore.yaml"
29     shell:
30         """
31         trim_galore --paired --nextera --cores {threads} \
32             --output_dir results/trimmed \
33             {input.r1} {input.r2}
34         """
35
36 rule bowtie2_align:
37     input:
38         r1 = "results/trimmed/{sample}_R1_val_1.fq.gz",
39         r2 = "results/trimmed/{sample}_R2_val_2.fq.gz"
40     output:
41         bam = temp("results/aligned/{sample}_raw.bam")
42     threads: 12
43     resources:
44         mem_mb = 32000
45     conda: "envs/bowtie2.yaml"
46     shell:
47         """
48         bowtie2 --very-sensitive -X 2000 --no-mixed \
49             --no-discordant --threads {threads} \
50             -x {GENOME} \
51             -1 {input.r1} -2 {input.r2} \
52             | samtools sort -@ {threads} -O bam \
53             -o {output.bam}
54         """
55
56 rule filter_bam:
57     input:

```

```

58     bam = "results/aligned/{sample}_raw.bam"
59 output:
60     bam = "results/filtered/{sample}_filtered.bam",
61     bai = "results/filtered/{sample}_filtered.bam.bai",
62     stats = "results/qc/{sample}_filter_stats.txt"
63 params:
64     blacklist = BLACKLIST
65 threads: 4
66 conda: "envs/samtools.yaml"
67 shell:
68     """
69     # Remove mito, low-quality, duplicates, blacklist
70     samtools view -h -f 2 -q 30 {input.bam} \
71         | grep -v 'chrM' \
72         | samtools sort -@ {threads} -O bam \
73         | samtools markdup -r -@ {threads} - - \
74         | bedtools intersect -v -abam stdin \
75             -b {params.blacklist} \
76         > {output.bam}
77     samtools index {output.bam}
78     samtools flagstat {output.bam} > {output.stats}
79     """
80
81 rule tn5_shift:
82     input:
83         bam = "results/filtered/{sample}_filtered.bam"
84     output:
85         bam = "results/shifted/{sample}_shifted.bam",
86         bai = "results/shifted/{sample}_shifted.bam.bai"
87     conda: "envs/deeptools.yaml"
88     shell:
89         """
90         alignmentSieve -b {input.bam} \
91             --ATACshift -o {output.bam}
92         samtools index {output.bam}
93         """
94
95 rule macs2_call_peaks:
96     input:
97         bam = "results/shifted/{sample}_shifted.bam"
98     output:
99         peaks = "results/peaks/{sample}_peaks.narrowPeak",
100         summits = "results/peaks/{sample}_summits.bed"
101     params:
102         name = "{sample}",
103         genome_size = EFFECTIVE_GENOME
104     conda: "envs/macs2.yaml"
105     shell:
106         """
107         macs2 callpeak -t {input.bam} \
108             -f BAMPE -g {params.genome_size} \
109             -n {params.name} \
110             --outdir results/peaks \
111             --nomodel --keep-dup all -q 0.01
112         """
113
114 rule make_bigwig:
115     input:

```

```

116     bam = "results/shifted/{sample}_shifted.bam",
117     bai = "results/shifted/{sample}_shifted.bam.bai"
118   output:
119     bw = "results/bigwig/{sample}.bw"
120   threads: 4
121   params:
122     genome_size = EFFECTIVE_GENOME
123   conda: "envs/deeptools.yaml"
124   shell:
125     """
126     bamCoverage -b {input.bam} -o {output.bw} \
127       --binSize 10 --normalizeUsing RPGC \
128       --effectiveGenomeSize {params.genome_size} \
129       -p {threads} --extendReads
130     """
131
132 rule fragment_size_plot:
133   input:
134     bam = "results/filtered/{sample}_filtered.bam",
135     bai = "results/filtered/{sample}_filtered.bam.bai"
136   output:
137     pdf = "results/qc/{sample}_fragment_sizes.pdf"
138   conda: "envs/deeptools.yaml"
139   shell:
140     """
141     bamPEFragmentSize -b {input.bam} \
142       --histogram {output.pdf} \
143       --maxFragmentLength 1000
144     """
145
146 rule homer_motifs:
147   input:
148     peaks = "results/peaks/{sample}_peaks.narrowPeak"
149   output:
150     dir = directory("results/motifs/homer_results"),
151     html = "results/motifs/homer_results/knownResults.html"
152   params:
153     genome = GENOME
154   threads: 4
155   conda: "envs/homer.yaml"
156   shell:
157     """
158     findMotifsGenome.pl {input.peaks} \
159       {params.genome} {output.dir} \
160       -size 200 -p {threads}
161     """
162
163 rule diffbind_analysis:
164   input:
165     peaks = expand(
166       "results/peaks/{sample}_peaks.narrowPeak",
167       sample=SAMPLES),
168     bams = expand(
169       "results/shifted/{sample}_shifted.bam",
170       sample=SAMPLES)
171   output:
172     results = "results/diffbind/differential_peaks.tsv"
173   conda: "envs/diffbind.yaml"

```

```

174     script:
175         "scripts/run_diffbind.R"
176
177 rule multiqc:
178     input:
179         expand("results/qc/{sample}_filter_stats.txt",
180             sample=SAMPLES),
181         expand("results/qc/{sample}_fragment_sizes.pdf",
182             sample=SAMPLES)
183     output:
184         "results/multiqc/multiqc_report.html"
185     conda: "envs/multiqc.yaml"
186     shell:
187         """
188         multiqc results/ -o results/multiqc --force
189         """

```

14.7.3 Nextflow Workflow

Listing 14.2: Nextflow DSL2 workflow for ATAC-seq analysis.

```

1  #!/usr/bin/env nextflow
2  nextflow.enable.dsl=2
3
4  params.reads          = "raw_data/*_R{1,2}.fastq.gz"
5  params.genome_index   = "reference/genome"
6  params.genome_fasta   = "reference/genome.fa"
7  params.blacklist      = "reference/blacklist.bed"
8  params.effective_size = "2.7e9"
9  params.outdir         = "results"
10
11 process TRIM_ADAPTERS {
12     tag "$sample_id"
13     cpus 4
14     publishDir "${params.outdir}/trimmed", mode: 'copy'
15
16     input:
17     tuple val(sample_id), path(reads)
18
19     output:
20     tuple val(sample_id), path("*_val_{1,2}.fq.gz"),
21                                     emit: trimmed
22     path "*_trimming_report.txt",    emit: reports
23
24     script:
25     """
26     trim_galore --paired --nextera \
27         --cores ${task.cpus} \
28         ${reads[0]} ${reads[1]}
29     """
30 }
31
32 process BOWTIE2_ALIGN {
33     tag "$sample_id"
34     cpus 12
35     memory '32 GB'
36

```

```

37   input:
38   tuple val(sample_id), path(reads)
39
40   output:
41   tuple val(sample_id), path("${sample_id}_raw.bam"),
42                                   emit: bam
43
44   script:
45   """
46   bowtie2 --very-sensitive -X 2000 \
47       --no-mixed --no-discordant \
48       --threads ${task.cpus} \
49       -x ${params.genome_index} \
50       -1 ${reads[0]} -2 ${reads[1]} \
51   | samtools sort -@ ${task.cpus} -O bam \
52       -o ${sample_id}_raw.bam
53   """
54 }
55
56 process FILTER_BAM {
57     tag "$sample_id"
58     cpus 4
59     publishDir "${params.outdir}/filtered", mode: 'copy'
60
61     input:
62     tuple val(sample_id), path(bam)
63
64     output:
65     tuple val(sample_id),
66         path("${sample_id}_filtered.bam"),
67         path("${sample_id}_filtered.bam.bai"),
68                                   emit: bam
69     path "${sample_id}_flagstat.txt",      emit: stats
70
71     script:
72     """
73     samtools view -h -f 2 -q 30 ${bam} \
74         | grep -v 'chrM' \
75         | samtools sort -@ ${task.cpus} -O bam \
76         | samtools markdup -r -@ ${task.cpus} - - \
77         | bedtools intersect -v -abam stdin \
78             -b ${params.blacklist} \
79         > ${sample_id}_filtered.bam
80
81     samtools index ${sample_id}_filtered.bam
82     samtools flagstat ${sample_id}_filtered.bam \
83         > ${sample_id}_flagstat.txt
84     """
85 }
86
87 process TN5_SHIFT {
88     tag "$sample_id"
89     publishDir "${params.outdir}/shifted", mode: 'copy'
90
91     input:
92     tuple val(sample_id), path(bam), path(bai)
93
94     output:

```

```

95     tuple val(sample_id),
96         path("${sample_id}_shifted.bam"),
97         path("${sample_id}_shifted.bam.bai"),
98         emit: bam
99
100     script:
101     """
102     alignmentSieve -b ${bam} --ATACshift \
103         -o ${sample_id}_shifted.bam
104     samtools index ${sample_id}_shifted.bam
105     """
106 }
107
108 process CALL_PEAKS {
109     tag "$sample_id"
110     publishDir "${params.outdir}/peaks", mode: 'copy'
111
112     input:
113     tuple val(sample_id), path(bam), path(bai)
114
115     output:
116     tuple val(sample_id),
117         path("${sample_id}_peaks.narrowPeak"),
118         emit: peaks
119
120     script:
121     """
122     macs2 callpeak -t ${bam} \
123         -f BAMPE -g ${params.effective_size} \
124         -n ${sample_id} \
125         --nomodel --keep-dup all -q 0.01
126     """
127 }
128
129 process BIGWIG {
130     tag "$sample_id"
131     cpus 4
132     publishDir "${params.outdir}/bigwig", mode: 'copy'
133
134     input:
135     tuple val(sample_id), path(bam), path(bai)
136
137     output:
138     path "${sample_id}.bw"
139
140     script:
141     """
142     bamCoverage -b ${bam} -o ${sample_id}.bw \
143         --binSize 10 --normalizeUsing RPGC \
144         --effectiveGenomeSize ${params.effective_size} \
145         -p ${task.cpus} --extendReads
146     """
147 }
148
149 process FOOTPRINTING {
150     tag "$sample_id"
151     cpus 4
152     publishDir "${params.outdir}/footprinting",

```



```

153         mode: 'copy'
154
155     input:
156     tuple val(sample_id), path(bam), path(bai)
157     tuple val(sample_id), path(peaks)
158
159     output:
160     path "TOBIAS_output/*"
161
162     script:
163     """
164     TOBIAS ATACorrect -b ${bam} \
165         -g ${params.genome_fasta} \
166         -p ${peaks} --cores ${task.cpus} \
167         --outdir TOBIAS_output
168
169     TOBIAS FootprintScores \
170         -s TOBIAS_output/*_corrected.bw \
171         -r ${peaks} --cores ${task.cpus} \
172         --outdir TOBIAS_output
173
174     TOBIAS BINDetect \
175         -m motif_database.jaspar \
176         -s TOBIAS_output/*_footprints.bw \
177         -b ${peaks} \
178         --genome ${params.genome_fasta} \
179         --cores ${task.cpus} \
180         --outdir TOBIAS_output/bindetect
181     """
182 }
183
184 // Main workflow
185 workflow {
186     reads_ch = Channel
187         .fromFilePairs(params.reads,
188             checkIfExists: true)
189
190     TRIM_ADAPTERS(reads_ch)
191     BOWTIE2_ALIGN(TRIM_ADAPTERS.out.trimmed)
192     FILTER_BAM(BOWTIE2_ALIGN.out.bam)
193     TN5_SHIFT(FILTER_BAM.out.bam)
194     CALL_PEAKS(TN5_SHIFT.out.bam)
195     BIGWIG(TN5_SHIFT.out.bam)
196     FOOTPRINTING(TN5_SHIFT.out.bam, CALL_PEAKS.out.peaks)
197 }

```

Practical Tip

The nf-core community maintains `nf-core/atacseq`, a production-grade Nextflow pipeline that handles all steps from raw reads through peak calling, differential analysis, and motif enrichment. It supports multiple genome builds, includes comprehensive QC metrics (TSS enrichment, FRiP, fragment distributions), and works on institutional HPC clusters and cloud platforms out of the box.

14.7.4 Differential Accessibility Analysis

To identify regions with significantly different accessibility between conditions:

1. **Create a consensus peak set:** Merge peaks from all samples (e.g., using `bedtools merge`), then count reads in each consensus peak for each sample using `featureCounts` or `bedtools multicov`.
2. **Normalise and test:** Use DESeq2 or edgeR on the count matrix to identify differentially accessible regions, or use the `DiffBind` R package, which wraps DESeq2/edgeR and provides ATAC-seq-specific functionality including affinity-based analysis.
3. **Annotate:** Use `ChIPseeker` or `HOMER annotatePeaks.pl` to assign peaks to nearest genes and genomic features (promoter, intron, intergenic, etc.).

14.7.5 Motif Analysis and Footprinting

Motif enrichment analysis identifies transcription factor binding motifs over-represented in accessible regions. Key tools:

- **HOMER:** `findMotifsGenome.pl` performs both known-motif enrichment and de novo motif discovery against a background set.
- **chromVAR:** Calculates per-cell TF motif accessibility deviation scores. Ideal for scATAC-seq and for comparing motif accessibility across conditions in bulk data.
- **MEME Suite:** Includes MEME (de novo motif finding), FIMO (individual motif scanning), and AME (motif enrichment testing).

TF footprinting goes beyond motif enrichment to detect evidence of actual TF occupancy. When a TF is bound to DNA, it protects a small region from Tn5 cutting, creating a “footprint” (a local dip in the otherwise high accessibility signal):

- **TOBIAS:** State-of-the-art ATAC-seq footprinting tool. Corrects Tn5 sequence bias, calculates footprint scores at motif instances, and uses `BINDetect` to identify differential TF binding between conditions.
- **HINT-ATAC:** Footprinting framework specifically designed for ATAC-seq data, with built-in bias correction models.

Important Caveat

Footprinting analysis requires substantial sequencing depth (>100 million uniquely mapped reads per sample) and careful Tn5 bias correction. At typical ATAC-seq depths (50–75 million reads), footprinting is often underpowered for all but the most abundantly expressed TFs. Motif enrichment analysis is more robust at standard sequencing depths and should be the primary analysis for most experiments.

14.7.6 TF Footprinting Scores: The TOBIAS Framework

TOBIAS (Transcription factor Occupancy prediction By Investigation of ATAC-seq Signal) provides a comprehensive framework for detecting transcription factor footprints from ATAC-seq data. The method proceeds through three stages: Tn5 bias correction, footprint scoring, and differential binding detection.

Tn5 Bias Correction (ATACCorrect)

The Tn5 transposase has an intrinsic sequence preference for insertion, which creates a non-uniform background of cut sites even in the absence of TF binding. If uncorrected, this bias can mimic or mask genuine footprints. TOBIAS corrects for Tn5 bias by:

1. Learning the **k-mer insertion preference** of Tn5 from the observed data. For each k-mer context (typically 6-mers) surrounding an insertion site, TOBIAS estimates the expected insertion frequency relative to the genomic average.
2. Computing an **expected insertion profile** for each genomic position based on the local sequence context and the learned Tn5 bias model.
3. Subtracting the expected (bias) profile from the observed insertion profile to produce a **bias-corrected insertion signal**. This corrected signal reflects chromatin accessibility and TF occupancy rather than enzyme preference.

Footprint Scoring (FootprintScores)

After bias correction, TOBIAS computes a **footprint score** at each potential TF binding site (defined by motif occurrences from databases such as JASPAR):

- At a motif site, TOBIAS examines the local pattern of Tn5 insertions. A bound TF creates a characteristic profile: **high insertion flanking the motif** (accessible DNA on either side of the TF) and **low insertion within the motif** (the TF physically blocks Tn5).
- The protection score quantifies this pattern:

$$\text{Footprint score} = \bar{S}_{\text{flanking}} - \bar{S}_{\text{motif}} \quad (14.5)$$

where $\bar{S}_{\text{flanking}}$ is the mean bias-corrected insertion signal in the flanking windows (typically ± 50 – 100 bp around the motif) and $\bar{S}_{\text{motif}}$ is the mean signal within the motif footprint. A high footprint score indicates strong TF protection (likely bound); a low or negative score suggests the motif is unoccupied.

Differential Binding Detection (BINDetect)

The final stage of TOBIAS compares footprint scores between conditions to identify TFs with **differential binding**:

- For each TF motif, TOBIAS collects the footprint scores at all motif instances genome-wide.
- It fits a mixture model to classify each motif instance as **bound** or **unbound** based on the footprint score distribution.
- Differential binding between conditions is assessed by comparing the proportion of bound motif instances and the distribution of footprint scores across conditions.
- Output includes a volcano-like plot showing each TF's change in binding activity versus significance.

14.7.7 Nucleosome Positioning from ATAC-seq Fragment Sizes

A unique feature of ATAC-seq is that the **fragment size distribution** encodes information about nucleosome positioning. Because Tn5 inserts into accessible linker DNA but is blocked by nucleosomes, the distance between two Tn5 insertion events on opposite sides of a nucleosome reflects the nucleosome footprint.

Fragment Size Classes

ATAC-seq fragments can be classified into three biologically meaningful categories based on the 147 bp nucleosome core particle size:

Table 14.2: Classification of ATAC-seq fragments by size and their chromatin structural interpretation.

Fragment Class	Size Range	Biological Interpretation
Sub-nucleosomal (NFR)	< 147 bp	Both Tn5 insertions in the same nucleosome-free region; represents open chromatin signal
Mono-nucleosomal	147–294 bp	Tn5 insertions flanking a single nucleosome; the fragment spans one nucleosome core particle
Di-nucleosomal	294–441 bp	Fragment spans two nucleosomes plus the intervening linker DNA
Multi-nucleosomal	> 441 bp	Fragment spans three or more nucleosomes; rare in well-prepared libraries

The 147 bp Periodicity

The characteristic “nucleosomal ladder” in the ATAC-seq fragment size distribution arises from the 147 bp periodicity imposed by the nucleosome core particle:

- **Peak at ~200 bp:** Mono-nucleosomal fragments (147 bp of wrapped DNA + ~50 bp of flanking linker DNA on each side).
- **Peak at ~400 bp:** Di-nucleosomal fragments (two nucleosomes + linker).
- **Peak at ~600 bp:** Tri-nucleosomal fragments (three nucleosomes + linkers).
- The exact peak positions depend on the linker DNA length, which varies between species, cell types, and genomic regions (typically 20–80 bp in mammals).

Inferring Nucleosome Occupancy

By separating fragments into sub-nucleosomal and nucleosomal size classes, one can generate distinct genome-wide signal tracks:

1. **NFR track** (fragments < 150 bp): represents the accessibility signal. This is the standard ATAC-seq open-chromatin signal used for peak calling.
2. **Nucleosome track** (fragments 150–300 bp): the midpoints of mono-nucleosomal fragments mark nucleosome dyad positions. A coverage track of these midpoints reveals nucleosome positioning patterns, including:
 - Well-positioned +1 and –1 nucleosomes flanking active TSSs.
 - Nucleosome-depleted regions (NDRs) at active regulatory elements.
 - Phased nucleosome arrays downstream of TSSs.
3. **Combined analysis:** Plotting the NFR signal and nucleosome occupancy together around genomic features (e.g., TSSs, TF motifs) provides a comprehensive view of the local chromatin architecture.

ATAC-seq as a Dual Assay

ATAC-seq simultaneously provides two complementary readouts: (1) **chromatin accessibility** from short, sub-nucleosomal fragments (< 147 bp), and (2) **nucleosome positioning** from mono-nucleosomal fragments (~147–294 bp). This duality is unique among accessibility assays and enables joint analysis of regulatory element activity and nucleosome architecture from a single experiment. Tools such as **NucleoATAC** and **HMMRATAC** exploit this property to jointly call accessible regions and position nucleosomes from ATAC-seq data.

14.8 Experimental Design Considerations

ATAC-seq Experimental Design Checklist

- **Cell number:** 50,000–100,000 cells per reaction (Omni-ATAC). Fresh cells preferred; flash-frozen nuclei are acceptable.
- **Replicates:** At least 2 biological replicates for peak calling, 3+ for differential analysis.
- **Sequencing:** Paired-end 50 or 75 bp; 50–75M read pairs per sample for bulk ATAC-seq.
- **Controls:** No input control is needed (unlike ChIP-seq), but include appropriate biological controls (e.g., untreated vs. treated).
- **Avoid:** over-tagmentation (destroys signal by fragmenting all chromatin), under-tagmentation (low library complexity), and over-amplification (excessive PCR duplicates).
- **Fresh Tn5:** Enzyme activity degrades over time; use aliquots and avoid repeated freeze-thaw cycles.

14.9 Chapter Summary

Chromatin accessibility profiling reveals the regulatory landscape of cells by mapping open chromatin regions genome-wide. ATAC-seq has become the dominant method due to its simplicity (50,000 cells, approximately one hour of bench work), and the Omni-ATAC variant further improves signal quality while reducing mitochondrial contamination. Single-cell ATAC-seq extends this approach to individual cells, albeit with extreme data sparsity that requires specialised computational approaches such as latent semantic indexing, topic modelling, and gene activity score estimation. The bioinformatics pipeline for ATAC-seq—from alignment and filtering through peak calling, differential accessibility, motif enrichment, and footprinting—provides a comprehensive view of the transcription factor binding landscape. Integration of accessibility data with gene expression (**scRNA-seq**), histone modifications (**ChIP-seq**/CUT&Tag), DNA methylation, and 3D genome architecture (**Hi-C**) data produces a multidimensional view of gene regulation that is far richer than any single assay alone.

Chapter 15

Histone Modifications and Protein–DNA Interactions

Key Concept

Histones are the protein spools around which DNA is wound in eukaryotic cells, forming nucleosomes—the fundamental units of chromatin. Chemical modifications to histone tails (methylation, acetylation, phosphorylation, ubiquitylation, and others) constitute a combinatorial **histone code** that regulates gene expression, DNA repair, replication, and chromosome condensation. Sequencing-based methods—ChIP-seq, CUT&RUN, and CUT&Tag—allow researchers to map the genome-wide locations of specific histone modifications and DNA-binding proteins, revealing the regulatory logic that governs cellular identity. This chapter covers the major histone marks, the experimental methods to profile them, and the bioinformatics workflows for their analysis.

15.1 The Histone Code

15.1.1 Histone Structure and Modifications

The nucleosome consists of 147 bp of DNA wrapped 1.65 times around an octamer of histone proteins (two copies each of H2A, H2B, H3, and H4). Each histone has an unstructured N-terminal “tail” that protrudes from the nucleosome and is subject to a wide array of post-translational modifications. These modifications are “written” by specific enzymes (methyltransferases, acetyltransferases), “erased” by others (demethylases, deacetylases), and “read” by effector proteins containing recognition domains (chromodomains, bromodomains, PHD fingers).

15.1.2 Key Histone Marks and Their Functions

15.1.3 Bivalent Chromatin

A particularly important concept is **bivalent chromatin**: the co-occurrence of the activating mark H3K4me3 and the repressive mark H3K27me3 at the same genomic locus. Bivalent domains are found at the promoters of key developmental genes in embryonic stem cells (ESCs). These genes are silenced but “poised” for rapid activation upon differentiation. During lineage commitment, bivalent domains resolve to either H3K4me3-only (activation) or H3K27me3-only (stable silencing).

Table 15.1: Major histone modifications profiled by sequencing-based methods and their associated chromatin states.

Mark	Type	Peak Shape	Biological Meaning
H3K4me1	Methylation	Broad	Active and poised enhancers
H3K4me3	Methylation	Sharp	Active promoters; marks TSSs of expressed genes
H3K27ac	Acetylation	Sharp/medium	Active regulatory elements (promoters and enhancers); distinguishes active from poised enhancers
H3K36me3	Methylation	Broad	Actively transcribed gene bodies; marks elongating RNA Pol II
H3K27me3	Methylation	Broad	Polycomb-mediated repression; silences developmental genes
H3K9me3	Methylation	Broad	Constitutive heterochromatin; marks pericentromeric repeats, transposable elements
H3K9me2	Methylation	Broad	Facultative heterochromatin; gene silencing in differentiated cells
H4K20me3	Methylation	Broad	Heterochromatin; DNA damage response
H3K4me2	Methylation	Sharp/medium	Active and poised regulatory elements
H2A.Z	Variant	Sharp	Nucleosome variant at promoters and enhancers; marks regulatory regions

15.1.4 The Combinatorial Histone Code

No single histone mark defines a chromatin state in isolation. Instead, combinations of marks define functional chromatin states. The **ChromHMM** algorithm (discussed in [Section 15.8.4](#)) uses hidden Markov models to learn chromatin states from multiple histone mark profiles simultaneously. Typical chromatin state models include 15–18 states such as:

- **Active TSS:** H3K4me3 + H3K27ac
- **Active enhancer:** H3K4me1 + H3K27ac
- **Poised enhancer:** H3K4me1 (without H3K27ac)
- **Strong transcription:** H3K36me3
- **Polycomb repressed:** H3K27me3
- **Heterochromatin:** H3K9me3
- **Quiescent:** no marks (desert)

Why Profile Multiple Histone Marks?

A minimum set of six histone marks—H3K4me3, H3K4me1, H3K27ac, H3K27me3, H3K9me3, and H3K36me3—is sufficient to define the major chromatin states in most cell types. This “core six” set is the basis for ENCODE and Roadmap Epigenomics chromatin state annotations. Profiling all six marks in your cell type of interest provides a comprehensive regulatory annotation of the genome.

15.2 ChIP-seq: Chromatin Immunoprecipitation Sequencing

15.2.1 Principle

chromatin immunoprecipitation sequencing (ChIP-seq) is the original and most widely used sequencing-based method for mapping protein–DNA interactions and histone modifications. The principle involves using an antibody specific to the target protein or modification to selectively enrich DNA fragments associated with that target:

1. **Crosslinking:** Treat cells with formaldehyde (1%, 10 minutes at room temperature) to covalently link proteins to nearby DNA.
2. **Cell lysis and chromatin shearing:** Lyse cells and shear chromatin into fragments of 200–500 bp by sonication or enzymatic digestion (MNase).
3. **Immunoprecipitation:** Incubate sheared chromatin with an antibody specific to the target (e.g., anti-H3K27ac). The antibody–chromatin complexes are captured on Protein A/G magnetic beads.
4. **Washing:** Wash beads stringently to remove non-specifically bound DNA.
5. **Reverse crosslinking:** Heat samples to reverse formaldehyde crosslinks and release DNA.
6. **DNA purification and library preparation:** Purify the enriched DNA and prepare a sequencing library (end repair, adapter ligation, PCR amplification).
7. **Sequencing:** Typically single-end 50–75 bp reads; 20–40 million uniquely mapped reads per sample.

ChIP-seq Protocol Summary

Starting material:	1–10 million cells per IP
Crosslinking:	1% formaldehyde, 10 min, RT
Fragmentation:	Sonication to 200–500 bp
Antibody:	2–10 μ g per IP; validated, lot-tested
IP incubation:	Overnight at 4°C
Washes:	4–6 washes, increasing stringency
Reverse crosslink:	65°C, 4–16 hours
Library prep:	Standard Illumina (e.g., NEBNext)
Sequencing:	SE50/75; 20–40M reads
Controls:	Input DNA (sheared, no IP) or IgG IP

15.2.2 Controls in ChIP-seq

- **Input control:** A fraction of the sheared chromatin that was not immunoprecipitated. This represents the background distribution of reads and corrects for biases in chromatin accessibility, mappability, and PCR amplification. *Input is the preferred control.*
- **IgG control:** An immunoprecipitation with a non-specific IgG antibody. Theoretically captures only non-specific binding. In practice, IgG controls often recover very little DNA and can be difficult to sequence, making input preferred.
- **Spike-in controls:** Adding a known amount of chromatin from another species (e.g., *Drosophila* chromatin with anti-H2Av antibody) to enable quantitative comparison between samples (spike-in normalisation).

15.2.3 Antibody Quality

Important Caveat

The quality of the ChIP-seq antibody is the single most critical factor determining experimental success. A poor antibody will produce high background, false-positive peaks, and unreliable results. ENCODE has established rigorous antibody validation standards:

- Western blot showing specificity (single band at expected size).
- Immunoprecipitation followed by mass spectrometry.
- Comparison of ChIP-seq signal with known genomic features (e.g., H3K4me3 at active TSSs).
- Lot-to-lot consistency testing.

Always use ENCODE-validated antibodies when available and check the antibody lot before committing to large-scale experiments.

15.2.4 Limitations of ChIP-seq

- Requires large cell numbers (1–10 million per IP).
- Antibody-dependent: quality and specificity vary.
- High background noise: only 1–5% of input DNA is typically recovered.
- Requires 20–40 million reads per sample.
- Crosslinking can introduce artefacts and is not fully reversible.
- Multi-day protocol (2–3 days minimum).

15.3 CUT&RUN: Cleavage Under Targets and Release Using Nuclease

15.3.1 Principle

CUT&RUN (Cleavage Under Targets and Release Using Nuclease), developed by Skene and Henikoff in 2017, addresses many limitations of ChIP-seq:

1. **Antibody binding:** Intact cells or nuclei (no crosslinking, no sonication) are incubated with a primary antibody against the target.
2. **Protein A–MNase binding:** A fusion protein of Protein A and micrococcal nuclease (pA-MN) is added and binds to the antibody.
3. **Targeted cleavage:** Addition of calcium activates the MNase, which cleaves DNA immediately adjacent to the antibody-bound target.
4. **Fragment release:** The small cleaved fragments (~150 bp) diffuse out of the nucleus into the supernatant.
5. **Collection and sequencing:** The released fragments are collected from the supernatant, purified, and sequenced.

Key advantages over ChIP-seq:

- **Lower input:** 100,000–500,000 cells (vs. 1–10 million for ChIP-seq).
- **Lower background:** only target-adjacent fragments are released; unbound chromatin stays in the nucleus.
- **Lower sequencing depth needed:** 3–8 million reads (vs. 20–40 million for ChIP-seq).
- **No crosslinking or sonication:** preserves native chromatin structure.
- **Faster:** can be completed in one day.

15.4 CUT&Tag: Cleavage Under Targets and Tagmentation

15.4.1 Principle

CUT&Tag (Cleavage Under Targets and Tagmentation), developed by Kaya-Okur et al. in 2019, further simplifies the workflow by replacing MNase cleavage with Tn5 transposase-mediated tagmentation:

1. **Antibody binding:** Intact nuclei are bound to concanavalin A magnetic beads, then incubated with primary antibody.
2. **Secondary antibody:** A secondary antibody is added to amplify the signal.
3. **pA-Tn5 or pAG-Tn5 binding:** A fusion protein of Protein A (or Protein A/G) and Tn5 transposase, pre-loaded with sequencing adapters, binds to the secondary antibody at target sites.
4. **Tagmentation:** Addition of magnesium activates Tn5, which inserts sequencing adapters into DNA near the target site.
5. **Library amplification:** DNA is purified and amplified with indexed primers. The library is ready for sequencing.

15.4.2 Advantages and Limitations of CUT&Tag

Advantages:

- Works with as few as **100–1,000 cells** (or even single cells).
- **Single-tube protocol:** no library preparation step—tagmentation directly inserts adapters.
- **Excellent signal-to-noise ratio:** very low background because only target-proximal DNA is tagmented.
- **Low sequencing cost:** 3–8 million reads are sufficient for most histone marks.
- **Fast:** can be completed in half a day.
- **No crosslinking or sonication required.**

Limitations:

- Works best for **abundant histone marks** (H3K27me3, H3K4me3, H3K27ac). Less reliable for transcription factors with small footprints and low abundance.
- Requires a **secondary antibody** (or Protein A/G-Tn5 that binds the primary directly).
- The pAG-Tn5 reagent may not be commercially available from all vendors; some labs produce it in-house.

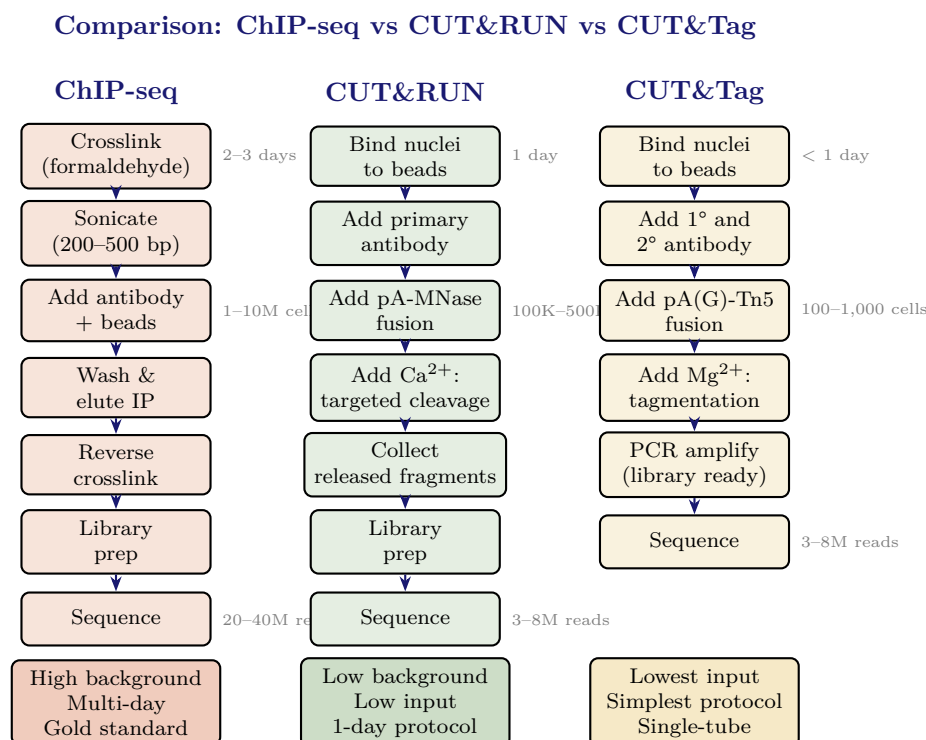


Figure 15.1: Side-by-side comparison of ChIP-seq, CUT&RUN, and CUT&Tag workflows. CUT&RUN and CUT&Tag progressively simplify the protocol, reduce input requirements, lower background, and decrease the sequencing depth needed. CUT&Tag is the simplest, requiring as few as 100–1,000 cells and producing sequencing-ready libraries in a single tube.

- Some bias from Tn5 sequence preferences (similar to ATAC-seq).

Practical Tip

For most new histone modification profiling experiments, CUT&Tag is the recommended starting point. It offers the simplest protocol, lowest input, and lowest cost. Reserve ChIP-seq for situations requiring ENCODE-standard comparability, for transcription factors that fail with CUT&Tag, or when large existing ChIP-seq datasets provide better statistical frameworks.

15.5 Single-Cell CUT&Tag

Single-cell CUT&Tag enables profiling of histone modifications in individual cells. Approaches include:

- **Droplet-based scCUT&Tag:** Nuclei are tagmented in bulk, then single nuclei are encapsulated in droplets (10x Genomics or similar) for barcoding, analogous to scATAC-seq.
- **Plate-based scCUT&Tag:** Individual cells or nuclei are sorted into 96/384-well plates, and the CUT&Tag reaction is performed in each well with unique index primers.
- **Multi-CUT&Tag:** Profiling multiple histone marks simultaneously in the same cell using different antibody–barcode combinations.

scCUT&Tag data shares the extreme sparsity challenges of scATAC-seq and uses similar computational approaches (LSI, clustering, pseudo-bulk analysis).

15.6 PLAC-seq and HiChIP

PLAC-seq (Proximity Ligation-Assisted ChIP-seq) and HiChIP combine chromatin immunoprecipitation with proximity ligation to map chromatin interactions at sites bound by a specific protein or bearing a specific histone mark:

- **HiChIP:** Cells are crosslinked, chromatin is digested with a restriction enzyme, and proximity ligation joins nearby fragments. The ligated chromatin is then immunoprecipitated with an antibody (e.g., anti-H3K27ac) and sequenced.
- **PLAC-seq:** Similar to HiChIP but with ChIP performed before proximity ligation.
- **Application:** H3K27ac HiChIP is widely used to map **active enhancer–promoter loops**, providing direct evidence for which enhancers regulate which genes.
- **Advantage over Hi-C:** By enriching for contacts at active regulatory elements, HiChIP requires far less sequencing depth than genome-wide Hi-C to detect enhancer–promoter interactions.

15.7 RNA–Protein Interaction Methods

15.7.1 CLIP-seq / eCLIP

Cross-Linking and Immunoprecipitation followed by sequencing (CLIP-seq) maps the binding sites of **RNA-binding proteins (RBPs)** on RNA:

1. UV crosslink cells (254 nm) to covalently link RBPs to bound RNA.
2. Lyse cells and fragment RNA (RNase treatment).
3. Immunoprecipitate with an anti-RBP antibody.
4. Ligate adapters to RNA fragments.
5. Reverse transcribe, amplify, and sequence.

Key variants:

- **iCLIP** (individual-nucleotide resolution CLIP): uses a circularisation step to capture cDNA truncation sites, providing nucleotide-resolution binding sites.
- **eCLIP** (enhanced CLIP): the ENCODE standard method. Includes size-matched input controls and an optimised protocol that improves library complexity and reduces artefacts. ENCODE has generated eCLIP datasets for >150 RBPs across multiple cell lines.
- **irCLIP** (infrared CLIP): uses infrared-labelled adapters for visualisation and optimised efficiency.

15.7.2 RIP-seq

RNA Immunoprecipitation sequencing (RIP-seq) is a simpler alternative to CLIP-seq. Cells are lysed, and an antibody against the RBP is used to immunoprecipitate the protein along with associated RNA, which is then sequenced. Unlike CLIP, RIP-seq does not use UV crosslinking, which makes it simpler but also means that some interactions may be lost or artefactual (due to post-lysis reassociation). RIP-seq provides transcript-level rather than nucleotide-level resolution.

15.7.3 DAP-seq

DNA Affinity Purification sequencing (DAP-seq) is an *in vitro* method for mapping transcription factor binding sites. Purified TF protein is incubated with naked genomic DNA (no chromatin context), and bound fragments are captured and sequenced. DAP-seq is widely used in plant genomics, where ChIP-grade antibodies are often unavailable, and has been applied to survey the binding specificities of hundreds of plant TFs.

15.8 Bioinformatics Workflow for ChIP-seq/CUT&Tag

15.8.1 Analysis Overview

The core bioinformatics pipeline for ChIP-seq, CUT&RUN, and CUT&Tag data follows a common structure, with minor differences in parameter settings:

1. **Quality control:** FastQC, MultiQC.
2. **Trimming:** Trim Galore or fastp (adapter removal).
3. **Alignment:** Bowtie2 (SE or PE). Use `-very-sensitive` mode. For CUT&Tag, use `-very-sensitive -no-mixed -no-discordant -X 700`.
4. **Filtering:** Remove duplicates, low-quality reads ($\text{MAPQ} < 30$), and reads in blacklist regions.
5. **Peak calling:**
 - **Sharp/narrow peaks** (H3K4me3, H3K27ac, TFs): MACS2 default settings.
 - **Broad peaks** (H3K27me3, H3K9me3, H3K36me3): MACS2 with `-broad` flag, or SICER2/epic2.
6. **Reproducibility:** IDR (Irreproducibility Discovery Rate) to assess peak reproducibility across replicates.
7. **Differential binding:** DiffBind or MANorm2.
8. **Visualisation:** deepTools (computeMatrix, plotHeatmap, plotProfile), IGV.
9. **Chromatin state segmentation:** ChromHMM (when profiling multiple marks).

Quality Metrics

- **FRiP (Fraction of Reads in Peaks):** ENCODE minimum is 1% for broad marks and 1% for narrow marks, though good experiments typically achieve 5–30%.
- **Cross-correlation (strand-shift) analysis:** Plots the Pearson correlation between read density on the + and – strands as a function of strand shift. A clear peak at the fragment length indicates successful enrichment. Metrics include NSC (Normalised Strand Coefficient) and RSC (Relative Strand Correlation).
- **IDR (Irreproducibility Discovery Rate):** Measures concordance between biological replicates. Peaks with $\text{IDR} < 0.05$ are considered reproducible.
- **Library complexity:** NRF (Non-Redundant Fraction), PBC1, PBC2 metrics from ENCODE.

ChIP Enrichment Statistics in Detail

The quality of a ChIP-seq or CUT&Tag experiment is determined by the degree of enrichment above background. Three key metrics—FRiP, NSC, and RSC—quantify this enrichment from complementary perspectives.

FRiP (Fraction of Reads in Peaks). FRiP measures the proportion of all mapped reads that fall within called peaks:

$$\text{FRiP} = \frac{\text{Number of reads in peaks}}{\text{Total number of mapped reads}} \quad (15.1)$$

ENCODE has established the following FRiP guidelines:

- **Sharp/narrow marks** (H3K4me3, H3K27ac, TFs): FRiP > 5% is acceptable; FRiP > 10% indicates a high-quality experiment. Top-performing experiments can achieve FRiP > 30%.
- **Broad marks** (H3K27me3, H3K9me3, H3K36me3): FRiP > 1% is the minimum ENCODE threshold. Because broad marks span large genomic domains, a lower FRiP can still represent genuine enrichment. Values of 5–15% are typical for well-performing experiments.
- FRiP is sensitive to peak calling parameters: more lenient thresholds inflate FRiP. Always report the peak caller and parameters used alongside FRiP values.

Cross-Correlation Analysis: NSC and RSC. The strand cross-correlation analysis exploits the directional asymmetry of ChIP-seq fragments. In a successful ChIP experiment, reads on the forward (+) strand are shifted upstream of the true binding site, while reads on the reverse (−) strand are shifted downstream, by a distance equal to the fragment length. Computing the Pearson correlation between + strand and − strand read density as a function of strand shift d produces a cross-correlation profile with:

- A peak at $d = \text{fragment length}$ (the true enrichment signal).
- A “phantom” peak at $d = \text{read length}$ (an artefact of mappability).

The two quality metrics derived from this profile are:

$$\text{NSC (Normalised Strand Coefficient)} = \frac{CC_{\text{fragment}}}{CC_{\text{min}}} \quad (15.2)$$

$$\text{RSC (Relative Strand Coefficient)} = \frac{CC_{\text{fragment}} - CC_{\text{min}}}{CC_{\text{phantom}} - CC_{\text{min}}} \quad (15.3)$$

where CC_{fragment} is the cross-correlation at the fragment-length shift, CC_{phantom} is the cross-correlation at the read-length shift, and CC_{min} is the minimum cross-correlation value.

ENCODE thresholds:

- **NSC ≥ 1.05** and **RSC ≥ 0.8**: acceptable quality.
- **NSC ≥ 1.1** and **RSC ≥ 1.0**: high quality.
- NSC or RSC values near 1.0 suggest weak enrichment or failed ChIP.

These metrics are computed by the `phantompeakqualtools` R package (also called `SPP`), which is integrated into the ENCODE pipeline and `nf-core/chipseq`.

Spike-in Normalisation for Quantitative Comparisons

Standard ChIP-seq normalisation methods (e.g., reads per million, RPGC) assume that the total amount of the target modification is approximately constant across samples. This assumption fails in experimental settings where the modification undergoes **global changes**—for example, when comparing cells treated with an HDAC inhibitor (global increase in acetylation) to untreated cells, or when studying H3K27me3 loss after *EZH2* knockout.

The spike-in approach. To enable quantitative comparisons when global histone modification levels change, a known amount of chromatin from a different species is added to each sample before immunoprecipitation:

1. Add a fixed amount of *Drosophila melanogaster* chromatin (typically 1–5% of the total chromatin mass) to each human/mouse sample prior to the ChIP step.
2. Perform ChIP using an antibody that recognises both the human/mouse target and the *Drosophila* ortholog (e.g., anti-H3K27me3 recognises the mark in both species), or add a second antibody against a *Drosophila*-specific histone variant (anti-H2Av).
3. After sequencing, align reads to a combined human/mouse + *Drosophila* reference genome.
4. Use the number of reads mapping to the *Drosophila* genome as a normalisation factor.

The scaling formula. For sample i , the spike-in scaling factor is:

$$\text{scale}_i = \frac{\text{median}_j(\text{spike}_j)}{\text{spike}_i} \quad (15.4)$$

where spike_i is the number of reads mapping to the *Drosophila* genome in sample i , and the median is taken across all samples in the experiment. Multiplying the human/mouse read counts by scale_i adjusts for differences in ChIP efficiency and global modification levels.

Important Caveat

Spike-in normalisation requires careful experimental execution. The *Drosophila* chromatin must be added at a consistent ratio before the immunoprecipitation step—not after. Variation in the spike-in amount across samples introduces normalisation artefacts. Additionally, the spike-in reads should constitute 1–10% of the total reads; too few spike-in reads lead to noisy normalisation, while too many waste sequencing capacity on non-target reads. Active Motif and EpiCypher offer commercial spike-in kits with calibrated standards.

15.8.2 Snakemake Workflow

Listing 15.1: Snakemake workflow for ChIP-seq/CUT&Tag analysis.

```

1 # Snakefile for ChIP-seq / CUT&Tag analysis
2 configfile: "config.yaml"
3
4 SAMPLES = config["samples"]          # Dict: sample -> {mark, condition}
5 CONTROLS = config["controls"]         # Dict: sample -> input_control
6 GENOME = config["genome_index"]      # Bowtie2 index prefix
7 BLACKLIST = config["blacklist"]
8 EFFECTIVE_GENOME = config["effective_genome_size"]
9
10 rule all:
11     input:
```



```

12     "results/multiqc/multiqc_report.html",
13     expand("results/peaks/{sample}_peaks.broadPeak",
14           sample=[s for s in SAMPLES
15                 if SAMPLES[s]["mark"] in
16                   ["H3K27me3", "H3K9me3", "H3K36me3"]]),
17     expand("results/peaks/{sample}_peaks.narrowPeak",
18           sample=[s for s in SAMPLES
19                 if SAMPLES[s]["mark"] in
20                   ["H3K4me3", "H3K27ac"]]),
21     expand("results/bigwig/{sample}.bw", sample=SAMPLES),
22     "results/deeptools/heatmap_TSS.pdf",
23     "results/diffbind/differential_binding.tsv",
24     "results/chromhmm/chromatin_states.bed"
25
26 rule trim_reads:
27     input:
28         r1 = "raw_data/{sample}_R1.fastq.gz",
29         r2 = "raw_data/{sample}_R2.fastq.gz"
30     output:
31         r1 = "results/trimmed/{sample}_R1_val_1.fq.gz",
32         r2 = "results/trimmed/{sample}_R2_val_2.fq.gz"
33     threads: 4
34     conda: "envs/trim_galore.yaml"
35     shell:
36         """
37         trim_galore --paired --cores {threads} \
38             --output_dir results/trimmed \
39             {input.r1} {input.r2}
40         """
41
42 rule bowtie2_align:
43     input:
44         r1 = "results/trimmed/{sample}_R1_val_1.fq.gz",
45         r2 = "results/trimmed/{sample}_R2_val_2.fq.gz"
46     output:
47         bam = "results/aligned/{sample}.bam",
48         bai = "results/aligned/{sample}.bam.bai"
49     threads: 12
50     resources:
51         mem_mb = 16000
52     conda: "envs/bowtie2.yaml"
53     shell:
54         """
55         bowtie2 --very-sensitive --no-mixed \
56             --no-discordant -X 700 \
57             --threads {threads} \
58             -x {GENOME} \
59             -1 {input.r1} -2 {input.r2} \
60             | samtools sort -@ {threads} -O bam -o - \
61             | samtools markdup -r -@ {threads} - {output.bam}
62         samtools index {output.bam}
63         """
64
65 rule filter_bam:
66     input:
67         bam = "results/aligned/{sample}.bam",
68         bai = "results/aligned/{sample}.bam.bai"
69     output:

```

```

70     bam = "results/filtered/{sample}_filtered.bam",
71     bai = "results/filtered/{sample}_filtered.bam.bai"
72     params:
73         blacklist = BLACKLIST
74     conda: "envs/samtools.yaml"
75     shell:
76         """
77         samtools view -b -f 2 -q 30 {input.bam} \
78             | bedtools intersect -v -abam stdin \
79             -b {params.blacklist} \
80             > {output.bam}
81         samtools index {output.bam}
82         """
83
84 rule macs2_narrow:
85     input:
86         treatment = "results/filtered/{sample}_filtered.bam",
87         control = lambda wc: "results/filtered/"
88             + CONTROLS[wc.sample] + "_filtered.bam"
89     output:
90         peaks = "results/peaks/{sample}_peaks.narrowPeak"
91     params:
92         name = "{sample}",
93         gs = EFFECTIVE_GENOME
94     conda: "envs/macs2.yaml"
95     shell:
96         """
97         macs2 callpeak -t {input.treatment} \
98             -c {input.control} \
99             -f BAMPE -g {params.gs} \
100             -n {params.name} --outdir results/peaks \
101             -q 0.01
102         """
103
104 rule macs2_broad:
105     input:
106         treatment = "results/filtered/{sample}_filtered.bam",
107         control = lambda wc: "results/filtered/"
108             + CONTROLS[wc.sample] + "_filtered.bam"
109     output:
110         peaks = "results/peaks/{sample}_peaks.broadPeak"
111     params:
112         name = "{sample}",
113         gs = EFFECTIVE_GENOME
114     conda: "envs/macs2.yaml"
115     shell:
116         """
117         macs2 callpeak -t {input.treatment} \
118             -c {input.control} \
119             -f BAMPE -g {params.gs} \
120             -n {params.name} --outdir results/peaks \
121             --broad --broad-cutoff 0.05
122         """
123
124 rule make_bigwig:
125     input:
126         bam = "results/filtered/{sample}_filtered.bam",
127         bai = "results/filtered/{sample}_filtered.bam.bai"

```

```

128     output:
129         bw = "results/bigwig/{sample}.bw"
130     threads: 4
131     params:
132         gs = EFFECTIVE_GENOME
133     conda: "envs/deeptools.yaml"
134     shell:
135         """
136         bamCoverage -b {input.bam} -o {output.bw} \
137             --binSize 10 --normalizeUsing RPGC \
138             --effectiveGenomeSize {params.gs} \
139             -p {threads} --extendReads
140         """
141
142 rule deeptools_heatmap:
143     input:
144         bw = expand("results/bigwig/{sample}.bw",
145                     sample=SAMPLES),
146         bed = config["tss_bed"]
147     output:
148         matrix = "results/deeptools/matrix_TSS.gz",
149         heatmap = "results/deeptools/heatmap_TSS.pdf",
150         profile = "results/deeptools/profile_TSS.pdf"
151     threads: 8
152     conda: "envs/deeptools.yaml"
153     shell:
154         """
155         computeMatrix reference-point \
156             -S {input.bw} -R {input.bed} \
157             --referencePoint TSS \
158             -b 3000 -a 3000 \
159             -p {threads} -o {output.matrix}
160
161         plotHeatmap -m {output.matrix} \
162             -o {output.heatmap} \
163             --colorMap RdBu_r \
164             --whatToShow 'heatmap and colorbar'
165
166         plotProfile -m {output.matrix} \
167             -o {output.profile} \
168             --perGroup
169         """
170
171 rule diffbind:
172     input:
173         peaks = expand("results/peaks/{sample}_peaks.narrowPeak",
174                       sample=SAMPLES),
175         bams = expand("results/filtered/{sample}_filtered.bam",
176                     sample=SAMPLES)
177     output:
178         "results/diffbind/differential_binding.tsv"
179     conda: "envs/diffbind.yaml"
180     script:
181         "scripts/run_diffbind.R"
182
183 rule chromhmm:
184     input:
185         bams = expand("results/filtered/{sample}_filtered.bam",

```

```

186         sample=SAMPLES)
187     output:
188         "results/chromhmm/chromatin_states.bed"
189     conda: "envs/chromhmm.yaml"
190     script:
191         "scripts/run_chromhmm.sh"
192
193 rule multiqc:
194     input:
195         expand("results/aligned/{sample}.bam", sample=SAMPLES)
196     output:
197         "results/multiqc/multiqc_report.html"
198     conda: "envs/multiqc.yaml"
199     shell:
200         "multiqc results/ -o results/multiqc --force"

```

15.8.3 Nextflow Workflow

Listing 15.2: Nextflow DSL2 workflow for ChIP-seq/CUT&Tag analysis.

```

1  #!/usr/bin/env nextflow
2  nextflow.enable.dsl=2
3
4  params.reads          = "raw_data/*_R{1,2}.fastq.gz"
5  params.genome_index   = "reference/genome"
6  params.genome_fasta   = "reference/genome.fa"
7  params.blacklist      = "reference/blacklist.bed"
8  params.effective_size = "2.7e9"
9  params.tss_bed        = "reference/tss_regions.bed"
10 params.sample_sheet   = "sample_sheet.csv"
11 params.outdir          = "results"
12
13 process TRIM_READS {
14     tag "$sample_id"
15     cpus 4
16     publishDir "${params.outdir}/trimmed", mode: 'copy'
17
18     input:
19     tuple val(sample_id), path(reads)
20
21     output:
22     tuple val(sample_id), path("*_val_{1,2}.fq.gz"),
23                                     emit: trimmed
24     path "*_trimming_report.txt",    emit: reports
25
26     script:
27     """
28     trim_galore --paired --cores ${task.cpus} \
29         ${reads[0]} ${reads[1]}
30     """
31 }
32
33 process BOWTIE2_ALIGN {
34     tag "$sample_id"
35     cpus 12
36     memory '16 GB'
37     publishDir "${params.outdir}/aligned", mode: 'copy'

```

```

38
39     input:
40     tuple val(sample_id), path(reads)
41
42     output:
43     tuple val(sample_id),
44         path("${sample_id}.bam"),
45         path("${sample_id}.bam.bai"),      emit: bam
46
47     script:
48     """
49     bowtie2 --very-sensitive --no-mixed \
50         --no-discordant -X 700 \
51         --threads ${task.cpus} \
52         -x ${params.genome_index} \
53         -1 ${reads[0]} -2 ${reads[1]} \
54         | samtools sort -@ ${task.cpus} -O bam \
55         | samtools markdup -r -@ ${task.cpus} - \
56         ${sample_id}.bam
57     samtools index ${sample_id}.bam
58     """
59 }
60
61 process FILTER_BAM {
62     tag "$sample_id"
63     publishDir "${params.outdir}/filtered", mode: 'copy'
64
65     input:
66     tuple val(sample_id), path(bam), path(bai)
67
68     output:
69     tuple val(sample_id),
70         path("${sample_id}_filt.bam"),
71         path("${sample_id}_filt.bam.bai"), emit: bam
72
73     script:
74     """
75     samtools view -b -f 2 -q 30 ${bam} \
76         | bedtools intersect -v -abam stdin \
77         -b ${params.blacklist} \
78         > ${sample_id}_filt.bam
79     samtools index ${sample_id}_filt.bam
80     """
81 }
82
83 process CALL_PEAKS_NARROW {
84     tag "$sample_id"
85     publishDir "${params.outdir}/peaks", mode: 'copy'
86
87     input:
88     tuple val(sample_id), path(treat_bam), path(treat_bai)
89     tuple val(control_id), path(ctrl_bam), path(ctrl_bai)
90
91     output:
92     tuple val(sample_id),
93         path("${sample_id}_peaks.narrowPeak"),
94         emit: peaks
95

```

```

96     script:
97     """
98     macs2 callpeak -t ${treat_bam} \
99         -c ${ctrl_bam} -f BAMPE \
100         -g ${params.effective_size} \
101         -n ${sample_id} -q 0.01
102     """
103 }
104
105 process CALL_PEAKS_BROAD {
106     tag "${sample_id}"
107     publishDir "${params.outdir}/peaks", mode: 'copy'
108
109     input:
110     tuple val(sample_id), path(treat_bam), path(treat_bai)
111     tuple val(control_id), path(ctrl_bam), path(ctrl_bai)
112
113     output:
114     tuple val(sample_id),
115         path("${sample_id}_peaks.broadPeak"),
116         emit: peaks
117
118     script:
119     """
120     macs2 callpeak -t ${treat_bam} \
121         -c ${ctrl_bam} -f BAMPE \
122         -g ${params.effective_size} \
123         -n ${sample_id} \
124         --broad --broad-cutoff 0.05
125     """
126 }
127
128 process BIGWIG {
129     tag "${sample_id}"
130     cpus 4
131     publishDir "${params.outdir}/bigwig", mode: 'copy'
132
133     input:
134     tuple val(sample_id), path(bam), path(bai)
135
136     output:
137     tuple val(sample_id), path("${sample_id}.bw"),
138         emit: bigwig
139
140     script:
141     """
142     bamCoverage -b ${bam} -o ${sample_id}.bw \
143         --binSize 10 --normalizeUsing RPGC \
144         --effectiveGenomeSize ${params.effective_size} \
145         -p ${task.cpus} --extendReads
146     """
147 }
148
149 process DEEPTOOLS_HEATMAP {
150     cpus 8
151     publishDir "${params.outdir}/deeptools", mode: 'copy'
152
153     input:

```

```

154 path bigwigs
155 path tss_bed
156
157 output:
158 path "heatmap_TSS.pdf"
159 path "profile_TSS.pdf"
160
161 script:
162 """
163 computeMatrix reference-point \
164     -S ${bigwigs} -R ${tss_bed} \
165     --referencePoint TSS \
166     -b 3000 -a 3000 \
167     -p ${task.cpus} -o matrix_TSS.gz
168
169 plotHeatmap -m matrix_TSS.gz \
170     -o heatmap_TSS.pdf \
171     --colorMap RdBu_r
172
173 plotProfile -m matrix_TSS.gz \
174     -o profile_TSS.pdf --perGroup
175 """
176 }
177
178 // Main workflow
179 workflow {
180     reads_ch = Channel
181         .fromFilePairs(params.reads,
182             checkIfExists: true)
183
184     TRIM_READS(reads_ch)
185     BOWTIE2_ALIGN(TRIM_READS.out.trimmed)
186     FILTER_BAM(BOWTIE2_ALIGN.out.bam)
187     BIGWIG(FILTER_BAM.out.bam)
188
189     DEEPTOOLS_HEATMAP(
190         BIGWIG.out.bigwig.map{ it[1] }.collect(),
191         file(params.tss_bed)
192     )
193 }

```

Practical Tip

The nf-core community provides `nf-core/chipseq` for ChIP-seq data and `nf-core/cutandrun` for CUT&RUN/CUT&Tag data. Both pipelines handle alignment, filtering, peak calling, QC, and visualisation with sensible defaults and extensive documentation. They support both narrow and broad peak calling modes and include IDR analysis for replicate concordance.

15.8.4 ChromHMM: Chromatin State Segmentation

When multiple histone marks have been profiled in the same cell type, **ChromHMM** uses a multivariate hidden Markov model to segment the genome into discrete chromatin states:

1. **Binarise:** Convert each histone mark's signal into binary (present/absent) calls in 200 bp genomic bins.

2. **Learn model:** Train an HMM with a user-specified number of states (typically 15–18). The model learns the emission probabilities (which combinations of marks define each state) and transition probabilities (how states change along the genome).
3. **Segment:** Apply the trained model to assign each 200 bp bin to a chromatin state.
4. **Annotate:** Compare the learned states to known genomic features (TSSs, gene bodies, enhancers, heterochromatin) to assign biological labels.

Typical ChromHMM States (15-State Model)

- States 1–3: Active TSS and flanking regions (H3K4me3, H3K27ac)
- States 4–5: Transcription (H3K36me3)
- States 6–7: Active enhancers (H3K4me1, H3K27ac)
- States 8–9: Zinc finger genes and repeats (H3K9me3, H3K36me3)
- States 10–11: Poised/bivalent elements (H3K4me1/me3, H3K27me3)
- States 12–13: Polycomb repressed (H3K27me3)
- States 14–15: Quiescent/heterochromatin (H3K9me3 or no marks)

ChromHMM Emission and Transition Model

ChromHMM formalises the combinatorial histone code as a **multivariate hidden Markov model (HMM)**. Understanding the mathematical framework clarifies how chromatin states are learned and why the resulting annotations are biologically meaningful.

Hidden states and observations. The genome is divided into fixed-size bins (typically 200 bp). At each bin t along the genome, the hidden state $q_t \in \{1, 2, \dots, K\}$ represents the unknown chromatin state, and the observation $\mathbf{o}_t = (o_{t,1}, o_{t,2}, \dots, o_{t,M})$ is a binary vector indicating the presence (1) or absence (0) of each of M histone marks. With six marks (the “core six”), each bin has a 6-dimensional binary observation vector, yielding $2^6 = 64$ possible observation patterns.

Emission probabilities. Each chromatin state k has an associated **emission probability matrix** that defines how likely each histone mark is to be observed when the genome is in state k . Assuming conditional independence of marks given the state (a key simplifying assumption), the emission probability for observing pattern $\mathbf{o}_t$ in state k is:

$$P(\mathbf{o}_t \mid q_t = k) = \prod_{m=1}^M p_{k,m}^{o_{t,m}} (1 - p_{k,m})^{1-o_{t,m}} \quad (15.5)$$

where $p_{k,m}$ is the probability of observing mark m in state k . The emission matrix $\mathbf{P} = [p_{k,m}]_{K \times M}$ is typically visualised as a heatmap, where each row is a state and each column is a mark. For example, the “Active TSS” state has high emission probabilities for H3K4me3 and H3K27ac but low probabilities for H3K27me3.

Transition probabilities. The **transition matrix** $\mathbf{A} = [a_{k,l}]_{K \times K}$ specifies the probability of transitioning from state k to state l between adjacent bins:

$$a_{k,l} = P(q_{t+1} = l \mid q_t = k) \quad (15.6)$$

with $\sum_{l=1}^K a_{k,l} = 1$ for all k . The transition matrix encodes the spatial organisation of chromatin states. For example, the “Active TSS” state has a high transition probability to the “Flanking TSS” state and a low probability of transitioning directly to “Heterochromatin.”

Training with Baum–Welch. The emission and transition parameters are learned from the data using the **Baum–Welch algorithm** (a special case of expectation-maximisation for HMMs):

1. **Initialisation:** Randomly initialise the emission matrix $\mathbf{P}$, transition matrix $\mathbf{A}$, and initial state distribution $\boldsymbol{\pi}$.
2. **E-step (Forward–Backward):** Compute the posterior probability $\gamma_t(k) = P(q_t = k \mid \mathbf{O})$ that each bin is in each state, given the full observation sequence $\mathbf{O}$ and current parameters.
3. **M-step:** Update the parameters to maximise the expected log-likelihood:

$$\begin{aligned} \bullet \hat{p}_{k,m} &= \sum_t \gamma_t(k) \cdot o_{t,m} / \sum_t \gamma_t(k) \\ \bullet \hat{a}_{k,l} &= \sum_t \xi_t(k, l) / \sum_t \gamma_t(k) \end{aligned}$$

where $\xi_t(k, l) = P(q_t = k, q_{t+1} = l \mid \mathbf{O})$.

4. **Iterate** E- and M-steps until convergence.

Decoding with Viterbi. Once the model is trained, the **Viterbi algorithm** finds the most probable sequence of hidden states (chromatin state assignment) for each chromosome:

$$\hat{\mathbf{q}} = \arg \max_{\mathbf{q}} P(\mathbf{q} \mid \mathbf{O}, \mathbf{A}, \mathbf{P}, \boldsymbol{\pi}) \quad (15.7)$$

This produces the final genome-wide chromatin state annotation—a segmentation of the genome into labelled states that can be viewed in a genome browser, compared across cell types, and used to annotate regulatory elements.

Choosing the number of states. ChromHMM requires the user to specify the number of states K . The standard Roadmap Epigenomics models use:

- **15-state model:** Sufficient for the “core six” marks; widely used and well-characterised.
- **18-state model:** Provides finer distinctions (e.g., separating genic enhancers from distal enhancers); used when additional marks (e.g., H3K4me2, H2A.Z) are available.
- Increasing K beyond the mark complexity leads to redundant states; decreasing K merges biologically distinct states. A useful diagnostic is to examine the emission heatmap—if two states have nearly identical emission patterns, K is too high.

15.8.5 Differential Binding Analysis

To compare histone modification or TF binding between conditions:

- **DiffBind:** R/Bioconductor package that counts reads in a consensus peak set across all samples, normalises using DESeq2 or edgeR, and identifies significantly differentially bound sites. Provides MA plots, PCA, and heatmaps for visualisation.
- **MAnorm2:** Normalises ChIP-seq signals using common peaks (regions with similar enrichment across conditions) as anchors, then tests for differential binding.
- **THOR:** HMM-based differential peak calling that does not require pre-called peaks; identifies differentially enriched regions directly from read data.

15.9 Experimental Design Considerations

Choosing Between ChIP-seq, CUT&RUN, and CUT&Tag

- **CUT&Tag:** First choice for histone marks (H3K4me3, H3K27me3, H3K27ac). Simplest protocol, lowest input, lowest cost.
- **CUT&RUN:** Good for transcription factors that do not work well with CUT&Tag.
- **ChIP-seq:** Reserve for ENCODE comparability, for targets where CUT&RUN/Tag fails, or for very well-established pipelines.
- **Replicates:** Minimum 2 biological replicates; 3+ recommended for differential analysis.
- **Controls:** Input control required for ChIP-seq; IgG control for CUT&RUN; no control or IgG for CUT&Tag.
- **Sequencing depth:** ChIP-seq 20–40M reads; CUT&RUN/Tag 3–8M reads for histone marks, 10–15M for TFs.

15.10 Chapter Summary

Histone modifications constitute a rich regulatory layer that controls gene expression, chromatin organisation, and cellular identity. The combinatorial histone code—read through methods like ChIP-seq, CUT&RUN, and CUT&Tag—reveals the functional annotation of every region in the genome. CUT&Tag has emerged as the most accessible method, enabling histone modification profiling from as few as hundreds of cells in a single-tube, half-day protocol. Bioinformatically, peak calling (MACS2 for narrow marks, SICER/epic2 for broad marks), differential binding analysis (DiffBind), chromatin state segmentation (ChromHMM), and signal visualisation (deepTools) form the core analytical toolkit. Combining histone modification data with chromatin accessibility (ATAC-seq), DNA methylation, and 3D genome organisation provides the most complete picture of the epigenomic landscape.

Chapter 16

Three-Dimensional Genome Architecture

Key Concept

The genome is not a linear string of DNA floating freely in the nucleus. Instead, it is elaborately folded in three-dimensional space, and this folding is functionally critical. Enhancers must physically contact the promoters they regulate, often across hundreds of kilobases of linear genomic distance. The genome is organised hierarchically—from chromosome territories and A/B compartments to topologically associating domains (TADs) and individual chromatin loops—and this organisation is mediated by architectural proteins such as CTCF and cohesin. Sequencing-based chromosome conformation capture methods, most notably Hi-C and its variants, allow researchers to map these three-dimensional interactions genome-wide, revealing the structural principles that govern gene regulation.

16.1 Why Three-Dimensional Genome Organisation Matters

The three-dimensional organisation of the genome has profound functional consequences:

- **Enhancer–promoter communication:** Enhancers often reside tens to hundreds of kilobases from their target promoters in linear distance but are brought into close physical proximity through chromatin looping. This spatial proximity is required for enhancer-mediated gene activation.
- **Gene insulation:** Topological boundaries prevent enhancers from activating inappropriate genes. Disruption of these boundaries (e.g., by structural variants that delete CTCF binding sites) can cause developmental disorders and cancer through enhancer hijacking.
- **Transcriptional regulation:** Genes located in the transcriptionally active A compartment tend to be expressed, while genes in the inactive B compartment tend to be silenced.
- **DNA replication:** Replication timing correlates with compartment identity: A compartment replicates early, B compartment replicates late.
- **DNA repair:** The spatial organisation of chromosomes influences the frequency and patterns of chromosomal translocations.

16.2 Levels of 3D Genome Organisation

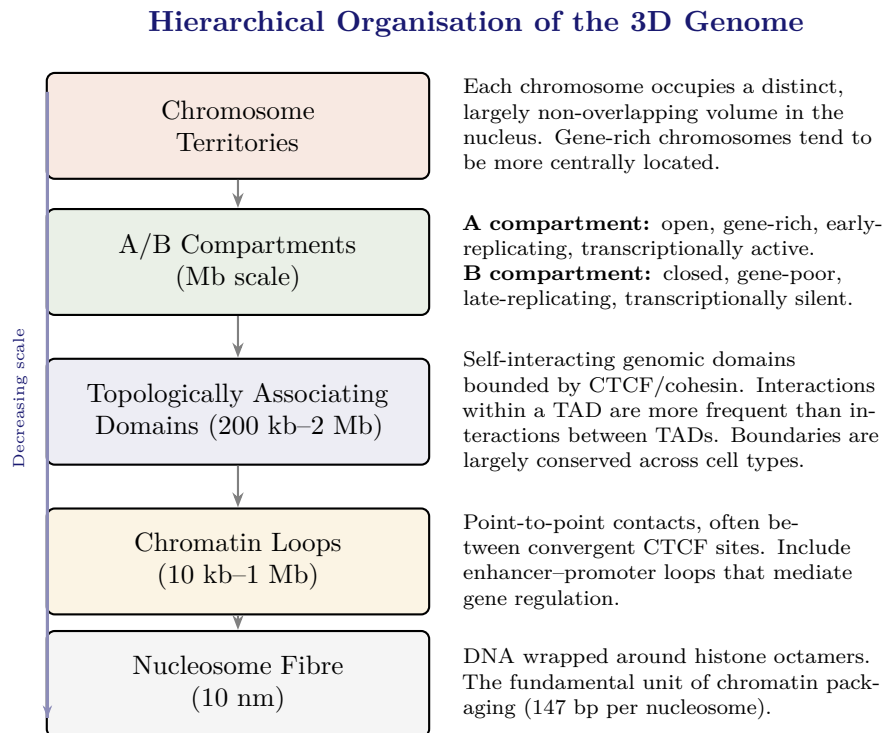


Figure 16.1: The 3D genome is organised hierarchically, from chromosome territories (largest scale) to individual nucleosomes (smallest scale). Each level can be interrogated by specific sequencing-based methods: chromosome territories and A/B compartments by Hi-C eigenvector analysis, TADs by Hi-C and Micro-C at moderate resolution, and chromatin loops by high-resolution Hi-C, HiChIP, or Capture Hi-C.

16.2.1 Chromosome Territories

Each chromosome occupies a distinct, largely non-overlapping volume in the interphase nucleus called a **chromosome territory**. Gene-rich chromosomes (e.g., chr19) tend to reside in the nuclear interior, while gene-poor chromosomes (e.g., chr18) tend to be positioned near the nuclear periphery (lamina-associated). Chromosome territories can be visualised by fluorescence in situ hybridisation (FISH) with whole-chromosome paints.

16.2.2 A/B Compartments

At the megabase scale, the genome is partitioned into two types of compartments:

- **A compartment:** euchromatic, gene-rich, transcriptionally active, early-replicating, enriched for active histone marks (H3K27ac, H3K4me3). Corresponds to open chromatin.
- **B compartment:** heterochromatic, gene-poor, transcriptionally silent, late-replicating, enriched for repressive marks (H3K27me3, H3K9me3) or lacking histone marks entirely. Corresponds to closed/compacted chromatin.

A/B compartments are identified from Hi-C data by performing principal component analysis (PCA) on the correlation matrix of Hi-C contact frequencies. The first eigenvector (PC1) separates the genome into A and B compartments. Compartment identity is dynamic and changes during differentiation, reprogramming, and disease.

16.2.3 Topologically Associating Domains (TADs)

TADs are contiguous genomic regions (typically 200 kb–2 Mb in mammals) within which chromatin interactions are substantially more frequent than interactions crossing TAD boundaries. TADs appear as prominent “triangles” along the diagonal of Hi-C contact maps.

Key properties of TADs:

- **Boundaries** are enriched for CTCF binding sites in convergent orientation, cohesin complex components, active promoters, and housekeeping genes.
- TAD boundaries are **largely conserved** across cell types and species, though sub-TAD organisation and loop contacts are more cell-type-specific.
- TADs serve as **regulatory domains**: enhancers within a TAD preferentially regulate genes within the same TAD.
- **Disruption of TAD boundaries** can cause enhancer hijacking—a mechanism implicated in developmental disorders (e.g., limb malformations) and cancer (e.g., oncogene activation).

16.2.4 Chromatin Loops

Chromatin loops are specific point-to-point contacts between two genomic loci. The most prominent loops are mediated by **CTCF and cohesin**:

- CTCF binds to its motif in a directional manner (the motif is asymmetric).
- Cohesin (a ring-shaped complex) extrudes chromatin until it encounters two convergently oriented CTCF sites (facing each other), forming a loop.
- This **loop extrusion model** explains how TADs and loops form: cohesin extrudes chromatin bidirectionally until blocked by convergent CTCF barriers.

The Loop Extrusion Model

The loop extrusion model proposes that cohesin (or condensin) complexes are loaded onto chromatin and actively extrude DNA through their ring structure, progressively enlarging a chromatin loop. Extrusion halts when cohesin encounters CTCF proteins bound in convergent orientation, establishing a stable loop with CTCF at its anchors. This model elegantly explains: (1) why loops form preferentially between convergent CTCF sites; (2) why TADs are insulated domains; and (3) why cohesin depletion eliminates loops and TADs but not compartments. The model has been supported by polymer simulations, cohesin depletion experiments (using auxin-inducible degen systems), and live-cell imaging.

16.3 Hi-C: Genome-Wide Chromosome Conformation Capture

16.3.1 Principle

Hi-C is the most widely used method for mapping genome-wide chromatin interactions. It captures all pairwise contacts in the genome simultaneously:

1. **Crosslinking:** Fix cells with formaldehyde to covalently link spatially proximal chromatin segments.

2. **Restriction enzyme digestion:** Digest chromatin with a restriction enzyme (e.g., MboI, DpnII, or HindIII). The choice of enzyme determines the theoretical resolution limit.
3. **Biotinylated fill-in:** Fill in the restriction enzyme overhangs with biotinylated nucleotides, marking the ligation junction.
4. **Proximity ligation:** Under dilute conditions (or in situ within the nucleus), ligate the blunt ends. Fragments that were spatially proximal (regardless of their linear genomic distance) are preferentially ligated.
5. **Biotin pull-down:** Shear the ligated DNA, pull down biotinylated junctions with streptavidin beads, and prepare a paired-end sequencing library.
6. **Sequencing and mapping:** Each read pair represents a contact between two genomic loci. Map each end independently to the reference genome to identify the interacting pair.

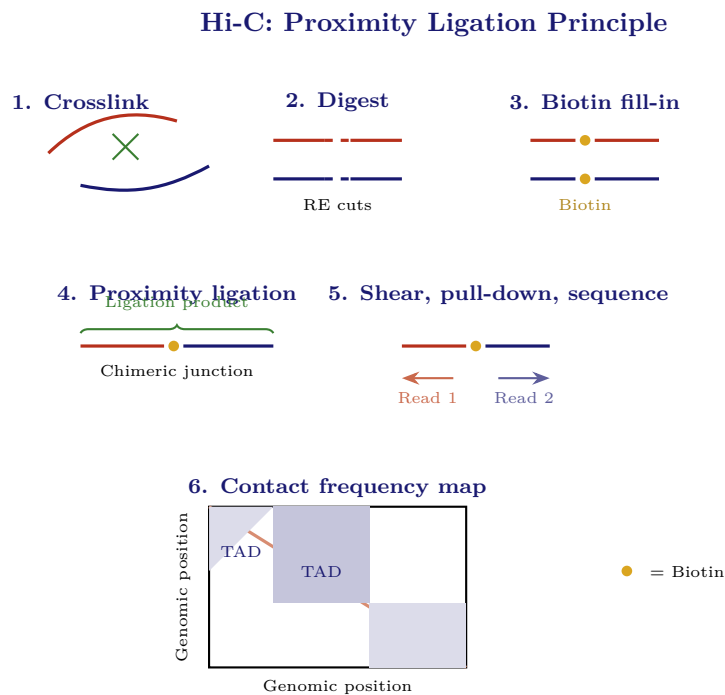


Figure 16.2: The Hi-C method. (1) Cells are crosslinked with formaldehyde to preserve spatial proximity. (2) Chromatin is digested with a restriction enzyme. (3) Overhangs are filled in with biotinylated nucleotides. (4) Spatially proximal fragments are ligated, creating chimeric junctions. (5) DNA is sheared, biotinylated junctions are pulled down, and paired-end sequencing is performed. (6) Read pairs are mapped to the genome to generate a contact frequency matrix (heatmap), where TADs appear as prominent triangles along the diagonal.

16.3.2 In Situ Hi-C vs. Dilution Hi-C

- **Dilution Hi-C (original method):** After restriction digestion, chromatin is highly diluted before ligation to promote intra-molecular (proximity) ligation over random inter-molecular ligation. This reduces noise but requires large cell numbers and has lower efficiency.

- **In situ Hi-C:** Ligation is performed inside intact nuclei, which naturally constrains ligation to spatially proximal fragments without the need for dilution. This produces higher-quality contact maps with less noise and is now the standard approach.

16.3.3 Resolution and Sequencing Depth

The resolution of a Hi-C contact map depends on sequencing depth:

Table 16.1: Hi-C resolution as a function of sequencing depth (human genome).

Resolution	Read Pairs Required	Features Resolved
1 Mb	~10 million	A/B compartments
100 kb	~100 million	TADs
40 kb	~300–500 million	TADs, sub-TADs
10 kb	~1 billion	Chromatin loops
5 kb	~2–3 billion	Fine loop architecture
1 kb	~10+ billion	Micro-C level detail

Important Caveat

Hi-C is one of the most sequencing-intensive methods in genomics. Achieving 1 kb resolution across a human genome requires >10 billion read pairs, costing tens of thousands of dollars. Most published Hi-C datasets are at 5–40 kb resolution. When designing a Hi-C experiment, carefully consider the resolution needed for your biological question and budget accordingly.

16.3.4 Hi-C Library Preparation Kits

- **Arima Hi-C:** Uses a cocktail of multiple restriction enzymes for finer fragmentation, achieving higher resolution per sequencing read. Widely used for both chromatin conformation and genome scaffolding.
- **Dovetail Omni-C:** Uses a sequence-independent chromatin fragmentation approach (DNase-based rather than restriction enzymes), eliminating restriction enzyme bias and providing more uniform coverage. Can be used for both 3D genome mapping and genome assembly scaffolding.
- **Phase Genomics ProxiMeta:** Hi-C kits optimised for metagenome deconvolution and assembly.

16.4 Micro-C

Micro-C replaces the restriction enzyme in Hi-C with micrococcal nuclease (MNase), which digests linker DNA between nucleosomes. This achieves **nucleosome-level resolution** (~200 bp):

- Improved resolution at short-range interactions (< 10 kb).
- Reveals fine-scale chromatin folding features (“stripes,” “dots”) not visible in standard Hi-C.
- Better signal-to-noise ratio for loop detection at moderate sequencing depths.
- Requires similar sequencing depths to Hi-C for genome-wide maps but provides superior resolution per read.

Practical Tip

For new chromatin conformation experiments where fine-scale loop detection is important, Micro-C offers advantages over restriction enzyme-based Hi-C. However, Hi-C remains the more established method with the larger existing dataset ecosystem and better standardised analysis pipelines.

16.5 Targeted and Enrichment-Based 3C Methods

16.5.1 Capture Hi-C / Promoter Capture Hi-C

Standard Hi-C distributes sequencing reads across all pairwise contacts genome-wide, making it expensive to achieve high resolution at specific loci. **Capture Hi-C** enriches Hi-C libraries for contacts involving specific regions of interest using hybridisation capture:

- **Promoter Capture Hi-C (PCHi-C):** Uses capture probes targeting all annotated gene promoters. Identifies enhancer–promoter interactions at specific genes without the cost of deep genome-wide Hi-C.
- **Region Capture Hi-C:** Capture probes target specific genomic regions (e.g., disease-associated loci) to map their interaction partners.
- Achieves high resolution (~ 1 – 4 restriction fragment resolution) at target loci with modest sequencing (100–200M read pairs).

16.5.2 HiChIP and PLAC-seq

HiChIP and PLAC-seq combine chromatin conformation capture with ChIP to map **contacts at specific protein binding sites or histone modifications**:

- **H3K27ac HiChIP:** The most widely used variant. Maps contacts between active regulatory elements (enhancers and promoters marked by H3K27ac), directly identifying active enhancer–promoter loops.
- **Cohesin HiChIP (SMC1A/RAD21):** Maps contacts at cohesin-mediated loops, identifying architectural loop anchors.
- **CTCF HiChIP:** Maps contacts at CTCF binding sites, revealing insulator-mediated loops and TAD boundaries.
- Requires far fewer reads than genome-wide Hi-C (50–200M per sample) because the antibody enrichment focuses sequencing depth on informative contacts.

16.5.3 ChIA-PET

Chromatin Interaction Analysis by Paired-End Tag sequencing (ChIA-PET) is an earlier method similar to HiChIP that uses ChIP followed by proximity ligation and paired-end tag sequencing. ChIA-PET was pioneered for CTCF and RNA Pol II and has been largely superseded by HiChIP due to the latter's simpler protocol and higher efficiency.

16.6 Long-Read and Alternative 3D Genome Methods

16.6.1 Pore-C (Oxford Nanopore)

Pore-C combines proximity ligation with nanopore long-read sequencing to capture **multi-way chromatin contacts**:

- Standard Hi-C reads contain only one ligation junction per read pair (pairwise contact). Pore-C long reads (>10 kb) can span **multiple ligation junctions**, revealing multi-way contacts (three or more fragments ligated together).
- Multi-way contacts provide information about **chromatin hubs**—regions where multiple genomic loci converge in 3D space simultaneously.
- Also useful for **genome scaffolding** in de novo assembly.

16.6.2 SPRITE

Split-Pool Recognition of Interactions by Tag Extension (SPRITE) captures multi-way chromatin contacts without proximity ligation:

1. Crosslink cells and fragment chromatin.
2. Split crosslinked chromatin complexes into 96-well plates and add a unique barcode (tag) in each well.
3. Pool and re-split into new plates for additional rounds of barcoding.
4. After multiple rounds, each chromatin complex carries a unique combination of barcodes (a “barcode string”) that identifies all fragments in the same complex.
5. Sequence and group fragments by shared barcode strings to identify multi-way contacts.

SPRITE can detect **higher-order interactions** (contacts among many loci simultaneously) and has revealed hub-like structures associated with nuclear bodies (nucleolus, nuclear speckles).

16.6.3 GAM (Genome Architecture Mapping)

GAM slices nuclei into thin cryosections, extracts DNA from each section, and sequences it. Loci that are spatially proximal are more likely to appear in the same nuclear section. By sequencing many sections, a contact frequency map is reconstructed. GAM does not require proximity ligation and can also detect multi-way contacts. It has been particularly useful for mapping **lamina-associated domains (LADs)**—regions of the genome that interact with the nuclear lamina and are typically transcriptionally silent.

16.6.4 DNA-FISH and Live Imaging

While not sequencing-based, imaging methods provide important complementary information:

- **DNA-FISH:** Fluorescence in situ hybridisation with locus-specific probes allows direct measurement of spatial distances between genomic loci in individual cells. Validates Hi-C-predicted contacts.
- **Multiplexed FISH (ORCA, chromatin tracing):** Sequential rounds of hybridisation and imaging map the 3D positions of hundreds to thousands of genomic loci in single cells, providing single-cell 3D genome structure data.
- **Live-cell imaging:** CRISPR-Cas9-based labelling or MS2/PP7 RNA tagging enables real-time tracking of locus dynamics in living cells.

Table 16.2: Comparison of major methods for mapping three-dimensional genome organisation.

Method	Resolution	Read Pairs	Multi-way?	Key Features
Hi-C	1 kb–1 Mb	100M–3B	No (pairwise)	Gold standard; genome-wide all-vs-all
Micro-C	~200 bp	100M–3B	No	Nucleosome resolution; superior short-range
Capture Hi-C	1–4 RE fragments	100–200M	No	Targeted enrichment; cost-effective for loci
HiChIP	5–10 kb	50–200M	No	ChIP + proximity ligation; active loops
ChIA-PET	5–10 kb	50–200M	No	Earlier version of HiChIP; largely replaced
Pore-C	~1 kb	50–200M reads	Yes	Long-read; multi-way contacts
SPRITE	~1 Mb	Variable	Yes	Ligation-free; nuclear body hubs
GAM	~30 kb	Variable	Yes	Cryosectioning; no ligation; LADs

16.7 Comprehensive Comparison of 3D Genome Methods

16.8 Bioinformatics Workflow for Hi-C

16.8.1 Analysis Overview

Hi-C data analysis follows a distinct pipeline from other sequencing assays because each read pair represents a chromatin contact rather than a contiguous genomic fragment:

1. **Read alignment:** Each mate of a read pair is aligned independently to the reference genome (because the two ends may originate from different chromosomes or distant loci). Aligners: BWA-MEM (with chimeric read handling), chromap, or the aligner built into specific pipelines.
2. **Contact pair identification:** Valid Hi-C contacts (“pairs”) are identified from read pairs that map to different restriction fragments. Invalid pairs (self-ligations, dangling ends, PCR duplicates) are filtered.
3. **Contact matrix generation:** Valid pairs are binned into a contact frequency matrix at a chosen resolution (bin size). Common formats: `.cool` (cooler), `.hic` (Juicer), or `.mcool` (multi-resolution cooler).
4. **Normalisation:** The raw contact matrix contains biases from GC content, mappability, restriction site density, and other factors. Normalisation methods include:
 - **ICE (Iterative Correction and Eigenvector decomposition):** Matrix balancing that equalises the total signal per row/column. The most widely used method. Implemented in cooler.
 - **KR (Knight-Ruiz):** A faster matrix balancing algorithm. Default in Juicer.
 - **VC (Vanilla Coverage):** Simple normalisation by the product of row and column sums.
5. **Feature calling:**

- **Compartments:** PCA on the contact correlation matrix (cooltools or HOMER).
 - **TADs:** TopDom, Arrowhead (Juicer), HiCExplorer `hicFindTADs`, or insulation score methods.
 - **Loops:** HICCUPS (Juicer), Mustache, Chromosight, or cooltools `dots`.
6. **Visualisation:** HiGlass (web-based interactive viewer), Juicebox (desktop application), HiCExplorer `hicPlotMatrix`, or cooltools for programmatic access.
 7. **Integration:** Overlay Hi-C contacts with epigenomic tracks (ATAC-seq, ChIP-seq, RNA-seq) to link 3D structure to function.

The following subsections describe the key computational methods for normalisation, TAD calling, compartment identification, and polymer physics models in greater detail.

16.8.2 Contact Matrix Normalisation Methods

Raw Hi-C contact matrices contain systematic biases arising from variable restriction site density, GC content, mappability, and chromatin accessibility across the genome. Normalisation removes these biases to produce matrices that more faithfully represent true contact frequencies. Three major normalisation approaches are in common use.

ICE (Iterative Correction and Eigenvector Decomposition)

ICE, implemented in the `cooler` ecosystem, is the most widely used normalisation method. It is based on the assumption that, in the absence of systematic biases, each genomic locus should participate in approximately the same total number of contacts. The algorithm proceeds iteratively:

1. Let $\mathbf{M}$ be the raw $N \times N$ contact matrix, where M_{ij} is the number of contacts between bins i and j .
2. At each iteration, compute the total contacts (row/column sum) for each bin: $S_i = \sum_j M_{ij}$.
3. Divide each row and column by the square root of its sum to produce a partially corrected matrix:

$$M_{ij}^{(t+1)} = \frac{M_{ij}^{(t)}}{\sqrt{S_i^{(t)} \cdot S_j^{(t)}}} \quad (16.1)$$

4. Repeat until convergence, defined as the point where all row and column sums are approximately equal.
5. The final normalised matrix $\tilde{\mathbf{M}}$ has balanced margins (equal row/column sums), and the bias vector $\mathbf{b}$ (product of all per-iteration normalisation factors) captures the per-bin biases.

Bins with very low contact counts (e.g., unmappable or repetitive regions) are excluded from balancing by applying a **mad-max filter**: bins whose log-transformed total counts deviate by more than a threshold (default: 5 median absolute deviations) from the median are masked.

KR (Knight–Ruiz) Balancing

The Knight–Ruiz (KR) algorithm achieves the same goal as ICE—equalising row and column sums—but uses a faster, mathematically rigorous matrix balancing procedure. KR finds diagonal matrices $\mathbf{D}_1$ and $\mathbf{D}_2$ such that the normalised matrix $\tilde{\mathbf{M}} = \mathbf{D}_1 \mathbf{M} \mathbf{D}_2$ has equal row sums and equal column sums. For symmetric matrices (as in Hi-C), $\mathbf{D}_1 = \mathbf{D}_2 = \mathbf{D}$, and the balanced matrix is:

$$\tilde{M}_{ij} = \frac{M_{ij}}{d_i \cdot d_j} \quad (16.2)$$

where d_i are the diagonal elements of $\mathbf{D}$, found by an iterative algorithm that converges faster than ICE. KR normalisation is the default in **Juicer** and produces results very similar to ICE.

VC (Vanilla Coverage) Normalisation

VC normalisation is the simplest approach: each contact count is divided by the product of the marginal sums of its row and column:

$$\tilde{M}_{ij} = \frac{M_{ij}}{S_i \cdot S_j} \times C \quad (16.3)$$

where $S_i = \sum_j M_{ij}$, $S_j = \sum_i M_{ij}$, and C is a constant (e.g., the total number of contacts) to keep values in a reasonable range. VC is a single-step procedure (no iteration) and is fast, but it over-corrects for highly interacting loci and is less robust than ICE or KR. It is mainly used as a baseline or for quick exploratory analyses.

Practical Tip

For most analyses, ICE (cooler) or KR (Juicer) normalisation is recommended. The two methods produce highly concordant results ($r > 0.99$ for most genomic regions). The choice is often driven by the analysis ecosystem: use ICE with the cooler/cooltools pipeline, and KR with the Juicer pipeline. Always exclude low-coverage bins from normalisation to prevent artefacts from unmappable regions.

16.8.3 TAD Calling Algorithms

Topologically associating domains (TADs) are self-interacting chromosomal regions where loci interact more frequently with each other than with loci in neighbouring domains. TAD boundaries are enriched for CTCF binding sites and often demarcate regulatory neighbourhoods. Two principal algorithms for TAD identification are the insulation score and the directionality index.

Insulation Score

The insulation score, originally proposed by Crane et al., quantifies the degree of insulation at each genomic bin by examining the average interaction frequency in a square diamond window centred on the bin's position along the matrix diagonal:

1. For each bin i , define a diamond-shaped window spanning w bins upstream and w bins downstream of i . This window captures contacts *across* position i —i.e., interactions between the region upstream of i and the region downstream.
2. Compute the insulation score as the mean contact frequency within this diamond:

$$\text{IS}(i) = \log_2 \left(\frac{1}{w^2} \sum_{i-w \leq a < i} \sum_{i < b \leq i+w} M_{a,b} \right) \quad (16.4)$$

3. Normalise the insulation scores to the genome-wide mean.
4. TAD boundaries correspond to **local minima** in the insulation score profile—positions where cross-boundary interactions are depleted relative to within-domain interactions.
5. The depth of the minimum (“boundary strength”) quantifies how strong the insulation is at each boundary.

The window size w (in bins) is a critical parameter. Smaller windows (~ 100 – 250 kb) detect smaller TADs; larger windows (~ 500 kb– 1 Mb) detect larger domains. The insulation score method is implemented in `cooltools insulation`.

Directionality Index

The directionality index (DI), introduced by Dixon et al., captures the asymmetry between upstream and downstream interactions at each bin. Within a TAD, bins near the upstream boundary interact preferentially downstream (toward the domain interior), while bins near the downstream boundary interact preferentially upstream:

$$\text{DI}(i) = \left(\frac{B_i - A_i}{|B_i - A_i|} \right) \cdot \frac{(A_i - E)^2 + (B_i - E)^2}{E} \quad (16.5)$$

where $A_i = \sum_{j=i-w}^{i-1} M_{ij}$ is the total upstream contact frequency, $B_i = \sum_{j=i+1}^{i+w} M_{ij}$ is the total downstream contact frequency, and $E = (A_i + B_i)/2$ is the expected value under the null hypothesis of no directional bias.

- **DI** > 0 (downstream bias): the bin interacts more with downstream neighbours, indicating the upstream portion of a TAD.
- **DI** < 0 (upstream bias): the bin interacts more with upstream neighbours, indicating the downstream portion of a TAD.
- **DI** $\approx$ 0: no directional preference (interior of a TAD or inter-TAD region).
- A **hidden Markov model** is then applied to the DI profile to segment the genome into domains. Transitions from negative to positive DI correspond to TAD boundaries.

The DI approach is implemented in the original `Juicer/Arrowhead` tool and in `HiCExplorer hicFindTADs`.

16.8.4 Compartment Analysis

At the megabase scale, the genome is partitioned into two types of compartments: **A compartments** (active, gene-rich, open chromatin) and **B compartments** (inactive, gene-poor, closed chromatin). These compartments manifest as a “checkerboard” (plaid) pattern in the Hi-C contact matrix.

Eigenvector Decomposition

Compartments are identified by **principal component analysis (PCA)** of the observed/expected (O/E) contact correlation matrix:

1. Compute the expected contact frequency E_{ij} for each pair of bins as a function of genomic distance (the average contact frequency at that distance).

2. Construct the O/E matrix: $OE_{ij} = M_{ij}/E_{ij}$.
3. Compute the **Pearson correlation matrix** of the O/E matrix: each row i of the O/E matrix defines a contact profile for bin i , and the correlation matrix captures the similarity of contact profiles between pairs of bins.
4. Perform eigendecomposition of the correlation matrix. The **first eigenvector (PC1)** captures the strongest pattern of co-variation in contact profiles, which corresponds to the A/B compartment structure.
5. Bins with **positive PC1 values** belong to one compartment, and bins with **negative PC1 values** belong to the other.

A/B assignment. The sign of the eigenvector is arbitrary (PCA does not assign biological labels). To resolve this ambiguity, the eigenvector is correlated with GC content or gene density: the compartment with higher GC content and gene density is labelled **A** (active), and the other is labelled **B** (inactive). In most cell types:

- **A compartments:** euchromatic, high gene density, high GC content, enriched for H3K27ac and H3K4me1, actively transcribed.
- **B compartments:** heterochromatic, low gene density, low GC content, enriched for H3K9me3 or H3K27me3, often lamina-associated.

Compartment analysis is performed at coarse resolution (50–250 kb bins) and is implemented in `cooltools eigs-cis` and `Juicer eigenvector`.

16.8.5 Polymer Physics Models of Chromatin Folding

The three-dimensional folding of chromatin can be described using concepts from **polymer physics**. These models provide a theoretical framework for interpreting the distance-dependent decay of contact frequencies observed in Hi-C data.

Contact Probability Scaling

A fundamental observation from Hi-C data is that the probability of two loci being in contact decreases as a power law with increasing genomic distance:

$$P(s) \propto s^{-\gamma} \quad (16.6)$$

where $P(s)$ is the contact probability at genomic separation s (in base pairs) and γ is the scaling exponent. The value of γ reveals information about the polymer organisation of chromatin:

- **Equilibrium globule** (random polymer in confinement): $\gamma \approx 1.5$. In this model, the polymer has reached thermodynamic equilibrium, and distant loci have very low contact probability. Early Hi-C experiments showed that chromatin does *not* behave as an equilibrium globule.
- **Fractal globule:** $\gamma \approx 1.0$. The fractal globule is a **knot-free** polymer conformation that has not reached equilibrium. It is characterised by self-similar organisation at multiple scales and the absence of topological entanglements. The original Lieberman-Aiden et al. (2009) Hi-C study found $\gamma \approx 1.08$ for human chromatin, closely matching fractal globule predictions.
- **Loop extrusion model:** γ varies with scale. At short ranges (< 100 kb), γ is influenced by loop extrusion dynamics; at intermediate ranges (100 kb–10 Mb), $\gamma \approx 0.75$ –1.0; at very long ranges (> 10 Mb), γ reflects compartmentalisation.

Loop Extrusion Model

The **loop extrusion model** is the current leading framework for explaining TAD formation and loop establishment:

1. A **cohesin ring complex** (SMC1/SMC3/RAD21/SA1–2) loads onto chromatin and begins **actively extruding a loop** by translocating along the chromatin fibre in both directions.
2. As cohesin moves, the chromatin on either side of the loading site is progressively reeled through the ring, forming an expanding loop.
3. When a cohesin subunit encounters a **CTCF protein bound in convergent orientation** (the CTCF motifs at the two anchors point toward each other: $\rightarrow \cdots \leftarrow$), loop extrusion is **blocked**. This stalling creates a stable loop with CTCF-bound anchors.
4. Loops formed by convergent CTCF sites correspond to the bright “dots” seen at the corners of TADs in Hi-C contact maps.
5. The loop extrusion process dynamically maintains TAD structures and is essential for proper enhancer–promoter regulation by confining regulatory contacts within domains.

The Loop Extrusion Paradigm

Loop extrusion by cohesin, blocked by convergent CTCF sites, explains three key features of 3D genome organisation simultaneously: (1) TAD boundaries coincide with convergent CTCF sites, (2) TAD structure is lost upon cohesin degradation (but restored upon cohesin recovery), and (3) CTCF site deletions or orientation inversions alter loop structure in predictable ways. This model has been validated by acute protein degradation experiments (auxin-inducible degron systems for RAD21 and CTCF), polymer simulations, and CRISPR-mediated CTCF site editing.

16.8.6 Snakemake Workflow

Listing 16.1: Snakemake workflow for Hi-C analysis.

```

1 # Snakefile for Hi-C analysis
2 configfile: "config.yaml"
3
4 SAMPLES = config["samples"]
5 GENOME = config["genome_fasta"]
6 GENOME_INDEX = config["genome_bwa_index"]
7 CHROMSIZES = config["chromsizes"]
8 RESOLUTION = config["resolution"] # e.g., 10000
9
10 rule all:
11     input:
12         expand("results/cool/{sample}.mcool", sample=SAMPLES),
13         expand("results/compartments/{sample}_compartments.bed",
14             sample=SAMPLES),
15         expand("results/tads/{sample}_tads.bed",
16             sample=SAMPLES),
17         expand("results/loops/{sample}_loops.bedpe",
18             sample=SAMPLES),
19         expand("results/plots/{sample}_contact_map.png",
20             sample=SAMPLES),
21         "results/multiqc/multiqc_report.html"
```

```

22
23 rule align_hic:
24     input:
25         r1 = "raw_data/{sample}_R1.fastq.gz",
26         r2 = "raw_data/{sample}_R2.fastq.gz"
27     output:
28         pairs = "results/pairs/{sample}.pairs.gz",
29         stats = "results/pairs/{sample}.stats"
30     threads: 16
31     resources:
32         mem_mb = 64000
33     conda: "envs/hic.yaml"
34     shell:
35         """
36         # Align with BWA-MEM, parse to pairs format
37         # using pairtools for chimeric read handling
38         bwa mem -t {threads} -SP5M {GENOME_INDEX} \
39             {input.r1} {input.r2} \
40             | pairtools parse --min-mapq 30 \
41                 --chroms-path {CHROMSIZES} \
42                 --walks-policy 5unique \
43                 --output-stats {output.stats} \
44             | pairtools sort --nproc {threads} \
45             | pairtools dedup \
46                 --max-mismatch 3 \
47                 --mark-dups \
48                 --output - \
49             | pairtools select '(chrom1 != "chrM") and \
50                 (chrom2 != "chrM")' \
51             | pairtools split \
52                 --output-pairs {output.pairs}
53         """
54
55 rule create_cool:
56     input:
57         pairs = "results/pairs/{sample}.pairs.gz"
58     output:
59         cool = "results/cool/{sample}.cool"
60     params:
61         resolution = RESOLUTION
62     conda: "envs/hic.yaml"
63     shell:
64         """
65         cooler cload pairs \
66             -c1 2 -p1 3 -c2 4 -p2 5 \
67             --assembly hg38 \
68             {CHROMSIZES}:{params.resolution} \
69             {input.pairs} {output.cool}
70         """
71
72 rule balance_and_zoomify:
73     input:
74         cool = "results/cool/{sample}.cool"
75     output:
76         mcool = "results/cool/{sample}.mcool"
77     conda: "envs/hic.yaml"
78     shell:
79         """

```



```

80     cooler balance {input.cool} --force
81     cooler zoomify {input.cool} \
82         --resolutions 1000,5000,10000,25000,50000, \
83     100000,250000,500000,1000000 \
84         --balance \
85         --out {output.mcool}
86     """
87
88 rule call_compartments:
89     input:
90         mcool = "results/cool/{sample}.mcool"
91     output:
92         compartments = "results/compartments/{sample}_compartments.bed",
93         eigenvector = "results/compartments/{sample}_eigenvector.bw"
94     params:
95         resolution = 100000
96     conda: "envs/hic.yaml"
97     shell:
98         """
99         cooltools eigs-cis \
100             {input.mcool}::resolutions/{params.resolution} \
101             --n-eigs 3 \
102             -o results/compartments/{wildcards.sample}
103
104         # Convert eigenvector to BED for A/B assignment
105         python scripts/eigenvector_to_compartments.py \
106             results/compartments/{wildcards.sample}.cis.vecs.tsv \
107             {output.compartments}
108         """
109
110 rule call_tads:
111     input:
112         mcool = "results/cool/{sample}.mcool"
113     output:
114         tads = "results/tads/{sample}_tads.bed",
115         insulation = "results/tads/{sample}_insulation.bw"
116     params:
117         resolution = 25000,
118         window = 500000
119     conda: "envs/hic.yaml"
120     shell:
121         """
122         cooltools insulation \
123             {input.mcool}::resolutions/{params.resolution} \
124             --window-pixels {params.window} \
125             -o results/tads/{wildcards.sample}_insulation.tsv
126
127         python scripts/insulation_to_tads.py \
128             results/tads/{wildcards.sample}_insulation.tsv \
129             {output.tads}
130         """
131
132 rule call_loops:
133     input:
134         mcool = "results/cool/{sample}.mcool"
135     output:
136         loops = "results/loops/{sample}_loops.bedpe"
137     params:

```

```

138     resolution = 10000
139     conda: "envs/hic.yaml"
140     shell:
141         """
142         mustache -f {input.mcool} \
143             -r {params.resolution} \
144             -o {output.loops} \
145             -pt 0.05 -st 0.88
146         """
147
148 rule plot_contact_map:
149     input:
150         mcool = "results/cool/{sample}.mcool"
151     output:
152         plot = "results/plots/{sample}_contact_map.png"
153     params:
154         resolution = 25000,
155         region = config.get("plot_region", "chr1:50000000-55000000")
156     conda: "envs/hic.yaml"
157     shell:
158         """
159         hicPlotMatrix --matrix {input.mcool} \
160             --region {params.region} \
161             --log1p \
162             --colorMap RdYlBu_r \
163             --outFileName {output.plot} \
164             --dpi 300
165         """
166
167 rule multiqc:
168     input:
169         expand("results/pairs/{sample}.stats", sample=SAMPLES)
170     output:
171         "results/multiqc/multiqc_report.html"
172     conda: "envs/multiqc.yaml"
173     shell:
174         """
175         multiqc results/ -o results/multiqc --force
176         """

```

16.8.7 Nextflow Workflow

Listing 16.2: Nextflow DSL2 workflow for Hi-C analysis.

```

1  #!/usr/bin/env nextflow
2  nextflow.enable.dsl=2
3
4  params.reads      = "raw_data/*_R{1,2}.fastq.gz"
5  params.genome     = "reference/genome.fa"
6  params.bwa_index  = "reference/genome"
7  params.chromsizes = "reference/chromsizes.txt"
8  params.resolution = 10000
9  params.outdir     = "results"
10 params.plot_region = "chr1:50000000-55000000"
11
12 process ALIGN_AND_PARSE {
13     tag "$sample_id"

```

```

14     cpus 16
15     memory '64 GB'
16     publishDir "${params.outdir}/pairs", mode: 'copy'
17
18     input:
19     tuple val(sample_id), path(reads)
20
21     output:
22     tuple val(sample_id),
23         path("${sample_id}.pairs.gz"),    emit: pairs
24     path "${sample_id}.stats",            emit: stats
25
26     script:
27     """
28     bwa mem -t ${task.cpus} -SP5M \
29         ${params.bwa_index} \
30         ${reads[0]} ${reads[1]} \
31     | pairtools parse --min-mapq 30 \
32         --chroms-path ${params.chromsizes} \
33         --walks-policy 5unique \
34         --output-stats ${sample_id}.stats \
35     | pairtools sort --nproc ${task.cpus} \
36     | pairtools dedup --max-mismatch 3 \
37         --mark-dups --output - \
38     | pairtools select \
39         '(chrom1 != "chrM") and (chrom2 != "chrM")' \
40     | pairtools split \
41         --output-pairs ${sample_id}.pairs.gz
42     """
43 }
44
45 process CREATE_COOLER {
46     tag "$sample_id"
47     publishDir "${params.outdir}/cool", mode: 'copy'
48
49     input:
50     tuple val(sample_id), path(pairs)
51
52     output:
53     tuple val(sample_id),
54         path("${sample_id}.mcool"),    emit: mcool
55
56     script:
57     """
58     cooler cload pairs \
59         -c1 2 -p1 3 -c2 4 -p2 5 \
60         --assembly hg38 \
61         ${params.chromsizes}:${params.resolution} \
62         ${pairs} ${sample_id}.cool
63
64     cooler balance ${sample_id}.cool --force
65
66     cooler zoomify ${sample_id}.cool \
67         --resolutions 1000,5000,10000,25000,50000,\
68     100000,250000,500000,1000000 \
69         --balance \
70         --out ${sample_id}.mcool
71     """

```

```

72 }
73
74 process CALL_COMPARTMENTS {
75     tag "$sample_id"
76     publishDir "${params.outdir}/compartments",
77         mode: 'copy'
78
79     input:
80     tuple val(sample_id), path(mcool)
81
82     output:
83     tuple val(sample_id),
84         path("${sample_id}_compartments.bed"),
85                                     emit: compartments
86     path "${sample_id}*",          emit: all_files
87
88     script:
89     """
90     cooltools eigs-cis \
91         ${mcool}::resolutions/100000 \
92         --n-eigs 3 \
93         -o ${sample_id}
94
95     python ${projectDir}/scripts/eigenvector_to_compartments.py \
96         ${sample_id}.cis.vecs.tsv \
97         ${sample_id}_compartments.bed
98     """
99 }
100
101 process CALL_TADS {
102     tag "$sample_id"
103     publishDir "${params.outdir}/tads", mode: 'copy'
104
105     input:
106     tuple val(sample_id), path(mcool)
107
108     output:
109     tuple val(sample_id),
110         path("${sample_id}_tads.bed"),    emit: tads
111
112     script:
113     """
114     cooltools insulation \
115         ${mcool}::resolutions/25000 \
116         --window-pixels 500000 \
117         -o ${sample_id}_insulation.tsv
118
119     python ${projectDir}/scripts/insulation_to_tads.py \
120         ${sample_id}_insulation.tsv \
121         ${sample_id}_tads.bed
122     """
123 }
124
125 process CALL_LOOPS {
126     tag "$sample_id"
127     publishDir "${params.outdir}/loops", mode: 'copy'
128
129     input:

```

```

130     tuple val(sample_id), path(mcool)
131
132     output:
133     tuple val(sample_id),
134         path("${sample_id}_loops.bedpe"),
135         emit: loops
136
137     script:
138     """
139     mustache -f ${mcool} \
140         -r ${params.resolution} \
141         -o ${sample_id}_loops.bedpe \
142         -pt 0.05 -st 0.88
143     """
144 }
145
146 process PLOT_CONTACT_MAP {
147     tag "$sample_id"
148     publishDir "${params.outdir}/plots", mode: 'copy'
149
150     input:
151     tuple val(sample_id), path(mcool)
152
153     output:
154     path "${sample_id}_contact_map.png"
155
156     script:
157     """
158     hicPlotMatrix --matrix ${mcool} \
159         --region ${params.plot_region} \
160         --log1p --colorMap RdYlBu_r \
161         --outFileName ${sample_id}_contact_map.png \
162         --dpi 300
163     """
164 }
165
166 // Main workflow
167 workflow {
168     reads_ch = Channel
169         .fromFilePairs(params.reads,
170             checkIfExists: true)
171
172     ALIGN_AND_PARSE(reads_ch)
173     CREATE_COOLER(ALIGN_AND_PARSE.out.pairs)
174     CALL_COMPARTMENTS(CREATE_COOLER.out.mcool)
175     CALL_TADS(CREATE_COOLER.out.mcool)
176     CALL_LOOPS(CREATE_COOLER.out.mcool)
177     PLOT_CONTACT_MAP(CREATE_COOLER.out.mcool)
178 }

```

Practical Tip

Several established Hi-C processing pipelines are available and well-maintained:

- **Juicer:** Developed by the Aiden lab. Produces `.hic` format files. Includes Arrowhead (TADs), HICUPS (loops), and compartment analysis.
- **HiC-Pro:** Standalone pipeline that handles alignment, filtering, and contact matrix generation. Outputs can be converted to `.cool` or `.hic` format.
- **pairtools + cooler + cooltools:** The “Open2C” ecosystem. A modular, Python-based toolkit that represents the modern standard for Hi-C data processing.
- **nf-core/hic:** A Nextflow pipeline for Hi-C data that handles alignment through feature calling.

For most new projects, the pairtools/cooler/cooltools ecosystem is recommended due to its flexibility, active development, and integration with the Python scientific computing stack.

16.8.8 Visualisation

- **HiGlass:** Interactive, web-based viewer for multi-resolution contact maps. Supports `.mcool` and `.hic` formats. Allows side-by-side comparison of multiple samples and overlay of 1D genomic tracks (genes, ChIP-seq, ATAC-seq). The most versatile viewer for exploration.
- **Juicebox:** Desktop application (Java) from the Aiden lab. Directly reads `.hic` files. Includes built-in annotations for loops, TADs, and compartments. User-friendly for browsing specific regions.
- **HiCExplorer:** Command-line tools (`hicPlotMatrix`, `hicPlotTADs`) for generating publication-quality figures. Supports plotting contact maps with overlaid TAD boundaries and 1D tracks (gene expression, histone marks, etc.).
- **cooltools:** Python library for programmatic analysis and plotting. Integrates with matplotlib and the broader Python data science ecosystem.
- **Cooler/HiGlass ecosystem:** The `.mcool` format (multi-resolution cooler) allows seamless zooming from chromosome-scale to kilobase-scale views.

16.8.9 Integration with Other Epigenomic Data

The greatest biological insight comes from integrating 3D genome data with other data types:

- **Hi-C + ATAC-seq:** Identify which accessible regulatory elements are in physical contact, linking enhancers to their target promoters.
- **Hi-C + H3K27ac ChIP/CUT&Tag:** Distinguish active enhancer–promoter loops from structural loops.
- **Hi-C + RNA-seq:** Correlate expression changes with changes in 3D genome topology.
- **Hi-C + GWAS data:** Map disease-associated non-coding variants to their target genes via 3D contacts. A variant in an enhancer may loop to a distant promoter, and Hi-C/HiChIP can identify that connection.

- **Hi-C + DNA methylation:** Examine the relationship between chromatin topology and the DNA methylation landscape.

Linking Non-Coding Variants to Target Genes

Genome-wide association studies (GWAS) identify genetic variants associated with disease risk, but >90% of these variants are in non-coding regions. Connecting a non-coding variant to its target gene requires evidence that the variant's region physically contacts a gene promoter. Hi-C, HiChIP (especially H3K27ac HiChIP), and Promoter Capture Hi-C provide this evidence by mapping the 3D contacts between variant-containing enhancers and gene promoters. This approach has been transformative for interpreting GWAS results and identifying therapeutic targets.

16.9 Experimental Design Considerations

Choosing a 3D Genome Method

- **Genome-wide, all-vs-all contacts:** Hi-C or Micro-C.
- **Active enhancer–promoter loops:** H3K27ac HiChIP or PLAC-seq.
- **Specific loci of interest:** Capture Hi-C or 4C-seq (one-vs-all from a single viewpoint).
- **Multi-way contacts:** Pore-C, SPRITE, or GAM.
- **Nucleosome-level resolution:** Micro-C.
- **Genome assembly scaffolding (dual purpose):** Hi-C with Arima or Omni-C kit.

Key design parameters:

1. **Cell numbers:** Hi-C typically requires 1–5 million cells. HiChIP requires 500K–5M cells depending on the antibody and mark abundance. Low-input Hi-C protocols can work with fewer cells but sacrifice resolution.
2. **Replicates:** At least 2 biological replicates, which can be pooled to increase sequencing depth. Use HiCRep (stratum-adjusted correlation coefficient) to assess reproducibility between replicates.
3. **Sequencing depth:** See [Table 16.1](#). For TAD-level analysis, 300–500 million valid pairs is a reasonable target. For loop calling, aim for ≥ 1 billion.
4. **Restriction enzyme:** 4-cutter enzymes (DpnII, MboI) provide higher resolution than 6-cutters (HindIII) because they cut more frequently. Multi-enzyme cocktails (Arima kit) provide the finest fragmentation.
5. **Crosslinking:** Standard formaldehyde crosslinking (1%, 10 min). Dual crosslinking (formaldehyde + EGS/DSG) can improve capture of long-range interactions.

Important Caveat

Hi-C and related methods are among the most technically demanding and expensive sequencing-based assays. A single human Hi-C library at loop-level resolution requires ~1 billion valid read pairs, which costs thousands of dollars in sequencing alone. Before embarking on a Hi-C experiment, carefully consider whether a more targeted approach (HiChIP, Capture Hi-C, or even mining existing publicly available Hi-C datasets from your cell type of interest) would answer your biological question at lower cost.

16.10 Chapter Summary

The three-dimensional organisation of the genome is a critical layer of gene regulation. Chromosomes are organised hierarchically into territories, A/B compartments, topologically associating domains (TADs), and chromatin loops, with CTCF and cohesin playing central roles in loop formation through the loop extrusion mechanism. Hi-C and its variants (Micro-C, HiChIP, Capture Hi-C, Pore-C) provide genome-wide maps of chromatin contacts at resolutions ranging from megabases to nucleosomes, depending on sequencing depth and the specific method used. The bioinformatics pipeline—alignment with chimeric read handling, contact matrix generation and normalisation, and feature calling for compartments, TADs, and loops—requires specialised tools (pairtools, cooler, cooltools, or Juicer) distinct from standard genomics pipelines. Integration of 3D genome data with chromatin accessibility, histone modifications, DNA methylation, and gene expression data provides the most complete picture of how the epigenome controls cell identity, and is essential for interpreting the functional impact of non-coding genetic variants identified by genome-wide association studies.

Part VI

Multionics and Emerging Frontiers

Chapter 17

Single-Cell and Spatial Multiomics

Key Concept

Multiomics refers to the simultaneous measurement of two or more molecular modalities—such as the transcriptome, epigenome, proteome, or genome—from the same individual cell or spatial location. By capturing multiple layers of biological information together, multiomics technologies resolve cellular states with far greater precision than any single assay alone, enabling the reconstruction of gene regulatory networks, lineage hierarchies, and tissue architectures that would otherwise remain hidden.

17.1 The Promise of Multiomics

Every cell in a multicellular organism contains the same genome, yet cells adopt spectacularly different identities. This diversity arises from the interplay of multiple regulatory layers: which genes are transcribed (transcriptome), which genomic regions are accessible (epigenome), which proteins are displayed on the cell surface (proteome), and how these layers interact in space and time. Traditional single-cell technologies measure only one of these layers at a time. When different modalities are profiled in separate experiments on separate cells, computational integration is required—and the results are inherently inferential rather than directly observed.

Multiomics technologies eliminate this inference gap by measuring two, three, or even more modalities from the *same individual cell*. This direct co-measurement has several transformative advantages:

- **Direct linkage:** One can directly correlate, for example, the accessibility of a chromatin peak with the expression of its target gene in the same cell, rather than relying on statistical association across different cell populations.
- **Regulatory network inference:** By observing which transcription factor binding sites are open and which genes are expressed in the same cell, one can reconstruct gene regulatory networks with far greater confidence.
- **Cell-type resolution:** Surface protein markers (measured by CITE-seq) can provide “ground truth” cell-type labels that are more robust than transcriptome-only clustering, particularly for closely related cell states.
- **Lineage and trajectory analysis:** Joint measurement of genetic barcodes (or somatic mutations) alongside transcriptomic state enables simultaneous tracking of cell identity and cell history.

Important Caveat

Multiomics experiments are substantially more expensive per cell than single-modality assays, and the data generated are more complex to analyse. In many cases, a well-designed single-modality experiment with appropriate computational integration will answer the biological question more cost-effectively. Multiomics is most valuable when direct co-measurement is scientifically essential—for example, linking specific regulatory elements to target genes or resolving ambiguous cell states.

17.2 Joint RNA + Chromatin Accessibility

The joint profiling of gene expression and chromatin accessibility within the same cell represents one of the most mature and widely adopted multiomics modalities. By measuring both the transcriptome (what is being expressed) and the accessible chromatin landscape (what *could* be expressed), these assays provide a direct window into the regulatory logic of individual cells.

17.2.1 10x Genomics Multiome (ATAC + GEX)

The 10x Genomics Multiome kit is currently the most widely used commercial platform for joint single-nucleus ATAC-seq and gene expression profiling. The assay works on *nuclei* rather than whole cells, which is important for understanding its characteristics.

10x Multiome ATAC + GEX Protocol Overview

1. **Nuclear isolation:** Cells are lysed and nuclei are isolated. This step is critical—overly harsh lysis destroys chromatin structure, while insufficient lysis leaves cytoplasmic RNA behind.
2. **Transposition:** Nuclei are incubated with Tn5 transposase loaded with sequencing adapters. Tn5 inserts adapters preferentially into accessible (open) chromatin regions.
3. **GEM generation:** Transposed nuclei are loaded onto the Chromium controller, where each nucleus is captured in a gel bead-in-emulsion (GEM) droplet with a uniquely barcoded gel bead.
4. **Barcoding:** Within each GEM, the nuclear RNA is reverse-transcribed and barcoded, while the transposed ATAC fragments are simultaneously barcoded with the same cell barcode.
5. **Library construction:** After breaking the emulsion, ATAC and GEX libraries are separated and independently amplified for sequencing.
6. **Sequencing:** Both libraries are sequenced on Illumina instruments. The shared cell barcode links ATAC and GEX data from the same nucleus.

Typical throughput is 500–10,000 nuclei per experiment, with the recommended sequencing depth of approximately 25,000 read pairs per nucleus for ATAC and 20,000 read pairs per nucleus for GEX.

17.2.2 SHARE-seq, SNARE-seq, and Paired-seq

Before the 10x Multiome became commercially available, several academic methods established the feasibility of joint RNA + ATAC profiling:

SHARE-seq (Simultaneous High-throughput ATAC and RNA Expression with Sequencing)

Developed by the Bhatt and Bhatt laboratories, SHARE-seq uses a split-pool combinatorial barcoding strategy rather than droplet-based encapsulation. This allows profiling of many more cells (up to hundreds of thousands) without requiring a microfluidics instrument. SHARE-seq was notably used to discover “domain of regulatory chromatin” (DORC) scores, linking accessible chromatin regions to gene expression.

SNARE-seq (Single-Nucleus Chromatin Accessibility and mRNA Expression Sequencing)

An earlier droplet-based method that captures both modalities from single nuclei. SNARE-seq2 improved throughput significantly using a modified barcoding strategy.

Paired-seq

Another combinatorial indexing approach that scales to hundreds of thousands of cells, with the advantage of being instrument-free (no microfluidics device required).

17.2.3 Analysis: Linking Peaks to Genes

The central analytical goal of joint RNA + ATAC data is to link cis-regulatory elements (ATAC peaks) to their target genes. Several strategies are used:

1. **Correlation-based linking:** For each peak–gene pair within a defined genomic window (e.g., 250 kb of the gene’s TSS), compute the correlation between peak accessibility and gene expression across cells. Statistically significant positive correlations suggest a regulatory relationship.
2. **Transcription factor motif enrichment:** Identify transcription factor (TF) binding motifs enriched in linked peaks, then correlate TF expression with both peak accessibility and target gene expression to build a regulatory circuit.
3. **Gene regulatory network (GRN) inference:** Tools such as SCENIC+ extend the SCENIC framework to jointly model TF expression, TF motif accessibility, and target gene expression, producing comprehensive GRN models.

Vertical vs. Diagonal Integration

When RNA and ATAC data come from the *same cells* (as in 10x Multiome), this is called **vertical integration**—one simply links the modalities via the shared cell barcode. When the two modalities are measured in *different cells* (e.g., separate scRNA-seq and scATAC-seq experiments), computational **diagonal integration** is needed, using methods such as those in Seurat v5, ArchR, or MultiVI. Vertical integration is always preferable when feasible because it avoids the assumptions inherent in cross-modality alignment.

17.2.4 Peak-to-Gene Correlation Framework

The correlation-based linking of ATAC peaks to target genes is the foundational analysis for joint RNA + chromatin data. The statistical framework operates as follows. For a given gene g and a candidate cis-regulatory peak p located within a genomic distance window $d_{\max}$ (typically 250–500 kb) of the gene’s transcription start site, one computes the correlation between chromatin accessibility and gene expression across all cells:

$$r_{pg} = \text{cor}(\mathbf{a}_p, \mathbf{x}_g), \quad (17.1)$$

where $\mathbf{a}_p = (a_{p1}, a_{p2}, \dots, a_{pN})$ is the vector of accessibility values for peak p across N cells, and $\mathbf{x}_g = (x_{g1}, x_{g2}, \dots, x_{gN})$ is the corresponding expression vector for gene g . Either Pearson or Spearman correlation may be used; Spearman is more robust to the zero-inflated nature of single-cell data.

Significance Testing by Permutation

Because the large number of peak–gene pairs tested creates a severe multiple testing burden, permutation-based significance is preferred over parametric p -values. For each peak–gene pair, the gene expression vector is permuted B times (typically $B = 1,000$ – $10,000$), and the correlation is recomputed for each permutation. The empirical p -value is:

$$p_{\text{perm}} = \frac{1 + \sum_{b=1}^B \mathbb{I}(|r_{pg}^{(b)}| \geq |r_{pg}|)}{1 + B}, \quad (17.2)$$

where $r_{pg}^{(b)}$ is the correlation from the b -th permutation. False discovery rate (FDR) correction (Benjamini–Hochberg) is applied across all tested pairs. Only peak–gene links with positive correlation and $\text{FDR} < 0.05$ are typically retained, as enhancers are expected to activate (not repress) their target genes.

The resulting set of significant peak–gene links provides a map of putative enhancer–gene regulatory relationships. A useful aggregate metric is the **DORC score** (Domain of Regulatory Chromatin), introduced with SHARE-seq. For each gene, the DORC score sums the accessibility of all significantly linked peaks:

$$\text{DORC}_g(i) = \sum_{p \in \mathcal{L}(g)} a_{pi}, \quad (17.3)$$

where $\mathcal{L}(g)$ is the set of peaks significantly linked to gene g and a_{pi} is the accessibility of peak p in cell i . Genes with high DORC scores are regulated by large cis-regulatory domains encompassing many accessible elements; these tend to be key cell-identity genes such as master transcription factors and lineage-defining markers. The DORC score provides a chromatin-derived proxy for gene regulatory potential that complements expression-based measures.

These links can be further validated by:

- Overlap with known enhancer databases (e.g., ENCODE cCREs, VISTA enhancer browser).
- Enrichment for transcription factor binding motifs consistent with the target gene’s regulatory logic.
- Consistency with Hi-C or promoter-capture Hi-C contact data, confirming physical proximity between the peak and the gene’s promoter.
- Perturbation experiments (e.g., CRISPRi targeting of individual peaks) to confirm functional regulatory activity.

Important Caveat

Correlation-based peak–gene linking has important caveats. First, the distance window $d_{\max}$ is arbitrary; some enhancers act over distances exceeding 1 Mb, while using large windows increases false positives. Second, the correlations are computed across cell types, so they primarily capture regulatory relationships that vary across cell identities rather than within a cell type. Third, negative correlations (suggesting repressive elements such as silencers) are typically discarded but may represent genuine biology. Fourth, the sparsity of single-cell ATAC data (most peaks are detected in only a small fraction of cells) limits statistical power, requiring large cell numbers ($>5,000$) for robust link detection.

17.2.5 Tools for Joint RNA + ATAC Analysis**ArchR**

A comprehensive R package for single-cell ATAC-seq analysis that natively supports multiome data. Provides peak calling, motif enrichment, peak-to-gene linking, trajectory analysis, and integration with scRNA-seq. Highly memory-efficient through the use of Arrow files.

Signac

An extension of the Seurat framework for chromatin data analysis. Provides functions for quality control, dimensionality reduction, motif analysis, and integration with gene expression data from the same or different cells.

MOFA+ (Multi-Omics Factor Analysis)

A Bayesian factor analysis method that identifies shared and modality-specific sources of variation across multiple data types. Particularly useful for exploratory analysis of multiomics datasets.

MultiVI

Part of the `scvi-tools` suite, MultiVI is a deep generative model that can jointly analyse RNA and ATAC data, handling both paired (multiome) and unpaired (separate experiments) data within a unified framework.

17.2.6 Variational Autoencoders for Multiomics: scVI, MultiVI, and the ELBO

Several of the most effective tools for single-cell and multiomics analysis—`scVI`, `TotalVI`, `MultiVI`, and `Cobolt`—are built on the **variational autoencoder** (VAE) framework. Understanding the underlying mathematics is essential for interpreting their outputs and choosing appropriate hyperparameters.

A VAE consists of two neural networks:

1. An **encoder** (inference network) $q_{\phi}(\mathbf{z}|\mathbf{x})$ that maps the observed data $\mathbf{x}$ (e.g., a cell's gene expression vector) to a distribution over a low-dimensional latent space $\mathbf{z}$.
2. A **decoder** (generative network) $p_{\theta}(\mathbf{x}|\mathbf{z})$ that reconstructs the observed data from a sample in the latent space.

The model is trained by maximizing the **Evidence Lower Bound (ELBO)** on the log-marginal likelihood of the data:

$$\text{ELBO}(\theta, \phi; \mathbf{x}) = \underbrace{\mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|\mathbf{z})]}_{\text{Reconstruction term}} - \underbrace{\text{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))}_{\text{Regularization term}}, \quad (17.4)$$

where $p(\mathbf{z})$ is the prior distribution over the latent space (typically a standard normal $\mathcal{N}(\mathbf{0}, \mathbf{I})$). The reconstruction term encourages the model to faithfully reproduce the input data, while the KL divergence regularizes the latent space to remain close to the prior, preventing overfitting and ensuring a smooth, interpretable latent representation.

scVI: Modelling Count Data with ZINB Likelihood

scVI (single-cell Variational Inference) tailors the VAE framework to the specific statistical properties of scRNA-seq data. Rather than using a Gaussian reconstruction loss, scVI models gene expression counts with a **zero-inflated negative binomial** (ZINB) distribution:

$$p(x_g|\mathbf{z}, s) = \pi_g \delta_0(x_g) + (1 - \pi_g) \text{NB}(x_g; \mu_g, \theta_g), \quad (17.5)$$

where x_g is the observed count for gene g , π_g is the zero-inflation probability (modelling technical dropouts), μ_g is the mean expression (a function of $\mathbf{z}$ and the library size s), and θ_g is the gene-specific dispersion parameter. The decoder network outputs π_g , μ_g , and θ_g for each gene. Batch effects are handled by conditioning the decoder on a batch indicator variable, enabling integrated analysis across experiments without explicit batch correction.

MultiVI extends the scVI framework to jointly model RNA and ATAC data. The key innovations are:

- A shared latent space $\mathbf{z}$ that captures biological variation common to both modalities.
- Modality-specific decoder heads: a ZINB decoder for RNA counts and a Bernoulli decoder for binarized ATAC peaks.
- The ability to handle **unpaired data**: cells with only RNA, cells with only ATAC, and cells with both modalities are all incorporated into a single model. Missing modalities are treated as latent variables and imputed by the decoder.
- The ELBO for MultiVI sums the reconstruction terms for both modalities:

$$\text{ELBO}_{\text{MultiVI}} = \mathbb{E}_{q(\mathbf{z}|\mathbf{x}_{\text{RNA}}, \mathbf{x}_{\text{ATAC}})} [\log p(\mathbf{x}_{\text{RNA}}|\mathbf{z}) + \log p(\mathbf{x}_{\text{ATAC}}|\mathbf{z})] - \text{KL}(q(\mathbf{z}|\mathbf{x})\|p(\mathbf{z})). \quad (17.6)$$

Practical Considerations for VAE-Based Models

When using scVI or MultiVI, several hyperparameters require attention:

- **Latent dimension:** Typically 10–30. Larger values capture more variation but risk overfitting with small datasets.
- **Number of hidden layers and neurons:** Default architectures (1–2 hidden layers, 128–256 neurons) work well for most datasets up to ~500,000 cells.
- **KL annealing:** Gradually increasing the weight of the KL term during training (“warm-up”) prevents posterior collapse, where the encoder ignores the data and collapses to the prior.
- **Training epochs:** Monitor the ELBO on a held-out validation set; training for 200–400 epochs is typical. Early stopping prevents overfitting.

17.2.7 Gene Regulatory Network Inference with SCENIC+

Gene regulatory network (GRN) inference from multiomics data aims to reconstruct the regulatory logic connecting transcription factors (TFs), their binding sites in accessible chromatin, and their target genes. The SCENIC+ pipeline is the most comprehensive approach for joint RNA + ATAC data and operates in three stages:

1. **TF motif enrichment in accessible regions:** For each cell cluster or cell type, SCENIC+ identifies ATAC peaks enriched for known TF binding motifs using tools such as `cisTarget`. This produces a set of candidate TF–region associations: TF t is linked to peak p if the motif for t is significantly enriched in p .
2. **TF–target gene regression (GRNBoost2):** GRNBoost2 uses gradient-boosted tree regression to model the expression of each gene as a function of TF expression levels across cells:

$$x_g = f_g(x_{t_1}, x_{t_2}, \dots, x_{t_K}) + \epsilon_g, \quad (17.7)$$

where x_g is the expression of target gene g , x_{t_k} is the expression of the k -th transcription factor, and f_g is a non-linear function learned by the ensemble of gradient-boosted trees. The importance score for each TF–gene pair is derived from the feature importance of the tree ensemble. GRNBoost2 is preferred over alternatives such as GENIE3 because of its substantially faster runtime on large single-cell datasets.

3. **Filtering by motif support:** TF–gene links from the regression step are retained only if the TF’s binding motif is found in an accessible peak linked to the target gene (from step 1). This intersection of expression-based and motif-based evidence yields high-confidence **regulons**—sets of target genes regulated by a common TF, supported by both co-expression and cis-regulatory motif evidence.

The final output of SCENIC+ is a GRN consisting of TFs, their regulons, and the associated cis-regulatory regions, providing a multi-layered view of gene regulation in each cell type. For each TF t and its regulon $\mathcal{R}_t$, SCENIC+ computes a **regulon activity score** (RAS) for each cell i :

$$\text{RAS}_t(i) = \frac{1}{|\mathcal{R}_t|} \sum_{g \in \mathcal{R}_t} \tilde{x}_{gi}, \quad (17.8)$$

where $\tilde{x}_{gi}$ is the normalized, scaled expression of target gene g in cell i , and $|\mathcal{R}_t|$ is the size of the regulon. More sophisticated scoring methods, such as the AUCell algorithm, rank genes by expression within each cell and compute the area under the recovery curve for regulon genes, providing a score that is robust to differences in sequencing depth and drop-out rates. The regulon activity matrix (TFs $\times$ cells) can be used for clustering, trajectory analysis, and identification of master regulators for each cell state.

Important Caveat

GRN inference from observational single-cell data is inherently correlative, not causal. Co-expression of a TF with a target gene and the presence of a TF motif in a nearby accessible peak are consistent with—but do not prove—a direct regulatory relationship. Validation through perturbation experiments (e.g., CRISPRi knockdown of the TF followed by scRNA-seq) is essential before drawing strong conclusions about regulatory causality. Additionally, GRNBoost2 and similar methods cannot distinguish direct from indirect regulation; a TF may appear to regulate a gene because both are downstream of a common upstream regulator.

Table 17.1: Comparison of gene regulatory network inference methods applicable to single-cell multiomics data.

Method	Approach	Uses ATAC	Scalability	Output
SCENIC+	GRNBoost2 re-gression + cis-Target motif filtering	Yes	High (>100K cells)	TF regulons with enhancer links
CellOracle	Bayesian re-gression with TF binding site priors from ATAC	Yes	Moderate	Directed GRN with perturbation simulation
Dictys	Context-specific GRN using joint RNA + ATAC with dynamic modelling	Yes	Moderate	Dynamic, context-specific GRN
GENIE3	Random forest re-gression of TF-gene relationships	No	Low-Moderate	Undirected TF-gene importance scores
GRaNIE	Integrates peak-gene correlations	Yes	High 348	TF-peak-gene triplets

17.3 Joint RNA + Protein (Surface Markers)

17.3.1 CITE-seq: Antibody-Derived Tags + Transcriptome

CITE-seq (Cellular Indexing of Transcriptomes and Epitopes by Sequencing) was developed by the Bhatt and Smibert laboratories and published in 2017. It enables simultaneous measurement of surface protein expression and the transcriptome from the same single cell.

CITE-seq Principle

1. Cells are stained with a panel of antibodies, each conjugated to a unique DNA oligonucleotide called an **antibody-derived tag (ADT)**.
2. Stained cells are loaded onto a droplet-based single-cell platform (e.g., 10x Chromium).
3. Within each droplet, both mRNA and ADT oligos are captured by the barcoded bead, reverse-transcribed, and barcoded with the same cell barcode.
4. After library preparation, RNA and ADT libraries are sequenced separately, then linked by cell barcode.
5. The number of ADT reads per cell is proportional to the abundance of the corresponding surface protein.

CITE-seq has become enormously popular because it provides protein-level phenotyping (similar to flow cytometry) alongside whole-transcriptome profiling. Large antibody panels of 200+ markers are now routinely used with commercial panels from BioLegend (TotalSeq).

17.3.2 REAP-seq and ASAP-seq

REAP-seq (RNA Expression and Protein Sequencing) is conceptually similar to CITE-seq but uses a different antibody–oligo conjugation chemistry developed by the Zhang laboratory at the New York Genome Center. **ASAP-seq** (ATAC with Select Antigen Profiling by Sequencing) extends the antibody-tagging concept to ATAC-seq, enabling joint measurement of chromatin accessibility and surface protein expression—a unique combination not achievable with CITE-seq alone.

17.3.3 TotalVI for Joint RNA + Protein Analysis

TotalVI (Total Variational Inference) is a deep generative model from the `scvi-tools` suite specifically designed for joint analysis of CITE-seq data. It models both RNA counts and protein (ADT) counts within a unified probabilistic framework, accounting for the distinct noise characteristics of each modality.

Key capabilities of TotalVI include:

- Joint dimensionality reduction that leverages both RNA and protein information.
- Denoising of protein counts, correcting for background antibody binding.
- Batch correction across experiments with different antibody panels.
- Differential expression analysis for both RNA and protein simultaneously.

17.3.4 Antibody Panel Design and Protein Normalization

Designing a CITE-seq antibody panel requires careful consideration:

- **Panel composition:** Include markers that distinguish the cell types of interest. For immune profiling, this typically includes lineage markers (CD3, CD4, CD8, CD19, CD56, CD14) and activation/differentiation markers.
- **Isotype controls:** Include isotype control antibodies (matched to each antibody species and isotype but without target specificity) to estimate non-specific background binding.
- **Titration:** Each antibody should be titrated to determine the optimal concentration that maximises signal-to-noise ratio. Over-concentrated antibodies increase background binding.

Protein Normalization Methods

Raw ADT counts require normalization to account for technical variation:

CLR (Centered Log-Ratio)

Divides each protein count by the geometric mean of all protein counts for that cell (or across cells), then applies a log transformation. Simple and widely used but can be biased by the panel composition.

DSB (Denoised and Scaled by Background)

Uses the isotype control antibodies and empty droplets to model and subtract non-specific background, then scales the signal. Generally more accurate than CLR but requires isotype controls and identifiable empty droplets.

TotalVI normalization

Learned denoised protein values from the generative model, which implicitly accounts for background binding and technical noise.

17.4 Joint RNA + DNA

17.4.1 G&T-seq: Genome and Transcriptome from the Same Cell

G&T-seq physically separates genomic DNA from mRNA within the same single cell using oligo-dT-coated magnetic beads. The mRNA (captured by its poly-A tail) is processed for scRNA-seq, while the remaining genomic DNA is processed for single-cell whole-genome amplification (WGA) and sequencing. This allows direct correlation of copy number alterations, point mutations, or structural variants with transcriptomic state in individual cells—critical for understanding tumour heterogeneity.

17.4.2 scDNA-seq + scRNA-seq Integration

When G&T-seq is not feasible at scale, separate scDNA-seq and scRNA-seq experiments can be computationally integrated. Copy number profiles from scDNA-seq (e.g., using 10x Genomics CNV or DLP+) can be matched to scRNA-seq clusters using inferred CNV signals from expression data (e.g., from *inferCNV* or *CopyKAT*). While not a true multiomics co-measurement, this approach can link genotype to phenotype across thousands of cells.

17.4.3 Lineage Tracing with CRISPR Barcodes

CRISPR-based lineage tracing introduces heritable, evolving barcodes into cells. As cells divide, Cas9-induced mutations accumulate in a barcode locus, creating a unique mutational history that records the cell's lineage. When combined with scRNA-seq, one can simultaneously determine each cell's transcriptomic state and its position in the lineage tree. Key methods include:

GESTALT (Genome Editing of Synthetic Target Arrays for Lineage Tracing)

Introduces an array of CRISPR target sites; progressive editing creates combinatorial barcodes.

CARLIN (CRISPR Array Repair Lineage tracing)

An inducible version that allows temporal control of barcode editing.

intMEMOIR

Uses integrase-based barcoding for irreversible binary recording in individual cells.

17.5 Triple-Omics and Beyond

The field has rapidly moved beyond dual-modality measurements to triple and even higher-order multiomics:

17.5.1 TEA-seq and DOGMA-seq

TEA-seq (Transcription, Epitopes, and Accessibility) and **DOGMA-seq** (an acronym referring to the central dogma) both achieve simultaneous measurement of three modalities—RNA, surface protein (ADT), and chromatin accessibility (ATAC)—from the same single cell. These represent the current state of the art in single-cell multiomics, enabling researchers to link chromatin state, gene expression, and protein output within individual cells.

The key technical challenge is performing Tn5 transposition (for ATAC) on nuclei that have already been stained with antibodies (for protein), while preserving mRNA for capture. Both TEA-seq and DOGMA-seq solve this with carefully optimized lysis and transposition buffers.

17.5.2 scNMT-seq: Nucleosome, Methylation, and Transcription

scNMT-seq (single-cell Nucleosome, Methylation, and Transcription sequencing) profiles three epigenomic and transcriptomic layers from the same cell:

1. **DNA methylation** via bisulfite sequencing of GpC methyltransferase-treated DNA.
2. **Nucleosome positioning / chromatin accessibility** using the same GpC methyltransferase (NOMe-seq principle: GpC methylation marks accessible chromatin, while endogenous CpG methylation is read separately).
3. **Gene expression** via Smart-seq2 on physically separated mRNA.

This triple assay is particularly powerful for studying the interplay between DNA methylation, chromatin accessibility, and transcription during dynamic processes such as embryonic development.

17.6 Spatial Multiomics

While single-cell dissociation-based methods provide exquisite molecular resolution, they destroy tissue architecture. Spatial multiomics technologies retain spatial information while measuring multiple molecular layers.

17.6.1 Spatial Chromatin Accessibility

Spatial-ATAC and **spatial-CUT&Tag** adapt chromatin profiling to tissue sections by performing transposition (ATAC) or targeted tagmentation (CUT&Tag) in situ, followed by spatially barcoded capture. These methods enable mapping of epigenomic landscapes across tissue architecture, revealing how chromatin states vary spatially in organs and tumours.

17.6.2 Spatial Proteomics

Several technologies measure protein expression with spatial resolution:

CODEX / PhenoCycler (Akoya Biosciences)

Uses iterative cycles of fluorescent oligonucleotide-labelled antibody hybridization and imaging. Each cycle reveals a subset of markers; 40–100+ protein markers can be measured per tissue section at single-cell resolution.

Imaging Mass Cytometry (IMC; Standard BioTools)

Stains tissue with metal-conjugated antibodies, then ablates the tissue with a laser, detecting the metal isotopes by mass spectrometry. Achieves ~40 markers per section with subcellular resolution (~1 μm pixel size).

MIBI (Multiplexed Ion Beam Imaging)

Similar to IMC but uses an ion beam for ablation, achieving higher spatial resolution. Both IMC and MIBI are destructive—the tissue is consumed during imaging.

17.6.3 10x Visium + Immunofluorescence and DBiT-seq

10x Visium can be combined with immunofluorescence (IF) staining on the same tissue section. The tissue is first stained with fluorescent antibodies and imaged, then permeabilized for mRNA capture. This provides joint spatial transcriptomics and protein expression, albeit at the Visium spot resolution (~55 μm).

DBiT-seq (Deterministic Barcoding in Tissue for spatial multi-omics) uses microfluidic channels to deliver spatial barcodes to a tissue section, enabling joint spatial profiling of mRNA and proteins (via antibody-derived tags) at 10–50 μm resolution. This approach achieves higher spatial resolution than Visium while measuring both modalities simultaneously.

17.7 Computational Integration Strategies

The integration of multiomics data—whether from the same cells or different experiments—is a central computational challenge. Three integration paradigms are recognized:

17.7.1 Vertical Integration (Same Cells, Multiple Modalities)

When multiple modalities are measured from the same cells (e.g., 10x Multiome, CITE-seq, TEA-seq), the data are linked by shared cell barcodes. The computational challenge is to combine these modalities into a unified representation that captures the information from all layers. Methods include:

- **WNN (Weighted Nearest Neighbor):** Implemented in Seurat v4/v5, WNN constructs a joint nearest-neighbor graph by weighting each modality according to its informativeness for each cell. Cells where one modality is noisy rely more on the other modality. The mathematical framework is described in detail below.
- **MOFA+:** Bayesian factor analysis that decomposes multiomics data into latent factors, identifying shared variation (factors active in multiple modalities) and modality-specific variation.
- **MultiVI / TotalVI:** Deep generative models that learn a joint latent space from paired multiomics data.
- **Cobolt:** A variational autoencoder designed for multiome data that handles the distinct statistical properties of RNA (count data) and ATAC (binary accessibility) data.

17.7.2 Weighted Nearest Neighbour (WNN) Mathematics

The **Weighted Nearest Neighbour** (WNN) algorithm, introduced in Seurat v4, provides a principled framework for constructing a joint cell-cell similarity graph from two or more modalities, where the contribution of each modality is determined independently for each cell. This cell-specific weighting is the key innovation: rather than assigning a single global weight to each modality, WNN allows a cell whose RNA data are high quality but whose ATAC data are noisy to rely predominantly on RNA for neighbourhood construction, and vice versa.

The algorithm proceeds as follows. For each cell i and each modality $m \in \{\text{RNA}, \text{ATAC}\}$:

1. **Within-modality nearest neighbour graphs:** Compute a k -nearest-neighbour (KNN) graph G_m in the reduced-dimensional space of modality m (e.g., PCA space for RNA, LSI space for ATAC). Let $\mathcal{N}_m(i)$ denote the set of k nearest neighbours of cell i in modality m .
2. **Cross-modality prediction:** For each cell i , use its neighbours $\mathcal{N}_m(i)$ in modality m to predict the cell's representation in the *other* modality. For example, using RNA neighbours to predict the cell's ATAC profile. The prediction accuracy (measured by the similarity between the predicted and observed cross-modality profile) quantifies how informative modality m is for cell i .
3. **Cell-specific modality weights:** The weight $w_m(i)$ assigned to modality m for cell i is computed from the cross-modality prediction accuracy. Specifically, let $s_m(i)$ denote the similarity between the predicted and observed cross-modality profile for cell i using modality m . The weights are obtained by a softmax transformation:

$$w_m(i) = \frac{\exp(s_m(i)/\tau)}{\sum_{m'} \exp(s_{m'}(i)/\tau)}, \quad (17.9)$$

where τ is a temperature parameter controlling the sharpness of the weighting. This ensures $w_{\text{RNA}}(i) + w_{\text{ATAC}}(i) = 1$ for all cells.

4. **Joint weighted graph:** The final WNN graph combines the modality-specific graphs using the cell-specific weights. For any two cells i and j , the joint edge weight in the WNN graph is:

$$W_{\text{WNN}}(i, j) = w_{\text{RNA}}(i) \cdot W_{\text{RNA}}(i, j) + w_{\text{ATAC}}(i) \cdot W_{\text{ATAC}}(i, j), \quad (17.10)$$

where $W_m(i, j)$ is the edge weight between cells i and j in the KNN graph for modality m (zero if $j \notin \mathcal{N}_m(i)$).

The resulting WNN graph can be used for all downstream analyses that operate on cell-cell graphs: UMAP visualization, Louvain/Leiden clustering, pseudotime trajectory inference, and differential abundance testing. Because the modality weights are cell-specific, the WNN graph naturally adapts to heterogeneous data quality across cells—a common situation in multiomics experiments where some cells have sparse ATAC coverage or low RNA capture efficiency.

Inspecting WNN Modality Weights

After running WNN in Seurat, it is informative to visualize the modality weights $w_{\text{RNA}}(i)$ and $w_{\text{ATAC}}(i)$ on the UMAP embedding. Cell types that are primarily distinguished by transcriptomic features (e.g., secretory vs. absorptive epithelial cells) will have higher RNA weights, while cell types distinguished by chromatin state (e.g., progenitor cells with bivalent promoters) may have higher ATAC weights. Extreme imbalance in weights across all cells may indicate that one modality has poor data quality globally.

17.7.3 Horizontal Integration (Different Cells, Same Modality)

This is the standard batch correction problem: aligning datasets from different experiments, donors, or conditions that measure the same modality. Methods include Harmony, Scanorama, scVI, and the RPCA/CCA approaches in Seurat.

17.7.4 Diagonal Integration (Different Cells, Different Modalities)

The most challenging scenario: integrating datasets where different cells were measured with different modalities (e.g., one experiment did scRNA-seq, another did scATAC-seq on a different sample). This requires identifying shared biological signals across fundamentally different data types. Approaches include:

- **Gene activity scores:** Convert ATAC data into a “gene activity” matrix (summing accessibility near gene bodies and promoters) to create a shared feature space with RNA data.
- **Canonical correlation analysis (CCA):** Find correlated dimensions between the two modalities.
- **Optimal transport:** Methods like GLUE (Graph-Linked Unified Embedding) use optimal transport to align cells across modalities.
- **MultiVI:** Can handle both paired and unpaired data in a single model, making it suitable for diagonal integration.

Choosing an Integration Strategy

The choice of integration strategy depends on the experimental design. If multiple modalities were measured from the same cells (vertical integration), this always provides stronger evidence than diagonal integration because the cell-level pairing removes ambiguity. Diagonal integration should be used only when co-measurement is not available, and results should be interpreted with appropriate caution regarding the assumptions of the alignment method.

17.7.5 Benchmarking: scIB and Beyond

The **scIB** (single-cell Integration Benchmarking) framework provides standardized metrics for evaluating integration methods, including:

- **Batch correction metrics:** kBET, batch-ASW, graph connectivity—measuring how well batch effects are removed.
- **Biological conservation metrics:** cell-type ASW, NMI, ARI—measuring how well biological structure is preserved.
- An overall score balancing batch correction and biological conservation.

17.8 Lineage Tracing Technologies

Understanding the clonal relationships between cells—which cells descended from which progenitor—is fundamental to developmental biology, stem cell research, and cancer biology. Lineage tracing technologies record cellular history alongside molecular state.

17.8.1 CRISPR-Based Lineage Barcoding

CRISPR-based approaches introduce evolving barcodes that accumulate mutations over cell divisions:

GESTALT

An array of CRISPR target sites is edited by Cas9, creating combinatorial deletion patterns. The barcode is read by bulk or single-cell sequencing.

CARLIN

An inducible system where barcode editing can be triggered at specific developmental timepoints by administering doxycycline. This adds temporal control to lineage recording.

intMEMOIR

Uses site-specific integrases (rather than CRISPR) to create irreversible binary switches at multiple genomic loci, creating a digital barcode.

17.8.2 Natural Lineage Markers

Mitochondrial mutations (MAESTER)

Somatic mutations in mitochondrial DNA (mtDNA) accumulate naturally during cell division. The **MAESTER** (Mitochondrial Alteration Enrichment with Single-cell Tiling and Sequencing to Evaluate Relationships) method enriches for mtDNA variants alongside scRNA-seq, using the naturally occurring mtDNA mutation pattern as a lineage barcode. The advantage is that no genetic engineering is required, making this applicable to primary human samples.

Somatic nuclear mutations

Nuclear somatic mutations (SNVs, indels, CNVs) also serve as natural lineage markers. Methods like **Phylovar** and **SCarches** use somatic variants detected from scRNA-seq or scDNA-seq data to reconstruct clonal phylogenies.

17.8.3 Computational Tools for Lineage Reconstruction

Cassiopeia

A scalable framework for CRISPR-barcode-based lineage tracing that implements multiple tree-reconstruction algorithms (maximum parsimony, neighbor joining, greedy heuristics) and provides benchmarking tools for comparing reconstruction accuracy.

CellTagging

A lentiviral barcoding system combined with scRNA-seq for multi-timepoint lineage tracing.

17.9 Perturbation-seq Approaches

Perturbation-seq technologies combine genetic perturbations (typically CRISPR-based) with single-cell readouts to systematically map gene function at scale.

17.9.1 Perturb-seq and CROP-seq

Perturb-seq and **CROP-seq** (CRISPR droplet sequencing) both transduce cells with a pooled CRISPR guide RNA (sgRNA) library, so that each cell receives (ideally) a single sgRNA targeting a single gene. The sgRNA identity is read alongside the transcriptome in a standard scRNA-seq workflow.

Perturb-seq / CROP-seq Workflow

1. Design a library of sgRNAs targeting genes of interest (typically 50–5,000 genes, with 3–5 sgRNAs per gene for redundancy).
2. Transduce cells with the sgRNA library at low multiplicity of infection ($\text{MOI} \approx 0.1\text{--}0.3$) to ensure most cells receive only one sgRNA.
3. Allow time for gene knockout/knockdown to take effect (typically 48–96 hours for CRISPRko, shorter for CRISPRi).
4. Perform scRNA-seq (e.g., 10x Chromium) with the sgRNA sequence captured alongside the transcriptome.
5. Associate each cell with its perturbation identity and compare gene expression profiles across perturbations.

A landmark 2022 study by the Bhatt laboratory performed Perturb-seq at genome scale, targeting all $\sim 20,000$ protein-coding genes in human cells and profiling over 2.5 million cells, creating the most comprehensive map of gene function to date.

17.9.2 ECCITE-seq: CRISPR + CITE-seq

ECCITE-seq (Expanded CRISPR-compatible CITE-seq) combines CRISPR perturbation identity, surface protein measurement (CITE-seq), and transcriptome profiling from the same cell. This triple readout enables researchers to observe how genetic perturbations affect both transcriptomic and proteomic phenotypes simultaneously.

17.9.3 Analysis of Perturbation Screens

Key analytical approaches for perturbation screens include:

- **Differential expression per perturbation:** Comparing cells with each sgRNA to non-targeting control cells to identify genes whose expression changes upon perturbation.
- **Perturbation signatures:** Computing a “perturbation embedding” for each gene knockout, then clustering perturbations to identify genes with similar loss-of-function phenotypes (implying shared pathways).
- **Mixscape** (Seurat): A method for classifying cells into “perturbed” and “escaped” groups based on transcriptomic response, accounting for the fact that not all sgRNAs successfully perturb their target.
- **Compositional analysis:** Identifying how perturbations shift the relative abundance of cell states in a heterogeneous population.

17.10 Multiomics Workflow Example

The following workflow examples illustrate the processing of 10x Multiome (ATAC + GEX) data, from raw FASTQ files through to integrated analysis.

17.10.1 Snakemake Workflow

Listing 17.1: Snakemake workflow for 10x Multiome processing.

```

1 # Snakefile for 10x Multiome (ATAC + GEX) Analysis
2 configfile: "config/multiome_config.yaml"
3
4 SAMPLES = config["samples"]
5
6 rule all:
7     input:
8         expand("results/{sample}/cellranger_arc/outs/"
9               "web_summary.html", sample=SAMPLES),
10        expand("results/{sample}/signac/"
11              "multiome_integrated.rds", sample=SAMPLES),
12        expand("results/{sample}/multivi/"
13              "multivi_model/", sample=SAMPLES),
14        "results/combined/peak_gene_links.csv"
15
16 rule cellranger_arc:
17     """Run Cell Ranger ARC for joint ATAC+GEX demux."""
18     input:
19         gex_fastqs="data/{sample}/gex/",
20         atac_fastqs="data/{sample}/atac/",
21         reference=config["cellranger_arc_ref"]
22     output:
23         summary="results/{sample}/cellranger_arc/outs/"
24               "web_summary.html",
25         h5="results/{sample}/cellranger_arc/outs/"
26           "filtered_feature_bc_matrix.h5"
27     params:
28         sample_id="{sample}",
29         outdir="results/{sample}/cellranger_arc"
30     threads: 16
31     resources:
32         mem_mb=64000,
33         time="24:00:00"
34     shell:
35         """
36         cellranger-arc count \
37             --id={params.sample_id} \
38             --reference={input.reference} \
39             --libraries=config/libraries_{wildcards.sample}.csv \
40             --localcores={threads} \
41             --localmem=60
42         mv {params.sample_id} {params.outdir}
43         """
44
45 rule qc_and_filter:
46     """Quality control and filtering of multiome data."""
47     input:
48         h5="results/{sample}/cellranger_arc/outs/"
49         "filtered_feature_bc_matrix.h5",
50         fragments="results/{sample}/cellranger_arc/outs/"
51         "atac_fragments.tsv.gz"
52     output:
53         rds="results/{sample}/signac/"
54         "filtered_multiome.rds",
55         qc_plots="results/{sample}/signac/qc_plots.pdf"
56     params:
57         min_genes=200,

```

```

58     max_genes=8000,
59     max_mt_pct=20,
60     min_tss_enrichment=2,
61     min_atac_fragments=1000
62 threads: 4
63 resources:
64     mem_mb=32000
65 script:
66     "scripts/qc_filter_multiome.R"
67
68 rule signac_analysis:
69     """Run Signac/Seurat WNN integration."""
70     input:
71         rds="results/{sample}/signac/"
72             "filtered_multiome.rds"
73     output:
74         rds="results/{sample}/signac/"
75             "multiome_integrated.rds",
76         umap="results/{sample}/signac/wnn_umap.pdf"
77     params:
78         n_dims_rna=30,
79         n_dims_atac=20,
80         resolution=0.8
81     threads: 8
82     resources:
83         mem_mb=48000
84     script:
85         "scripts/signac_wnn_analysis.R"
86
87 rule multivi_analysis:
88     """Run MultiVI for joint latent space."""
89     input:
90         h5="results/{sample}/cellranger_arc/outs/"
91             "filtered_feature_bc_matrix.h5"
92     output:
93         model=directory("results/{sample}/multivi/"
94             "multivi_model/"),
95         latent="results/{sample}/multivi/"
96             "multivi_latent.csv"
97     threads: 4
98     resources:
99         mem_mb=32000,
100         gpu=1
101     conda:
102         "envs/scvi.yaml"
103     script:
104         "scripts/run_multivi.py"
105
106 rule peak_gene_links:
107     """Link ATAC peaks to target genes."""
108     input:
109         rds=expand("results/{sample}/signac/"
110             "multiome_integrated.rds",
111             sample=SAMPLES)
112     output:
113         links="results/combined/peak_gene_links.csv",
114         plot="results/combined/peak_gene_heatmap.pdf"
115     threads: 8

```

```

116 resources:
117     mem_mb=64000
118 script:
119     "scripts/link_peaks_to_genes.R"

```

17.10.2 Nextflow Workflow

Listing 17.2: Nextflow workflow for 10x Multiome processing.

```

1 #!/usr/bin/env nextflow
2  nextflow.enable.dsl=2
3
4  params.samples      = "data/samples.csv"
5  params.reference    = "/ref/cellranger-arc/GRCh38"
6  params.min_genes    = 200
7  params.max_genes    = 8000
8  params.max_mt_pct   = 20
9  params.outdir       = "results"
10
11 // Read sample sheet: sample_id, gex_path, atac_path
12 Channel
13     .fromPath(params.samples)
14     .splitCsv(header: true)
15     .map { row -> tuple(row.sample_id,
16                         file(row.gex_path),
17                         file(row.atac_path)) }
18     .set { sample_ch }
19
20 process CELLRANGER_ARC {
21     tag "$sample_id"
22     cpus 16
23     memory '64 GB'
24     time '24h'
25     publishDir "${params.outdir}/${sample_id}/cellranger_arc",
26               mode: 'copy'
27
28     input:
29     tuple val(sample_id), path(gex_fastqs),
30         path(atac_fastqs)
31
32     output:
33     tuple val(sample_id),
34         path("outs/filtered_feature_bc_matrix.h5"),
35         path("outs/atac_fragments.tsv.gz"),
36         path("outs/atac_fragments.tsv.gz.tbi"),
37         emit: arc_out
38     path("outs/web_summary.html"), emit: summary
39
40     script:
41     """
42     cellranger-arc count \
43         --id=${sample_id} \
44         --reference=${params.reference} \
45         --libraries=libraries.csv \
46         --localcores=${task.cpus} \
47         --localmem=60
48     mv ${sample_id}/outs .

```

```

49     """
50 }
51
52 process QC_AND_FILTER {
53     tag "$sample_id"
54     cpus 4
55     memory '32 GB'
56     publishDir "${params.outdir}/${sample_id}/qc",
57         mode: 'copy'
58     container 'bioconductor/signac:latest'
59
60     input:
61     tuple val(sample_id), path(h5),
62         path(fragments), path(fragments_idx)
63
64     output:
65     tuple val(sample_id),
66         path("filtered_multiome.rds"),
67         emit: filtered
68     path("qc_plots.pdf"), emit: qc_report
69
70     script:
71     """
72     Rscript ${projectDir}/scripts/qc_filter.R \
73         --h5 ${h5} \
74         --fragments ${fragments} \
75         --min_genes ${params.min_genes} \
76         --max_genes ${params.max_genes} \
77         --max_mt_pct ${params.max_mt_pct} \
78         --output filtered_multiome.rds \
79         --plot_output qc_plots.pdf
80     """
81 }
82
83 process WNN_INTEGRATION {
84     tag "$sample_id"
85     cpus 8
86     memory '48 GB'
87     publishDir "${params.outdir}/${sample_id}/integrated",
88         mode: 'copy'
89     container 'bioconductor/signac:latest'
90
91     input:
92     tuple val(sample_id), path(rds)
93
94     output:
95     tuple val(sample_id),
96         path("wnn_integrated.rds"),
97         emit: integrated
98     path("wnn_umap.pdf"), emit: umap_plot
99
100     script:
101     """
102     Rscript ${projectDir}/scripts/wnn_integrate.R \
103         --input ${rds} \
104         --output wnn_integrated.rds \
105         --plot wnn_umap.pdf
106     """

```

```

107 }
108
109 process PEAK_GENE_LINKS {
110   cpus 8
111   memory '64 GB'
112   publishDir "${params.outdir}/combined", mode: 'copy'
113
114   input:
115   path(rds_files)
116
117   output:
118   path("peak_gene_links.csv"), emit: links
119   path("peak_gene_heatmap.pdf"), emit: plot
120
121   script:
122   """
123   Rscript ${projectDir}/scripts/link_peaks.R \
124     --input_dir . \
125     --output peak_gene_links.csv \
126     --plot peak_gene_heatmap.pdf
127   """
128 }
129
130 workflow {
131   CELLRANGER_ARC(sample_ch)
132   QC_AND_FILTER(CELLRANGER_ARC.out.arc_out)
133   WNN_INTEGRATION(QC_AND_FILTER.out.filtered)
134
135   // Collect all integrated RDS files
136   WNN_INTEGRATION.out.integrated
137   .map { sample_id, rds -> rds }
138   .collect()
139   .set { all_rds }
140
141   PEAK_GENE_LINKS(all_rds)
142 }

```

17.11 Multiomics Approach Comparison

17.12 Visualising the Multiomics Landscape

17.13 Summary

This chapter has surveyed the rapidly expanding field of single-cell and spatial multiomics. Key takeaways include:

- **Joint RNA + chromatin:** Technologies such as 10x Multiome and SHARE-seq enable direct linking of regulatory elements to target genes by measuring chromatin accessibility and gene expression in the same cell.
- **Joint RNA + protein:** CITE-seq and related methods provide protein-level cell-type markers alongside whole-transcriptome profiling, bridging the gap between flow cytometry and scRNA-seq.
- **Triple-omics:** TEA-seq and DOGMA-seq achieve simultaneous measurement of RNA, chromatin, and protein, providing the most comprehensive view of single-cell state

Table 17.2: Comparison of major single-cell multiomics technologies. Throughput indicates typical cells per experiment.

Technology	Modalities Measured	Throughput	Platform	Commercial
10x Multiome	RNA + ATAC	500–10K	Droplet	Yes
SHARE-seq	RNA + ATAC	10K–100K+	Split-pool	No
SNARE-seq2	RNA + ATAC	5K–70K	Droplet	No
CITE-seq	RNA + Protein (200+ ADTs)	5K–20K	Droplet	Yes
ASAP-seq	ATAC + Protein	5K–20K	Droplet	No
REAP-seq	RNA + Protein	5K–20K	Droplet	No
TEA-seq	RNA + ATAC + Protein	5K–20K	Droplet	No
DOGMA-seq	RNA + ATAC + Protein	5K–20K	Droplet	No
scNMT-seq	RNA + Methylation + Accessibility	100–1K	Plate	No
G&T-seq	RNA + DNA (WGS)	10–200	Plate	No
Perturb-seq	RNA + CRISPR perturbation	10K–100K+	Droplet	No
ECCITE-seq	RNA + Protein + CRISPR	5K–20K	Droplet	No

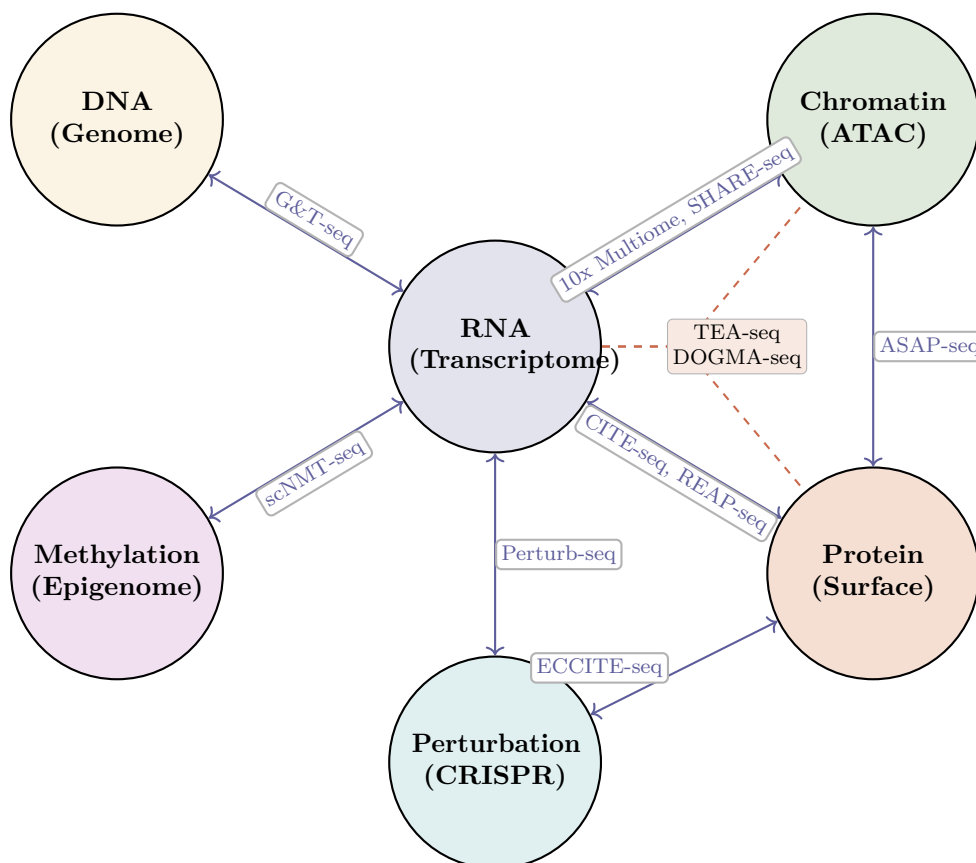


Figure 17.1: Overview of single-cell multiomics technologies and the molecular modalities they connect. Solid blue arrows indicate dual-modality assays; dashed red lines indicate triple-omics assays. Each technology simultaneously measures the connected modalities from the same individual cell.

currently available.

- **Spatial multiomics:** Technologies including spatial-ATAC, CODEX, and DBiT-seq extend multiomics measurements to intact tissues, preserving spatial context.
- **Lineage tracing:** CRISPR barcoding and natural lineage markers (mitochondrial mutations) enable reconstruction of cell lineage trees alongside molecular state.
- **Perturbation screens:** Perturb-seq and CROP-seq map gene function at genome scale by combining CRISPR perturbations with scRNA-seq readout.
- **Computational integration:** The distinction between vertical (same cells), horizontal (same modality), and diagonal (different cells, different modalities) integration is fundamental to choosing appropriate analytical methods.

As multiomics technologies continue to mature, their decreasing cost and increasing throughput will make multi-modal profiling the default rather than the exception in single-cell biology, providing an ever more complete picture of cellular identity and function.

Chapter 18

Specialized Sequencing Applications

Key Concept

Beyond the “core” applications of genome sequencing, transcriptomics, and epigenomics, a growing constellation of specialized sequencing assays addresses specific biological questions that standard methods cannot answer. These include ribosome profiling for measuring translation in real time, CRISPR screens for systematic gene function mapping, liquid biopsy for non-invasive cancer detection, epitranscriptomics for profiling RNA modifications, and adaptive nanopore sequencing for software-defined target enrichment. Each represents a creative repurposing of sequencing technology to read a distinct layer of biology.

18.1 Ribosome Profiling (Ribo-seq)

18.1.1 Principle: Measuring Translation in Real Time

While **RNA-seq** measures which genes are transcribed and at what level, it does not reveal which mRNAs are actively being translated into protein. The correlation between mRNA abundance and protein output is notoriously imperfect—post-transcriptional regulation, mRNA stability, and translational control introduce substantial discrepancies. **Ribosome profiling** (Ribo-seq), developed by Jonathan Weissman and Nicholas Ingolia in 2009, directly measures translation by sequencing the mRNA fragments that are physically protected by actively translating ribosomes.

Ribo-seq Protocol

1. **Translation arrest:** Treat cells with a translation elongation inhibitor (classically cycloheximide, CHX) to freeze ribosomes in place on their mRNA templates. Alternative inhibitors include lactimidomycin (which captures initiating ribosomes) and harringtonine (which also captures initiation).
2. **Cell lysis:** Lyse cells under conditions that preserve ribosome–mRNA complexes.
3. **RNase digestion:** Treat the lysate with RNase I (or micrococcal nuclease). This digests all mRNA *not* physically protected by a ribosome, leaving ~28–30 nucleotide (nt) “footprints” corresponding to the mRNA segment enclosed within the ribosome’s mRNA channel.
4. **Ribosome isolation:** Purify monosome-protected fragments by sucrose gradient centrifugation or size-exclusion chromatography.
5. **Fragment isolation and library preparation:** Extract the ~28–30 nt RNA fragments, ligate adapters, reverse-transcribe, and prepare a sequencing library (similar to small RNA library prep).
6. **Sequencing:** Single-end sequencing on Illumina (typically SE50 or SE75) yields the identity and position of each ribosome footprint.
7. **Parallel RNA-seq:** A matched total RNA or mRNA-seq library is prepared from the same sample to enable calculation of translational efficiency.

18.1.2 P-site Mapping and Translational Efficiency

After sequencing, Ribo-seq reads are mapped to the transcriptome. The key analytical concepts are:

P-site assignment

Each ribosome footprint corresponds to a specific codon being decoded. The “P-site offset” (typically 12–13 nt from the 5’ end of the footprint for 28-mers) maps each read to the precise codon in the ribosome’s peptidyl (P) site. This provides **codon-level resolution** of translation.

Three-nucleotide periodicity

Because the ribosome advances three nucleotides (one codon) per step, Ribo-seq data show a characteristic 3-nt periodicity when reads are mapped at single-nucleotide resolution. This periodicity is a hallmark of genuine translation signal and is used as a quality metric.

Translational efficiency (TE)

Defined as the ratio of ribosome footprint density (RPF) to mRNA abundance:

$$\text{TE}_{\text{gene}} = \frac{\text{RPF}_{\text{gene}}/\text{RPF}_{\text{total}}}{\text{mRNA}_{\text{gene}}/\text{mRNA}_{\text{total}}}$$

Genes with $\text{TE} > 1$ are translated more efficiently than average; genes with $\text{TE} < 1$ are translationally repressed.

18.1.3 P-site Offset Determination and Metagene Analysis

Accurate assignment of ribosome-protected fragments (RPFs) to the codon being decoded requires careful determination of the **P-site offset**—the distance from the 5’ end of the RPF

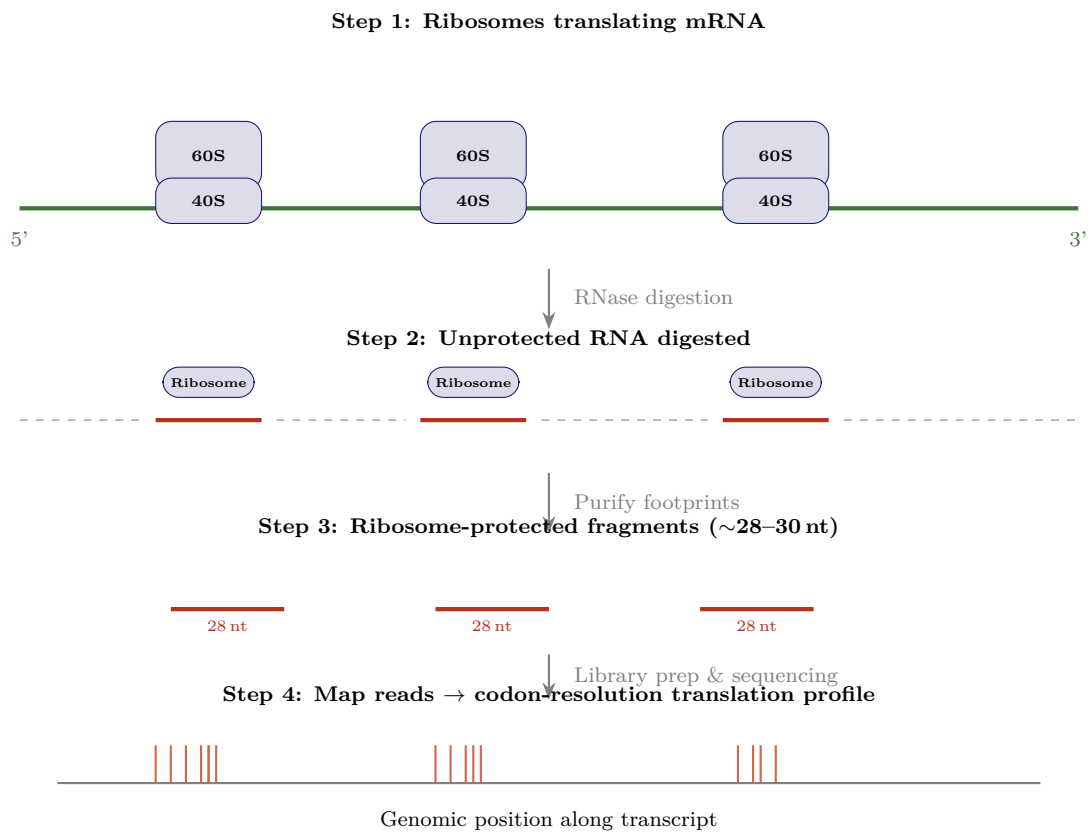


Figure 18.1: Ribosome profiling (Ribo-seq) captures the ~28–30 nucleotide mRNA fragments protected by actively translating ribosomes after RNase digestion, providing a snapshot of translation at codon-level resolution.

to the codon positioned in the ribosome's peptidyl-transfer (P) site. This offset is not a single fixed value but varies systematically with fragment length.

Fragment-Length-Dependent P-site Offsets

The standard RPF length distribution from eukaryotic ribosomes centres on 28–30 nt, with minor populations at 20–22 nt (corresponding to ribosomes in a rotated state). The P-site offset for each fragment length is determined empirically by metagene analysis:

- For **28-mers**: the P-site offset is typically 12 nt from the 5' end.
- For **29-mers**: the offset is typically 12 nt.
- For **30-mers**: the offset is typically 13 nt.
- For **shorter fragments** (20–22 nt): the offset is typically 9–10 nt.

These values are organism- and protocol-specific and must be calibrated for each experiment.

The calibration procedure uses **metagene analysis around annotated start codons**. For a given fragment length L , all RPFs of length L whose 5' end maps within a window of ± 50 nt around annotated AUG start codons are aggregated. The P-site offset δ_L is the displacement from the 5' end to the AUG position that produces the sharpest peak. Formally:

$$\delta_L = \operatorname{argmax}_d \sum_{g \in \mathcal{G}} \text{count}_L(\text{TSS}_g - d), \quad (18.1)$$

where $\mathcal{G}$ is the set of annotated coding genes, TSS_g is the position of the AUG start codon for gene g , and $\text{count}_L(p)$ is the number of RPFs of length L with their 5' end at position p . The offset δ_L that maximizes this signal corresponds to the correct P-site assignment for fragments of length L .

A complementary analysis uses **stop codons**: when the P-site offset is correctly assigned, the metagene profile around stop codons should show a sharp drop in RPF density immediately after the stop codon (as ribosomes terminate translation and dissociate). The presence of clear three-nucleotide periodicity throughout the coding sequence, with the majority of P-site-assigned reads falling in the first reading frame (frame 0), serves as the primary quality metric:

$$F_0 = \frac{\text{RPFs in frame 0}}{\text{RPFs in frame 0} + \text{RPFs in frame 1} + \text{RPFs in frame 2}}. \quad (18.2)$$

High-quality Ribo-seq datasets typically achieve $F_0 > 0.7$, with the best datasets exceeding 0.9. Tools such as **RiboWaltz** and **Plastid** automate this calibration process, producing fragment-length-specific offset tables and periodicity plots.

Important Caveat

The choice of translation inhibitor affects the fragment length distribution and P-site offset. Cycloheximide (CHX) pre-treatment produces a broader footprint distribution and can introduce artefacts near start codons due to ribosome run-off during the drug treatment period. Flash-freezing protocols (no drug pre-treatment) or the use of initiation-specific inhibitors (lactimidomycin, harringtonine) produce different footprint profiles. Always calibrate P-site offsets independently for each experimental condition rather than using default values from the literature.

18.1.4 Tools for Ribo-seq Analysis

RiboCode

A Python tool for de novo identification of translated open reading frames (ORFs) based on the 3-nt periodicity signal. Can detect canonical ORFs, upstream ORFs (uORFs), downstream ORFs, and ORFs in long non-coding RNAs.

ORFquant

An R package for quantifying translation at the ORF level, distinguishing between multiple ORFs on the same transcript.

Ribotricker

Uses a Bayesian changepoint model to detect actively translated ORFs, with particularly good performance on noisy datasets.

RiboWaltz

Optimizes P-site offset assignment and provides quality metrics for Ribo-seq data.

18.1.5 Discovering Novel Translated ORFs

One of the most exciting applications of Ribo-seq is the discovery of previously unannotated translated regions:

- **Upstream ORFs (uORFs):** Short ORFs in the 5'UTR that are translated before the main coding sequence. uORFs regulate translation of the downstream CDS and are often responsive to stress conditions.
- **Small ORFs (smORFs):** ORFs encoding micropeptides shorter than 100 amino acids, which were historically overlooked by gene annotation pipelines. Ribo-seq has revealed thousands of actively translated smORFs.
- **Alternative reading frames:** Ribo-seq can detect ribosomes translating in unexpected reading frames, including frameshifts.
- **Non-coding RNA translation:** Some annotated “long non-coding RNAs” (lncRNAs) show Ribo-seq signal, suggesting they encode short peptides.

Important Caveat

Ribo-seq data interpretation requires caution. Cycloheximide pre-treatment can cause run-off of some ribosomes before arrest, introducing artefacts near translation initiation and termination sites. Flash-freezing cells without drug treatment (“flash-freeze” Ribo-seq) can mitigate this but may introduce other biases. Additionally, the RNase digestion step does not always produce perfectly uniform footprints, and contaminating non-ribosomal RNA fragments (e.g., from RNA-binding protein complexes) can create false signals.

18.2 CRISPR Screens

18.2.1 Pooled CRISPR Knockout Screens

Pooled CRISPR screens enable systematic loss-of-function studies at genome scale. The principle is to introduce a library of guide RNAs (sgRNAs) targeting every gene in the genome, apply selective pressure, and identify which gene knockouts confer a fitness advantage or disadvantage.

Pooled CRISPR Knockout Screen Workflow

1. **Library design:** Select a genome-wide sgRNA library (e.g., Brunello library for human, Brie for mouse, each targeting ~20,000 genes with 4 sgRNAs per gene plus ~1,000 non-targeting controls).
2. **Library production:** Clone the sgRNA library into a lentiviral backbone containing a selection marker (e.g., puromycin resistance).
3. **Cell transduction:** Transduce Cas9-expressing cells at low MOI (~0.3) to ensure ~90% of transduced cells carry only one sgRNA. Aim for $\geq 500\times$ representation (at least 500 cells per sgRNA).
4. **Selection:** Apply the selective condition of interest. For a drug resistance screen, treat with the drug; for a fitness screen, simply passage cells for multiple population doublings.
5. **Genomic DNA extraction and sgRNA amplification:** Extract gDNA from the surviving cell population and a reference (early timepoint or untreated) population. PCR-amplify the sgRNA cassette from gDNA.
6. **Sequencing:** Sequence the sgRNA amplicons on Illumina (typically SE75 or PE75). Only ~20 million reads are needed for a genome-wide library.
7. **Analysis:** Count sgRNA reads and compare guide representation between treated and control samples.

18.2.2 CRISPRi and CRISPRa Screens

CRISPRi (CRISPR interference)

Uses a catalytically dead Cas9 (dCas9) fused to a transcriptional repressor domain (KRAB). The sgRNA directs dCas9 to the target gene's promoter, where KRAB silences transcription without cutting DNA. Advantages over CRISPRko: no DNA damage response, titratable repression, reversible.

CRISPRa (CRISPR activation)

Uses dCas9 fused to transcriptional activator domains (e.g., VP64, p65, Rta in the "VPR" system, or the synergistic activation mediator SAM system). This enables gain-of-function screens that complement loss-of-function approaches. CRISPRa guides target promoter regions to upregulate endogenous gene expression.

18.2.3 Guide RNA Library Design

Effective sgRNA design is critical for screen success. Key considerations include:

- **On-target efficiency:** Algorithms such as Rule Set 2 (Doench et al.) and DeepCas9 predict sgRNA cutting efficiency based on sequence features.
- **Off-target effects:** Tools such as CRISPRscan, Cas-OFFinder, and GuideScan rank guides by predicted specificity.
- **Target location:** For CRISPRko, guides targeting early constitutive exons are most effective. For CRISPRi/a, guides near the transcription start site (TSS) are optimal (−50 to +300 bp for CRISPRi; −400 to −50 bp for CRISPRa).
- **Controls:** Include non-targeting controls (guides with no match in the genome), safe-harbour targeting controls (guides targeting inert genomic loci), and positive controls (guides targeting known essential or drug-target genes).

18.2.4 Analysis Tools for CRISPR Screens

MAGeCK (Model-based Analysis of Genome-wide CRISPR-Cas9 Knockout)

The most widely used tool. MAGeCK uses a negative binomial model to identify sgRNAs with significantly altered abundance, then aggregates sgRNA-level results to gene-level scores using robust rank aggregation (RRA) or maximum likelihood estimation (MLE). MAGeCK-VISPR provides a visualization platform.

BAGEL2 (Bayesian Analysis of Gene Essentiality with Location)

Calculates a Bayes Factor for each gene, representing the likelihood that the gene is essential vs. non-essential, using reference sets of known essential and non-essential genes.

CRISPRcleanR

Corrects for copy-number-dependent sgRNA depletion bias that can confound screen results in aneuploid cell lines (common in cancer cells).

18.2.5 MAGeCK Statistical Framework

MAGeCK (Model-based Analysis of Genome-wide CRISPR-Cas9 Knockout) is the most widely used statistical framework for analysing pooled CRISPR screen data. Its two-stage approach—sgRNA-level modelling followed by gene-level aggregation—provides robust identification of screen hits even in the presence of variable sgRNA efficiency and noisy count data.

sgRNA-Level Negative Binomial Model

For each sgRNA j , MAGeCK models the read count Y_{jc} in condition c (treatment or control) using a **negative binomial** (NB) distribution:

$$Y_{jc} \sim \text{NB}(\mu_{jc}, \alpha_j), \quad (18.3)$$

where μ_{jc} is the expected count for sgRNA j in condition c , and α_j is the sgRNA-specific dispersion parameter. The mean μ_{jc} is modelled as:

$$\mu_{jc} = s_c \cdot \mu_{j0} \cdot 2^{\beta_j}, \quad (18.4)$$

where s_c is a sample-specific size factor (estimated by median-ratio normalization), μ_{j0} is the baseline mean count for sgRNA j , and β_j is the $\log_2$ fold change for sgRNA j between treatment and control. The negative binomial model accounts for both Poisson sampling noise and biological overdispersion—the latter arising from stochastic variation in lentiviral integration, cell growth, and PCR amplification.

The dispersion parameter α_j is estimated using a mean-variance relationship: MAGeCK fits a smooth curve $\hat{\alpha}(\mu)$ relating the variance to the mean across all sgRNAs, then uses this curve as an empirical Bayes prior to regularize per-sgRNA dispersion estimates. This borrowing of information across sgRNAs stabilizes estimation for low-count guides.

A p -value for each sgRNA is obtained by testing $H_0 : \beta_j = 0$ against the appropriate one-sided alternative (depletion or enrichment), using the NB likelihood ratio test or Wald test.

Gene-Level Aggregation via Robust Rank Aggregation

Because each gene is targeted by multiple sgRNAs (typically 4–10), MAGeCK aggregates sgRNA-level evidence to the gene level using **Robust Rank Aggregation** (RRA). For each gene g targeted by sgRNAs $\{j_1, j_2, \dots, j_{n_g}\}$:

1. Rank all sgRNAs in the library by their p -values (from the NB test).

2. Extract the normalized ranks $r_{j_1}, r_{j_2}, \dots, r_{j_{n_g}}$ for the sgRNAs targeting gene g , where each rank is divided by the total number of sgRNAs to obtain a value in $[0, 1]$.
3. Under the null hypothesis (gene g has no effect), these normalized ranks are uniformly distributed on $[0, 1]$. The RRA score for gene g is based on the minimum-order-statistic p -value:

$$\rho_g = \min_{k=1, \dots, n_g} \beta_{\min}(r_{g(k)}; k, n_g - k + 1), \quad (18.5)$$

where $r_{g(k)}$ is the k -th smallest normalized rank among the sgRNAs targeting gene g , and $\beta_{\min}(\cdot; a, b)$ is the CDF of the k -th order statistic from a $\text{Uniform}(0, 1)$ sample, which follows a $\text{Beta}(k, n_g - k + 1)$ distribution.

4. The RRA score ρ_g represents the probability of observing ranks as extreme as those of gene g 's sgRNAs by chance. Lower scores indicate stronger evidence that the gene is a screen hit.

Why RRA Is Robust

The power of RRA lies in its robustness to individual sgRNA failure. A gene is called significant even if one or two of its sgRNAs show no effect (due to poor cutting efficiency or off-target integration), as long as the remaining sgRNAs show consistent enrichment or depletion. This is critical because sgRNA efficiency varies substantially, and requiring all guides to show an effect would dramatically reduce sensitivity. The α -RRA variant (used by default in MAGeCK) further improves robustness by only considering the top α fraction of sgRNAs per gene, discarding obvious non-functional guides.

MAGeCK also provides a **maximum likelihood estimation** (MLE) module for more complex experimental designs (e.g., multiple conditions, time series, paired samples), which jointly estimates gene-level effect sizes within a generalized linear model framework. The MLE module models the mean count for sgRNA j in sample c as:

$$\log \mu_{jc} = \log s_c + \beta_0 + \beta_{g(j)} \cdot d_c, \quad (18.6)$$

where β_0 is the baseline, $\beta_{g(j)}$ is the gene-level effect size for the gene targeted by sgRNA j , and d_c is the design matrix entry for sample c . This formulation allows multi-factor designs including drug treatment, time course, and interaction effects.

Table 18.1: Comparison of statistical frameworks for pooled CRISPR screen analysis.

Tool	Statistical Model	Multi-condition	Copy-number correction
MAGeCK-RRA	Negative binomial + Robust Rank Aggregation	No	No
MAGeCK-MLE	Negative binomial GLM with gene-level effects	Yes	No
BAGEL2	Bayes Factor using essential/non-essential reference sets	No	No
CRISPRcleanR	Unsupervised segmentation to correct CN bias	No	Yes
JACKS	Joint Analysis of CRISPR/-Cas9 Knockout Screens	Yes (multi-replicate)	No

Genome-wide vs. Focused Screens

Genome-wide screens (~80,000–100,000 sgRNAs for human) require large cell numbers (>50 million cells for 500× representation at MOI 0.3) and are best suited for unbiased discovery. Focused screens targeting specific gene sets (e.g., kinases, transcription factors, drug targets) can be performed with fewer cells, more sgRNAs per gene, and deeper sequencing, yielding higher statistical power for the genes included. Consider your biological question when choosing between the two approaches.

18.3 Cell-Free DNA and Liquid Biopsy

18.3.1 Circulating Tumor DNA (ctDNA)

Cell-free DNA (cfDNA) consists of short DNA fragments circulating in the bloodstream, released primarily by apoptotic and necrotic cells. In healthy individuals, cfDNA is predominantly derived from hematopoietic cells. In cancer patients, a fraction of cfDNA originates from tumour cells—this fraction is called **circulating tumour DNA (ctDNA)**.

Key characteristics of cfDNA:

- Fragment size peaks at ~167 bp, corresponding to one nucleosome of wrapped DNA (~147 bp) plus linker DNA (~20 bp). A secondary peak at ~334 bp (dinucleosomal) is often observed.
- ctDNA fraction varies enormously: from >50% in advanced metastatic cancers to <0.01% in early-stage cancers.
- ctDNA carries the same somatic mutations, copy number alterations, and methylation patterns as the tumour of origin.

18.3.2 Non-Invasive Prenatal Testing (NIPT)

NIPT was the first clinical application of cfDNA sequencing. Cell-free fetal DNA (cffDNA), shed from the placenta into the maternal bloodstream, represents ~5–20% of total maternal cfDNA from approximately 10 weeks of gestation. By shallow whole-genome sequencing (~0.1–0.5× coverage) of maternal plasma cfDNA, chromosomal aneuploidies in the fetus (trisomy 21, 18, 13) can be detected as subtle shifts in the proportion of reads mapping to the affected chromosome.

NIPT Statistical Principle

If the fetus has trisomy 21, chromosome 21 constitutes a slightly larger fraction of total cfDNA than expected (because the placenta sheds three copies instead of two). With sufficient sequencing depth, this excess—on the order of a few percent of the fetal fraction—can be detected statistically. The higher the fetal fraction, the more reliable the test. This is why NIPT is typically performed after 10 weeks of gestation, when fetal fraction is sufficient.

18.3.3 Cancer Detection Applications

Multi-cancer early detection (MCED)

The GRAIL Galleri test uses cfDNA methylation patterns (rather than mutations) to detect over 50 cancer types from a single blood draw, with the ability to predict the tissue of origin. The test sequences cfDNA after methylation-sensitive enrichment and uses a machine learning classifier trained on thousands of cancer and non-cancer samples.

Minimal residual disease (MRD)

After cancer treatment (surgery, chemotherapy), ctDNA monitoring can detect residual disease months before imaging detects recurrence. Tumor-informed assays (e.g., Signatera by Natera) design patient-specific panels based on the tumour's mutation profile, achieving sensitivity to detect ctDNA fractions as low as $\sim 0.001\%$.

Treatment monitoring

Serial ctDNA quantification tracks treatment response in real time. Rising ctDNA levels during therapy indicate treatment failure before clinical or imaging progression.

18.3.4 Fragmentomics

The fragmentation pattern of cfDNA carries biological information beyond the sequence itself:

Fragment size distribution

Tumour-derived cfDNA fragments tend to be slightly shorter than normal cfDNA. Fragment size profiling can enhance ctDNA detection.

DELFI (DNA Evaluation of Fragments for early Interception)

Analyses genome-wide fragmentation patterns (the ratio of short to long fragments in 5 Mb bins) to detect cancer. The rationale is that altered chromatin organization in cancer cells produces distinct fragmentation patterns when DNA is released into the blood.

Nucleosome positioning

cfDNA fragments reflect the nucleosome landscape of their cell of origin. Regions protected by nucleosomes produce fragments, while nucleosome-free regions (e.g., active promoters) are depleted. This can be used to infer gene expression and cell-of-origin from cfDNA.

18.3.5 cfDNA Fragmentomics: From Size Distributions to Nucleosome Footprinting

The fragmentation pattern of cfDNA encodes substantial biological information that extends far beyond simple tumour detection. Understanding the biophysical basis of cfDNA fragmentation is essential for interpreting fragmentomic data.

Nucleosomal Fragment Size Distribution

cfDNA fragments are not randomly sized. Their length distribution reflects the nucleosomal organization of the chromatin in the cells of origin. The dominant peak at ~ 167 bp corresponds to the **mono-nucleosomal unit**: 147 bp of DNA wrapped around a histone octamer plus ~ 20 bp of linker DNA protected by histone H1. Subsequent peaks at ~ 334 bp (di-nucleosomal) and ~ 501 bp (tri-nucleosomal) represent fragments spanning two and three nucleosomes, respectively.

Within the mono-nucleosomal peak, finer structure is apparent in high-depth sequencing data:

- A **sub-nucleosomal peak at ~ 145 bp** corresponds to the nucleosome core particle without linker DNA, observed when H1 is absent or displaced.
- Fragments shorter than 145 bp (~ 80 – 120 bp) likely arise from nucleosome remodelling events or represent DNA protected by non-histone protein complexes (e.g., transcription factors).

- A 10 bp periodicity within the mono-nucleosomal peak reflects the helical pitch of DNA on the nucleosome surface, with preferential cleavage occurring where the minor groove faces outward.

Nucleosome Footprinting of Transcription Factor Binding

At genomic regions bound by transcription factors in the cells of origin, cfDNA shows characteristic **nucleosome depletion**—a reduction in fragment coverage centred on the TF binding site, flanked by well-positioned nucleosomes that produce phased fragment ends. This pattern enables inference of TF binding activity and, by extension, the cell type of origin from cfDNA data alone.

The nucleosome footprinting approach works as follows:

1. Align cfDNA reads to the reference genome and extract fragment start/end positions.
2. For each known TF binding site (from databases such as ENCODE or JASPAR), aggregate the coverage of cfDNA fragment midpoints in a window around the site.
3. Sites actively bound by TFs in the cells of origin show a characteristic “V-shaped” depletion profile, while unbound sites show uniform nucleosomal coverage.
4. By comparing the depth of the nucleosome-free region across thousands of TF binding sites, one can infer which TFs are active in the cells contributing cfDNA to the plasma.

ctDNA Detection Sensitivity

The sensitivity of ctDNA detection depends on two key parameters: the **tumour fraction** f (the proportion of cfDNA molecules derived from the tumour) and the **sequencing depth** N (the number of unique cfDNA molecules sampled). For a given somatic mutation present in the tumour, the expected number of mutant molecules in a cfDNA sample is $n_{\text{mut}} = f \cdot N$. To detect a mutation with confidence, one requires n_{mut} to exceed the background error rate of the assay.

For standard Illumina sequencing (error rate $\epsilon \approx 10^{-3}$), the limit of detection is approximately $f \geq 1\%$. With UMI-based error correction ($\epsilon \approx 10^{-5}$), detection extends to $f \geq 0.01\%$. With duplex sequencing ($\epsilon \approx 10^{-7}$), even lower tumour fractions become theoretically accessible, though the number of unique molecules sampled becomes the limiting factor.

The probability of detecting at least one mutant molecule follows a binomial model:

$$P(\text{detect}) = 1 - (1 - f)^N \approx 1 - e^{-fN}, \quad \text{for small } f, \quad (18.7)$$

which shows that for a tumour fraction of $f = 0.01\%$, one needs to sample $N \geq 30,000$ unique molecules to achieve $>95\%$ probability of observing at least one mutant molecule. In practice, achieving this many unique molecules requires deep sequencing (typically $>50,000\times$ raw coverage, reduced to $\sim 30,000\times$ unique coverage after UMI deduplication) or enrichment for the target locus using hybrid capture panels.

Tumour-Informed vs. Tumour-Agnostic Assays

Tumour-informed assays (e.g., Signatera, Archer) design patient-specific panels targeting mutations identified from the primary tumour, achieving higher sensitivity by interrogating known variants. Tumour-agnostic assays (e.g., Guardant360, FoundationOne Liquid CDx) use fixed panels of cancer-associated genes, sacrificing some sensitivity for broader applicability when tumour tissue is unavailable. For minimal residual disease (MRD) monitoring, tumour-informed assays can detect ctDNA at fractions as low as $\sim 0.001\%$, while tumour-agnostic panels typically require $\geq 0.1\%$ tumour fraction.

18.3.6 Methylation-Based Cancer Detection

cfMeDIP-seq (cell-free Methylated DNA ImmunoPrecipitation sequencing) uses an antibody against 5-methylcytosine to enrich methylated cfDNA fragments, followed by sequencing. Because DNA methylation patterns are highly cell-type-specific and extensively reprogrammed in cancer, methylation-based approaches can both detect cancer and identify the tissue of origin.

18.3.7 Technical Challenges and Error Correction

Detecting rare ctDNA variants (at allele fractions below 1%) requires error rates far below standard Illumina sequencing ($\sim 0.1\%$ per base). Two key strategies address this:

UMI-based error correction

Unique molecular identifiers (UMIs) are ligated to each original DNA molecule before amplification. After sequencing, reads sharing the same UMI are grouped into “families” derived from the same original molecule. Errors introduced during PCR or sequencing will appear in only some family members, while true variants will be present in all. Consensus calling across UMI families suppresses errors by several orders of magnitude.

Duplex sequencing

Tags both strands of the original DNA duplex with complementary UMIs, so that reads from the Watson and Crick strands can be identified. A true mutation must be present on *both* strands; PCR errors or sequencing errors will appear on only one strand. This achieves error rates approaching 10^{-7} per base—the lowest of any sequencing method.

Duplex Sequencing

Duplex sequencing achieves the lowest error rate of any sequencing approach ($\sim 10^{-7}$ per base) by requiring that a variant be present on both complementary strands of the original DNA molecule. This is mathematically equivalent to requiring two independent mutations at the same position—an extraordinarily unlikely event. Applications include ultra-rare variant detection, somatic mutagenesis studies (measuring mutation rates in normal tissues), and forensic DNA analysis.

18.4 Adaptive Sampling (Oxford Nanopore)

18.4.1 Real-Time Selective Sequencing

One of the most distinctive capabilities of Oxford Nanopore sequencing is **adaptive sampling** (also called “Read Until”): the ability to selectively sequence specific genomic regions in real time, *without any physical enrichment or capture step*. The principle is elegantly simple:

1. A DNA molecule enters a nanopore and begins translocating.
2. The first few hundred bases of the molecule are sequenced and mapped in real time against a reference genome.
3. If the molecule maps to a region of interest, translocation continues and the full read is collected.
4. If the molecule maps to an off-target region, the pore voltage is reversed, ejecting the molecule and making the pore available for a new molecule.

This software-defined enrichment achieves $\sim 2\text{--}5\times$ enrichment for target regions relative to random sequencing, without any wet-lab modification to the library preparation.

18.4.2 Software and Applications

ReadFish

A Python framework for real-time adaptive sampling that integrates with the MinKNOW acquisition software. Users specify target regions as BED files, and ReadFish handles the real-time basecalling, mapping, and accept/reject decisions.

MinKNOW adaptive sampling

ONT's built-in adaptive sampling feature, available directly in the MinKNOW software without additional tools.

Applications

Adaptive sampling is particularly valuable for:

- Targeted sequencing of specific genes or regions without capture probes.
- Pathogen enrichment from clinical metagenomic samples (accept pathogen reads, reject host reads).
- Human host depletion (reject human reads to enrich for microbial sequences).
- Selective sequencing of specific chromosomes or genomic regions of interest.

Practical Tip

Adaptive sampling works best for applications where the target represents a minority of the total DNA. If the target is $>50\%$ of the genome, there is no enrichment benefit. The enrichment factor also depends on read length: longer molecules have higher mapping confidence from the initial fragment, enabling more accurate accept/reject decisions. Use long DNA extraction protocols ($>10\text{ kb}$ fragment length) for optimal adaptive sampling performance.

18.5 Optical Genome Mapping (Bionano Saphyr)

Optical genome mapping (OGM) is not sequencing in the traditional sense, but it is an increasingly important complement to sequencing for structural variant (SV) detection. The Bionano Saphyr platform works by:

1. Extracting ultra-high molecular weight DNA (molecules $>150\text{ kb}$, ideally $>250\text{ kb}$).
2. Labelling the DNA at specific sequence motifs (typically the 6-bp motif CTTAAG) with fluorescent tags using the DLE-1 enzyme.

3. Linearizing the labelled molecules in nanochannel arrays and imaging the fluorescent label pattern.
4. Comparing the observed label pattern to the expected pattern from a reference genome to detect structural variants.

OGM is particularly powerful for detecting:

- Large structural variants (>500 bp) including deletions, duplications, inversions, and translocations.
- Complex rearrangements in cancer genomes and constitutional chromosomal abnormalities.
- Repeat expansions (e.g., in trinucleotide repeat disorders such as Friedreich’s ataxia, Fragile X syndrome).
- Balanced translocations that are invisible to short-read sequencing and copy-number arrays.

Important Caveat

OGM does not provide nucleotide-level sequence information. It detects structural variants based on label pattern disruption, but it cannot resolve the exact breakpoint sequence. For full characterization, OGM findings should be confirmed and refined by long-read sequencing or targeted PCR and Sanger sequencing of breakpoint junctions.

18.6 Epitranscriptomics: RNA Modifications

18.6.1 The RNA Modification Landscape

Just as DNA is chemically modified (methylation, hydroxymethylation), RNA carries over 170 known chemical modifications that regulate its function. The most abundant and well-studied modifications in mRNA include:

N⁶-methyladenosine (m⁶A)

The most abundant internal modification in eukaryotic mRNA. Found predominantly near stop codons and in 3’UTRs. Regulated by “writer” enzymes (METTL3/METTL14 complex), “eraser” enzymes (FTO, ALKBH5), and “reader” proteins (YTHDF1/2/3, YTHDC1/2). Influences mRNA stability, splicing, translation efficiency, and nuclear export.

5-methylcytosine (m⁵C)

Found in both mRNA and tRNA. Written by NSUN2 and DNMT2. Implicated in mRNA stability and nuclear–cytoplasmic export.

Pseudouridine (Ψ)

The most abundant RNA modification overall (highly prevalent in rRNA and tRNA). In mRNA, Ψ can affect translation fidelity and mRNA stability. Detected by Ψ-seq or bisulfite-based methods.

N¹-methyladenosine (m¹A)

Found in tRNA and rRNA; less common in mRNA. m¹A in the 5’UTR may promote cap-independent translation under stress conditions.

18.6.2 Sequencing-Based Detection of RNA Modifications

MeRIP-seq / m⁶A-seq

The most widely used method for transcriptome-wide m⁶A mapping. mRNA is fragmented (~100 nt), immunoprecipitated with an anti-m⁶A antibody, and sequenced. Enriched regions (“m⁶A peaks”) are identified by comparing IP to input libraries. Resolution is limited to ~100–200 nt (the fragment size).

miCLIP (m⁶A individual-nucleotide-resolution cross-linking and immunoprecipitation)

Achieves single-nucleotide resolution by UV cross-linking the anti-m⁶A antibody to the RNA, creating characteristic mutation signatures at the cross-link site.

DART-seq (Deamination Adjacent to RNA modification Targets)

An antibody-free approach that fuses the m⁶A-binding YTH domain to the cytidine deaminase APOBEC1. The fusion protein binds m⁶A sites and deaminates nearby cytidines to uridines, creating C-to-U mutations detectable by standard RNA-seq. DART-seq requires much less input RNA than MeRIP-seq.

MAZTER-seq

Exploits the m⁶A-sensitive RNase MazF, which cleaves at unmethylated ACA motifs but not at methylated m⁶AACA motifs. Methylated sites appear as uncleaved positions in the sequencing data.

Direct nanopore detection

Oxford Nanopore direct RNA sequencing can detect RNA modifications in native RNA without any chemical treatment. The modifications alter the electrical signal as the RNA translocates through the pore. Computational tools (m6Anet, ELIGOS, Nanom6A) use neural networks to classify modified vs. unmodified bases from the raw signal. This is the only method that preserves the full-length transcript context and avoids all biochemical biases.

Important Caveat

MeRIP-seq/m⁶A-seq has significant limitations: low resolution (~100–200 nt), high input requirements (>5 µg poly-A+ RNA), antibody cross-reactivity (the anti-m⁶A antibody can also bind m⁶Am, the cap-adjacent modification), and batch-dependent variability. Results should be validated with orthogonal methods (e.g., miCLIP, DART-seq, or site-specific qPCR after SELECT) before drawing strong biological conclusions.

18.7 Telomere and Centromere Sequencing: T2T Approaches

The Telomere-to-Telomere (T2T) Consortium completed the first truly complete human genome assembly in 2022, filling in the ~8% of the genome (nearly 200 Mb) that was missing from the GRCh38 reference. This achievement was enabled by:

- **Ultra-long nanopore reads** (>100 kb) that spanned centromeric satellite repeats.
- **PacBio HiFi reads** that provided the high accuracy needed for consensus polishing.
- The use of a complete hydatidiform mole cell line (CHM13), which is effectively haploid, eliminating the challenge of separating homologous chromosomes.

The T2T-CHM13 assembly revealed complete centromere structures (megabases of alpha-satellite repeats), full-length ribosomal DNA arrays, segmental duplications, and hundreds

of previously missing protein-coding genes. Assembling telomeric and centromeric regions remains challenging even with current long-read technology, requiring specialized assembly algorithms (e.g., `verkko`) and careful manual curation.

18.8 Chromosome Conformation in Clinical Settings

While Hi-C and related chromosome conformation capture methods were originally research tools, they are increasingly entering clinical settings, particularly for:

- **Structural variant detection:** Hi-C data intrinsically detect inter-chromosomal translocations (visible as off-diagonal contact signal) and large-scale inversions (visible as altered contact patterns), including balanced rearrangements that are invisible to copy-number arrays.
- **Haplotype phasing:** Hi-C links distant genomic loci that reside on the same chromosome, enabling chromosome-scale haplotype phasing.
- **Cancer karyotyping:** Hi-C-based approaches (e.g., from Arima Genomics or Phase Genomics) can replace traditional cytogenetic karyotyping for detecting chromosomal rearrangements in haematological malignancies.

18.9 Practical Workflow: Ribo-seq Processing

18.9.1 Snakemake Workflow

Listing 18.1: Snakemake workflow for Ribo-seq analysis.

```

1 # Snakefile for Ribo-seq Analysis Pipeline
2 configfile: "config/riboseq_config.yaml"
3
4 SAMPLES = config["samples"]
5 GENOME = config["genome_fasta"]
6 ANNOTATION = config["gtf"]
7 RRNA_INDEX = config["rrna_bowtie2_index"]
8 STAR_INDEX = config["star_index"]
9
10 rule all:
11     input:
12         expand("results/{sample}/ribocode/"
13             "ORF_result.txt", sample=SAMPLES),
14         expand("results/{sample}/translational_efficiency.tsv",
15             sample=SAMPLES),
16         "results/qc/multiqc_report.html"
17
18 rule trim_adapters:
19     """Trim adapters from Ribo-seq reads."""
20     input:
21         fastq="data/{sample}.fastq.gz"
22     output:
23         trimmed="results/{sample}/trimmed/{sample}.fastq.gz"
24     threads: 4
25     shell:
26         """
27         cutadapt -a AGATCGGAAGAGCACACGTCT \
28             --minimum-length 20 --maximum-length 35 \
29             -j {threads} \

```

```

30         -o {output.trimmed} {input.fastq}
31     """
32
33 rule remove_rrna:
34     """Remove rRNA contamination with Bowtie2."""
35     input:
36         fastq="results/{sample}/trimmed/{sample}.fastq.gz"
37     output:
38         non_rrna="results/{sample}/filtered/"
39             "{sample}_no_rrna.fastq.gz",
40         rrna_log="results/{sample}/filtered/rrna_stats.txt"
41     params:
42         index=RRNA_INDEX
43     threads: 8
44     shell:
45         """
46         bowtie2 -x {params.index} \
47             -U {input.fastq} \
48             --un-gz {output.non_rrna} \
49             -p {threads} --very-sensitive \
50             -S /dev/null \
51             2> {output.rrna_log}
52         """
53
54 rule star_align:
55     """Align to genome/transcriptome with STAR."""
56     input:
57         fastq="results/{sample}/filtered/"
58             "{sample}_no_rrna.fastq.gz"
59     output:
60         bam="results/{sample}/aligned/"
61             "{sample}_Aligned.toTranscriptome.out.bam",
62         genome_bam="results/{sample}/aligned/"
63             "{sample}_Aligned.sortedByCoord.out.bam"
64     params:
65         index=STAR_INDEX,
66         prefix="results/{sample}/aligned/{sample}_",
67     threads: 12
68     resources:
69         mem_mb=40000
70     shell:
71         """
72         STAR --runThreadN {threads} \
73             --genomeDir {params.index} \
74             --readFilesIn {input.fastq} \
75             --readFilesCommand zcat \
76             --outSAMtype BAM SortedByCoordinate \
77             --quantMode TranscriptomeSAM \
78             --outFilterMultimapNmax 1 \
79             --outFilterMismatchNmax 2 \
80             --alignEndsType EndToEnd \
81             --outFileNamePrefix {params.prefix}
82         """
83
84 rule psite_offset:
85     """Determine P-site offset from read length."""
86     input:
87         bam="results/{sample}/aligned/"

```

```

88         "{sample}_Aligned.toTranscriptome.out.bam"
89     output:
90         offsets="results/{sample}/psite/"
91             "psite_offsets.txt",
92         plots="results/{sample}/psite/periodicity.pdf"
93     params:
94         annotation=ANNOTATION
95     script:
96         "scripts/calculate_psite_offset.R"
97
98 rule ribocode:
99     """Identify translated ORFs with RiboCode."""
100     input:
101         bam="results/{sample}/aligned/"
102             "{sample}_Aligned.toTranscriptome.out.bam"
103     output:
104         orfs="results/{sample}/ribocode/ORF_result.txt"
105     params:
106         annotation=ANNOTATION,
107         outdir="results/{sample}/ribocode"
108     conda:
109         "envs/ribocode.yaml"
110     shell:
111         """
112         prepare_transcripts -g {params.annotation} \
113         -o {params.outdir}/annot
114         RiboCode -a {params.outdir}/annot \
115         -c {params.outdir}/config.txt \
116         -l no -g -o {params.outdir}/ORF_result.txt
117         """
118
119 rule translational_efficiency:
120     """Calculate TE as RPF density / mRNA density."""
121     input:
122         rpf_bam="results/{sample}/aligned/"
123             "{sample}_Aligned.sortedByCoord.out.bam",
124         mrna_counts="data/{sample}_rnaseq_counts.tsv"
125     output:
126         te="results/{sample}/translational_efficiency.tsv"
127     params:
128         annotation=ANNOTATION
129     script:
130         "scripts/calc_te.R"
131
132 rule multiqc:
133     """Aggregate QC reports."""
134     input:
135         expand("results/{sample}/aligned/"
136             "{sample}_Aligned.sortedByCoord.out.bam",
137             sample=SAMPLES)
138     output:
139         "results/qc/multiqc_report.html"
140     shell:
141         "multiqc results/ -o results/qc/"

```

18.9.2 Nextflow Workflow

Listing 18.2: Nextflow workflow for Ribo-seq analysis.

```

1 #!/usr/bin/env nextflow
2  nextflow.enable.dsl=2
3
4  params.reads      = "data/*.fastq.gz"
5  params.rrna_index = "/ref/rrna_bowtie2"
6  params.star_index = "/ref/star_index"
7  params.gtf        = "/ref/genecode.v44.annotation.gtf"
8  params.outdir     = "results"
9  params.adapter    = "AGATCGGAAGAGCACACGTCT"
10 params.min_length = 20
11 params.max_length = 35
12
13 Channel
14   .fromPath(params.reads)
15   .map { file ->
16       def sample = file.simpleName
17       tuple(sample, file)
18   }
19   .set { reads_ch }
20
21 process TRIM_ADAPTERS {
22     tag "$sample"
23     cpus 4
24     container 'quay.io/biocontainers/cutadapt:4.4'
25     publishDir "${params.outdir}/${sample}/trimmed"
26
27     input:
28     tuple val(sample), path(reads)
29
30     output:
31     tuple val(sample), path("${sample}_trimmed.fq.gz"),
32         emit: trimmed
33
34     script:
35     """
36     cutadapt -a ${params.adapter} \
37         --minimum-length ${params.min_length} \
38         --maximum-length ${params.max_length} \
39         -j ${task.cpus} \
40         -o ${sample}_trimmed.fq.gz ${reads}
41     """
42 }
43
44 process REMOVE_RRNA {
45     tag "$sample"
46     cpus 8
47     memory '16 GB'
48     container 'quay.io/biocontainers/bowtie2:2.5.2'
49
50     input:
51     tuple val(sample), path(trimmed)
52
53     output:
54     tuple val(sample),
55         path("${sample}_no_rrna.fq.gz"),
56         emit: filtered
57 }

```

```

58     script:
59     """
60     bowtie2 -x ${params.rrna_index} \
61         -U ${trimmed} \
62         --un-gz ${sample}_no_rrna.fq.gz \
63         -p ${task.cpus} --very-sensitive \
64         -S /dev/null
65     """
66 }
67
68 process STAR_ALIGN {
69     tag "$sample"
70     cpus 12
71     memory '40 GB'
72     container 'quay.io/biocontainers/star:2.7.11b'
73     publishDir "${params.outdir}/${sample}/aligned",
74         mode: 'copy'
75
76     input:
77     tuple val(sample), path(filtered)
78
79     output:
80     tuple val(sample),
81         path("${sample}_Aligned.toTranscriptome.out.bam"),
82         emit: transcriptome_bam
83     tuple val(sample),
84         path("${sample}_Aligned.sortedByCoord.out.bam"),
85         emit: genome_bam
86
87     script:
88     """
89     STAR --runThreadN ${task.cpus} \
90         --genomeDir ${params.star_index} \
91         --readFilesIn ${filtered} \
92         --readFilesCommand zcat \
93         --outSAMtype BAM SortedByCoordinate \
94         --quantMode TranscriptomeSAM \
95         --outFilterMultimapNmax 1 \
96         --outFilterMismatchNmax 2 \
97         --alignEndsType EndToEnd \
98         --outFileNamePrefix ${sample}_
99     """
100 }
101
102 process RIBOCODE {
103     tag "$sample"
104     cpus 4
105     memory '16 GB'
106     container 'ribocode:latest'
107     publishDir "${params.outdir}/${sample}/ribocode",
108         mode: 'copy'
109
110     input:
111     tuple val(sample), path(trans_bam)
112
113     output:
114     tuple val(sample), path("ORF_result.txt"),
115         emit: orfs

```

```

116
117     script:
118     """
119     prepare_transcripts -g ${params.gtf} -o annot
120     RiboCode -a annot \
121         -c config.txt -l no -g \
122         -o ORF_result.txt
123     """
124 }
125
126 workflow {
127     TRIM_ADAPTERS(reads_ch)
128     REMOVE_RRNA(TRIM_ADAPTERS.out.trimmed)
129     STAR_ALIGN(REMOVE_RRNA.out.filtered)
130     RIBOCODE(STAR_ALIGN.out.transcriptome_bam)
131 }

```

18.10 Comparison of Specialized Applications

Table 18.2: Comparison of specialized sequencing applications covered in this chapter, summarizing what each measures, typical sequencing platform, and key analytical tools.

Application	What It Measures	Key Tools	Platform
Ribo-seq	Active translation at codon resolution	RiboCode, ORFquant, Ribotricer	Illumina SE50
CRISPR screens	Gene fitness / function via guide enrichment	MAGeCK, BAGEL2, CRISPRcleanR	Illumina SE/PE75
cfDNA / liquid biopsy	Tumour mutations, methylation from blood	UMI tools, DELFI, Galleri	Illumina (deep)
Duplex seq.	Ultra-rare variants (10^{-7} error rate)	fgbio, AGeNT	Illumina (deep)
Adaptive sampling	Targeted regions (software-defined)	ReadFish, MinKNOW	ONT
Optical genome mapping	Structural variants, repeat expansions	Bionano Access/Solve	Bionano Saphyr
MeRIP-seq	m ⁶ A RNA methylation	MACS2, exomePeak2	Illumina PE150
Nanopore RNA mods	RNA modifications (native RNA)	m6Anet, ELIGOS	ONT direct RNA

18.11 Summary

This chapter has explored specialized sequencing applications that extend the reach of NGS into domains beyond standard genomics and transcriptomics. Key takeaways include:

- **Ribo-seq** captures the translational landscape at codon-level resolution, revealing novel ORFs and translational regulation that RNA-seq alone cannot detect. The characteristic three-nucleotide periodicity provides a unique quality control signature.

- **Pooled CRISPR screens** enable systematic, genome-scale mapping of gene function by combining CRISPR perturbation with sequencing readout of guide representation. CRISPRi and CRISPRa extend this to repression and activation modalities, respectively.
- **Cell-free DNA analysis** enables non-invasive cancer detection, prenatal testing, and treatment monitoring from a simple blood draw. Fragmentomics and methylation-based approaches add additional layers of information beyond mutation detection.
- **Error-corrected sequencing** (UMI consensus, duplex sequencing) pushes error rates to 10^{-7} , enabling detection of ultra-rare variants in applications ranging from cancer liquid biopsy to mutagenesis studies.
- **Adaptive sampling** on Oxford Nanopore platforms provides software-defined enrichment, removing the need for physical capture in targeted sequencing applications.
- **Epitranscriptomics** reveals the regulatory layer of RNA modifications, with methods ranging from antibody-based enrichment (MeRIP-seq) to direct nanopore detection of modifications in native RNA.
- **T2T sequencing** and **optical genome mapping** address genomic regions—centromeres, telomeres, complex structural variants—that remain challenging for standard short-read approaches.

Each of these applications leverages the same fundamental sequencing platforms discussed in earlier chapters, illustrating the remarkable versatility of modern sequencing technology when combined with creative library preparation strategies and targeted analytical approaches.

Chapter 19

Emerging Technologies and Future Directions

Key Concept

The sequencing field is evolving at an extraordinary pace, with new technologies, computational paradigms, and applications emerging each year. This chapter surveys the frontier: protein sequencing technologies that extend beyond nucleic acids, in situ sequencing that reads genomes within intact tissues, artificial intelligence that is transforming every stage of the sequencing pipeline, portable sequencing that brings genomics to the point of care, pangenomics that moves beyond a single reference genome, and the ethical considerations that accompany these powerful capabilities. Understanding these emerging directions is essential for anyone planning research that will unfold over the coming years.

19.1 Protein Sequencing by Sequencing-Adjacent Technologies

While this book focuses on nucleic acid sequencing, several emerging platforms aim to bring sequencing-like approaches to the proteome. These technologies are relevant because they share conceptual and sometimes physical infrastructure with NGS and because proteomics is increasingly integrated with genomics and transcriptomics.

19.1.1 Quantum-Si: Single-Molecule Protein Sequencing

Quantum-Si is developing a semiconductor-based platform that detects individual amino acids as they are cleaved from a protein molecule. The principle uses time-domain fluorescence detection: fluorescently labelled amino acid recognizers bind to the N-terminal amino acid, and the identity is determined by the fluorescence lifetime (not just wavelength or intensity). After identification, an aminopeptidase cleaves the terminal residue, exposing the next amino acid for the cycle to repeat.

Key characteristics:

- Single-molecule sensitivity—no amplification required.
- Semiconductor chip fabrication enables scaling and cost reduction analogous to DNA sequencing chips.
- Currently limited in the number of amino acids that can be distinguished per cycle, but active development is expanding coverage.

19.1.2 Nautilus Biotechnology: Proteome Analysis Platform

Nautilus takes a different approach: rather than sequencing amino acids sequentially, it uses a large panel of affinity reagents (engineered protein binders) that each recognize short amino acid motifs. By iteratively probing single protein molecules on a surface with different affinity reagents, Nautilus builds up a combinatorial fingerprint that identifies each protein. This approach does not provide a complete amino acid sequence but can identify and quantify thousands of proteins in a sample.

19.1.3 Nanopore-Based Protein Sequencing: Signal Models and Challenges

Beyond fluorescence-based approaches, an emerging frontier is the use of **nanopore technology for direct protein sequencing**. The fundamental principle mirrors nanopore DNA sequencing: a protein molecule is threaded through a nanoscale pore, and the ionic current modulation caused by amino acid residues transiting the pore sensing region is recorded and decoded. However, protein sequencing by nanopore presents formidable challenges that do not arise in nucleic acid sequencing.

Alphabet complexity

DNA has 4 canonical bases; proteins have 20 standard amino acids (plus post-translational modifications), creating a vastly larger signal alphabet. The current signal for each amino acid depends on its size, charge, hydrophobicity, and interactions with neighbouring residues in the pore. With a sensing region spanning ~ 4 – 5 residues, the effective k -mer space is $20^5 = 3.2$ million possible combinations, compared to $4^5 = 1,024$ for DNA.

Variable charge and translocation control

DNA is uniformly negatively charged, enabling electrophoretic driving through the pore at a controllable rate. Proteins carry variable charge (positive, negative, and neutral residues), making voltage-driven translocation unreliable. Current approaches use **unfoldases** (e.g., ClpXP) or molecular motors to ratchet the polypeptide through the pore at a controlled rate, analogous to the helicase motor used in nanopore DNA sequencing.

Protein unfolding

Unlike the linear polymer DNA, proteins fold into complex three-dimensional structures that must be unfolded before they can transit the pore. Denaturation (chemical or thermal) or enzymatic unfolding adds a required preprocessing step that can introduce bias (some proteins are more resistant to unfolding than others).

Signal modelling

The current signal $I(t)$ at time t as a residue k -mer transits the pore is modelled as:

$$I(t) = g(a_i, a_{i+1}, \dots, a_{i+k-1}) + \epsilon(t), \quad (19.1)$$

where a_i through a_{i+k-1} are the amino acids in the sensing region, $g(\cdot)$ is the signal function that maps a k -mer to an expected current level, and $\epsilon(t)$ is noise. Learning $g(\cdot)$ requires extensive training data from proteins of known sequence—a challenge being addressed by sequencing synthetic peptide libraries of defined composition.

Important Caveat

Nanopore protein sequencing remains in the early research phase, with proof-of-concept demonstrations limited to short peptides or individual amino acid discrimination. Achieving the throughput, accuracy, and generality needed for proteome-scale applications will require substantial advances in pore engineering, motor protein development, and signal processing algorithms. Nevertheless, the potential to perform single-molecule, modification-aware protein sequencing without the need for mass spectrometry makes this an area of intense research interest.

Practical Tip

Protein sequencing technologies are not yet at the maturity level of DNA/RNA sequencing, but they represent a convergent trajectory. As these platforms mature, they will increasingly complement nucleic acid sequencing for integrated multi-omics studies, potentially enabling direct measurement of the proteome alongside the genome and transcriptome from the same sample.

19.2 In Situ Sequencing in Tissues

In situ sequencing reads nucleic acid sequences directly within intact tissue sections, preserving spatial context while providing sequence-level information. Unlike spatial transcriptomics methods that capture or probe known sequences, in situ sequencing can discover sequence variants and gene expression patterns simultaneously within their tissue architecture.

19.2.1 Key In Situ Sequencing Technologies

STARmap (Spatially-resolved Transcript Amplicon Readout Mapping)

Performs in situ reverse transcription, rolling circle amplification, and sequencing by ligation within thin tissue sections. Can simultaneously detect hundreds to thousands of genes at subcellular resolution in intact brain tissue.

ExSeq (Expansion Sequencing)

Combines tissue expansion microscopy (physically swelling the tissue to increase effective resolution) with in situ sequencing. By expanding the tissue $\sim 4\times$, ExSeq achieves nanoscale resolution of RNA molecules within cells, enabling subcellular localization of transcripts within organelles and cellular compartments.

BaristaSeq

A padlock-probe-based in situ sequencing method optimized for profiling barcoded neurons in connectomics studies. BaristaSeq reads barcode sequences in neurons that have been virally labelled, enabling mapping of neural connectivity alongside gene expression.

In situ genome sequencing (IGS)

Extends in situ sequencing beyond the transcriptome to read genomic DNA within intact cells. IGS can detect copy number alterations, structural variants, and point mutations in their spatial context within tissues, which is particularly valuable for understanding tumour heterogeneity.

In Situ vs. Capture-Based Spatial Methods

In situ sequencing methods (STARmap, ExSeq) perform the sequencing reaction within the tissue, preserving subcellular localization but requiring specialized microscopy and enzymatic chemistry. Capture-based spatial methods (10x Visium, Slide-seq) capture RNA on a barcoded surface for external sequencing, which is simpler to implement but loses subcellular resolution. Imaging-based methods (MERFISH, seqFISH+) use combinatorial FISH probes to identify transcripts in situ, achieving subcellular resolution for predefined gene panels. Each approach occupies a distinct niche in the resolution-throughput-discovery space.

19.3 Single-Cell Proteomics by Mass Spectrometry

While CITE-seq and similar methods (see [Chapter 17](#)) measure a predefined panel of surface proteins via antibody tags, mass spectrometry-based single-cell proteomics aims to measure the intracellular proteome without relying on antibodies.

SCoPE2 (Single Cell ProtEomics by Mass Spectrometry, version 2)

Uses isobaric tandem mass tags (TMT) to multiplex single cells, with a carrier channel of ~200 cells providing signal for peptide identification. SCoPE2 can quantify >1,000 proteins per cell across hundreds of cells.

plexDIA (Multiplexed Data-Independent Acquisition)

Combines DIA mass spectrometry with multiplexing via non-isobaric labels, achieving higher throughput than SCoPE2 while maintaining quantitative accuracy.

These technologies provide context for understanding the broader single-cell landscape: while sequencing-based methods provide the deepest transcriptomic and epigenomic profiling, mass spectrometry-based proteomics offers direct protein measurement without the translation-rate assumptions inherent in RNA-based protein inference.

19.4 Programmable Biology and Synthetic Long Reads

19.4.1 Linked-Read Technologies

Linked-read technologies partition long DNA molecules into droplets or wells, add a shared barcode to all short-read fragments derived from the same long molecule, and then generate standard short reads. The shared barcode “links” the short reads back to their molecule of origin, enabling long-range information from short-read data.

10x Genomics Chromium Genome (deprecated)

The original linked-read platform, which partitioned ~50 kb DNA molecules into GEM droplets for barcoded library construction. 10x discontinued this product in 2020, but the conceptual approach remains influential.

TELL-seq (Transposase Enzyme Linked Long-read Sequencing)

Uses transposase-loaded microbeads to tag long DNA molecules with shared barcodes in a single tube reaction, without microfluidics. Achieves linked-read libraries from nanogram-level DNA input.

stLFR (Single-Tube Long Fragment Read)

Developed by BGI/MGI, stLFR captures long DNA molecules on barcoded beads in a single tube, generating linked reads compatible with BGI sequencing platforms. stLFR has been used for diploid genome assembly and structural variant detection.

19.4.2 Universal Sequencing Technology

Universal Sequencing Technology (UST) is developing a pore-based electronic sequencing approach that uses synthetic polymer tags attached to each nucleotide. Rather than reading the nucleotide directly, the instrument detects the electronic signature of the polymer tag as it passes through a solid-state nanopore. This approach potentially combines the long-read capabilities of nanopore sequencing with higher accuracy and the scalability of semiconductor fabrication.

19.5 Ultra-Long Nanopore Sequencing Advances

Oxford Nanopore sequencing has achieved some of the longest individual reads in sequencing history, with multiple reports of reads exceeding 4 Mb (4 million bases) in length. These ultra-long reads are transformative for genome assembly, particularly in regions that are intractable for shorter reads.

19.5.1 Ultra-High Molecular Weight DNA Extraction

Achieving ultra-long reads requires ultra-high molecular weight (UHMW) DNA, which in turn requires specialized extraction protocols:

Ultra-Long DNA Extraction Key Principles

- **Minimal shearing:** Avoid vortexing, pipetting through narrow tips, or any mechanical force that fragments DNA. Use wide-bore pipette tips or cut tips.
- **Agarose plug lysis:** Embed cells in low-melting-point agarose plugs before lysis. The agarose matrix protects DNA from mechanical damage during enzymatic digestion.
- **Phenol-chloroform extraction with phase-lock tubes:** Gentle organic extraction preserves molecular weight better than column-based methods.
- **Needle-based shearing avoidance:** Use gravity-based or slow rotation mixing instead of pipetting.
- **Quality assessment:** Use pulsed-field gel electrophoresis (PFGE) or the Femto Pulse system (Agilent) to verify that extracted DNA exceeds 100–200 kb.

19.5.2 Applications of Ultra-Long Reads

- **Centromere assembly:** Human centromeres consist of megabases of highly repetitive alpha-satellite DNA. Only reads that span entire higher-order repeat units (>100 kb) can resolve centromeric structure, as demonstrated by the T2T Consortium.
- **Complex repeat resolution:** Segmental duplications, satellite arrays, and tandem repeat expansions that confound short reads and even standard long reads can be resolved with ultra-long reads.
- **Structural variant phasing:** Ultra-long reads can phase structural variants with nearby SNPs across hundreds of kilobases, enabling allele-specific SV analysis.
- **Complete bacterial genomes:** Many bacterial genomes (<5 Mb) can be assembled into a single contig from ultra-long reads alone, with no gaps.

19.6 AI and Machine Learning in Sequencing

Artificial intelligence and machine learning have permeated every stage of the sequencing pipeline, from signal processing to biological interpretation.

19.6.1 Neural Network Basecallers

ONT Dorado

Oxford Nanopore’s current basecaller uses transformer-based neural networks to convert raw electrical signal (“squiggle”) into nucleotide sequence. Successive model generations have dramatically improved accuracy: from ~85% in early MinION runs to >99% with the latest “super-accurate” models on R10.4.1 flow cells. Dorado also performs modified base detection (5mC, 6mA) as part of basecalling.

PacBio DeepConsensus

Uses a transformer neural network to generate consensus sequences from the multiple passes of a circular consensus sequencing (CCS/HiFi) read. DeepConsensus improves yield at Q30+ accuracy by 1–9% over the standard CCS algorithm, effectively converting borderline reads into usable HiFi reads.

19.6.2 AI-Powered Variant Calling

DeepVariant (Google)

A convolutional neural network (CNN) that treats variant calling as an image classification problem. Pileup images (visualizations of reads aligned at a candidate variant position) are classified as homozygous reference, heterozygous, or homozygous variant. DeepVariant consistently achieves the highest accuracy in PrecisionFDA truth challenges and works across Illumina, PacBio, and ONT data (with platform-specific models).

PEPPER-Margin-DeepVariant

An extension of DeepVariant optimized for long-read data, using the PEPPER neural network for candidate variant identification and Margin for haplotype-aware read phasing before final variant calling with DeepVariant.

Clair3

A neural network variant caller specifically designed for long-read data (ONT and PacBio), achieving competitive accuracy with DeepVariant while being significantly faster.

19.6.3 Genomic Language Models and Foundation Models

A new paradigm in computational genomics is the application of large language model architectures to biological sequences:

Evo A genomic foundation model trained on 2.7 million prokaryotic and phage genomes. Evo uses a StripedHyena architecture (a hybrid of attention and state-space models) with a context window of up to 131 kilobases. It can predict gene essentiality, generate functional DNA sequences, and model multi-element genetic interactions across long genomic distances.

Nucleotide Transformers

A family of transformer models (from the Instadeep/BioNTech collaboration) trained on reference genomes and whole-genome sequencing data. These models generate contextual embeddings for DNA sequences that capture functional information, enabling zero-shot prediction of variant effects, regulatory element activity, and splicing outcomes.

scGPT

A generative pre-trained transformer for single-cell biology, trained on over 33 million single-cell transcriptomic profiles. scGPT learns a universal representation of cell states and can be fine-tuned for cell-type annotation, perturbation response prediction, gene network inference, and multi-batch integration.

Geneformer

A transformer model pre-trained on ~30 million single-cell transcriptomic profiles from Genecorpus-30M. Geneformer treats each cell's expression profile as a "sentence" of gene "tokens" ranked by expression, learning contextual gene representations. It can predict disease-related gene regulatory network dynamics and identify therapeutic targets.

scBERT

A BERT-based model for single-cell transcriptomics that generates cell embeddings useful for cell-type annotation, particularly in atlas-scale datasets where traditional methods become computationally prohibitive.

Foundation Models in Genomics

Foundation models are large neural networks pre-trained on vast amounts of unlabelled data, which can then be fine-tuned for specific downstream tasks with relatively little labelled data. In genomics, foundation models pre-trained on millions of genomes or single-cell profiles learn general-purpose representations of biological sequences and cell states. These representations encode functional information (regulatory grammar, cell-type identity, variant effects) that emerges from the training data without explicit supervision. The analogy to large language models in natural language processing is direct: just as GPT learns language structure from text, genomic foundation models learn biological grammar from sequences.

19.6.4 Transformer Architecture for Genomic Sequences

The core computational mechanism underlying most genomic foundation models is the **transformer** architecture, specifically the self-attention mechanism that enables the model to capture long-range dependencies in sequence data.

Given an input sequence of tokens (nucleotides, k -mers, or gene identifiers), the self-attention mechanism computes a weighted representation of each token based on its relationships to all other tokens in the sequence. For each attention head, the input embeddings are projected into three matrices—queries (Q), keys (K), and values (V)—and the attention output is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (19.2)$$

where d_k is the dimensionality of the key vectors and the scaling factor $1/\sqrt{d_k}$ prevents the dot products from becoming too large. The softmax function converts the raw attention scores into a probability distribution, so each position attends to a weighted combination of all other positions. Multi-head attention runs h parallel attention computations with independent learned projections, allowing the model to attend to different types of relationships simultaneously.

For DNA foundation models, this architecture is adapted in several ways:

DNABERT-2

Tokenizes DNA sequences using byte pair encoding (BPE) rather than fixed k -mers, producing a vocabulary of variable-length tokens learned from genome data. DNABERT-2 is pre-trained with masked language modelling (predicting masked tokens from context)

on multi-species genome data. The BPE tokenization substantially reduces sequence length compared to single-nucleotide tokenization, enabling longer genomic context windows within the same computational budget.

Nucleotide Transformer

A family of models ranging from 50 million to 2.5 billion parameters, trained on 3,202 reference genomes plus 850 diverse human genomes. Uses 6-mer tokenization and is pre-trained with masked language modelling. The resulting embeddings capture regulatory element activity, splicing signals, and variant effects, enabling zero-shot or few-shot prediction on these tasks.

Evo Uses a hybrid **StripedHyena** architecture combining attention layers with state-space model (SSM) layers, achieving a context window of 131 kilobases—far longer than standard transformer models. The SSM layers provide efficient processing of long sequences with sub-quadratic computational complexity (standard self-attention scales as $O(n^2)$ in sequence length n), while the interspersed attention layers capture specific long-range interactions. Evo is trained on 2.7 million prokaryotic and phage genomes using next-token prediction.

These models are applied to downstream tasks through either **fine-tuning** (adjusting model weights on a labelled dataset) or **zero-shot prediction** (using the pre-trained model directly). Key applications include:

- **Variant effect prediction:** Compute the log-likelihood ratio of the reference vs. alternate allele in the model’s learned distribution to score variant deleteriousness.
- **Regulatory element annotation:** Classify genomic sequences as promoters, enhancers, or silencers based on learned sequence representations.
- **Functional sequence design:** Use the generative capabilities of the model to design novel regulatory sequences or gene constructs with desired properties.

19.6.5 AlphaFold and the Sequence-to-Structure Connection

While not a sequencing technology, **AlphaFold** (DeepMind) represents the most dramatic demonstration of AI’s impact on biology. AlphaFold2 (2020) and AlphaFold3 (2024) predict three-dimensional protein structures from amino acid sequences with near-experimental accuracy. The connection to sequencing is fundamental: the amino acid sequence predicted by AlphaFold is ultimately derived from the DNA/RNA sequence produced by sequencing. The combination of genome sequencing → gene prediction → protein structure prediction → functional annotation creates a complete pipeline from sequence to molecular function.

19.7 Cloud and On-Premise Computing

The computational demands of modern sequencing analysis—from petabyte-scale storage to GPU-intensive deep learning—have driven the development of specialized cloud platforms and high-performance computing solutions.

19.7.1 Major Cloud Platforms for Genomics

Terra / AnVIL (Broad Institute)

Terra is a cloud-based platform built on Google Cloud that provides a workspace for running genomic analyses at scale. The NHGRI AnVIL (Analysis, Visualization, and Informatics Lab-space) project provides controlled access to large genomic datasets

(e.g., gnomAD, GTEx, CCDG) alongside analytical tools within Terra. Workflows are typically written in WDL (Workflow Description Language) and executed by the Cromwell engine.

DNAnexus

A commercial cloud platform that provides secure, compliant (HIPAA, FedRAMP) genomic data analysis. Supports multiple workflow languages and provides a curated app library. Used by the UK Biobank for analysis of 500,000 whole-genome sequences.

Illumina Connected Analytics (ICA)

Illumina's cloud platform for analysis of data from Illumina sequencing instruments. Provides DRAGEN-based secondary analysis pipelines and integrates with Illumina's BaseSpace Sequence Hub for data management.

Illumina DRAGEN

A hardware-accelerated (FPGA-based) bioinformatics pipeline that performs alignment, variant calling, and other analyses at dramatically faster speeds than software-only tools. DRAGEN can process a 30× whole-genome in ~25 minutes, compared to hours for standard BWA-MEM + GATK pipelines. Available both on-premise (DRAGEN server) and in the cloud.

Seven Bridges

A cloud genomics platform that supports CWL and WDL workflows, with integration to multiple cloud providers (AWS, Google Cloud, Azure). Used by the NCI Cancer Genomics Cloud.

Nextflow Tower / Seqera Platform

A management and monitoring platform for Nextflow workflows that provides a web interface for launching, monitoring, and managing pipeline executions across cloud and HPC environments. Supports AWS Batch, Google Cloud Life Sciences, Azure Batch, and traditional HPC schedulers (SLURM, PBS, LSF).

19.7.2 GPU Acceleration

NVIDIA Clara Parabricks

A suite of GPU-accelerated genomics tools that reimplements standard bioinformatics algorithms (BWA-MEM, GATK HaplotypeCaller, DeepVariant, STAR) on GPUs. A 30× whole-genome analysis that takes 24+ hours on a 16-core CPU can be completed in under 30 minutes on a single GPU node. Parabricks is available on-premise and in all major cloud environments.

Choosing Between Cloud and On-Premise

The choice between cloud and on-premise computing depends on several factors:

- **Data volume:** For occasional, burst-intensive analyses, cloud is cost-effective. For continuous high-volume sequencing (e.g., a production genome center), on-premise HPC clusters may be more economical.
- **Data sensitivity:** Clinical and human genetic data may have regulatory requirements (HIPAA, GDPR) that constrain where data can be stored and processed. Many cloud platforms now offer compliant environments.
- **Egress costs:** Downloading data from the cloud (“egress”) incurs fees that can be substantial for petabyte-scale datasets. “Bring compute to the data” strategies (analyzing data where it is stored) mitigate this.
- **Reproducibility:** Cloud platforms with containerized workflows (Docker/Singularity) and workflow managers (Nextflow, Snakemake, WDL/Cromwell) provide excellent computational reproducibility.

19.8 Portable and Point-of-Care Sequencing

The Oxford Nanopore MinION—a USB-powered sequencer that weighs 87 grams and costs approximately \$1,000—has enabled sequencing in environments that were previously unimaginable.

19.8.1 Field Sequencing Deployments

Ebola outbreak (2015–2016)

Josh Quick and Nicholas Loman deployed MinION sequencers to Guinea during the West African Ebola epidemic, performing real-time viral genome sequencing in field laboratories with minimal infrastructure. Results were available within 24 hours of sample collection, enabling tracking of viral transmission chains.

International Space Station

NASA astronaut Kate Rubins performed the first DNA sequencing in space using a MinION aboard the ISS in 2016, demonstrating that sequencing works in microgravity. Subsequent experiments have sequenced microbial communities from ISS surfaces.

Antarctic and Arctic research

MinION sequencers have been deployed to Antarctic research stations and Arctic field camps for environmental DNA surveys of marine and terrestrial ecosystems, where samples cannot be easily shipped to sequencing facilities.

Rainforest biodiversity

Researchers have performed real-time species identification in tropical rainforests using MinION sequencing of environmental DNA and barcoding amplicons, enabling immediate feedback for biodiversity surveys.

19.8.2 Clinical Rapid Sequencing

Pathogen identification

Metagenomic nanopore sequencing can identify bacterial, viral, and fungal pathogens from clinical samples (blood, cerebrospinal fluid, respiratory secretions) in 4–8 hours,

compared to days for culture-based methods. This is particularly valuable for critically ill patients with infections of unknown etiology.

Rapid whole-genome sequencing for neonates

Stanford University's rapid WGS program demonstrated diagnosis of genetic diseases in critically ill newborns within 8–14 hours from blood draw to diagnosis, using ultra-rapid library preparation and accelerated Illumina sequencing combined with DRAGEN analysis.

Antimicrobial resistance profiling

Nanopore sequencing of bacterial isolates can identify resistance genes and predict antibiotic susceptibility within hours, guiding antibiotic therapy decisions far faster than phenotypic susceptibility testing.

19.8.3 Veterinary and Agricultural Diagnostics

Portable sequencing is increasingly used outside of human medicine:

- Detection of plant pathogens in the field, enabling rapid quarantine decisions.
- Identification of animal diseases at the farm level without shipping samples to central laboratories.
- Food safety testing: rapid identification of contaminating organisms in food production facilities.
- Aquaculture: monitoring for fish pathogens in farmed fish populations.

19.9 Democratising Genomics and Healthcare

For most of the history of genome sequencing, access to genomic information was restricted to well-funded academic institutions and specialist clinical centres. That is changing rapidly. Falling sequencing costs, the rise of direct-to-consumer (DTC) genomics, open-source bioinformatics, and cloud computing are converging to make it plausible for individuals to obtain, process, and interpret their own genome sequences—a shift with profound implications for preventive medicine, personalised drug prescribing, and health equity.

19.9.1 The Falling Cost of Personal Genome Sequencing

The cost of sequencing a human genome has fallen from approximately \$3 billion in 2003 (the Human Genome Project) to below \$200 for 30× short-read WGS in 2024. Several commercial providers now offer clinical-grade (ISO 15189) whole-genome sequencing directly to consumers or via physicians at a price accessible to middle-income individuals in high-income countries. Continued cost reductions driven by new sequencing chemistry (Illumina NovaSeq X, Complete Genomics DNBSEQ-T20x2, Ultima UG 100) and increasing competition are expected to push the price of a complete genome below \$100 within the coming years.

Why Your Genome Is Worth Sequencing Once

Unlike most laboratory tests, a genome sequence is a permanent record: once sequenced, it can be reanalysed repeatedly as knowledge grows. Variants of uncertain significance today may be reclassified as pathogenic or benign in future database updates. A genome sequenced at age 30 can yield new clinically actionable insights at age 50 without any additional sequencing cost.

19.9.2 Options for Obtaining Your Genome Sequence

Individuals seeking to obtain genome-scale data have several pathways at different cost and resolution points:

DTC genotyping arrays (\$50–150)

Companies such as 23andMe and AncestryDNA genotype ~600,000–900,000 SNPs using microarrays. Array data is inexpensive and can be used for ancestry estimation, disease-risk polygenic scores, and carrier status for many Mendelian conditions. Limitations: only interrogates known variant positions, cannot detect novel mutations, structural variants, or indels outside the array design. Raw data files (PLINK format) can be downloaded and analysed independently.

Low-pass WGS (0.5–1×; \$50–100)

Companies such as Nebula Genomics offer low-coverage whole-genome sequencing followed by statistical imputation to infer genotypes at millions of SNP positions. Imputation accuracy is high for common variants (MAF >5%) and diminishes for rare variants. Suitable for polygenic risk scores but not for rare Mendelian diagnosis.

30× short-read WGS (\$200–400)

Full 30× Illumina WGS through providers such as Nebula Genomics (research tier), Dante Labs, or Sequencing.com provides a complete FASTQ/BAM file for personal download. This is equivalent to a clinical-grade WGS and enables detection of SNPs, indels, structural variants, and copy number variants genome-wide. A physician's prescription or clinical intermediary may be required in some jurisdictions.

HiFi long-read WGS (\$600–1,200)

PacBio HiFi or Oxford Nanopore sequencing now enables phased diploid assemblies and direct methylation profiling. Services from providers such as Phase Genomics or specialty clinical labs offer long-read WGS for individuals seeking full haplotype resolution, characterisation of complex structural variants, or repeat expansions (e.g., in trinucleotide repeat diseases).

Clinical WGS via healthcare system

In an increasing number of countries (UK NHS, Australia, parts of Europe), whole-genome or whole-exome sequencing is reimbursed for specific indications (rare disease, oncology, neonatal intensive care). For individuals with a clinical indication, this is the highest-quality route, with interpretation by accredited clinical geneticists.

Data Privacy and Ownership

Before uploading genomic data to any third-party service, understand the company's data-sharing policies. Several DTC companies have shared or sold anonymised genetic data to pharmaceutical companies. Your genome contains identifying information not only about you but about your relatives. Opt-out from research-sharing programmes if privacy is a concern, and prefer services that allow you to download raw data and delete stored copies.

19.9.3 Processing Your Own Genome: A Practical Guide

If you have obtained raw FASTQ files from a WGS service, the following open-source pipeline produces a fully annotated variant call set using commodity cloud computing at minimal cost.

Step 1: Infrastructure

The analysis can be run on a personal laptop (feasible but slow), a university HPC cluster (free for students/staff), or cloud computing. For individuals without HPC access, cloud options include:

- **Google Colab Pro** (\$10/month): 52 GB RAM, A100 GPU access; sufficient for small analyses.
- **AWS / Google Cloud / Azure spot/preemptible instances**: A 30× WGS analysis (BWA-MEM2 + GATK) costs approximately \$2–5 in cloud compute on a 32-core spot instance. Tools such as **Terra** (Broad Institute) or **DNAnexus** provide managed platforms that handle cloud resource provisioning automatically.
- **Seven Bridges / Galaxy**: Browser-based platforms offering pre-built pipelines (GATK Best Practices, nf-core/sarek) that require no command-line expertise.

Step 2: Short-Read WGS Analysis Pipeline

Listing 19.1: Personal genome analysis using nf-core/sarek (Nextflow).

```

1 # Install Nextflow and pull the sarek pipeline
2 curl -s https://get.nextflow.io | bash
3 nextflow pull nf-core/sarek
4
5 # Create a minimal samplesheet (CSV)
6 cat > samplesheet.csv <<'EOF'
7 patient,sample,lane,fastq_1,fastq_2
8 individual1,genome_seq,1,reads_R1.fastq.gz,reads_R2.fastq.gz
9 EOF
10
11 # Run the full germline variant calling pipeline
12 nextflow run nf-core/sarek \
13   --input samplesheet.csv \
14   --genome GRCh38 \
15   --tools haplotypcaller,vep,snpEff \
16   --outdir results/ \
17   -profile docker
18
19 # Key outputs:
20 #   results/variant_calling/haplotypcaller/*.vcf.gz  -- raw SNP/indel calls
21 #   results/annotation/vep/*.ann.vcf.gz           -- annotated variants
22 #   results/multiqc/multiqc_report.html           -- QC summary

```

The **nf-core/sarek** pipeline implements the GATK Best Practices workflow and handles all steps automatically: adapter trimming (**fastp**), alignment (**BWA-MEM2**), duplicate marking (**GATK MarkDuplicates**), base quality score recalibration (**BQSR**), **HaplotypeCaller** genotyping, variant filtration (**VQSR** or hard filters), and annotation with **VEP** or **snpEff**.

Step 3: Interpreting Your Variants

After generating a variant call file (VCF), interpretation requires cross-referencing against established databases:

- **ClinVar** (ncbi.nlm.nih.gov/clinvar): Aggregates variant–disease associations curated by clinical laboratories worldwide. Filter your VCF for ClinVar **CLNSIG=Pathogenic** or **Likely_pathogenic** variants.
- **OMIM** (Online Mendelian Inheritance in Man): Comprehensive catalogue of Mendelian disease genes and allelic variants.

- **gnomAD**: Population allele frequencies from >150,000 individuals. Variants absent from gnomAD or with allele frequency <0.001 in relevant populations are candidates for rare disease investigation.
- **PharmGKB / CPIC**: Pharmacogenomic variant–drug interaction databases that identify variants affecting drug metabolism (e.g., *CYP2C19* variants affecting clopidogrel response, *TPMT* variants affecting thiopurine toxicity).
- **PGS Catalog**: Repository of validated polygenic score models. Tools such as `pgsc_calc` compute polygenic risk scores for dozens of common conditions (type 2 diabetes, coronary artery disease, breast cancer) from your VCF.

Using Phenopackets and Matchmaker Exchange

If you suspect a rare Mendelian condition, encoding your phenotype as a **Phenopacket** (using Human Phenotype Ontology, HPO, terms) and submitting to **Matchmaker Exchange** (matchmakerexchange.org) can connect you with researchers and clinicians studying the same gene or phenotype globally—a powerful tool for ultra-rare disease diagnosis.

Step 4: Methylation and Long-Read Analysis

If you have obtained ONT or PacBio HiFi data, additional analysis layers are available:

Listing 19.2: Personal methylome analysis with Oxford Nanopore data.

```

1 # Basecall with Dorado (includes 5mC methylation calling)
2 dorado basecaller dna_r10.4.1_e8.2_400bps_sup@v4.3.0 \
3   --modified-bases 5mC_CpG \
4   pod5_dir/ > calls.bam
5
6 # Align to reference
7 samtools fastq calls.bam | \
8   minimap2 -ax map-ont GRCh38.fa - | \
9   samtools sort -o aligned.bam && samtools index aligned.bam
10
11 # Extract per-CpG methylation
12 modkit pileup aligned.bam methylation.bedmethyl \
13   --ref GRCh38.fa --preset traditional
14
15 # Compute global CpG methylation statistics
16 awk 'NR>1 {sum+=$11; count++} END {print "Mean CpG methylation:", sum/count "%"}' \
17   methylation.bedmethyl

```

19.9.4 Equity and the Digital Divide in Genomic Medicine

While falling costs are democratising access to genomic data, significant barriers remain:

- **Underrepresentation of non-European populations** in reference databases (ClinVar, gnomAD) means that variants common in African, Asian, and Indigenous populations may be misclassified as rare or of uncertain significance. The H3Africa initiative, GenomeAsia100K, and the NHLBI TOPMed programme are working to rectify this imbalance.
- **Interpretation expertise**: A VCF file is not a medical report. Without access to a clinical geneticist or genetic counsellor, most individuals cannot safely act on variant findings. Expanding training and telehealth provision of genetic counselling is essential.

- **Infrastructure:** High-throughput sequencing requires reliable electricity, high-bandwidth internet, and computing resources—prerequisites not universally available in low- and middle-income countries (LMICs). Portable sequencers (Oxford Nanopore MinION) and offline analysis tools (EPI2ME, minimap2 on laptops) are beginning to address this.
- **Regulatory environments:** In several countries, DTC genomics is restricted or prohibited to protect consumers from direct access to clinically sensitive information without medical supervision.

Table 19.1: Pathways for obtaining personal genome sequencing at different price points (2024 prices, approximate).

Approach	Cost	Coverage	Variants detected	Best use case
DTC SNP array	\$50–150	0.03%	Common SNPs only	Ancestry, poly-genic scores
Low-pass WGS (0.5×)	\$50–100	100% (imputed)	Common SNPs (imputed)	Polygenic scores
30× short-read WGS	\$200–400	100%	SNPs, indels, SVs, CNVs	Complete variant profiling
HiFi long-read WGS	\$600–1,200	100% (phased)	All + repeat expansions	Phasing, structural variants
Clinical WGS (NHS/insured)	Covered	100%	All + clinical reporting	Rare disease, oncology

19.9.5 The Personal Genome as a Longitudinal Health Tool

A DNA sequence is fixed from birth; the transcriptome, methylome, and proteome change throughout life in response to ageing, environment, and disease. Combining periodic **multi-omic snapshots** (RNA-seq from blood, cell-free DNA, proteomics) with a fixed whole-genome reference is emerging as a framework for proactive, personalised health monitoring. Studies such as the **Stanford Wellness Study** (Michael Snyder’s group) demonstrated that longitudinal multi-omic profiling can detect pre-symptomatic disease states—including nascent type 2 diabetes—before clinical symptoms appear. As the cost of these measurements continues to fall, periodic multi-omic health checks are likely to transition from research tools to mainstream preventive medicine.

19.10 Integration of Multi-Omics with Electronic Health Records

As genomic sequencing becomes routine in clinical care, integrating sequencing results with electronic health records (EHRs) presents both opportunities and challenges:

- **Pharmacogenomic decision support:** Embedding pharmacogenomic variant information in the EHR and triggering clinical decision support alerts when a drug–gene interaction is relevant. Programs at Vanderbilt (PREDICT), St. Jude (PG4KDS), and the University of Chicago have demonstrated feasibility.
- **Variant reanalysis:** As knowledge of variant pathogenicity evolves, genomic data stored in EHRs can be periodically reanalysed, potentially yielding new diagnoses years after initial sequencing.
- **Data standards:** HL7 FHIR Genomics provides a standard framework for representing genomic data in EHR systems, enabling interoperability across institutions.
- **Challenges:** Variant interpretation updates, data storage costs, patient consent for long-term data retention, and computational infrastructure for reanalysis all remain active areas of development.

19.11 Pangenomics: Moving Beyond Single Reference Genomes

19.11.1 The Limitations of a Single Reference

Since the completion of the Human Genome Project, nearly all genomic analyses have relied on a single linear reference genome (currently GRCh38/hg38, or the T2T-CHM13 assembly). This approach introduces **reference bias**: sequence reads containing variants not present in the reference may fail to align or align with lower quality, systematically underdetecting variation in individuals whose genomes diverge most from the reference. Because the reference genome is predominantly derived from a small number of donors of European ancestry, this bias disproportionately affects underrepresented populations.

19.11.2 The Human Pangenome Reference Consortium

The **Human Pangenome Reference Consortium (HPRC)** aims to create a reference that represents the full spectrum of human genomic diversity. The initial release includes high-quality phased diploid assemblies from 47 genetically diverse individuals, with a goal of 350 individuals representing global genetic diversity. These assemblies are being integrated into a unified pangenome reference.

19.11.3 Graph-Based References

Pangenome references are represented as **sequence graphs** rather than linear sequences. In a graph-based reference, common variants are represented as alternative paths through the graph, allowing reads from any individual to find their correct alignment regardless of which alleles they carry.

vg (variation graph)

The most established graph-based genome toolkit. **vg** constructs a variation graph from a reference genome plus known variants (or from multiple assemblies), indexes the graph for fast read mapping, and calls variants against the graph. Graph-based mapping with **vg giraffe** has been shown to reduce reference bias and improve variant calling accuracy, particularly for structural variants and in regions of high diversity (e.g., the MHC/HLA region).

minigraph

Developed by Heng Li (author of BWA and minimap2), **minigraph** constructs a pangenome graph from multiple genome assemblies, with a focus on structural variants.

It represents SVs as alternative paths in the graph, enabling SV genotyping by mapping reads to the graph.

minigraph-cactus

Combines **minigraph** with the Cactus whole-genome aligner to produce a comprehensive pangenome graph that captures both small variants and structural variants from multiple assemblies.

19.11.4 Pangenome Graph Formalism and the GFA Format

The mathematical representation of a pangenome as a graph provides a rigorous framework for encoding, indexing, and querying genomic variation. A **variation graph** (or sequence graph) is defined as a directed graph $G = (V, E)$ where:

- Each node $v \in V$ represents a **sequence segment**—a string of nucleotides corresponding to a stretch of DNA shared by one or more haplotypes.
- Each directed edge $(u, v) \in E$ represents **adjacency**: the sequence of node u is immediately followed by the sequence of node v in at least one haplotype.
- A **path** through the graph corresponds to a complete haplotype sequence, traversing a series of nodes connected by edges. The reference genome is one such path; each alternative haplotype defines another.

At a variant site (e.g., a SNP or indel), the graph branches into alternative nodes representing the reference and alternate alleles, then merges back at the downstream shared sequence. For a simple biallelic SNP, the graph contains a “bubble” structure: one node for the reference allele and one for the alternate allele, both sharing the same predecessor and successor nodes. Structural variants (insertions, deletions, inversions) produce more complex graph topologies, including nested bubbles and non-trivial edge structures.

The standard interchange format for pangenome graphs is the **Graphical Fragment Assembly (GFA)** format. GFA version 1 uses three record types:

S (Segment)

Defines a node: **S** <name> <sequence>. Each segment has a unique name and an associated nucleotide sequence.

L (Link)

Defines an edge: **S** <from> <from_orient> <to> <to_orient> <overlap>. Links connect the end of one segment to the beginning of another, with orientation (+ or – strand) and optional overlap specification.

P (Path)

Defines a haplotype path: **P** <name> <segment_names> <overlaps>. Paths trace named sequences through the graph, recording which segments are traversed and in which orientation.

The **vg** toolkit operates on variation graphs in GFA or its own internal formats (XG, GBWT, GBZ). The key operations supported by **vg** include:

1. **Graph construction** (**vg construct**, **vg autoindex**): Build a variation graph from a linear reference plus VCF of known variants, or from multiple genome assemblies via **minigraph-cactus**.
2. **Read mapping** (**vg giraffe**): Map short reads to the graph using minimizer-based seeding and graph-aware extension. **vg giraffe** achieves speed comparable to linear aligners (BWA-MEM2) while reducing reference bias by considering all paths through variant sites.

3. **Variant calling** (`vg call`, `vg pack`): Genotype variants by examining read support for alternative paths through graph bubbles.
4. **Surjection** (`vg surject`): Project graph alignments back onto a linear reference coordinate system, producing standard BAM files compatible with existing tools.

Graph-based mapping with `vg giraffe` has been benchmarked to reduce reference bias by 5–15% in highly polymorphic regions (such as the MHC/HLA locus) and to improve structural variant genotyping accuracy by providing explicit graph representation of known SVs. As the HPRC pangenome reference matures and is adopted by major variant calling pipelines, graph-based analysis is expected to become the standard approach for human genome analysis.

19.11.5 Population-Specific References

Some projects have created population-specific reference genomes or pangenomes tailored to specific ancestral groups (e.g., African, East Asian, South Asian). These references can improve variant calling accuracy in the target population by reducing reference bias for population-specific structural variants and common polymorphisms.

Important Caveat

While population-specific references can improve technical accuracy for a given population, they risk reinforcing the problematic concept of discrete biological “races.” The pangenome approach—representing the full continuum of human diversity in a single graph-based reference—is conceptually and practically preferable to a collection of population-specific linear references.

19.12 Ethical Considerations

The increasing power, accessibility, and scope of sequencing technologies raise profound ethical questions that the genomics community must actively address.

19.12.1 Genetic Privacy and Insurance Discrimination

In the United States, the **Genetic Information Nondiscrimination Act (GINA)** of 2008 prohibits health insurers and employers from using genetic information to make coverage or employment decisions. However, GINA has important gaps: it does not cover life insurance, disability insurance, or long-term care insurance. Similar legislation exists in other countries but with varying scope and enforcement.

As sequencing becomes routine in clinical care and direct-to-consumer testing, the volume of genetic data in existence is expanding rapidly, increasing the potential for misuse.

19.12.2 Direct-to-Consumer Genetic Testing

Companies such as 23andMe, AncestryDNA, and others offer genotyping (and increasingly sequencing) services directly to consumers. These services raise concerns about:

- **Informed consent:** Do consumers understand the implications of learning about disease risk, carrier status, or pharmacogenomic variants?
- **Data security:** Consumer genomic databases have been targeted by hackers and used by law enforcement (via genetic genealogy).
- **Clinical validity:** Some DTC health reports lack the clinical validation and contextual interpretation provided by certified genetic counsellors and clinical geneticists.

19.12.3 Incidental Findings in Clinical Sequencing

When a patient undergoes clinical whole-genome or whole-exome sequencing for one condition, the data may reveal **incidental (or secondary) findings**—pathogenic variants unrelated to the primary indication. The American College of Medical Genetics and Genomics (ACMG) maintains a list of ~80 genes for which incidental findings should be reported to patients because they are medically actionable. Deciding which findings to disclose, and how to counsel patients about unexpected results, remains an area of active ethical debate.

19.12.4 Data Sovereignty for Indigenous Populations

Genomic research involving Indigenous communities raises unique ethical considerations. Historically, some studies collected samples from Indigenous populations without adequate consent, shared data without community oversight, or published findings that conflicted with community values. Increasingly, Indigenous communities are asserting **data sovereignty**—the principle that genomic data from a community belongs to and should be governed by that community. Frameworks such as the CARE Principles for Indigenous Data Governance (Collective benefit, Authority to control, Responsibility, Ethics) complement the FAIR data principles (Findable, Accessible, Interoperable, Reusable) to ensure that genomic research benefits the communities from which data are derived.

19.12.5 Equitable Access to Sequencing Technologies

Despite dramatic cost reductions, access to sequencing technologies remains unevenly distributed globally:

- Many low- and middle-income countries lack the sequencing infrastructure, trained personnel, and computational resources to implement clinical genomics.
- Genomic databases and reference panels are heavily biased toward individuals of European ancestry, reducing the clinical utility of sequencing in underrepresented populations.
- The cost of interpretation (genetic counselling, bioinformatics expertise) remains high even as the cost of sequencing itself decreases.

Addressing these disparities requires investment in local sequencing capacity, training programs, diverse reference panels (such as the HPRC), and open-source computational tools.

19.13 Future Outlook: Where Is the Field Heading?

Several trends are likely to shape the sequencing field over the coming decade:

- **Sequencing as a clinical default:** Whole-genome sequencing will increasingly replace targeted panels as the first-line clinical test for genetic diseases, cancer profiling, and prenatal screening, enabled by sub-\$100 sequencing costs and faster turnaround times.
- **Multionics as standard practice:** Single-cell and spatial multionics will become routine rather than specialized, as commercial platforms reduce costs and simplify workflows.
- **AI-native analysis:** Foundation models will become the backbone of genomic analysis, enabling zero-shot variant interpretation, cell-type annotation, and treatment prediction without task-specific training data.

- **Real-time clinical genomics:** The combination of rapid sequencing (ONT, ultrafast Illumina protocols), hardware acceleration (DRAGEN, Parabricks), and cloud computing will enable same-day genomic diagnosis in critical care settings.
- **Pangenome adoption:** Graph-based pangenome references will gradually replace linear references as the default for variant calling and genome analysis, reducing reference bias and improving accuracy for all populations.
- **Direct protein and metabolite sequencing:** As protein sequencing technologies (Quantum-Si, Nautilus) mature, integrated nucleic acid + protein + metabolite profiling from the same sample will become feasible.
- **Decentralized sequencing:** Portable and point-of-care sequencing will expand beyond research into routine clinical, agricultural, and environmental monitoring applications, making sequencing as commonplace as PCR testing.

19.14 Practical Workflow: Pangenome Alignment

The following workflow examples illustrate alignment of short reads to a pangenome graph reference, representing the shift from linear reference-based analysis to graph-based approaches.

19.14.1 Snakemake Workflow

Listing 19.3: Snakemake workflow for pangenome-based variant calling using vg giraffe.

```

1 # Snakefile for Pangenome Variant Calling with vg
2 configfile: "config/pangenome_config.yaml"
3
4 SAMPLES = config["samples"]
5 GRAPH = config["pangenome_gbz"] # .gbz graph index
6 DIST = config["pangenome_dist"] # .dist distance index
7 MINIM = config["pangenome_min"] # .min minimizer index
8 REF_FASTA = config["linear_ref"] # for surjection
9
10 rule all:
11     input:
12         expand("results/{sample}/deepvariant/"
13             "{sample}.vcf.gz", sample=SAMPLES),
14         "results/combined/multisample.vcf.gz"
15
16 rule vg_giraffe_map:
17     """Map reads to pangenome graph with vg giraffe."""
18     input:
19         r1="data/{sample}_R1.fastq.gz",
20         r2="data/{sample}_R2.fastq.gz"
21     output:
22         gam="results/{sample}/mapped/{sample}.gam"
23     params:
24         gbz=GRAPH,
25         dist=DIST,
26         minim=MINIM
27     threads: 16
28     resources:
29         mem_mb=32000
30     shell:
31         """
32         vg giraffe \

```

```

33         -Z {params.gbz} \
34         -m {params.minim} \
35         -d {params.dist} \
36         -f {input.r1} -f {input.r2} \
37         -t {threads} \
38         --output-format gam \
39         > {output.gam}
40     """
41
42 rule surject_to_bam:
43     """Project graph alignments to linear reference."""
44     input:
45         gam="results/{sample}/mapped/{sample}.gam"
46     output:
47         bam="results/{sample}/mapped/{sample}.sorted.bam",
48         bai="results/{sample}/mapped/{sample}.sorted.bam.bai"
49     params:
50         gbz=GRAPH
51     threads: 8
52     resources:
53         mem_mb=16000
54     shell:
55         """
56         vg surject -x {params.gbz} \
57             -t {threads} -b \
58             {input.gam} \
59             | samtools sort -@ 4 -o {output.bam}
60         samtools index {output.bam}
61         """
62
63 rule deepvariant:
64     """Call variants with DeepVariant."""
65     input:
66         bam="results/{sample}/mapped/{sample}.sorted.bam",
67         bai="results/{sample}/mapped/{sample}.sorted.bam.bai",
68         ref=REF_FASTA
69     output:
70         vcf="results/{sample}/deepvariant/"
71             "{sample}.vcf.gz"
72     threads: 16
73     resources:
74         mem_mb=64000
75     singularity:
76         "docker://google/deepvariant:latest"
77     shell:
78         """
79         /opt/deepvariant/bin/run_deepvariant \
80             --model_type=WGS \
81             --ref={input.ref} \
82             --reads={input.bam} \
83             --output_vcf={output.vcf} \
84             --num_shards={threads}
85         """
86
87 rule merge_vcfs:
88     """Merge per-sample VCFs into a multi-sample VCF."""
89     input:
90         vcfs=expand("results/{sample}/deepvariant/"

```

```

91         "{sample}.vcf.gz", sample=SAMPLES)
92     output:
93         merged="results/combined/multisample.vcf.gz"
94     threads: 4
95     shell:
96         """
97         bcftools merge {input.vcfs} \
98             --threads {threads} \
99             -Oz -o {output.merged}
100         tabix -p vcf {output.merged}
101         """

```

19.14.2 Nextflow Workflow

Listing 19.4: Nextflow workflow for pangenome-based variant calling.

```

1  #!/usr/bin/env nextflow
2  nextflow.enable.dsl=2
3
4  params.samples      = "data/samples.csv"
5  params.pangenome_gbz = "/ref/hprc-v1.1-mc-grch38.gbz"
6  params.pangenome_dist = "/ref/hprc-v1.1-mc-grch38.dist"
7  params.pangenome_min  = "/ref/hprc-v1.1-mc-grch38.min"
8  params.linear_ref     = "/ref/GRCh38.fa"
9  params.outdir        = "results"
10
11  // Read sample sheet: sample_id, r1, r2
12  Channel
13      .fromPath(params.samples)
14      .splitCsv(header: true)
15      .map { row -> tuple(row.sample_id,
16                          file(row.r1),
17                          file(row.r2)) }
18      .set { sample_ch }
19
20  process VG_GIRAFFE {
21      tag "$sample_id"
22      cpus 16
23      memory '32 GB'
24      container 'quay.io/vgteam/vg:latest'
25      publishDir "${params.outdir}/${sample_id}/mapped",
26                  mode: 'copy'
27
28      input:
29      tuple val(sample_id), path(r1), path(r2)
30
31      output:
32      tuple val(sample_id), path("${sample_id}.gam"),
33          emit: gam
34
35      script:
36      """
37      vg giraffe \
38          -Z ${params.pangenome_gbz} \
39          -m ${params.pangenome_min} \
40          -d ${params.pangenome_dist} \
41          -f ${r1} -f ${r2} \

```

```

42     -t ${task.cpus} \
43     --output-format gam \
44     > ${sample_id}.gam
45     ""
46 }
47
48 process SURJECT_TO_BAM {
49     tag "${sample_id}"
50     cpus 8
51     memory '16 GB'
52     container 'quay.io/vgteam/vg:latest'
53
54     input:
55     tuple val(sample_id), path(gam)
56
57     output:
58     tuple val(sample_id),
59         path("${sample_id}.sorted.bam"),
60         path("${sample_id}.sorted.bam.bai"),
61         emit: bam
62
63     script:
64     ""
65     vg surject -x ${params.pangenome_gbz} \
66         -t ${task.cpus} -b ${gam} \
67     | samtools sort -@ 4 \
68         -o ${sample_id}.sorted.bam
69     samtools index ${sample_id}.sorted.bam
70     ""
71 }
72
73 process DEEPVARIANT {
74     tag "${sample_id}"
75     cpus 16
76     memory '64 GB'
77     container 'google/deepvariant:latest'
78     publishDir "${params.outdir}/${sample_id}/variants",
79         mode: 'copy'
80
81     input:
82     tuple val(sample_id), path(bam), path(bai)
83
84     output:
85     tuple val(sample_id),
86         path("${sample_id}.vcf.gz"),
87         emit: vcf
88
89     script:
90     ""
91     /opt/deepvariant/bin/run_deepvariant \
92         --model_type=WGS \
93         --ref=${params.linear_ref} \
94         --reads=${bam} \
95         --output_vcf=${sample_id}.vcf.gz \
96         --num_shards=${task.cpus}
97     ""
98 }
99

```

```

100 process MERGE_VCFS {
101   cpus 4
102   memory '8 GB'
103   publishDir "${params.outdir}/combined", mode: 'copy'
104
105   input:
106   path(vcfs)
107
108   output:
109   path("multisample.vcf.gz"), emit: merged
110   path("multisample.vcf.gz.tbi"), emit: index
111
112   script:
113   """
114   bcftools merge ${vcfs} \
115     --threads ${task.cpus} \
116     -Oz -o multisample.vcf.gz
117   tabix -p vcf multisample.vcf.gz
118   """
119 }
120
121 workflow {
122   VG_GIRAFFE(sample_ch)
123   SURJECT_TO_BAM(VG_GIRAFFE.out.gam)
124   DEEPVARIANT(SURJECT_TO_BAM.out.bam)
125
126   DEEPVARIANT.out.vcf
127     .map { sample_id, vcf -> vcf }
128     .collect()
129     .set { all_vcfs }
130
131   MERGE_VCFS(all_vcfs)
132 }

```

19.15 Visualising the Evolving Technology Landscape

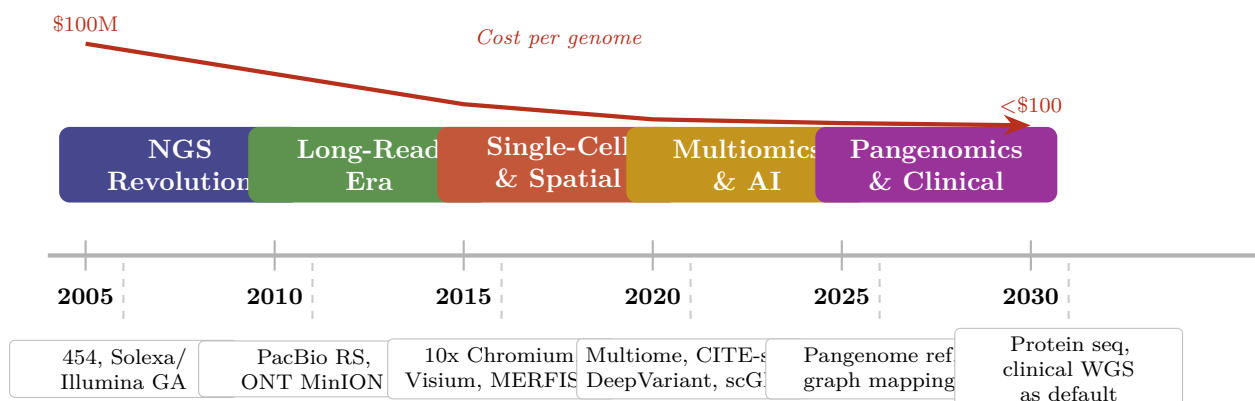


Figure 19.1: Timeline of the sequencing technology landscape, showing major eras, representative technologies, and the continuing decline in per-genome cost. Each era builds upon the previous ones; earlier technologies do not disappear but are augmented by newer capabilities.

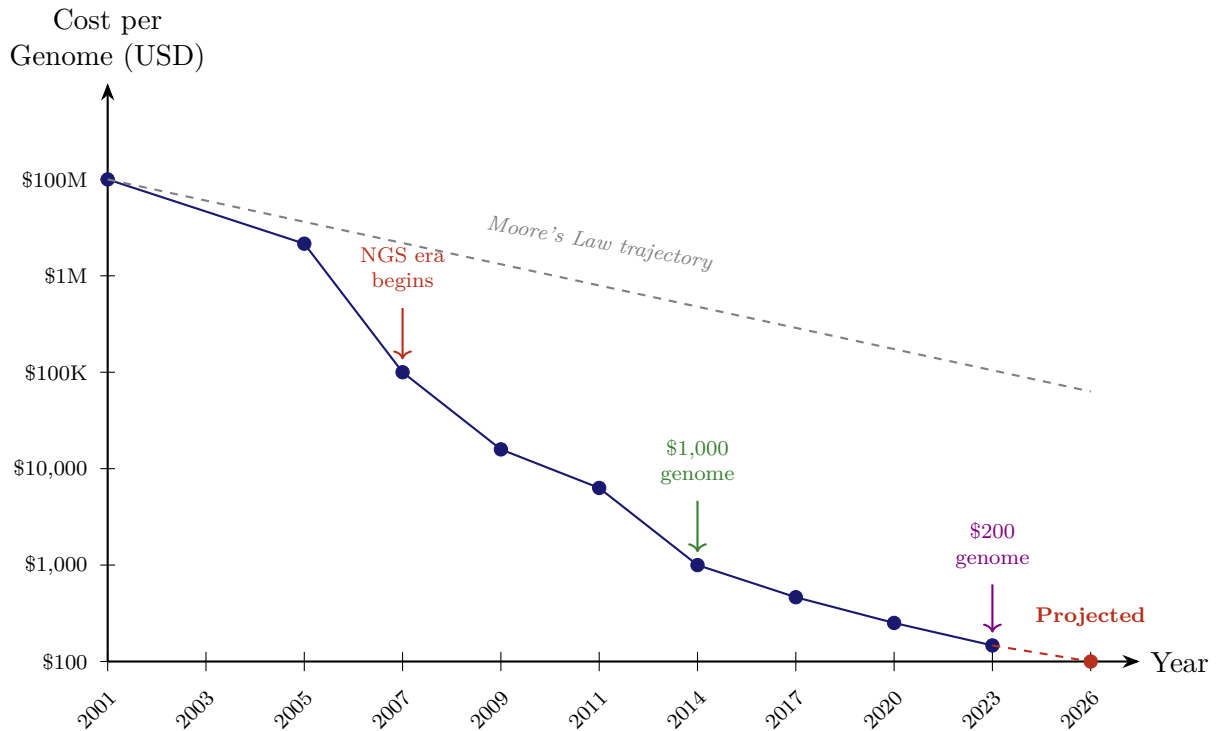


Figure 19.2: The cost of sequencing a human genome from 2001 to 2026 (projected), shown on a logarithmic scale. The precipitous drop beginning around 2007 coincides with the introduction of NGS technologies. The decline has consistently outpaced Moore’s Law. The dashed segment indicates projected costs. Data based on NHGRI estimates.

19.16 Summary

This chapter has surveyed the frontiers of sequencing technology and the broader genomics landscape. Key takeaways include:

- **Protein sequencing** technologies (Quantum-Si, Nautilus) are bringing sequencing-like approaches to the proteome, promising integrated multi-molecular profiling.
- **In situ sequencing** (STARmap, ExSeq) reads sequences directly within intact tissues at subcellular resolution, merging the spatial and molecular information domains.
- **Synthetic long reads** and linked-read technologies (TELL-seq, stLFR) extract long-range information from short-read platforms, providing a cost-effective alternative to true long-read sequencing for some applications.
- **Ultra-long nanopore reads** (>4 Mb) enable assembly of the most complex genomic regions, including centromeres and large structural variants.
- **AI and machine learning** have transformed basecalling (Dorado, DeepConsensus), variant calling (DeepVariant), and biological interpretation (genomic language models, single-cell foundation models such as scGPT and Geneformer).
- **Cloud computing and GPU acceleration** (Terra, DRAGEN, Parabricks) are making large-scale genomic analysis accessible and fast, while workflow managers (Nextflow Tower/Sequera) provide the orchestration layer.
- **Portable sequencing** has taken genomics to the field—from Ebola outbreak zones to the International Space Station—and is enabling rapid clinical diagnosis in hours rather

than days.

- **Pangenomics** and graph-based references (vg, minigraph) are replacing the single linear reference genome paradigm, reducing reference bias and improving accuracy for all populations.
- **Ethical considerations**—genetic privacy, equitable access, data sovereignty, and responsible use of genomic information—are as important as the technologies themselves and must be actively addressed by the genomics community.

The sequencing revolution that began with Sanger’s dideoxy method in 1977 shows no signs of slowing. If anything, the pace of innovation is accelerating, driven by the convergence of molecular biology, engineering, and artificial intelligence. The researchers and clinicians who will shape the next decade of genomics are those who combine deep understanding of the biological questions with fluency in the rapidly evolving technological landscape described throughout this book.

Part VII

Bioinformatics and Data Analysis

Chapter 20

Bioinformatics Fundamentals

Key Concept

Bioinformatics is the interdisciplinary field that develops and applies computational methods to analyse, interpret, and manage biological data—particularly the vast quantities of sequence data generated by modern NGS platforms. Mastering bioinformatics fundamentals is essential for any researcher who generates or analyses sequencing data: without appropriate computational tools and practices, even the highest-quality sequencing data cannot be translated into biological insight.

20.1 What Is Bioinformatics?

At its core, bioinformatics sits at the intersection of biology, computer science, mathematics, and statistics. It encompasses the development of algorithms and software tools for processing raw sequencing data (alignment, quality control, variant calling), the design of databases and ontologies for organising biological knowledge, and the application of statistical and machine-learning methods for extracting meaningful patterns from high-dimensional datasets.

For the genomics practitioner, bioinformatics is not a separate discipline but an integral part of every sequencing experiment. The moment reads leave the sequencer, the project becomes a computational endeavour. Understanding the computing environment, file formats, key tools, programming languages, and workflow management systems described in this chapter is therefore a prerequisite for every subsequent analysis chapter in this book.

20.2 Computing Environment Setup

20.2.1 Linux/Unix Basics for Bioinformatics

Nearly all bioinformatics software is developed for, and best supported on, Linux and Unix-like operating systems. Even researchers who use macOS or Windows on their personal machines will typically run analyses on a Linux-based high-performance computing (HPC) cluster or cloud instance. The command-line interface (CLI) is the primary means of interaction.

Essential command-line skills include:

- **Navigation:** `cd`, `ls`, `pwd`, `tree`.
- **File manipulation:** `cp`, `mv`, `rm`, `mkdir`, `ln -s` (symbolic links).
- **Text processing:** `cat`, `head`, `tail`, `less`, `wc`, `grep`, `sed`, `awk`, `cut`, `sort`, `uniq`.
- **Compression:** `gzip`/`gunzip`, `bgzip` (block-gzip for indexed access), `zcat`.

- **Pipes and redirection:** combining commands with `|`, `>`, `»`, `2>`.
- **Process management:** `top`/`htop`, `ps`, `kill`, `nohup`, `screen`/`tmux`.
- **Permissions:** `chmod`, `chown`; understanding `rw``x` triplets.
- **Environment variables:** `export`, `PATH`, `.bashrc`/`.bash_profile`.

Screen and Tmux

Use `screen` or `tmux` to create persistent terminal sessions on remote servers. If your SSH connection drops during a long-running job, the process continues safely inside the session. This is indispensable for interactive bioinformatics work on HPC clusters.

Listing 20.1: Essential Linux commands for bioinformatics.

```

1 # Count reads in a FASTQ file (4 lines per read)
2 wc -l sample.fastq | awk '{print $1/4}'
3
4 # Extract chromosome 1 alignments from a BAM file
5 samtools view -b input.bam chr1 > chr1.bam
6
7 # Sort a BED file and merge overlapping intervals
8 sort -k1,1 -k2,2n peaks.bed | bedtools merge > merged.bed
9
10 # View the first few records of a gzipped VCF
11 zcat variants.vcf.gz | grep -v "^#" | head -20
12
13 # Monitor disk usage of a project directory
14 du -sh /data/project/*

```

20.2.2 Package Managers

Installing bioinformatics software can be challenging due to complex dependency chains. Modern package managers greatly simplify this process:

Conda / Mamba

Conda is the most widely used package manager in bioinformatics. It manages both software binaries and their dependencies within isolated *environments*, preventing version conflicts between projects. The `bioconda` channel provides over 11,000 bioinformatics packages. `Mamba` is a drop-in replacement for `conda` with significantly faster dependency resolution.

Pip Python's native package manager. Used for Python-specific packages not available through Conda (e.g., niche libraries on PyPI). Best used *within* a Conda environment to avoid conflicts with system Python.

Bioconductor

The primary repository for R-based bioinformatics packages. Bioconductor provides a curated, versioned ecosystem of packages for genomic data analysis, including `DESeq2`, `edgeR`, `GenomicRanges`, and hundreds of others. Packages are installed via `BiocManager::install()`.

Listing 20.2: Setting up a Conda environment for bioinformatics.

```

1 # Install Miniforge (includes mamba)
2 curl -L -O
   https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Linux-x86_64.sh

```

```

3 bash Miniforge3-Linux-x86_64.sh
4
5 # Create an environment for an RNA-seq project
6 mamba create -n rnaseq -c conda-forge -c bioconda \
7     fastqc=0.12.1 fastp=0.23.4 star=2.7.11b \
8     subread=2.0.6 samtools=1.19 multiqc=1.21
9
10 # Activate the environment
11 mamba activate rnaseq
12
13 # Export the environment for reproducibility
14 mamba env export > environment.yml

```

20.2.3 Containers: Docker and Singularity/Apptainer

Containers provide a higher level of reproducibility than package managers by encapsulating the entire software environment—operating system libraries, tools, and configuration—into a portable, immutable image.

Docker

The most widely used container platform. Docker images are available for virtually every bioinformatics tool (e.g., on Docker Hub or Quay.io). Docker requires root privileges to run, which limits its use on shared HPC clusters.

Singularity / Apptainer

Designed for HPC environments where users do not have root access. Singularity (now rebranded as Apptainer) can run Docker images directly without modification. Most HPC centres support Singularity/Apptainer.

Listing 20.3: Using containers for bioinformatics.

```

1 # Pull and run a Docker image for FastQC
2 docker pull biocontainers/fastqc:v0.12.1
3 docker run --rm -v $(pwd):/data biocontainers/fastqc:v0.12.1 \
4     fastqc /data/sample_R1.fastq.gz /data/sample_R2.fastq.gz
5
6 # On HPC: convert Docker image to Singularity/Apptainer
7 singularity pull docker://biocontainers/fastqc:v0.12.1
8 singularity exec fastqc_v0.12.1.sif fastqc sample_R1.fastq.gz
9
10 # BioContainers provides images for most bioinformatics tools
11 # See: https://biocontainers.pro

```

Containers for Reproducibility

A Conda environment specifies what software to install; a container specifies the exact binary files, libraries, and operating system state. Containers provide “bit-for-bit” reproducibility—the same container will produce identical results on any machine, even years later. For published analyses, containers are the gold standard for computational reproducibility.

20.2.4 HPC Clusters: SLURM, PBS/Torque, SGE

Most bioinformatics analyses require computational resources beyond what a laptop or desktop can provide. High-performance computing (HPC) clusters provide shared pools of CPUs,

RAM, and storage. Jobs are submitted through a **workload manager** (also called a job scheduler):

SLURM (Simple Linux Utility for Resource Management)

The most widely deployed scheduler in academic HPC. Jobs are submitted with `sbatch`, monitored with `squeue`, and cancelled with `scancel`.

PBS/Torque

An older but still common scheduler. Jobs are submitted with `qsub` and monitored with `qstat`.

SGE (Sun Grid Engine) / Open Grid Scheduler

Legacy scheduler found in some older installations. Also uses `qsub/qstat` syntax.

Listing 20.4: SLURM job submission for a bioinformatics task.

```
1 #!/bin/bash
2 #SBATCH --job-name=bwa_align
3 #SBATCH --output=logs/bwa_%j.out
4 #SBATCH --error=logs/bwa_%j.err
5 #SBATCH --cpus-per-task=16
6 #SBATCH --mem=32G
7 #SBATCH --time=06:00:00
8 #SBATCH --partition=standard
9
10 # Load modules or activate environment
11 source activate wgs_pipeline
12
13 # Run BWA-MEM2 alignment
14 bwa-mem2 mem -t 16 \
15 /ref/GRCh38.fa \
16 sample_R1.fastq.gz sample_R2.fastq.gz \
17 | samtools sort -@ 4 -o sample.sorted.bam -
18
19 samtools index sample.sorted.bam
```

20.2.5 Cloud Computing for Genomics

Cloud computing offers elastic, on-demand computational resources without the capital investment of purchasing hardware. The three major cloud providers each offer genomics-specific services:

- **Amazon Web Services (AWS):** AWS HealthOmics for managed genomics workflows; S3 for storage; EC2 for compute; AWS Batch for job scheduling. The Registry of Open Data on AWS hosts public genomics datasets (1000 Genomes, TCGA, gnomAD).
- **Google Cloud Platform (GCP):** Google Cloud Life Sciences API; BigQuery for large-scale variant analysis; Terra platform (Broad Institute) for WDL-based workflows running on GCP.
- **Microsoft Azure:** Azure Genomics; Cromwell on Azure for WDL workflows; integration with Microsoft's healthcare data solutions.

Cloud Cost Management

Cloud computing can become expensive if not carefully managed. A single misconfigured pipeline can generate thousands of dollars in compute charges within hours. Always set budget alerts, use spot/preemptible instances for fault-tolerant workloads (50–80% cost savings), and shut down instances when not in use. Estimate costs before running large analyses using the provider’s pricing calculators.

20.3 Bioinformatics File Formats

A thorough understanding of file formats is essential for working with sequencing data. Each format serves a specific purpose in the analysis pipeline, and knowing their structure helps in debugging problems and writing custom analyses.

20.3.1 FASTA: Reference Sequences

The FASTA format is one of the simplest and oldest bioinformatics formats, used to store nucleotide or protein sequences. Each record consists of a header line beginning with > followed by one or more lines of sequence:

Listing 20.5: FASTA format example.

```
>chr1 Homo sapiens chromosome 1
NNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNN
NNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNGATAATCTACTACTACTCAG
TGGCCAAACCTGATCCTGACAGCTGGAGTAAGGAACCTGAAGTCCCTA
AAACTCATCAATGTTCTTTAGAGACTTACCAGGACCACTTCGTGAGGGA
>chr2 Homo sapiens chromosome 2
NNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNN
AGAGATCAGCTCAGGAGAGTCTCTTGAAGAAATCTGATTCACTGTATGG
```

Reference genomes (e.g., GRCh38/hg38 for human) are distributed in FASTA format. For efficient random access, FASTA files are indexed with `samtools faidx`, producing a `.fai` index file. Many tools also require a sequence dictionary (`.dict`) created with Picard or `samtools`.

20.3.2 FASTQ: Raw Sequencing Reads

FASTQ is the universal output format for sequencing instruments. Each read consists of exactly four lines:

Listing 20.6: FASTQ format example with Phred quality encoding.

@NB501234:45:H3YTNBGX3:1:11101:24563:1038 1:N:O:ACGTACGT
NTGCAAGCAGTTTCAGGATCAGTCGAGACTTCAATGTCGATCTACGTAGC
+
#AAFFJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJ

Line 1 (@)

Read identifier containing instrument name, run ID, flow cell coordinates, and index sequence. Paired reads share the same ID but differ in the 1: or 2: field.

Line 2

The nucleotide sequence (A, C, G, T, N).

Line 3 (+)

A separator line (sometimes repeats the read ID, but usually just +).

Line 4

ASCII-encoded quality scores, one character per base. Each character encodes a **Phred quality score**: $Q = -10 \log_{10}(P_{\text{error}})$.

Phred Quality Scores

Phred scores encode the probability of a base call being incorrect:

Phred Score	Error Probability	Accuracy
10	1 in 10	90%
20	1 in 100	99%
30	1 in 1,000	99.9%
40	1 in 10,000	99.99%

Modern Illumina instruments use the **Phred+33** encoding: the ASCII character code minus 33 gives the quality score. For example, the character J has ASCII code 74, so $Q = 74 - 33 = 41$. The character # has ASCII code 35, so $Q = 35 - 33 = 2$ (very low quality).

FASTQ files are almost always gzip-compressed (`.fastq.gz`) because raw data is enormous—a single 30× human genome generates approximately 90 Gb of compressed FASTQ data.

20.3.3 SAM/BAM/CRAM: Alignment Files

The Sequence Alignment/Map (SAM) format and its binary counterpart BAM store read alignments against a reference genome. CRAM is a more compressed alternative that uses reference-based compression.

Each alignment record contains the following key fields:

Table 20.1: Key fields in SAM/BAM alignment records.

Field	Description
QNAME	Read name (query name). Identical for both reads of a pair.
FLAG	Bitwise flag encoding read properties: paired, properly paired, unmapped, mate unmapped, reverse strand, first/second in pair, duplicate, supplementary, secondary.
RNAME	Reference sequence name (chromosome) to which the read is aligned.
POS	1-based leftmost mapping position on the reference.
MAPQ	Mapping quality score (Phred-scaled probability that the mapping position is wrong). MAPQ 0 means the read maps equally well to multiple locations.
CIGAR	Compact string describing the alignment: M (match/mismatch), I (insertion to reference), D (deletion from reference), S (soft clipping), H (hard clipping), N (skipped region, e.g., intron in RNA-seq). Example: 76M2I50M3S means 76 matched bases, 2-bp insertion, 50 matched bases, 3 soft-clipped bases.
RNEXT	Reference name of the mate's alignment (for paired-end reads).
PNEXT	Position of the mate's alignment.
TLEN	Observed template length (insert size).

Field	Description
SEQ	Read sequence.
QUAL	Base quality string (same encoding as FASTQ).

Listing 20.7: SAM format example (two paired-end reads).

```

1 @HD VN:1.6 SO:coordinate
2 @SQ SN:chr1 LN:248956422
3 @RG ID:sample1 SM:sample1 PL:ILLUMINA
4 read001 99 chr1 10050 60 151M = 10250 351 ACGTACGT... JJJJJJJJ...
5 read001 147 chr1 10250 60 151M = 10050 -351 TGCATGCA... JJJJJJJJ...
```

SAM FLAG Field

The FLAG field is a bitwise integer that encodes multiple properties simultaneously. Use `samtools flags` to decode a FLAG value, or the SAM flag decoder at <https://broadinstitute.github.io/picard/explain-flags.html>. Common FLAG values: 99 (paired, properly paired, mate on reverse strand, first in pair), 147 (paired, properly paired, read on reverse strand, second in pair), 4 (unmapped).

BAM files are always sorted (by coordinate or by name) and indexed (`.bai` or `.csi` index) for efficient random access. CRAM achieves 30–60% smaller file sizes than BAM by storing only the differences from the reference.

20.3.4 BED: Genomic Intervals

BED (Browser Extensible Data) format represents genomic intervals with a simple tab-delimited structure. Coordinates are **0-based, half-open**: the start position is 0-based and inclusive, the end position is exclusive.

Listing 20.8: BED format variants (BED3, BED6, BED12).

```

1 # BED3: chromosome, start, end
2 chr1 1000 2000
3 chr1 3000 4500
4
5 # BED6: adds name, score, strand
6 chr1 1000 2000 peak1 500 +
7 chr1 3000 4500 peak2 300 -
8
9 # BED12: adds thickStart, thickEnd, itemRgb, blockCount,
10 #      blockSizes, blockStarts (used for transcript models)
11 chr1 11873 14409 NR_046018 0 + 11873 11873 0 3 354,109,1189, 0,739,1347,
```

Coordinate Systems

BED uses 0-based, half-open coordinates. SAM/BAM and VCF use 1-based, fully closed coordinates. GFF/GTF uses 1-based, fully closed coordinates. The position “chr1:1000–2000” in BED corresponds to “chr1:1001–2000” in SAM. Coordinate system mismatches are one of the most common sources of off-by-one errors in bioinformatics.

20.3.5 GFF/GTF: Gene Annotations

GFF3 (General Feature Format version 3) and GTF (Gene Transfer Format, essentially GFF2) store gene and transcript annotations. These files define the locations of genes, transcripts, exons, coding sequences, and other genomic features.

Listing 20.9: GTF format example (GENCODE).

```

1 chr1 HAVANA gene 11869 14409 . + . gene_id "ENSG00000223972";
   gene_name "DDX11L1";
2 chr1 HAVANA transcript 11869 14409 . + . gene_id "ENSG00000223972";
   transcript_id "ENST00000456328";
3 chr1 HAVANA exon 11869 12227 . + . gene_id "ENSG00000223972";
   transcript_id "ENST00000456328";
4 chr1 HAVANA exon 12613 12721 . + . gene_id "ENSG00000223972";
   transcript_id "ENST00000456328";
5 chr1 HAVANA exon 13221 14409 . + . gene_id "ENSG00000223972";
   transcript_id "ENST00000456328";

```

GENCODE annotations (the standard for human and mouse) are available in both GTF and GFF3 formats. Most RNA-seq quantification tools (STAR, featureCounts, Salmon) accept GTF.

20.3.6 VCF/BCF: Variant Calls

The Variant Call Format (VCF) stores genetic variants (SNPs, indels, structural variants). BCF is the binary, indexed version. The VCF format is critical for genomics and clinical sequencing:

Listing 20.10: VCF format example with key fields.

```

1 ##fileformat=VCFv4.3
2 ##INFO=<ID=DP,Number=1,Type=Integer,Description="Total Depth">
3 ##INFO=<ID=AF,Number=A,Type=Float,Description="Allele Frequency">
4 ##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
5 ##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
6 ##FORMAT=<ID=GQ,Number=1,Type=Integer,Description="Genotype Quality">
7 #CHROM POS ID REF ALT QUAL FILTER INFO FORMAT SAMPLE1
8 chr1 12345 rs123456 A G 99 PASS DP=50;AF=0.5 GT:DP:GQ 0/1:50:99
9 chr1 67890 . CT C 85 PASS DP=40;AF=0.3 GT:DP:GQ 0/1:40:85

```

Key VCF concepts:

- **INFO field:** site-level annotations (total depth, allele frequency, functional impact).
- **FORMAT field:** defines per-sample fields that follow.
- **Genotype (GT):** 0/0 = homozygous reference, 0/1 = heterozygous, 1/1 = homozygous alternate, ./ = missing.
- **FILTER:** PASS if the variant passes all quality filters; otherwise a semicolon-separated list of filter names.

20.3.7 BigWig and BedGraph: Coverage Tracks

BigWig and BedGraph files represent continuous signal tracks across the genome, such as read coverage depth or ChIP-seq signal intensity. BedGraph is a text format; BigWig is a compressed, indexed binary format suitable for genome browser visualization. BigWig files are created from BAM files using tools like `deepTools bamCoverage` or `bedtools genomecov`.

20.3.8 HDF5 and AnnData (h5ad): Single-Cell Data

Single-cell datasets are stored in HDF5-based formats. The **.h5ad** format (AnnData) is the standard for the Scanpy ecosystem in Python, storing the expression matrix, cell metadata, gene metadata, dimensionality reductions, and graphs in a single file. The Seurat ecosystem in R uses **.rds** (serialized R objects) or the newer **.h5Seurat** format. Loom (**.loom**) is another HDF5-based format used by some tools.

20.3.9 Hi-C Contact Matrices: Pairs, Cool, and .hic

Hi-C data (3D genome organization) uses specialized formats:

- **.pairs**: Text format listing pairs of interacting genomic loci, defined by the 4DN consortium.
- **.cool** / **.mcool**: HDF5-based format for contact matrices at one or multiple resolutions, used by the **cooler** library.
- **.hic**: Binary format used by **Juicer** and **JuiceBox** for contact matrix storage and visualization.

20.3.10 10x Genomics MEX Format

The Market Exchange (MEX) format from Cell Ranger consists of three files stored together in a directory:

- **matrix.mtx.gz**: Sparse matrix in Matrix Market format (rows = genes, columns = barcodes).
- **barcodes.tsv.gz**: Cell barcodes (one per line).
- **features.tsv.gz**: Gene/feature identifiers and names.

This format is read by **Scanpy** (**sc.read_10x_mtx()**), **Seurat** (**Read10X()**), and other single-cell tools.

20.4 Key Bioinformatics Tools

20.4.1 samtools: The Swiss Army Knife of BAM Files

samtools is the most fundamental bioinformatics tool, providing operations on SAM/BAM/CRAM files:

Listing 20.11: Essential samtools commands.

```

1 # View BAM header
2 samtools view -H sample.bam
3
4 # Convert SAM to sorted BAM
5 samtools sort -@ 8 -o sample.sorted.bam sample.sam
6
7 # Index a sorted BAM file
8 samtools index sample.sorted.bam
9
10 # View alignments in a region
11 samtools view sample.sorted.bam chr1:1000000-2000000
12
13 # Filter: keep only properly paired, uniquely mapped reads (MAPQ >= 30)
14 samtools view -b -f 2 -q 30 sample.sorted.bam > filtered.bam

```

```

15
16 # Generate alignment statistics
17 samtools flagstat sample.sorted.bam
18 samtools idxstats sample.sorted.bam
19 samtools stats sample.sorted.bam > sample.stats.txt
20
21 # Calculate per-base depth
22 samtools depth -a sample.sorted.bam > depth.txt
23
24 # Merge multiple BAM files
25 samtools merge merged.bam sample1.bam sample2.bam sample3.bam

```

20.4.2 bedtools: Interval Arithmetic

bedtools provides a comprehensive suite of operations for manipulating genomic intervals:

Listing 20.12: Common bedtools operations.

```

1 # Intersect peaks with gene promoters
2 bedtools intersect -a peaks.bed -b promoters.bed -wa -wb > overlap.bed
3
4 # Find peaks that do NOT overlap with blacklist regions
5 bedtools intersect -a peaks.bed -b blacklist.bed -v > clean_peaks.bed
6
7 # Merge overlapping intervals
8 bedtools merge -i sorted_peaks.bed -d 100 > merged.bed
9
10 # Find closest gene to each peak
11 bedtools closest -a peaks.bed -b genes.bed -d > closest.bed
12
13 # Compute genome-wide coverage from a BAM file
14 bedtools genomecov -ibam sample.bam -bg > coverage.bedgraph
15
16 # Generate windows across the genome
17 bedtools makewindows -g genome.sizes -w 10000 > windows.bed
18
19 # Compute Jaccard similarity between two sets of intervals
20 bedtools jaccard -a set1.bed -b set2.bed

```

20.4.3 Picard: Duplicate Marking and Metrics

Picard (Broad Institute) provides utilities for BAM manipulation, with MarkDuplicates being the most commonly used:

Listing 20.13: Picard MarkDuplicates and metrics collection.

```

1 # Mark PCR duplicates
2 java -jar picard.jar MarkDuplicates \
3   I=sample.sorted.bam \
4   O=sample.dedup.bam \
5   M=sample.dup_metrics.txt \
6   REMOVE_DUPLICATES=false
7
8 # Collect alignment summary metrics
9 java -jar picard.jar CollectAlignmentSummaryMetrics \
10   R=reference.fa I=sample.dedup.bam O=alignment_metrics.txt
11
12 # Collect insert size distribution

```

```

13 java -jar picard.jar CollectInsertSizeMetrics \
14     I=sample.dedup.bam O=insert_metrics.txt H=insert_hist.pdf

```

20.4.4 IGV: Visual Inspection

The Integrative Genomics Viewer (IGV) is an essential desktop application for visually inspecting alignments, variants, and other genomic features. Load BAM files, VCF files, BED files, and BigWig tracks to examine specific loci. Visual inspection is critical for validating computational results—many false-positive variant calls or artefacts become obvious upon manual review in IGV.

Always Inspect Key Results in IGV

Before reporting important findings—a novel variant, an unexpected peak, a suspicious fusion event—take five minutes to look at the raw data in IGV. Computational tools can make systematic errors that are immediately obvious to a human viewer. Journals and reviewers increasingly expect IGV screenshots for key findings.

20.4.5 deepTools: Coverage and Visualization

deepTools provides powerful commands for computing and visualizing coverage from BAM files:

Listing 20.14: deepTools for coverage analysis and visualization.

```

1  # Generate normalized coverage track (RPKM)
2  bamCoverage -b sample.bam -o sample.bw \
3      --normalizeUsing RPKM --binSize 10 -p 8
4
5  # Compute correlation between replicates
6  multiBamSummary bins -b rep1.bam rep2.bam rep3.bam \
7      -o results.npz -p 8
8  plotCorrelation -in results.npz --corMethod spearman \
9      -o correlation_heatmap.pdf --plotFileFormat pdf
10
11 # Generate heatmap of signal around TSS
12 computeMatrix reference-point -S chip.bw input.bw \
13     -R genes.bed -a 3000 -b 3000 -o matrix.gz
14 plotHeatmap -m matrix.gz -o heatmap.pdf

```

20.4.6 MultiQC: Aggregating QC Reports

MultiQC scans a project directory for output files from dozens of bioinformatics tools (FastQC, STAR, samtools, Picard, featureCounts, etc.) and compiles them into a single interactive HTML report. This is invaluable for identifying outlier samples and systematic quality issues across a cohort:

Listing 20.15: Running MultiQC to aggregate quality reports.

```

1  # Run MultiQC on an entire project directory
2  multiqc /project/results/ -o /project/multiqc_report/
3
4  # Run with specific module selection and title
5  multiqc . --module fastqc --module star --module picard \
6      --title "RNA-seq QC Report" -o qc_report/

```

20.5 Algorithms and Data Structures for Sequence Analysis

Understanding the algorithmic foundations of sequence alignment and search is essential for choosing the right tool for a given problem and for interpreting performance characteristics. This section covers the key data structures and algorithms that underpin modern read aligners and sequence analysis tools.

20.5.1 Suffix Arrays

Suffix Array

A **suffix array** (SA) of a string T of length n is an array of integers that specifies the lexicographic ordering of all n suffixes of T . Formally, $\text{SA}[i] = j$ means that the j -th suffix (i.e., $T[j..n]$) is the i -th smallest suffix in lexicographic order.

Given a reference genome T of length n , the suffix array enables exact string matching in $O(m \log n)$ time for a pattern of length m , using binary search over the sorted suffixes. More advanced techniques reduce this to $O(m + \log n)$ using the longest common prefix (LCP) array.

Suffix arrays can be constructed in $O(n \log n)$ time using the prefix-doubling algorithm, or in $O(n)$ time using the SA-IS (Suffix Array by Induced Sorting) algorithm. In practice, the SA-IS algorithm is preferred for genome-scale strings:

$$\text{SA construction: } O(n) \text{ time, } O(n) \text{ space (SA-IS algorithm)} \quad (20.1)$$

Suffix arrays are closely related to the Burrows-Wheeler Transform: given the suffix array, the BWT can be computed in $O(n)$ time, and vice versa. This relationship is exploited by read aligners such as BWA and Bowtie2.

20.5.2 Burrows-Wheeler Transform and FM-Index

Burrows-Wheeler Transform

The **Burrows-Wheeler Transform** (BWT) of a string T is constructed by:

1. Appending a sentinel character $\$$ (lexicographically smaller than all others) to T .
2. Generating all cyclic rotations of the string.
3. Sorting these rotations lexicographically.
4. Taking the **last column** of the sorted rotation matrix.

The resulting string $\text{BWT}(T)$ is a permutation of T that is highly compressible because characters from similar sequence contexts are grouped together.

The BWT has a remarkable property called the **LF mapping** (Last-to-First mapping): the i -th occurrence of character c in the last column corresponds to the i -th occurrence of c in the first column. Since the first column is simply the sorted characters of T , it can be represented implicitly. The LF mapping enables navigation through the original string without storing it explicitly.

The **FM-index** (Full-text Minute-space index) combines the BWT with auxiliary data structures—the **C array** (cumulative character counts) and the **Occ table** (occurrence counts of each character up to each position in the BWT)—to support exact pattern matching in $O(m)$ time, where m is the pattern length:

$$\text{Backward search step: } \text{top} = C[c] + \text{Occ}(c, \text{top} - 1) + 1 \quad (20.2)$$

$$\text{bot} = C[c] + \text{Occ}(c, \text{bot}) \quad (20.3)$$

Starting from the last character of the query pattern and proceeding backwards, each step narrows the suffix array interval $[\text{top}, \text{bot})$ to those suffixes that match an increasingly longer suffix of the pattern. When $\text{top} \geq \text{bot}$, the pattern does not occur in T ; otherwise, the number of occurrences is $\text{bot} - \text{top}$.

Why BWA-MEM Uses the FM-Index

BWA-MEM and BWA-MEM2 use the FM-index to find exact matches (seeds) between the read and the reference genome in $O(m)$ time per seed, independent of genome size. The FM-index of the human genome (~ 3 Gb) requires approximately 3–5 GB of RAM, making it practical for commodity hardware. Without the FM-index, naive string searching would require $O(nm)$ time per read, which is infeasible for billions of reads against a multi-gigabase genome.

20.5.3 Smith-Waterman vs Seed-and-Extend

Smith-Waterman Local Alignment

The **Smith-Waterman** (SW) algorithm finds the optimal local alignment between two sequences using dynamic programming. Given a query of length m and a reference of length n , the recurrence is:

$$H(i, j) = \max \begin{cases} 0 \\ H(i-1, j-1) + s(a_i, b_j) \\ H(i-1, j) - d \\ H(i, j-1) - d \end{cases} \quad (20.4)$$

where $s(a_i, b_j)$ is the substitution score (positive for match, negative for mismatch) and d is the gap penalty. The algorithm runs in $O(mn)$ time and $O(mn)$ space (reducible to $O(m)$ space with traceback modifications).

For aligning a single 150 bp read against the 3 Gb human genome, Smith-Waterman would require $\sim 150 \times 3 \times 10^9 = 4.5 \times 10^{11}$ operations per read—clearly impractical for millions of reads. Even with SIMD-accelerated implementations, full Smith-Waterman against a whole genome is too slow for routine alignment.

Seed-and-Extend Strategy

The **seed-and-extend** paradigm makes alignment tractable for whole genomes:

1. **Seeding:** Find exact k -mer matches (seeds) between the read and the reference using an index (FM-index, hash table, or minimizer index). Seeds are short exact matches that identify candidate alignment locations.
2. **Chaining:** Group co-linear seeds that are consistent with a single alignment (same strand, approximately correct distance apart). This step identifies the most promising candidate regions.
3. **Extension:** Perform banded Smith-Waterman (or similar DP) alignment only in a narrow band around the chained seeds. Banded alignment restricts the DP matrix to a diagonal band of width w , reducing time complexity to $O(mw)$ where $w \ll n$.

BWA-MEM2

Uses FM-index-based seeding with super-maximal exact matches (SMEMs), followed by Smith-Waterman extension with affine gap penalties. **BWA-MEM2** additionally uses AVX-512 SIMD instructions for 2–3× speedup over the original **BWA-MEM**.

minimap2

Uses a minimizer-based index (rather than an FM-index) for seeding, followed by chaining with a gap-cost-aware dynamic programming algorithm, and base-level alignment with the KSW2 library. **minimap2** supports both short reads and long reads (PacBio, Oxford Nanopore) by adjusting seeding and chaining parameters.

20.5.4 Algorithm Complexity Reference

Table 20.2: Time and space complexity of key sequence analysis algorithms.

Algorithm	Time	Space	Used By
BWT construction (SA-IS)	$O(n)$	$O(n)$	BWA, Bowtie2 (index build)
FM-index query	$O(m)$ per pattern	$O(n)$	BWA-MEM, BWA-MEM2, Bowtie2
Suffix array (binary search)	$O(m \log n)$ per query	$O(n)$	Suffix array-based tools
Smith-Waterman (full DP)	$O(mn)$	$O(mn)$	SSW library, Parasail, BLAST
Banded Smith-Waterman	$O(mw)$	$O(mw)$	BWA-MEM2, minimap2 (extension)
Seed-and-extend (overall)	$O(m + \text{hits} \cdot w)$	$O(n)$ index	BWA-MEM2, minimap2, STAR
Minimizer index construction	$O(n)$	$O(n/w)$	minimap2, Kraken2, Strobealign
minimap2 chaining	$O(N \log N)$	$O(N)$	minimap2 (N = seed count)
De Bruijn graph construction	$O(n)$	$O(\Sigma ^k)$ worst	SPAdes, Velvet, Trinity (assembly)

Practical vs Theoretical Complexity

Asymptotic complexity does not capture the full performance picture. SIMD vectorisation (used by BWA-MEM2 and minimap2) can accelerate Smith-Waterman alignment by 10–50×. Cache locality, I/O bandwidth, and memory access patterns often dominate real-world runtime. For example, **minimap2** uses minimizers to reduce the index size by $\sim w$ -fold compared to a full suffix array, making it faster in practice despite similar asymptotic complexity. Always benchmark tools on representative data rather than relying solely on complexity analysis.

20.6 Programming Languages for Bioinformatics

20.6.1 Python for Bioinformatics

Python has become the dominant general-purpose programming language in bioinformatics, valued for its readability, extensive libraries, and strong community support.

Biopython

Reading/writing FASTA/FASTQ, BLAST parsing, sequence manipulation.

pysam

Python wrapper for htslib; reading/writing SAM/BAM/VCF files programmatically.

Scanpy

The standard framework for single-cell RNA-seq analysis (preprocessing, clustering, visualization).

pandas

Tabular data manipulation (metadata tables, count matrices).

NumPy / SciPy

Numerical computing, statistical tests.

matplotlib / seaborn

Publication-quality plotting.

pyGenomeTracks

Visualization of genomic tracks (BigWig, BED, Hi-C).

Listing 20.16: Python example: reading a BAM file with pysam.

```

1 import pysam
2
3 # Open BAM file and count reads per chromosome
4 bamfile = pysam.AlignmentFile("sample.sorted.bam", "rb")
5 for chrom in bamfile.references:
6     count = bamfile.count(contig=chrom)
7     print(f"{chrom}: {count} reads")
8
9 # Extract reads with MAPQ >= 30 from a specific region
10 high_qual = [read for read in bamfile.fetch("chr1", 1000000, 2000000)
11              if read.mapping_quality >= 30 and not read.is_duplicate]
12 print(f"High-quality reads in region: {len(high_qual)}")
13 bamfile.close()

```

20.6.2 R for Bioinformatics

R, through the Bioconductor ecosystem, remains the language of choice for statistical genomics and many specialized analysis workflows:

DESeq2 / edgeR / limma-voom

Differential gene expression analysis from RNA-seq count data.

GenomicRanges / GenomicAlignments

Efficient representation and manipulation of genomic intervals in R.

Seurat

Single-cell RNA-seq analysis (alternative to Scanpy).

ggplot2

The grammar of graphics—the gold standard for publication-quality statistical graphics.

clusterProfiler

Gene ontology and pathway enrichment analysis.

DiffBind

Differential binding analysis for ChIP-seq and ATAC-seq.

Listing 20.17: R example: basic DESeq2 differential expression analysis.

```

1 library(DESeq2)
2
3 # Load count matrix and sample metadata
4 counts <- read.csv("counts.csv", row.names = 1)
5 coldata <- read.csv("sample_info.csv", row.names = 1)
6
7 # Create DESeq2 dataset
8 dds <- DESeqDataSetFromMatrix(countData = counts,
9                               colData = coldata,
10                              design = ~ condition)
11
12 # Run differential expression analysis
13 dds <- DESeq(dds)
14 res <- results(dds, contrast = c("condition", "treated", "control"))
15
16 # Filter significant genes (adjusted p-value < 0.05, |log2FC| > 1)
17 sig <- subset(res, padj < 0.05 & abs(log2FoldChange) > 1)
18 cat("Significant DE genes:", nrow(sig), "\n")

```

20.6.3 When to Use Which Language

Table 20.3: Choosing between Python and R for bioinformatics tasks.

Task	Python	R (Bioconductor)
Single-cell RNA-seq	Scanpy (dominant)	Seurat (dominant)
Differential expression	pyDESeq2 (newer)	DESeq2, edgeR (gold standard)
Genomic interval operations	pybedtools, pysam	GenomicRanges
Statistical modeling	statsmodels, scipy	Built-in, extensive
Machine learning	scikit-learn, PyTorch	caret, mlr3
Workflow scripting	Excellent	Less common
Plotting	matplotlib, seaborn	ggplot2 (gold standard)
Pipeline integration	Snakemake (Python-native)	Both via CLI calls

Use Both Languages

Most bioinformatics projects benefit from using both Python and R. A common pattern: use Python (with Snakemake or Nextflow) for the upstream pipeline (alignment, counting, QC), then switch to R for the downstream statistical analysis (differential expression, enrichment) and publication-quality figures. The **reticulate** package in R and **rpy2** in Python allow cross-language communication when needed.

20.7 Workflow Management Systems

20.7.1 Why Workflow Managers?

A typical bioinformatics analysis involves dozens of sequential and parallel processing steps, each with specific software, parameters, input files, and output files. Running these steps manually is error-prone, difficult to reproduce, and impossible to scale. Workflow management systems address these challenges by providing:

- **Reproducibility:** The workflow definition is a complete, executable record of the analysis.
- **Scalability:** Automatic parallelization across samples, with support for HPC clusters and cloud.
- **Portability:** Workflows can run on laptops, clusters, or cloud with minimal modification.
- **Resumability:** If a step fails, the workflow resumes from where it left off after the issue is fixed.
- **Provenance tracking:** Automatic logging of which inputs, software versions, and parameters produced each output.

20.7.2 Snakemake

Philosophy and Core Concepts

Snakemake is a Python-based workflow engine that uses a declarative, file-centric approach inspired by GNU Make. The central abstraction is the **rule**, which defines how to create output files from input files:

- **Rules:** Each rule specifies **input**, **output**, and a **shell** command (or **run** block for Python code).
- **Wildcards:** Placeholders (e.g., {sample}) that are automatically resolved, enabling rules to apply to any sample.
- **expand():** Function that generates file lists by combining wildcards with sample lists.
- **Directed Acyclic Graph (DAG):** Snakemake infers the dependency graph from input/output relationships and executes steps in the correct order.
- **Conda integration:** Each rule can specify a Conda environment YAML file, ensuring tool versions are pinned per step.
- **Container support:** Rules can use Singularity/Apptainer containers for maximum reproducibility.
- **Cluster execution:** Native support for SLURM, PBS, SGE, and cloud (AWS, GCP) via profiles.

Complete Snakemake QC Pipeline Example

Listing 20.18: Complete Snakemake workflow for a basic FASTQ QC pipeline.

```

1 # Snakefile for basic QC pipeline
2 configfile: "config.yaml"
3
4 SAMPLES = config["samples"]
5
6 rule all:
7     input:
```

```

8      expand("results/fastqc/{sample}_{read}_fastqc.html",
9            sample=SAMPLES, read=["R1", "R2"]),
10     expand("results/fastp/{sample}_R1.trimmed.fastq.gz",
11           sample=SAMPLES),
12     "results/multiqc/multiqc_report.html"
13
14 rule fastqc_raw:
15     input:
16         "data/{sample}_{read}.fastq.gz"
17     output:
18         html="results/fastqc/{sample}_{read}_fastqc.html",
19         zip="results/fastqc/{sample}_{read}_fastqc.zip"
20     threads: 2
21     conda:
22         "envs/qc.yaml"
23     shell:
24         "fastqc -t {threads} -o results/fastqc/ {input}"
25
26 rule fastp_trim:
27     input:
28         r1="data/{sample}_R1.fastq.gz",
29         r2="data/{sample}_R2.fastq.gz"
30     output:
31         r1="results/fastp/{sample}_R1.trimmed.fastq.gz",
32         r2="results/fastp/{sample}_R2.trimmed.fastq.gz",
33         html="results/fastp/{sample}_fastp.html",
34         json="results/fastp/{sample}_fastp.json"
35     threads: 4
36     conda:
37         "envs/qc.yaml"
38     params:
39         qual=config.get("min_quality", 20),
40         len=config.get("min_length", 50)
41     shell:
42         """
43         fastp -i {input.r1} -I {input.r2} \
44             -o {output.r1} -O {output.r2} \
45             -h {output.html} -j {output.json} \
46             --qualified_quality_phred {params.qual} \
47             --length_required {params.len} \
48             --detect_adapter_for_pe \
49             --thread {threads}
50         """
51
52 rule multiqc:
53     input:
54         expand("results/fastqc/{sample}_{read}_fastqc.zip",
55             sample=SAMPLES, read=["R1", "R2"]),
56         expand("results/fastp/{sample}_fastp.json",
57             sample=SAMPLES)
58     output:
59         "results/multiqc/multiqc_report.html"
60     conda:
61         "envs/qc.yaml"
62     shell:
63         "multiqc results/ -o results/multiqc/ --force"

```

Listing 20.19: config.yaml for the Snakemake QC pipeline.

```

1 # config.yaml
2 samples:
3   - sample1
4   - sample2
5   - sample3
6
7 min_quality: 20
8 min_length: 50

```

Listing 20.20: Conda environment file for QC tools (envs/qc.yaml).

```

1 # envs/qc.yaml
2 name: qc
3 channels:
4   - conda-forge
5   - bioconda
6 dependencies:
7   - fastqc=0.12.1
8   - fastp=0.23.4
9   - multiqc=1.21

```

Listing 20.21: Running the Snakemake QC pipeline.

```

1 # Dry run (show what would be executed)
2 snakemake -n --cores 8
3
4 # Execute locally with 8 cores
5 snakemake --cores 8 --use-conda
6
7 # Execute on a SLURM cluster
8 snakemake --cores 100 --use-conda --slurm \
9   --default-resources slurm_partition=standard mem_mb=8000
10
11 # Generate the workflow DAG visualization
12 snakemake --dag | dot -Tpdf > dag.pdf

```

20.7.3 Nextflow

Philosophy and Core Concepts

Nextflow uses a **dataflow programming model** where data flows through a network of processes connected by **channels**. It is written in Groovy (a JVM language) and uses DSL2 syntax for modular workflow design:

- **Processes:** Independent computational steps, each with defined inputs, outputs, and a script.
- **Channels:** Asynchronous queues that connect process inputs to outputs. Data items flow through channels one at a time, enabling automatic parallelism.
- **Channel operators:** `map`, `filter`, `collect`, `combine`, `groupTuple`, `flatten`—powerful transformations for routing and reshaping data.
- **DSL2 modules:** Processes can be defined in separate module files and imported, promoting reuse.

- **Profiles:** Named configuration sets (e.g., `-profile docker`, `-profile slurm`) that switch between execution environments.
- **nf-core:** A community of curated, production-ready Nextflow pipelines (over 100 pipelines for genomics, transcriptomics, epigenomics, metagenomics, and more).
- **Sequera Platform (Nextflow Tower):** A web-based monitoring and management platform for Nextflow pipelines.

Complete Nextflow QC Pipeline Example

Listing 20.22: Complete Nextflow DSL2 workflow for a basic FASTQ QC pipeline (main.nf).

```

1  #!/usr/bin/env nextflow
2  nextflow.enable.dsl = 2
3
4  // Parameters
5  params.reads      = "data/*_{R1,R2}.fastq.gz"
6  params.outdir     = "results"
7  params.min_qual   = 20
8  params.min_len    = 50
9
10 log.info """
11  =====
12  Q C   P I P E L I N E
13  =====
14  reads   : ${params.reads}
15  outdir  : ${params.outdir}
16  min_qual: ${params.min_qual}
17  min_len : ${params.min_len}
18  =====
19  """.stripIndent()
20
21 process FASTQC {
22     tag "${sample_id}"
23     publishDir "${params.outdir}/fastqc", mode: 'copy'
24     conda 'bioconda::fastqc=0.12.1'
25     cpus 2
26
27     input:
28     tuple val(sample_id), path(reads)
29
30     output:
31     path("*.html"), emit: html
32     path("*.zip"),  emit: zip
33
34     script:
35     """
36     fastqc -t ${task.cpus} ${reads}
37     """
38 }
39
40 process FASTP {
41     tag "${sample_id}"
42     publishDir "${params.outdir}/fastp", mode: 'copy'
43     conda 'bioconda::fastp=0.23.4'
44     cpus 4
45
46     input:

```



```

47 tuple val(sample_id), path(reads)
48
49 output:
50 tuple val(sample_id), path("*_trimmed.fastq.gz"), emit: trimmed
51 path("*.html"), emit: html
52 path("*.json"), emit: json
53
54 script:
55 def (r1, r2) = reads
56 """
57 fastp -i ${r1} -I ${r2} \
58     -o ${sample_id}_R1_trimmed.fastq.gz \
59     -O ${sample_id}_R2_trimmed.fastq.gz \
60     -h ${sample_id}_fastp.html \
61     -j ${sample_id}_fastp.json \
62     --qualified_quality_phred ${params.min_qual} \
63     --length_required ${params.min_len} \
64     --detect_adapter_for_pe \
65     --thread ${task.cpus}
66 """
67 }
68
69 process MULTIQC {
70     publishDir "${params.outdir}/multiqc", mode: 'copy'
71     conda 'bioconda::multiqc=1.21'
72
73     input:
74     path('*')
75
76     output:
77     path("multiqc_report.html")
78
79     script:
80     """
81     multiqc . -o . --force
82     """
83 }
84
85 workflow {
86     read_pairs_ch = Channel
87         .fromFilePairs(params.reads, checkIfExists: true)
88
89     FASTQC(read_pairs_ch)
90     FASTP(read_pairs_ch)
91
92     ch_multiqc = FASTQC.out.zip
93         .mix(FASTP.out.json)
94         .collect()
95
96     MULTIQC(ch_multiqc)
97 }

```

Listing 20.23: nextflow.config for the QC pipeline.

```

1 // nextflow.config
2 profiles {
3     standard {
4         process.executor = 'local'
5         process.cpus = 4

```

```

6     process.memory = '8 GB'
7 }
8 slurm {
9     process.executor = 'slurm'
10    process.queue = 'standard'
11    process.cpus = 4
12    process.memory = '8 GB'
13    process.time = '2h'
14 }
15 docker {
16     docker.enabled = true
17 }
18 singularity {
19     singularity.enabled = true
20     singularity.autoMounts = true
21 }
22 }
23
24 process {
25     errorStrategy = 'retry'
26     maxRetries = 2
27 }

```

Listing 20.24: Running the Nextflow QC pipeline.

```

1  # Run locally
2  nextflow run main.nf --reads "data/*_{R1,R2}.fastq.gz"
3
4  # Run with SLURM profile
5  nextflow run main.nf -profile slurm
6
7  # Run with Docker containers
8  nextflow run main.nf -profile docker
9
10 # Resume a failed/interrupted run
11 nextflow run main.nf -resume
12
13 # Run an nf-core pipeline directly
14 nextflow run nf-core/rnaseq -r 3.14.0 \
15     --input samplesheet.csv --genome GRCh38 -profile singularity

```

20.7.4 Snakemake vs Nextflow: Detailed Comparison

Choosing a Workflow Manager

Both Snakemake and Nextflow are excellent choices. If your group uses Python extensively and values the Make-like file-centric model, Snakemake may feel more natural. If you prefer dataflow programming, need robust cloud integration, or want to leverage the extensive nf-core pipeline library, Nextflow is an excellent choice. Many bioinformatics groups successfully use both, selecting the tool that best fits each project.

20.7.5 WDL (Workflow Description Language)

WDL is a workflow language developed by the Broad Institute, primarily used with the Cromwell execution engine and the Terra platform (a cloud-based genomics platform running on Google Cloud). GATK best-practices pipelines are officially maintained as WDL workflows.

Table 20.4: Comprehensive comparison of Snakemake and Nextflow workflow management systems.

Feature	Snakemake	Nextflow
Language	Python-based	Groovy-based (JVM)
Paradigm	Declarative, file-centric (Make-like)	Dataflow, channel-based
Dependency resolution	From file I/O relationships	From channel connections
Learning curve	Lower (Python knowledge helps)	Steeper (new channel/operator concepts)
Wildcard system	Powerful, built-in wildcards	Channels and tuple patterns
Modularity	Include/module rules, wrappers	DSL2 modules, sub-workflows
Conda integration	Per-rule <code>conda</code> directive	Per-process <code>conda</code> directive
Container support	Singularity, Docker	Docker, Singularity, Podman
Cloud support	AWS, GCP (via profiles)	AWS, GCP, Azure (native executors)
HPC support	SLURM, PBS, SGE (profiles)	SLURM, PBS, SGE, LSF, etc.
Community pipelines	Snakemake Workflow Catalog	nf-core (100+ curated pipelines)
Monitoring	Panoptes (limited)	Seqera Platform / Nextflow Tower
Caching/resume	Based on file timestamps/checksums	Based on task hash (robust)
Reporting	Built-in HTML report	Nextflow reports, Tower dashboard
Popularity	Strong in academia (Europe)	Strong in industry and nf-core community

WDL uses a task-and-workflow structure similar to Nextflow’s process-and-workflow model. While less widely adopted in academia than Snakemake or Nextflow, WDL is prominent in clinical genomics and large-scale variant calling projects.

20.7.6 nf-core: Community-Curated Pipelines

The nf-core community maintains a library of over 100 production-ready Nextflow pipelines, each following strict quality standards:

- Standardised input formats (samplesheets)
- Comprehensive documentation
- Continuous integration testing
- Container support (Docker, Singularity)
- Built-in MultiQC reporting
- Reference genome handling via iGenomes or custom references

Key nf-core pipelines include: `nf-core/rnaseq` (RNA-seq), `nf-core/sarek` (WGS/WES variant calling), `nf-core/atacseq` (ATAC-seq), `nf-core/chipseq` (ChIP-seq), `nf-core/methylseq` (bisulfite sequencing), `nf-core/ampliseq` (16S amplicon), `nf-core/scrnaseq` (single-cell RNA-seq), and `nf-core/hicar` (Hi-C analysis).

20.8 Version Control with Git

Version control is essential for managing bioinformatics code, workflow definitions, configuration files, and analysis scripts. Git, combined with hosting platforms like GitHub or GitLab, provides:

- **History tracking:** Every change to your analysis code is recorded with a message, author, and timestamp.
- **Branching:** Experiment with alternative analysis approaches without affecting the main codebase.
- **Collaboration:** Multiple analysts can work on the same project simultaneously.
- **Reproducibility:** Tag specific commits that correspond to published analyses.

Listing 20.25: Essential Git workflow for a bioinformatics project.

```

1 # Initialize a repository
2 git init my_ngs_project && cd my_ngs_project
3
4 # Create a .gitignore for bioinformatics
5 cat > .gitignore << 'EOF'
6 # Data files (too large for Git)
7 *.fastq.gz
8 *.bam
9 *.bam.bai
10 *.vcf.gz
11 *.bigwig
12 *.h5ad
13
14 # Results (regenerated by pipeline)
15 results/
16
17 # Environment files
18 .snakemake/
19 work/
20 .nextflow*
21 EOF
22
23 # Stage and commit workflow files
24 git add Snakefile config.yaml envs/ scripts/
25 git commit -m "Add initial QC pipeline"
26
27 # Tag a version corresponding to a publication
28 git tag -a v1.0 -m "Version used in Smith et al. 2026"
29 git push origin main --tags

```

Never Commit Large Data Files

Git is designed for text files (code, configuration, documentation). Never commit large binary files (FASTQ, BAM, VCF, BigWig) to a Git repository. Use Git LFS (Large File Storage) for moderate-sized files, or track data locations in a separate manifest. Raw sequencing data should be archived in public repositories (SRA/ENA) or institutional storage systems.

20.9 Reproducibility Best Practices

Reproducibility—the ability for another researcher (or your future self) to obtain the same results from the same data using the same methods—is a cornerstone of scientific integrity. In bioinformatics, achieving reproducibility requires attention to several layers:

Reproducibility Checklist

1. **Pin software versions:** Record exact versions of every tool used (e.g., via Conda environment files or container digests).
2. **Pin reference data:** Record the exact genome build, annotation version, and database versions used (e.g., GENCODE v44, dbSNP b156).
3. **Use a workflow manager:** Encode your entire analysis in a Snakemake or Nextflow pipeline.
4. **Use containers:** Provide Docker/Singularity images for each step.
5. **Use version control:** Track all code and configuration in Git.
6. **Document parameters:** Record all command-line parameters and configuration choices, with justification.
7. **Deposit data:** Submit raw data to SRA/ENA and processed data to GEO or equivalent repositories.
8. **Archive code:** Deposit code on Zenodo (for DOI assignment) in addition to GitHub.
9. **Use literate programming:** Combine analysis code, results, and narrative in R Markdown or Jupyter notebooks for downstream statistical analyses.
10. **Set random seeds:** When using stochastic methods (e.g., UMAP, t-SNE, bootstrapping), set and record the random seed.

20.10 A Typical Bioinformatics Computing Stack

Figure 20.1 illustrates the layers of a modern bioinformatics computing stack, from hardware infrastructure at the base to biological interpretation at the top.

20.11 Summary

This chapter has introduced the foundational elements of bioinformatics that underpin all subsequent analysis chapters:

- **Computing environment:** Linux command-line skills, package managers (Conda/-Mamba), containers (Docker/Singularity), HPC clusters (SLURM), and cloud computing form the infrastructure layer.
- **File formats:** FASTA, FASTQ, SAM/BAM/CRAM, BED, GFF/GTF, VCF/BCF, BigWig, h5ad, and specialized formats for Hi-C and single-cell data are the lingua franca of bioinformatics.
- **Core tools:** samtools, bedtools, Picard, IGV, deepTools, and MultiQC are used in virtually every analysis pipeline.
- **Programming languages:** Python (with Biopython, pysam, Scanpy) and R (with Bioconductor, DESeq2, ggplot2) are complementary and both essential.
- **Workflow management:** Snakemake and Nextflow provide reproducibility, scalability, and portability for complex analysis pipelines. The nf-core community provides production-ready Nextflow pipelines.

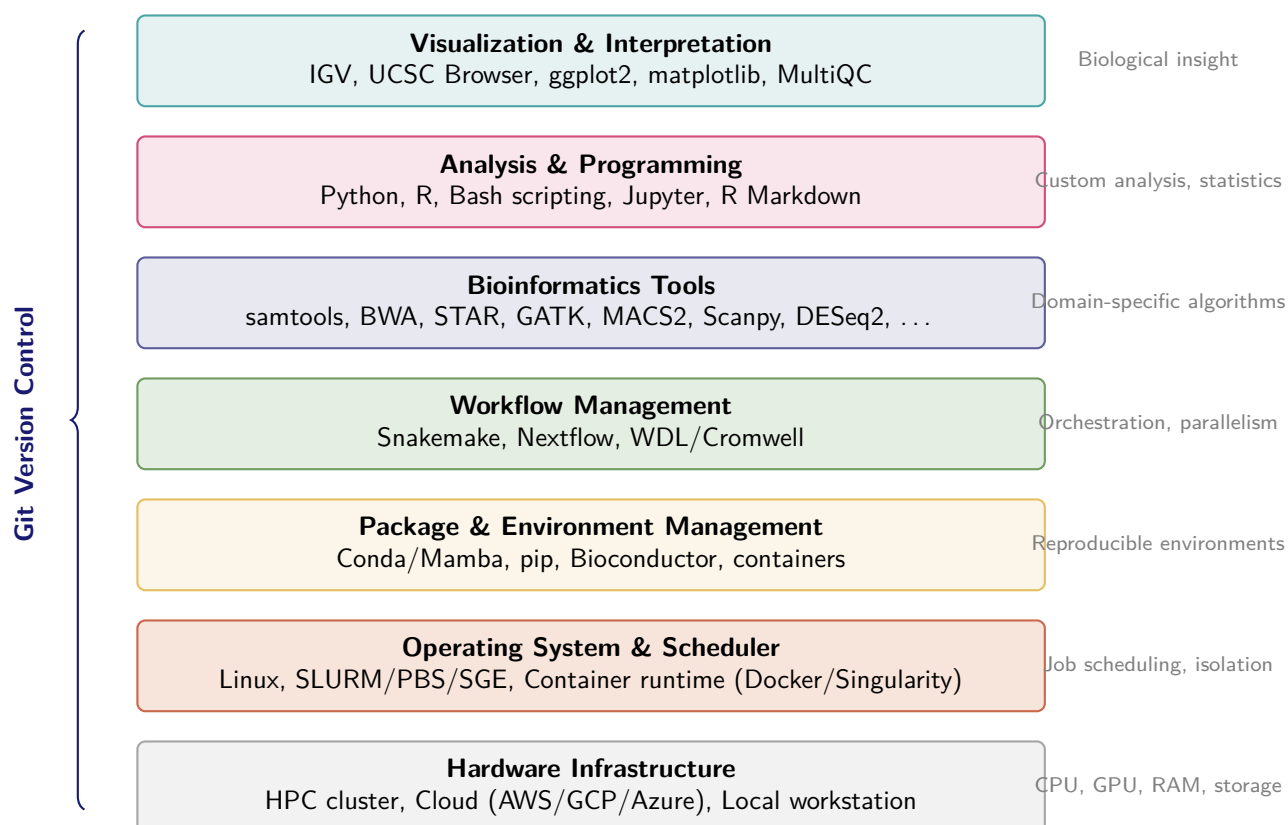


Figure 20.1: The layers of a modern bioinformatics computing stack. Version control (Git) spans all layers. Reproducibility requires attention at every level—from pinning software versions (environment management) to using workflow managers and containers.

- **Reproducibility:** Version control (Git), software version pinning, containers, workflow managers, and proper documentation are all essential for reproducible bioinformatics.

With these fundamentals in place, the next chapter presents complete, runnable analysis workflows for the major sequencing applications covered in this book.

Chapter 21

Complete Analysis Workflows

Key Concept

This chapter provides complete, practical workflow implementations for the major NGS applications covered in this book. For each application, we present both a **Snakemake** and a **Nextflow** implementation, explain every processing step, discuss key parameter choices, and highlight common pitfalls. These workflows are designed to serve as starting points that you can adapt to your specific data and research questions.

Each workflow follows the same structure: (1) a brief overview of the analysis logic, (2) a step-by-step explanation of what each tool does and why, (3) a complete Snakemake implementation, (4) a complete Nextflow DSL2 implementation, (5) expected outputs, and (6) common pitfalls and tips.

21.1 WGS Variant Calling Pipeline

21.1.1 Pipeline Overview

Whole-genome sequencing variant calling identifies single nucleotide variants (SNVs), insertions/deletions (indels), and potentially structural variants from short-read sequencing data. The GATK Best Practices pipeline is the most widely used approach for germline variant calling:

1. **FastQC**: Raw read quality assessment.
2. **fastp**: Adapter trimming and quality filtering.
3. **BWA-MEM2**: Alignment to the reference genome (faster successor to BWA-MEM).
4. **samtools sort**: Coordinate-sort the alignments.
5. **sambamba markdup**: Mark PCR duplicates (faster alternative to Picard MarkDuplicates).
6. **GATK HaplotypeCaller**: Per-sample variant calling in GVCF mode.
7. **GATK GenotypeGVCFs**: Joint genotyping across samples.
8. **GATK VQSR**: Variant Quality Score Recalibration for filtering.
9. **VEP**: Variant Effect Predictor for functional annotation.

Why GVCF Mode?

GATK HaplotypeCaller in GVCF mode (`-ERC GVCF`) produces a genomic VCF that records confidence at every position, not just variant sites. This enables efficient joint genotyping: new samples can be added to a cohort without re-calling all existing samples. For single-sample analyses, you can skip GVCF mode and call variants directly.

21.1.2 Complete Snakemake WGS Workflow

Listing 21.1: Snakemake WGS variant calling pipeline (Snakefile).

```

1 # Snakefile -- WGS Germline Variant Calling (GATK Best Practices)
2 configfile: "config.yaml"
3
4 SAMPLES = config["samples"]
5 REF      = config["reference"]
6 KNOWN    = config["known_sites"] # dbSNP, Mills, 1000G
7
8 rule all:
9     input:
10         "results/annotation/all_samples.annotated.vcf",
11         "results/multiqc/multiqc_report.html"
12
13 rule fastp_trim:
14     input:
15         r1="data/{sample}_R1.fastq.gz",
16         r2="data/{sample}_R2.fastq.gz"
17     output:
18         r1="results/trimmed/{sample}_R1.trimmed.fastq.gz",
19         r2="results/trimmed/{sample}_R2.trimmed.fastq.gz",
20         json="results/trimmed/{sample}_fastp.json",
21         html="results/trimmed/{sample}_fastp.html"
22     threads: 4
23     conda: "envs/qc.yaml"
24     shell:
25         """
26         fastp -i {input.r1} -I {input.r2} \
27             -o {output.r1} -O {output.r2} \
28             -j {output.json} -h {output.html} \
29             --detect_adapter_for_pe --thread {threads}
30         """
31
32 rule bwa_mem2_align:
33     input:
34         r1="results/trimmed/{sample}_R1.trimmed.fastq.gz",
35         r2="results/trimmed/{sample}_R2.trimmed.fastq.gz",
36         ref=REF
37     output:
38         bam="results/aligned/{sample}.sorted.bam"
39     threads: 16
40     params:
41         rg=lambda wc: f"@RG\tID:{wc.sample}\tSM:{wc.sample}\tPL:ILLUMINA"
42     conda: "envs/align.yaml"
43     shell:
44         """
45         bwa-mem2 mem -t {threads} -R '{params.rg}' {input.ref} \

```

```

46         {input.r1} {input.r2} \
47         | samtools sort -@ 4 -o {output.bam} -
48         samtools index {output.bam}
49         """
50
51 rule mark_duplicates:
52     input:
53         "results/aligned/{sample}.sorted.bam"
54     output:
55         bam="results/dedup/{sample}.dedup.bam",
56         metrics="results/dedup/{sample}.dup_metrics.txt"
57     threads: 8
58     conda: "envs/align.yaml"
59     shell:
60         """
61         sambamba markdup -t {threads} \
62             {input} {output.bam} 2> {output.metrics}
63         """
64
65 rule haplotype_caller:
66     input:
67         bam="results/dedup/{sample}.dedup.bam",
68         ref=REF
69     output:
70         gvcf="results/gvcf/{sample}.g.vcf.gz"
71     threads: 4
72     conda: "envs/gatk.yaml"
73     shell:
74         """
75         gatk HaplotypeCaller \
76             -R {input.ref} -I {input.bam} \
77             -O {output.gvcf} -ERC GVCF \
78             --native-pair-hmm-threads {threads}
79         """
80
81 rule combine_gvcfs:
82     input:
83         gvcfs=expand("results/gvcf/{sample}.g.vcf.gz",
84                     sample=SAMPLES),
85         ref=REF
86     output:
87         "results/genotyped/combined.g.vcf.gz"
88     params:
89         gvcf_args=lambda wc, input: " ".join(
90             [f"-V {g}" for g in input.gvcfs])
91     conda: "envs/gatk.yaml"
92     shell:
93         """
94         gatk CombineGVCFs -R {input.ref} {params.gvcf_args} \
95             -O {output}
96         """
97
98 rule genotype_gvcfs:
99     input:
100         gvcf="results/genotyped/combined.g.vcf.gz",
101         ref=REF
102     output:
103         "results/genotyped/all_samples.vcf.gz"

```

```

104     conda: "envs/gatk.yaml"
105     shell:
106         """
107         gatk GenotypeGVCFs -R {input.ref} -V {input.gvcf} \
108             -O {output}
109         """
110
111 rule vqsr_snps:
112     input:
113         vcf="results/genotyped/all_samples.vcf.gz",
114         ref=REF
115     output:
116         recal="results/vqsr/snps.recal",
117         tranches="results/vqsr/snps.tranches",
118         vcf="results/vqsr/snps_recalibrated.vcf.gz"
119     params:
120         hapmap=config["hapmap"],
121         omni=config["omni"],
122         g1k=config["g1k_snps"],
123         dbsnp=config["dbsnp"]
124     conda: "envs/gatk.yaml"
125     shell:
126         """
127         gatk VariantRecalibrator -R {input.ref} -V {input.vcf} \
128             --resource:hapmap,known=false,training=true,truth=true,prior=15.0
129         {params.hapmap} \
130             --resource:omni,known=false,training=true,truth=false,prior=12.0
131         {params.omni} \
132             --resource:1000G,known=false,training=true,truth=false,prior=10.0
133         {params.g1k} \
134             --resource:dbsnp,known=true,training=false,truth=false,prior=2.0
135         {params.dbsnp} \
136             -an QD -an MQ -an MQRankSum -an ReadPosRankSum -an FS -an SOR \
137             -mode SNP -O {output.recal} --tranches-file {output.tranches}
138
139         gatk ApplyVQSR -R {input.ref} -V {input.vcf} \
140             --recal-file {output.recal} --tranches-file {output.tranches} \
141             --truth-sensitivity-filter-level 99.5 -mode SNP \
142             -O {output.vcf}
143         """
144
145 rule vep_annotate:
146     input:
147         "results/vqsr/snps_recalibrated.vcf.gz"
148     output:
149         "results/annotation/all_samples.annotated.vcf"
150     threads: 4
151     conda: "envs/annotation.yaml"
152     shell:
153         """
154         vep --input_file {input} --output_file {output} \
155             --format vcf --vcf --fork {threads} \
156             --cache --offline --assembly GRCh38 \
157             --sift b --polyphen b --symbol --numbers \
158             --canonical --biotype --af --af_1kg --af_gnomade
159         """
160
161 rule multiqc:

```

```

158 input:
159     expand("results/trimmed/{sample}_fastp.json", sample=SAMPLES),
160     expand("results/dedup/{sample}.dup_metrics.txt", sample=SAMPLES)
161 output:
162     "results/multiqc/multiqc_report.html"
163 conda: "envs/qc.yaml"
164 shell:
165     "multiqc results/ -o results/multiqc/ --force"

```

21.1.3 Complete Nextflow WGS Workflow

Listing 21.2: Nextflow DSL2 WGS variant calling pipeline (main.nf).

```

1  #!/usr/bin/env nextflow
2  nextflow.enable.dsl = 2
3
4  params.reads      = "data/*_{R1,R2}.fastq.gz"
5  params.ref        = "ref/GRCh38.fa"
6  params.outdir     = "results"
7  params.dbsnp      = "ref/dbsnp_156.vcf.gz"
8
9  process FASTP {
10     tag "${sample_id}"
11     publishDir "${params.outdir}/trimmed", mode: 'copy'
12     cpus 4
13
14     input: tuple val(sample_id), path(reads)
15     output:
16         tuple val(sample_id), path("*_trimmed.fastq.gz"), emit: trimmed
17         path("*.json"), emit: json
18
19     script:
20     def (r1, r2) = reads
21     """
22     fastp -i ${r1} -I ${r2} \
23         -o ${sample_id}_R1_trimmed.fastq.gz \
24         -O ${sample_id}_R2_trimmed.fastq.gz \
25         -j ${sample_id}_fastp.json \
26         --detect_adapter_for_pe --thread ${task.cpus}
27     """
28 }
29
30 process BWA_MEM2 {
31     tag "${sample_id}"
32     publishDir "${params.outdir}/aligned", mode: 'copy'
33     cpus 16
34     memory '32 GB'
35
36     input:
37         tuple val(sample_id), path(reads)
38         path(ref)
39
40     output:
41         tuple val(sample_id), path("*.sorted.bam"), path("*.sorted.bam.bai"), emit:
42         bam
43
44     script:

```

```

44     def (r1, r2) = reads
45     """
46     bwa-mem2 mem -t ${task.cpus} \
47       -R "@RG\tID:${sample_id}\tSM:${sample_id}\tPL:ILLUMINA" \
48       ${ref} ${r1} ${r2} \
49       | samtools sort -@ 4 -o ${sample_id}.sorted.bam -
50     samtools index ${sample_id}.sorted.bam
51     """
52 }
53
54 process MARKDUP {
55     tag "${sample_id}"
56     publishDir "${params.outdir}/dedup", mode: 'copy'
57     cpus 8
58
59     input: tuple val(sample_id), path(bam), path(bai)
60     output: tuple val(sample_id), path("*.dedup.bam"), path("*.dedup.bam.bai"),
61             emit: bam
62
63     script:
64     """
65     sambamba markdup -t ${task.cpus} ${bam} ${sample_id}.dedup.bam
66     samtools index ${sample_id}.dedup.bam
67     """
68 }
69
70 process HAPLOTYPE_CALLER {
71     tag "${sample_id}"
72     publishDir "${params.outdir}/gvcf", mode: 'copy'
73     cpus 4
74     memory '16 GB'
75
76     input:
77         tuple val(sample_id), path(bam), path(bai)
78         path(ref)
79
80     output: tuple val(sample_id), path("*.g.vcf.gz"), path("*.g.vcf.gz.tbi"), emit:
81             gvcf
82
83     script:
84     """
85     gatk HaplotypeCaller -R ${ref} -I ${bam} \
86       -O ${sample_id}.g.vcf.gz -ERC GVCF \
87       --native-pair-hmm-threads ${task.cpus}
88     """
89 }
90
91 process JOINT_GENOTYPE {
92     publishDir "${params.outdir}/genotyped", mode: 'copy'
93     memory '32 GB'
94
95     input:
96         path(gvcfs)
97         path(tbis)
98         path(ref)
99
100     output: path("all_samples.vcf.gz"), emit: vcf

```

```

100     script:
101     def gvcf_args = gvcfs.collect { "-V ${it}" }.join(' ')
102     """
103     gatk CombineGVCFs -R ${ref} ${gvcf_args} -O combined.g.vcf.gz
104     gatk GenotypeGVCFs -R ${ref} -V combined.g.vcf.gz \
105         -O all_samples.vcf.gz
106     """
107 }
108
109 process VEP_ANNOTATE {
110     publishDir "${params.outdir}/annotation", mode: 'copy'
111     cpus 4
112
113     input: path(vcf)
114     output: path("annotated.vcf")
115
116     script:
117     """
118     vep --input_file ${vcf} --output_file annotated.vcf \
119         --format vcf --vcf --fork ${task.cpus} \
120         --cache --offline --assembly GRCh38 \
121         --sift b --polyphen b --symbol --canonical --af_gnomade
122     """
123 }
124
125 workflow {
126     reads_ch = Channel.fromFilePairs(params.reads, checkIfExists: true)
127     ref_ch = Channel.value(file(params.ref))
128
129     FASTP(reads_ch)
130     BWA_MEM2(FASTP.out.trimmed, ref_ch)
131     MARKDUP(BWA_MEM2.out.bam)
132     HAPLOTYPE_CALLER(MARKDUP.out.bam, ref_ch)
133
134     gvcfs = HAPLOTYPE_CALLER.out.gvcf.map { it[1] }.collect()
135     tbis = HAPLOTYPE_CALLER.out.gvcf.map { it[2] }.collect()
136
137     JOINT_GENOTYPE(gvcfs, tbis, ref_ch)
138     VEP_ANNOTATE(JOINT_GENOTYPE.out.vcf)
139 }

```

VQSR Requires Large Cohorts

GATK VQSR (Variant Quality Score Recalibration) requires at least 30 whole-genome samples to build a reliable model. For smaller cohorts, use hard filtering instead: `gatk VariantFiltration` with thresholds on QD, FS, SOR, MQ, MQRankSum, and ReadPosRankSum. The nf-core/sarek pipeline handles this distinction automatically.

Production Alternative: nf-core/sarek

For production use, consider nf-core/sarek, which implements the complete GATK Best Practices pipeline with extensive QC, supports multiple variant callers (GATK, DeepVariant, Strelka2, Manta), handles both germline and somatic calling, and includes tumor-normal workflows. Run with: `nextflow run nf-core/sarek -input samplesheet.csv -genome GRCh38 -profile singularity`.

21.2 RNA-seq Differential Expression Pipeline

21.2.1 Pipeline Overview

RNA-seq differential expression analysis quantifies gene expression and identifies genes that are significantly up- or down-regulated between conditions. We present two approaches: (1) alignment-based (STAR + featureCounts + DESeq2) and (2) pseudo-alignment-based (Salmon + tximeta + DESeq2).

1. **FastQC / fastp**: Quality assessment and adapter trimming.
2. **STAR alignment** (approach 1): Splice-aware alignment to the genome.
3. **featureCounts** (approach 1): Count reads per gene from BAM files.
4. **Salmon** (approach 2): Quasi-mapping and transcript-level quantification.
5. **tximeta/tximport**: Import Salmon counts to gene level for DESeq2.
6. **DESeq2**: Differential expression testing with shrinkage estimation.

21.2.2 Complete Snakemake RNA-seq Workflow

Listing 21.3: Snakemake RNA-seq differential expression pipeline.

```

1 # Snakefile -- RNA-seq Differential Expression
2 configfile: "config.yaml"
3
4 SAMPLES = config["samples"]
5 REF      = config["reference"]
6 GTF      = config["gtf"]
7 STAR_IDX = config["star_index"]
8
9 rule all:
10     input:
11         "results/deseq2/results_table.csv",
12         "results/multiqc/multiqc_report.html"
13
14 rule fastp_trim:
15     input:
16         r1="data/{sample}_R1.fastq.gz",
17         r2="data/{sample}_R2.fastq.gz"
18     output:
19         r1="results/trimmed/{sample}_R1.trimmed.fastq.gz",
20         r2="results/trimmed/{sample}_R2.trimmed.fastq.gz",
21         json="results/trimmed/{sample}_fastp.json"
22     threads: 4
23     conda: "envs/qc.yaml"
24     shell:
25         """
26         fastp -i {input.r1} -I {input.r2} \
27             -o {output.r1} -O {output.r2} \
28             -j {output.json} --detect_adapter_for_pe \
29             --thread {threads}
30         """
31
32 rule star_align:
33     input:
34         r1="results/trimmed/{sample}_R1.trimmed.fastq.gz",

```



```

35     r2="results/trimmed/{sample}_R2.trimmed.fastq.gz",
36     idx=STAR_IDX
37   output:
38     bam="results/star/{sample}/Aligned.sortedByCoord.out.bam",
39     log="results/star/{sample}/Log.final.out"
40   threads: 12
41   conda: "envs/align.yaml"
42   params:
43     prefix="results/star/{sample}/"
44   shell:
45     """
46     STAR --runThreadN {threads} \
47         --genomeDir {input.idx} \
48         --readFilesIn {input.r1} {input.r2} \
49         --readFilesCommand zcat \
50         --outSAMtype BAM SortedByCoordinate \
51         --outFileNamePrefix {params.prefix} \
52         --quantMode GeneCounts \
53         --outSAMattributes NH HI NM MD AS
54     samtools index {output.bam}
55     """
56
57 rule featurecounts:
58   input:
59     bams=expand("results/star/{sample}/Aligned.sortedByCoord.out.bam",
60                 sample=SAMPLES),
61     gtf=GTF
62   output:
63     counts="results/counts/gene_counts.txt",
64     summary="results/counts/gene_counts.txt.summary"
65   threads: 8
66   conda: "envs/count.yaml"
67   shell:
68     """
69     featureCounts -T {threads} -p --countReadPairs \
70         -a {input.gtf} -o {output.counts} \
71         -s 2 --extraAttributes gene_name \
72         {input.bams}
73     """
74
75 rule deseq2:
76   input:
77     counts="results/counts/gene_counts.txt",
78     metadata=config["metadata"]
79   output:
80     results="results/deseq2/results_table.csv",
81     ma_plot="results/deseq2/ma_plot.pdf",
82     volcano="results/deseq2/volcano_plot.pdf",
83     pca="results/deseq2/pca_plot.pdf"
84   conda: "envs/deseq2.yaml"
85   script:
86     "scripts/run_deseq2.R"
87
88 rule multiqc:
89   input:
90     expand("results/trimmed/{sample}_fastp.json", sample=SAMPLES),
91     expand("results/star/{sample}/Log.final.out", sample=SAMPLES),
92     "results/counts/gene_counts.txt.summary"

```

```

93 output:
94     "results/multiqc/multiqc_report.html"
95 conda: "envs/qc.yaml"
96 shell:
97     "multiqc results/ -o results/multiqc/ --force"

```

21.2.3 Complete Nextflow RNA-seq Workflow

Listing 21.4: Nextflow DSL2 RNA-seq pipeline with Salmon pseudo-alignment.

```

1  #!/usr/bin/env nextflow
2  nextflow.enable.dsl = 2
3
4  params.reads      = "data/*_{R1,R2}.fastq.gz"
5  params.salmon_idx = "ref/salmon_index"
6  params.gtf        = "ref/gencode.v44.annotation.gtf"
7  params.outdir     = "results"
8  params.metadata   = "samplesheet.csv"
9
10 process FASTP {
11     tag "${sample_id}"
12     publishDir "${params.outdir}/trimmed", mode: 'copy'
13     cpus 4
14
15     input: tuple val(sample_id), path(reads)
16     output:
17         tuple val(sample_id), path("*_trimmed.fastq.gz"), emit: trimmed
18         path("*.json"), emit: json
19
20     script:
21     def (r1, r2) = reads
22     """
23     fastp -i ${r1} -I ${r2} \
24         -o ${sample_id}_R1_trimmed.fastq.gz \
25         -O ${sample_id}_R2_trimmed.fastq.gz \
26         -j ${sample_id}_fastp.json \
27         --detect_adapter_for_pe --thread ${task.cpus}
28     """
29 }
30
31 process SALMON_QUANT {
32     tag "${sample_id}"
33     publishDir "${params.outdir}/salmon", mode: 'copy'
34     cpus 8
35     memory '16 GB'
36
37     input:
38         tuple val(sample_id), path(reads)
39         path(index)
40
41     output:
42         path("${sample_id}"), emit: quant_dir
43
44     script:
45     def (r1, r2) = reads
46     """
47     salmon quant -i ${index} -l A \

```

```

48     -1 ${r1} -2 ${r2} \
49     -p ${task.cpus} --validateMappings \
50     --gcBias --seqBias \
51     -o ${sample_id}
52     ""
53 }
54
55 process DESEQ2 {
56     publishDir "${params.outdir}/deseq2", mode: 'copy'
57
58     input:
59         path(salmon_dirs)
60         path(metadata)
61
62     output:
63         path("results_table.csv")
64         path("*.pdf")
65
66     script:
67         ""
68         Rscript ${projectDir}/scripts/salmon_deseq2.R \
69             --salmon_dir . --metadata ${metadata}
70         ""
71 }
72
73 process MULTIQC {
74     publishDir "${params.outdir}/multiqc", mode: 'copy'
75
76     input: path('*')
77     output: path("multiqc_report.html")
78
79     script:
80         ""
81         multiqc . --force
82         ""
83 }
84
85 workflow {
86     reads_ch = Channel.fromFilePairs(params.reads, checkIfExists: true)
87     salmon_idx = Channel.value(file(params.salmon_idx))
88     metadata_ch = Channel.value(file(params.metadata))
89
90     FASTP(reads_ch)
91     SALMON_QUANT(FASTP.out.trimmed, salmon_idx)
92     DESEQ2(SALMON_QUANT.out.quant_dir.collect(), metadata_ch)
93     MULTIQC(FASTP.out.json.collect())
94 }

```

STAR vs Salmon

STAR performs full alignment to the genome, producing BAM files that can be used for many downstream analyses (variant calling from RNA-seq, splicing analysis, coverage visualization). Salmon performs pseudo-alignment directly to the transcriptome, which is 10–20 times faster and uses far less memory, but does not produce BAM files. For standard differential expression analysis, Salmon is recommended; for analyses requiring read-level information, STAR is necessary.

Production Alternative: nf-core/rnaseq

The `nf-core/rnaseq` pipeline implements both STAR and Salmon paths with comprehensive QC, strandedness detection, UMI handling, rRNA removal, and MultiQC reporting. It supports dozens of organisms via iGenomes and is the recommended starting point for production RNA-seq analysis.

21.3 ATAC-seq Pipeline

21.3.1 Pipeline Overview

ATAC-seq maps open chromatin regions. The analysis requires careful filtering steps specific to this assay:

1. **FastQC / fastp:** Adapter trimming (Nextera adapters for ATAC-seq).
2. **Bowtie2:** Alignment (Bowtie2 is preferred over BWA for ATAC-seq due to better handling of short fragments).
3. **Filtering:** Remove mitochondrial reads, PCR duplicates, reads with low MAPQ, and reads in blacklist regions.
4. **MACS2:** Peak calling with ATAC-seq-specific parameters.
5. **Fragment size QC:** Verify the characteristic nucleosomal ladder pattern.
6. **DiffBind:** Differential accessibility analysis between conditions.
7. **HOMER:** Motif enrichment analysis in accessible regions.

21.3.2 Complete Snakemake ATAC-seq Workflow

Listing 21.5: Snakemake ATAC-seq pipeline.

```

1 # Snakefile -- ATAC-seq Pipeline
2 configfile: "config.yaml"
3
4 SAMPLES = config["samples"]
5 REF      = config["reference"]
6 BT2_IDX  = config["bowtie2_index"]
7 BLACKLIST = config["blacklist"]
8
9 rule all:
10     input:
11         expand("results/peaks/{sample}_peaks.narrowPeak", sample=SAMPLES),
12         expand("results/bigwig/{sample}.bw", sample=SAMPLES),
13         expand("results/qc/{sample}_fragment_sizes.pdf", sample=SAMPLES),
14         "results/multiqc/multiqc_report.html"
15
16 rule fastp_trim:
17     input:
18         r1="data/{sample}_R1.fastq.gz",
19         r2="data/{sample}_R2.fastq.gz"
20     output:
21         r1="results/trimmed/{sample}_R1.trimmed.fastq.gz",
22         r2="results/trimmed/{sample}_R2.trimmed.fastq.gz",
23         json="results/trimmed/{sample}_fastp.json"
24     threads: 4

```

```

25     conda: "envs/qc.yaml"
26     shell:
27         """
28         fastp -i {input.r1} -I {input.r2} \
29             -o {output.r1} -O {output.r2} \
30             -j {output.json} --detect_adapter_for_pe \
31             --thread {threads}
32         """
33
34 rule bowtie2_align:
35     input:
36         r1="results/trimmed/{sample}_R1.trimmed.fastq.gz",
37         r2="results/trimmed/{sample}_R2.trimmed.fastq.gz"
38     output:
39         bam="results/aligned/{sample}.bam"
40     threads: 12
41     params:
42         idx=BT2_IDX
43     conda: "envs/align.yaml"
44     shell:
45         """
46         bowtie2 -x {params.idx} -1 {input.r1} -2 {input.r2} \
47             -X 2000 --very-sensitive --no-mixed --no-discordant \
48             --threads {threads} \
49             | samtools sort -@ 4 -o {output.bam} -
50         samtools index {output.bam}
51         """
52
53 rule filter_bam:
54     input:
55         "results/aligned/{sample}.bam"
56     output:
57         bam="results/filtered/{sample}.filtered.bam"
58     threads: 8
59     params:
60         blacklist=BLACKLIST
61     conda: "envs/align.yaml"
62     shell:
63         """
64         # Remove mitochondrial, low MAPQ, unmapped, not proper pair
65         samtools view -b -h -f 2 -q 30 {input} \
66             | grep -v "chrM" \
67             | samtools view -b -o temp_{wildcards.sample}.bam -
68         # Remove duplicates
69         sambamba markdup -t {threads} --remove-duplicates \
70             temp_{wildcards.sample}.bam temp2_{wildcards.sample}.bam
71         # Remove blacklist regions
72         bedtools intersect -v -abam temp2_{wildcards.sample}.bam \
73             -b {params.blacklist} > {output.bam}
74         samtools index {output.bam}
75         rm -f temp_{wildcards.sample}.bam* temp2_{wildcards.sample}.bam*
76         """
77
78 rule macs2_peaks:
79     input:
80         "results/filtered/{sample}.filtered.bam"
81     output:
82         "results/peaks/{sample}_peaks.narrowPeak"

```

```

83   params:
84       prefix="{sample}",
85       outdir="results/peaks"
86   conda: "envs/peaks.yaml"
87   shell:
88       """
89       macs2 callpeak -t {input} \
90           -f BAMPE -g hs --nomodel --shift -100 --extsize 200 \
91           --keep-dup all -q 0.05 \
92           -n {params.prefix} --outdir {params.outdir}
93       """
94
95   rule bamcoverage:
96       input:
97           "results/filtered/{sample}.filtered.bam"
98       output:
99           "results/bigwig/{sample}.bw"
100      threads: 8
101      conda: "envs/deeptools.yaml"
102      shell:
103          """
104          bamCoverage -b {input} -o {output} \
105              --normalizeUsing RPKM --binSize 10 \
106              --extendReads -p {threads}
107          """
108
109   rule fragment_sizes:
110       input:
111           "results/filtered/{sample}.filtered.bam"
112       output:
113           "results/qc/{sample}_fragment_sizes.pdf"
114       conda: "envs/deeptools.yaml"
115       shell:
116           """
117           bamPEFragmentSize -b {input} \
118               -o {output} --maxFragmentLength 1000
119           """
120
121   rule multiqc:
122       input:
123           expand("results/trimmed/{sample}_fastp.json", sample=SAMPLES),
124           expand("results/peaks/{sample}_peaks.narrowPeak", sample=SAMPLES)
125       output:
126           "results/multiqc/multiqc_report.html"
127       conda: "envs/qc.yaml"
128       shell:
129           "multiqc results/ -o results/multiqc/ --force"

```

21.3.3 Complete Nextflow ATAC-seq Workflow

Listing 21.6: Nextflow DSL2 ATAC-seq pipeline.

```

1  #!/usr/bin/env nextflow
2  nextflow.enable.dsl = 2
3
4  params.reads      = "data/*_{R1,R2}.fastq.gz"
5  params.bt2_index = "ref/GRCh38_bt2"

```

```

6 params.blacklist = "ref/hg38-blacklist.v2.bed"
7 params.outdir    = "results"
8
9 process FASTP {
10     tag "${sample_id}"
11     cpus 4
12     input: tuple val(sample_id), path(reads)
13     output: tuple val(sample_id), path("*_trimmed.fastq.gz"), emit: trimmed
14     script:
15     def (r1, r2) = reads
16     """
17     fastp -i ${r1} -I ${r2} \
18         -o ${sample_id}_R1_trimmed.fastq.gz \
19         -O ${sample_id}_R2_trimmed.fastq.gz \
20         --detect_adapter_for_pe --thread ${task.cpus}
21     """
22 }
23
24 process BOWTIE2 {
25     tag "${sample_id}"
26     cpus 12
27     memory '16 GB'
28     input:
29         tuple val(sample_id), path(reads)
30         val(bt2_idx)
31     output: tuple val(sample_id), path("*.sorted.bam"), path("*.sorted.bam.bai"),
32     emit: bam
33     script:
34     def (r1, r2) = reads
35     """
36     bowtie2 -x ${bt2_idx} -1 ${r1} -2 ${r2} \
37         -X 2000 --very-sensitive --no-mixed --no-discordant \
38         --threads ${task.cpus} \
39         | samtools sort -@ 4 -o ${sample_id}.sorted.bam -
40     samtools index ${sample_id}.sorted.bam
41     """
42 }
43
44 process FILTER_BAM {
45     tag "${sample_id}"
46     cpus 8
47     input:
48         tuple val(sample_id), path(bam), path(bai)
49         path(blacklist)
50     output: tuple val(sample_id), path("*.filtered.bam"),
51     path("*.filtered.bam.bai"), emit: bam
52     script:
53     """
54     # Filter: proper pairs, MAPQ>=30, no chrM
55     samtools view -b -h -f 2 -q 30 ${bam} \
56         $(samtools idxstats ${bam} | cut -f1 | grep -v chrM | tr '\n' ' ') \
57         | samtools sort -o no_mito.bam -
58     # Remove duplicates
59     sambamba markdup -t ${task.cpus} --remove-duplicates no_mito.bam dedup.bam
60     # Remove blacklist
61     bedtools intersect -v -abam dedup.bam -b ${blacklist} >
62     ${sample_id}.filtered.bam
63     samtools index ${sample_id}.filtered.bam

```

```

61     """
62 }
63
64 process MACS2 {
65     tag "${sample_id}"
66     publishDir "${params.outdir}/peaks", mode: 'copy'
67     input: tuple val(sample_id), path(bam), path(bai)
68     output: path("${sample_id}_peaks.narrowPeak"), emit: peaks
69     script:
70     """
71     macs2 callpeak -t ${bam} -f BAMPE -g hs \
72         --nomodel --shift -100 --extsize 200 \
73         --keep-dup all -q 0.05 -n ${sample_id}
74     """
75 }
76
77 process BIGWIG {
78     tag "${sample_id}"
79     publishDir "${params.outdir}/bigwig", mode: 'copy'
80     cpus 8
81     input: tuple val(sample_id), path(bam), path(bai)
82     output: path("*.bw")
83     script:
84     """
85     bamCoverage -b ${bam} -o ${sample_id}.bw \
86         --normalizeUsing RPKM --binSize 10 --extendReads -p ${task.cpus}
87     """
88 }
89
90 workflow {
91     reads_ch = Channel.fromFilePairs(params.reads, checkIfExists: true)
92     bt2_idx_ch = Channel.value(params.bt2_index)
93     bl_ch = Channel.value(file(params.blacklist))
94
95     FASTP(reads_ch)
96     BOWTIE2(FASTP.out.trimmed, bt2_idx_ch)
97     FILTER_BAM(BOWTIE2.out.bam, bl_ch)
98     MACS2(FILTER_BAM.out.bam)
99     BIGWIG(FILTER_BAM.out.bam)
100 }

```

ATAC-seq-Specific Filtering

ATAC-seq data typically has 30–80% of reads mapping to mitochondrial DNA, because mitochondria lack chromatin and are highly accessible to the transposase. These reads must be removed before peak calling. Additionally, the Tn5 transposase introduces a 9-bp duplication at the insertion site; the `-shift -100 -extsize 200` MACS2 parameters or the alternative `-shift -75 -extsize 150` account for this offset to centre peaks on the actual cut site.

Fragment Size Distribution QC

A successful ATAC-seq library shows a characteristic nucleosomal ladder: a large peak at <150 bp (nucleosome-free fragments), a smaller peak at ~200 bp (mono-nucleosome), ~400 bp (di-nucleosome), etc. If the sub-nucleosomal peak is absent, the library quality is poor. Generate this plot with `bamPEFragmentSize` from `deepTools` or custom scripts.

21.4 ChIP-seq / CUT&Tag Pipeline

21.4.1 Pipeline Overview

ChIP-seq and CUT&Tag identify genomic regions bound by specific proteins (transcription factors, histone modifications). Key differences from ATAC-seq include: (1) an input or IgG control is required, (2) peak calling uses the control for background estimation, and (3) CUT&Tag may use spike-in normalization.

1. **fastp**: Adapter trimming.
2. **Bowtie2**: Alignment (including spike-in genome for CUT&Tag if applicable).
3. **Filtering**: Remove duplicates, low MAPQ reads, blacklist regions.
4. **MACS2** (narrow peaks, e.g., transcription factors, H3K4me3) or **SICER2/epic2** (broad peaks, e.g., H3K27me3, H3K36me3).
5. **BigWig generation**: Normalized coverage tracks.
6. **deepTools**: Heatmaps and profile plots around features.

21.4.2 Complete Snakemake ChIP-seq Workflow

Listing 21.7: Snakemake ChIP-seq pipeline with input control.

```

1 # Snakefile -- ChIP-seq Pipeline
2 configfile: "config.yaml"
3
4 SAMPLES = config["chip_samples"] # e.g., [H3K4me3_rep1, H3K4me3_rep2]
5 INPUTS = config["input_controls"] # e.g., {H3K4me3_rep1: input_rep1}
6 PEAK_TYPE = config.get("peak_type", "narrow") # "narrow" or "broad"
7 BT2_IDX = config["bowtie2_index"]
8 BLACKLIST = config["blacklist"]
9
10 rule all:
11     input:
12         expand("results/peaks/{sample}_peaks.{ext}",
13               sample=SAMPLES,
14               ext="narrowPeak" if PEAK_TYPE=="narrow" else "broadPeak"),
15         expand("results/bigwig/{sample}.bw", sample=SAMPLES),
16         "results/deeptools/heatmap.pdf"
17
18 rule fastp_trim:
19     input:
20         r1="data/{sample}_R1.fastq.gz",
21         r2="data/{sample}_R2.fastq.gz"
22     output:
23         r1="results/trimmed/{sample}_R1.trimmed.fastq.gz",
24         r2="results/trimmed/{sample}_R2.trimmed.fastq.gz",

```

```

25     json="results/trimmed/{sample}_fastp.json"
26     threads: 4
27     conda: "envs/qc.yaml"
28     shell:
29         """
30         fastp -i {input.r1} -I {input.r2} \
31             -o {output.r1} -O {output.r2} \
32             -j {output.json} --detect_adapter_for_pe --thread {threads}
33         """
34
35 rule bowtie2_align:
36     input:
37         r1="results/trimmed/{sample}_R1.trimmed.fastq.gz",
38         r2="results/trimmed/{sample}_R2.trimmed.fastq.gz"
39     output:
40         bam="results/aligned/{sample}.sorted.bam"
41     threads: 12
42     params: idx=BT2_IDX
43     conda: "envs/align.yaml"
44     shell:
45         """
46         bowtie2 -x {params.idx} -1 {input.r1} -2 {input.r2} \
47             --very-sensitive --threads {threads} \
48             | samtools sort -@ 4 -o {output.bam} -
49         samtools index {output.bam}
50         """
51
52 rule filter_and_dedup:
53     input:
54         "results/aligned/{sample}.sorted.bam"
55     output:
56         bam="results/filtered/{sample}.filtered.bam"
57     threads: 8
58     conda: "envs/align.yaml"
59     shell:
60         """
61         samtools view -b -f 2 -q 30 {input} \
62             | samtools sort -o temp.bam -
63         sambamba markdup -t {threads} --remove-duplicates \
64             temp.bam {output.bam}
65         samtools index {output.bam}
66         rm -f temp.bam*
67         """
68
69 rule macs2_callpeak:
70     input:
71         treatment="results/filtered/{sample}.filtered.bam",
72         control=lambda wc: f"results/filtered/{INPUTS[wc.sample]}.filtered.bam"
73     output:
74         "results/peaks/{sample}_peaks.narrowPeak" if PEAK_TYPE=="narrow" else
75         "results/peaks/{sample}_peaks.broadPeak"
76     params:
77         mode="--broad --broad-cutoff 0.1" if PEAK_TYPE=="broad" else "",
78         name="{sample}",
79         outdir="results/peaks"
80     conda: "envs/peaks.yaml"
81     shell:
82         """

```

```

83     macs2 callpeak -t {input.treatment} -c {input.control} \
84         -f BAMPE -g hs {params.mode} \
85         -q 0.05 -n {params.name} --outdir {params.outdir}
86     """"
87
88 rule bamcoverage:
89     input:
90         "results/filtered/{sample}.filtered.bam"
91     output:
92         "results/bigwig/{sample}.bw"
93     threads: 8
94     conda: "envs/deeptools.yaml"
95     shell:
96         """"
97         bamCoverage -b {input} -o {output} \
98             --normalizeUsing RPKM --binSize 10 -p {threads}
99         """"
100
101 rule deeptools_heatmap:
102     input:
103         bw=expand("results/bigwig/{sample}.bw", sample=SAMPLES),
104         bed=config["gene_bed"]
105     output:
106         "results/deeptools/heatmap.pdf"
107     conda: "envs/deeptools.yaml"
108     shell:
109         """"
110         computeMatrix reference-point \
111             -S {input.bw} -R {input.bed} \
112             -a 3000 -b 3000 -o matrix.gz
113         plotHeatmap -m matrix.gz -o {output} --colorMap RdBu
114         """"

```

21.4.3 Complete Nextflow ChIP-seq Workflow

Listing 21.8: Nextflow DSL2 ChIP-seq pipeline.

```

1  #!/usr/bin/env nextflow
2  nextflow.enable.dsl = 2
3
4  params.samplesheet = "samplesheet.csv" // sample_id,fastq_1,fastq_2,control_id
5  params.bt2_index    = "ref/GRCh38_bt2"
6  params.blacklist    = "ref/hg38-blacklist.v2.bed"
7  params.outdir       = "results"
8  params.peak_type    = "narrow" // or "broad"
9
10 process FASTP {
11     tag "${sample_id}"
12     cpus 4
13     input: tuple val(sample_id), path(r1), path(r2)
14     output: tuple val(sample_id), path("*_R1_trimmed.fastq.gz"),
15             path("*_R2_trimmed.fastq.gz"), emit: trimmed
16     script:
17         """"
18         fastp -i ${r1} -I ${r2} \
19             -o ${sample_id}_R1_trimmed.fastq.gz \
20             -O ${sample_id}_R2_trimmed.fastq.gz \

```

```

20     --detect_adapter_for_pe --thread ${task.cpus}
21     ""
22 }
23
24 process BOWTIE2_ALIGN {
25     tag "${sample_id}"
26     cpus 12
27     input:
28         tuple val(sample_id), path(r1), path(r2)
29         val(idx)
30     output: tuple val(sample_id), path("*.filtered.bam"),
31             path("*.filtered.bam.bai"), emit: bam
32     script:
33         ""
34         bowtie2 -x ${idx} -1 ${r1} -2 ${r2} --very-sensitive \
35             --threads ${task.cpus} \
36             | samtools view -b -f 2 -q 30 - \
37             | samtools sort -o sorted.bam -
38         sambamba markdup -t ${task.cpus} --remove-duplicates sorted.bam
39         ${sample_id}.filtered.bam
40         samtools index ${sample_id}.filtered.bam
41     ""
42 }
43
44 process MACS2_CALLPEAK {
45     tag "${sample_id}"
46     publishDir "${params.outdir}/peaks", mode: 'copy'
47     input:
48         tuple val(sample_id), path(treatment_bam), path(treatment_bai),
49             path(control_bam), path(control_bai)
50     output: path("${sample_id}_peaks.*Peak"), emit: peaks
51     script:
52         def broad_flag = params.peak_type == "broad" ? "--broad --broad-cutoff 0.1" : ""
53         ""
54         macs2 callpeak -t ${treatment_bam} -c ${control_bam} \
55             -f BAMPE -g hs ${broad_flag} -q 0.05 -n ${sample_id}
56     ""
57 }
58
59 workflow {
60     // Parse samplesheet: sample_id, fastq_1, fastq_2, control_id
61     ch_input = Channel.fromPath(params.samplesheet)
62     .splitCsv(header: true)
63     .map { row -> tuple(row.sample_id, file(row.fastq_1), file(row.fastq_2),
64         row.control_id) }
65
66     ch_reads = ch_input.map { it[0..2] }
67     bt2_idx_ch = Channel.value(params.bt2_index)
68
69     FASTP(ch_reads)
70     BOWTIE2_ALIGN(FASTP.out.trimmed, bt2_idx_ch)
71
72     // Join treatment with control BAMs
73     ch_with_control = ch_input.map { [it[0], it[3]] } // sample_id, control_id
74     ch_treatment = BOWTIE2_ALIGN.out.bam // sample_id, bam, bai
75     ch_control = BOWTIE2_ALIGN.out.bam // also includes controls
76
77     ch_paired = ch_treatment

```

```

75     .combine(ch_with_control, by: 0)
76     .map { sample_id, t_bam, t_bai, ctrl_id -> [sample_id, t_bam, t_bai,
ctrl_id] }
77     .combine(ch_control.map { [it[0], it[1], it[2]] }, by: 0)
78
79     // Simplified: directly call peaks
80     MACS2_CALLPEAK(ch_paired)
81 }

```

Narrow vs Broad Peaks

Narrow peaks are sharp, well-defined binding sites typical of transcription factors and active histone marks (H3K4me3, H3K27ac). Use MACS2 default mode.

Broad peaks are diffuse domains spanning kilobases to megabases, typical of repressive marks (H3K27me3, H3K9me3) and gene-body marks (H3K36me3). Use MACS2 with `-broad` or dedicated tools like SICER2/epic2.

21.5 scRNA-seq Pipeline

21.5.1 Pipeline Overview

Single-cell RNA-seq analysis begins with raw read processing (Cell Ranger or STARsolo) to generate a count matrix, followed by a series of computational steps to identify cell populations and their marker genes:

1. **Cell Ranger / STARsolo:** Demultiplexing, alignment, UMI counting, cell barcode filtering.
2. **Quality control:** Filter cells by total UMIs, genes detected, and mitochondrial fraction.
3. **Normalization:** Library size normalization and log transformation.
4. **Highly variable genes (HVGs):** Select informative features for dimensionality reduction.
5. **PCA:** Reduce dimensionality; select significant components.
6. **Neighbourhood graph and UMAP:** Build cell-cell similarity graph; embed in 2D for visualization.
7. **Clustering:** Leiden or Louvain community detection on the graph.
8. **Marker gene identification:** Find genes differentially expressed between clusters.

21.5.2 Complete Snakemake scRNA-seq Workflow

Listing 21.9: Snakemake scRNA-seq pipeline with STARsolo and Scanpy.

```

1 # Snakefile -- scRNA-seq Pipeline
2 configfile: "config.yaml"
3
4 SAMPLES      = config["samples"]
5 STAR_IDX     = config["star_index"]
6 WHITELIST    = config["barcode_whitelist"]
7
8 rule all:
9     input:

```

```

10     expand("results/scanpy/{sample}_processed.h5ad", sample=SAMPLES),
11     "results/scanpy/integrated.h5ad"
12
13 rule starsolo:
14     input:
15         r1="data/{sample}_R1.fastq.gz", # cDNA read
16         r2="data/{sample}_R2.fastq.gz"  # barcode+UMI read
17     output:
18         bam="results/starsolo/{sample}/Aligned.sortedByCoord.out.bam",
19         mtx="results/starsolo/{sample}/Solo.out/Gene/filtered/matrix.mtx"
20     threads: 16
21     params:
22         idx=STAR_IDX,
23         wl=WHITELIST,
24         outdir="results/starsolo/{sample}/"
25     conda: "envs/starsolo.yaml"
26     shell:
27         """
28         STAR --runThreadN {threads} \
29             --genomeDir {params.idx} \
30             --readFilesIn {input.r1} {input.r2} \
31             --readFilesCommand zcat \
32             --soloType CB_UMI_Simple \
33             --soloCBwhitelist {params.wl} \
34             --soloCBstart 1 --soloCBlen 16 \
35             --soloUMIstart 17 --soloUMIlen 12 \
36             --outSAMtype BAM SortedByCoordinate \
37             --outFileNamePrefix {params.outdir} \
38             --soloFeatures Gene \
39             --soloCellFilter EmptyDrops_CR
40         """
41
42 rule scanpy_analysis:
43     input:
44         mtx="results/starsolo/{sample}/Solo.out/Gene/filtered/matrix.mtx"
45     output:
46         h5ad="results/scanpy/{sample}_processed.h5ad"
47     conda: "envs/scanpy.yaml"
48     script:
49         "scripts/run_scanpy.py"
50
51 rule integrate_samples:
52     input:
53         expand("results/scanpy/{sample}_processed.h5ad", sample=SAMPLES)
54     output:
55         "results/scanpy/integrated.h5ad"
56     conda: "envs/scanpy.yaml"
57     script:
58         "scripts/integrate_scanpy.py"

```

21.5.3 Complete Nextflow scRNA-seq Workflow

Listing 21.10: Nextflow DSL2 scRNA-seq pipeline.

```

1 #!/usr/bin/env nextflow
2 nextflow.enable.dsl = 2
3

```

```

4 params.samplesheet = "samplesheet.csv"
5 params.star_index = "ref/star_index"
6 params.whitelist = "ref/3M-february-2018.txt"
7 params.outdir = "results"
8
9 process STARSOLO {
10     tag "${sample_id}"
11     publishDir "${params.outdir}/starsolo/${sample_id}", mode: 'copy'
12     cpus 16
13     memory '64 GB'
14
15     input:
16         tuple val(sample_id), path(r1), path(r2)
17         path(star_index)
18         path(whitelist)
19
20     output:
21         tuple val(sample_id), path("Solo.out/Gene/filtered/"), emit: counts
22         path("Log.final.out"), emit: log
23
24     script:
25         """
26         STAR --runThreadN ${task.cpus} \
27             --genomeDir ${star_index} \
28             --readFilesIn ${r1} ${r2} \
29             --readFilesCommand zcat \
30             --soloType CB_UMI_Simple \
31             --soloCBwhitelist ${whitelist} \
32             --soloCBstart 1 --soloCBlen 16 \
33             --soloUMIstart 17 --soloUMIlen 12 \
34             --outSAMtype BAM SortedByCoordinate \
35             --soloFeatures Gene \
36             --soloCellFilter EmptyDrops_CR
37         """
38 }
39
40 process SCANPY_PROCESS {
41     tag "${sample_id}"
42     publishDir "${params.outdir}/scanpy", mode: 'copy'
43
44     input: tuple val(sample_id), path(counts_dir)
45     output: tuple val(sample_id), path("${sample_id}_processed.h5ad"), emit: h5ad
46
47     script:
48         """
49         python ${projectDir}/scripts/run_scanpy.py \
50             --input ${counts_dir} --sample ${sample_id} \
51             --output ${sample_id}_processed.h5ad
52         """
53 }
54
55 process INTEGRATE {
56     publishDir "${params.outdir}/scanpy", mode: 'copy'
57
58     input: path(h5ad_files)
59     output: path("integrated.h5ad")
60
61     script:

```

```

62     """
63     python ${projectDir}/scripts/integrate_scanpy.py \
64         --input_files ${h5ad_files} --output integrated.h5ad
65     """
66 }
67
68 workflow {
69     ch_input = Channel.fromPath(params.samplesheet)
70         .splitCsv(header: true)
71         .map { row -> tuple(row.sample_id, file(row.fastq_r1), file(row.fastq_r2)) }
72
73     star_idx = Channel.value(file(params.star_index))
74     wl       = Channel.value(file(params.whitelist))
75
76     STARSOLO(ch_input, star_idx, wl)
77     SCANPY_PROCESS(STARSOLO.out.counts)
78     INTEGRATE(SCANPY_PROCESS.out.h5ad.map { it[1] }.collect())
79 }

```

Scanpy Processing Script (scripts/run_scanpy.py)

The core Scanpy workflow, typically implemented as a Python script called by the workflow manager:

1. Load count matrix: `sc.read_10x_mtx()` or `sc.read_10x_h5()`
2. QC filtering: remove cells with <200 genes, >20% mitochondrial reads
3. Normalization: `sc.pp.normalize_total(target_sum=10000)`, `sc.pp.log1p()`
4. HVG selection: `sc.pp.highly_variable_genes(n_top_genes=2000)`
5. PCA: `sc.tl.pca(n_comps=50)`
6. Neighbors: `sc.pp.neighbors(n_neighbors=15, n_pcs=30)`
7. UMAP: `sc.tl.umap()`
8. Leiden clustering: `sc.tl.leiden(resolution=0.5)`
9. Marker genes: `sc.tl.rank_genes_groups('leiden', method='wilcoxon')`
10. Save: `adata.write('output.h5ad')`

21.6 Single-Cell Quantification Tool Comparison

Several tools can generate the cell-by-gene count matrix from raw single-cell RNA-seq FASTQ files. Choosing between them depends on speed, memory constraints, and compatibility requirements.

21.6.1 Cell Ranger vs STARSolo vs alevin-fry

Cell Ranger (10x Genomics)

The official quantification tool from 10x Genomics. Cell Ranger is straightforward to run and produces well-documented outputs, but it is closed-source, relatively slow, and requires substantial memory.

Listing 21.11: Cell Ranger count command.

```

1 cellranger count --id=sample1 \
2   --transcriptome=/ref/refdata-gex-GRCh38-2024-A \
3   --fastqs=/data/sample1/ \
4   --sample=sample1 \
5   --localcores=16 --localmem=64

```


STARsolo

The single-cell mode of the STAR aligner. STARsolo produces output fully compatible with Cell Ranger (MEX format) while running 5–10× faster. It supports 10x Chromium v2/v3, Drop-seq, and other barcoding schemes.

Listing 21.12: STARsolo command for 10x Chromium v3.

```

1 STAR --runThreadN 16 \
2   --genomeDir /ref/star_index \
3   --readFilesIn sample1_R2.fastq.gz sample1_R1.fastq.gz \
4   --readFilesCommand zcat \
5   --soloType CB_UMI_Simple \
6   --soloCBwhitelist 3M-february-2018.txt \
7   --soloCBstart 1 --soloCBlen 16 \
8   --soloUMIstart 17 --soloUMIlen 12 \
9   --outSAMtype BAM SortedByCoordinate \
10  --soloFeatures Gene GeneFull \
11  --soloCellFilter EmptyDrops_CR

```

alevin-fry (Salmon)

A lightweight, pseudo-alignment-based quantification tool. **alevin-fry** uses **Salmon** for mapping and a Rust-based framework for UMI resolution. It is the fastest option and uses the least memory, but does not produce BAM files.

Listing 21.13: alevin-fry quantification pipeline.

```

1 # Step 1: Generate RAD file with Salmon alevin
2 salmon alevin -l ISR -i /ref/salmon_index \
3   --chromiumV3 -1 sample1_R1.fastq.gz -2 sample1_R2.fastq.gz \
4   -p 16 -o sample1_alevin --sketch
5
6 # Step 2: Generate permit list
7 alevin-fry generate-permit-list -d fw -k \
8   -i sample1_alevin -o sample1_quant
9
10 # Step 3: Collate RAD file
11 alevin-fry collate -r sample1_alevin -i sample1_quant -t 16
12
13 # Step 4: Quantify
14 alevin-fry quant -m /ref/t2g.tsv -i sample1_quant \
15   -o sample1_quant_res -t 16 -r cr-like --use-mtx

```

21.6.2 Single-Cell Tool Recommendation Table

Which Tool to Choose?

For most analyses, **STARsolo** is the recommended default: it is fast, open-source, produces Cell Ranger-compatible output, and generates BAM files for downstream use (e.g., RNA velocity with **velocity**). Use **alevin-fry** when processing very large datasets where speed and memory are critical. Use **Cell Ranger** when you need 10x-specific features or when your collaborators require Cell Ranger output for regulatory or reproducibility reasons.

21.7 Bisulfite Sequencing Pipeline

Table 21.1: Comparison of single-cell quantification tools for 10x Chromium data.

Tool	Speed	Peak RAM	Cell Ranger Compat.	Notes
Cell Ranger	Slow	~64 GB	Native	Closed-source; required for some 10x features (e.g., ATAC, multiome)
STARsolo	5–10× faster	~32 GB	Full (MEX output)	Open-source; drop-in replacement; produces BAM
alevin-fry	10–20× faster	~8–16 GB	Via conversion	Fastest; smallest memory; no BAM; requires Salmon index
kallisto bustools	10–15× faster	~8–12 GB	Via conversion	Pseudo-alignment; similar speed to alevin-fry

21.7.1 Pipeline Overview

Whole-genome bisulfite sequencing (WGBS) measures DNA methylation at single-base resolution. Bisulfite treatment converts unmethylated cytosines to uracil (read as thymine), while methylated cytosines are protected. This creates a reduced-complexity genome that requires specialized alignment:

1. **FastQC:** Raw read quality assessment.
2. **Trim Galore:** Adapter and quality trimming (Trim Galore handles the RRBS or WGBS-specific trimming needs).
3. **Bismark:** Bisulfite-aware alignment to in-silico converted genomes.
4. **Bismark deduplicate:** Remove PCR duplicates.
5. **Bismark methylation extractor:** Extract per-CpG methylation levels.
6. **DMR calling:** Identify differentially methylated regions (e.g., with `dmrseq` or `DSS` in R).

21.7.2 Complete Snakemake Bisulfite-seq Workflow

Listing 21.14: Snakemake WGBS pipeline with Bismark.

```

1 # Snakefile -- WGBS Bisulfite Sequencing Pipeline
2 configfile: "config.yaml"
3
4 SAMPLES      = config["samples"]
5 BISMARK_IDX  = config["bismark_index"]
6
7 rule all:
8     input:
9         expand("results/methylation/{sample}.bismark.cov.gz", sample=SAMPLES),
10        "results/multiqc/multiqc_report.html"
11
12 rule trim_galore:
13     input:
14         r1="data/{sample}_R1.fastq.gz",

```

```

15     r2="data/{sample}_R2.fastq.gz"
16     output:
17         r1="results/trimmed/{sample}_R1_val_1.fq.gz",
18         r2="results/trimmed/{sample}_R2_val_2.fq.gz"
19     threads: 4
20     conda: "envs/trim.yaml"
21     shell:
22         """
23         trim_galore --paired --cores {threads} \
24             -o results/trimmed/ \
25             {input.r1} {input.r2}
26         """
27
28 rule bismark_align:
29     input:
30         r1="results/trimmed/{sample}_R1_val_1.fq.gz",
31         r2="results/trimmed/{sample}_R2_val_2.fq.gz"
32     output:
33         bam="results/bismark/{sample}_pe.bam",
34         report="results/bismark/{sample}_PE_report.txt"
35     threads: 8
36     params:
37         idx=BISMARK_IDX
38     conda: "envs/bismark.yaml"
39     shell:
40         """
41         bismark --genome {params.idx} \
42             -1 {input.r1} -2 {input.r2} \
43             --parallel {threads} --bowtie2 \
44             -o results/bismark/ \
45             --temp_dir results/bismark/tmp
46         """
47
48 rule bismark_dedup:
49     input:
50         "results/bismark/{sample}_pe.bam"
51     output:
52         "results/bismark/{sample}_pe.deduplicated.bam"
53     conda: "envs/bismark.yaml"
54     shell:
55         """
56         deduplicate_bismark --bam --paired \
57             -o results/bismark/ {input}
58         """
59
60 rule methylation_extract:
61     input:
62         "results/bismark/{sample}_pe.deduplicated.bam"
63     output:
64         "results/methylation/{sample}.bismark.cov.gz"
65     threads: 8
66     params:
67         idx=BISMARK_IDX
68     conda: "envs/bismark.yaml"
69     shell:
70         """
71         bismark_methylation_extractor --paired-end \
72             --comprehensive --merge_non_CpG \

```

```

73     --cytosine_report --genome_folder {params.idx} \
74     --parallel {threads} --gzip \
75     -o results/methylation/ \
76     {input}
77     """
78
79 rule multiqc:
80     input:
81         expand("results/bismark/{sample}_PE_report.txt", sample=SAMPLES),
82         expand("results/methylation/{sample}.bismark.cov.gz", sample=SAMPLES)
83     output:
84         "results/multiqc/multiqc_report.html"
85     conda: "envs/qc.yaml"
86     shell:
87         "multiqc results/ -o results/multiqc/ --force"

```

21.7.3 Complete Nextflow Bisulfite-seq Workflow

Listing 21.15: Nextflow DSL2 WGBS pipeline with Bismark.

```

1  #!/usr/bin/env nextflow
2  nextflow.enable.dsl = 2
3
4  params.reads      = "data/*_{R1,R2}.fastq.gz"
5  params.bismark_idx = "ref/bismark_index"
6  params.outdir     = "results"
7
8  process TRIM_GALORE {
9      tag "${sample_id}"
10     cpus 4
11     input: tuple val(sample_id), path(reads)
12     output: tuple val(sample_id), path("*_val_{1,2}.fq.gz"), emit: trimmed
13     script:
14         def (r1, r2) = reads
15         """
16         trim_galore --paired --cores ${task.cpus} ${r1} ${r2}
17         """
18 }
19
20 process BISMARK_ALIGN {
21     tag "${sample_id}"
22     cpus 8
23     memory '32 GB'
24     input:
25         tuple val(sample_id), path(reads)
26         path(bismark_idx)
27     output: tuple val(sample_id), path("*.bam"), emit: bam
28     script:
29         def (r1, r2) = reads
30         """
31         bismark --genome ${bismark_idx} \
32             -1 ${r1} -2 ${r2} \
33             --parallel ${task.cpus} --bowtie2 --temp_dir tmp
34         """
35 }
36
37 process BISMARK_DEDUP {

```

```

38   tag "${sample_id}"
39   input: tuple val(sample_id), path(bam)
40   output: tuple val(sample_id), path("*.deduplicated.bam"), emit: bam
41   script:
42   """
43   deduplicate_bismark --bam --paired ${bam}
44   """
45 }
46
47 process METHYL_EXTRACT {
48   tag "${sample_id}"
49   publishDir "${params.outdir}/methylation", mode: 'copy'
50   cpus 8
51   input:
52     tuple val(sample_id), path(bam)
53     path(bismark_idx)
54   output: path("*.cov.gz"), emit: coverage
55   script:
56   """
57   bismark_methylation_extractor --paired-end \
58     --comprehensive --merge_non_CpG \
59     --cytosine_report --genome_folder ${bismark_idx} \
60     --parallel ${task.cpus} --gzip ${bam}
61   """
62 }
63
64 workflow {
65   reads_ch = Channel.fromFilePairs(params.reads, checkIfExists: true)
66   idx_ch   = Channel.value(file(params.bismark_idx))
67
68   TRIM_GALORE(reads_ch)
69   BISMARK_ALIGN(TRIM_GALORE.out.trimmed, idx_ch)
70   BISMARK_DEDUP(BISMARK_ALIGN.out.bam)
71   METHYL_EXTRACT(BISMARK_DEDUP.out.bam, idx_ch)
72 }

```

Production Alternative: nf-core/methylseq

The `nf-core/methylseq` pipeline supports both Bismark and `bwa-meth` alignment strategies, handles RRBS and WGBS protocols, includes comprehensive QC metrics, and provides publication-ready methylation reports.

21.8 16S Amplicon Metagenomics Pipeline

21.8.1 Pipeline Overview

16S rRNA gene amplicon sequencing identifies bacterial community composition. The DADA2 pipeline is the current standard, replacing older OTU-based approaches with exact amplicon sequence variants (ASVs):

1. **Cutadapt:** Remove primer sequences from read ends.
2. **DADA2 (in R):** Quality filtering, error learning, denoising, merging paired reads, chimera removal, and ASV table generation.
3. **Taxonomic classification:** Assign taxonomy to ASVs using SILVA, Greengenes2, or GTDB databases.

4. **Diversity analysis:** Alpha diversity (Shannon, Simpson, observed ASVs), beta diversity (Bray-Curtis, UniFrac), ordination (PCoA), and differential abundance testing.

21.8.2 Complete Snakemake 16S Workflow

Listing 21.16: Snakemake 16S amplicon metagenomics pipeline.

```

1 # Snakefile -- 16S Amplicon Metagenomics (DADA2)
2 configfile: "config.yaml"
3
4 SAMPLES = config["samples"]
5 FWD_PRIMER = config["fwd_primer"] # e.g., CCTACGGGNGGCWGCAG (341F)
6 REV_PRIMER = config["rev_primer"] # e.g., GACTACHVGGGTATCTAATCC (785R)
7 SILVA_DB = config["silva_db"] # SILVA 138.1 training set
8
9 rule all:
10     input:
11         "results/dada2/asv_table.tsv",
12         "results/dada2/taxonomy.tsv",
13         "results/diversity/alpha_diversity.pdf",
14         "results/diversity/beta_pcoa.pdf"
15
16 rule cutadapt_primers:
17     input:
18         r1="data/{sample}_R1.fastq.gz",
19         r2="data/{sample}_R2.fastq.gz"
20     output:
21         r1="results/trimmed/{sample}_R1.trimmed.fastq.gz",
22         r2="results/trimmed/{sample}_R2.trimmed.fastq.gz"
23     params:
24         fwd=FWD_PRIMER,
25         rev=REV_PRIMER
26     threads: 4
27     conda: "envs/cutadapt.yaml"
28     shell:
29         """
30         cutadapt -g {params.fwd} -G {params.rev} \
31             -o {output.r1} -p {output.r2} \
32             --discard-untrimmed --cores {threads} \
33             {input.r1} {input.r2}
34         """
35
36 rule dada2_pipeline:
37     input:
38         r1=expand("results/trimmed/{sample}_R1.trimmed.fastq.gz",
39                 sample=SAMPLES),
40         r2=expand("results/trimmed/{sample}_R2.trimmed.fastq.gz",
41                 sample=SAMPLES)
42     output:
43         asv="results/dada2/asv_table.tsv",
44         tax="results/dada2/taxonomy.tsv",
45         seqs="results/dada2/asv_sequences.fasta"
46     params:
47         silva=SILVA_DB,
48         trunc_f=config.get("truncLenF", 280),
49         trunc_r=config.get("truncLenR", 200)
50     conda: "envs/dada2.yaml"
51     script:

```

```

52     "scripts/run_dada2.R"
53
54 rule diversity_analysis:
55     input:
56         asv="results/dada2/asv_table.tsv",
57         tax="results/dada2/taxonomy.tsv",
58         metadata=config["metadata"]
59     output:
60         alpha="results/diversity/alpha_diversity.pdf",
61         beta="results/diversity/beta_pcoa.pdf"
62     conda: "envs/phyloseq.yaml"
63     script:
64         "scripts/diversity_analysis.R"

```

21.8.3 Complete Nextflow 16S Workflow

Listing 21.17: Nextflow DSL2 16S amplicon metagenomics pipeline.

```

1 #!/usr/bin/env nextflow
2  nextflow.enable.dsl = 2
3
4  params.reads      = "data/*_{R1,R2}.fastq.gz"
5  params.fwd_primer = "CCTACGGGNGGCWGCAG"
6  params.rev_primer = "GACTACHVGGGTATCTAATCC"
7  params.silva_db    = "ref/silva_nr99_v138.1_train_set.fa.gz"
8  params.outdir      = "results"
9  params.truncLenF   = 280
10 params.truncLenR   = 200
11
12 process CUTADAPT {
13     tag "${sample_id}"
14     cpus 4
15     input: tuple val(sample_id), path(reads)
16     output: tuple val(sample_id), path("*_trimmed.fastq.gz"), emit: trimmed
17     script:
18         def (r1, r2) = reads
19         """
20         cutadapt -g ${params.fwd_primer} -G ${params.rev_primer} \
21             -o ${sample_id}_R1_trimmed.fastq.gz \
22             -p ${sample_id}_R2_trimmed.fastq.gz \
23             --discard-untrimmed --cores ${task.cpus} \
24             ${r1} ${r2}
25         """
26 }
27
28 process DADA2 {
29     publishDir "${params.outdir}/dada2", mode: 'copy'
30     memory '16 GB'
31     input:
32         path(trimmed_files)
33         path(silva_db)
34     output:
35         path("asv_table.tsv"), emit: asv
36         path("taxonomy.tsv"), emit: tax
37         path("asv_sequences.fasta"), emit: seqs
38     script:
39         """

```

```

40   Rscript ${projectDir}/scripts/run_dada2.R \
41     --input_dir . \
42     --silva ${silva_db} \
43     --truncLenF ${params.truncLenF} \
44     --truncLenR ${params.truncLenR}
45   ""
46 }
47
48 process DIVERSITY {
49   publishDir "${params.outdir}/diversity", mode: 'copy'
50   input:
51     path(asv_table)
52     path(taxonomy)
53   output:
54     path("*.pdf")
55   script:
56     ""
57   Rscript ${projectDir}/scripts/diversity_analysis.R \
58     --asv ${asv_table} --tax ${taxonomy}
59   ""
60 }
61
62 workflow {
63   reads_ch = Channel.fromFilePairs(params.reads, checkIfExists: true)
64   silva_ch = Channel.value(file(params.silva_db))
65
66   CUTADAPT(reads_ch)
67   trimmed_all = CUTADAPT.out.trimmed.map { it[1] }.collect()
68   DADA2(trimmed_all, silva_ch)
69   DIVERSITY(DADA2.out.asv, DADA2.out.tax)
70 }

```

DADA2 Truncation Length

The `truncLen` parameters in DADA2 (`truncLenF` for forward, `truncLenR` for reverse) determine where reads are cut based on quality. These values must be chosen after inspecting the quality profiles (`plotQualityProfile()` in R). Set truncation where median quality drops below Q25–30. After truncation, the remaining forward and reverse reads must still overlap by at least 20 bp for successful merging.

Production Alternative: nf-core/ampliseq

`nf-core/ampliseq` provides a comprehensive Nextflow pipeline for amplicon sequencing analysis, supporting DADA2 and QIIME2 backends, multiple taxonomic databases, and extensive diversity analyses. It handles both 16S and ITS (fungal) amplicons.

21.9 Master Comparison of nf-core Pipelines

Table 21.2: Summary of key nf-core pipelines for NGS data analysis.

Pipeline	Application	Key Features
nf-core/sarek	WGS/WES variant calling	GATK, DeepVariant, Strelka2; germline & somatic; tumor-normal
nf-core/rnaseq	RNA-seq	STAR, Salmon, HISAT2; strandedness detection; UMI support
nf-core/atacseq	ATAC-seq	Bowtie2, MACS2; fragment size QC; IDR for replicates
nf-core/chipseq	ChIP-seq	Bowtie2, MACS2; input control; IDR; broad/narrow peaks
nf-core/scrnaseq	scRNA-seq	Cell Ranger, STARsolo, Salmon alevin; multiple protocols
nf-core/methylseq	Bisulfite-seq	Bismark, bwa-meth; WGBS and RRBS support
nf-core/ampliseq	16S/ITS amplicon	DADA2, QIIME2; multiple taxonomic databases; diversity
nf-core/hic	Hi-C	Mapping, filtering, contact matrices, TAD calling
nf-core/cutandrun	CUT&Run/CUT&Tag	Bowtie2; spike-in normalization; SEACR peak calling
nf-core/smrnaseq	Small RNA-seq	miRNA quantification; novel miRNA prediction
nf-core/nanoseq	Oxford Nanopore	Basecalling, alignment, QC for long-read data
nf-core/mag	Metagenome assembly	Assembly, binning, taxonomic classification

21.10 General Best Practices for All Workflows

Workflow Best Practices Checklist

1. **Always run FastQC/MultiQC** before and after trimming to verify data quality.
2. **Check expected outputs at each step:** alignment rates should be >80%; duplication rates should be reasonable for your assay; insert sizes should match expectations.
3. **Use consistent reference genomes and annotations:** Download genome FASTA and GTF from the same source (e.g., GENCODE or Ensembl) and the same release version.
4. **Document all parameters:** Store them in configuration files (config.yaml or nextflow.config), not hardcoded in the workflow.
5. **Pin software versions:** Use Conda environment files or container image digests.
6. **Test on a small subset first:** Run your pipeline on one or two samples (or a downsampled dataset) before processing the full cohort.
7. **Monitor resource usage:** Check CPU, memory, and disk usage to optimize resource requests on HPC/cloud.
8. **Archive raw data and results:** Submit raw data to SRA/ENA before publication; archive processed results with documentation.

21.11 Resource Requirements by Analysis Type

Planning computational resources before launching a pipeline prevents out-of-memory failures, disk exhaustion, and unnecessarily long runtimes. Table 21.3 provides representative resource estimates for common NGS analyses.

Table 21.3: Approximate computational resource requirements per sample for common NGS analyses. Values assume a human genome and typical parameter settings.

Analysis Type	CPU Cores	RAM (GB)	Storage (GB)	Approx. Time
WGS 30× (BWA-MEM2 + GATK)	16	32–64	200–300	8–16 h/sample
RNA-seq 20 samples (STAR + DESeq2)	12	32	50–100	4–8 h total
scRNA-seq 10K cells (STARsolo + Scanpy)	16	32–64	20–50	1–3 h/sample
scRNA-seq 10K cells (alevin-fry + Scanpy)	8	8–16	10–30	0.5–1 h/sample
ATAC-seq (Bowtie2 + MACS2)	12	16	30–60	2–4 h/sample
ChIP-seq (Bowtie2 + MACS2)	12	16	20–50	2–4 h/sample
WGBS (Bismark)	8	16–32	150–250	12–24 h/sample
16S amplicon (DADA2, 100 samples)	4	8–16	5–10	1–4 h total
Hi-C 1 kb resolution	16	64–128	300–500	24–48 h/sample

Disk Space Is Often the Bottleneck

Intermediate files (unsorted BAMs, temporary alignment files) can consume 2–5× the final output size. A single 30× WGS sample can require over 300 GB of temporary disk space during processing. Always provision at least 3× the expected final output size, and clean up intermediate files as soon as they are no longer needed. On cloud platforms, use local SSD scratch space for intermediate files and persistent storage only for final outputs.

21.12 Error Handling Patterns in Workflows

Production bioinformatics pipelines must handle failures gracefully. Common failure modes include disk exhaustion, out-of-memory (OOM) kills, corrupt or truncated BAM files, and transient network errors when downloading reference data. Both Snakemake and Nextflow provide mechanisms for robust error handling.

21.12.1 Common Failure Modes

Disk full

Pipelines that produce large intermediate files (e.g., unsorted BAM, temporary STAR genome loading) can exhaust disk space. Symptoms: truncated output files, cryptic I/O errors. Prevention: monitor disk usage, clean temporary files, use `-tmp-dir` options.

OOM kill

The Linux kernel’s OOM killer terminates processes that exceed available memory. STAR genome loading (~32 GB for human), GATK HaplotypeCaller, and de novo

assembly are common culprits. Symptoms: process killed with signal 9 (SIGKILL), no error message in tool log. Prevention: request sufficient memory in job scheduler, monitor with `htop`.

Corrupt BAM/CRAM

Truncated BAM files from interrupted jobs cause downstream tools to fail. Always validate BAM integrity with `samtools quickcheck` before using them as input.

Stale NFS locks (HPC)

On shared filesystems, stale file locks can cause tools to hang or produce empty files. Prevention: use local scratch for temporary files, avoid writing to the same directory from multiple concurrent jobs.

21.12.2 Error Handling in Snakemake

Listing 21.18: Snakemake error handling and restart strategies.

```

1 # Retry failed jobs up to 3 times (e.g., transient NFS or OOM)
2 snakemake --cores 16 --use-conda --restart-times 3
3
4 # Use checkpoints for rules that dynamically determine output files
5 # (e.g., when the number of output chunks is not known in advance)
6
7 # Set per-rule resource limits in the Snakefile
8 rule star_align:
9     resources:
10         mem_mb=32000,
11         disk_mb=100000,
12         tmpdir="/scratch/tmp"
13     retries: 2 # Snakemake 8+ per-rule retry

```

21.12.3 Error Handling in Nextflow

Listing 21.19: Nextflow error handling configuration.

```

1 // nextflow.config -- Error handling strategies
2 process {
3     // Retry failed tasks up to 3 times
4     errorStrategy = 'retry'
5     maxRetries = 3
6
7     // Dynamically increase memory on retry
8     memory = { 16.GB * task.attempt }
9     time    = { 4.h * task.attempt }
10
11     // For specific processes, ignore failures (non-critical QC)
12     withName: 'FASTQC' {
13         errorStrategy = 'ignore'
14     }
15
16     // Terminate the pipeline on critical failures
17     withName: 'HAPLOTYPE_CALLER' {
18         errorStrategy = { task.attempt <= 3 ? 'retry' : 'terminate' }
19         memory = { 16.GB * Math.pow(2, task.attempt - 1) }
20     }
21 }

```

Validate Intermediate Files

Add validation steps between pipeline stages. For example, run `samtools quickcheck` after alignment and `bcftools stats` after variant calling. In Snakemake, use the `onstart` and `onsuccess` handlers; in Nextflow, add a validation command in the process script before emitting output.

21.13 Summary

This chapter has provided complete, practical workflow implementations for seven major NGS applications:

- **WGS variant calling:** BWA-MEM2, GATK HaplotypeCaller, VQSR, VEP annotation.
- **RNA-seq differential expression:** STAR + featureCounts + DESeq2 and Salmon + tximeta + DESeq2.
- **ATAC-seq:** Bowtie2, careful filtering (mitochondrial, duplicates, blacklist), MACS2 peak calling.
- **ChIP-seq / CUT&Tag:** Input-controlled peak calling, narrow vs broad peak modes.
- **scRNA-seq:** STARsolo/Cell Ranger count matrix generation, Scanpy processing workflow.
- **Bisulfite-seq:** Trim Galore, Bismark alignment and methylation extraction.
- **16S amplicon metagenomics:** Cutadapt primer removal, DADA2 ASV inference, diversity analysis.

For each pipeline, both Snakemake and Nextflow implementations were provided, along with pointers to the corresponding nf-core production pipelines ([Table 21.2](#)). These workflows should serve as starting points; adapt parameters and tools to your specific organism, sequencing platform, and research questions.

Chapter 22

Experimental Design and Best Practices

Key Concept

Experimental design is the single most important factor in determining whether a sequencing experiment will produce reliable, interpretable results. No amount of computational sophistication can compensate for a fundamentally flawed experimental design. Careful planning of replicates, sequencing depth, controls, and batch structure before the first sample is collected saves enormous time, money, and frustration.

22.1 Why Experimental Design Matters

The principle of “garbage in, garbage out” applies with particular force to NGS experiments. A poorly designed experiment can lead to:

- **False discoveries:** Confounded variables produce spurious biological conclusions.
- **Missed discoveries:** Insufficient statistical power prevents detection of real effects.
- **Wasted resources:** Sequencing at the wrong depth, with too few replicates, or with inappropriate read lengths wastes budget.
- **Irreproducible results:** Uncontrolled batch effects and undocumented variables prevent replication.

This chapter covers the key design decisions for NGS experiments: biological versus technical replication, sequencing depth, read length, power analysis, batch effects, controls, multiplexing, sample handling, and budgeting.

22.2 Biological Replicates vs Technical Replicates

Replicates

Biological replicates are independent biological samples—different animals, different cell culture flasks, different patients. They capture the natural biological variability in the system under study and are essential for statistical inference.

Technical replicates are repeated measurements of the same biological sample—the same RNA extraction sequenced twice, or the same library loaded on two lanes. They capture measurement variability but do not capture biological variability.

For most NGS applications, **biological replicates are far more important than technical replicates**. Modern sequencing platforms have sufficiently low technical variability that technical replication adds little value. Investment in additional biological replicates almost always provides more statistical power than technical replicates.

Common Mistake: Pseudoreplication

Pseudoreplication occurs when non-independent samples are treated as independent replicates. Examples include: (1) sequencing three aliquots of the same RNA extraction and calling them “triplicates,” (2) treating individual cells from the same animal as biological replicates in a single-cell experiment, or (3) splitting a single library across three lanes and treating the results as replicates. Pseudoreplication inflates statistical significance and produces unreliable results.

22.2.1 Minimum Replicates by Assay Type

The number of replicates required depends on the assay, the effect size you expect to detect, and the variability of your biological system. The following guidelines represent community consensus:

Table 22.1: Recommended minimum biological replicates by assay type.

Assay	Min Replicates	Notes
RNA-seq (differential expression)	3 (5+ recommended)	3 is the bare minimum; 5+ replicates per condition recommended for detecting moderate fold changes (<2-fold). With 3 replicates, only large effects are detectable.
ChIP-seq	2	ENCODE standard: 2 biological replicates per condition. Consistency measured by IDR (Irreproducible Discovery Rate).
ATAC-seq	2+	2 minimum; 3+ recommended for differential accessibility analysis.
WGS (germline)	1 per individual	Each individual is its own biological unit. No replication needed per individual.
WGS (somatic/cancer)	1 tumor + 1 normal	Matched tumor-normal pairs from the same patient.
scRNA-seq	2+ per condition	Biological replicates = independent samples (different animals/donors). Cells within a sample are NOT replicates.
WGBS	2-3	At least 2 per condition for DMR calling.
Hi-C	1-2	Deeply sequenced; replicates improve TAD boundary resolution.
16S amplicon	5-10+	High variability in microbiome studies demands more replicates.

More Replicates Are Always Better

If your budget allows, add more biological replicates rather than increasing sequencing depth beyond the recommended minimum. A study with 3 replicates at 50M reads each will almost always outperform a study with 2 replicates at 100M reads each for detecting differentially expressed genes.

22.3 Sequencing Depth Guidelines

Sequencing depth (the number of reads or coverage) must be matched to the application. Insufficient depth leads to incomplete data; excessive depth wastes resources with diminishing returns.

Depth vs Coverage vs Reads

These terms are often used interchangeably but have distinct meanings:

- **Reads:** The total number of sequencing reads generated (e.g., 30 million).
- **Coverage (fold coverage):** The average number of times each base in the target region is sequenced. For WGS: $\text{coverage} \approx (\text{number of reads} \times \text{read length}) / \text{genome size}$.
- **Depth:** Typically synonymous with coverage, but sometimes used to refer to the number of reads at a specific position.

Always specify whether you mean total reads, uniquely mapped reads, or fold coverage, as the distinction matters for experimental planning.

22.4 Read Length Considerations

The choice of read length and single-end (SE) versus paired-end (PE) sequencing depends on the application:

Paired-End vs Single-End

Paired-end (PE) sequencing reads both ends of each fragment, providing information about fragment size (insert size) and improving read mapping, especially in repetitive regions. PE is required for applications that rely on fragment size information (ATAC-seq, Hi-C) and strongly recommended for WGS and RNA-seq.

Single-end (SE) sequencing reads only one end, halving the data volume but losing fragment size information. SE is acceptable for ChIP-seq peak calling and some scRNA-seq protocols where the second read is short (barcode/UMI only).

22.5 Power Analysis

Power analysis estimates the number of replicates and sequencing depth needed to detect effects of a given size with a specified confidence level. It should be performed *before* the experiment to ensure adequate statistical power.

22.5.1 RNA-seq Power Analysis

RNASeqPower (Bioconductor)

Estimates power for detecting differential expression as a function of depth, coefficient

Table 22.2: Recommended sequencing depth by application.

Application	Recommended Depth	Notes
WGS (germline, human)	30× coverage	Standard for clinical and research germline calling. Higher (40–60×) for rare variant detection or structural variant analysis.
WGS (somatic, tumor)	60–100× tumor, 30–40× normal	Higher tumor depth needed to detect low-frequency somatic mutations. Ultra-deep (>200×) for cfDNA/liquid biopsy.
WES (exome)	100–200× on target	Actual unique coverage depends on enrichment uniformity.
RNA-seq (differential expression)	20–30M read pairs	Sufficient for robust differential expression of medium-to-high expressed genes.
RNA-seq (isoform/transcript discovery)	50–100M read pairs	Deeper sequencing needed to detect lowly expressed isoforms and novel transcripts.
scRNA-seq (10x Chromium)	20,000–50,000 reads per cell	Trade-off: more cells at lower depth vs fewer cells at higher depth. For most applications, more cells is preferred. Minimum 5,000 cells per sample recommended.
ATAC-seq	50–75M unique fragments per replicate	After filtering (mitochondrial, duplicates), expect 25–50M usable fragments. Paired-end sequencing required.
ChIP-seq (sharp marks)	20–40M reads	For transcription factors and H3K4me3. ENCODE recom-

Table 22.3: Read length recommendations by application.

Application	Recommended	Rationale
WGS	PE 150 bp	Standard; good balance of mappability and throughput.
WES	PE 75–150 bp	PE 100 or PE 150 both work well.
RNA-seq (DE)	PE 75–100 bp	Sufficient for gene-level quantification. Longer reads provide marginal benefit for DE.
RNA-seq (isoforms)	PE 150 bp	Longer reads improve splice junction detection and transcript assembly.
scRNA-seq (10x 3')	SE 28 + SE 91 (or SE 150)	R1 is barcode+UMI (28 bp); R2 is cDNA. SE 91 sufficient for gene counting.
ATAC-seq	PE 50–75 bp	Short PE reads are sufficient since fragments are short. PE required for fragment size analysis.
ChIP-seq	SE 50–75 bp or PE 75 bp	SE 50 is cost-effective for peak calling. PE improves duplicate detection.
CUT&Tag	PE 50 bp	Short PE reads; fragments are typically <500 bp.
Hi-C	PE 50–150 bp	PE 50 minimizes chimeric-read artefacts; PE 150 better for long-range interactions.
WGBS	PE 100–150 bp	Longer reads improve mapping of bisulfite-converted reads.
16S amplicon	PE 250–300 bp	Reads must overlap to span the full amplicon (e.g., V3–V4 $\approx$ 460 bp).

of variation (CV), fold change, sample size, and significance level. Based on the negative binomial model used by DESeq2/edgeR.

powsimR

A simulation-based power analysis tool that can model realistic RNA-seq data characteristics including mean-variance relationships, dropout, and multiple testing correction.

how_many_cells

Specifically designed for scRNA-seq power analysis: estimates the number of cells needed to detect a cell type or state of a given frequency in the population.

Listing 22.1: Power analysis for RNA-seq with RNASeqPower.

```

1 library(RNASeqPower)
2
3 # How many replicates do I need?
4 # Parameters: 30M reads, CV=0.4, effect=2-fold, alpha=0.05
5 rnapower(depth = 30, cv = 0.4, effect = 2, alpha = 0.05, power = 0.8)
6 # Returns: n = 3.2 -> need at least 4 replicates per condition
7
8 # Power at different replicate numbers
9 for (n in 2:8) {
10   pwr <- rnapower(depth = 30, cv = 0.4, effect = 2, alpha = 0.05, n = n)
11   cat(sprintf("n=%d: power=%.2f\n", n, pwr))
12 }
13 # Typical output:
14 # n=2: power=0.49
15 # n=3: power=0.73
16 # n=4: power=0.86

```

```

17 # n=5: power=0.92
18 # n=6: power=0.96

```

The Depth vs Replicates Trade-Off

For a fixed budget, how should you allocate resources between sequencing depth and replicates? Research consistently shows that **adding replicates provides more power than adding depth** once a minimum depth threshold is reached (approximately 10–15M reads for RNA-seq). Sequencing 3 replicates at 30M reads each is far more powerful than sequencing 2 replicates at 45M reads each. Prioritize replicates over depth.

22.5.2 Formal Power Analysis for Differential Expression

Power for detecting differential expression in RNA-seq depends on five key parameters: sample size n per group, effect size δ ($\log_2$ fold change), biological coefficient of variation ϕ (dispersion), sequencing depth d (in millions of reads), and significance threshold α . Under the negative binomial model used by DESeq2 and edgeR, the approximate power for a single gene is:

$$\text{Power} = \Phi \left(\frac{|\delta| \sqrt{n/(1/\mu + \phi)}}{\sigma_\delta} - z_{1-\alpha/2} \right) \quad (22.1)$$

where Φ is the standard normal CDF, μ is the mean expression (proportional to depth d), ϕ is the dispersion parameter (related to the coefficient of variation by $\text{CV}^2 = 1/\mu + \phi$), and $z_{1-\alpha/2}$ is the critical value. As depth d increases, $1/\mu$ becomes negligible and the variance is dominated by ϕ , the biological variability. This is why increasing replicates (which reduces the effective variance by $1/n$) is more effective than increasing depth once μ is large.

Listing 22.2: Formal power analysis with power curves.

```

1 library(RNASeqPower)
2
3 # Power as a function of sample size for different effect sizes
4 sample_sizes <- 2:12
5 cv <- 0.4 # Biological CV (typical for human tissue)
6 depth <- 20 # Millions of reads (mapped)
7
8 # Generate power curves
9 par(mar = c(4, 4, 2, 1))
10 plot(NA, xlim = c(2, 12), ylim = c(0, 1),
11      xlab = "Replicates per group (n)",
12      ylab = "Power",
13      main = "RNA-seq Power Analysis")
14 abline(h = 0.8, lty = 2, col = "gray")
15
16 effects <- c(1.5, 2.0, 2.5, 3.0) # Fold changes
17 cols <- c("red", "blue", "darkgreen", "purple")
18
19 for (i in seq_along(effects)) {
20   powers <- sapply(sample_sizes, function(n) {
21     rnapower(depth = depth, cv = cv, effect = effects[i],
22             alpha = 0.05, n = n)
23   })
24   lines(sample_sizes, powers, col = cols[i], lwd = 2)
25 }
26

```

```

27 legend("bottomright", legend = paste0(effects, "-fold"),
28      col = cols, lwd = 2, title = "Effect size")

```

Interpreting Power Curves

Power curves reveal several important patterns:

- For 2-fold changes (common in biology), at least 5–6 replicates per group are needed to reach 80% power with $CV = 0.4$.
- For small effects (<1.5 -fold), even 10+ replicates may provide insufficient power—consider whether the biological question is realistic.
- Diminishing returns set in beyond ~ 8 replicates; the power gain from replicate 9 to 10 is much smaller than from replicate 3 to 4.
- Reducing CV (by using more homogeneous samples, e.g., cell lines vs primary tissue) dramatically increases power at every sample size.

22.5.3 The Benjamini-Hochberg Procedure

When testing thousands of genes simultaneously, the multiple testing problem must be addressed. The **Benjamini-Hochberg (BH) procedure** controls the **false discovery rate (FDR)**—the expected proportion of false positives among all rejected hypotheses.

Benjamini-Hochberg Step-by-Step

Given m hypothesis tests with p-values $p_1, p_2, \dots, p_m$:

1. **Sort** the p-values: $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$.
2. **Find the largest** k such that $p_{(k)} \leq \frac{k}{m} \cdot \alpha$, where α is the desired FDR level (typically 0.05).
3. **Reject** all hypotheses $H_{(1)}, H_{(2)}, \dots, H_{(k)}$ (i.e., all hypotheses with $p \leq p_{(k)}$).

The BH procedure guarantees that the FDR is controlled at level α when the tests are independent or positively dependent. In DESeq2, the `padj` column contains BH-adjusted p-values (also called q-values).

Bonferroni vs BH: When It Matters

With $m = 20,000$ genes tested at $\alpha = 0.05$: Bonferroni requires $p < 0.05/20,000 = 2.5 \times 10^{-6}$ to reject, while BH can reject genes with much larger p-values as long as they fall below the BH threshold line. In a typical RNA-seq experiment, Bonferroni might find 50 DE genes while BH finds 500. For discovery-oriented research, BH is almost always the appropriate choice.

22.5.4 Pilot Study Design

A pilot study is a small preliminary experiment designed to estimate key parameters (biological variability, effect sizes) needed to power the full study.

Table 22.4: Comparison of multiple testing correction methods.

Method	Controls	Stringency	When to Use
Bonferroni	FWER	Most conservative	Few tests; when any false positive is unacceptable (e.g., clinical validation of specific variants)
Benjamini-Hochberg	FDR	Moderate	Standard for genome-wide discovery (DE genes, peaks, DMRs); accepts some false positives in exchange for greater sensitivity
Storey q-value	FDR	Less conservative	Estimates π_0 (proportion of true nulls) for higher power; used by <code>qvalue</code> R package
Independent Hypothesis Weighting (IHW)	FDR	Adaptive	Weights hypotheses by an independent covariate (e.g., mean expression); can increase power for RNA-seq

Designing a Pilot Study

1. **Determine the goal:** Estimate the biological coefficient of variation (ϕ) and approximate effect sizes for your system.
2. **Sample size:** Use 3–5 biological replicates per group. Fewer than 3 gives unreliable variance estimates; more than 5 is wasteful for a pilot.
3. **Sequencing depth:** Use the target depth for the full study (e.g., 20–30M reads for RNA-seq) to get realistic quality metrics.
4. **Process identically:** Use the same library preparation, sequencing platform, and bioinformatics pipeline planned for the full study.
5. **Estimate parameters:** From pilot data, estimate per-gene dispersion (ϕ) using DESeq2 or edgeR, and compute the distribution of observed fold changes.
6. **Power the full study:** Use the estimated ϕ and desired effect size in the RNASeqPower formula (Eq. (22.1)) to determine the required sample size for the main experiment.

Listing 22.3: Using pilot data to estimate dispersion and power the full study.

```

1 library(DESeq2)
2 library(RNASeqPower)
3
4 # Analyse pilot data (3 vs 3, treated vs control)
5 pilot_dds <- DESeqDataSetFromMatrix(
6   countData = pilot_counts, colData = pilot_meta,
7   design = ~ condition)

```

```

8 pilot_dds <- DESeq(pilot_dds)
9
10 # Extract median dispersion (biological CV2 approximation)
11 dispersions <- dispersions(pilot_dds)
12 median_disp <- median(dispersions, na.rm = TRUE)
13 estimated_cv <- sqrt(median_disp)
14 cat("Estimated biological CV:", round(estimated_cv, 2), "\n")
15
16 # Power analysis for full study using pilot estimates
17 for (n in c(3, 5, 7, 10, 12)) {
18   pwr <- rnapower(depth = 20, cv = estimated_cv,
19                 effect = 2, alpha = 0.05, n = n)
20   cat(sprintf("n=%2d per group: power = %.2f\n", n, pwr))
21 }

```

Common Pilot Study Mistakes

- **Too few replicates:** With only 2 per group, variance estimates are extremely noisy, leading to unreliable power calculations.
- **Incorporating pilot data into the main study:** If the pilot reveals issues (contamination, unexpected batch effects, protocol changes), the pilot data may not be combinable with the main study. Decide in advance whether pilot samples will be included.
- **Ignoring the pilot results:** Some researchers run a pilot but proceed with the planned sample size regardless. If the pilot reveals higher-than-expected variability, increase the sample size.
- **Different protocols:** A pilot run with a different library prep kit, extraction method, or sequencing platform provides misleading variance estimates.

22.5.5 Cost-Optimisation Model

For a fixed total budget B , the key trade-off in RNA-seq experimental design is between the number of samples n and the sequencing depth per sample d . The total cost can be modelled as:

$$B = n \cdot (C_{\text{lib}} + d \cdot C_{\text{seq}}) \quad (22.2)$$

where C_{lib} is the per-sample library preparation cost (typically \$50–200), d is the sequencing depth in millions of reads, and C_{seq} is the cost per million reads (typically \$1–5 depending on the platform and throughput).

Solving for n at a given depth:

$$n = \frac{B}{C_{\text{lib}} + d \cdot C_{\text{seq}}} \quad (22.3)$$

Listing 22.4: Cost-optimisation: finding the optimal allocation of budget to samples vs depth.

```

1 library(RNASeqPower)
2
3 # Budget parameters
4 budget <- 10000      # Total budget in USD
5 C_lib <- 150         # Library prep cost per sample
6 C_seq <- 3           # Cost per million reads
7
8 # Scan different depths to find optimal allocation

```

```

9 depths <- seq(5, 60, by = 1)
10 cv <- 0.4; effect <- 2; alpha <- 0.05
11
12 results <- data.frame(depth = depths, n = NA, power = NA)
13
14 for (i in seq_along(depths)) {
15   d <- depths[i]
16   n <- floor(budget / (C_lib + d * C_seq) / 2) # per group
17   if (n < 2) next
18   pwr <- rnapower(depth = d, cv = cv, effect = effect,
19                 alpha = alpha, n = n)
20   results$n[i] <- n
21   results$power[i] <- pwr
22 }
23
24 # Find optimal depth
25 best <- results[which.max(results$power), ]
26 cat(sprintf("Optimal: depth=%dM, n=%d per group, power=%.2f\n",
27           best$depth, best$n, best$power))
28
29 # Plot the trade-off
30 plot(results$depth, results$power, type = "l", lwd = 2,
31       xlab = "Depth (millions of reads)",
32       ylab = "Power",
33       main = sprintf("Budget = $%d: Samples vs Depth Trade-off", budget))
34 abline(v = best$depth, lty = 2, col = "red")
35 text(best$depth + 2, best$power, sprintf("n=%d/group", best$n),
36       col = "red", adj = 0)

```

The Cost-Optimisation Principle

For differential expression studies, the optimal budget allocation almost always favours more replicates at moderate depth over fewer replicates at high depth. This is because:

1. Statistical power depends on n (through the $\sqrt{n}$ term in the test statistic), but only weakly on depth once a threshold is reached.
2. Biological variance (ϕ) dominates technical variance ($1/\mu$) at moderate sequencing depths (>10–15M reads for RNA-seq).
3. The marginal cost of adding depth increases linearly, while the marginal power gain diminishes rapidly.

A typical optimum for RNA-seq with a \$10,000 budget is 8–10 replicates per group at 15–20M reads, rather than 4 replicates at 40M reads.

22.6 Batch Effects

Batch effects are systematic, non-biological variations introduced by technical factors during sample processing. They are one of the most common and insidious problems in NGS experiments.

22.6.1 Common Sources of Batch Effects

- **Different processing days:** Samples processed on Monday vs Friday may differ systematically.
- **Different operators:** Variation in technique between lab members.

- **Different reagent lots:** Enzyme, buffer, or kit lot-to-lot variation.
- **Different flow cells / lanes:** Sequencing run-to-run variation.
- **Different library preparation batches:** The most common source of batch effects.
- **Different sample storage conditions:** Fresh vs frozen, different storage durations.
- **Different extraction methods:** Changing protocols mid-experiment.

22.6.2 How to Minimize Batch Effects

Batch Effect Prevention Strategy

1. **Process all samples simultaneously** whenever possible. If not possible, distribute conditions evenly across batches.
2. **Randomize** sample assignment to library preparation batches, flow cell lanes, and sequencing runs.
3. **Never confound batch with biological condition.** If all treated samples are processed in batch 1 and all controls in batch 2, batch and condition effects are indistinguishable. This is the cardinal sin of experimental design.
4. **Use the same operator, reagent lots, and protocol** throughout the experiment.
5. **Include reference samples** across batches to quantify and correct batch effects.
6. **Record metadata:** Date of processing, operator, reagent lot, flow cell ID, lane. You cannot correct for batch effects you did not record.

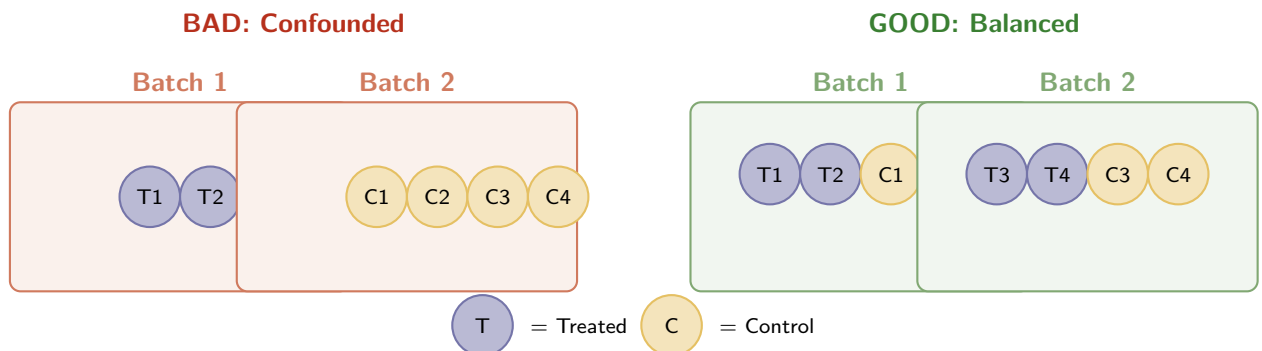


Figure 22.1: Left: a confounded design where batch and condition are indistinguishable. Right: a balanced design where both conditions are represented in every batch. The balanced design allows batch effects to be estimated and removed computationally.

22.6.3 Computational Batch Effect Correction

When batch effects cannot be completely prevented (e.g., multi-site studies, longitudinal experiments), they can be mitigated computationally—but only if batch is not confounded with condition:

ComBat / ComBat-seq

Empirical Bayes framework for batch correction. ComBat works on normalized data;

ComBat-seq works directly on count matrices (preferred for RNA-seq). Available in the `sva` R package.

limma::removeBatchEffect

Adjusts expression values for known batch variables. Useful for visualization (PCA, heatmaps) but not for differential expression testing—instead, include batch as a covariate in the DESeq2 design formula.

Harmony

Fast integration method for single-cell data. Corrects batch effects in PCA space, preserving biological variation. Widely used for integrating scRNA-seq datasets from different experiments or technologies.

Scanorama / scVI / BBKNN

Alternative single-cell integration methods. scVI uses a variational autoencoder; BBKNN modifies the nearest-neighbor graph. Each has different strengths depending on the severity of batch effects.

Batch Correction Cannot Fix Confounded Designs

If batch is perfectly confounded with the biological condition of interest (all treated samples in batch 1, all controls in batch 2), no computational method can distinguish biological effects from batch effects. The results will be unreliable regardless of how sophisticated the correction. Prevention through balanced experimental design is the only solution.

22.7 Controls

Appropriate controls are essential for distinguishing true signal from background noise and technical artefacts.

22.7.1 Sequencing Controls

PhiX spike-in

A balanced-composition bacteriophage genome library included at 1–5% in every Illumina sequencing run. PhiX serves as a quality control: it provides an independent estimate of error rate, helps with color calibration, and boosts sequence diversity for low-complexity libraries. For amplicon sequencing on two-channel instruments (NextSeq, NovaSeq X), spike in 20–50% PhiX.

Negative controls (no-template controls)

Process a “blank” sample (water or empty well) through the entire library preparation and sequencing workflow. Contamination in the negative control reveals kit reagent contamination or cross-contamination during processing. Critical for low-input protocols and metagenomics.

Positive controls

Include a well-characterised reference sample with known properties. Examples: the NA12878/HG001 reference genome for WGS variant calling validation, ERCC spike-in RNA mixes for RNA-seq, or a mock microbial community for 16S metagenomics.

22.7.2 Assay-Specific Controls

Input control (ChIP-seq)

Chromatin that has been cross-linked and sonicated but not immunoprecipitated. Serves

as the background model for peak calling. Essential for ChIP-seq; typically not needed for CUT&Tag (which uses IgG control or no-antibody control instead).

ERCC spike-ins (RNA-seq)

External RNA Controls Consortium (ERCC) synthetic RNA molecules added at known concentrations before library preparation. Enable assessment of technical variation, detection sensitivity, and (with appropriate normalization) absolute quantification.

Spike-in normalization (CUT&Tag)

Escherichia coli DNA carried over from the pA-Tn5 production in CUT&Tag experiments can serve as a spike-in for normalization. Alternatively, *Drosophila* chromatin can be added as an exogenous spike-in for cross-sample normalization.

Synthetic methylation standards (WGBS)

Fully methylated and fully unmethylated lambda phage or synthetic DNA spike-ins to assess bisulfite conversion efficiency. Conversion rates should exceed 99%.

22.8 Multiplexing Strategy

Multiplexing allows multiple samples to be sequenced on the same flow cell by adding unique index (barcode) sequences to each library. Proper multiplexing strategy is critical for cost-effective and accurate sequencing.

22.8.1 How Many Samples per Flow Cell

The number of samples that can be multiplexed depends on the required depth per sample and the flow cell output:

$$\text{Number of samples} = \frac{\text{Flow cell output (reads)}}{\text{Required reads per sample}} \quad (22.4)$$

For example, a NovaSeq X 10B flow cell producing 10 billion reads can accommodate approximately 330 WGS samples at 30× coverage (~30M read pairs each with PE150 for a 3 Gb genome), or 333 RNA-seq samples at 30M reads each.

22.8.2 Index Compatibility and Distance

Multiplexed libraries must have index sequences that are sufficiently different from each other to prevent misassignment. Key considerations:

- **Hamming distance:** Use indexes with a minimum Hamming distance of 3 (at least 3 positions differ), enabling 1-error correction.
- **Color balance:** On two-channel Illumina instruments, ensure that each cycle of the index read has a mixture of labelled and unlabelled bases (balanced A/C and G/T ratios).
- **Unique dual indexes (UDIs):** On patterned flow cells (NovaSeq, NextSeq 1000/2000), use UDIs—unique combinations of i7 and i5 indexes—to prevent index hopping. Combinatorial dual indexing (reusing the same i7 with different i5 across samples) is vulnerable to hopping on these platforms.

Index Hopping on Patterned Flow Cells

On patterned flow cells that use exclusion amplification (ExAmp), free adapter molecules can hybridize to the wrong nanowell, causing reads to be assigned to the wrong sample (“index hopping”). Rates of 0.1–2% have been reported. Use unique dual indexes (UDIs) rather than combinatorial dual indexes. Libraries from unrelated projects sharing a flow cell should be especially careful.

22.9 Sample Collection and Storage

The quality of sequencing results ultimately depends on the quality of the starting material. Nucleic acids are fragile molecules that can be degraded by nucleases, heat, pH extremes, and repeated freeze-thaw cycles.

22.9.1 DNA Storage

- **Fresh tissue:** Snap-freeze in liquid nitrogen immediately after collection. Store at -80°C .
- **Blood:** Collect in EDTA tubes (EDTA inhibits nucleases). Process within 24 hours or freeze.
- **FFPE (formalin-fixed, paraffin-embedded):** Standard for pathology archives. DNA from FFPE is degraded and chemically modified; special extraction kits and library preparation protocols are required. Expect shorter fragments and higher error rates.
- **Extracted DNA:** Store in TE buffer (10 mM Tris, 1 mM EDTA, pH 8.0) at -20°C for long-term storage or 4°C for short-term use. Avoid repeated freeze-thaw cycles.

22.9.2 RNA Storage

RNA is much more labile than DNA due to the ubiquity of RNases:

- **Fresh tissue:** Snap-freeze immediately or place in **RNAlater** (a stabilization reagent that penetrates tissue and inactivates RNases). RNAlater-preserved samples can be stored at room temperature for up to one week, or at -80°C for long-term storage.
- **Cultured cells:** Lyse directly in TRIzol or equivalent extraction reagent. Do not pellet cells and freeze without stabilization.
- **Blood RNA:** Collect in PAXgene or Tempus RNA blood tubes, which lyse cells and stabilize RNA immediately upon collection.
- **FFPE:** RNA from FFPE is highly degraded. The RNA Integrity Number (RIN) is typically <3 (vs 8–10 for high-quality RNA). Use FFPE-specific library preparation kits.

RNA Quality Assessment

Always assess RNA quality before library preparation using the Agilent Bioanalyzer, TapeStation, or equivalent. The **RNA Integrity Number (RIN)** ranges from 1 (fully degraded) to 10 (intact). For standard RNA-seq, a $\text{RIN} \geq 7$ is recommended. Some protocols (e.g., ribosomal RNA depletion) can handle more degraded RNA (RIN 3–7), but expect reduced data quality.

22.10 Cost Estimation and Budgeting

A realistic budget for an NGS experiment must account for all components of the workflow:

1. **Sample collection and processing:** IRB approval, sample collection supplies, nucleic acid extraction kits.
2. **Quality assessment:** Bioanalyzer/TapeStation chips (\$15–30 per sample), Qubit assays (\$2–5 per sample).
3. **Library preparation:** \$50–200 per sample depending on the kit and protocol.
4. **Sequencing:** The largest variable cost, ranging from \$50 per sample (low-coverage WGS or RNA-seq on a high-throughput instrument) to \$500+ (deep WGS on an individual flow cell).
5. **Computational:** HPC time, cloud computing charges, data storage (\$20–30 per TB per month for cloud storage).
6. **Personnel:** Bioinformatics analyst time is often the largest hidden cost.

Get Core Facility Quotes

Contact your institutional sequencing core facility or commercial provider early in the planning process. Core facilities can provide accurate per-sample costs, advise on multiplexing strategies, and often handle library preparation and QC. Commercial services (Novogene, BGI, Genewiz/Azenta, etc.) may offer lower per-sample costs for large projects.

22.11 Decision Tree for Choosing a Sequencing Strategy

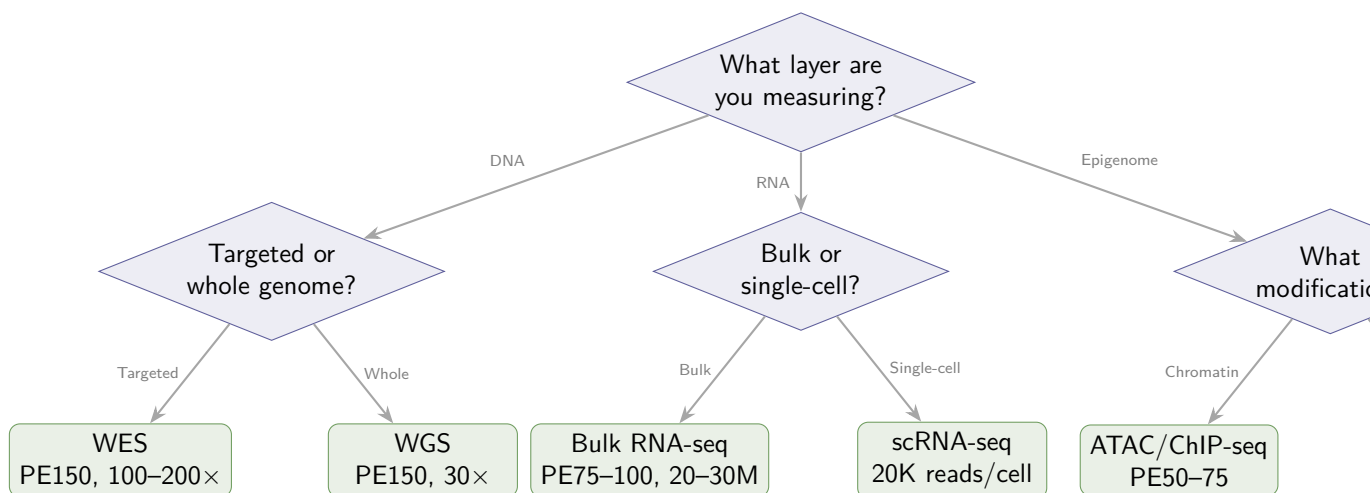


Figure 22.2: Simplified decision tree for selecting a sequencing strategy. Start with the biological layer of interest, then refine based on scope (whole-genome vs targeted) and resolution (bulk vs single-cell).

22.12 Comprehensive Experimental Design Reference Table

Table 22.5: Master experimental design reference: depth, replicates, read length, and key considerations for common NGS assays.

Assay	Depth	Replicates	Read Config	Key Design Notes
WGS (germline)	30×	1/individual	PE150	Matched normal for somatic analysis
WGS (somatic)	60–100×	1 T + 1 N	PE150	Higher depth for low-frequency variants
WES	100–200×	1/individual	PE100–150	Account for capture uniformity
RNA-seq (DE)	20–30M	3–5+/condition	PE75–100	Prioritize replicates over depth
RNA-seq (isoform)	50–100M	3+/condition	PE150	Long reads help with novel isoforms
scRNA-seq	20–50K/cell	2+/condition	R1:28, R2:91	More cells usually beats more depth
ATAC-seq	50–75M unique	2–3/condition	PE50–75	Remove mitochondrial reads
ChIP-seq (narrow)	20–40M	2/condition	SE50–75	Input control required
ChIP-seq (broad)	40–60M	2/condition	PE75	Broad peaks need more reads
CUT&Tag	3–8M	2/condition	PE50	Very low background; spike-in norm.
WGBS	10–30×	2–3/condition	PE100–150	Check bisulfite conversion rate
Hi-C	50M–2B	1–2	PE50–150	Depth determines resolution
16S amplicon	10–100K/sample	5–10+	PE250–300	Must overlap for amplicon assembly

22.13 Summary

Effective experimental design for NGS experiments requires careful consideration of multiple interacting factors:

- **Replication:** Biological replicates are essential; technical replicates are usually unnecessary. The minimum number of replicates depends on the assay and the effect size you need to detect.
- **Sequencing depth:** Must be matched to the application. Too little depth misses signal; too much wastes resources. When in doubt, more replicates at moderate depth outperform fewer replicates at higher depth.
- **Read length:** Choose based on the application. PE150 is the standard for WGS; shorter reads (PE50–100) are sufficient for many assays.
- **Power analysis:** Perform before the experiment to size the study appropriately. Tools like RNASeqPower and powsimR make this straightforward.
- **Batch effects:** The most common preventable source of error. Balance conditions across batches and never confound batch with condition.
- **Controls:** Include appropriate controls for every experiment—PhiX for sequencing QC, input/IgG for ChIP-seq, ERCC spike-ins for RNA-seq, negative controls for contamination detection.
- **Multiplexing:** Use unique dual indexes on patterned flow cells. Ensure index compatibility and color balance.
- **Sample quality:** Invest in proper sample collection, storage, and nucleic acid extraction. No amount of sequencing can rescue degraded samples.

Planning the experiment carefully before the first sample is collected is the single most impactful action you can take to ensure your sequencing project succeeds.

Chapter 23

Data Standards, Sharing, and Reproducibility

Key Concept

Genomics data has maximal value when it is shared openly, described with rich metadata, stored in standardised formats, and produced by reproducible methods. Data sharing is not merely a journal requirement—it is a fundamental pillar of the scientific process that enables reuse, meta-analysis, validation, and discovery far beyond the original study. This chapter describes the repositories, standards, and best practices for making NGS data findable, accessible, interoperable, and reusable.

23.1 Why Data Sharing and Standards Matter

The genomics community generates petabytes of data annually. Without standards and public repositories, this data would be siloed in individual laboratories, inaccessible for reuse, and lost when hard drives fail or students graduate. The benefits of data sharing are substantial:

- **Reproducibility:** Independent researchers can verify published results using the original data.
- **Discovery:** Public datasets enable new analyses that the original authors did not envision. Many high-impact papers are entirely based on re-analysis of public data.
- **Meta-analysis:** Combining data across studies increases statistical power and reveals patterns invisible in individual datasets.
- **Training resources:** Public data provides invaluable training material for bioinformatics education.
- **Cost savings:** Reusing existing data avoids redundant sequencing of the same biological materials.
- **Equity:** Open data enables researchers in resource-limited settings to participate in cutting-edge genomics research.

Most funding agencies (NIH, ERC, Wellcome Trust, BBSRC, DFG) and journals (Nature, Science, Cell, PNAS, Genome Biology, etc.) now mandate data deposition in public repositories as a condition of funding or publication.

23.2 Public Repositories for Raw Sequencing Data

23.2.1 The International Nucleotide Sequence Database Collaboration (INSDC)

The three primary archives for raw sequencing data are part of the INSDC, which synchronizes data globally:

NCBI SRA (Sequence Read Archive)

The largest raw sequencing data archive, hosted by the US National Center for Biotechnology Information. SRA stores raw reads in a compressed format and provides programmatic access via the SRA Toolkit. As of 2026, SRA contains over 80 petabases of data from millions of experiments.

ENA (European Nucleotide Archive)

The European counterpart to SRA, hosted by EMBL-EBI. ENA stores the same data (mirrored from SRA) and provides a more flexible submission interface. ENA additionally stores assembled sequences and functional annotations.

DDBJ (DNA Data Bank of Japan)

Japan's nucleotide archive, operated by the National Institute of Genetics. Also mirrors INSDC data.

Data submitted to any one of these archives is automatically shared with the other two within 24–48 hours. You only need to submit once.

23.2.2 How to Submit Data to SRA/ENA

Data submission requires three levels of metadata organized in a hierarchical structure:

1. **BioProject:** A top-level umbrella describing the overall research project. Contains a title, description, and organism. Each publication typically corresponds to one BioProject.
2. **BioSample:** Describes each biological sample (organism, tissue type, genotype, treatment, collection date). One BioSample per biological unit (e.g., per patient, per cell line).
3. **Experiment / Run:** Links a BioSample to a library preparation strategy and sequencing run. Contains library metadata (insert size, read length, platform, instrument model) and the actual FASTQ files.

Listing 23.1: Submitting data to SRA using the SRA Toolkit.

```

1 # Register BioProject and BioSamples via NCBI Submission Portal
2 # https://submit.ncbi.nlm.nih.gov/
3
4 # Upload FASTQ files using the SRA submission tool or Aspera
5 # Using Aspera (much faster than FTP for large files)
6 ascp -i ~/.aspera/connect/etc/asperaweb_id_dsa.openssh \
7     -QT -l 1000m -k 1 \
8     sample1_R1.fastq.gz sample1_R2.fastq.gz \
9     subasp@upload.ncbi.nlm.nih.gov:uploads/your_key_folder/
10
11 # Alternative: submit via ENA using their Webin-CLI tool
12 java -jar webin-cli.jar -context reads \

```



```

13 -manifest manifest.txt \
14 -userName Webin-XXXXX -password XXXXX \
15 -submit

```

23.2.3 How to Download Data from SRA/ENA

Listing 23.2: Downloading data from SRA.

```

1 # Using fasterq-dump (faster successor to fastq-dump)
2 fasterq-dump --split-files --threads 8 SRR12345678
3 # Produces: SRR12345678_1.fastq and SRR12345678_2.fastq
4 gzip SRR12345678_*.fastq
5
6 # Download multiple accessions with a loop
7 while read acc; do
8     fasterq-dump --split-files --threads 8 "$acc"
9     gzip "${acc}"_*.fastq
10 done < accession_list.txt
11
12 # Alternative: download directly from ENA (already in FASTQ format)
13 wget
14     ftp://ftp.sra.ebi.ac.uk/vol1/fastq/SRR123/078/SRR12345678/SRR12345678_1.fastq.gz
15
16 # Programmatic access with pysradb (Python)
17 pip install pysradb
18 pysradb metadata --detailed SRP123456
19 pysradb download -t 8 SRP123456

```

ENA vs SRA for Downloads

Downloading from ENA is often faster and more convenient than from SRA, because ENA provides files directly in FASTQ format (no conversion needed), while SRA stores data in its own SRA format requiring conversion with **fasterq-dump**. ENA also provides direct FTP and Aspera download links.

23.2.4 Snakemake Workflow for SRA Data Download

Listing 23.3: Snakemake workflow for downloading SRA data.

```

1 # Snakefile -- Download data from SRA
2 configfile: "config.yaml"
3
4 ACCESSIONS = config["sra_accessions"]
5
6 rule all:
7     input:
8         expand("data/{acc}_1.fastq.gz", acc=ACCESSIONS),
9         expand("data/{acc}_2.fastq.gz", acc=ACCESSIONS)
10
11 rule download_sra:
12     output:
13         r1="data/{acc}_1.fastq.gz",
14         r2="data/{acc}_2.fastq.gz"
15     threads: 8
16     conda: "envs/sra.yaml"
17     shell:

```

```

18     """
19     fasterq-dump --split-files --threads {threads} \
20         --outdir data/ {wildcards.acc}
21     gzip data/{wildcards.acc}_1.fastq
22     gzip data/{wildcards.acc}_2.fastq
23     """

```

23.2.5 Nextflow Workflow for SRA Data Download

Listing 23.4: Nextflow process for downloading SRA data.

```

1  #!/usr/bin/env nextflow
2  nextflow.enable.dsl = 2
3
4  params.accessions = "accession_list.txt"
5  params.outdir     = "data"
6
7  process DOWNLOAD_SRA {
8      tag "${accession}"
9      publishDir "${params.outdir}", mode: 'copy'
10     cpus 8
11
12     input: val(accession)
13     output: tuple val(accession), path("*.fastq.gz"), emit: fastq
14
15     script:
16     """
17     fasterq-dump --split-files --threads ${task.cpus} ${accession}
18     gzip *.fastq
19     """
20 }
21
22 workflow {
23     accessions_ch = Channel
24         .fromPath(params.accessions)
25         .splitText()
26         .map { it.trim() }
27
28     DOWNLOAD_SRA(accessions_ch)
29 }

```

23.3 Processed Data Repositories

Beyond raw sequencing data, processed results (gene expression matrices, called peaks, variant lists) are deposited in specialized repositories:

23.3.1 Gene Expression Omnibus (GEO)

NCBI GEO is the most widely used repository for processed functional genomics data. A GEO submission typically includes:

- Processed data files (count matrices, normalized expression values, peak files)
- Sample metadata (conditions, treatments, organism, tissue)
- A link to the corresponding SRA submission for raw data

GEO assigns **GSE** accessions (for series/datasets) and **GSM** accessions (for individual samples). Researchers can browse and download GEO data via the web interface or programmatically using the `GEOquery` R package.

23.3.2 ArrayExpress / BioStudies

EMBL-EBI's equivalent to GEO, now transitioning to the BioStudies platform. ArrayExpress/BioStudies stores processed data with rich metadata in the MAGE-TAB format. Data is cross-referenced with ENA for raw reads.

23.3.3 Consortium Data Portals

Several large-scale consortia maintain their own data portals with curated, consistently processed datasets:

ENCODE (Encyclopedia of DNA Elements)

Gold-standard ChIP-seq, ATAC-seq, RNA-seq, and Hi-C datasets with uniform processing pipelines. The ENCODE portal (encodeproject.org) provides downloadable processed files (BigWig, BED, BAM) and comprehensive quality metrics. ENCODE data quality standards serve as benchmarks for the field.

Roadmap Epigenomics

Comprehensive epigenomic maps (histone modifications, DNA methylation, chromatin accessibility) across 111 human tissues and cell types. Data available from the Roadmap Epigenomics portal.

GTEx (Genotype-Tissue Expression)

Gene expression data across 54 human tissues from nearly 1,000 post-mortem donors. GTEx is the primary resource for understanding tissue-specific gene expression and expression quantitative trait loci (eQTLs).

TCGA / GDC (The Cancer Genome Atlas / Genomic Data Commons)

Multi-omics data (WGS, WES, RNA-seq, methylation, miRNA, proteomics) for over 11,000 tumors across 33 cancer types. Accessible through the NCI Genomic Data Commons portal. Some data requires dbGaP access approval due to patient privacy.

Human Cell Atlas (HCA)

A global effort to create reference single-cell maps of every cell type in the human body. Data available through the HCA Data Portal.

CellxGene

A platform maintained by the Chan Zuckerberg Initiative for exploring and downloading curated single-cell datasets. Provides interactive visualization and standardized AnnData (h5ad) downloads.

UCSC Genome Browser and Table Browser

Comprehensive genome annotations, conservation tracks, regulatory elements, and variation data. The Table Browser allows bulk download of genomic tracks in BED, BigWig, and other formats.

Ensembl and BioMart

Genome annotations, gene models, orthology information, and variation data for hundreds of species. BioMart provides a powerful query interface for programmatic data retrieval.

23.4 Reference Genomes and Annotations

Consistent use of reference genomes and gene annotations is essential for reproducibility and cross-study comparability.

23.4.1 Where to Download References

GENCODE

The gold standard for human and mouse gene annotations. Provides comprehensive annotations including protein-coding genes, lncRNAs, pseudogenes, and other non-coding elements. Available from gencodegenes.org. Current release: GENCODE v46 (human) based on GRCh38.

Ensembl

Annotations for hundreds of species. Ensembl annotations for human are closely related to GENCODE but follow a different release numbering. Available from ensembl.org.

RefSeq (NCBI)

NCBI's curated reference sequence database. RefSeq annotations tend to be more conservative (fewer predicted transcripts) than GENCODE. Used in clinical settings where annotation confidence is prioritized.

UCSC Genome Browser

Downloads for genome sequences, annotations, and tracks at hgdownload.soe.ucsc.edu.

23.4.2 GENCODE vs RefSeq

Table 23.1: Comparison of GENCODE and RefSeq annotations.

Feature	GENCODE	RefSeq
Provider	EMBL-EBI / Wellcome Sanger	NCBI
Approach	Comprehensive (manual + automated)	Curated (conservative)
Transcript count	Higher (more predicted isoforms)	Lower (more stringent evidence)
Identifier format	ENSG/ENST (Ensembl IDs)	NM_, NR_, XM_ (RefSeq IDs)
Non-coding genes	Extensive annotation	More conservative
Clinical use	Research standard	Often preferred in clinical settings
Update frequency	Biannual releases	Continuous updates

Mixing References Is Dangerous

Never mix genome builds or annotation versions within a single analysis. Using a GRCh37 VCF with a GRCh38 annotation file, or a GENCODE GTF with RefSeq gene names, can produce incorrect or uninterpretable results. Document the exact genome build, annotation source, and release version in every analysis.

23.4.3 Genome Builds and LiftOver

When comparing results across studies that used different genome builds (e.g., GRCh37/hg19 vs GRCh38/hg38), coordinate conversion is necessary:

Listing 23.5: Converting coordinates between genome builds with liftOver.

```
1 # Download the chain file for hg19 -> hg38 conversion
```

```

2 wget
   https://hgdownload.soe.ucsc.edu/goldenPath/hg19/liftOver/hg19ToHg38.over.chain.gz
3
4 # Convert BED coordinates from hg19 to hg38
5 liftOver input_hg19.bed hg19ToHg38.over.chain.gz \
6   output_hg38.bed unmapped.bed
7
8 # In R, use rtracklayer
9 library(rtracklayer)
10 chain <- import.chain("hg19ToHg38.over.chain")
11 gr_hg38 <- liftOver(gr_hg19, chain)

```

23.5 Metadata Standards

Metadata—data about data—is what makes raw sequencing files interpretable. Without knowing the organism, tissue type, experimental condition, library preparation method, and sequencing parameters, a FASTQ file is just a meaningless stream of letters.

23.5.1 Key Metadata Standards

MINSEQE (Minimum Information about a high-throughput Nucleotide Sequencing Experiment)

The community standard for NGS metadata, analogous to MIAME for microarrays. MINSEQE specifies the minimum information needed to interpret and reproduce a sequencing experiment: (1) raw data, (2) processed data, (3) experiment design including sample descriptions, (4) experimental protocols, and (5) data processing protocols.

MIAME (Minimum Information About a Microarray Experiment)

The historical predecessor to MINSEQE, established for microarray experiments. Many of its principles apply to sequencing experiments.

Dublin Core

A general-purpose metadata standard (title, creator, date, description, identifier) used for cataloguing research outputs. Forms the basis for many repository metadata schemas.

23.5.2 Ontologies for Standardized Terminology

Free-text metadata fields (e.g., “brain tissue,” “brain,” “cerebral cortex”) create inconsistencies that hinder data integration. Ontologies provide controlled vocabularies:

EFO (Experimental Factor Ontology)

Standardized terms for experimental variables: diseases, cell types, anatomical locations, organisms, and assay types. Used by GEO, ArrayExpress, and GWAS Catalog.

OBI (Ontology for Biomedical Investigations)

Terms for experimental protocols, instruments, and data transformation processes.

CL (Cell Ontology)

Standardized cell type names. Essential for single-cell studies where consistent cell type labelling is critical for cross-study comparisons.

UBERON

Cross-species anatomical ontology. Enables comparison of tissues across organisms (e.g., human “liver” maps to mouse “liver”).

23.6 FAIR Principles

The **FAIR principles** provide a framework for making data maximally useful:

FAIR Data Principles

Findable Data should have globally unique, persistent identifiers (DOIs, accession numbers). Rich metadata should be registered in searchable resources.

Accessible Data should be retrievable using standardised, open protocols. Even if data access requires authentication (e.g., for patient data), the metadata should always be accessible.

Interoperable Data should use standardised formats (FASTQ, BAM, VCF) and controlled vocabularies (ontologies). This enables integration across datasets and tools.

Reusable Data should include clear provenance information, usage licenses, and detailed metadata following community standards. This enables meaningful reuse for new purposes.

For NGS data, FAIR compliance means:

- Submitting raw data to SRA/ENA with complete BioProject/BioSample metadata.
- Using standard file formats (FASTQ, BAM, VCF, BED, BigWig) rather than custom formats.
- Providing processing code in public repositories (GitHub/GitLab) with DOIs (Zenodo).
- Using ontology terms rather than free text for sample descriptions.
- Publishing under open-access licenses when possible.

23.7 Reproducibility in Bioinformatics

Computational reproducibility—the ability to regenerate the same results from the same input data—remains a significant challenge in bioinformatics. Studies have shown that a large fraction of published bioinformatics analyses cannot be reproduced, even by the original authors, due to undocumented software versions, missing code, or unavailable data.

23.7.1 Pillars of Computational Reproducibility

1. **Version pinning for software:** Record the exact version of every tool used. A Conda environment file (`environment.yml`) or container image digest provides this.
2. **Version pinning for reference data:** Record the exact genome build (e.g., GRCh38 patch 14), annotation version (e.g., GENCODE v44), and database versions (e.g., dbSNP b156, gnomAD v4.0).
3. **Containers (Docker, Singularity):** Package the complete software environment—operating system, libraries, and tools—into a portable, immutable image. The same container produces identical results regardless of the host system.
4. **Workflow managers (Snakemake, Nextflow):** Encode the complete analysis pipeline as executable code. The workflow file serves as a precise, machine-readable protocol.

5. **Literate programming:** Combine narrative, code, and results in a single document using R Markdown, Quarto, or Jupyter notebooks. This ensures that figures and tables in a manuscript can be traced directly to the code that generated them.
6. **Code repositories:** Host all analysis code on GitHub or GitLab. Tag specific commits corresponding to publication versions.
7. **Zenodo:** Archive code repositories on Zenodo to obtain a DOI. GitHub repositories can change or be deleted; Zenodo provides permanent archival with a citable DOI.

23.7.2 Example Reproducible Project Structure

Listing 23.6: Recommended directory structure for a reproducible bioinformatics project.

```

1 my_ngs_project/
2 |-- README.md           # Project description and instructions
3 |-- LICENSE             # Open-source license (e.g., MIT, GPL)
4 |-- Snakefile           # Main Snakemake workflow (or main.nf)
5 |-- nextflow.config     # Nextflow configuration (if using NF)
6 |-- config.yaml         # Project-specific parameters
7 |-- envs/               # Conda environment files
8 |   |-- align.yaml
9 |   |-- count.yaml
10 |   |-- deseq2.yaml
11 |-- containers/         # Dockerfiles (if not using Biocontainers)
12 |   |-- Dockerfile
13 |-- scripts/           # Custom analysis scripts
14 |   |-- run_deseq2.R
15 |   |-- plot_figures.R
16 |   |-- run_scanpy.py
17 |-- notebooks/         # Exploratory analyses
18 |   |-- exploration.Rmd
19 |   |-- qc_report.ipynb
20 |-- data/              # Raw data (NOT committed to Git)
21 |   |-- README.md       # How to obtain the data
22 |-- ref/               # Reference files (NOT committed to Git)
23 |   |-- README.md       # Download URLs and versions
24 |-- results/           # Pipeline outputs (NOT committed to Git)
25 |-- .gitignore          # Exclude data/, results/, ref/

```

23.7.3 Snakemake Example for Reproducible Analysis

Listing 23.7: Snakemake workflow for a reproducible project with containers.

```

1 # Snakefile -- Reproducible analysis with containers
2 configfile: "config.yaml"
3
4 # Use Singularity containers for every rule
5 singularity: "docker://continuumio/miniconda3:latest"
6
7 rule all:
8     input:
9         "results/final_report.html"
10
11 rule align:
12     input:
13         r1="data/{sample}_R1.fastq.gz",

```

```

14     r2="data/{sample}_R2.fastq.gz"
15     output:
16         "results/aligned/{sample}.bam"
17     threads: 16
18     container:
19         "docker://biocontainers/bwa:v0.7.17_cv1"
20     shell:
21         """
22         bwa mem -t {threads} {config[reference]} \
23             {input.r1} {input.r2} \
24             | samtools sort -o {output} -
25         """
26
27 rule report:
28     input:
29         expand("results/aligned/{s}.bam", s=config["samples"])
30     output:
31         "results/final_report.html"
32     container:
33         "docker://rocker/tidyverse:4.3.2"
34     script:
35         "scripts/generate_report.Rmd"

```

23.7.4 Nextflow Example for Reproducible Analysis

Listing 23.8: Nextflow workflow with container-based reproducibility.

```

1  #!/usr/bin/env nextflow
2  nextflow.enable.dsl = 2
3
4  params.reads = "data/*_{R1,R2}.fastq.gz"
5  params.ref = "ref/GRCh38.fa"
6  params.outdir = "results"
7
8  process ALIGN {
9      container 'biocontainers/bwa:v0.7.17_cv1'
10     cpus 16
11     publishDir "${params.outdir}/aligned", mode: 'copy'
12
13     input:
14         tuple val(sample_id), path(reads)
15         path(ref)
16
17     output:
18         tuple val(sample_id), path("*.sorted.bam"), emit: bam
19
20     script:
21     def (r1, r2) = reads
22     """
23     bwa mem -t ${task.cpus} ${ref} ${r1} ${r2} \
24         | samtools sort -o ${sample_id}.sorted.bam -
25     """
26 }
27
28 process REPORT {
29     container 'rocker/tidyverse:4.3.2'
30     publishDir "${params.outdir}", mode: 'copy'

```



```

31  input: path(bams)
32  output: path("final_report.html")
33
34
35  script:
36  """
37  Rscript -e "rmarkdown::render('${projectDir}/scripts/report.Rmd', \
38    output_dir = '.')"
39  """
40 }
41
42 workflow {
43   reads_ch = Channel.fromFilePairs(params.reads, checkIfExists: true)
44   ref_ch   = Channel.value(file(params.ref))
45
46   ALIGN(reads_ch, ref_ch)
47   REPORT(ALIGN.out.bam.map { it[1] }.collect())
48 }

```

23.8 Data Management Plans

Most funding agencies require a Data Management Plan (DMP) as part of grant applications. A DMP describes how data will be generated, managed, shared, and preserved.

23.8.1 What Funding Agencies Require

NIH (USA)

Since January 2023, the NIH Data Management and Sharing Policy requires all funded research to submit a DMP describing: (1) data types generated, (2) data standards and metadata, (3) preservation and access plans, (4) timeline for sharing, and (5) oversight. Genomic data must be shared through approved repositories (SRA, GEO, dbGaP for controlled-access data).

ERC (European Research Council)

Requires compliance with the EU Open Science policy. Research data should be “as open as possible, as closed as necessary.” Open access to publications and FAIR data sharing are expected.

Wellcome Trust (UK)

Expects all research outputs, including data and code, to be shared openly. Requires a DMP and supports data deposition costs.

23.8.2 Storage Considerations

Data Retention Strategy

Not all data needs to be kept permanently. Raw FASTQ files should be archived in SRA/ENA (free, permanent storage). Intermediate files (unsorted BAMs, temporary files) can be deleted after the pipeline completes. Final processed results (VCFs, count matrices, peak files) should be retained locally and deposited in GEO or equivalent. BAM files can often be regenerated from FASTQ, so consider deleting them after analysis if storage is limited.

Table 23.2: Typical data volumes and storage requirements for NGS projects.

Data Type	Size per Sample	Compression	Storage Notes
FASTQ (30× WGS)	60–90 Gb	gzip: 3–4×	Archive in SRA; delete local copies after verification
BAM (30× WGS)	50–80 Gb	CRAM: 30–60% smaller	Regenerable from FASTQ; archive or delete
VCF	0.1–1 Gb	bgzip + tabix	Small; retain permanently
RNA-seq FASTQ	3–8 Gb	gzip	Archive in SRA
scRNA-seq (h5ad)	0.5–5 Gb	zlib compressed	Retain processed object
BigWig tracks	0.1–1 Gb	Pre-compressed	Retain for visualization

23.8.3 Data Security and Patient Consent

Genomic data from human subjects requires careful attention to privacy and consent:

HIPAA (USA)

The Health Insurance Portability and Accountability Act governs the use and disclosure of protected health information. Genomic data linked to patient identifiers is subject to HIPAA. De-identification is required for open sharing; controlled-access repositories (dbGaP) are used for data that cannot be fully de-identified.

GDPR (EU)

The General Data Protection Regulation classifies genetic data as “special category” personal data, subject to strict processing requirements. Explicit consent, data minimization, and purpose limitation principles apply. Cross-border data transfer requires specific legal mechanisms.

Controlled-access repositories

Human genomic data that poses re-identification risk is deposited in controlled-access repositories (dbGaP in the US, EGA in Europe). Researchers must apply for access, agree to data use conditions, and have their institution sign a data access agreement.

Genomic Data Is Inherently Identifiable

Even “de-identified” genomic data can potentially be re-identified by matching variant patterns to public genealogy databases or other genomic datasets. A study demonstrated that individuals could be re-identified from as few as 75 independent SNPs. Always follow your institution’s IRB guidelines and obtain appropriate consent for data sharing.

23.8.4 HIPAA vs GDPR Detailed Comparison

Researchers working with human genomic data must navigate different regulatory frameworks depending on the jurisdiction. Table 23.3 compares the two most important frameworks.

Practical Implications for Genomics Researchers

If your study involves collaborators or participants across US and EU jurisdictions, you must comply with *both* HIPAA and GDPR. In practice, this means: (1) obtain explicit consent that satisfies GDPR requirements (which are stricter), (2) use controlled-access repositories (dbGaP, EGA) for data sharing, (3) implement data processing agreements for cross-border transfers, and (4) consult your institutional privacy officer before beginning the study.

Table 23.3: Comparison of HIPAA (USA) and GDPR (EU) for genomic data.

Aspect	HIPAA (USA)	GDPR (EU)
Scope	Applies to “covered entities” (healthcare providers, insurers) and their business associates	Applies to all organisations processing personal data of EU residents, regardless of location
Genomic data classification	Protected Health Information (PHI) when linked to identifiers; not explicitly classified as a special category	Explicitly classified as “genetic data” under Article 9 (special category), requiring additional safeguards
Consent requirements	Authorization required for use/disclosure; research exemption via IRB-approved waiver possible	Explicit consent required for processing genetic data; broad consent for future research possible under certain conditions
De-identification	Safe Harbor (remove 18 identifiers) or Expert Determination method; de-identified data is no longer PHI	Pseudonymisation reduces risk but data remains “personal data” under GDPR; full anonymisation (irreversible) removes GDPR obligations
Breach notification	Within 60 days to HHS and affected individuals; without unreasonable delay	Within 72 hours to supervisory authority; without undue delay to affected individuals if high risk
Cross-border data transfer	No specific restrictions within the US; international transfers follow institutional policy	Transfers outside EU/EEA require adequacy decision, Standard Contractual Clauses (SCCs), or Binding Corporate Rules
Penalties	Up to \$1.9 million per violation category per year; criminal penalties possible	Up to 4% of global annual turnover or €20 million, whichever is higher
Right to deletion	No general right to deletion of PHI; retention required for specified periods	“Right to be forgotten” (Article 17); individuals can request deletion of their genetic data
Research exemption	Common Rule (45 CFR 46) provides exemptions for de-identified data and IRB-waived consent	Research exemption available under Article 89, subject to appropriate safeguards

23.8.5 Controlled Access Repositories and Federated Analysis

Human genomic data that cannot be fully de-identified is deposited in controlled-access repositories. Access requires a formal application, institutional approval, and a signed data access agreement.

dbGaP (Database of Genotypes and Phenotypes, USA)

Operated by NCBI. Hosts individual-level genotype and phenotype data from NIH-funded studies (GWAS, WGS, WES, RNA-seq). Access is granted by the relevant Data Access Committee (DAC) after institutional sign-off. Typical approval time: 2–6 weeks. Cloud access is available through the NHGRI AnVIL platform and the NCI Cancer Research Data Commons.

EGA (European Genome-phenome Archive)

Operated by EMBL-EBI and the Centre for Genomic Regulation (CRG). The European counterpart to dbGaP. Hosts individual-level data from European and international studies. Access is controlled by study-specific DACs appointed by the data generators.

DDBJ-JGA (Japanese Genotype-phenome Archive)

Operated by DDBJ/NIG in Japan. Hosts controlled-access data from Japanese research projects. Functions similarly to dbGaP and EGA.

Federated Analysis: Data Stays, Compute Moves

Traditional data sharing requires downloading data to local infrastructure, creating security risks and bandwidth bottlenecks. **Federated analysis** inverts this model: instead of moving data to the researcher, the researcher's analysis code moves to the data.

The **GA4GH** (Global Alliance for Genomics and Health) is developing standards that enable federated analysis:

Beacon API Allows querying whether a specific variant exists in a dataset without revealing individual-level data. Beacon v2 supports richer queries (phenotypes, cohort-level statistics).

Passports A standard for researcher identity and access credentials that enables single sign-on across federated data nodes.

DRS (Data Repository Service) A standardised API for locating and accessing data objects across distributed repositories.

TES/WES (Task/Workflow Execution Service) Standards for submitting analysis workflows to remote compute platforms where the data resides.

23.8.6 Cross-Repository Data Discovery

Public genomic data is distributed across multiple repositories (GEO, SRA, ENA, DDBJ, ArrayExpress). Finding relevant datasets requires searching across these sources.

NCBI Entrez (unified search)

The NCBI Entrez system provides a single search interface across all NCBI databases (SRA, GEO, BioProject, BioSample, PubMed). Use the **esearch** and **efetch** command-line tools from Entrez Direct, or the **rentrez** R package for programmatic access.

BioProject/BioSample hierarchy

Every INSDC submission is organised into a BioProject (study-level) and BioSample (sample-level) hierarchy. Searching by BioProject accession retrieves all associated runs, samples, and metadata. This hierarchy connects raw data (SRA), processed data (GEO), and publications (PubMed).

EBI Search

EMBL-EBI provides a similar unified search across ENA, ArrayExpress/BioStudies, UniProt, and other EBI resources.

SRA Run Selector

The NCBI SRA Run Selector allows filtering and downloading metadata for all runs associated with a BioProject, enabling programmatic selection of specific samples.

Listing 23.9: Searching across repositories with NCBI Entrez Direct.

```

1 # Install Entrez Direct
2 sh -c "$(curl -fsSL
   https://ftp.ncbi.nlm.nih.gov/entrez/entrezdirect/install-edirect.sh)"
3
4 # Search SRA for human ATAC-seq datasets from 2024-2025
5 esearch -db sra -query "ATAC-seq[Strategy] AND Homo sapiens[Organism] \
6   AND 2024:2025[Publication Date]" | \
7   efetch -format runinfo | head -20
8
9 # Find all SRA runs for a specific BioProject
10 esearch -db sra -query "PRJNA123456[BioProject]" | \
11   efetch -format runinfo > runinfo.csv
12
13 # Search GEO for scRNA-seq datasets in a specific tissue
14 esearch -db gds -query "scRNA-seq AND brain AND Homo sapiens" | \
15   efetch -format summary | head -50

```

Listing 23.10: Programmatic GEO search with the GEOquery R package.

```

1 library(GEOquery)
2
3 # Download a GEO dataset and explore metadata
4 gse <- getGEO("GSE123456", GSEMatrix = TRUE)
5 metadata <- pData(gse[[1]])
6 head(metadata[, c("title", "geo_accession", "source_name_ch1")])
7
8 # Search GEO using rentrez
9 library(rentrez)
10 search_results <- entrez_search(
11   db = "gds",
12   term = "scRNA-seq[Title] AND Homo sapiens[Organism] AND 2024[PDAT]",
13   retmax = 50
14 )
15 cat("Found", search_results$count, "datasets\n")

```

23.8.7 Long-Term Storage Cost Modelling

Genomic data volumes grow faster than storage costs decline, making storage cost modelling essential for long-term project planning.

CRAM vs BAM compression: CRAM files use reference-based compression, typically achieving 40–60% of the corresponding BAM file size. For a 30× WGS sample, a BAM file of ~70 GB becomes a CRAM file of ~30–40 GB. Over many samples and years, this compression translates to substantial savings:

$$\text{Annual storage cost} = V \cdot r \cdot C_{\text{tier}} \cdot 12 \quad (23.1)$$

where V is the data volume in TB, r is the compression ratio (1.0 for BAM, ~0.5 for CRAM, ~0.3 for FASTQ.gz), and C_{tier} is the monthly cost per TB for the chosen storage tier.

Table 23.4: AWS S3 storage tiers and approximate costs (as of 2025). Costs vary by region.

Storage Tier	Cost/TB/month	Retrieval Cost	Retrieval Time	Best For
S3 Standard	\$23	Free	Instant	Active analysis data
S3 Infrequent Access (IA)	\$12.50	\$0.01/GB	Instant	Intermediate files accessed occasionally
S3 Glacier Instant	\$4	\$0.03/GB	Instant	Processed results archive
S3 Glacier Flexible	\$3.60	\$0.03/GB	3–5 hours	Raw data archive
S3 Glacier Deep Archive	\$0.99	\$0.02/GB	12–48 hours	Long-term archival (FASTQ, old BAM)

Listing 23.11: Storage cost projection for a sequencing facility.

```

1  # Storage cost projection for 100 WGS samples per year
2  samples_per_year = 100
3  bam_size_tb = 0.07      # 70 GB per sample
4  cram_size_tb = 0.035    # ~50% of BAM
5  fastq_size_tb = 0.09    # 90 GB compressed FASTQ
6
7  # Storage tier costs (USD per TB per month)
8  tiers = {
9      "S3 Standard":      23.0,
10     "S3 IA":            12.5,
11     "Glacier Flexible":  3.6,
12     "Deep Archive":      0.99,
13 }
14
15 # Strategy: keep BAM in Standard for 3 months, then
16 # convert to CRAM in IA; archive FASTQ in Deep Archive
17 # (FASTQ already in SRA, so local copy is backup)
18 annual_bam_active = samples_per_year * bam_size_tb * 23.0 * 3
19 annual_cram_ia = samples_per_year * cram_size_tb * 12.5 * 9
20 annual_fastq_archive = samples_per_year * fastq_size_tb * 0.99 * 12
21
22 total = annual_bam_active + annual_cram_ia + annual_fastq_archive
23 print(f"Annual storage cost: ${total:,.0f}")
24 print(f"  Active BAM (3 months):    ${annual_bam_active:,.0f}")
25 print(f"  CRAM in IA (9 months):    ${annual_cram_ia:,.0f}")
26 print(f"  FASTQ Deep Archive (12m):  ${annual_fastq_archive:,.0f}")

```

When to Store Raw vs Processed Data

- **Always archive raw FASTQ in SRA/ENA** (free, permanent, public or embargoed).
- **Store CRAM instead of BAM** for long-term retention—convert with `samtools view -C -T ref.fa -o out.cram in.bam`.
- **Delete intermediate files** (unsorted BAMs, temporary indices) after pipeline completion.
- **Retain final results permanently** (VCF, count matrices, peak files, h5ad)—these are small and expensive to regenerate.
- **Consider regenerability:** If a file can be regenerated from archived inputs in under 24 hours of compute, it may not be worth storing long-term.

23.9 Community Standards and Consortia

Several international consortia have established quality standards and best practices that serve as benchmarks for the field:

ENCODE Consortium

ENCODE has established detailed quality standards for ChIP-seq, ATAC-seq, RNA-seq, and other assays. Their standards specify minimum replicates, sequencing depth, quality metrics (e.g., IDR for ChIP-seq, TSS enrichment for ATAC-seq), and data processing pipelines. The ENCODE quality metrics are widely adopted by the community as benchmarks for new experiments.

IHEC (International Human Epigenome Consortium)

Coordinates the generation of reference epigenome maps across multiple countries. IHEC standards specify data quality requirements for histone ChIP-seq, DNA methylation, and chromatin accessibility assays.

Human Cell Atlas (HCA)

Establishing standards for single-cell data: recommended protocols, metadata standards, cell type annotation guidelines, and data formats.

GA4GH (Global Alliance for Genomics and Health)

Develops standards and frameworks for responsible genomic data sharing. Key outputs include:

- **htsget:** Protocol for streaming BAM/CRAM/VCF data from remote servers.
- **Beacon:** Protocol for querying the existence of specific variants across federated databases without transferring raw data.
- **DRS (Data Repository Service):** Standardized API for accessing data objects across repositories.
- **WES (Workflow Execution Service):** Standardized API for running workflows on remote compute platforms.
- **Phenopackets:** Standardized format for clinical phenotype data linked to genomic findings.

23.10 The Data Lifecycle

Figure 23.1 illustrates the complete lifecycle of NGS data, from experimental planning through long-term archival and reuse.

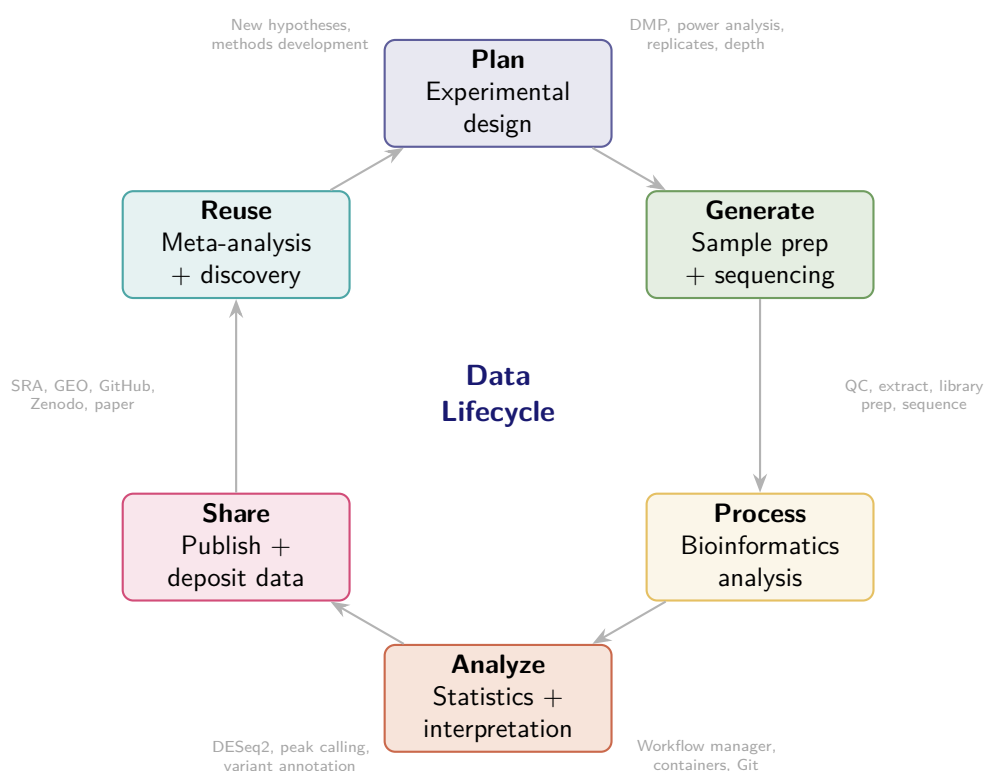


Figure 23.1: The data lifecycle for NGS projects. Data flows from planning through generation, processing, analysis, sharing, and reuse. The cycle is iterative: reuse of public data generates new hypotheses that drive new experiments. Each phase has associated standards, tools, and best practices described throughout this book.

23.11 Practical Data Submission Checklist

Pre-Publication Data Submission Checklist

Complete these steps before submitting your manuscript:

1. Raw data to SRA/ENA:

- Register a BioProject with a descriptive title and abstract.
- Create BioSamples with complete metadata (organism, tissue, genotype, condition).
- Upload FASTQ files and link to BioSamples.
- Set a release date (can be set to future date for embargo until publication).

2. Processed data to GEO:

- Prepare processed files (count matrices, peak BED files, BigWig tracks).
- Create a GEO submission with sample-level metadata.
- Link to SRA BioProject for raw data.

3. Code to GitHub + Zenodo:

- Push all analysis code (Snakemake/Nextflow workflows, scripts) to GitHub.
- Include a README with instructions for reproducing the analysis.
- Create a Zenodo release to get a DOI for permanent archival.
- Include Conda environment files or Dockerfiles.

4. In the manuscript:

- Report all accession numbers (BioProject, GEO, SRA).
- Report all software versions.
- Report all key parameters.
- Provide the GitHub/Zenodo DOI for analysis code.

23.12 Summary

Responsible data management, sharing, and reproducibility practices are not optional extras—they are integral to high-quality genomics research:

- **Public repositories:** SRA/ENA for raw data, GEO/ArrayExpress for processed data. Data submitted to any INSDC member is automatically shared globally.
- **Consortium portals:** ENCODE, GTEx, TCGA, HCA, and CellxGene provide curated, uniformly processed datasets that serve as community resources.
- **Reference genomes:** Use consistent genome builds and annotations from GENCODE, Ensembl, or RefSeq. Document exact versions.
- **Metadata standards:** MINSEQE defines minimum metadata requirements. Use controlled vocabularies (EFO, CL, OBI) rather than free text.
- **FAIR principles:** Make data Findable (persistent identifiers), Accessible (standard protocols), Interoperable (standard formats), and Reusable (rich metadata and clear licenses).
- **Reproducibility:** Pin software and reference versions, use containers and workflow managers, version-control code with Git, archive on Zenodo for DOIs, and combine code with narrative using literate programming.

- **Data management plans:** Required by most funding agencies. Plan for data generation, storage, sharing, and long-term preservation from the start of the project.
- **Data security:** Human genomic data requires careful attention to consent, de-identification, and controlled access. Follow HIPAA, GDPR, and institutional IRB requirements.
- **Community standards:** ENCODE, IHEC, HCA, and GA4GH establish quality benchmarks and interoperability standards that benefit the entire field.

By following these standards and practices, you ensure that your sequencing data contributes not only to your immediate research goals but to the broader scientific community for years to come.

Part VIII

Democratisation of Sequencing

Chapter 24

Accessible Sequencing: Democratising Genomics

Key Concept

Sequencing technology is no longer the exclusive province of well-funded core facilities and genome centres. The dramatic reduction in instrument costs, the emergence of portable nanopore sequencers, and the growth of community laboratories have opened genomics to individual researchers, educators, citizen scientists, and field biologists working in remote locations. This chapter maps the landscape of accessible sequencing—from budget planning and platform selection to field deployment, educational applications, and the legal and ethical frameworks that govern this expanding frontier. The goal is to provide a realistic, practical guide for anyone who wants to generate sequence data outside a traditional institutional setting.

24.1 The Landscape of Accessible Sequencing

The cost of sequencing a human genome has fallen from approximately \$2.7 billion (the Human Genome Project, completed 2003) to under \$200 for consumables on the latest Illumina platforms. Yet the cost of *entry*—the capital expenditure required to begin generating sequence data independently—remained prohibitively high until the mid-2010s. The introduction of Oxford Nanopore’s MinION in 2014 fundamentally changed this equation, creating a category of sequencing devices that cost less than a high-end laptop.

Today, a researcher, educator, or citizen scientist can enter the sequencing space at several cost tiers:

- **Under \$1,000:** A MinION starter pack provides the sequencing device itself (the MinION Mk1B) plus initial flow cells and reagents. This is sufficient for bacterial genomes, amplicon sequencing, and educational demonstrations.
- **\$1,000–\$5,000:** Adding a Flongle adapter, additional flow cells, a basic PCR thermocycler, and a used laptop with a GPU enables a modest but capable sequencing programme.
- **\$5,000–\$10,000:** The Illumina iSeq 100 (list price ~\$20,000 new, often available refurbished for \$8,000–\$12,000) or the ONT PromethION P2 Solo (~\$9,800) provides substantially higher throughput while remaining within reach of individual laboratories.
- **\$10,000–\$50,000:** Shared-access models with benchtop instruments such as the Illumina MiSeq or NextSeq 550 become feasible for departments, teaching institutions, and

community laboratories.

Table 24.1 compares the key accessible sequencing platforms available as of 2025.

Table 24.1: Comparison of accessible sequencing platforms. Costs are approximate and represent list prices for the instrument plus a single run of consumables; actual costs vary by region and negotiation.

Platform	Vendor	Output	Read Length	Accuracy	Cost ^a	Best For
MinION Mk1B	ONT	50 Gb	>100 kb	Q20+ (sup)	\$1,000	Portability, long reads, rapid prototyping
Flongle	ONT	2.8 Gb	>100 kb	Q20+ (sup)	\$140/run	Low-cost amplicons, targeted panels, teaching
P2 Solo	ONT	290 Gb	>100 kb	Q20+ (sup)	\$9,800	Human WGS, large genomes, production
iSeq 100	Illumina	1.2 Gb	2 × 150 bp	>Q30	\$20,000	Small panels, 16S, low-throughput short reads

^a MinION cost reflects the starter pack; Flongle cost is per-flow-cell consumable; P2 Solo and iSeq 100 costs are instrument list prices.

Refurbished and Decommissioned Instruments

Illumina MiSeq and Ion Torrent PGM instruments are increasingly available on the secondary market as institutions upgrade. A refurbished MiSeq can often be obtained for \$15,000–\$25,000—a fraction of its original list price. Ensure that service contracts or at minimum a warranty period are included, and verify that the instrument firmware is compatible with current reagent kits before purchasing.

The choice between platforms involves trade-offs that depend on the application. Short-read platforms (iSeq 100, MiSeq) offer high per-base accuracy and compatibility with the vast existing ecosystem of Illumina-based bioinformatics tools (see Chapter 3). Long-read platforms (ONT MinION, Flongle, P2 Solo) offer portability, real-time data streaming, the ability to detect base modifications natively, and ultra-long reads that simplify de novo assembly (see Chapter 5).

24.2 Oxford Nanopore for the Individual Researcher

Oxford Nanopore Technologies has done more than any other company to lower the barrier to entry for independent sequencing. The MinION—a USB-powered sequencing device roughly the size of a stapler—was the first sequencing instrument that a single researcher could purchase, unbox, and begin using in an afternoon.

24.2.1 MinION and Mk1C

The **MinION Mk1B** is the flagship accessible device. It contains no internal compute; sequencing data is streamed via USB 3.0 to a host computer where basecalling is performed.

The **MinION Mk1C** integrates an NVIDIA Jetson Xavier GPU module with a touchscreen, making it a self-contained sequencing-and-basecalling device—particularly useful for field deployment where carrying a separate laptop is impractical.

Key specifications of the MinION platform:

- **Flow cell:** R10.4.1 (current generation), containing $\sim 2,048$ individually addressable nanopores arranged in 512 channels.
- **Output:** Up to 50 Gb per flow cell under ideal conditions; typical yields range from 10–30 Gb depending on library quality and pore occupancy.
- **Read length:** Limited primarily by input DNA fragment length, not the device. Reads exceeding 4 Mb have been demonstrated.
- **Run time:** Up to 72 hours, but most runs yield the majority of data within 24–48 hours.
- **Accuracy:** With R10.4.1 chemistry and the “super-accurate” (sup) basecalling model, simplex read accuracy exceeds Q20 ($>99\%$ per-base). Duplex reads (both strands of the same molecule) achieve Q30+ ($>99.9\%$).
- **Power draw:** Approximately 2–3 W for the MinION device itself; the host computer or Mk1C draws an additional 10–30 W.

24.2.2 Flongle: The Disposable Flow Cell

The **Flongle** is a single-use flow cell adapter for the MinION that contains 126 channels (compared to 512 on a standard MinION flow cell). It produces up to 2.8 Gb of data—sufficient for many targeted applications—at a consumable cost of approximately \$140 per flow cell (including the adapter, which is reusable).

The Flongle is ideally suited for:

- Bacterial genome assembly (a 5 Mb genome at $30\times$ coverage requires only 150 Mb of data).
- Amplicon sequencing (16S, ITS, [PCR](#) products for genotyping).
- [QC](#) runs to verify library quality before committing a full MinION flow cell.
- Teaching and training exercises.
- Rapid pathogen identification in point-of-care settings.

The economics of the Flongle are compelling: at \$140 per flow cell, the per-sample cost for a bacterial genome (including library preparation) is approximately \$200–\$250. This is competitive with outsourcing to a core facility, with the added benefit of turnaround times measured in hours rather than weeks.

24.2.3 PromethION P2 Solo

For researchers who need higher throughput but cannot justify a full PromethION 48 ($>\$200,000$), the **PromethION P2 Solo** offers two PromethION flow cells (each with $\sim 2,675$ channels) in a benchtop format priced at approximately \$9,800. Each P2 Solo flow cell can yield up to 145 Gb, making $30\times$ human [WGS](#) feasible from a single flow cell.

The P2 Solo requires a dedicated compute resource for basecalling, ideally a workstation equipped with a high-end NVIDIA GPU (see [Table 24.2](#)).

24.2.4 Computing Requirements for Nanopore Basecalling

Basecalling—the process of translating raw electrical signal into nucleotide sequences—is the most computationally intensive step in the ONT workflow. The basecalling model used dramatically affects both accuracy and compute time. Table 24.2 provides benchmarks for basecalling a typical MinION run (~20 Gb) on different hardware configurations.

Table 24.2: Approximate basecalling times for 20 Gb of MinION R10.4.1 data using Dorado on different hardware configurations. Times are wall-clock hours for a complete run.

Hardware	Specification	Fast	HAC	SUP
CPU only	16-core Xeon / Ryzen 9	8 h	48 h	>200 h
Entry GPU	NVIDIA RTX 3060 (12 GB)	0.5 h	2.5 h	12 h
Mid-range GPU	NVIDIA RTX 4080 (16 GB)	0.3 h	1.2 h	5 h
High-end GPU	NVIDIA A100 (80 GB)	0.2 h	0.6 h	2 h
Mk1C (internal)	Jetson Xavier NX	4 h	18 h	N/A ^b
Apple Silicon	M2 Pro (16 GB unified)	1.5 h	6 h	28 h

^b The Mk1C hardware is insufficient for SUP basecalling in a practical time frame; use an external GPU workstation for SUP accuracy.

Basecalling Model Selection

The choice of basecalling model involves a direct trade-off between accuracy and compute cost. The “fast” model (approximately Q8–Q10) is suitable for real-time species identification during a run but insufficient for variant calling. The “high accuracy” (HAC, Q15–Q18) model is appropriate for most applications. The “super-accurate” (SUP, Q20+) model is recommended for variant calling, methylation detection, and any application where per-base accuracy is critical—but requires a capable GPU. Running SUP basecalling on a CPU is not practical for production use.

Listing 24.1: Basecalling with Dorado on a GPU workstation.

```

1 # Download the appropriate model
2 dorado download --model dna_r10.4.1_e8.2_400bps_sup@v5.0.0
3
4 # Basecall a directory of POD5 files using the SUP model
5 dorado basecaller \
6   dna_r10.4.1_e8.2_400bps_sup@v5.0.0 \
7   /data/pod5/ \
8   --device cuda:0 \
9   --emit-moves \
10  > basecalled.bam
11
12 # Convert to FASTQ if needed
13 samtools fastq basecalled.bam > basecalled.fastq
14
15 # For methylation-aware basecalling (5mC and 5hmC)
16 dorado basecaller \
17   dna_r10.4.1_e8.2_400bps_sup@v5.0.0 \
18   /data/pod5/ \
19   --device cuda:0 \

```



```

20 --modified-bases 5mCG_5hmCG \
21 > basecalled_mods.bam

```

24.3 Budget Planning and Cost Modelling

One of the most common mistakes made by researchers entering the accessible sequencing space is underestimating total project costs. The sequencing device itself is often the smallest line item; reagents, library preparation kits, sample preparation consumables, compute resources, and personnel time dominate the budget. This section provides itemised cost models for five common project types.

24.3.1 Itemised Cost Models

Table 24.3 provides realistic, itemised cost estimates for five representative projects using ONT MinION or Flongle as the primary sequencing platform. All costs are approximate and based on 2025 list prices.

Table 24.3: Itemised cost models for five representative projects. Costs assume a MinION Mk1B already in hand (starter pack, \$1,000, not included in per-project costs). Library preparation costs include consumables only.

Cost Item	Bacterial Genome	16S Amplicon	Human WGS	eDNA Biodiv.	Food Auth.
DNA extraction kit	\$50	\$120	\$30	\$200	\$80
Library prep kit	\$120	\$60	\$180	\$90	\$90
Flow cell (type)	\$140 ^c	\$140 ^c	\$900 ^d	\$140 ^c	\$140 ^c
Barcoding expansion	—	\$45	—	\$45	\$45
PCR primers / reagents	—	\$30	—	\$30	\$30
Compute (cloud GPU)	\$5	\$2	\$40	\$5	\$3
Consumables (tips, tubes)	\$15	\$25	\$20	\$30	\$20
Total per project	\$330	\$422	\$1,170	\$540	\$408
Samples per run	1	24	1	12	12
Cost per sample	\$330	\$18	\$1,170	\$45	\$34

^c Flongle flow cell. ^d Standard MinION R10.4.1 flow cell.

Hidden Costs to Budget For

Beyond the obvious consumables, accessible sequencing projects should budget for:

- **Sample collection and transport:** Preservation reagents, cold chain, shipping.
- **Data storage:** A single MinION run generates 50–200 GB of raw POD5 files. Cloud storage costs accumulate.
- **Compute:** GPU basecalling, alignment, assembly. Cloud GPU instances (e.g., AWS g5.xlarge) cost \$1–\$3/hour.
- **Failed runs:** Budget for a 10–20% failure rate due to clogged pores, degraded reagents, or library preparation issues.
- **Personnel time:** Even “automated” protocols require skilled hands. Factor in training time.

24.3.2 Reagent Shelf Life

Reagent degradation is a frequently overlooked cost driver, particularly for low-throughput users who may not consume reagents before expiration. Key shelf-life considerations include:

- **ONT flow cells:** Stored at 2–8°C, flow cells have a shelf life of approximately 8–12 weeks from manufacture. Using flow cells near their expiration date results in reduced pore counts and lower yields. Order flow cells as close to the planned experiment date as possible.
- **Library preparation kits:** Most ONT library prep kits (Ligation Sequencing Kit, Rapid Barcoding Kit) have a shelf life of 3–6 months at –20°C. Kits contain enzymes that degrade with repeated freeze–thaw cycles; aliquot on first use.
- **DNA extraction reagents:** Column-based kits (Qiagen, Zymo) are stable for 12–18 months at room temperature. Enzymatic lysis reagents may have shorter shelf lives.
- **PCR reagents:** Polymerases and master mixes are typically stable for 6–12 months at –20°C. Primers dissolved in TE buffer are stable for years at –20°C.

Group Purchasing for Low-Throughput Users

If you sequence infrequently (fewer than one run per month), consider coordinating with nearby laboratories or community lab members to place joint orders, share flow cells, and run samples together. A Flongle flow cell with native barcoding can accommodate up to 24 samples—pooling samples from multiple users minimises per-sample cost and reduces reagent waste.

24.3.3 Shared Equipment Models and Cost-per-Sample Analysis

For institutions considering a shared sequencing resource, the cost per sample can be modelled as:

$$C_{\text{sample}} = \frac{C_{\text{instrument}}}{N_{\text{lifetime}} \times S_{\text{run}}} + \frac{C_{\text{consumable}}}{S_{\text{run}}} + C_{\text{extraction}} + C_{\text{overhead}} \quad (24.1)$$

where $C_{\text{instrument}}$ is the capital cost of the instrument, N_{lifetime} is the total number of runs over the instrument’s lifetime, S_{run} is the number of samples per run, $C_{\text{consumable}}$ is the per-run consumable cost (flow cell + library prep), $C_{\text{extraction}}$ is the per-sample DNA extraction cost, and C_{overhead} captures personnel time, compute, storage, and facility costs.

For example, consider a community laboratory that purchases a MinION starter pack (\$1,000) and plans to perform 100 Flongle runs over two years, with 12 barcoded samples per run:

$$\begin{aligned} C_{\text{sample}} &= \frac{\$1,000}{100 \times 12} + \frac{\$140 + \$90}{12} + \$10 + \$5 \\ &= \$0.83 + \$19.17 + \$10 + \$5 \\ &= \$35.00 \text{ per sample} \end{aligned} \quad (24.2)$$

This is competitive with commercial sequencing services, with the added advantages of rapid turnaround (same-day results), complete control over the protocol, and the educational value of performing sequencing in-house.

24.4 Community Labs and Maker Spaces

Community biology laboratories—also known as biohacker spaces or DIY biology labs—have emerged as important venues for accessible sequencing. These spaces provide shared equipment, training, and a collaborative community that lowers the barrier to entry for individuals who lack institutional affiliation.

24.4.1 Prominent Community Laboratories

Several community labs have pioneered accessible sequencing programmes:

Genspace (New York City, USA)

Founded in 2010, Genspace is one of the world's first community biology laboratories. It offers courses in DNA extraction, **PCR**, gel electrophoresis, and nanopore sequencing. Genspace operates as a non-profit with membership fees (\$100/month) and course-based revenue. The lab maintains MinION devices available for member use and has contributed to citizen science projects including environmental DNA monitoring of the Gowanus Canal.

BioCurious (Santa Clara, California, USA)

A community laboratory in Silicon Valley that has hosted sequencing workshops and synthetic biology projects since 2011. BioCurious provides a BSL-1 workspace, shared equipment, and mentorship from experienced molecular biologists.

La Paillasse (Paris, France)

One of Europe's leading community biology spaces, La Paillasse has been involved in open-source biology projects since 2014. It has hosted nanopore sequencing workshops and contributed to food safety and environmental monitoring projects.

BioFoundry (Sydney, Australia)

Australia's first community laboratory, BioFoundry offers nanopore sequencing training and has supported eDNA biodiversity monitoring projects in urban waterways.

24.4.2 Setting Up a Sequencing Corner

A functional sequencing workspace can be established within a community laboratory, teaching institution, or even a well-organised home laboratory, provided that appropriate biosafety measures are in place. The following requirements apply to **BSL-1** (Biosafety Level 1) work only—that is, work with organisms and samples that pose no known risk to healthy adults.

Minimum Requirements for a BSL-1 Sequencing Workspace

1. **Work surface:** A cleanable, non-porous bench surface (stainless steel, epoxy resin, or chemical-resistant laminate). Minimum 1.5 m × 0.6 m of clear bench space.
2. **Decontamination:** 10% bleach solution or 70% ethanol for surface decontamination. An autoclave or pressure cooker for waste decontamination.
3. **Personal protective equipment (PPE):** Disposable nitrile gloves, laboratory coat, and safety glasses.
4. **Waste disposal:** Biohazard bags for contaminated consumables; sharps container if using scalpels or needles.
5. **Hand washing:** A sink with soap and water within the workspace.
6. **No food or drink:** The workspace must be separated from food preparation and consumption areas.
7. **Ventilation:** Adequate general ventilation. A biosafety cabinet is *not* required for BSL-1 work but is recommended for handling samples that may contain allergens or irritants.
8. **Documentation:** A laboratory notebook (physical or electronic) for recording all procedures, reagent lot numbers, and results.
9. **Training:** All users should receive basic laboratory safety training covering chemical hazards, biological waste handling, and emergency procedures.

24.4.3 Minimum Equipment for a Sequencing Laboratory

Table 24.4 lists the minimum equipment required for a functional nanopore sequencing laboratory, along with approximate costs and potential budget alternatives.

3D-Printed and Open-Source Lab Equipment

The open-source hardware movement has produced functional designs for many common laboratory instruments. Magnetic racks, tube holders, gel electrophoresis chambers, and even centrifuges can be 3D-printed at a fraction of commercial cost. Resources such as the Open Bioeconomy Lab, NIH 3D Print Exchange, and Thingiverse host verified designs. The OpenPCR project provides plans for a thermocycler that can be assembled for under \$500.

24.5 Field Sequencing

One of the most transformative aspects of portable sequencing is the ability to generate sequence data in the field—at the point of sample collection, in remote locations without reliable electricity or internet, and in outbreak settings where rapid pathogen identification saves lives. The MinION's small size, low power consumption, and tolerance of temperature variation make it uniquely suited to field deployment.

24.5.1 Power Requirements

The MinION Mk1B draws approximately 2–3 W via USB, while the host laptop typically requires 30–65 W depending on whether GPU basecalling is performed. Total system power

Table 24.4: Minimum equipment for a basic nanopore sequencing laboratory. “Budget alternative” indicates lower-cost substitutes suitable for educational or low-throughput settings.

Equipment	Cost	Essential?	Budget Alternative
MinION Mk1B	\$1,000	Yes	—
Laptop (16 GB RAM, GPU)	\$1,200	Yes	Used/refurbished; cloud basecalling
Micropipettes (P2, P20, P200, P1000)	\$1,500	Yes	Used pipettes (\$200–\$500)
Minicentrifuge	\$200	Yes	—
Vortex mixer	\$150	Yes	Manual flicking (not recommended)
Heat block or thermocycler	\$300	Yes	Sous vide cooker for lysis steps
Magnetic rack (for SPRI beads)	\$50	Yes	3D-printed rack with neodymium magnets
Qubit fluorometer	\$3,000	Recommended	NanoDrop (\$1,500) or gel estimation
Gel electrophoresis system	\$500	Recommended	Pre-cast gel kits (\$100)
Mini PCR thermocycler	\$500	For amplicons	miniPCR (\$500, educational pricing)
−20°C freezer	\$500	Yes	Shared freezer; dorm-style mini-freezer
Total (essential only)	\$4,900		
Total (with recommended)	\$9,100		

for a MinION + laptop configuration is therefore 35–70 W.

- **USB power banks:** A 100 Wh USB-C PD power bank can sustain a MinION + efficient laptop for approximately 2–3 hours of sequencing. Multiple power banks can be carried for extended field work.
- **Solar panels:** A 100 W portable solar panel with a LiFePO₄ battery station (e.g., 500 Wh capacity) can sustain continuous sequencing during daylight hours and provide 7–10 hours of overnight operation.
- **Vehicle power:** A car cigarette lighter or 12 V outlet with an inverter provides reliable power for roadside or vehicle-based sequencing.
- **Generator:** For multi-day field campaigns, a small (<1 kW) inverter generator provides indefinite runtime.

24.5.2 Temperature Constraints

ONT flow cells are sensitive to temperature. The recommended operating range is 20–35°C; the optimal range is 22–30°C. Outside this range:

- **Below 15°C:** Enzyme activity decreases, translocation speed slows, and basecalling accuracy degrades. In cold environments, insulate the MinION in a small cooler with a chemical hand warmer.
- **Above 35°C:** Pore lifetime decreases and DNA may denature. In hot environments, shade the device and use a small USB fan for convective cooling. Avoid direct sunlight.

24.5.3 Data Transfer in the Field

A single MinION run generates 50–200 GB of raw signal data (POD5 format). Transferring this volume of data over satellite or cellular networks is often impractical. Strategies include:

- **On-device basecalling:** Perform basecalling locally (fast or HAC model) to reduce data to 1–5 GB of FASTQ/BAM, which is transferable over a cellular connection.
- **Real-time selective sequencing:** Use ReadFish or ONT's adaptive sampling to reject off-target reads, reducing data volume and focusing on sequences of interest.
- **Portable storage:** Carry ruggedised external SSDs (2–4 TB) and transfer data physically when returning from the field.
- **Satellite internet:** Starlink terminals (~100 Mbps) can enable data transfer in remote locations with clear sky access.

24.5.4 Case Studies in Field Sequencing

Real-Time Genomic Surveillance of Ebola (2015)

The seminal demonstration of field sequencing was performed by Quick et al. (2016) during the West African Ebola outbreak. Using MinION devices deployed to Guinea, the team sequenced 142 Ebola virus genomes within 48 hours of sample collection. Real-time phylogenetic analysis revealed transmission chains, identified cross-border transmission events, and demonstrated that the outbreak was sustained by human-to-human transmission rather than repeated zoonotic introductions. The entire sequencing laboratory fit in a single suitcase, operated on a generator, and demonstrated that genomic surveillance could be performed in resource-limited settings with minimal infrastructure.

eDNA Biodiversity Monitoring

Environmental DNA (eDNA) is DNA shed by organisms into their environment—water, soil, air—and can be collected non-invasively to survey biodiversity. Portable sequencing brings the analysis to the field site, enabling same-day species detection. Notable deployments include:

- Detection of aquatic species in Amazonian rivers using water samples filtered and sequenced on-site with MinION.
- Monitoring of invasive species in the Great Lakes using portable eDNA sequencing at boat launches.
- Marine biodiversity surveys at remote coral reef sites, where MinION sequencing of the mitochondrial *COI* barcode region identified fish species present in water samples within hours of collection.

These applications are discussed further in the context of metagenomic methods in Chapter 9.

24.5.5 Field Sequencing Setup

Figure 24.1 illustrates a minimal field sequencing setup. The key principle is to minimise the number of components while maintaining a complete workflow from sample to sequence.

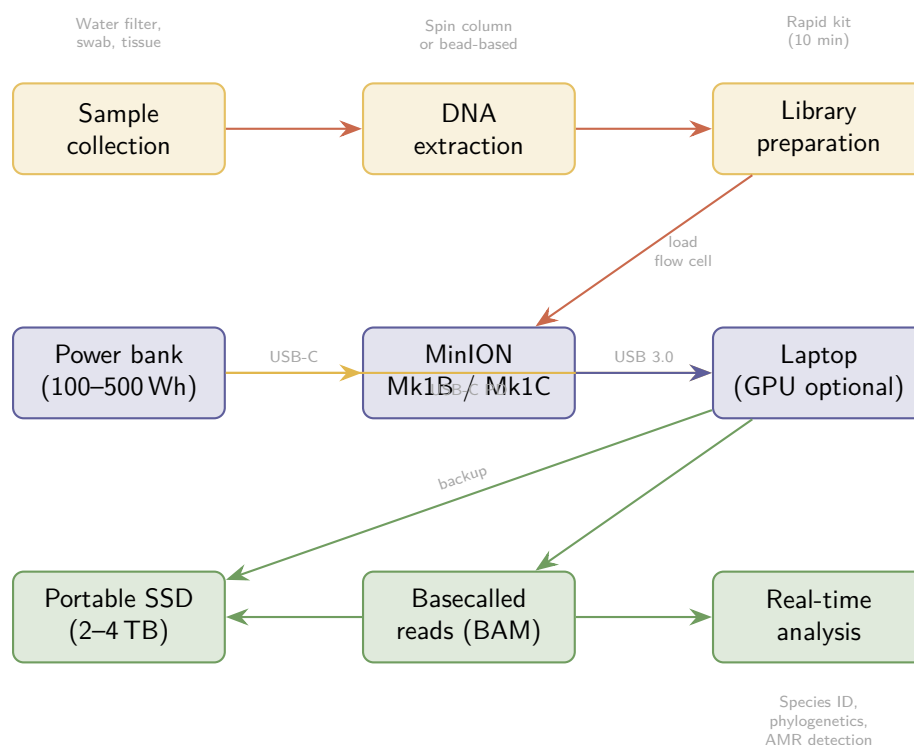


Figure 24.1: Schematic of a minimal field sequencing setup showing the three phases of the workflow: sample preparation (top, gold), sequencing hardware (middle, blue), and data handling (bottom, green). Arrows indicate sample flow (red), power delivery (gold), and data flow (blue/green).

Field Sequencing Packing Checklist

The following items constitute a complete field sequencing kit that fits in a single carry-on-sized case:

1. MinION Mk1B (or Mk1C) with USB cable
2. 2–4 Flongle or MinION flow cells (in cold pack pouch, 2–8°C)
3. Rapid Barcoding Kit (SQK-RBK114.24)
4. Spin-column DNA extraction kit (e.g., Qiagen DNeasy or Zymo Quick-DNA)
5. Micropipettes (P20, P200) with filter tips
6. Magnetic rack and AMPure XP beads (or Zymo Select-a-Size)
7. Minicentrifuge (palm-sized, battery-operated)
8. Laptop with MinKNOW and Dorado pre-installed
9. 100 Wh USB-C PD power bank (for MinION) + laptop charger
10. Portable SSD for data backup
11. Nitrile gloves, 70% isopropanol wipes, waste bag
12. Laboratory notebook and permanent marker

Total weight: approximately 5–7 kg including the laptop.

24.6 Educational and Outreach Applications

Sequencing technology is an extraordinarily powerful tool for science education. The ability to extract DNA from an organism, generate a sequence, and identify it computationally provides a visceral, end-to-end experience of molecular biology that textbooks alone cannot replicate. The accessibility of nanopore sequencing has made it feasible to bring real sequencing into undergraduate laboratories, high school classrooms, and public outreach events.

24.6.1 Designing a Teaching Laboratory Sequencing Module

A successful teaching laboratory sequencing module must be:

- **Robust:** Protocols must work reliably in the hands of novices. Use the simplest possible library preparation kit (e.g., [ONT Rapid Barcoding Kit](#), which requires 10 minutes of hands-on time and no [PCR](#)).
- **Fast:** Students must see results within a single laboratory session (3–4 hours). Nanopore sequencing produces data in real time, allowing species identification within 30–60 minutes of loading.
- **Affordable:** Per-student costs must be manageable within institutional teaching budgets.
- **Safe:** All work must be BSL-1 compatible. Use non-pathogenic organisms or environmental samples that pose no known health risk.
- **Pedagogically meaningful:** The experiment should connect to core concepts in molecular biology, genetics, and bioinformatics as covered in the foundational chapters of this book (see [Chapter 3](#)).

24.6.2 Suggested Curriculum Outline

The following five-session curriculum is designed for an undergraduate molecular biology or genomics course. Each session is 3–4 hours.

Session 1: DNA Extraction and Quantification

Students extract DNA from an assigned sample (environmental water, soil, cheek swab, food product, or bacterial culture). They quantify DNA using a Qubit fluorometer and assess quality by gel electrophoresis or NanoDrop A260/A280 ratio. *Learning objectives:* Cell lysis mechanisms, DNA purification chemistry, quality metrics.

Session 2: Library Preparation and Sequencing

Students prepare a nanopore library using the Rapid Barcoding Kit. Each student receives a unique barcode. Barcoded libraries are pooled and loaded onto a single Flongle or MinION flow cell. Sequencing begins during the session and runs overnight. *Learning objectives:* Library preparation principles (see [Chapter 6](#)), adapter ligation, multiplexing by barcoding.

Session 3: Basecalling and Quality Control

Students retrieve basecalled FASTQ data and perform [QC](#) using [NanoPlot](#) and [FastQC](#). They examine read length distributions, quality scores, and per-barcode yields. *Learning objectives:* Phred quality scores, read length distributions, demultiplexing.

Session 4: Bioinformatics Analysis

Depending on the sample type: (a) for amplicon data, students run [BLAST](#) or [minimap2](#) against reference databases for species identification; (b) for bacterial genomes, students

perform de novo assembly with **FLYe** and assess assembly quality with **QUAST**. *Learning objectives:* Sequence alignment, de novo assembly, genome annotation.

Session 5: Interpretation and Presentation

Students interpret their results in biological context, write a short report, and present findings to the class. For eDNA projects, this may include comparing detected species to known local biodiversity. *Learning objectives:* Scientific communication, critical evaluation of sequence data.

24.6.3 Cost Per Student

Assuming a class of 24 students sharing a single Flongle flow cell with barcoding:

$$\begin{aligned}
 C_{\text{student}} &= \frac{C_{\text{flowcell}} + C_{\text{libprep}} + C_{\text{barcoding}}}{24} + C_{\text{extraction}} + C_{\text{consumables}} \\
 &= \frac{\$140 + \$60 + \$45}{24} + \$8 + \$5 \\
 &= \$10.21 + \$8 + \$5 \\
 &= \$23.21 \text{ per student}
 \end{aligned} \tag{24.3}$$

This is comparable to the cost of a conventional molecular biology teaching lab exercise involving **PCR** and gel electrophoresis, and far less expensive than outsourcing Sanger sequencing of individual student samples.

Oxford Nanopore Educational Discounts

ONT offers an educational programme with discounted starter packs and flow cells for registered academic institutions. Contact the **ONT** education team or check the Oxford Nanopore store for current academic pricing. Several grant programmes (e.g., the HHMI Inclusive Excellence programme in the US, the Wellcome Trust Public Engagement Grants in the UK) specifically fund sequencing equipment for teaching laboratories.

24.6.4 High School Protocols

Sequencing can be brought into high school settings with appropriate simplification:

- **Pre-extracted DNA:** Provide students with pre-extracted, quantified DNA to avoid the time and consumable cost of extraction in a high school setting.
- **Teacher-performed library preparation:** The teacher or a trained assistant prepares the library. Students participate in loading the flow cell and monitoring the sequencing run.
- **Simplified bioinformatics:** Use **EPI2ME** (Oxford Nanopore’s cloud-based analysis platform) or pre-built **Galaxy** workflows rather than command-line tools.
- **Focus on species identification:** The “What’s in my sample?” question (species identification from environmental DNA, food products, or bacterial cultures) is inherently engaging and requires minimal bioinformatics infrastructure.
- **Partner with a local university:** University researchers can provide equipment, expertise, and mentorship. Many universities have outreach programmes that actively seek partnerships with local schools.

24.7 What Can You Realistically Do with \$1,000?

The MinION starter pack costs approximately \$1,000 and includes the device, two Flongle flow cells, one MinION flow cell, and a wash kit. With this initial investment, what sequencing projects are realistically achievable? [Table 24.5](#) provides a feasibility matrix.

Table 24.5: Feasibility matrix for sequencing projects within a \$1,000 total budget (including the MinION starter pack). Coverage is estimated for the included flow cells. “Feasible” indicates that useful results can be obtained; “marginal” indicates that results are possible but may be incomplete.

Project	Genome Size	Data Needed	Achievable?	Notes
Bacterial genome (single)	5 Mb	150 Mb (30×)	Feasible	Flongle; excellent results
16S amplicon (24 samples)	N/A	50 Mb	Feasible	Flongle + barcoding
Fungal genome	30–40 Mb	1.2 Gb (30×)	Feasible	Flongle; sufficient yield
Plant/animal genome	200 Mb–2 Gb	6–60 Gb (30×)	Marginal	MinION flow cell; low coverage for large genomes
Human WGS (30×)	3.1 Gb	93 Gb	Not feasible	Requires multiple flow cells (\$900+ each)
Viral genome	10–30 kb	<1 Mb	Feasible	Flongle; massive over-coverage
eDNA species detection	N/A	100 Mb	Feasible	Flongle; amplicon approach
Food authentication	N/A	100 Mb	Feasible	Flongle; barcode approach

Honest Expectations

The \$1,000 entry point is real but comes with important caveats:

- The starter pack does *not* include DNA extraction kits, library preparation reagents, pipettes, or any other laboratory equipment. Total out-of-pocket cost for a first project is realistically \$1,300–\$1,800.
- The included flow cells have limited shelf life. If you do not use them within 8–12 weeks, pore counts degrade significantly.
- Basecalling at SUP accuracy requires a computer with a capable GPU (\$800+ if purchased new) or access to cloud computing.
- Nanopore sequencing has a steeper learning curve than users accustomed to “send sample, receive data” core facility models. Expect your first 2–3 runs to be practice runs with suboptimal results.
- Assembly, annotation, and downstream analysis require bioinformatics skills. Free resources (e.g., [Galaxy](#), [ONT’s EPI2ME](#), this textbook) can help, but plan for a learning investment.

Maximising the Value of Your First Flow Cell

Your first MinION or Flongle run should be a low-stakes experiment designed to familiarise you with the workflow. Suggestions:

- Extract DNA from an easily lysed bacterium (e.g., *Escherichia coli* DH5 α) and prepare a rapid library. The genome is well-characterised, so you can assess your results against the known reference.
- Use the [ONT](#) Lambda phage control DNA included in many kits as a positive control alongside your experimental sample.
- Do not attempt barcoding on your first run—run a single unbarcoded library to simplify troubleshooting.

24.8 Citizen Science Sequencing Projects

The accessibility of nanopore sequencing has enabled a new category of citizen science: projects in which non-professional scientists generate and analyse sequence data to address questions of community relevance. Unlike traditional citizen science (which typically involves observation and data collection), sequencing-based citizen science produces molecular data that can contribute to professional research.

24.8.1 Environmental DNA Barcoding

Environmental DNA (eDNA) barcoding is perhaps the most natural fit for citizen science sequencing. Participants collect water, soil, or air samples from their local environment, extract DNA, amplify taxonomic barcode regions (mitochondrial *COI* for animals, *rbcL/matK* for plants, 16S for bacteria, ITS for fungi), and sequence the amplicons on a MinION or Flongle.

A well-designed eDNA citizen science project includes:

1. **Standardised sampling protocol:** Define sample volume, filtration method, preservation buffer, and collection metadata (GPS coordinates, date, water temperature, pH).
2. **Centralised library preparation:** Participants send extracted DNA (or even raw samples with preservation buffer) to a central laboratory for library preparation and

sequencing. This ensures consistent quality.

3. **Open data portal:** Results are uploaded to a public database where participants and the broader community can explore detected species, visualise biodiversity maps, and track changes over time.
4. **Quality assurance:** Include negative controls (blank extractions, no-template PCR controls) and positive controls (spike-in of known DNA) in every batch.

24.8.2 Food Fraud Detection

Food fraud—the deliberate mislabelling or adulteration of food products—is a global problem affecting seafood, meat, spices, olive oil, and honey. DNA barcoding can detect species substitution (e.g., tilapia sold as red snapper) with high sensitivity and specificity.

Citizen science food authentication projects have been conducted by community laboratories, journalism organisations, and consumer advocacy groups. A typical workflow involves:

- Purchasing food products from local markets and supermarkets.
- Extracting DNA using commercial kits optimised for processed food matrices.
- Amplifying species-specific markers (*COI*, *cytb*, 16S) with universal or taxon-specific primers.
- Sequencing amplicons on a Flongle and querying results against BOLD (Barcode of Life Data System) or NCBI BLAST.

DNA Barcoding for Species Identification

DNA barcoding uses a short, standardised gene region to identify the species of origin of a biological sample. The most commonly used barcode regions are:

COI Cytochrome c oxidase subunit I (~658 bp). The standard barcode for animals.

Deposited in the BOLD database (>14 million records).

rbcL RuBisCO large subunit. A standard plant barcode.

matK Maturase K. Used in combination with *rbcL* for plants.

ITS Internal transcribed spacer of ribosomal DNA. The standard fungal barcode.

16S rRNA Used for bacterial and archaeal identification (see Chapter 9).

24.8.3 Antimicrobial Resistance Monitoring in Waterways

Antimicrobial resistance (AMR) genes in environmental water are a growing public health concern. Rivers, lakes, and wastewater effluent harbour bacteria carrying resistance genes that can transfer to human pathogens. Citizen science monitoring of AMR in local waterways can:

- Establish baseline levels of resistance genes in local water bodies.
- Detect hotspots near hospitals, farms, and wastewater treatment plants.
- Track seasonal and temporal trends.
- Generate data that informs public health policy.

A nanopore-based AMR monitoring workflow involves collecting water samples, extracting metagenomic DNA, performing either shotgun sequencing or targeted amplification of known resistance genes, and querying results against the CARD (Comprehensive Antibiotic Resistance Database) or ResFinder databases.

Listing 24.2: AMR gene detection from environmental water using nanopore data.

```

1 # Basecall raw data
2 dorado basecaller \
3   dna_r10.4.1_e8.2_400bps_hac@v5.0.0 \
4   /data/pod5/ --device cuda:0 > reads.bam
5
6 # Convert to FASTQ
7 samtools fastq reads.bam > reads.fastq
8
9 # Map reads against CARD database
10 minimap2 -a card_database.fasta reads.fastq | \
11   samtools sort -o aligned_card.bam
12 samtools index aligned_card.bam
13
14 # Alternatively, use ABRicate for rapid AMR screening
15 abricate --db card reads.fasta > amr_results.tab
16 abricate --summary amr_results.tab > amr_summary.tab
17
18 # Visualise results
19 cat amr_summary.tab | column -t

```

24.8.4 Concrete Project Examples

Table 24.6 summarises several concrete citizen science sequencing projects that have been successfully conducted or are currently active.

Table 24.6: Examples of citizen science sequencing projects. Projects range from community laboratory initiatives to coordinated national programmes.

Project	Target	Platform	Description
Gowanus Canal eDNA	Aquatic biodiv.	MinION	Genspace members sampled water from a superfund site; identified fish, invertebrate, and microbial species.
Sushi DNA Test	Food fraud	Flongle	High school students tested sushi from NYC restaurants; found 30% mislabelling rate.
Thames AMR Watch	AMR monitoring	MinION	Community scientists monitored AMR genes in the River Thames at 12 sites over 12 months.
BioBlitz Sequencing	Terrestrial biodiv.	Flongle	Weekend bioblitz events where participants collected insects and sequenced <i>COI</i> barcodes on-site.
Honey Authentication	Food fraud	Flongle	Beekeeping clubs tested commercial honey for adulteration with corn syrup and species verification of pollen DNA.

24.9 Legal, Ethical, and Biosafety Considerations

The democratisation of sequencing brings with it significant legal, ethical, and biosafety responsibilities. The ability of any individual to generate genomic data—from environmental

samples, food products, microorganisms, and potentially human tissues—raises questions that the genomics community has traditionally addressed within institutional frameworks. As sequencing moves outside these frameworks, practitioners must be aware of and comply with applicable regulations.

24.9.1 The Nagoya Protocol and Access and Benefit Sharing

The **Nagoya Protocol** on Access to Genetic Resources and the Fair and Equitable Sharing of Benefits Arising from their Utilization is an international agreement (effective October 2014) supplementing the Convention on Biological Diversity. It establishes a legal framework for:

- **Prior informed consent (PIC):** Researchers must obtain consent from the country of origin before accessing genetic resources.
- **Mutually agreed terms (MAT):** Benefits arising from the use of genetic resources must be shared with the providing country under terms agreed upon before access.
- **Digital sequence information (DSI):** The status of DNA sequence data (as opposed to physical genetic material) under the Nagoya Protocol remains contested. A landmark decision at COP-15 (2022) established a multilateral mechanism for benefit sharing from DSI, with implementation details still being negotiated.

Nagoya Protocol Compliance

The Nagoya Protocol applies to anyone accessing genetic resources from another country, regardless of whether you are an academic researcher, a citizen scientist, or a hobbyist. Collecting soil, water, or biological samples abroad and sequencing them without the appropriate permits may constitute a violation of international law. Before collecting biological samples internationally, check:

1. Whether the country of collection has ratified the Nagoya Protocol.
2. What permits are required from the national competent authority.
3. Whether your institution has an Access and Benefit Sharing (ABS) policy.

Ignorance of the Nagoya Protocol does not constitute a legal defence.

24.9.2 Dual-Use Research Concerns

Dual-use research of concern (DURC) refers to research that, while intended for beneficial purposes, could be misused to threaten public health, agriculture, or national security. In the context of accessible sequencing, potential dual-use concerns include:

- **Pathogen characterisation:** Sequencing pathogenic organisms to identify virulence factors or antimicrobial resistance mechanisms. While this research is scientifically valuable, the resulting sequence data could theoretically inform the engineering of more dangerous pathogens.
- **Synthesis-enabling information:** Complete genome sequences of dangerous pathogens (e.g., *Variola major*, 1918 influenza virus) are publicly available. Sequencing technology *per se* does not create a dual-use risk, but combined with DNA synthesis capabilities, it could theoretically enable recreation of dangerous organisms.
- **Environmental surveillance:** Monitoring for the presence of regulated or dangerous organisms could reveal information about biosecurity vulnerabilities.

In practice, the dual-use risk of accessible sequencing is low relative to the benefits. Sequencing is fundamentally a *reading* technology, not a *writing* technology. The ability

to sequence a pathogen genome does not confer the ability to synthesise or engineer that pathogen. Nevertheless, community laboratories and citizen scientists should be aware of dual-use frameworks (e.g., the US Government Policy for Institutional DURC Oversight, the EU Code of Conduct for Responsible Nanosciences and Nanotechnologies Research) and exercise appropriate judgement.

24.9.3 Ethics Approval for Human Samples

Any project that involves sequencing human-derived biological material—including cheek swabs, saliva, blood, tissue biopsies, or cell lines—requires ethics approval from an Institutional Review Board (IRB) or Research Ethics Committee (REC). This requirement applies regardless of the institutional setting:

- **Academic institutions:** Submit a protocol to the institutional IRB/REC. Include details on informed consent, data storage, de-identification, and participant privacy protections.
- **Community laboratories:** If the community lab is not affiliated with an institution that has an IRB, ethical oversight must be arranged through a commercial IRB (e.g., WCG IRB, Advarra) or by partnering with a local university.
- **Self-experimentation:** Sequencing your own genome does not typically require IRB approval (you are both the researcher and the subject). However, you should carefully consider the implications of incidental findings (e.g., discovering a pathogenic variant in *BRCA1* or *APOE*) before proceeding.
- **Educational settings:** Students sequencing their own DNA (e.g., cheek swabs) requires careful ethical consideration. Many institutions prohibit this practice or require IRB approval, informed consent from students, and the option to use an alternative sample (e.g., bacterial culture) without academic penalty.

Informed Consent for Genomic Studies

Informed consent for genomic sequencing must address several unique considerations:

- **Scope of data:** Genomic data is inherently identifiable. Even “anonymised” genomic data can potentially be re-identified through linkage to public databases.
- **Incidental findings:** Whole-genome or whole-exome sequencing may reveal clinically significant variants unrelated to the study purpose. The consent process must specify whether and how incidental findings will be communicated.
- **Data sharing:** Participants must be informed about whether their data will be deposited in public repositories and the implications of this (see [Chapter 23](#)).
- **Future use:** Broad consent models allow data to be used for future, unspecified research. Narrow consent limits use to the specific study.
- **Withdrawal:** Participants must be able to withdraw consent, though complete data deletion from distributed databases may be technically challenging.

24.9.4 Data Sharing Obligations

Many funding agencies and journals mandate that sequence data be deposited in public repositories (e.g., NCBI SRA, ENA) as a condition of publication (see [Chapter 23](#)). Even in the absence of a publication mandate, open data sharing is a core principle of genomics:

- **Environmental and microbial data:** There are generally no restrictions on sharing environmental metagenomic data, though Nagoya Protocol obligations may apply.

- **Human data:** Controlled-access repositories (e.g., dbGaP, EGA) must be used for human genomic data to protect participant privacy. Never deposit identifiable human sequence data in open-access repositories.
- **Pathogen data:** Pathogen genome sequences from outbreak surveillance are typically expected to be shared rapidly via GISAID, GenBank, or the relevant public health authority.

24.9.5 Biosafety Levels and Regulated Organisms

Work in community laboratories and non-institutional settings is typically restricted to Biosafety Level 1 (BSL-1). This means working only with organisms that are “not known to consistently cause disease in healthy adult humans.” Key points:

BSL-1 organisms (permitted)

Non-pathogenic *Escherichia coli* strains (e.g., K-12, DH5 α), *Saccharomyces cerevisiae*, environmental samples from non-clinical sources, food products, plant material.

BSL-2 organisms (NOT permitted in community labs)

Organisms that can cause disease but for which treatments exist: *Staphylococcus aureus*, *Salmonella* spp., *Candida albicans*, human cell lines (due to potential blood-borne pathogen contamination), clinical specimens.

BSL-3 and BSL-4 organisms (specialised facilities only)

Mycobacterium tuberculosis, SARS-CoV-2, Ebola virus, *Francisella tularensis*. These require specialised containment facilities and are far outside the scope of accessible sequencing.

Environmental Samples May Contain Pathogens

Environmental samples (soil, water, sewage) may contain BSL-2 organisms even though the sample itself was collected from a non-clinical source. For example, urban waterways may harbour *Salmonella*, *Legionella*, and antibiotic-resistant *Enterobacteriaceae*. While the risk to healthy adults from handling these samples with standard BSL-1 precautions (gloves, hand washing, no mouth pipetting) is low, practitioners should:

1. Never attempt to culture organisms from environmental samples in a BSL-1 facility.
2. Treat all environmental samples as potentially containing pathogens.
3. Decontaminate all waste (autoclave or 10% bleach for 30 minutes) before disposal.
4. Wash hands thoroughly after handling samples.

24.9.6 Regulatory Frameworks by Region

Regulations governing the use of sequencing technology outside institutional settings vary by jurisdiction:

- **United States:** The NIH Guidelines for Research Involving Recombinant or Synthetic Nucleic Acid Molecules apply to all NIH-funded research. The CDC Select Agent Program regulates possession and use of certain dangerous pathogens. Community laboratories that do not receive federal funding are not bound by NIH Guidelines but may be subject to state and local regulations.
- **European Union:** The EU Directive 2009/41/EC on contained use of genetically modified micro-organisms applies to any facility working with GMOs. Sequencing of

naturally occurring organisms does not fall under this directive. GDPR applies to any project generating identifiable human data.

- **United Kingdom:** The Health and Safety Executive (HSE) regulates work with biological agents under COSHH regulations. The Human Tissue Act 2004 governs the use of human tissue for research.
- **Australia:** The Office of the Gene Technology Regulator (OGTR) oversees work with GMOs. Sequencing of non-GMO material is generally unregulated at the federal level.

24.10 Building a Sustainable Accessible Sequencing Programme

Accessible sequencing is most impactful when it is sustained over time rather than limited to one-off demonstrations. This section addresses practical strategies for building a lasting programme.

24.10.1 Funding Models

- **Grant funding:** Many funding agencies now support public engagement, citizen science, and science education. NSF CAREER awards, NIH SEPA (Science Education Partnership Awards), Wellcome Trust Public Engagement Grants, and ERC Consolidator Grants with outreach components can fund sequencing equipment and consumables.
- **Crowdfunding:** Platforms such as Experiment.com and Kickstarter have successfully funded community sequencing projects. A well-designed crowdfunding campaign with clear deliverables (e.g., “We will sequence 100 local water samples and publish a biodiversity map”) can raise \$5,000–\$20,000.
- **Membership fees:** Community laboratories sustain operations through monthly membership fees (\$50–\$150/month) that cover rent, utilities, and shared consumables. Sequencing supplies can be funded through supplemental project-specific fees.
- **Industry partnerships:** ONT and other sequencing companies have programmes that provide discounted or donated equipment to community labs and educational institutions.

24.10.2 Training and Skill Development

The bottleneck for accessible sequencing is often not hardware or reagent costs but human expertise. Effective training programmes should cover:

1. **Wet-lab skills:** Micropipetting technique, DNA extraction, library preparation, flow cell loading. Hands-on practice with supervision is essential—no amount of video tutorials substitutes for guided bench work.
2. **Computational skills:** Command-line basics (navigating directories, running programs, reading output files), basecalling, alignment, assembly. Free resources include the ONT nanopore sequencing tutorials, Software Carpentry workshops, and online courses on platforms such as Coursera and edX.
3. **Experimental design:** Controls, replicates, appropriate sequencing depth, and statistical considerations (see [Chapter 22](#) for a detailed treatment).
4. **Data management:** File naming conventions, metadata recording, backup strategies, and data deposition in public repositories.

Start Simple, Iterate

The most successful accessible sequencing programmes begin with a single, well-defined project (e.g., “sequence the bacteria in our local creek”) and expand gradually as participants gain skills and confidence. Resist the temptation to attempt a complex multi-omic project on day one. Master the basics of DNA extraction, library preparation, and basecalling before adding complexity.

24.10.3 Data Quality and Reproducibility

One concern about accessible sequencing is data quality. Are sequences generated in community laboratories, field settings, or teaching labs of sufficient quality for research? The answer depends on adherence to good laboratory practices:

- **Use positive controls:** Include a known DNA sample (e.g., [ONT](#) Lambda control DNA, *E. coli* K-12 genomic DNA) in every sequencing run. If the positive control fails, the entire run should be treated as suspect.
- **Use negative controls:** Include a no-template control in every [PCR](#) amplification and a blank extraction control for every batch of extractions. Negative controls that show amplification indicate contamination.
- **Record metadata:** Document every detail of the protocol—reagent lot numbers, incubation times, flow cell IDs, software versions, basecalling models. This is essential for troubleshooting and reproducibility.
- **Compare to references:** Whenever possible, validate results against known reference sequences or orthogonal methods (e.g., Sanger sequencing of a subset of samples).

24.11 Looking Forward: The Future of Accessible Sequencing

The trend toward accessible sequencing is accelerating. Several developments on the horizon will further lower barriers:

- **Sample-to-answer devices:** Integrated devices that combine sample lysis, DNA extraction, library preparation, and sequencing in a single consumable—eliminating the need for laboratory equipment and pipetting skills entirely. [ONT](#)’s prototype “VolTRAX” microfluidic device represents an early step in this direction.
- **Improved basecalling on edge devices:** Advances in neural network quantisation and hardware acceleration will enable SUP-quality basecalling on low-power devices (phones, tablets, embedded systems), removing the GPU bottleneck.
- **Falling consumable costs:** Flow cell costs continue to decline. If Flongle-class flow cells reach \$50 per unit, the per-sample cost for bacterial genomes will approach \$100, making sequencing as routine as PCR.
- **Protein sequencing:** [ONT](#) and other companies are developing nanopore-based protein sequencing, which would extend the accessible-sequencing paradigm beyond nucleic acids.
- **Regulatory clarity:** As community sequencing becomes more common, regulatory frameworks will mature to address the specific circumstances of non-institutional research, providing clearer guidance and reducing uncertainty for practitioners.

The democratisation of sequencing is not merely a technological achievement—it is a societal one. When a high school student can identify the species in a local pond, when a community group can monitor antimicrobial resistance in their watershed, and when a field biologist can sequence a viral genome during an outbreak without waiting weeks for results from a distant laboratory, the power of genomics is multiplied far beyond what any single institution could achieve. The challenge now is to ensure that this expansion is accompanied by appropriate training, ethical awareness, and commitment to data quality.

The Accessible Sequencing Ecosystem

Accessible sequencing succeeds when four elements come together:

1. **Affordable hardware:** Portable, low-cost sequencing devices (MinION, Flongle).
2. **Simple protocols:** Rapid library preparation kits that require minimal equipment and expertise (see [Chapter 6](#)).
3. **User-friendly analysis:** Cloud-based or graphical bioinformatics tools (EPI2ME, Galaxy) that do not require command-line proficiency.
4. **Community support:** Training resources, community laboratories, online forums, and mentorship networks that help newcomers overcome the learning curve.

When all four elements are in place, sequencing transitions from a specialised technique to a broadly accessible scientific tool—much as microscopy did in the 19th century and PCR did in the late 20th century.

24.12 Summary

This chapter has surveyed the expanding landscape of accessible sequencing, from the practical mechanics of budget planning and platform selection to the broader implications of democratised genomics. The key takeaways are:

1. **Entry costs are genuinely low:** A complete nanopore sequencing setup can be assembled for \$1,000–\$5,000, enabling bacterial genomes, amplicon surveys, and small targeted projects.
2. **Budget realistically:** Consumables, compute, and personnel time dominate costs. The sequencing device itself is often the smallest line item.
3. **Field sequencing is proven:** Portable sequencing has been deployed in outbreak response, biodiversity monitoring, and remote research settings with real scientific impact.
4. **Education benefits enormously:** Real sequencing in teaching labs provides an unmatched pedagogical experience at a per-student cost comparable to traditional molecular biology exercises.
5. **Citizen science is growing:** eDNA barcoding, food fraud detection, and [AMR](#) monitoring are all feasible as community-driven projects.
6. **Legal and ethical frameworks apply:** The Nagoya Protocol, biosafety regulations, ethics approval for human samples, and data sharing obligations apply regardless of the institutional setting.
7. **Quality requires discipline:** Positive controls, negative controls, metadata recording, and reproducible protocols are non-negotiable, whether sequencing is performed in a genome centre or a community laboratory.

The tools are available. The protocols are established. The cost barrier has fallen. The remaining challenge is to ensure that the growing community of accessible sequencing practitioners operates with scientific rigour, ethical responsibility, and a commitment to open science.

Chapter 25

Practical Protocols: From Sample to Sequencer

Key Concept

This chapter provides detailed, step-by-step laboratory protocols for the complete nanopore sequencing workflow—from sample collection and DNA extraction through library preparation, flow cell loading, and sequencing. Each protocol has been optimised for reliability and is presented in a format that can be printed and brought directly to the bench. Where previous chapters discussed the *theory* behind library preparation (Chapter 6) and experimental design (Chapter 22), this chapter provides the *practice*: precise volumes, incubation times, decision trees, and troubleshooting tables to ensure success on the first attempt.

25.1 Safety First: Laboratory Hazards and Precautions

Before beginning any sequencing workflow, it is essential to understand the safety requirements of your laboratory environment. Sequencing protocols involve biological samples, chemical reagents, and electrical equipment—each with specific hazard profiles.

25.1.1 Biosafety Levels

The Centres for Disease Control and Prevention (CDC) and the World Health Organisation (WHO) classify laboratories into four **Biosafety Levels** (BSL) based on the risk posed by the organisms handled:

BSL-1

Work with well-characterised agents not known to cause disease in healthy adults. Standard microbiological practices; no special containment equipment required. *Examples*: non-pathogenic *Escherichia coli* strains, environmental DNA from water samples, human saliva for self-experimentation.

BSL-2

Work with agents that pose moderate hazards to personnel and the environment. Requires restricted access, biohazard warning signs, biosafety cabinets for aerosol-generating procedures, and autoclaving of waste before disposal. *Examples*: human blood, clinical samples (stool, sputum), pathogenic bacteria such as *Staphylococcus aureus* or *Salmonella* spp.

BSL-3

Work with agents that can cause serious or potentially lethal disease through inhalation. Requires respiratory protection, double-door entry, directional airflow, and HEPA filtration of exhaust air. *Examples: Mycobacterium tuberculosis*, SARS-CoV-2 viral cultures, *Coxiella burnetii*.

BSL-4

Work with agents that pose a high risk of life-threatening disease, aerosol transmission, and for which no vaccine or therapy is available. Requires positive-pressure protective suits or Class III biological safety cabinets. *Examples: Ebola virus, Marburg virus, Nipah virus.*

Most Sequencing Labs Operate at BSL-1 or BSL-2

The vast majority of **NGS** sequencing laboratories operate at BSL-1 (environmental samples, cell lines, purified DNA) or BSL-2 (human clinical samples, pathogen surveillance). If your samples are purified DNA or RNA—already extracted and inactivated—they are generally considered BSL-1 regardless of the original source. Always consult your institutional biosafety committee (IBC) before beginning work with new sample types.

25.1.2 Personal Protective Equipment (PPE)

Required **PPE** varies by biosafety level and reagent exposure:

- **BSL-1 minimum:** Laboratory coat, closed-toe shoes, nitrile gloves. Safety glasses when handling chemicals.
- **BSL-2 additions:** Face protection (safety glasses or face shield) when risk of splash exists. Lab coat must be removed before leaving the laboratory.
- **Chemical handling:** Phenol-chloroform work requires a chemical fume hood, double gloving (nitrile outer, latex inner), chemical splash goggles, and a lab coat with long sleeves.
- **UV exposure:** When using UV transilluminators for gel imaging, wear UV-blocking face shields and ensure exposed skin is covered.

25.1.3 Chemical Hazards in Sequencing Workflows**Never Mix Guanidinium Salts with Bleach**

Chaotropic lysis buffers contain guanidinium salts. Mixed with bleach, these produce highly toxic cyanide and chlorine gases. **Never** add bleach to extraction kit waste. Neutralise with 10× volume 1 M NaOH before disposal, or dispose as hazardous chemical waste per your institution's EHS protocols.

25.1.4 Waste Disposal

Segregate waste into four streams: (1) **biological waste** (contaminated tips, tubes, swabs—autoclave at BSL-2 or biohazard bag for incineration); (2) **chemical waste** (phenol-chloroform, guanidinium lysis buffers, ethidium bromide—labelled containers via EHS); (3) **sharp**s (scalpel blades, broken glass—puncture-resistant containers); (4) **general waste** (clean consumables, packaging).

Table 25.1: Common chemical hazards encountered in NGS sample preparation workflows.

Chemical	Hazard Class	Risk	Handling
Phenol-chloroform	Acute toxicity, corrosive	Burns, liver/kidney damage, CNS depression	Fume hood only
Guanidinium thiocyanate	Acute toxicity	Harmful if swallowed or inhaled; reacts violently with bleach to produce toxic gas	Avoid bleach contact
Ethanol (100%)	Flammable	Flash point 13°C; ignition risk near open flames	No open flames
Isopropanol	Flammable, irritant	Flash point 12°C; eye and respiratory irritant	Ventilated area
β -Mercaptoethanol	Acute toxicity, stench	Highly toxic if inhaled; penetrates nitrile gloves	Fume hood; double glove
SYBR Green/EtBr	Mutagen (EtBr)	Potential carcinogenicity (ethidium bromide)	Designated staining area
Sodium hypochlorite	Corrosive, oxidiser	Skin burns; toxic chlorine gas if mixed with acids	Separate from acids

Pre-Experiment Safety Checklist

Before beginning any protocol in this chapter, verify:

- ☐ You have read and understand the Safety Data Sheets (SDS) for all reagents in the protocol.
- ☐ Appropriate PPE is available and donned: lab coat, nitrile gloves, safety glasses.
- ☐ Chemical fume hood is operational (if phenol-chloroform or guanidinium reagents are used).
- ☐ Biological waste, chemical waste, and sharps containers are accessible and not full.
- ☐ First aid kit location is known; eyewash station is functional.
- ☐ Emergency contact numbers (EHS, campus emergency) are posted.
- ☐ You have institutional biosafety approval (IBC protocol) for the sample types you will handle.
- ☐ A lab partner or colleague knows you are performing the experiment (do not work alone with hazardous chemicals).

25.2 DNA Extraction Protocols

High-quality DNA is the foundation of every successful sequencing experiment. This section provides three complete extraction protocols optimised for different sample types commonly encountered in teaching and research laboratories.

What Makes DNA “High Quality” for Nanopore Sequencing?

For **ONT** nanopore sequencing, the ideal DNA has:

- **Purity:** A260/A280 ratio of 1.8–2.0 (free of protein contamination). A260/A230 ratio of 2.0–2.2 (free of organic solvent and salt contamination).
- **Integrity:** High molecular weight (HMW), ideally >10 kb fragments. For ultra-long read applications, >50 kb is desirable.
- **Concentration:** Sufficient mass input for the library preparation kit (typically 200–1,000 ng for rapid kits, 1,000 ng for ligation kits).
- **No inhibitors:** Free of **PCR** inhibitors (humic acids, polysaccharides, haemoglobin, melanin) that can interfere with enzymatic steps.

25.2.1 Protocol A: DNA from Human Saliva

Saliva is one of the easiest and least invasive sources of human genomic DNA, making it ideal for teaching laboratories and citizen science projects. Commercial saliva collection kits stabilise DNA at room temperature for weeks to months.

Table 25.2: Materials list for saliva DNA extraction.

Item	Supplier	Approx. Cost	Notes
DNA Genotek OG-500 kit	DNA Genotek	\$25–30/unit	Includes collection tube with stabiliser
QIAGEN DNeasy Blood & Tissue Kit	QIAGEN	\$5–7/prep	Cat. No. 69504 (50 preps)
Proteinase K (20 mg/mL)	Included in kit	—	Also available separately
Ethanol (96–100%)	General supplier	\$0.10/prep	Molecular biology grade
Phosphate-buffered saline (PBS)	General supplier	\$0.05/prep	Sterile, 1×
Microcentrifuge (up to 20,000 × <i>g</i>)	—	—	Benchtop model
Heat block or water bath (56°C)	—	—	Temperature-controlled
NanoDrop spectrophotometer	Thermo Fisher	—	For A260/280 and A260/230
Qubit fluorometer + dsDNA BR kit	Thermo Fisher	\$2–3/assay	Accurate dsDNA quantification

Materials

Protocol: DNA Extraction from Human Saliva

Estimated time: 2–3 hours (including incubation). **Expected yield:** 1–5 µg gDNA. **Sample collection** (the day before or same day):

1. Instruct the donor not to eat, drink, smoke, or chew gum for 30 minutes before collection.
2. Have the donor provide 2 mL of saliva into the OG-500 collection tube. Close the cap to release the stabilising buffer and mix by inverting 5–10 times.
3. If not extracting immediately, store at room temperature for up to 12 months (DNA is stabilised in the buffer).

DNA extraction (DNeasy Blood & Tissue kit protocol, adapted):

1. Transfer 500 µL of stabilised saliva to a 1.5 mL microcentrifuge tube.
2. Add 20 µL Proteinase K and 500 µL Buffer AL. Vortex thoroughly for 15 s.
3. Incubate at 56°C for 30 minutes (or up to 1 hour for maximum yield).
4. Briefly centrifuge to collect condensation from the lid.
5. Add 500 µL ethanol (96–100%). Mix by pulse-vortexing for 15 s.
6. Transfer 600 µL of the mixture to a DNeasy Mini spin column in a collection tube. Centrifuge at 6,000 × *g* for 1 minute. Discard flow-through and collection tube.
7. Repeat step 6 with remaining mixture using a new collection tube.
8. Add 500 µL Buffer AW1 to the column. Centrifuge at 6,000 × *g* for 1 minute. Discard flow-through and collection tube.
9. Add 500 µL Buffer AW2 to the column. Centrifuge at 20,000 × *g* for 3 minutes to dry the membrane.
10. Transfer the column to a clean, labelled 1.5 mL microcentrifuge tube.
11. Add 100 µL Buffer AE (prewarmed to 56°C) directly onto the membrane. Incubate at room temperature for 5 minutes.
12. Centrifuge at 6,000 × *g* for 1 minute to elute DNA.
13. **Optional second elution:** Repeat steps 11–12 with a second 100 µL Buffer AE into a separate tube to recover additional DNA (typically 15–30% extra yield but at lower concentration).

Quality control:

1. Measure A260/A280 and A260/A230 on NanoDrop (use 1.5 µL).
2. Quantify with Qubit dsDNA BR assay (use 1 µL of sample in 199 µL working solution).
3. If yield >200 ng and ratios are acceptable, proceed to library preparation.

Increasing Saliva DNA Yield

If yield is below 1 μg : (1) increase saliva input to 750 μL (scale reagents 1.5 $\times$); (2) extend Proteinase K to 1 h at 56°C; (3) perform the second elution and pool with the first. Gently scraping the inner cheek with a cytology brush before collection substantially increases yield.

Troubleshooting: Saliva DNA Extraction

Table 25.3: Troubleshooting guide for saliva DNA extraction.

Problem	Likely Cause	Solution
Very low yield ($<0.5 \mu\text{g}$)	Insufficient buccal cells	Have donor scrape inner cheeks gently before collection; increase saliva volume to 1 mL
A260/A280 < 1.7	Protein contamination	Extend Proteinase K incubation to 1 h; add additional 10 μL Proteinase K
A260/A230 < 1.5	Guanidinium salt carry-over	Perform an additional AW2 wash; ensure 3-min spin at $20,000 \times g$
Viscous lysate, hard to pipette	Mucus in saliva	Add 5 μL RNase A before AL buffer; vortex more vigorously; pass through 20G needle
Clogged spin column	Cellular debris	Pre-clear lysate by centrifuging at $10,000 \times g$ for 2 min and transfer supernatant
Degraded DNA on gel	Old saliva without stabiliser	Use collection kit with stabiliser; do not freeze unstabilised saliva
High Qubit but low NanoDrop	RNA co-purification	Add RNase A treatment step; Qubit is correct for dsDNA
Low Qubit but high NanoDrop	Free nucleotides / contaminants	Trust Qubit value; NanoDrop overestimates in presence of contaminants

25.2.2 Protocol B: Environmental DNA (eDNA) from Water Samples

Environmental DNA (eDNA) analysis detects organisms from DNA shed into their environment—skin cells, mucus, faeces, gametes—without capturing or observing the organisms. Water is the most common eDNA matrix.

Filtration Setup and Equipment

The critical first step is **filtration**: concentrating trace DNA from large volumes of water onto a filter membrane.

- **Filter type:** Enclosed filter capsules (e.g., Sterivex 0.45 μm) or open filter cups with polycarbonate, polyethersulfone (PES), or cellulose nitrate membranes (47 mm diameter).
- **Pore size:** 0.22 μm (captures bacteria and free DNA) or 0.45 μm (faster flow rate, captures most cellular material). For metabarcoding of eukaryotes, 0.45 μm is standard.

- **Volume:** Filter 0.5–2 L per sample for freshwater; 1–4 L for marine water (lower eDNA concentration). Stop if the filter clogs before reaching target volume and record actual volume filtered.
- **Filtration method:** Peristaltic pump (field-portable) or vacuum manifold (laboratory). Syringe filtration is acceptable for small volumes (<500 mL) using Sterivex capsules.

eDNA Degrades Rapidly

eDNA in water degrades continuously (UV, nucleases, pH, temperature); half-life is 1–14 days. **Filter within 6 hours of collection** or store on ice and filter within 24 hours. Add preservation buffer (Longmire's, ATL, or ethanol) to the filter immediately and store at -20°C .

Protocol: eDNA Extraction from Water (DNeasy PowerWater Kit)

Estimated time: 1.5–2 hours (after filtration). **Kit:** QIAGEN DNeasy PowerWater Kit (Cat. No. 14900-50-NF).

Filtration (field or laboratory):

1. Set up vacuum manifold or peristaltic pump with a 47 mm polycarbonate filter (0.45 μm pore size) in a sterile filter funnel.
2. Filter 1 L of water sample. Record exact volume filtered if the filter clogs before 1 L.
3. Using sterile forceps, fold the filter in half (sample side inward) and place in a provided PowerWater bead tube. Alternatively, for Sterivex capsules, remove the inlet, add 750 μL ATL buffer, cap, and store.
4. If not extracting immediately, add 1 mL PW1 buffer and store at -20°C .

Extraction:

1. Ensure the filter membrane is in the PowerWater bead tube with solution PW1 (1 mL).
2. Secure tubes horizontally on a vortex adapter. Vortex at maximum speed for 5 minutes.
3. Centrifuge at $4,000 \times g$ for 1 minute. Transfer supernatant ($\sim 900 \mu\text{L}$) to a clean 2 mL tube.
4. Add 200 μL solution IRS (inhibitor removal solution). Vortex briefly. Incubate at 4°C for 5 minutes.
5. Centrifuge at $13,000 \times g$ for 1 minute. Avoiding the pellet, transfer supernatant to a clean 2 mL tube.
6. Add 650 μL solution PW3. Vortex briefly.
7. Load 650 μL onto a spin filter. Centrifuge at $13,000 \times g$ for 1 minute. Discard flow-through. Repeat until all lysate has been passed through the filter.
8. Add 650 μL solution PW4 (ethanol wash). Centrifuge at $13,000 \times g$ for 1 minute. Discard flow-through.
9. Add 650 μL solution PW5 (second ethanol wash). Centrifuge at $13,000 \times g$ for 1 minute. Discard flow-through.
10. Centrifuge empty column at $13,000 \times g$ for 2 minutes to dry the membrane.
11. Transfer column to a clean 1.5 mL tube. Add 50 μL solution EB (10 mM Tris, pH 8.0, prewarmed to 50°C) to the centre of the membrane. Incubate 5 minutes.
12. Centrifuge at $13,000 \times g$ for 1 minute. The eluate is your purified eDNA.

Quality control:

1. Measure concentration by Qubit dsDNA HS (high sensitivity) assay—eDNA concentrations are often very low (0.1–10 ng/ μL).
2. NanoDrop is unreliable at low concentrations; use only for A260/280 ratio if concentration is $>5 \text{ ng}/\mu\text{L}$.
3. If concentration is below the library prep input requirement, consider whole-genome amplification or concentration by vacuum centrifugation.

Expected eDNA Yields by Water Type**Field Controls for eDNA**

Always collect **field negative controls**: filter 1 L of distilled water through your equipment at each site. Process identically through extraction and sequencing. Without field negatives, true detections cannot be distinguished from contamination.

Table 25.4: Typical eDNA yields from 1 L water samples using DNeasy PowerWater kit.

Water Type	Typical Yield	DNA Quality	Notes
Freshwater pond/lake	0.5–5 ng/ μ L	Variable	High microbial DNA; may need inhibitor removal
River/stream	0.2–3 ng/ μ L	Moderate	Turbidity affects filtration; pre-filter if needed
Marine (coastal)	0.1–2 ng/ μ L	Good	Salts may carry over; extra wash recommended
Marine (open ocean)	0.01–0.5 ng/ μ L	Good	Very low biomass; filter 2–4 L
Aquarium/tank	2–20 ng/ μ L	Good	High organism density; dilution may be needed
Wastewater	5–50 ng/ μ L	Poor	Heavy PCR inhibitor load; PowerSoil Pro kit may be better

25.2.3 Protocol C: DNA from Food and Plant Samples

Food authenticity testing and plant genomics present unique challenges: high polysaccharide and polyphenol content, PCR inhibitors from processing, and degraded DNA in processed products.

CTAB Extraction for Fresh Plant Tissue

The cetyltrimethylammonium bromide (CTAB) method is the gold standard for plant tissues rich in polysaccharides and secondary metabolites.

Protocol: CTAB DNA Extraction from Plant Tissue

Estimated time: 3–4 hours. **Expected yield:** 5–50 μg from 100 mg tissue.

Reagent preparation (make fresh or use within 1 month):

- 2 $\times$ CTAB buffer: 2% CTAB, 100 mM Tris-HCl (pH 8.0), 20 mM EDTA, 1.4 M NaCl, 1% PVP-40. Autoclave. Add 0.2% β -mercaptoethanol fresh before use.
- Chloroform:isoamyl alcohol (24:1 v/v).
- Isopropanol (molecular biology grade, at -20°C).
- 70% ethanol.
- TE buffer (10 mM Tris-HCl, 1 mM EDTA, pH 8.0).
- RNase A (10 mg/mL).

Protocol:

1. Preheat 2 $\times$ CTAB buffer (with β -mercaptoethanol) to 65°C in a water bath.
2. Weigh 100 mg of fresh leaf tissue (young, actively growing tissue gives highest quality DNA). Flash-freeze in liquid nitrogen.
3. Grind frozen tissue to a fine powder in a pre-chilled mortar and pestle with liquid nitrogen. Transfer powder to a 2 mL tube before it thaws.
4. Add 800 μL of preheated 2 $\times$ CTAB buffer. Mix by gentle inversion 10 times.
5. Incubate at 65°C for 30–60 minutes, inverting every 10 minutes.
6. Cool to room temperature. Add 800 μL chloroform:isoamyl alcohol (24:1). Invert gently 50 times.
7. Centrifuge at $12,000 \times g$ for 10 minutes at room temperature.
8. Carefully transfer the upper aqueous phase ($\sim 600 \mu\text{L}$) to a new 1.5 mL tube. **Do not** disturb the interface.
9. Add 5 μL RNase A (10 mg/mL). Incubate at 37°C for 30 minutes.
10. Add 0.7 $\times$ volume ($\sim 420 \mu\text{L}$) ice-cold isopropanol. Invert gently 20 times. Incubate at -20°C for 30 minutes (or overnight for maximum recovery).
11. Centrifuge at $12,000 \times g$ for 15 minutes at 4°C . A white DNA pellet should be visible.
12. Carefully decant the supernatant. Wash pellet with 500 μL ice-cold 70% ethanol.
13. Centrifuge at $12,000 \times g$ for 5 minutes at 4°C . Remove all ethanol with a pipette.
14. Air-dry the pellet for 5–10 minutes (do not over-dry; the pellet becomes difficult to resuspend).
15. Resuspend in 50–100 μL TE buffer. Allow to dissolve at 4°C for 1–2 hours (or overnight) with occasional gentle flicking.

Chloroform Safety

Chloroform is a suspected carcinogen. Always work in a fume hood. Dispose of organic waste in halogenated solvent containers—never down the drain.

Commercial Kits for Processed Food

For processed foods, use the QIAGEN DNeasy Mericon Food Kit (Cat. No. 69514): 200 mg homogenised input, overnight Proteinase K lysis at 60°C for heavily processed samples, expected yield 0.1–10 μg .

PCR Inhibitor Mitigation Strategies

PCR inhibitors are the primary challenge with plant and food DNA extracts. Common strategies include:

Table 25.5: PCR inhibitor mitigation strategies for challenging samples.

Inhibitor	Source	Mitigation
Humic acids	Soil, water, leaf litter	PowerClean Pro cleanup kit; PVP-40 in CTAB buffer
Polysaccharides	Plant tissue, algae	Increase NaCl to 1.4 M in CTAB; add high-salt wash step
Polyphenols	Tea, wine, berries, wood	PVP-40 or PVPP in lysis buffer; activated charcoal
Melanin	Squid ink, sepia, dark soils	Dilution; BSA addition (400 ng/ μ L) to PCR reaction
Haemoglobin	Blood in meat/tissue	DNeasy Blood & Tissue optimised protocol; dilution
Maillard products	Cooked/processed foods	OneStep PCR Inhibitor Removal Kit (Zymo D6030)
Fat/lipids	Dairy, adipose tissue	Pre-wash with hexane; skim fat layer after lysis

The “Dilution Solution”

A simple 1:10 dilution often reduces inhibitor concentrations below the threshold affecting enzymatic reactions. Always include an internal positive control (IPC) to detect inhibition—if the IPC fails, inhibitors are present.

25.2.4 QC Decision Tree

After extraction, every DNA sample must pass through a quality control decision tree before proceeding to library preparation. The following figure provides a visual guide.

NanoDrop vs. Qubit: When They Disagree

NanoDrop measures absorbance of *everything* at 260 nm, while Qubit uses a dye specific to dsDNA. When NanoDrop reads much higher than Qubit, non-DNA contaminants are present. **Always trust Qubit for concentration, NanoDrop for purity ratios only.**

25.3 Nanopore Rapid Barcoding Kit (SQK-RBK114.96)

The Rapid Barcoding Kit SQK-RBK114.96 is designed for high-throughput multiplexing with minimal hands-on time, using a transposase to simultaneously fragment DNA and attach barcoded adapters.

Rapid Barcoding: Speed vs. Read Length

The transposase simultaneously cleaves DNA and attaches barcode adapters, achieving ~15 minutes hands-on time but limiting N50 read length to 5–10 kb. If long reads (>20 kb) are critical (structural variants, phasing, assembly), use the Ligation Kit SQK-LSK114 instead (Section 25.4).

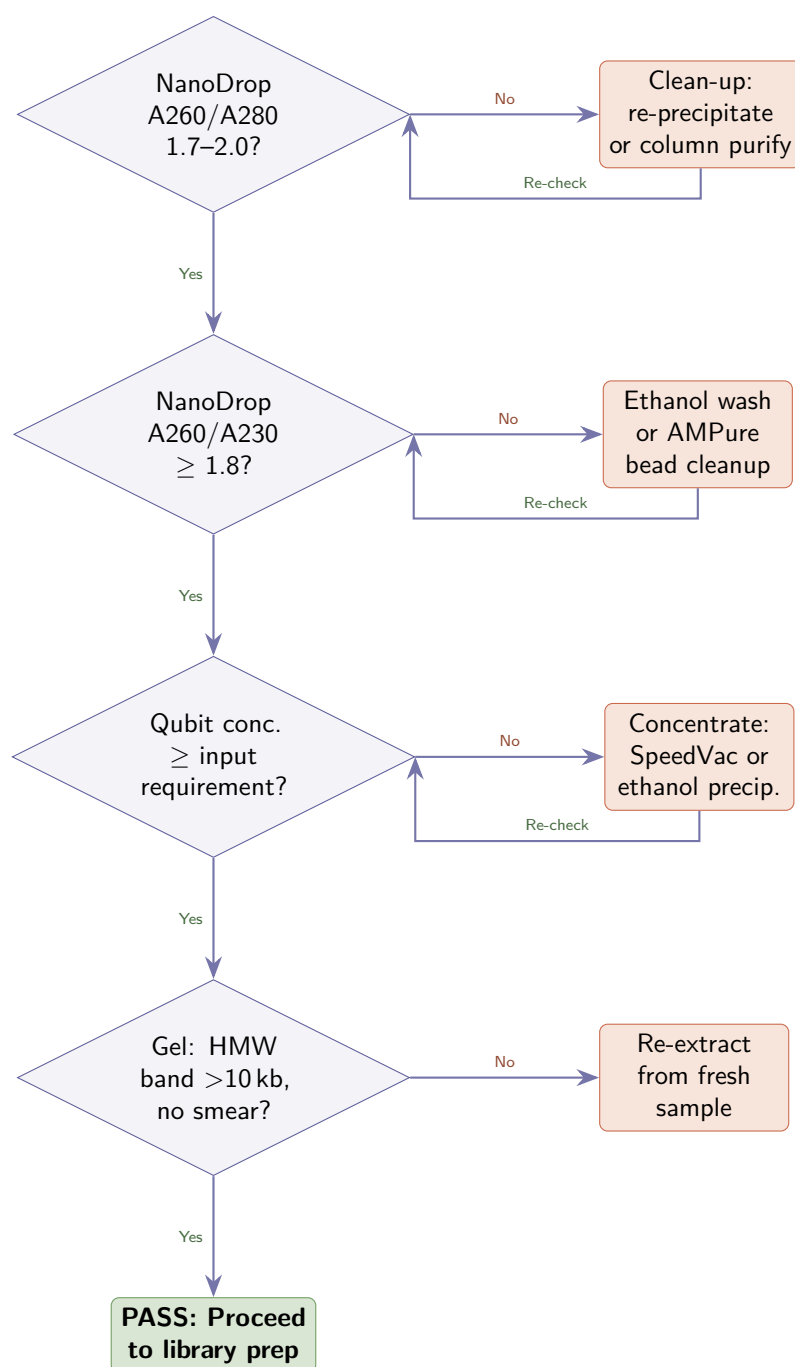


Figure 25.1: Quality control decision tree for extracted DNA. Each sample must pass four sequential checkpoints—NanoDrop purity (A260/A280 and A260/A230), Qubit concentration, and gel electrophoresis—before proceeding to library preparation. Failed samples are routed to cleanup or re-extraction steps.

Table 25.6: SQK-RBK114.96 kit specifications and input requirements.

Parameter	Specification
Kit name	Rapid Barcoding Kit 96 V14
Kit code	SQK-RBK114.96
Chemistry	V14 (Kit 14) with 5 kHz sampling
Number of barcodes	96 unique barcodes
Input DNA requirement	50–200 ng per sample (100 ng recommended)
Input DNA quality	A260/A280 ≥ 1.8 ; HMW preferred but not required
Hands-on time	~15 minutes
Compatible flow cells	R10.4.1 (FLO-MIN114, FLO-PRO114M)
Output read N50	5–10 kb (input-dependent)
Recommended applications	Amplicon sequencing, surveillance, species ID, low-complexity samples

Protocol: Nanopore Rapid Barcoding (SQK-RBK114.96)

Estimated time: 15 minutes hands-on + 30 minutes barcoding incubation. **Perform at room temperature unless stated otherwise.**

Before you start:

- Thaw Rapid Adapter F (RAP F) on ice and mix by flicking. Spin down briefly.
- Thaw barcodes (RB01–RB96) at room temperature. Spin down briefly.
- Ensure all DNA samples are quantified by Qubit and normalised to 50 ng in 5 μL with nuclease-free water.

Barcoding:

1. In a 96-well PCR plate or individual PCR strip tubes, combine per sample:
 - 5 μL DNA (50 ng)
 - 5 μL barcoded rapid adapter (e.g., RB01 for sample 1, RB02 for sample 2, etc.)
2. Mix by pipetting up and down 5 times. Do not vortex.
3. Incubate at 30°C for 2 minutes in a thermal cycler.
4. Incubate at 80°C for 2 minutes to heat-inactivate the transposase.
5. Briefly spin down the plate/tubes. Place on ice.

Pooling and adapter attachment:

6. Pool all barcoded samples into a single 1.5 mL DNA LoBind tube (total volume = n samples $\times$ 10 μL).
7. If pooling >24 samples, use only 5 μL from each reaction to keep total volume manageable.
8. Add 1 $\times$ volume of AMPure XP beads (e.g., 240 μL beads if total pool is 240 μL). Mix by gentle flicking.
9. Incubate at room temperature for 5 minutes.
10. Place on a magnetic rack for 2 minutes until solution is clear.
11. Remove and discard the supernatant.
12. With the tube still on the magnet, wash twice with 200 μL freshly prepared 80% ethanol. Wait 30 s per wash before removing ethanol.
13. Remove all residual ethanol with a fine pipette tip. Air-dry for 30 seconds (**do not over-dry**—nanopore adapters are sensitive to bead over-drying).
14. Remove tube from magnet. Resuspend beads in 15 μL Elution Buffer (EB).
15. Incubate at room temperature for 2 minutes. Return to magnet for 2 minutes.
16. Transfer 15 μL eluate to a new 1.5 mL DNA LoBind tube.
17. Add 1 μL Rapid Adapter F (RAP F). Mix by gentle flicking 10 times.
18. Incubate at room temperature for 5 minutes.

The library is now ready for loading. Proceed to flow cell priming and loading (Section 25.6).

When to Use Rapid Barcoding vs. Ligation

Choose Rapid Barcoding when speed is paramount, you need >24-plex multiplexing, for amplicon sequencing, when input DNA is limited (<200 ng), or for surveillance applications. **Choose Ligation** when long reads (>10 kb) are essential (assembly, SVs, phasing), for maximum data yield (1.5–2× more), or when native DNA modifications must be preserved.

25.4 Nanopore Ligation Sequencing Kit (SQK-LSK114)

The Ligation Sequencing Kit SQK-LSK114 is the workhorse kit for nanopore sequencing when read length and data yield must be maximised. It uses a multi-step enzymatic preparation (end repair, dA-tailing, adapter ligation) that preserves the full length of input DNA.

Ligation Libraries Preserve Read Length

Without transposase fragmentation, read length directly reflects the input DNA fragment length. With HMW DNA (>50 kb) and gentle handling (wide-bore tips, no vortexing), reads exceeding 100 kb are achievable. The trade-off: 60–90 minutes hands-on and 1,000 ng input recommended.

Protocol: Nanopore Ligation Sequencing (SQK-LSK114)

Estimated time: 60–90 minutes hands-on. **Input:** 1,000 ng gDNA in 47 μL nuclease-free water.

Before you start:

- Thaw NEBNext Companion Module reagents: FFPE Repair Buffer, FFPE Repair Mix, Ultra II End Prep Reaction Buffer, Ultra II End Prep Enzyme Mix. Keep on ice.
- Thaw Ligation Adapter (LA), Ligation Buffer (LNB), and Quick T4 DNA Ligase. Keep LNB and LA on ice; T4 ligase at -20°C until needed.
- Thaw Elution Buffer (EB) and Short Fragment Buffer (SFB). Keep at room temperature.
- Prepare fresh 80% ethanol ($\geq 500\ \mu\text{L}$).
- Resuspend AMPure XP beads by vortexing. Equilibrate to room temperature.
- Use wide-bore pipette tips throughout to minimise DNA shearing.

Step 1: DNA repair and end-prep (combines FFPE repair and end-prep in a single reaction):

1. In a 0.2 mL thin-wall PCR tube, combine:
 - 47 μL DNA (1,000 ng)
 - 1 μL FFPE DNA Repair Buffer
 - 1 μL FFPE DNA Repair Mix
 - 3.5 μL Ultra II End Prep Reaction Buffer
 - 1.5 μL Ultra II End Prep Enzyme Mix
 - 1 μL nuclease-free water

Total volume: 55 μL .

2. Mix gently by flicking the tube 10–15 times. Do not vortex.
3. Spin down briefly in a mini centrifuge.
4. Incubate in a thermal cycler: 20°C for 5 minutes, then 65°C for 5 minutes. Hold at 4°C .

Step 2: AMPure XP cleanup (removes enzymes, buffers, and short fragments):

5. Add 55 μL ($1\times$ volume) AMPure XP beads to the end-prepped DNA. Mix by gentle flicking.
6. Incubate at room temperature for 5 minutes.
7. Place on magnetic rack. Wait 2 minutes until solution clears.
8. Remove and discard supernatant.
9. Wash twice with 200 μL freshly prepared 80% ethanol. Wait 30 s each wash.
10. Remove residual ethanol with a fine tip. Air-dry for 30 seconds. **Do not over-dry.**
11. Remove from magnet. Resuspend in 61 μL nuclease-free water. Incubate 2 minutes.
12. Return to magnet for 2 minutes. Transfer 61 μL eluate to a clean tube.

Step 3: Adapter ligation:

13. In the eluate tube (61 μL end-prepped DNA), add:
 - 25 μL Ligation Buffer (LNB)
 - 10 μL Quick T4 DNA Ligase
 - 5 μL Ligation Adapter (LA)

Total volume: 101 μL .

14. Mix by gentle flicking 10 times. Spin down briefly.
15. Incubate at room temperature for 10 minutes.

Step 4: AMPure XP cleanup with Short Fragment Buffer:

16. Add 40 μL AMPure XP beads ($0.4\times$ ratio for long-fragment selection). Mix gently.
17. Incubate at room temperature for 5 minutes.
18. Place on magnetic rack for 2 minutes.
19. Remove and discard supernatant. 557
20. Wash twice with 250 μL Short Fragment Buffer (SFB). **Not ethanol**—SFB is specifically formulated for this step.
21. Remove residual SFB. Air-dry for 30 seconds.

Do Not Use Ethanol Washes After Ligation

After ligation, wash with Short Fragment Buffer (SFB), **not** ethanol. Ethanol denatures the motor protein on the adapter, destroying the library—this is the most common ligation prep mistake and produces zero data. SFB removes short fragments while preserving motor protein integrity.

25.4.1 AMPure XP Bead Ratio Optimisation

The ratio of AMPure XP beads to sample volume determines the size cutoff for DNA fragment retention. Lower ratios select for longer fragments. This is critical for tuning read length distributions.

Table 25.7: AMPure XP bead ratio and approximate fragment size cutoff for DNA selection.

Bead:Sample Ratio	Size Cutoff	Recovery %	Use Case
2.0×	>100 bp	>95%	Amplicon cleanup; retain all fragments
1.0×	>200 bp	>90%	Standard cleanup; general-purpose
0.8×	>400 bp	80–90%	Post end-prep cleanup; remove adapter dimers
0.6×	>800 bp	70–85%	Select for longer fragments; reduce short-fragment noise
0.45×	>1.5 kb	60–75%	Long-read enrichment; post-ligation
0.4×	>2 kb	50–70%	Aggressive long-read selection; recommended for LSK114
0.35×	>3 kb	40–60%	Ultra-long read enrichment; significant yield loss

Bead Ratio Fine-Tuning

Exact cutoffs depend on DNA concentration, buffer composition, and temperature. For critical applications, test 3–4 ratios empirically on a gel or TapeStation. A 0.4× ratio is the default for SQK-LSK114 post-ligation cleanup; going below 0.35× causes unacceptable yield loss.

25.5 16S/ITS Amplicon Library Preparation

Amplicon sequencing of phylogenetic marker genes is one of the most accessible NGS applications. Nanopore sequencing enables full-length 16S (~1,500 bp) and ITS sequencing, providing superior taxonomic resolution compared to short-read approaches targeting single variable regions.

Table 25.8: Recommended primers for full-length amplicon nanopore sequencing.

Target	Primer	Sequence (5'→3')	Amplicon	Notes
16S (bacteria, archaea)	27F	AGAGTTTGATCMTGGCTCAG	~1,465 bp	Universal bacterial; covers V1–V9
	1492R	TACGGYTACCTTGTTACGACTT		
ITS (fungi)	ITS1f	CTTGGTCATTTAGAGGAAGTAA	500–850 bp	Fungal-biased forward primer; broad coverage
	ITS4	TCCTCCGCTTATTGATATGC		
18S (eukaryotes)	Euk-A	AACCTGGTTGATCCTGCCAGT	~1,800 bp	Broad eukaryotic coverage
	Euk-B	TGATCCTTCTGCAGGTTTCACCTAC		

25.5.1 Primer Selection

Adding Barcode Tails to Primers

With SQK-RBK114.96, no primer modification is needed—the transposase attaches barcodes directly. For native barcoding (SQK-NBD114.96), add barcode sequences as 5' tails on primers. Consult the [ONT](#) protocol for current tail sequences (these change between kit versions).

25.5.2 PCR Cycling Conditions

Table 25.9: PCR cycling conditions for full-length 16S and ITS amplicon generation.

Step	Temperature	Time	Cycles	Notes
Initial denaturation	95°C	3 min	1	Complete template denaturation
Denaturation	95°C	30 s		Reduce cycles to 25 for high-input samples
Annealing	55°C	30 s	25–30	55°C works for both 27F/1492R and ITS1f/ITS4
Extension	72°C	90 s		1 min/kb; 90 s for ~1.5 kb 16S
Final extension	72°C	5 min	1	Complete all partial products
Hold	4°C	∞	—	Short-term storage

Protocol: 16S Amplicon PCR (25 μ L Reaction)**Per reaction:**

- 12.5 μ L LongAmp Taq 2 $\times$ Master Mix (NEB, Cat. No. M0287)
- 1 μ L forward primer 27F (10 μ M stock)
- 1 μ L reverse primer 1492R (10 μ M stock)
- 1 μ L template DNA (1–10 ng)
- 9.5 μ L nuclease-free water

Cycling: Use the conditions in [Table 25.9](#) with 25–30 cycles.

Post-PCR:

1. Confirm amplification by running 2 μ L on a 1% agarose gel (expect a single band at $\sim$ 1,465 bp).
2. Clean up with 0.8 $\times$ AMPure XP beads to remove primers and dimers.
3. Quantify by Qubit dsDNA BR assay.
4. Normalise all samples to the same concentration before pooling for barcoding.

25.5.3 Barcoding Strategy and Controls

For multiplexed amplicon experiments, proper barcoding and controls are essential:

- **Barcode assignment:** Assign unique barcodes to each sample with a mapping sheet linking barcodes to sample identifiers and metadata.
- **Positive control:** Include a mock community (e.g., ZymoBIOMICS Microbial Community Standard, D6300: 8 bacteria + 2 fungi at known ratios) to assess classification accuracy.
- **Negative controls:** Include **both** an extraction blank and a [PCR](#) no-template control (NTC). Both must be barcoded and sequenced. Taxa in negatives indicate contamination.
- **Balanced pooling:** Aim for equimolar pooling to ensure even read distribution across samples.

The “Kitome” Problem

Extraction kits and [PCR](#) reagents contain trace bacterial DNA (“kitome”). In low-biomass samples these contaminants can dominate results. Common kitome genera: *Bradyrhizobium*, *Ralstonia*, *Methylobacterium*, *Sphingomonas*. Extraction blanks are the only way to identify and remove these contaminants. Never omit them.

25.6 Loading and Running the Flow Cell

The final steps are flow cell quality assessment, priming, library loading, and run monitoring.

25.6.1 Flow Cell QC Check

Before loading any library, verify that the flow cell has sufficient active nanopores.

Protocol: Flow Cell Quality Check

1. Insert the R10.4.1 flow cell (FLO-MIN114) into the MinION or GridION device.
2. Open MinKNOW software. Navigate to the flow cell check (or “Platform QC”) function.
3. Initiate the flow cell check. The software will apply a voltage and measure current through each pore (takes ~5 minutes).
4. Read the result:
 - **>800 active pores:** Excellent. Proceed with loading.
 - **600–800 active pores:** Acceptable. Expect slightly reduced throughput.
 - **<600 active pores:** Contact [ONT](#) support for a replacement. Running a library on a low-pore flow cell is not cost-effective.
5. Record the pore count in your laboratory notebook for later reference.

25.6.2 Flow Cell Priming

Priming replaces storage buffer with sequencing-compatible buffer and removes air bubbles from the sensor array.

Protocol: Flow Cell Priming (R10.4.1)**Prepare the priming mix:**

1. Thaw one tube of Flow Cell Flush (FCF) at room temperature. Briefly spin down.
2. Add 30 μL of Flow Cell Tether (FCT) to the FCF tube. Mix by gentle inversion. This is the Priming Mix.

Prime the flow cell:

3. Open the priming port on the flow cell by sliding the cover clockwise. **Do not remove the SpotON port cover yet.**
4. Set a P1000 pipette to 200 μL . Insert the tip into the priming port. Turn the pipette dial to draw back a small amount of buffer (~20–30 μL) until you see a small amount of buffer entering the tip. This removes any air bubble from the sensor array.
5. Load 800 μL of Priming Mix via the priming port. Allow to flow through by gravity (do not force). Wait 5 minutes.
6. Open the SpotON port cover.
7. Load another 200 μL of Priming Mix via the priming port.

25.6.3 Library Loading

Protocol: Library Loading

Prepare the loading mix:

1. In a clean 1.5 mL tube, combine:
 - 37 μ L Sequencing Buffer (SB)
 - 25.5 μ L Loading Beads (LB)—**mix thoroughly by pipetting immediately before use**, as beads settle rapidly
 - 12 μ L prepared library (from SQK-LSK114 or SQK-RBK114.96)

Total volume: 75 μ L.

2. Mix by gentle pipetting. Do not vortex.

Load the library:

3. Using a P200 pipette, add the entire 75 μ L **drop-by-drop** through the SpotON port. Ensure each drop is drawn into the flow cell before adding the next (you will see each drop wick into the port).
4. Close the SpotON port cover.
5. Close the priming port cover.
6. Close the MinION/GridION lid.

Loading Beads Must Be Mixed

Loading Beads settle within seconds. Pipette-mix thoroughly immediately before adding to the loading mix and load promptly. Settled beads cause very low pore occupancy and minimal output.

25.6.4 MinKNOW Run Setup and Monitoring

Protocol: Starting a Sequencing Run in MinKNOW

1. In MinKNOW, select “Start Sequencing.”
2. Configure run parameters:
 - **Kit:** Select the kit used (SQK-RBK114.96 or SQK-LSK114).
 - **Basecalling:** Select “Super accuracy” (SUP) model for highest accuracy (>Q20 modal). Alternatively, use “Fast” basecalling if real-time analysis is needed and accuracy requirements are lower.
 - **Barcoding:** If using rapid barcoding, enable barcode detection and select the appropriate barcode kit.
 - **Run length:** Default 72 hours. For rapid experiments or flow cell reuse, set to 24 hours.
 - **Output format:** FASTQ (for immediate analysis) or POD5 (raw signal, for re-basecalling).
3. Click “Start” to begin sequencing.

Monitoring parameters to check during the run:

- **Pore occupancy:** Target >50%. If below 20%, the library loading was unsuccessful—check loading bead mixing.
- **Strand rate:** Number of reads translating per minute. Should be steady or slowly declining.
- **Read length distribution:** Should match expectations for your library type (5–10 kb N50 for rapid, >10 kb for ligation).
- **Quality score distribution:** Modal quality should be >Q15 (fast basecalling) or >Q20 (SUP basecalling).
- **Active pore count:** Will decline over 72 hours. A rapid drop in the first hour may indicate a problem (contaminated library, air bubbles, or membrane damage).
- **Temperature:** The device should maintain 35°C. Overheating causes increased noise.

25.6.5 Nuclease Flush and Reload

If a run completes before 72 hours and active pores remain, flush and reload a new library to maximise flow cell utilisation.

Protocol: Nuclease Flush and Reload

1. In MinKNOW, stop the current sequencing run.
2. Prepare wash solution: add 380 μL nuclease-free water to one tube of DNase I from the Flow Cell Wash Kit (EXP-WSH004). Mix by inversion.
3. Open the priming port and SpotON port covers on the flow cell.
4. Using a P1000 pipette, slowly draw off all the library from the priming port. Discard.
5. Load 400 μL of the DNase I wash solution via the priming port.
6. Close both port covers. Leave at room temperature for 30 minutes. The DNase will digest remaining DNA and free the pores.
7. After 30 minutes, open both port covers and remove the wash solution via the priming port.
8. Re-prime the flow cell with fresh Priming Mix (as in the priming protocol, [Section 25.6.2](#)).
9. Perform a flow cell check to assess remaining active pore count.
10. If >400 pores remain, load a new library via the SpotON port.
11. Start a new sequencing run in MinKNOW.

Maximising Flow Cell Value

A single R10.4.1 flow cell can be reused 2–3 times with nuclease flushes. Use the first load for the most important library (best pore count). Each reload typically yields 50–70% of the previous run's output. Store partially used flow cells at 4°C for up to one week.

25.7 Troubleshooting

Keep a Sequencing Log

Record every run: date, operator, flow cell ID, pore count, kit, DNA input, run time, total reads, N50, and anomalies. This record is invaluable for diagnosing problems and tracking performance.

25.8 Sample Collection Best Practices

The best laboratory protocols cannot rescue a poorly collected sample. This section covers collection, preservation, storage, and documentation best practices.

25.8.1 Field Preservation Methods

Different sample types require different preservation strategies to maintain DNA integrity:

Cold chain (ice/dry ice)

The default method. Wet ice (4°C) for transport <6 hours; dry ice (−78°C) for overnight shipping. Transfer to −20°C or −80°C upon arrival.

Chemical preservation

For field situations without cold chain: **95% ethanol** ($\geq 10\times$ volume, stable months at RT); **RNAlater** (RNA preservation, tissue <0.5 cm in $5\times$ volume; can interfere with DNA kits); **DESS buffer** (20% DMSO, 0.25 M EDTA, NaCl-saturated, pH 8.0;

Table 25.10: Comprehensive troubleshooting guide for nanopore sequencing workflows.

Problem	Likely Cause	Solution
Very low yield after extraction	Insufficient lysis or input material	Increase incubation time with Proteinase K; increase sample input volume; check expiry of lysis reagents
A260/A280 < 1.7	Protein or phenol contamination	Re-precipitate DNA; run through a cleanup column; extend Proteinase K digestion
A260/A230 < 1.5	Guanidinium, ethanol, or chaotropic salt carry-over	Additional ethanol wash steps; column cleanup; ensure full 3-min drying spin
Degraded DNA (smear on gel)	Nuclease activity, improper storage, or excessive handling	Use fresh reagents; store DNA at -20°C ; minimise freeze-thaw cycles; add EDTA to storage buffer
Low pore occupancy (<20%)	Insufficient library, loading bead settling, or air bubbles	Re-quantify library; ensure beads are mixed immediately before loading; remove air from priming port
Pore occupancy drops rapidly in first hour	Library quality issue or membrane contamination	Check for contaminants (detergent, organic solvents); re-prepare library from clean DNA
Short reads (<1 kb N50) with ligation kit	DNA shearing during preparation	Use wide-bore pipette tips; do not vortex; handle tubes gently; avoid freeze-thaw of HMW DNA
No reads after starting run	Adapter not attached or motor protein denatured	Verify adapter ligation with Qubit (concentration should not drop >50%); check that SFB (not ethanol) was used for post-ligation wash
High adapter dimer reads	Excessive adapter in ligation reaction or insufficient cleanup	Lower adapter input; use a stricter AMPure bead ratio (0.4 $\times$); check for DNA input below minimum
Low quality scores (<Q10 modal)	Old flow cell, basecalling model mismatch, or temperature issue	Use SUP basecalling model; check flow cell manufacture date; ensure device is not overheating
Uneven barcode distribution	Unequal DNA input or PCR amplification bias	Quantify all samples by Qubit and normalise to equimolar before pooling; reduce PCR cycle number
High “unclassified” barcode reads	Poor barcode ligation or chimeric reads	Increase barcode stringency in MinKNOW; check DNA input was not degraded
No amplicon band on gel	Primer failure, inhibitors, or template issue	Check primer sequences and concentrations; run positive control; dilute template 1:10 to reduce inhibitors
Multiple amplicon bands	Non-specific priming or contamination	Increase annealing temperature (+2–5 $^{\circ}\text{C}$); reduce primer concentration; use fresh aliquot of primers
MinKNOW software crashes	Insufficient disk space or RAM	Ensure >100 GB free disk space; close other applications; restart software
Temperature warning in MinKNOW	Blocked ventilation or ambient temperature too high	Ensure device ventilation is unobstructed; reduce ambient temperature; use external cooling fan
Flow cell check shows 0 pores	Flow cell not inserted properly or damaged	Remove and reinsert flow cell firmly; check connector pins; try a different flow cell

versatile); **Longmire's buffer** (100 mM Tris, 100 mM EDTA, 10 mM NaCl, 0.5% SDS, pH 8.0; eDNA filters and wildlife tissue).

Desiccation

Silica gel packets in sealed bags for plant leaf tissue. DNA stable for years when dry.

FTA cards

Cellulose cards lyse cells on contact; DNA stable at RT for years. Punch a disc directly into the [PCR](#) reaction.

Saliva kits

DNA Genotek OG-500/OG-600 tubes preserve DNA at RT for >12 months. See [Section 25.2.1](#).

25.8.2 Storage Temperature Requirements

Table 25.12: Recommended storage conditions and maximum holding times for common sample types.

Sample Type	Temperature	Max. Hold Time	Notes
Whole blood (EDTA)	4°C	7 days	DNA quality declines after 3 days; freeze for longer storage
Whole blood (EDTA)	−20°C	5 years	Avoid repeated freeze-thaw
Saliva (stabilised)	Room temp	12 months	DNA Genotek OG-500 buffer
Fresh tissue	4°C	24 hours	Extract ASAP or snap-freeze
Fresh tissue	−80°C	10+ years	Flash-freeze in liquid nitrogen before transfer
Tissue in 95% ethanol	Room temp	6 months	Use $\geq 10\times$ volume ethanol
Tissue in RNAlater	−20°C	Indefinite	Best for RNA; acceptable for DNA
Stool (fresh)	4°C	24 hours	Use OMNIgene Gut kit for room temp stability
eDNA filter (dry)	−20°C	6 months	Add preservation buffer for longer storage
eDNA filter (in buffer)	−20°C	12 months	Longmire's or ATL buffer
Extracted DNA	−20°C	2+ years	TE buffer or nuclease-free water; avoid freeze-thaw
Extracted DNA	−80°C	10+ years	Long-term archival storage
Prepared library	4°C	1 week	Load promptly for best results
FFPE blocks	Room temp	10+ years	DNA is degraded; expect 200–500 bp fragments
Plant leaf (silica-dried)	Room temp	5+ years	Use indicating silica gel; keep dry

25.8.3 Chain of Custody Documentation

For clinical, forensic, or regulatory applications, a formal chain of custody must be maintained. Even for research, robust sample tracking prevents costly errors. Assign a **unique sample ID** at collection using a systematic scheme (e.g., “EDNA-LK01-20260315-W01”). Label tubes with both human-readable text and machine-readable identifiers (barcode/QR code labels; use cryogenic labels for -80°C storage). Maintain a **chain of custody log** recording collector, date, location (GPS coordinates), preservation method, and every transfer between personnel. Photograph collection sites and sample tubes. Keep both physical (lab notebook) and digital (spreadsheet, LIMS) records.

25.8.4 Metadata Recording

The biological value of sequencing data depends critically on its associated metadata. Record the following at minimum:

Table 25.13: Essential metadata fields for sequencing sample documentation.

Field	Format	Description
Sample ID	Text	Unique identifier linking to all downstream data
Collection date	YYYY-MM-DD	ISO 8601 format
Collection time	HH:MM (24h)	Local time with timezone noted
Collector	Text	Name of person who collected the sample
Location	Decimal degrees	Latitude, longitude (GPS); include altitude if relevant
Habitat/site	Text	Description of collection environment
Sample type	Controlled vocab	e.g., “saliva”, “water-eDNA”, “leaf-tissue”, “soil”
Organism	Binomial	<i>Homo sapiens</i> , “environmental”, or target species
Preservation	Controlled vocab	e.g., “ethanol-95pct”, “frozen-80C”, “OG500-stabiliser”
Storage temp	$^{\circ}\text{C}$	Temperature at which sample is stored
Extraction date	YYYY-MM-DD	Date DNA was extracted
Extraction method	Text	Kit name, catalogue number, and any modifications
DNA conc. (Qubit)	$\text{ng}/\mu\text{L}$	Qubit fluorometric measurement
A260/A280	Ratio	NanoDrop purity ratio
A260/A230	Ratio	NanoDrop purity ratio
Notes	Free text	Any anomalies, deviations from protocol, or observations

Use Standardised Metadata Standards

Align metadata with GSC MIxS (Minimum Information about any (x) Sequence) checklists and [SRA](#) submission requirements from the outset—recording these at collection saves hours of retroactive data entry. See ?? for details on data standards.

Metadata Cannot Be Reconstructed

Metadata lost at collection is gone forever. Make metadata recording a non-negotiable part of every field protocol. Pre-print field data sheets or use mobile apps (KoboToolbox, ODK Collect) to ensure completeness.

25.9 Chapter Summary

Key Takeaways

1. **Safety is non-negotiable.** Understand your BSL requirements, PPE needs, and chemical hazards before starting any protocol.
2. **DNA quality determines sequencing success.** Use the [QC](#) decision tree ([Fig. 25.1](#)) to verify every sample before committing it to an expensive flow cell.
3. **Choose the right kit for the job.** Rapid Barcoding (SQK-RBK114.96) for speed and high multiplexing; Ligation (SQK-LSK114) for long reads and maximum yield.
4. **Never use ethanol washes after ligation.** Use Short Fragment Buffer (SFB) to preserve motor protein integrity.
5. **Mix loading beads immediately before use.** Settled beads result in zero pore occupancy.
6. **Controls are essential.** Every amplicon experiment requires positive controls (mock community) and negative controls (extraction blank + [PCR](#) NTC).
7. **Metadata matters as much as sequence data.** Record comprehensive metadata at the point of collection—it cannot be reconstructed later.
8. **Keep a sequencing log.** Track every run's parameters and performance for long-term quality management.

Chapter 26

DIY Analysis: From Raw Reads to Results

Key Concept

This chapter is your hands-on laboratory notebook for long-read nanopore data analysis. We walk through every step—from configuring your computer and basecalling raw signal data to assembling bacterial genomes, identifying species from metagenomic samples, and calling variants in human genomes. Every command is given in full, every parameter is explained, and complete workflow implementations in both **Snakemake** and **Nextflow** are provided so that you can reproduce each analysis end-to-end. Whether you are a bench scientist running your first MinION experiment or a bioinformatician building production pipelines, this chapter gives you the tools to go from raw reads to publication-ready results on your own hardware.

Throughout this chapter we assume you are working with Oxford Nanopore Technologies (ONT) data, although many of the downstream tools (assembly, annotation, variant calling) apply equally to PacBio long reads. Where commands differ between platforms, we note the alternatives. All tools are open-source and freely available through `conda/mamba`.

26.1 Setting Up Your Environment

Before you can analyse a single read, you need a properly configured computing environment. This section covers hardware requirements, software installation, and test datasets you can use to validate your setup.

26.1.1 Hardware Requirements

The computational demands of long-read analysis vary enormously depending on the task. [Table 26.1](#) summarises minimum and recommended hardware for the analyses covered in this chapter.

Storage Fills Up Fast

A single MinION flow cell produces 10–50 GB of POD5 raw signal data. After basecalling, the resulting **BAM** file can be 5–20 GB. During assembly, temporary files can consume 3–5× the input size. Monitor disk usage with `df -h` and plan for at least 3× the raw data size in free space. NVMe SSDs are strongly recommended—assembly tools like **Flye** perform heavy random I/O that is painfully slow on spinning hard drives.

Table 26.1: Hardware requirements for nanopore data analysis.

Component	Minimum	Recommended
CPU	4 cores (Intel i5 / AMD Ryzen 5)	8+ cores (Intel i7/i9 / AMD Ryzen 7/9)
RAM	8 GB	32 GB (64 GB for human genomes)
Storage	256 GB SSD (SATA)	1 TB+ NVMe SSD
GPU	None (CPU basecalling only)	NVIDIA RTX 3060+ (8 GB VRAM) for Dorado
OS	Ubuntu 20.04+ / macOS 12+	Ubuntu 22.04 LTS
Network	Broadband for tool downloads	High-speed for SRA downloads

26.1.2 Installing Software with Conda

We use `mamba` (a fast drop-in replacement for `conda`) and the `bioconda` channel to install all tools into isolated environments, preventing version conflicts.

Listing 26.1: Installing Miniforge and creating the analysis environment.

```

1 # Step 1: Install Miniforge (includes mamba)
2 curl -L -O https://github.com/conda-forge/miniforge/releases/\
3 latest/download/Miniforge3-Linux-x86_64.sh
4 bash Miniforge3-Linux-x86_64.sh
5 # Restart your shell or run: source ~/.bashrc
6
7 # Step 2: Create the nanopore analysis environment
8 mamba create -n nanopore -c conda-forge -c bioconda \
9     dorado=0.8.0 \
10     nanoplot=1.42.0 \
11     pycoqc=2.5.2 \
12     fastqc=0.12.1 \
13     multiqc=1.21 \
14     chopper=0.9.0 \
15     flye=2.9.5 \
16     medaka=2.0.1 \
17     quast=5.2.0 \
18     busco=5.7.1 \
19     bakta=1.9.4 \
20     kraken2=2.1.3 \
21     bracken=2.9 \
22     krona=2.8.1 \
23     minimap2=2.28 \
24     samtools=1.21 \
25     bcftools=1.21 \
26     snakemake=8.25.0 \
27     nextflow=24.04.0
28
29 # Step 3: Activate the environment
30 mamba activate nanopore
31
32 # Step 4: Verify key tools
33 dorado --version
34 flye --version
35 minimap2 --version
36 samtools --version | head -1
37
38 # Step 5: Export for reproducibility

```



```
39 mamba env export > environment.yml
```

Separate Environments for Conflicting Tools

Some tools have conflicting dependencies. If `mamba create` fails due to unsolvable conflicts, split tools into separate environments. For example, `Clair3` often requires its own environment. Use `mamba create -n clair3 -c bioconda clair3` separately. Snakemake and Nextflow can activate different environments per rule/process automatically.

26.1.3 Test Datasets

Practise with real-world datasets before analysing your own data. Table 26.2 lists publicly available nanopore datasets suitable for the walkthroughs in this chapter.

Table 26.2: Recommended test datasets from the [SRA](#) and other public repositories.

Walkthrough	Accession	Description	Size
Bacterial assembly	SRR28370649	<i>E. coli</i> K-12 MG1655, R10.4.1, SUP basecall	~1.5 GB
Bacterial assembly	ERR3813594	<i>E. coli</i> K-12, R9.4.1	~2.0 GB
Metagenomics	SRR27498888	ZymoBIOMICS mock community, R10.4.1	~3.0 GB
Human variant calling	HG002 (GIAB)	Genome in a Bottle, ONT R10.4.1, chr20 subset	~5.0 GB

Listing 26.2: Downloading test data from the SRA.

```
1 # Install SRA Toolkit if not already present
2 mamba install -c bioconda sra-tools
3
4 # Download E. coli test dataset
5 prefetch SRR28370649
6 fasterq-dump --threads 4 SRR28370649 -O data/ecoli/
7
8 # For BAM input (preferred for ONT data)
9 # Many ONT datasets are deposited as uBAM; check the SRA entry
10 wget -P data/ecoli/ \
11     https://sra-pub-src-1.s3.amazonaws.com/SRR28370649/reads.bam
12
13 # Verify download integrity
14 samtools quickcheck data/ecoli/reads.bam && echo "OK" || echo "CORRUPT"
```

26.2 Basecalling with Dorado

Basecalling is the process of converting raw electrical signal data (POD5 files) into nucleotide sequences. Dorado is Oxford Nanopore's current production basecaller, replacing the older Guppy. It supports GPU acceleration and produces output directly in [BAM](#) format.

26.2.1 Basecalling Models

Dorado offers three model tiers that trade speed for accuracy. Table 26.3 compares them.

Table 26.3: Dorado basecalling model comparison (R10.4.1 chemistry, v4.3.0 models).

Model	Simplex Accuracy	Speed (GPU)	Use Case	GPU Required?
fast	~Q15 (96.8%)	~5×10 ⁶ bases/s	Real-time monitoring, previews	Recommended
hac	~Q20 (99.0%)	~2×10 ⁶ bases/s	Standard analyses, metagenomics	Recommended
sup	~Q23 (99.5%)	~0.5×10 ⁶ bases/s	Variant calling, assembly, publication-quality	Required

Which Model Should I Use?

For bacterial genome assembly and metagenomics, **hac** (high-accuracy) is usually sufficient and 4× faster than **sup**. For variant calling in human or complex genomes where every tenth of a percent of accuracy matters, always use **sup** (super-accuracy). Never use **fast** for any downstream analysis that requires accurate consensus—it is intended only for real-time monitoring during sequencing.

26.2.2 Running Dorado

Listing 26.3: Basecalling with Dorado.

```

1 # List available models
2 dorado download --list
3
4 # Download the SUP model for R10.4.1 chemistry
5 dorado download --model dna_r10.4.1_e8.2_400bps_sup@v4.3.0
6
7 # Basecall POD5 files to unaligned BAM
8 dorado basecaller \
9   dna_r10.4.1_e8.2_400bps_sup@v4.3.0 \
10  pod5_dir/ \
11  --device cuda:0 \
12  --emit-moves \
13  > basecalled.bam
14
15 # Check the output
16 samtools view -c basecalled.bam # total reads
17 samtools stats basecalled.bam | grep "average length"
18
19 # If you need FASTQ output instead (some tools require it)
20 samtools fastq basecalled.bam | gzip > basecalled.fastq.gz

```

GPU vs CPU Basecalling

CPU basecalling with the **sup** model is extremely slow—a single MinION flow cell can take **days to weeks** on a modern CPU. With an NVIDIA RTX 4090, the same data takes approximately 2–4 hours. If you do not have a GPU, consider: (1) using the **fast** or **hac** model, (2) using a cloud GPU instance (see [Section 26.9](#)), or (3) using the basecalled data already produced by MinKNOW during the sequencing run. The `-device cpu` flag forces CPU mode: `dorado basecaller model pod5/ -device cpu > out.bam`.

26.2.3 Understanding BAM Output

Dorado produces **unaligned BAM** files (uBAM). Unlike traditional aligned BAM files, these contain sequence and quality scores but no alignment information. The key advantage of uBAM over **FASTQ** is that it preserves rich metadata:

- **Move table:** Signal-to-base correspondence for re-basecalling or modified-base calling.
- **Read group information:** Flow cell ID, run ID, basecall model version.
- **Base modification tags:** If modified-base calling was enabled (`-modified-bases 5mCG_5hmCG`).
- **Duplex information:** If duplex basecalling was performed.

Listing 26.4: Inspecting uBAM metadata.

```

1 # View the header (contains read group and program info)
2 samtools view -H basecalled.bam | head -20
3
4 # View the first read with all tags
5 samtools view basecalled.bam | head -1 | tr '\t' '\n'
6
7 # Summary statistics
8 samtools stats basecalled.bam > basecall_stats.txt
9 grep ^SN basecall_stats.txt | head -15

```

26.3 Quality Control

Quality control (QC) is the critical checkpoint between raw data and analysis. Poor-quality data will produce poor-quality results regardless of how sophisticated your pipeline is. This section covers **QC** tools for nanopore data and provides a decision framework for identifying problems.

26.3.1 NanoPlot: Nanopore-Specific QC

NanoPlot generates publication-quality plots and summary statistics tailored to long-read data.

Listing 26.5: Running NanoPlot on basecalled data.

```

1 # From BAM file (recommended)
2 NanoPlot --bam basecalled.bam \
3   --outdir qc/nanoplot \
4   --threads 8 \
5   --loglength \
6   --plots dot kde \
7   --N50 \
8   --title "E. coli MinION Run"

```

```

9
10 # From FASTQ file
11 NanoPlot --fastq basecalled.fastq.gz \
12     --outdir qc/nanoplot_fq \
13     --threads 8
14
15 # From sequencing_summary.txt (fastest, but less detail)
16 NanoPlot --summary sequencing_summary.txt \
17     --outdir qc/nanoplot_summary \
18     --threads 4

```

Key metrics to examine in the NanoPlot output:

- **Mean/median read length:** Expect 5–20 kb for standard ligation libraries, 200–500 bp for rapid barcoding.
- **N50 read length:** The length at which 50% of total bases are in reads of this length or longer. Higher is better; >10 kb is good for assembly.
- **Mean/median quality score:** Q15+ for hac, Q20+ for sup models.
- **Total bases:** Divide by genome size to estimate coverage (e.g., 500 Mb / 4.6 Mb $\approx 109\times$ for *E. coli*).
- **Read length distribution:** Look for a unimodal distribution. A spike at very short lengths may indicate DNA degradation or adapter artifacts.

26.3.2 pycoQC: Sequencing Run QC

pycoQC uses the `sequencing_summary.txt` file generated by MinKNOW to provide an interactive HTML report focused on the sequencing run itself (rather than the reads).

Listing 26.6: Running pycoQC for run-level quality control.

```

1 # Generate interactive HTML report
2 pycoQC \
3     --summary_file sequencing_summary.txt \
4     --html_outfile qc/pycoqc_report.html \
5     --min_pass_qual 8
6
7 # Generate JSON for programmatic parsing
8 pycoQC \
9     --summary_file sequencing_summary.txt \
10    --json_outfile qc/pycoqc_report.json

```

pycoQC reports include pore activity over time, read length vs quality scatter plots, channel activity heatmaps, and throughput curves. These are particularly useful for diagnosing flow cell problems: a sudden drop in active pores may indicate a blockage or temperature issue, while a bimodal quality distribution could suggest a mixed library.

26.3.3 FastQC and MultiQC

Although designed for short reads, **FastQC** provides useful per-base quality profiles and adapter content checks for any **FASTQ** data. **MultiQC** aggregates reports from multiple tools into a single interactive dashboard.

Listing 26.7: Running FastQC and MultiQC.

```

1 # Run FastQC on basecalled reads
2 fastqc basecalled.fastq.gz \
3     --outdir qc/fastqc \
4     --threads 4
5

```

```

6 # Aggregate all QC results with MultiQC
7 multiqc qc/ \
8   --outdir qc/multiqc \
9   --title "Nanopore Run QC Summary" \
10  --force

```

26.3.4 QC Decision Table

Table 26.4 provides a systematic framework for interpreting QC results and deciding on corrective actions.

Table 26.4: QC metrics, acceptable ranges, and recommended actions for nanopore data.

Metric	Acceptable Range	Action if Outside Range
Mean read quality	$\geq Q10$ (hac), $\geq Q18$ (sup)	Re-basecall with correct model; check chemistry version
Median read length	>1 kb (ligation library)	Check DNA extraction quality; fragment analysis may show degradation
N50 read length	>5 kb (ligation)	Optimise DNA extraction (avoid vortexing); use HMW extraction protocol
Total yield	>1 Gb (MinION)	Check pore count; may need a new flow cell or longer run
Active pores at start	>800 (MinION)	Flow cell quality issue; contact ONT support
Pore occupancy	>50%	Adjust library loading amount (too little DNA)
Read length distribution	Unimodal, no spike at <200 bp	Filter short reads with chopper ; adapter trimming issue
Adapter content	<5% of reads with residual adapters	Re-run adapter trimming; check library prep protocol
Chimeric reads	<2% estimated chimeras	Filter with quality/length cut-offs; avoid rapid library prep for assembly
Coverage uniformity	<3 $\times$ coefficient of variation across genome	Increase total coverage; check for GC bias

When to Stop Sequencing

For bacterial genome assembly, you typically need 50–100 $\times$ coverage. For a 5 Mb genome, that is 250–500 Mb of data. Check your coverage during the run: if NanoPlot shows you have 300 Mb with a good N50, you can safely stop the run and wash the flow cell for reuse. For human variant calling, aim for 30 $\times$ minimum ($\sim$ 90 Gb).

26.4 Walkthrough 1: Bacterial Genome Assembly

In this walkthrough, we assemble an *Escherichia coli* genome from nanopore reads. The pipeline proceeds through six steps: read filtering, assembly, polishing, quality assessment, completeness checking, and annotation.

26.4.1 Step 1: Read Filtering with Chopper

chopper (part of the **Nanopack** suite) filters reads by quality and length. Removing very short and low-quality reads improves assembly contiguity and accuracy.

Listing 26.8: Filtering reads with **chopper**.

```

1 # Filter reads: minimum quality Q10, minimum length 1000 bp
2 # Input from BAM, output as FASTQ
3 samtools fastq basecalled.bam | \
4   chopper \
5     --quality 10 \
6     --minlength 1000 \
7     --threads 8 | \
8   gzip > filtered_reads.fastq.gz
9
10 # Check how many reads survived filtering
11 echo "Before filtering:"
12 samtools view -c basecalled.bam
13 echo "After filtering:"
14 zcat filtered_reads.fastq.gz | awk 'NR%4==1' | wc -l

```

Choosing Filter Thresholds

For bacterial genome assembly with SUP-basecalled data:

- **Quality ≥ 10 :** Removes the lowest-quality 5–10% of reads. For hac-basecalled data, use Q8.
- **Length ≥ 1000 bp:** Removes very short fragments that contribute noise but little contiguity.
- Do *not* over-filter: removing too many reads can reduce coverage below what the assembler needs. Aim to retain $>50\times$ coverage after filtering.

26.4.2 Step 2: Assembly with Flye

Flye is a de novo assembler designed for noisy long reads. It uses a repeat graph approach that handles the higher error rates of nanopore data gracefully.

Listing 26.9: De novo assembly with **Flye**.

```

1 # Assemble with Flye
2 flye \
3   --nano-hq filtered_reads.fastq.gz \
4   --out-dir assembly/flye \
5   --threads 8 \
6   --genome-size 4.6m \
7   --iterations 1
8
9 # Key output files:
10 # assembly/flye/assembly.fasta      - the assembled contigs
11 # assembly/flye/assembly_info.txt   - contig statistics
12 # assembly/flye/assembly_graph.gfa  - assembly graph (view in Bandage)
13
14 # Quick check: how many contigs?
15 grep -c ">" assembly/flye/assembly.fasta

```

Key **Flye** parameters:

-nano-hq Use this for Q20+ (SUP or duplex) basecalled data. Use **-nano-raw** for Q10 data (older chemistry or hac basecalling).

- genome-size** An approximate expected genome size. This helps Flye calibrate its coverage-based heuristics. Use 4.6m for *E. coli*, 5m for a typical bacterium, 3g for human.
- iterations** Number of polishing rounds internal to Flye (default 1). One iteration is usually sufficient when using high-accuracy reads and planning to polish with Medaka.

26.4.3 Step 3: Polishing with Medaka

Medaka uses a neural network to correct remaining errors in the assembly consensus by re-examining the raw read alignments. This step typically improves accuracy from ~Q30–Q40 to ~Q50+ for SUP-basecalled data.

Listing 26.10: Polishing the assembly with Medaka.

```

1 # Polish with Medaka
2 medaka_polisher \
3   --threads 8 \
4   --model r1041_e82_400bps_sup_v4.3.0 \
5   --bacteria \
6   -i filtered_reads.fastq.gz \
7   -d assembly/flye/assembly.fasta \
8   -o assembly/medaka
9
10 # The polished assembly is:
11 # assembly/medaka/consensus.fasta
12
13 # Alternative: use medaka_polisher for more control
14 # medaka_polisher aligns reads, runs inference, and stitches results

```

Medaka Model Must Match Basecalling Model

Medaka's neural network was trained on data basecalled with a specific model. Using the wrong Medaka model can *decrease* assembly quality. Always match the Medaka model to your basecalling chemistry and model. Check available models with: `medaka -help` and look for models matching your flowcell (e.g., `r1041_e82_400bps_sup_v4.3.0` for R10.4.1 SUP data). If in doubt, run `medaka smolecule -help` or check the Medaka GitHub repository.

26.4.4 Step 4: Assembly Assessment with QUAST

QUAST provides comprehensive assembly quality statistics, optionally comparing against a reference genome.

Listing 26.11: Assessing assembly quality with QUAST.

```

1 # Download E. coli K-12 reference for comparison
2 wget -O ref/ecoli_k12.fasta.gz \
3   "https://ftp.ncbi.nlm.nih.gov/genomes/all/GCF/000/005/845/\
4   GCF_000005845.2_ASM584v2/GCF_000005845.2_ASM584v2_genomic.fna.gz"
5 gunzip ref/ecoli_k12.fasta.gz
6
7 # Run QUAST with reference
8 quast \
9   assembly/medaka/consensus.fasta \
10  -r ref/ecoli_k12.fasta \
11  -o qc/quast \
12  --threads 4 \
13  --min-contig 500 \

```

```

14  --labels "Flye+Medaka"
15
16  # View the key metrics
17  cat qc/quast/report.txt

```

Key QUAST metrics to examine:

Number of contigs Ideally 1 for a bacterial genome (single circular chromosome).

Total length Should be close to the expected genome size (4,641,652 bp for *E. coli* K-12 MG1655).

N50 For a complete bacterial assembly, N50 equals the genome size.

Genome fraction (%) Percentage of the reference covered by the assembly. Should be >99%.

Misassemblies Should be 0 or very low for a well-assembled bacterial genome.

Mismatches per 100 kb Indicates per-base accuracy. <5 is good; <1 is excellent.

26.4.5 Step 5: Completeness with BUSCO

BUSCO (Benchmarking Universal Single-Copy Orthologs) assesses assembly completeness by searching for a set of genes expected to be present in all organisms of a given lineage.

Listing 26.12: Running BUSCO for completeness assessment.

```

1  # Run BUSCO with the enterobacterales lineage
2  busco \
3    -i assembly/medaka/consensus.fasta \
4    -o busco_results \
5    -m genome \
6    -l enterobacterales_odb10 \
7    --cpu 8 \
8    --out_path qc/
9
10 # For a general bacterial assembly (if lineage is unknown)
11 busco \
12   -i assembly/medaka/consensus.fasta \
13   -o busco_bacteria \
14   -m genome \
15   -l bacteria_odb10 \
16   --cpu 8 \
17   --out_path qc/
18
19 # View the short summary
20 cat qc/busco_results/short_summary.specific.\
21 enterobacterales_odb10.busco_results.txt

```

A good bacterial assembly should show >98% Complete BUSCOs, with <1% Missing. If completeness is low, the assembly may be fragmented or contaminated.

26.4.6 Step 6: Annotation with Bakta

Bakta is a rapid, comprehensive bacterial genome annotation tool that identifies coding sequences, rRNAs, tRNAs, ncRNAs, CRISPR arrays, and more.

Listing 26.13: Annotating the assembly with Bakta.

```

1  # Download the Bakta database (run once, ~1.5 GB)
2  bakta_db download --output db/bakta --type light
3
4  # Annotate the polished assembly

```



```

5 bakta \
6   assembly/medaka/consensus.fasta \
7   --db db/bakta/db-light \
8   --output annotation/bakta \
9   --prefix ecoli_assembly \
10  --threads 8 \
11  --genus Escherichia \
12  --species coli \
13  --strain "K-12 MG1655" \
14  --complete # flag indicating closed genome
15
16 # Key output files:
17 ls annotation/bakta/
18 # ecoli_assembly.gff3 - gene annotations (GFF3 format)
19 # ecoli_assembly.gbff - GenBank flat file
20 # ecoli_assembly.faa - protein FASTA
21 # ecoli_assembly.ffn - nucleotide FASTA of features
22 # ecoli_assembly.tsv - tab-separated feature table
23 # ecoli_assembly.embl - EMBL format

```

26.4.7 Expected Results

Table 26.5 shows expected results for common bacterial genome assemblies with 50–100× nanopore coverage using SUP basecalling.

Table 26.5: Expected results for bacterial genome assembly from nanopore data (SUP basecalling, Flye + Medaka).

Organism	Size	Contigs	Accuracy	BUSCO	Genes
<i>E. coli</i> K-12	4.64 Mb	1	>99.9% (Q30+)	>99%	~4,300
<i>S. aureus</i>	2.81 Mb	1	>99.9%	>99%	~2,600
<i>P. aeruginosa</i>	6.26 Mb	1	>99.8%	>98%	~5,700
<i>M. tuberculosis</i>	4.41 Mb	1	>99.5%	>97%	~4,000
<i>K. pneumoniae</i>	5.33 Mb	1–3*	>99.8%	>98%	~5,100

* Additional contigs from plasmids.

26.4.8 Complete Snakemake Workflow

Listing 26.14: Snakemake workflow for bacterial genome assembly (Snakefile).

```

1 # Snakefile -- Bacterial Genome Assembly Pipeline (Nanopore)
2 configfile: "config.yaml"
3
4 SAMPLE      = config["sample"]
5 RAW_BAM     = config["raw_bam"]
6 GENOME_SIZE = config["genome_size"] # e.g., "4.6m"
7 MEDAKA_MODEL = config["medaka_model"] # e.g., "r1041_e82_400bps_sup_v4.3.0"
8 BUSCO_LINEAGE = config["busco_lineage"] # e.g., "enterobacterales_odb10"
9 BAKTA_DB     = config["bakta_db"]
10 REFERENCE   = config.get("reference", "") # optional, for QUAST
11
12 rule all:
13     input:
14         f"qc/nanoplot/{SAMPLE}/NanoPlot-report.html",

```

```

15     f"assembly/medaka/{SAMPLE}/consensus.fasta",
16     f"qc/quast/{SAMPLE}/report.html",
17     f"qc/busco/{SAMPLE}/short_summary.txt",
18     f"annotation/{SAMPLE}/{SAMPLE}.gff3"
19
20 rule bam_to_fastq:
21     input:
22         RAW_BAM
23     output:
24         f"data/{SAMPLE}/basecalled.fastq.gz"
25     threads: 4
26     shell:
27         "samtools fastq -@ {threads} {input} | gzip > {output}"
28
29 rule nanoplot:
30     input:
31         f"data/{SAMPLE}/basecalled.fastq.gz"
32     output:
33         f"qc/nanoplot/{SAMPLE}/NanoPlot-report.html"
34     params:
35         outdir=f"qc/nanoplot/{SAMPLE}"
36     threads: 4
37     shell:
38         """
39         NanoPlot --fastq {input} \
40             --outdir {params.outdir} \
41             --threads {threads} --loglength --N50
42         """
43
44 rule chopper_filter:
45     input:
46         f"data/{SAMPLE}/basecalled.fastq.gz"
47     output:
48         f"data/{SAMPLE}/filtered.fastq.gz"
49     threads: 4
50     shell:
51         """
52         zcat {input} | chopper \
53             --quality 10 --minlength 1000 \
54             --threads {threads} | gzip > {output}
55         """
56
57 rule flye_assembly:
58     input:
59         f"data/{SAMPLE}/filtered.fastq.gz"
60     output:
61         fasta=f"assembly/flye/{SAMPLE}/assembly.fasta",
62         info=f"assembly/flye/{SAMPLE}/assembly_info.txt"
63     params:
64         outdir=f"assembly/flye/{SAMPLE}",
65         genome_size=GENOME_SIZE
66     threads: 8
67     shell:
68         """
69         flye --nano-hq {input} \
70             --out-dir {params.outdir} \
71             --threads {threads} \
72             --genome-size {params.genome_size}

```

```

73     """
74
75 rule medaka_polish:
76     input:
77         reads=f"data/{SAMPLE}/filtered.fastq.gz",
78         assembly=f"assembly/flye/{SAMPLE}/assembly.fasta"
79     output:
80         f"assembly/medaka/{SAMPLE}/consensus.fasta"
81     params:
82         outdir=f"assembly/medaka/{SAMPLE}",
83         model=MEDAKA_MODEL
84     threads: 8
85     shell:
86         """
87         medaka_polisher \
88             -i {input.reads} \
89             -d {input.assembly} \
90             -o {params.outdir} \
91             --model {params.model} \
92             --threads {threads} --bacteria
93         """
94
95 rule quast:
96     input:
97         assembly=f"assembly/medaka/{SAMPLE}/consensus.fasta"
98     output:
99         f"qc/quast/{SAMPLE}/report.html"
100    params:
101        outdir=f"qc/quast/{SAMPLE}",
102        ref_flag=f"-r {REFERENCE}" if REFERENCE else ""
103    threads: 4
104    shell:
105        """
106        quast {input.assembly} \
107            {params.ref_flag} \
108            -o {params.outdir} \
109            --threads {threads} --min-contig 500
110        """
111
112 rule busco:
113     input:
114         f"assembly/medaka/{SAMPLE}/consensus.fasta"
115     output:
116         f"qc/busco/{SAMPLE}/short_summary.txt"
117     params:
118         outdir="qc/busco",
119         lineage=BUSCO_LINEAGE
120     threads: 8
121     shell:
122         """
123         busco -i {input} \
124             -o {SAMPLE} -m genome \
125             -l {params.lineage} \
126             --cpu {threads} \
127             --out_path {params.outdir}
128         cp {params.outdir}/{SAMPLE}/short_summary.*.txt \
129             {output}
130         """

```

```

131
132 rule bakta_annotate:
133     input:
134         f"assembly/medaka/{SAMPLE}/consensus.fasta"
135     output:
136         f"annotation/{SAMPLE}/{SAMPLE}.gff3"
137     params:
138         outdir=f"annotation/{SAMPLE}",
139         db=BAKTA_DB
140     threads: 8
141     shell:
142         """
143         bakta {input} \
144             --db {params.db} \
145             --output {params.outdir} \
146             --prefix {SAMPLE} \
147             --threads {threads} --complete
148         """

```

The corresponding configuration file:

Listing 26.15: Configuration file for the bacterial assembly Snakemake pipeline (config.yaml).

```

1 # config.yaml -- Bacterial Genome Assembly
2 sample: "ecoli_k12"
3 raw_bam: "data/ecoli/basecalled.bam"
4 genome_size: "4.6m"
5 medaka_model: "r1041_e82_400bps_sup_v4.3.0"
6 busco_lineage: "enterobacterales_odb10"
7 bakta_db: "db/bakta/db-light"
8 reference: "ref/ecoli_k12.fasta" # optional

```

26.4.9 Complete Nextflow Workflow

Listing 26.16: Nextflow DSL2 bacterial genome assembly pipeline (main.nf).

```

1 #!/usr/bin/env nextflow
2 nextflow.enable.dsl = 2
3
4 // === Parameters ===
5 params.sample      = "ecoli_k12"
6 params.raw_bam     = "data/ecoli/basecalled.bam"
7 params.genome_size = "4.6m"
8 params.medaka_model = "r1041_e82_400bps_sup_v4.3.0"
9 params.busco_lineage = "enterobacterales_odb10"
10 params.bakta_db    = "db/bakta/db-light"
11 params.reference   = ""
12 params.outdir      = "results"
13
14 // === Processes ===
15 process BAM_TO_FASTQ {
16     tag "${params.sample}"
17     cpus 4
18
19     input: path(bam)
20     output: path("*.fastq.gz"), emit: fastq
21
22     script:

```

```

23     """
24     samtools fastq -@ ${task.cpus} ${bam} | gzip > basecalled.fastq.gz
25     """
26 }
27
28 process NANOPILOT {
29     tag "${params.sample}"
30     publishDir "${params.outdir}/qc/nanoplot", mode: 'copy'
31     cpus 4
32
33     input: path(fastq)
34     output: path("nanoplot_out/*"), emit: report
35
36     script:
37         """
38         NanoPlot --fastq ${fastq} \
39             --outdir nanoplot_out \
40             --threads ${task.cpus} --loglength --N50
41         """
42 }
43
44 process CHOPPER {
45     tag "${params.sample}"
46     cpus 4
47
48     input: path(fastq)
49     output: path("filtered.fastq.gz"), emit: filtered
50
51     script:
52         """
53         zcat ${fastq} | chopper \
54             --quality 10 --minlength 1000 \
55             --threads ${task.cpus} | gzip > filtered.fastq.gz
56         """
57 }
58
59 process FLYE {
60     tag "${params.sample}"
61     publishDir "${params.outdir}/assembly/flye", mode: 'copy'
62     cpus 8
63     memory '16 GB'
64
65     input: path(reads)
66     output:
67         path("flye_out/assembly.fasta"), emit: assembly
68         path("flye_out/assembly_info.txt"), emit: info
69
70     script:
71         """
72         flye --nano-hq ${reads} \
73             --out-dir flye_out \
74             --threads ${task.cpus} \
75             --genome-size ${params.genome_size}
76         """
77 }
78
79 process MEDAKA {
80     tag "${params.sample}"

```

```

81     publishDir "${params.outdir}/assembly/medaka", mode: 'copy'
82     cpus 8
83     memory '16 GB'
84
85     input:
86         path(reads)
87         path(assembly)
88     output: path("medaka_out/consensus.fasta"), emit: polished
89
90     script:
91     """
92     medaka_polisher \
93         -i ${reads} -d ${assembly} \
94         -o medaka_out \
95         --model ${params.medaka_model} \
96         --threads ${task.cpus} --bacteria
97     """
98 }
99
100 process QUAST {
101     tag "${params.sample}"
102     publishDir "${params.outdir}/qc/quast", mode: 'copy'
103     cpus 4
104
105     input:
106         path(assembly)
107         val(reference)
108     output: path("quast_out/*"), emit: report
109
110     script:
111     def ref_flag = reference ? "-r ${reference}" : ""
112     """
113     quast ${assembly} ${ref_flag} \
114         -o quast_out --threads ${task.cpus} --min-contig 500
115     """
116 }
117
118 process BUSCO {
119     tag "${params.sample}"
120     publishDir "${params.outdir}/qc/busco", mode: 'copy'
121     cpus 8
122
123     input: path(assembly)
124     output: path("busco_out/*"), emit: report
125
126     script:
127     """
128     busco -i ${assembly} \
129         -o busco_out -m genome \
130         -l ${params.busco_lineage} \
131         --cpu ${task.cpus}
132     """
133 }
134
135 process BAKTA {
136     tag "${params.sample}"
137     publishDir "${params.outdir}/annotation", mode: 'copy'
138     cpus 8

```

```

139     memory '8 GB'
140
141     input:
142         path(assembly)
143         path(db)
144     output: path("bakta_out/*"), emit: annotation
145
146     script:
147         """
148         bakta ${assembly} \
149             --db ${db} \
150             --output bakta_out \
151             --prefix ${params.sample} \
152             --threads ${task.cpus} --complete
153         """
154     }
155
156     // === Workflow ===
157     workflow {
158         bam_ch = Channel.fromPath(params.raw_bam, checkIfExists: true)
159         db_ch  = Channel.fromPath(params.bakta_db, checkIfExists: true)
160
161         BAM_TO_FASTQ(bam_ch)
162         NANOPLLOT(BAM_TO_FASTQ.out.fastq)
163         CHOPPER(BAM_TO_FASTQ.out.fastq)
164         FLYE(CHOPPER.out.filtered)
165         MEDAKA(CHOPPER.out.filtered, FLYE.out.assembly)
166         QUAST(MEDAKA.out.polished, params.reference)
167         BUSCO(MEDAKA.out.polished)
168         BAKTA(MEDAKA.out.polished, db_ch)
169     }

```

26.5 Walkthrough 2: Species Identification with Kraken2

Taxonomic classification—identifying which organisms are present in a sample—is one of the most common applications of nanopore sequencing. Kraken2 uses exact k -mer matching against a reference database to classify reads at remarkable speed.

26.5.1 Database Setup

The database is the foundation of any Kraken2 analysis. Different databases offer different trade-offs between comprehensiveness and resource requirements.

Listing 26.17: Setting up Kraken2 databases.

```

1  # Option 1: Standard database (~70 GB, requires 64 GB RAM)
2  # Includes archaea, bacteria, viral, plasmid, human, UniVec_Core
3  kraken2-build --standard --db db/kraken2_standard --threads 8
4
5  # Option 2: PlusPF database (includes protozoa & fungi; ~100 GB)
6  # Recommended for environmental samples
7  kraken2-build --standard --db db/kraken2_pluspf \
8      --threads 8 \
9      --protein # include protein-level matches
10
11 # Option 3: Pre-built databases (fastest setup!)
12 # Download from https://benlangmead.github.io/aws-indexes/k2

```

```

13 wget -P db/ \
14     https://genome-index.s3.amazonaws.com/kraken/k2_standard_20240904.tar.gz
15 mkdir -p db/kraken2_standard
16 tar -xzf db/k2_standard_20240904.tar.gz -C db/kraken2_standard
17
18 # Option 4: Custom database (e.g., fish species for eDNA)
19 kraken2-build --download-taxonomy --db db/kraken2_fish
20 # Add custom genomes
21 for genome in custom_genomes/*.fasta; do
22     kraken2-build --add-to-library "$genome" --db db/kraken2_fish
23 done
24 kraken2-build --build --db db/kraken2_fish --threads 8

```

Kraken2 Database Memory Requirements

Kraken2 loads the entire database into RAM during classification. The standard database requires ~64 GB of RAM, and PlusPF requires ~100 GB. If you have limited RAM, use one of the smaller pre-built databases (e.g., the “MinusDB” variants, ~8 GB) or use Kraken2 with the `-memory-mapping` flag (slower but uses disk instead of RAM). The *k*-mer length is fixed at 35 bp by default.

26.5.2 Running Classification

Listing 26.18: Classifying reads with Kraken2.

```

1 # Classify nanopore reads
2 kraken2 \
3     --db db/kraken2_standard \
4     --threads 8 \
5     --output kraken2/classifications.txt \
6     --report kraken2/report.txt \
7     --report-minimizer-data \
8     --minimum-hit-groups 3 \
9     reads.fastq.gz
10
11 # Key output files:
12 # classifications.txt - per-read classification (C/U, read ID, taxon)
13 # report.txt         - hierarchical abundance report
14
15 # Quick summary: what percentage was classified?
16 head -5 kraken2/report.txt

```

The `--minimum-hit-groups 3` parameter requires at least 3 groups of consecutive *k*-mers to match a taxon before classification, which reduces false positives for long nanopore reads.

26.5.3 Abundance Estimation with Bracken

Bracken re-estimates species-level abundances from Kraken2 results by redistributing reads assigned to higher taxonomic levels.

Listing 26.19: Running Bracken for abundance estimation.

```

1 # Build Bracken database (one-time step, per Kraken2 DB)
2 bracken-build -d db/kraken2_standard -t 8 -l 1000
3
4 # Run Bracken at species level
5 bracken \

```



```

6  -d db/kraken2_standard \
7  -i kraken2/report.txt \
8  -o bracken/species_abundance.txt \
9  -w bracken/report_bracken.txt \
10 -r 1000 \
11 -l S \
12 -t 10
13
14 # -r 1000 : read length (approximate; use your median read length)
15 # -l S    : taxonomic level (S=species, G=genus, F=family)
16 # -t 10   : minimum reads to keep a taxon
17
18 # View top species
19 sort -t$'\t' -k7 -rn bracken/species_abundance.txt | head -20

```

26.5.4 Interactive Visualization with Krona

Krona creates interactive, zoomable pie charts for exploring taxonomic composition.

Listing 26.20: Creating Krona visualisations.

```

1  # Convert Kraken2 report to Krona-compatible format
2  kreport2krona.py \
3    -r kraken2/report.txt \
4    -o kraken2/krona_input.txt
5
6  # Generate interactive HTML
7  ktImportText \
8    kraken2/krona_input.txt \
9    -o kraken2/krona_plot.html
10
11 # Open in your browser
12 # On macOS: open kraken2/krona_plot.html
13 # On Linux: xdg-open kraken2/krona_plot.html

```

26.5.5 Example 1: Environmental DNA from a Pond

Environmental DNA (eDNA) analysis involves filtering water, extracting DNA, and sequencing to detect which organisms are present. Here is a complete analysis for a freshwater pond sample:

Listing 26.21: eDNA pond water analysis pipeline.

```

1  # Assume basecalled reads are in: data/pond_edna/reads.fastq.gz
2
3  # Step 1: Quality filter
4  zcat data/pond_edna/reads.fastq.gz | \
5    chopper --quality 8 --minlength 500 --threads 4 | \
6    gzip > data/pond_edna/filtered.fastq.gz
7
8  # Step 2: Classify with PlusPF database (includes protozoa, fungi)
9  kraken2 \
10    --db db/kraken2_pluspf \
11    --threads 8 \
12    --output results/pond/kraken2_output.txt \
13    --report results/pond/kraken2_report.txt \
14    --minimum-hit-groups 3 \
15    data/pond_edna/filtered.fastq.gz

```

```

16
17 # Step 3: Bracken species-level estimates
18 bracken \
19   -d db/kraken2_pluspf \
20   -i results/pond/kraken2_report.txt \
21   -o results/pond/bracken_species.txt \
22   -r 1000 -l S -t 5
23
24 # Step 4: Visualise
25 kreport2krona.py -r results/pond/kraken2_report.txt \
26   -o results/pond/krona_input.txt
27 ktImportText results/pond/krona_input.txt \
28   -o results/pond/pond_krona.html
29
30 # Step 5: Extract vertebrate detections (for biodiversity monitoring)
31 grep -i "Vertebrata" results/pond/kraken2_report.txt

```

Typical findings from a pond eDNA sample include: dominant bacterial taxa (e.g., *Flavobacterium*, *Pseudomonas*), algae, protists, and—critically for biodiversity monitoring—trace amounts of vertebrate DNA from fish, amphibians, and waterfowl that have shed cells into the water.

26.5.6 Example 2: Sushi Species Authentication

Nanopore sequencing is increasingly used for food authentication—verifying that the species on the label matches the species on the plate.

Listing 26.22: Sushi species authentication analysis.

```

1 # Basecalled reads from sushi sample: data/sushi/reads.fastq.gz
2
3 # Classify against the standard database
4 kraken2 \
5   --db db/kraken2_standard \
6   --threads 8 \
7   --output results/sushi/classifications.txt \
8   --report results/sushi/report.txt \
9   --minimum-hit-groups 3 \
10  data/sushi/reads.fastq.gz
11
12 # Extract fish-related hits
13 grep -E "Actinopterygii|Chondrichthyes" results/sushi/report.txt
14
15 # Bracken estimation at species level
16 bracken \
17   -d db/kraken2_standard \
18   -i results/sushi/report.txt \
19   -o results/sushi/bracken_species.txt \
20   -r 1000 -l S -t 5
21
22 # Common findings for mislabelled sushi:
23 # "White tuna" often = Escolar (Lepidocybium flavobrunneum)
24 # "Red snapper" often = Tilapia (Oreochromis niloticus)
25 # Salmon should be = Salmo salar or Oncorhynchus spp.

```

26.5.7 Interpretation Guidelines

- **Classification rate:** For a pure bacterial culture, expect >90% of reads classified. For environmental samples, 20–60% is typical (many environmental organisms lack reference genomes).
- **Confidence threshold:** Reads classified at genus level are more reliable than species-level assignments. Report genus-level results when species-level classification has low *k*-mer coverage.
- **False positives:** Closely related species (e.g., *E. coli* vs. *Shigella*) can be confused. Validate key findings with a second tool (e.g., Centrifuge or MetaPhlAn).
- **Database matters:** If your target organism is not in the database, it cannot be detected. Always check database contents before drawing negative conclusions.
- **Quantification caveats:** Relative abundance estimates from Bracken reflect read proportions, not cell counts. Differences in genome size, cell lysis efficiency, and DNA extraction bias all affect the relationship between read abundance and true organism abundance.

26.6 Walkthrough 3: Variant Calling

Variant calling from nanopore data identifies genetic differences between a sequenced sample and a reference genome. While short-read variant calling is well established (see [Chapter 8](#)), long-read variant calling uses specialised tools trained on the unique error profile of nanopore data.

26.6.1 Step 1: Alignment with minimap2

minimap2 is the standard aligner for long reads. Its `-ax map-ont` preset is optimised for nanopore data.

Listing 26.23: Aligning nanopore reads with minimap2.

```

1 # Download human reference genome (if not already present)
2 wget -P ref/ \
3   https://ftp.ncbi.nlm.nih.gov/genomes/all/GCA/000/001/405/\
4   GCA_000001405.15_GRCh38/seqs_for_alignment_pipelines.ucsc_ids/\
5   GCA_000001405.15_GRCh38_no_alt_analysis_set.fna.gz
6 gunzip ref/GCA_000001405.15_GRCh38_no_alt_analysis_set.fna.gz
7 mv ref/GCA_000001405.15_GRCh38_no_alt_analysis_set.fna ref/GRCh38.fa
8 samtools faidx ref/GRCh38.fa
9
10 # Align with minimap2
11 minimap2 \
12   -ax map-ont \
13   -t 16 \
14   --MD \
15   -R "@RG\tID:sample1\tSM:HG002\tPL:ONT" \
16   ref/GRCh38.fa \
17   basecalled.fastq.gz | \
18   samtools sort -@ 8 -o aligned/HG002.sorted.bam -
19
20 # Index the BAM file
21 samtools index aligned/HG002.sorted.bam
22
23 # Quick alignment statistics
24 samtools flagstat aligned/HG002.sorted.bam
25 samtools coverage aligned/HG002.sorted.bam | head -25

```

Key minimap2 parameters:

- ax map-ont** Preset for Oxford Nanopore reads: uses longer minimiser windows, higher error tolerance, and ONT-specific scoring.
- MD** Adds the MD tag to the output, which records the reference sequence at mismatching positions. Required by some variant callers.
- R** Adds a read group header, required by GATK-based tools and useful for tracking samples in multi-sample analyses.
- t 16** Number of alignment threads. minimap2 scales well to 16–32 threads.

26.6.2 Step 2: Variant Calling with Clair3

Clair3 is a deep-learning-based variant caller specifically designed for long-read sequencing data. It supports both nanopore and PacBio data with platform-specific models.

Listing 26.24: Calling variants with Clair3.

```

1 # Create a separate environment for Clair3 (dependency conflicts)
2 mamba create -n clair3 -c bioconda clair3
3 mamba activate clair3
4
5 # Run Clair3
6 run_clair3.sh \
7   --bam_fn=aligned/HG002.sorted.bam \
8   --ref_fn=ref/GRCh38.fa \
9   --output=variants/clair3 \
10  --threads=16 \
11  --platform="ont" \
12  --model_path="${CONDA_PREFIX}/bin/models/ont" \
13  --sample_name="HG002" \
14  --include_all_ctgs
15
16 # Key output files:
17 # variants/clair3/merge_output.vcf.gz - all variants (SNPs + indels)
18 # variants/clair3/pileup.vcf.gz      - pileup model variants
19 # variants/clair3/full_alignment.vcf.gz - full-alignment model variants
20
21 # Count variants
22 bcftools stats variants/clair3/merge_output.vcf.gz | grep "number of records"
```

Clair3 Model Selection

Clair3 ships with pre-trained models for different platforms and chemistries. Use the correct model:

- **ONT R10.4.1 SUP:** ont_r1041_e82_400bps_sup_v430 (best current performance)
- **ONT R10.4.1 HAC:** ont_r1041_e82_400bps_hac_v430
- **ONT R9.4.1 SUP:** ont_r941_e81_sup_v340
- **PacBio HiFi:** hifi

Models are located in `$CONDA_PREFIX/bin/models/`. Using the wrong model will produce unreliable variant calls.

26.6.3 Step 3: Variant Filtering with bcftools

Raw variant calls should be filtered to remove low-confidence calls before downstream analysis.

Listing 26.25: Filtering variants with bcftools.

```

1 # Filter by quality and depth
2 bcftools filter \
3   -i 'QUAL>=20 && INFO/DP>=10' \
4   -o variants/clair3/filtered.vcf.gz \
5   -O z \
6   variants/clair3/merge_output.vcf.gz
7
8 # Index the filtered VCF
9 bcftools index variants/clair3/filtered.vcf.gz
10
11 # Separate SNPs and indels
12 bcftools view -v snps \
13   variants/clair3/filtered.vcf.gz \
14   -o variants/clair3/snps.vcf.gz -O z
15
16 bcftools view -v indels \
17   variants/clair3/filtered.vcf.gz \
18   -o variants/clair3/indels.vcf.gz -O z
19
20 # Summary statistics
21 bcftools stats variants/clair3/filtered.vcf.gz > variants/clair3/stats.txt

```

26.6.4 Expected Variant Counts

Table 26.6 shows typical variant counts for a human genome at 30× nanopore coverage.

Table 26.6: Expected variant counts per human genome (30× ONT, SUP basecalling, Clair3).

Variant Type	Expected Count	Clair3 F1 Score	Notes
SNVs	4.0–4.5 million	>99.5%	Performance comparable to short-read callers
Indels (1–50 bp)	500,000–700,000	~85–92%	Homopolymer indels remain challenging
Heterozygous SNVs	~2.5 million	>99%	Well-detected by long reads
Homozygous ALT SNVs	~1.6 million	>99.5%	Very reliable
Ti/Tv ratio	2.05–2.15	—	Values outside this range suggest systematic errors

26.6.5 Comparison: Clair3 vs Medaka for Variant Calling

Both Clair3 and Medaka can call variants, but they are designed for different use cases:

Rule of Thumb for Variant Caller Selection

Use Clair3 for diploid organisms (human, animal, plant) where you need accurate genotype calls (0/0, 0/1, 1/1). Use Medaka for haploid organisms (bacteria) or when you primarily need consensus correction rather than per-site variant calls. For structural variant calling (>50 bp), use Sniffles2 or cuteSV instead—neither Clair3 nor Medaka is designed for large structural variants.

Table 26.7: Comparison of Clair3 and Medaka for variant calling from nanopore data.

Feature	Clair3	Medaka (variant mode)
Primary use	Germline variant calling (human, diploid)	Consensus/variant calling (haploid, bacterial)
SNV accuracy	Excellent (>99.5% F1)	Good (~98% F1)
Indel accuracy	Good (~85–92% F1)	Moderate (~75–85% F1)
Diploid genotyping	Yes (proper GT field)	Limited (designed for haploid)
Speed (30× human)	~4–8 hours (16 threads)	~12–24 hours
GPU support	Optional (speeds pileup model)	Optional
Best for	Human/diploid variant calling	Bacterial SNP calling, assembly polishing

26.7 Visualization with IGV

The Integrative Genomics Viewer (IGV) is an essential tool for visually inspecting alignments, variants, and annotations. No bioinformatics analysis is complete without manual inspection of key results.

26.7.1 Loading and Navigating Data

Listing 26.26: Preparing files for IGV and launching from the command line.

```

1 # Ensure BAM is sorted and indexed
2 samtools sort aligned/HG002.sorted.bam -o aligned/HG002.sorted.bam
3 samtools index aligned/HG002.sorted.bam
4
5 # Ensure VCF is sorted, compressed, and indexed
6 bcftools sort variants/clair3/filtered.vcf.gz \
7   -o variants/clair3/filtered.sorted.vcf.gz
8 bcftools index variants/clair3/filtered.sorted.vcf.gz
9
10 # Launch IGV (if installed locally)
11 # Download from https://igv.org/doc/desktop/#DownloadPage/
12 igv.sh
13
14 # Or use IGV-Web at https://igv.org/app/

```

Once IGV is open:

1. **Load the reference genome:** *Genomes* → *Load Genome from Server* → select “Human (GRCh38/hg38)”.
2. **Load the BAM file:** *File* → *Load from File* → select `HG002.sorted.bam`.
3. **Load the VCF file:** *File* → *Load from File* → select `filtered.sorted.vcf.gz`.
4. **Navigate:** Type a gene name (e.g., *BRCA1*) or coordinates (e.g., `chr17:43,044,295–43,125,364`) in the search bar.

26.7.2 Interpreting Alignment Displays

When viewing long-read alignments in IGV, pay attention to:

Read colour Grey indicates a proper alignment. Red, blue, or other colours indicate structural variant signatures (mate mapped to different chromosome, unexpected insert size, etc.).

Coloured bases within reads These mark mismatches relative to the reference. A column of consistent mismatches across many reads is likely a true variant; scattered mismatches are sequencing errors.

Insertions (purple “I”) Small purple marks indicate inserted bases relative to the reference. Hover over them to see the inserted sequence.

Deletions (thin lines) Horizontal lines within a read indicate deleted bases. A consistent deletion across many reads suggests a true deletion variant.

Soft-clipped regions Coloured bars at read ends indicate portions of the read that did not align. Consistent soft-clipping at the same position can indicate a structural variant breakpoint.

26.7.3 Batch Screenshots

For publications or systematic review of multiple loci, IGV supports batch scripting:

Listing 26.27: IGV batch script for automated screenshots.

```

1 # Create a batch script: igv_batch.txt
2 cat > igv_batch.txt << 'BATCH'
3 new
4 genome hg38
5 load aligned/HG002.sorted.bam
6 load variants/clair3/filtered.sorted.vcf.gz
7 snapshotDirectory igv_screenshots
8 maxPanelHeight 500
9 goto chr17:43,044,295-43,125,364
10 sort base
11 snapshot BRCA1_overview.png
12 goto chr17:43,092,900-43,093,100
13 snapshot BRCA1_exon10_zoom.png
14 goto chr13:32,315,474-32,400,266
15 snapshot BRCA2_overview.png
16 goto chr7:117,120,017-117,120,201
17 snapshot CFTR_p.F508del.png
18 exit
19 BATCH
20
21 # Run IGV in batch mode
22 igv.sh --batch igv_batch.txt

```

26.8 Galaxy: No-Code Option

Not everyone is comfortable with the command line, and not every analysis requires a custom pipeline. Galaxy (<https://usegalaxy.org>) provides a web-based, graphical interface for bioinformatics analysis that requires no programming.

26.8.1 Getting Started with Galaxy

Galaxy is a free, open-source platform available at three main public servers:

- **usegalaxy.org** (USA, hosted at Penn State/Johns Hopkins)
- **usegalaxy.eu** (Europe, hosted at University of Freiburg)
- **usegalaxy.org.au** (Australia, hosted at Melbourne Bioinformatics)

To begin:

1. Create a free account at <https://usegalaxy.org>.
2. Upload your **FASTQ** or **BAM** files via the upload tool (supports FTP for large files).

3. Search for tools in the left panel (e.g., “NanoPlot”, “Flye”, “Kraken2”).
4. Configure parameters via the graphical form and click “Run Tool”.
5. Results appear in your history panel on the right.

26.8.2 Building Visual Workflows

Galaxy’s Workflow Editor allows you to chain tools together visually:

1. Go to *Workflow* → *Create New Workflow*.
2. Drag tools from the toolbox onto the canvas.
3. Connect outputs to inputs by dragging lines between tool boxes.
4. Set parameters for each tool.
5. Save and run the workflow on your data.

Galaxy also hosts a large library of pre-built, community-curated workflows. The Galaxy Training Network (<https://training.galaxyproject.org>) provides step-by-step tutorials for nanopore analysis, including bacterial genome assembly and metagenomics.

26.8.3 Limitations vs Command Line

Table 26.8: Galaxy vs command-line analysis: trade-offs.

Aspect	Galaxy	Command Line
Ease of use	Point-and-click; no coding	Requires terminal familiarity
Reproducibility	Workflows are shareable and versioned	Requires Snakemake/Nextflow discipline
Tool availability	~5,000+ tools, but not always latest versions	Full access to any tool, any version
Customisation	Limited to available tool wrappers	Unlimited
Large datasets	Upload limits; queue times on public servers	Limited only by your hardware
GPU basecalling	Not available on public servers	Full GPU support
Collaboration	Shared histories and workflows	Requires Git, shared filesystems
Cost	Free on public servers (with quotas)	Hardware and electricity costs

When to Use Galaxy

Galaxy is an excellent choice when: (1) you are a wet-lab scientist who needs quick results without learning the command line, (2) you want to teach students bioinformatics concepts without setup overhead, (3) you need to run a well-established workflow and do not need cutting-edge tool versions, or (4) you want to share a reproducible analysis with collaborators. Switch to the command line when you need the latest tools, GPU basecalling, custom scripts, or are processing very large datasets.

26.9 Cloud Computing on a Budget

When your local hardware is insufficient—particularly for GPU basecalling, human genome assembly, or large-scale metagenomics—cloud computing offers a scalable alternative.

26.9.1 When Local Isn’t Enough

Consider cloud computing when:

- You need a GPU for basecalling but do not own one.
- Your dataset exceeds local RAM (e.g., human genome assembly needs 32–64 GB).
- You need to process many samples in parallel (e.g., a sequencing core facility).
- You need temporary burst capacity (e.g., a one-time large experiment).
- Your institution does not provide [HPC](#) access.

26.9.2 Cloud Platform Comparison

Table 26.9: Cloud computing options for nanopore analysis (approximate pricing as of 2024).

Platform	Instance Type	On-Demand	Spot/Preemptible	Best For
AWS EC2	g5.2xlarge (1× A10G, 32 GB RAM)	\$1.21/hr	\$0.36–0.50/hr	GPU basecalling
AWS EC2	r6i.4xlarge (16 vCPU, 128 GB RAM)	\$1.01/hr	\$0.30–0.40/hr	Genome assembly
GCP	g2-standard-8 (1× L4, 32 GB RAM)	\$0.98/hr	\$0.29–0.39/hr	GPU basecalling
GCP	n2-highmem-16 (16 vCPU, 128 GB RAM)	\$0.95/hr	\$0.28–0.38/hr	Assembly, variant calling
Google Colab	Free tier (T4 GPU, 12 GB RAM)	Free	—	Small datasets, learning
Google Colab Pro	A100 GPU, 52 GB RAM	\$10/month	—	Moderate basecalling
Terra (Broad)	Configurable	Variable	—	GATK pipelines, human genomics

Spot Instances Save 60–80%

Spot instances (AWS) or preemptible VMs (GCP) offer the same hardware at 60–80% discount, but can be terminated with 2 minutes’ notice. This is acceptable for workflows managed by Snakemake or Nextflow, which can automatically resume interrupted jobs. Always use spot/preemptible instances for non-interactive batch processing.

26.9.3 Google Colab for Small Analyses

Google Colab provides free GPU access through a Jupyter notebook interface. It is ideal for learning and processing small datasets (e.g., a single bacterial genome).

Listing 26.28: Setting up a nanopore analysis environment in Google Colab.

```

1 # In a Colab notebook cell:
2 # Install conda in Colab
3 !pip install -q condaolab

```

```

4 import condacolab
5 condacolab.install()
6
7 # Install tools
8 !mamba install -y -c conda-forge -c bioconda \
9     nanoplot flye medaka minimap2 samtools
10
11 # Upload your data (or download from SRA)
12 !pip install -q sra-tools
13 !prefetch SRR28370649
14 !fasterq-dump SRR28370649 -O data/
15
16 # Run NanoPlot
17 !NanoPlot --fastq data/SRR28370649.fastq --outdir qc/ --threads 2

```

Colab Limitations

Google Colab sessions are ephemeral: they disconnect after ~90 minutes of inactivity (free tier) or 24 hours maximum. All data and installed software are lost. Save results to Google Drive (from `google.colab import drive; drive.mount('/content/drive')`) before the session ends. RAM is limited to 12 GB (free) or 52 GB (Pro), which is insufficient for Kraken2 with the standard database.

26.9.4 Terra for Production Genomics

Terra (<https://terra.bio>), developed by the Broad Institute, is a cloud-native platform designed specifically for genomics research. It runs on Google Cloud and provides:

- Pre-configured analysis environments with GATK, Cromwell, and other tools.
- WDL (Workflow Description Language) workflow execution.
- Direct access to large public datasets (TCGA, gnomAD, 1000 Genomes).
- Built-in Jupyter notebooks for interactive analysis.
- Fine-grained billing control with workspace budgets.

Terra is particularly well-suited for clinical genomics labs processing many human genomes through standardised GATK pipelines.

26.10 Complete End-to-End Snakemake Workflow

This section provides a comprehensive Snakemake workflow that chains every step from basecalling through annotation into a single, reproducible pipeline.

Listing 26.29: Complete end-to-end Snakemake workflow for nanopore analysis (Snakefile).

```

1 # Snakefile -- Complete Nanopore Analysis Pipeline
2 # Basecalling -> QC -> Assembly -> Polishing -> Annotation
3 configfile: "config.yaml"
4
5 SAMPLE          = config["sample"]
6 POD5_DIR        = config["pod5_dir"]
7 DORADO_MODEL     = config["dorado_model"]
8 GENOME_SIZE     = config["genome_size"]
9 MEDAKA_MODEL     = config["medaka_model"]
10 BUSCO_LIN       = config["busco_lineage"]
11 BAKTA_DB        = config["bakta_db"]
12 REFERENCE       = config.get("reference", "")
13 DEVICE         = config.get("device", "cuda:0")

```

```

14
15 rule all:
16     input:
17         f"results/{SAMPLE}/qc/nanoplot/NanoPlot-report.html",
18         f"results/{SAMPLE}/qc/multiqc/multiqc_report.html",
19         f"results/{SAMPLE}/assembly/medaka/consensus.fasta",
20         f"results/{SAMPLE}/qc/quast/report.html",
21         f"results/{SAMPLE}/qc/busco/short_summary.txt",
22         f"results/{SAMPLE}/annotation/{SAMPLE}.gff3"
23
24 # ---- Basecalling ----
25 rule dorado_basecall:
26     input:
27         POD5_DIR
28     output:
29         f"results/{SAMPLE}/basecall/basecalled.bam"
30     params:
31         model=DORADO_MODEL,
32         device=DEVICE
33     shell:
34         """
35         dorado basecaller {params.model} {input} \
36             --device {params.device} --emit-moves \
37             > {output}
38         """
39
40 rule bam_to_fastq:
41     input:
42         f"results/{SAMPLE}/basecall/basecalled.bam"
43     output:
44         f"results/{SAMPLE}/basecall/basecalled.fastq.gz"
45     threads: 4
46     shell:
47         "samtools fastq -@ {threads} {input} | gzip > {output}"
48
49 # ---- Quality Control ----
50 rule nanoplot_qc:
51     input:
52         f"results/{SAMPLE}/basecall/basecalled.fastq.gz"
53     output:
54         f"results/{SAMPLE}/qc/nanoplot/NanoPlot-report.html"
55     params:
56         outdir=f"results/{SAMPLE}/qc/nanoplot"
57     threads: 4
58     shell:
59         """
60         NanoPlot --fastq {input} --outdir {params.outdir} \
61             --threads {threads} --loglength --N50 \
62             --title "{SAMPLE} QC"
63         """
64
65 rule fastqc:
66     input:
67         f"results/{SAMPLE}/basecall/basecalled.fastq.gz"
68     output:
69         f"results/{SAMPLE}/qc/fastqc/basecalled_fastqc.html"
70     params:
71         outdir=f"results/{SAMPLE}/qc/fastqc"

```

```

72     threads: 2
73     shell:
74         "fastqc {input} --outdir {params.outdir} --threads {threads}"
75
76 rule multiqc:
77     input:
78         f"results/{SAMPLE}/qc/nanoplot/NanoPlot-report.html",
79         f"results/{SAMPLE}/qc/fastqc/basecalled_fastqc.html"
80     output:
81         f"results/{SAMPLE}/qc/multiqc/multiqc_report.html"
82     params:
83         indir=f"results/{SAMPLE}/qc",
84         outdir=f"results/{SAMPLE}/qc/multiqc"
85     shell:
86         "multiqc {params.indir} -o {params.outdir} --force"
87
88 # ---- Read Filtering ----
89 rule chopper_filter:
90     input:
91         f"results/{SAMPLE}/basecall/basecalled.fastq.gz"
92     output:
93         f"results/{SAMPLE}/filtered/reads.fastq.gz"
94     threads: 4
95     shell:
96         """
97         zcat {input} | chopper --quality 10 --minlength 1000 \
98         --threads {threads} | gzip > {output}
99         """
100
101 # ---- Assembly ----
102 rule flye_assemble:
103     input:
104         f"results/{SAMPLE}/filtered/reads.fastq.gz"
105     output:
106         fasta=f"results/{SAMPLE}/assembly/flye/assembly.fasta",
107         info=f"results/{SAMPLE}/assembly/flye/assembly_info.txt"
108     params:
109         outdir=f"results/{SAMPLE}/assembly/flye",
110         gsize=GENOME_SIZE
111     threads: 8
112     shell:
113         """
114         flye --nano-hq {input} \
115         --out-dir {params.outdir} \
116         --threads {threads} \
117         --genome-size {params.gsize}
118         """
119
120 # ---- Polishing ----
121 rule medaka_polish:
122     input:
123         reads=f"results/{SAMPLE}/filtered/reads.fastq.gz",
124         draft=f"results/{SAMPLE}/assembly/flye/assembly.fasta"
125     output:
126         f"results/{SAMPLE}/assembly/medaka/consensus.fasta"
127     params:
128         outdir=f"results/{SAMPLE}/assembly/medaka",
129         model=MEDAKA_MODEL

```

```

130 threads: 8
131 shell:
132     """
133     medaka_polisher \
134         -i {input.reads} -d {input.draft} \
135         -o {params.outdir} \
136         --model {params.model} \
137         --threads {threads} --bacteria
138     """
139
140 # ---- Quality Assessment ----
141 rule quast_assess:
142     input:
143         f"results/{SAMPLE}/assembly/medaka/consensus.fasta"
144     output:
145         f"results/{SAMPLE}/qc/quast/report.html"
146     params:
147         outdir=f"results/{SAMPLE}/qc/quast",
148         ref=f"-r {REFERENCE}" if REFERENCE else ""
149     threads: 4
150     shell:
151         """
152         quast {input} {params.ref} \
153             -o {params.outdir} \
154             --threads {threads} --min-contig 500
155         """
156
157 rule busco_assess:
158     input:
159         f"results/{SAMPLE}/assembly/medaka/consensus.fasta"
160     output:
161         f"results/{SAMPLE}/qc/busco/short_summary.txt"
162     params:
163         outdir=f"results/{SAMPLE}/qc/busco",
164         lineage=BUSCO_LIN
165     threads: 8
166     shell:
167         """
168         busco -i {input} -o busco_run -m genome \
169             -l {params.lineage} --cpu {threads} \
170             --out_path {params.outdir}
171         cp {params.outdir}/busco_run/short_summary.*.txt {output}
172         """
173
174 # ---- Annotation ----
175 rule bakta_annotate:
176     input:
177         f"results/{SAMPLE}/assembly/medaka/consensus.fasta"
178     output:
179         f"results/{SAMPLE}/annotation/{SAMPLE}.gff3"
180     params:
181         outdir=f"results/{SAMPLE}/annotation",
182         db=BAKTA_DB
183     threads: 8
184     shell:
185         """
186         bakta {input} --db {params.db} \
187             --output {params.outdir} \

```

```

188     --prefix {SAMPLE} \
189     --threads {threads} --complete
190     """

```

The corresponding configuration file:

Listing 26.30: Configuration file for the end-to-end Snakemake pipeline (config.yaml).

```

1 # config.yaml -- Complete Nanopore Pipeline Configuration
2 sample: "ecoli_k12"
3 pod5_dir: "data/pod5/"
4 dorado_model: "dna_r10.4.1_e8.2_400bps_sup@v4.3.0"
5 genome_size: "4.6m"
6 medaka_model: "r1041_e82_400bps_sup_v4.3.0"
7 busco_lineage: "enterobacterales_odb10"
8 bakta_db: "db/bakta/db-light"
9 reference: "ref/ecoli_k12.fasta"
10 device: "cuda:0" # use "cpu" if no GPU available

```

Listing 26.31: Running the complete Snakemake pipeline.

```

1 # Dry run: see what will be executed
2 snakemake --configfile config.yaml -n
3
4 # Run locally with 8 cores
5 snakemake --configfile config.yaml --cores 8
6
7 # Run with a SLURM cluster profile
8 snakemake --configfile config.yaml \
9     --profile slurm/ \
10    --jobs 10
11
12 # Generate a pipeline DAG visualisation
13 snakemake --configfile config.yaml --dag | dot -Tpng > dag.png

```

Snakemake Best Practices

- Always run `snakemake -n` (dry run) first to verify the execution plan.
- Use `-use-conda` with per-rule `conda:` directives for fully reproducible environments.
- Use `-configfile` to separate parameters from logic.
- Add `resources: mem_mb=16000` to rules for cluster job submission.
- Use `snakemake -lint` to check for common mistakes.
- Pin tool versions in your environment YAML files.

26.11 Complete End-to-End Nextflow Workflow

The equivalent pipeline in Nextflow DSL2, with the same functionality as the Snakemake workflow above.

Listing 26.32: Complete end-to-end Nextflow pipeline for nanopore analysis (main.nf).

```

1 #!/usr/bin/env nextflow
2 nextflow.enable.dsl = 2
3
4 // ===== Parameters =====
5 params.sample = "ecoli_k12"

```

```

6 params.pod5_dir      = "data/pod5"
7 params.dorado_model  = "dna_r10.4.1_e8.2_400bps_sup@v4.3.0"
8 params.genome_size    = "4.6m"
9 params.medaka_model   = "r1041_e82_400bps_sup_v4.3.0"
10 params.busco_lineage = "enterobacterales_odb10"
11 params.bakta_db       = "db/bakta/db-light"
12 params.reference      = ""
13 params.device         = "cuda:0"
14 params.outdir         = "results"
15
16 // ===== Processes =====
17
18 process DORADO_BASECALL {
19     tag "${params.sample}"
20     publishDir "${params.outdir}/basecall", mode: 'copy'
21     label 'process_gpu'
22
23     input: path(pod5_dir)
24     output: path("basecalled.bam"), emit: bam
25
26     script:
27     """
28     dorado basecaller ${params.dorado_model} ${pod5_dir} \
29         --device ${params.device} --emit-moves \
30         > basecalled.bam
31     """
32 }
33
34 process BAM_TO_FASTQ {
35     tag "${params.sample}"
36     cpus 4
37
38     input: path(bam)
39     output: path("basecalled.fastq.gz"), emit: fastq
40
41     script:
42     """
43     samtools fastq -@ ${task.cpus} ${bam} | gzip > basecalled.fastq.gz
44     """
45 }
46
47 process NANOPILOT {
48     tag "${params.sample}"
49     publishDir "${params.outdir}/qc/nanoplot", mode: 'copy'
50     cpus 4
51
52     input: path(fastq)
53     output: path("nanoplot_out/*"), emit: report
54
55     script:
56     """
57     NanoPlot --fastq ${fastq} --outdir nanoplot_out \
58         --threads ${task.cpus} --loglength --N50 \
59         --title "${params.sample} QC"
60     """
61 }
62
63 process FASTQC {

```

```

64     tag "${params.sample}"
65     publishDir "${params.outdir}/qc/fastqc", mode: 'copy'
66     cpus 2
67
68     input: path(fastq)
69     output: path("*_fastqc.{html,zip}"), emit: report
70
71     script:
72     """
73     fastqc ${fastq} --threads ${task.cpus}
74     """
75 }
76
77 process MULTIQC {
78     tag "${params.sample}"
79     publishDir "${params.outdir}/qc/multiqc", mode: 'copy'
80
81     input: path(reports)
82     output: path("multiqc_report.html"), emit: report
83
84     script:
85     """
86     multiqc . -o . --force
87     """
88 }
89
90 process CHOPPER {
91     tag "${params.sample}"
92     cpus 4
93
94     input: path(fastq)
95     output: path("filtered.fastq.gz"), emit: filtered
96
97     script:
98     """
99     zcat ${fastq} | chopper --quality 10 --minlength 1000 \
100       --threads ${task.cpus} | gzip > filtered.fastq.gz
101     """
102 }
103
104 process FLYE {
105     tag "${params.sample}"
106     publishDir "${params.outdir}/assembly/flye", mode: 'copy'
107     cpus 8
108     memory '16 GB'
109
110     input: path(reads)
111     output:
112         path("flye_out/assembly.fasta"), emit: assembly
113         path("flye_out/assembly_info.txt"), emit: info
114
115     script:
116     """
117     flye --nano-hq ${reads} \
118       --out-dir flye_out \
119       --threads ${task.cpus} \
120       --genome-size ${params.genome_size}
121     """

```



```

122 }
123
124 process MEDAKA {
125     tag "${params.sample}"
126     publishDir "${params.outdir}/assembly/medaka", mode: 'copy'
127     cpus 8
128     memory '16 GB'
129
130     input:
131         path(reads)
132         path(draft)
133     output: path("medaka_out/consensus.fasta"), emit: polished
134
135     script:
136         """
137     medaka_polisher -i ${reads} -d ${draft} \
138         -o medaka_out \
139         --model ${params.medaka_model} \
140         --threads ${task.cpus} --bacteria
141         """
142 }
143
144 process QUAST {
145     tag "${params.sample}"
146     publishDir "${params.outdir}/qc/quast", mode: 'copy'
147     cpus 4
148
149     input:
150         path(assembly)
151         val(reference)
152     output: path("quast_out/*"), emit: report
153
154     script:
155     def ref_flag = reference ? "-r ${reference}" : ""
156     """
157     quast ${assembly} ${ref_flag} \
158         -o quast_out --threads ${task.cpus} --min-contig 500
159     """
160 }
161
162 process BUSCO {
163     tag "${params.sample}"
164     publishDir "${params.outdir}/qc/busco", mode: 'copy'
165     cpus 8
166
167     input: path(assembly)
168     output: path("busco_out/*"), emit: report
169
170     script:
171         """
172     busco -i ${assembly} -o busco_out -m genome \
173         -l ${params.busco_lineage} --cpu ${task.cpus}
174         """
175 }
176
177 process BAKTA {
178     tag "${params.sample}"
179     publishDir "${params.outdir}/annotation", mode: 'copy'

```

```

180   cpus 8
181   memory '8 GB'
182
183   input:
184     path(assembly)
185     path(db)
186   output: path("bakta_out/*"), emit: annotation
187
188   script:
189     """
190     bakta ${assembly} --db ${db} \
191       --output bakta_out --prefix ${params.sample} \
192       --threads ${task.cpus} --complete
193     """
194 }
195
196 // ===== Workflow =====
197 workflow {
198   // Input channels
199   pod5_ch = Channel.fromPath(params.pod5_dir, checkIfExists: true)
200   db_ch    = Channel.fromPath(params.bakta_db, checkIfExists: true)
201
202   // Basecalling
203   DORADO_BASECALL(pod5_ch)
204   BAM_TO_FASTQ(DORADO_BASECALL.out.bam)
205
206   // Quality control (runs in parallel)
207   NANOPLLOT(BAM_TO_FASTQ.out.fastq)
208   FASTQC(BAM_TO_FASTQ.out.fastq)
209   qc_reports = NANOPLLOT.out.report.mix(FASTQC.out.report).collect()
210   MULTIQC(qc_reports)
211
212   // Filtering and assembly
213   CHOPPER(BAM_TO_FASTQ.out.fastq)
214   FLYE(CHOPPER.out.filtered)
215   MEDAKA(CHOPPER.out.filtered, FLYE.out.assembly)
216
217   // Assessment (runs in parallel)
218   QUAST(MEDAKA.out.polished, params.reference)
219   BUSCO(MEDAKA.out.polished)
220
221   // Annotation
222   BAKTA(MEDAKA.out.polished, db_ch)
223 }

```

The corresponding Nextflow configuration:

Listing 26.33: Nextflow configuration file (nextflow.config).

```

1 // nextflow.config
2 params {
3   sample      = "ecoli_k12"
4   pod5_dir    = "data/pod5"
5   dorado_model = "dna_r10.4.1_e8.2_400bps_sup@v4.3.0"
6   genome_size = "4.6m"
7   medaka_model = "r1041_e82_400bps_sup_v4.3.0"
8   busco_lineage = "enterobacterales_odb10"
9   bakta_db     = "db/bakta/db-light"
10  reference    = ""

```

```

11     device      = "cuda:0"
12     outdir      = "results"
13 }
14
15 profiles {
16     standard {
17         process.executor = 'local'
18         process.cpus     = 4
19         process.memory   = '8 GB'
20     }
21     slurm {
22         process.executor = 'slurm'
23         process.queue    = 'standard'
24         process.time     = '6h'
25         process.clusterOptions = '--account=myproject'
26     }
27     docker {
28         docker.enabled = true
29     }
30     singularity {
31         singularity.enabled = true
32     }
33 }
34
35 process {
36     withLabel: 'process_gpu' {
37         accelerator = 1
38         clusterOptions = '--gres=gpu:1'
39         queue = 'gpu'
40         memory = '32 GB'
41     }
42 }
43
44 // Timeline and report
45 timeline {
46     enabled = true
47     file    = "${params.outdir}/pipeline_info/timeline.html"
48 }
49 report {
50     enabled = true
51     file    = "${params.outdir}/pipeline_info/report.html"
52 }

```

Listing 26.34: Running the complete Nextflow pipeline.

```

1  # Run locally
2  nextflow run main.nf -profile standard
3
4  # Run on a SLURM cluster with Singularity containers
5  nextflow run main.nf -profile slurm,singularity
6
7  # Run with custom parameters
8  nextflow run main.nf \
9      --sample my_sample \
10     --pod5_dir /data/run01/pod5 \
11     --genome_size 5m \
12     --outdir /data/results/run01
13
14 # Resume a failed/interrupted run

```

```

15 nextflow run main.nf -resume
16
17 # View execution report after completion
18 open results/pipeline_info/report.html

```

Nextflow vs Snakemake: Which to Choose?

Both Snakemake and Nextflow are excellent workflow managers. Some practical guidance:

- **Snakemake** is Python-based, intuitive for Python users, and uses a “pull” model (define desired outputs, Snakemake figures out what to run). Excellent `conda`: integration per rule.
- **Nextflow** is Groovy/Java-based, uses a “push” model (data flows through connected processes), and has native support for cloud executors (AWS Batch, Google Life Sciences). The `nf-core` community provides >100 production-grade pipelines.
- Choose Snakemake if your team knows Python and works primarily on HPC clusters. Choose Nextflow if you need cloud portability or want to leverage `nf-core` pipelines.

26.12 Putting It All Together: A Practical Checklist

Complete Analysis Checklist

1. **Before sequencing:** Estimate required coverage. For bacteria: 50–100×; for human variant calling: 30× minimum.
2. **Basecalling:** Use Dorado with the SUP model for publication-quality data. Match the model to your chemistry version.
3. **QC checkpoint:** Run NanoPlot. Check median quality ($\geq Q15$ for hac, $\geq Q20$ for sup), N50 (>5 kb for ligation libraries), and total yield. If metrics are poor, troubleshoot before proceeding.
4. **Read filtering:** Apply mild quality and length filters with `chopper`. Do not over-filter.
5. **Analysis:** Choose the appropriate walkthrough:
 - Bacterial genome? → Flye + Medaka + QUAST + BUSCO + Bakta (Section 26.4).
 - Species identification? → Kraken2 + Bracken + Krona (Section 26.5).
 - Human variants? → minimap2 + Clair3 + bcftools (Section 26.6).
6. **Visual inspection:** Always look at your data in IGV (Section 26.7). Automated pipelines can produce plausible but incorrect results.
7. **Reproducibility:** Wrap your analysis in a Snakemake or Nextflow workflow. Export your `conda` environment. Document everything.
8. **Share:** Deposit raw data in the [SRA](#). Share your workflow on GitHub. Publish your results.

The Most Important Skill

The single most important skill in bioinformatics is not knowing every command by heart—it is knowing how to *check your results*. Every tool can fail silently, producing output files that look correct but contain errors. Always verify: Is the assembly the expected size? Does BUSCO show high completeness? Does the Ti/Tv ratio make sense? Are there unexpected species in your metagenomics results? Does the variant count match expectations? Build verification steps into every pipeline, and never trust a result you have not inspected.

List of Acronyms

AMR antimicrobial resistance. 534, 541

ATAC-seq Assay for Transposase-Accessible Chromatin using sequencing. 9, 12, 15, 23, 31, 34, 63, 68, 276

BAM Binary Alignment Map. 569, 571, 593

ChIP-seq chromatin immunoprecipitation sequencing. 9, 12, 15, 31, 34, 68, 295, 299

CNV copy number variation. 33

eDNA environmental DNA. 587, 588

ENCODE Encyclopedia of DNA Elements. 276

FASTQ FASTA with Quality. 11, 50, 53, 573, 574, 593

Hi-C high-throughput chromosome conformation capture. 9, 15, 23

HPC high-performance computing. 595

indel insertion or deletion. 33

NGS next-generation sequencing. xxxi, 5, 6, 8, 15, 415, 443, 544, 545, 558

ONT Oxford Nanopore Technologies. 40, 41, 49, 519, 520, 522–524, 527, 528, 530, 531, 533, 539, 540, 546, 559, 561, 569

PacBio Pacific Biosciences. 41

PCR polymerase chain reaction. 10, 19, 20, 30, 41, 46, 51, 255, 519, 521, 523–525, 527, 530, 531, 534, 540, 546, 551, 552, 560, 566, 568

PPE personal protective equipment. 544

QC quality control. 521, 530, 568, 573, 575

RNA-seq RNA sequencing. 9, 12, 14, 27–29, 63, 67, 68, 90, 92, 189, 190, 192, 365

SBS sequencing by synthesis. 38, 64

scRNA-seq single-cell RNA sequencing. 9, 14, 100, 211, 295

SMRT sequencing single-molecule real-time sequencing. 41, 72

SNP single nucleotide polymorphism. 32, 33

SNV single nucleotide variant. 63, 71

SRA Sequence Read Archive. xxxi, 567, 571, 606

SV structural variant. 33, 71

UMI unique molecular identifier. 51, 212, 376

WES whole-exome sequencing. 7, 9, 11

WGBS whole-genome bisulfite sequencing. 9, 15, 30, 63, 258

WGS whole-genome sequencing. 7, 9, 11, 12, 33, 63, 66–68, 70, 75, 81, 84, 397, 520, 521, 532